Cacti kurulum ve işletim kılavuzu

Cacti 1.3 development series

Documentation source base: d8a327c12a91
Built: 2026-10-27

This snapshot follows the development documentation. The accompanying build manifest records each source file hash.

For Cacti 1.2, use the maintenance manual for your installed release.

Video thumbnails are represented by links; local screenshots are embedded.

Copyright (c) 2004-2026 The Cacti Group

Contents

1. Cacti kurulum ve işletim kılavuzu 5
2. Cacti (tm) Documentation 7
3. Complete Manual Index 10
4. Current Manuals and Contributed Archives 15
5. Requirements 18
6. General Installing Instructions 21
7. Installing on Enterprise Linux with Apache 28
8. Installing Cacti Development on Ubuntu or Debian with Apache 31
9. Installing Under Windows 34
10. Upgrading Cacti Under UNIX Compatible Operating Systems 40
11. Upgrading Cacti under Windows 43
12. Upgrading Cacti Under FreeBSD 45
13. Hardening a Cacti Installation 47
14. Configuring SSL for Cacti 52
15. Version Specific Release Notes 55
16. Navigating the Cacti User Interface 57
17. Principles of Operation 62
18. How to Graph Your Network 65
19. Device Management 68
20. Graph Overview 71
21. Graph Management 75
22. Viewing Graphs 76
23. Creating a Graph for a Single SNMP OID 80
24. Site Management 83
25. Cacti Trees 85
26. Data Source Management 89
27. Aggregate Graphs 91
28. Automation Networks 92
29. Automation Discovered Devices 94
30. Automation Device Rules 96
31. Automation Graph Rules 97
32. Automation Tree Rules 100
33. Automation SNMP Options 102
34. User Management 104
35. User Group Management 120
36. Login Providers 123
37. User Domain Management 125
38. Reports Administration 127
39. Reports User 130
40. Report Items 133
41. Report Preview 136
42. Report Events 137
43. Other Report Management Options 138
44. System Utilities 140

45. Data Collectors ... 146
46. Spine ... 149
47. Poller Settings ... 152
48. PHP Script Server ... 155
49. Performance Boosting Settings ... 159
50. rrdproxy ... 164
51. Remote Poller Setup ... 167
52. Change poller interval from 5 minutes to 1 minute ... 169
53. How To Set Up SSH Tunnels to Graph a Remote Unix Server ... 173
54. How To Extend snmpd with custom script/data ... 176
55. Templates ... 178
56. How To Work with Templates ... 190
57. Device Templates ... 197
58. Graph Templates ... 199
59. Data Source Templates ... 203
60. Aggregate Templates ... 206
61. Color Templates ... 210
62. Data Input Methods ... 212
63. Data Input Methods - From Script Output to Complete Graph ... 215
64. Data Queries ... 230
65. How To Work with Data Queries ... 239
66. SNMP Data Queries ... 242
67. SNMP Data Query Walkthrough ... 250
68. Script Data Query Walkthrough ... 268
69. How To Determine a Cacti Template Version ... 291
70. Import Packages ... 292
71. Template Import ... 293
72. Template Exporting ... 295
73. Create Packages ... 303
74. Package Repositories ... 305
75. Package Public Keys ... 306
76. Apache Template Config ... 307
77. ESXi/VMware Template ... 309
78. Proxmox - external storage usage ... 310
79. Data Source Debugging ... 312
80. The Cacti Log Interface ... 315
81. Debugging ... 317
82. Frequently Asked Questions ... 320
83. Known Issues ... 326
84. Spikekill ... 327
85. Settings ... 330
86. General Settings ... 331
87. Path Settings ... 336
88. Device Default Settings ... 339
89. Data Settings ... 342
90. Visual Settings ... 345
91. Performance Settings (Boost) ... 348

92. Spike Kill ... 350
93. Mail/Reporting/DNS Settings/Proxy ... 352
94. Authentication ... 357
95. Local Authentication ... 359
96. LDAP Authentication ... 361
97. Basic Authentication ... 368
98. Multiple LDAP/AD Domains ... 370
99. Configuration - Users, Groups and Login Providers ... 372
100. Data Source Profiles ... 373
101. CDEFs ... 375
102. VDEFs ... 378
103. Colors ... 380
104. GPRINT Presets ... 382
105. Variables ... 383
106. RRDtool Specific Features ... 391
107. Command Line Scripts (CLI) ... 404
108. Plugin Management ... 423
109. Supported plugins ... 425
110. syslog ... 429
111. MacTrack ... 438
112. Plugin Development ... 440
113. Plugin Guidelines ... 444
114. Creating a plugin ... 445
115. Reference Plugins ... 449
116. Plugin Hook API Reference ... 452
117. Contributing ... 477
118. Creating Translations ... 479
119. Documentation Standards ... 480
120. Formatting Guidelines ... 483
121. PHP Specific Constructs ... 490
122. File System Layout ... 491
123. Patch Creation ... 493
124. SQL Standards ... 495
125. Security ... 498
126. Architecture Decision Records ... 500
127. ADR-0001: Establish bounded contexts and framework-independent domain code ... 501
128. ADR-0002: Model installation as durable, resumable install runs ... 502
129. Project Credits and Licensing ... 503
130. External Links ... 505
131. Enterprise Linux LAMP Installation ... 507
132. Enterprise Linux LEMP Installation ... 508
133. Installing on Rocky Linux 9 and 10 with nginx ... 509
134. Installing Cacti on Ubuntu Server with nginx ... 513
135. Converting a Debian or Ubuntu Package Installation to Source ... 516
136. ADR-0003: Provide a versioned plugin extension boundary ... 517
137. ADR-0004: Persist workflow state separately from diagnostic artifacts ... 518

Cacti kurulum ve işletim kılavuzu

Bu belge, yeni kurulumlar için eski Türkçe katkının yerini alan yeni bir kılavuzdur. Henüz yayımlanmamış Cacti 1.3 `develop` dalını anlatır. Cacti 1.2 kullanıyorsanız 1.2.x el kitabını izleyin. Türkçe başlangıç bölümü yeniden yazılmıştır; eksiksiz ek bölüm güncel İngilizce bölümleri içerir. Bu belge, el kitabının tamamının Türkçe çevirisi değildir.

Sürüm ve platform seçimi

Önce gereksinimleri denetleyin. Güvenlik güncellemeleri alan bir işletim sistemi ve seçilen Cacti dalına uygun PHP, veritabanı ve RRDtool kullanın. Komut satırı PHP'si ile web sunucusunun kullandığı PHP farklı ayar dosyaları yükleyebilir; ikisini de kurucuda doğrulayın.

Linux için Ubuntu/Debian veya Enterprise Linux kurulumunu izleyin. Apache ve nginx yapılandırmalarını birbirinin yerine kullanmayın. FreeBSD paketleri için genel kurulumdaki PHP çeşidi ve dizin bilgilerini okuyun. Bir Cacti 1.2 paketi, geliştirme sürümüne eşdeğer değildir.

Kuruluma hazırlık

Mevcut bir sistemi değiştirmeden önce veritabanını, yapılandırmayı ve RRD verilerini yedekleyin. Yedeğin geri yüklenebildiğini sınayın. Seçtiğiniz platform kılavuzuna göre PHP, veritabanı, RRDtool ve Net-SNMP kurun. Kaynak koddan kurulumda, dağıtım sahibi olarak kilitlenmiş bağımlılıkları kurun:

```
composer install --no-dev
composer check-platform-reqs --no-dev
php --ini
php -m
```

Platform denetimi başarısız olursa devam etmeyin. `cacti.sql` dosyasını yalnızca yeni bir veritabanına aktarın; mevcut veritabanının üzerine aktarmayın. Mevcut kurulum için Cacti'nin yükseltme işlemini kullanın. Parolaları komut satırında, ekran görüntülerinde veya destek iletilerinde paylaşmayın.

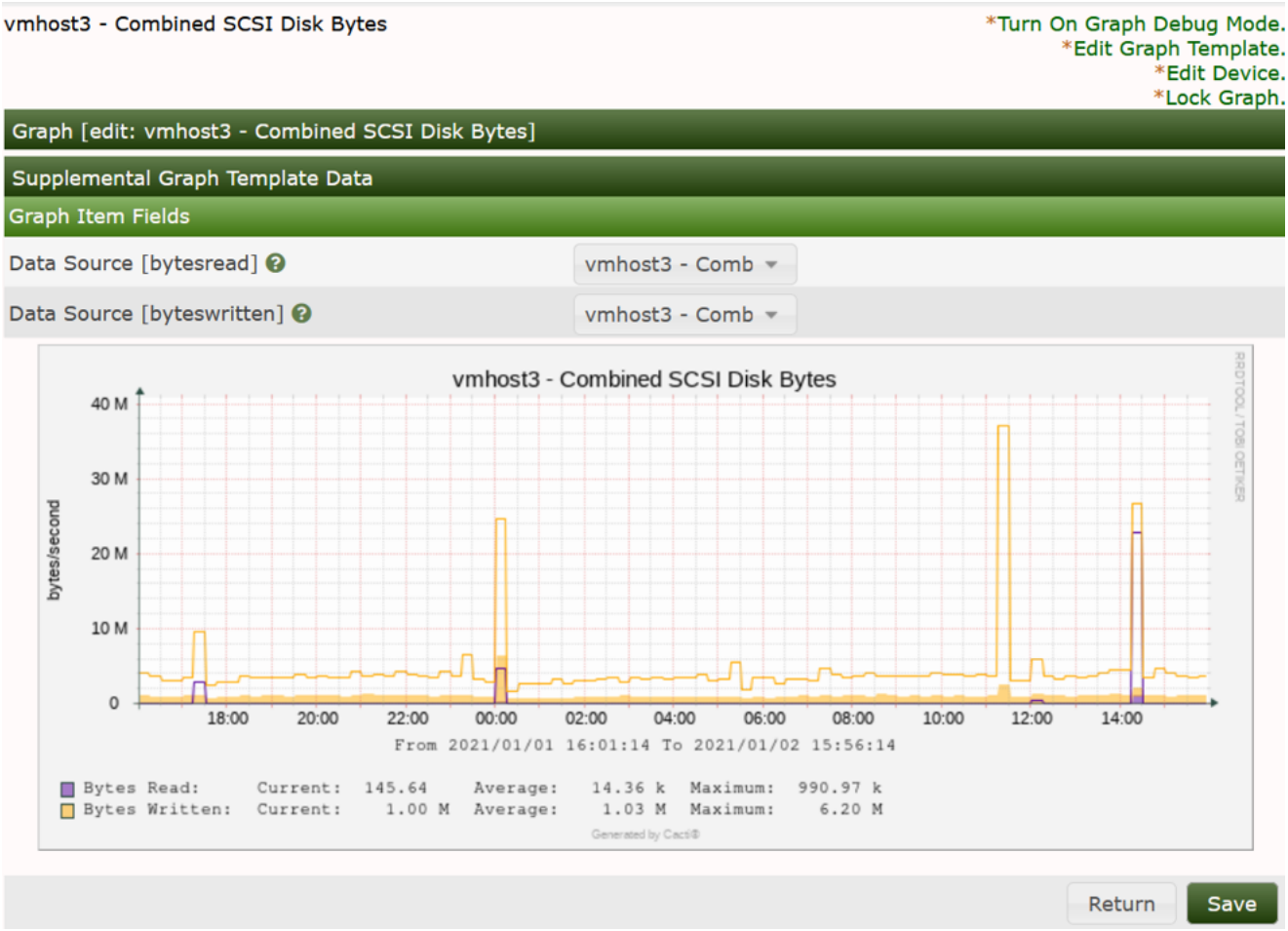
Web erişimi ve veri toplama

Gerçek veritabanı bağlantı bilgilerini `include/config.php` içinde tanımlayın. Bu dosyayı ve uygulama kodunu web sunucusunun değiştirmesine izin vermeyin. Veri toplama hesabına yalnızca gerekli çalışma dizinlerinde yazma izni verin. Web sunucusu ve poller için ayrı, yetkileri sınırlı hesaplar kullanın.

Kurulum sırasında web erişimini sınırlayın ve HTTPS yapılandırın. Web kurucunun PHP, dizin, saat dilimi ve veritabanı denetimlerini tamamlayın. İlk oturumda başlangıç parolasını değiştirin. PHP ve `poller.php` için mutlak yolları kullanan tek bir zamanlama oluşturun. Aynı anda ikinci poller çalıştırmayın. Spine isteğe bağlıdır ve Cacti sürümüyle uyumlu olmalıdır.

İlk cihaz ve grafikler

Console > Devices bölümünde uygun şablonla cihaz ekleyin. SNMP sürümünü ve kimlik bilgilerini cihazın ayarlarıyla eşleştirin. Cihazın SNMP erişimini collector adresiyle sınırlayın. Grafik oluşturun ve bir ağaca ekleyin. Sonuçları değerlendirmeden önce birkaç veri toplama aralığı bekleyin. İlk grafik adımlarını izleyin.



Görüntü eski bir arayüzü ve geçmiş örnek ölçümleri gösterir.

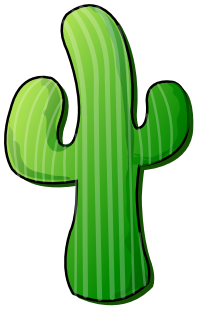
Sorun giderme ve yükseltme

Önce servisleri, sistem saatini, CLI ve web PHP ayarlarını, çalıştırılabilir dosya yollarını ve hesap izinlerini denetleyin. Cacti günlüğüne bakın ve tek bir veri kaynağı için hata ayıklama araçlarını kullanın. Ayrıntılı loglamayı yalnızca gerekli cihazlarda geçici olarak açın. Boş bir grafik nedeniyle RRD dosyalarını silmeyin veya veritabanını yeniden oluşturmayın.

Yükseltmeden önce sürüm notlarını ve yükseltme kılavuzunu okuyun. Veritabanı, RRD verisi, yapılandırma, eklentiler ve zamanlamaları yedekleyin. Önce deneme ortamında doğrulayın. Veritabanı yükseltmesini bir kez çalıştırın. Kurulumu kullanıcılara açmadan önce güvenlik adımlarını uygulayın.

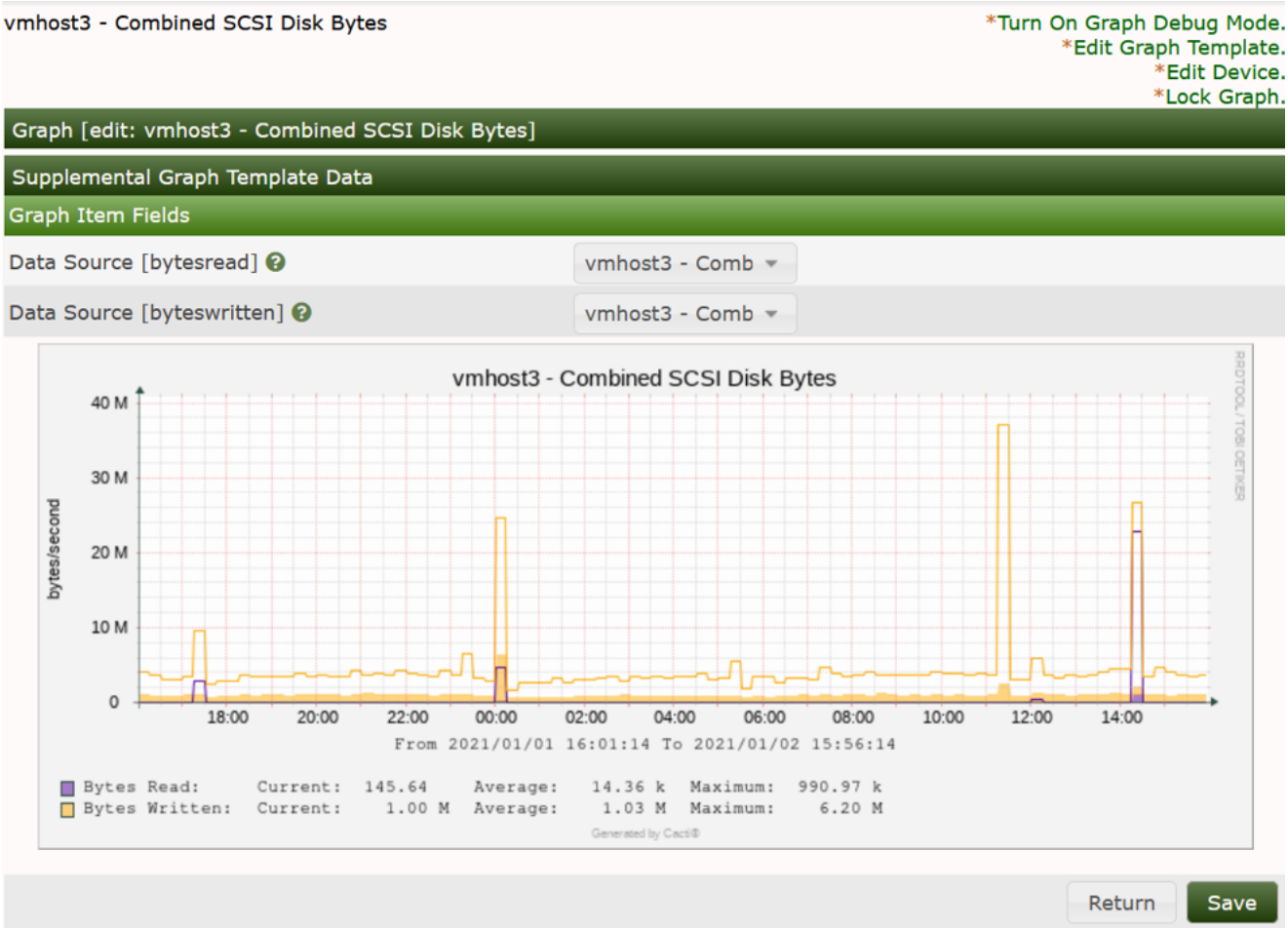
Copyright (c) 2004-2026 The Cacti Group

Cacti (tm) Documentation



Cacti collects measurements from network devices, servers, and applications and uses RRDtool to store time-series data and display graphs. Use this manual to install Cacti, configure monitoring, and keep your installation running.

Version: This manual follows **develop**, the unreleased Cacti 1.3 series. For released Cacti 1.2 installations, use the 1.2.x manual and the requirements supplied with your installed version.



This example from Graph Overview contains historical sample data and shows an earlier interface. Menu labels can differ in your version.

Current editions and contributed archives include PDF, text, localized installation guides, and the development schema reference.

What Do You Want to Do

- **Install or upgrade Cacti**: Check requirements, choose your platform, install, upgrade, and secure Cacti.

- **Create your first graphs**: Add a device, configure SNMP, create graphs, and verify data collection.
- **Manage your installation**: Organize devices and graphs, automate discovery, manage access, and report.

- **Configure data collection**: Set up collectors, Spine, remote polling, scheduling, and performance.
- **Create and reuse templates**: Work with templates, data queries, and package import and export.
- **Troubleshoot a problem**: Investigate missing data, failed SNMP queries, and polling errors.

Getting Started

Follow this sequence for a new installation:

1. Check requirements.
2. Install Cacti for your platform.
3. Add a device and create its graphs.
4. Verify data collection and check the Cacti log.
5. Secure the installation before making it available to other users.

For the concepts behind these steps, read Principles of Operation. To find your way around Cacti, use Navigating the User Interface.

Troubleshoot a Problem

- **Graphs have no data**: Check polling and RRD updates.
- **Find errors in the log**: Inspect collection and system logs.
- **Investigate SNMP and graph failures**: Follow the debugging guide for polling and rendering problems.
- **Common questions and polling problems**: Find answers about SNMP, scheduling, password recovery, and graph errors.

- **Known issues**: Check documented limitations and where to report a problem.

Manual and Reference

Browse the **complete manual index** for every guide, including procedures grouped with their related subjects.

Use the settings reference, command-line scripts, and graph and data reference when you need a particular option, command, variable, or graph expression. Check release notes for version-specific changes.

Extend and Contribute

- **Use and develop plugins**: Install extensions or find plugin guidelines, hooks, and references.

- **Contribute to Cacti**: Report bugs, improve code or documentation, and help translate Cacti.

- **HTML manual content and publication**: Build all chapters and preserve links from the archived HTML manual.
- **Development standards**: Follow the documentation, PHP, SQL, and security conventions for your target branch.
- **Architecture decisions**: Read proposals and accepted decisions guiding development.
- **Video tutorials**: Watch tutorials on the official Cacti channel. Check the demonstrated version against your installation.

Project Information

See project credits, acknowledgments, and licensing. For downloads and news, visit the Cacti website. For community support, use the Cacti Forums.

Copyright (c) 2004-2026 The Cacti Group

Complete Manual Index

Return to the documentation front page.

This index follows **develop**, the unreleased Cacti 1.3 series. For released Cacti 1.2 installations, use the 1.2.x manual. Menu visibility also depends on permissions, enabled features, and plugins. Screenshots can show an earlier release or a different theme.

Current editions and contributed archives include PDF, text, localized installation guides, and the development schema reference.

Install or Upgrade Cacti

Choose the guide for your operating system and installation method. Earlier LAMP and LEMP guide URLs point to the maintained Enterprise Linux instructions.

- Requirements
- General Installing Instructions
- Enterprise Linux with Apache (LAMP)
- Rocky Linux 9 and 10 with Nginx (LEMP)
- Ubuntu/Debian with Apache (LAMP)
- Ubuntu Server 26.04 LTS and 26.10 with Nginx (LEMP)
- Installing Under Windows
- Upgrading Cacti Under UNIX Compatible Operating Systems
- Upgrading Cacti Under Windows
- Upgrading Cacti Under FreeBSD
- Convert a Debian/Ubuntu package install to source
- Hardening a Cacti Installation
- Configuring SSL for Cacti
- Version Specific Release Notes

Create Your First Graphs

Start with the interface and concepts, then add a device and its graphs.

- Navigating the Cacti User Interface
- Principles of Operation
- How to Graph Your Network
- Device Management
- Graph Overview
- Graph Management
- Viewing Graphs
- Creating a Graph for a Single SNMP OID

Manage Your Installation

Organize monitoring, automate discovery, manage access, and create reports.

- Site Management

- Cacti Trees
- Data Source Management
- Aggregate Graphs
- Automation Networks
- Automation Discovered Devices
- Automation Device Rules
- Automation Graph Rules
- Automation Tree Rules
- Automation SNMP Options
- User Management
- User Group Management
- Login Providers
- User Domains (1.2)
- Reports Administration
- Reports User
- Report Items
- Report Preview
- Report Events
- Other Report Management Options
- System Utilities

Configure Data Collection

Configure local and remote collection and diagnose performance constraints.

- Data Collectors
- Spine
- Poller Settings
- PHP Script Server
- Performance Boosting Settings
- rrdproxy
- Remote Poller Setup
- Change the polling interval
- SSH tunnels for remote Unix monitoring
- How To Extend snmpd with custom script/data

Create and Reuse Templates

Find template concepts, query procedures, packaging, and device-specific guides.

- Templates
- How To Work with Templates
- Device Templates
- Graph Templates
- Data Source Templates
- Aggregate Templates
- Color Templates

- Data Input Methods
- Create a data input method
- Data Queries
- How To Work with Data Queries
- SNMP Data Queries
- SNMP Data Query Walkthrough
- Script Data Query Walkthrough
- Determine a template version
- Import Packages
- Template Import
- Template Exporting
- Create Packages
- Package Repositories
- Package Public Keys
- Apache Template Config
- ESXi/VMware Template
- Proxmox - external storage usage

Troubleshoot a Problem

Start with the symptom, polling output, and log messages.

- Data Source Debugging
- The Cacti Log Interface
- Debugging
- Frequently Asked Questions
- Known Issues
- Spikekill

Settings Reference

Use the settings for your branch. Development and 1.2 authentication differ.

- Settings
- General Settings
- Path Settings
- Device Default Settings
- Data Settings
- Visual Settings
- Performance Settings (Boost)
- Spike Kill
- Mail/Reporting/DNS Settings/Proxy
- Authentication
- Local Authentication
- LDAP Authentication
- Basic Authentication
- Multiple LDAP/AD Domains (1.2)

- Users, groups, and login providers

Graph and Data Reference

Look up data profiles, expressions, formatting, and command-line tools.

- Data Source Profiles
- CDEFs
- VDEFs
- Colors
- GPRINT Presets
- Variables
- RRDtool Specific Features
- Command Line Scripts (CLI)

Plugins

Install and use plugins, or develop an extension.

- Plugin Management
- Supported plugins
- syslog
- MacTrack
- Plugin Development
- Plugin Guidelines
- Creating a plugin
- Reference Plugins
- Plugin Hook API Reference
- ReportIt plugin manual (historical PDF)

Contributing

Contribute code, documentation, translations, and support.

- Contributing
- Creating Translations

Development Standards

Follow the target branch's contributor instructions and checks.

- Documentation Standards
- Formatting Guidelines
- PHP Specific Constructs
- File System Layout
- Patch Creation
- SQL Standards
- Security

Architecture Decisions

Read the decision index for the status of each proposal.

- Architecture Decision Records
- ADR-0001: Bounded contexts
- ADR-0002: Durable install runs
- ADR-0003: Plugin extension boundary
- ADR-0004: Workflow storage and observability

Project and Community

Find project credits, licensing, support, and additional resources.

- Project Credits and Licensing
- External Links
- Official video tutorials
- Cacti Forums

Copyright (c) 2004-2026 The Cacti Group

Current Manuals and Contributed Archives

These replacements follow **develop**, the unreleased Cacti 1.3 series. Use the 1.2.x manual for released Cacti 1.2. The version guidance follows the website's distinction between released and development documentation. The manuals reuse the maintained chapter content and existing documentation styles. They do not import the website's visual design.

Complete maintained manuals

- PDF manual, with all current main chapters, bookmarks, screenshots, and links between chapters.
- UTF-8 text manual, with the same main chapters, command examples, image descriptions, and full link destinations.
- HTML manual publication, including all chapters and legacy chapter URLs.
- Historical ReportIt manual. The newer plugin manual is maintained separately from these Cacti editions.

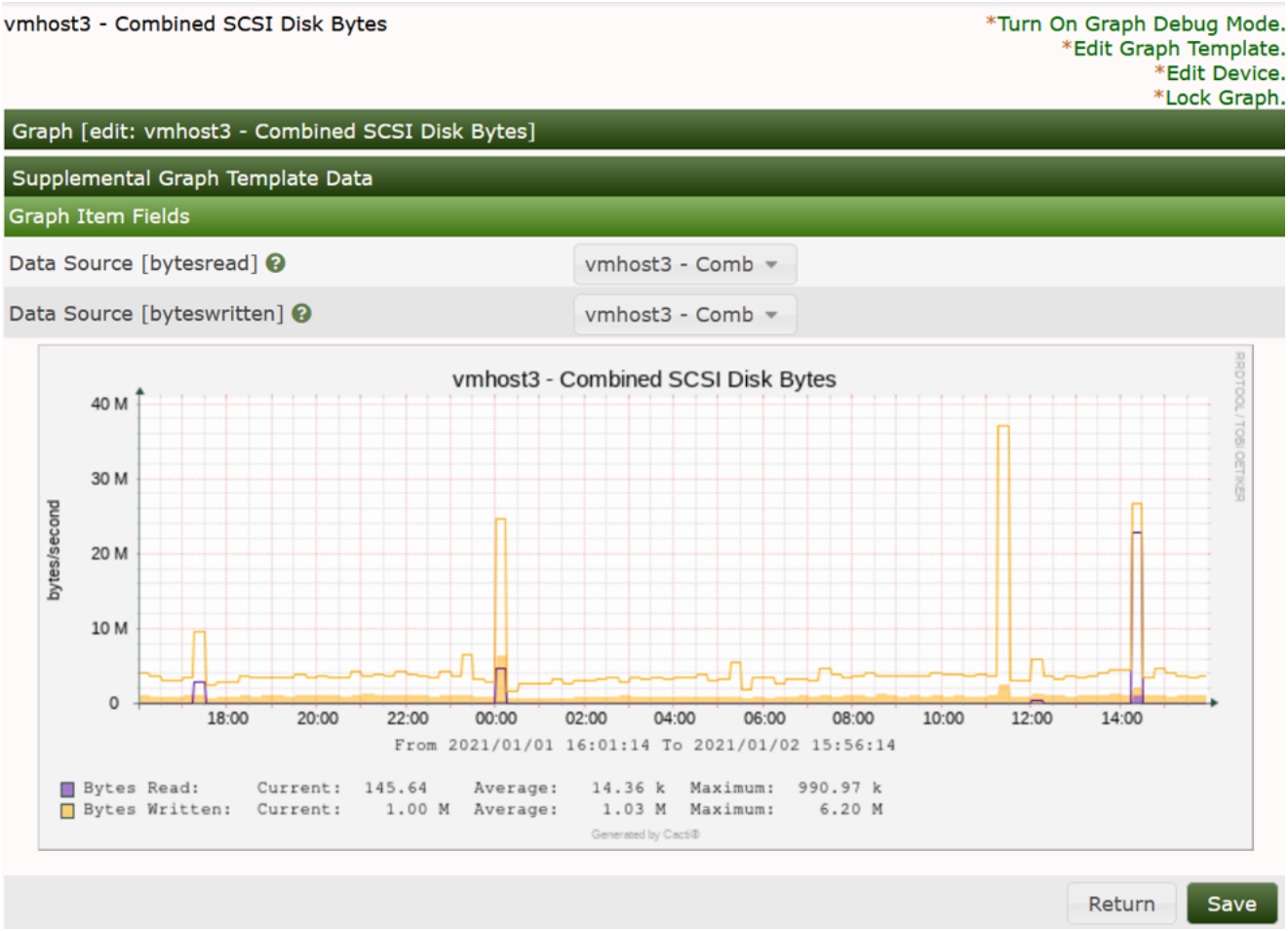
Current replacements for contributed guides

The new guides are separate Cacti documentation, not revisions attributed to the original contributors. German, Turkish, and Russian editions contain a new localized introduction and complete relevant English chapters in their PDF and text appendices. They are not full translations. Language review is welcome; platform checks and runtime limitations still apply.

Guide	Read online	PDF	Text
German installation and operation	Deutsch	PDF	Text
Turkish installation and operation	Türkçe	PDF	Text
Linux installation	English	PDF	Text
Russian UNIX installation	Русский	PDF	Text
Russian Windows installation	Русский	PDF	Text
Windows installation	English	PDF	Text
Migration from Solaris	English	PDF	Text
Development database schema	Reference	PDF	Text

The German and Turkish appendices include every current main manual chapter. Installation editions include the complete platform, first-graph, troubleshooting, security, and upgrade chapters selected in editions.json. Solaris has a migration guide rather than an unverified new-install procedure. The Windows guides explain the current development dependency restriction before any install instructions.

The schema SQL contains only the pinned upstream `CREATE TABLE` statements. It is a reference, not an installer or upgrade script. It has no seed data and does not describe plugin tables. Use the schema supplied with your exact Cacti release for installation.



This screenshot shows historical sample measurements and an earlier interface. It is included in the operating guides, not evidence of a deployment on every listed platform.

Historical contributions

These originals remain archives at their existing URLs, with their original credits and rights notices. Their obsolete dependencies and commands do not apply to current installations. Do not overwrite them with the new guides or present the new guides as work by the original contributors.

Original	Credit from the original	Status
German documentation	Sebastian Larisch, August 2006	Archive
Turkish handbook	Özgür Karataş, February 2006	Archive
Linux how-to	Lee Carter, 2004	Archive
Russian UNIX installation	Translation by Александр Цапкин, December 2006	Archive
Russian Windows installation	Translation by Александр Цапкин, December 2006	Archive
Solaris installation	Original credit and notice remain in the archive	Archive
Windows installation	Original credit and notice remain in the archive	Archive
Previous PDF manual	Original Cacti manual credits	Archive
Previous text manual	Original Cacti manual credits	Archive
Database schema PDF	Original notice remains in the archive	Archive

Original	Credit from the original	Status
phpMyAdmin SQL	Original notice remains in the archive	Archive

The archive inventory records each original URL, byte count, and SHA-256 captured on 26 October 2026. Original documents are not copied or translated into the new guides.

Build and publish

Follow manual publication to rebuild and validate every edition. The edition manifest records exact sources and checksums. The PDF builder also records its locked toolchain in the PDF manifest.

CI publishes review artifacts; it does not deploy the public file host. Maintainers should publish the new editions under distinct versioned names, link this index from the download page, and retain the original archive URLs. Publish the main manual separately from the historic `manual.pdf` and `manual.txt` names so an existing archive citation keeps its meaning.

Copyright (c) 2004-2026 The Cacti Group

Requirements

This manual follows the unreleased Cacti 1.3 `develop` branch. Use the 1.2.x manual for released Cacti 1.2 installations. Verify the requirements of the exact revision or release you deploy; development requirements can change.

Supported operating systems

Platform guidance checked on 26 October 2026. Use the current security-updated point release in a vendor-supported stable branch. The examples below target these maintained operating systems; vendor OS support does not certify every Cacti, PHP, database, or third-party binary combination.

Operating system	Versions used for installation guidance	Lifecycle source
RHEL	Current supported minor release of 9 or 10	Red Hat lifecycle
Rocky Linux	Current minor release of 9 or 10	Rocky releases
AlmaLinux	Current minor release of 9 or 10	AlmaLinux releases
Ubuntu Server	24.04 LTS, 26.04 LTS, or 26.10	Ubuntu lifecycle
Debian	13 stable; 12 only with applicable Debian LTS coverage	Debian lifecycle
FreeBSD	14.5-RELEASE or 15.1-RELEASE, with current patches	FreeBSD security support
Windows Server	2022 or 2025, with current updates	Server 2022, Server 2025

Check the linked lifecycle information before deployment. LTS coverage can exclude architectures or packages. Use a maintained PHP and database version that also meets this Cacti branch's requirements. A supported OS with an incompatible default PHP version is not ready for installation. Do not use a beta OS or an expired point release for these procedures.

Software Requirements

Component	Development requirement or recommendation
Web server	PHP-capable Apache, nginx, or IIS FastCGI
PHP	PHP 8.3 or later in the PHP 8 series (`^8.3` in Composer)
Database	MySQL 8.4+ or MariaDB 10.11+ recommended by the installer
RRDtool	Version 1.5 or later
SNMP	Net-SNMP command-line tools for operations using external binaries
Spine	Optional; use a version compatible with Cacti and its runtime libraries

The PHP requirement comes from the dependency manifest and runtime minimum. The database values are the branch's installer recommendations, rather than a certification of every vendor release. Check the actual server and extensions before installation.

PHP extensions

Cacti checks the CLI and web PHP environments separately. Install the extensions required by the selected revision's Composer manifest and lock file, and satisfy the installer checks. The combined requirements include:

- `ctype`, `curl`, `date`, `dom`, `filter`, `gd`, `gmp`, `hash`, `intl`, `json`, `ldap`, `mbstring`, `mysqlnd`, `openssl`, `pcre`, `PDO`, `pdo_mysql`, `Phar`, `session`, `simplexml`, `sockets`, `spl`, `sqlite3`, `standard`, `xml`, and `zlib`.
- `posix` and CLI `pcntl` on Unix.
- `com_dotnet` for the Windows installer checks.

Some modules are built into PHP. Check `php -v`, `php --ini`, and `php -m` for the CLI, then check web PHP through the installer. Enabling an extension in one PHP configuration does not enable it in the other.

The PHP SNMP extension is optional. External Net-SNMP tools are still needed for collection paths such as some SNMPv3 and bulk requests. Configure their paths and verify execution under the actual web and collector accounts.

Composer and source deployments

A source checkout or GitHub source archive needs its locked PHP dependencies. Run these commands from the Cacti source root as the deployment owner:

```
composer install --no-dev
composer check-platform-reqs --no-dev
```

Use `composer install`, which reads `composer.lock`. Do not run `composer update` during installation or upgrade. A complete release bundle can already contain `include/vendor/autoload.php`; verify its generated platform checks and the release's instructions rather than assuming every archive contains vendor files.

The current manifest requires `ext-pcntl` and `ext-posix` unconditionally. Native Windows PHP cannot supply these modules. Source dependency installation is therefore blocked on native Windows, even though the installer has Windows-specific checks. A bundled artifact must also pass its runtime platform check. See Installing Under Windows before choosing a build. Do not bypass platform checks to force installation.

Hardware Sizing

Size from the polling interval, number of data sources, retention profiles, concurrent graph requests, collector settings, and measured storage latency. Device count alone is not a reliable sizing rule. Test representative polling and graphing load before increasing the deployment.

Use storage that can sustain RRD updates and database writes within the polling window. Measure CPU, memory, I/O latency, database connections, and poll completion time. Remote collectors can distribute collection work; they also require connectivity, synchronization, and operational monitoring. Spine process and thread counts should be tuned against those measurements.

Database configuration

Enable timezone support and grant the application account `SELECT` access to `mysql.time_zone_name`. The grant does not populate that table. Import the vendor's timezone data using the procedure for your operating system.

Use `utf8mb4` and the `utf8mb4_unicode_ci` collation consistently for the Cacti database. Changing server defaults does not convert existing tables; back up and plan any conversion of an installed database separately.

Start with the database vendor's defaults and review the installer's checks for the installed engine and version. Do not copy a fixed `my.cnf` block from another server. Size global buffers and per-connection buffers together with PHP and operating-system memory. Estimate Spine's connection demand as:

```
total_connections = total_processes * (total_threads + script_servers + 1)
```

Allow additional connections for web requests, plugins, and maintenance. Inspect actual storage engines before applying memory-table sizing guidance.

Keep InnoDB doublewrite protection enabled. Disabling it removes protection against incomplete page writes; an SSD alone does not establish atomic page writes. Review the durability consequences before changing commit flushing or other crash-recovery settings.

Database variables differ by engine and release. Removed settings can prevent startup. For example, MySQL 8 no longer accepts `innodb_file_format`, `innodb_large_prefix`, or `innodb_additional_mem_pool_size`, and MariaDB 10.6 removed `innodb_buffer_pool_instances`. Validate settings against the installed server's documentation and Cacti's version-aware checks before restarting. See the MySQL reference or MariaDB reference.

Accounts and platform controls

Keep application code owned by the administrator or deployment account. Give web and collector identities the read, execute, and runtime-directory write access they need. Grant temporary installation or import access only to the required paths, and remove it afterward. Keep database credentials, CSRF secrets, private caches, RRD files, and backups inaccessible over HTTP.

Configure SELinux, AppArmor, container permissions, or IIS request filtering for the required operations. Do not remove confinement globally to make installation checks pass. Follow the platform installation guide and Hardening a Cacti Installation.

Copyright (c) 2004-2026 The Cacti Group

General Installing Instructions

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

Install the packages required by the selected operating system and verify that the configured web server, PHP handler, and database start at boot.

Containers

Use an unprivileged container with its normal security profile where possible. Diagnose the specific ICMP, socket, or filesystem permission that fails and adjust only the required capability or policy. Do not remove AppArmor confinement or make the entire container privileged as a general prerequisite.

Required Packages for Most Operating Systems

Depending on your operating system and PHP version, certain packages are required for Cacti. The largest variable in these requirements come with regard to PHP and MySQL/MariaDB.

Installation requirements include the packages below. The installation of these packages will vary by operating system.

Base OS

- apache, IIS, or nginx
- net-snmp, net-snmp-utils
- rrdtool
- help2man (for spine)
- dos2unix (for spine)
- development packages (gcc, automake, autoconf, libtool, either mysql-devel or mariadb-devel, net-snmp-devel, help2man)

  (for spine)

Database

Use the database versions and configuration described in Requirements. The development branch recommends MySQL 8.4+ or MariaDB 10.11+. Check the installed server with the Cacti installer.

- mysql
- mysql-server
- libmysqlclient

or

- mariadb
- mariadb-server
- libmariadbclient

PHP Modules

Use PHP 8.3 or later in the PHP 8 series and the extensions listed in Requirements. Verify the CLI with `php -v` and `php -m` and check web PHP separately in the installer. Source deployments also need Composer and all extensions required by the selected revision's lock file. Run `composer install --no-dev` as the deployment owner, then `composer check-platform-reqs --no-dev`. Use the lock file rather than updating dependencies during installation.

Systems using PHP-FPM

After changing PHP configuration, test and restart the actual FPM service. Service names can be versioned, for example on Debian and Ubuntu. Check the installed unit rather than assuming that `php-fpm` is the service name. Confirm its socket path and permissions match the web-server handler.

FreeBSD

Use a patched release listed in supported operating systems. Choose binary packages or the Ports Collection using the FreeBSD Handbook. Run package and service administration commands as root. Use the dedicated poller account for collection.

Choose the Cacti version and PHP flavor

The FreeBSD `net-mgmt/cacti` port currently packages Cacti 1.2.31, not the unreleased Cacti 1.3 development branch covered by this manual. For a packaged release, follow its installed instructions and the 1.2.x manual. For development source, meet Requirements and follow the source-install steps below using FreeBSD paths and service configuration. Do not overlay a source checkout onto files owned by `pkg`.

Cacti packages have PHP flavors. Discover the names available in your configured repository before selecting one:

```
pkg update
pkg search -S name -x '^cacti(-php[0-9]+)?$'
```

For example, if the repository offers `cacti-php84` and the installation uses PHP 8.4, review and install that flavor:

```
pkg install -n cacti-php84
pkg install cacti-php84
pkg info -D cacti-php84
pkg info -l cacti-php84
```

Replace the example name with the selected flavor. Read the transaction before accepting it, particularly changes to PHP extensions and database clients. Dependencies do not configure a web server or create a database server. Set up a compatible local MariaDB/MySQL service, or an existing remote database, and configure Apache or nginx with the matching PHP handler separately. Use FreeBSD `service` and `sysrc` for installed rc.d services; Linux `systemctl` examples do not apply.

Build from ports when custom options are required

Use a maintained ports tree and read `/usr/ports/UPDATING`. Clone a tree only if `/usr/ports` does not already contain one:

```
pkg install git
# New tree only:
git clone --depth 1 https://git.FreeBSD.org/ports.git /usr/ports
# Existing Git tree instead:
git -C /usr/ports pull --ff-only
```

Build the chosen flavor, for example:

```
cd /usr/ports/net-mgmt/cacti
make FLAVOR=php84 config
make FLAVOR=php84 install clean
```

The supported flavors and dependencies come from that ports revision. Follow the handbook's dependency-management guidance before mixing custom ports with binary packages. Rebuilding a port does not turn the packaged release into Cacti development source.

Check installed paths and services

The Cacti port uses these default locations. Custom ports options and existing Cacti settings can change them; inspect the installed files and application configuration before copying commands.

Item	Default location or account
Application and bundled schema	`/usr/local/share/cacti`
Database configuration	`/usr/local/share/cacti/include/config.php`
PHP configuration	`/usr/local/etc/php.ini`
RRD data	`/var/db/cacti/rra`
Log directory	`/var/log/cacti`
PHP CLI	`/usr/local/bin/php`
Packaged poller account	`cacti`
Web-server account	Commonly `www`; verify the service configuration

The port supplies `include/config.php.sample`; preserve an existing `include/config.php` and use the sample when creating a new one. Set the intended timezone in both CLI and web PHP. Initialize a new database from the bundled `cacti.sql`, grant the application account access to its database and `mysql.time_zone_name`, and verify database timezone data is loaded. Never import `cacti.sql` over an existing installation.

Check package-specific patches as well as upstream instructions. For example, the current port sets `$php_snmp_support = false` for new configurations; verify the installed setting and Net-SNMP paths when troubleshooting polling.

Configure write access only for the runtime directories needed by the actual poller and web processes. Keep application code and database credentials protected. Preserve Cacti's web-server restrictions on configuration, CLI, resource, and data directories.

Create one Cacti schedule. For a five-minute launcher, an example entry in the `cacti` user's crontab is:

```
*/5 * * * * /usr/local/bin/php -q /usr/local/share/cacti/poller.php
```

Match this cadence to **Cron/Daemon Interval** in Cacti. A system crontab needs the account field as well. Check for an existing cron entry or daemon first; do not run both. Use Data Source Debugging and the Cacti log to verify collection after completing the web installer.

Optional Spine poller

The Spine port is `net-mgmt/cacti-spine`; its package is `spine`. Install it only when its version matches the Cacti release you use:

```
pkg install -n spine
pkg install spine
```

Its default binary is `/usr/local/bin/spine`, with `/usr/local/etc/spine.conf.sample` as the configuration example. Supply the correct database connection in `spine.conf` and select the binary in Cacti. Verify the poller account can run it. Grant ICMP privileges only when the selected polling method requires them; do not apply a blanket setuid command.

For an existing installation, follow Upgrading Under FreeBSD.

Configure PHP

Verify that the modules are installed and configured correctly. There are several ways to do so - consult PHP configuration instructions for a complete description.

It is imperative that you set the `date.timezone` in your `/etc/php.ini`, or `/etc/phpX/apache/php.ini` and `/etc/phpX/cli/php.ini` files. Failure to do so will result in errors after the install is complete.

Most other PHP configuration is done automatically by the base OS, so there is no need to discuss that here.

Configure the Webserver (Apache)

Use the distribution's Apache/PHP integration, selecting either its PHP-FPM handler or a compatible Apache PHP module. Do not configure both handlers for the same files. The Enterprise Linux LAMP guide and Ubuntu/Debian guide provide examples.

Verify Apache's configuration before reloading. Restrict access during setup, use HTTPS, and ensure that PHP requests execute rather than expose source. Keep Cacti's shipped `.htaccess` restrictions active, or reproduce them in the server configuration if overrides are disabled. nginx and IIS need their own restrictions; they do not interpret Apache `.htaccess` files.

Configure MySQL/MariaDB

Initialize and secure the database using the procedure for the installed MySQL or MariaDB package. Use the administrator authentication method configured by that package; some Unix packages use local socket authentication. Store credentials securely and do not put passwords directly on shell command lines.

You must also load timezone information into the database. This is required for various plugin use. Later, you will be required to grant access to the `time_zone_name` table during the final installation steps.

```
mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -u root -p mysql
```

Since Cacti 1.x is supporting internationalization (i18n), it is important that the default character set for MySQL/MariaDB be i18n compatible. The Cacti installer will make specific recommendations on MySQL/MariaDB settings. Follow those as applicable for your OS.

Galera clustering: There are several tables which are set to use the MEMORY storage engine which do not get replicated among nodes which can cause problems. If you configure Cacti to only connect to one node of your cluster and are not load balancing this does not apply to you.

If you are running multiple nodes in a load-balanced environment where you connect to a VIP you should remove all but one node from rotation during Cacti installation or update. After the installation/update login to your MySQL server and execute the following commands to update those tables to use the InnoDB engine:

```
MariaDB [(none)]> use cacti;
MariaDB [cacti]> ALTER TABLE `automation_ips` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `automation_processes` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `data_source_stats_hourly_cache` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `data_source_stats_hourly_last` ENGINE=InnoDB;
```

```
MariaDB [cacti]> ALTER TABLE `poller_output` ENGINE=InnoDB;
MariaDB [cacti]> ALTER TABLE `poller_output_boost_processes` ENGINE=InnoDB;
```

Before restoring database load balancing, verify the table-engine changes, replication state and successful polling on the configured topology. A fixed number of polling cycles is not proof that the cluster is ready.

Install and Configure Cacti

These are fresh source-install examples for Cacti development (1.3). Package-managed installations should use the maintainer's procedure instead. For an existing installation, follow Upgrading Cacti. The examples use `/var/www/html/cacti`; adjust that path and the service accounts for your OS. Use administrator privileges for software installation and permission changes.

Extract the source and create the database

Obtain the intended source revision in a new installation directory. Do not run these commands over an existing deployment:

```
git clone --branch develop --depth 1 https://github.com/Cacti/cacti.git cacti
mv cacti /var/www/html/cacti
cd /var/www/html/cacti
composer install --no-dev
composer check-platform-reqs --no-dev
```

The installation root should now contain `cacti.sql`, `poller.php` and `include/config.php.dist`, rather than another nested release directory.

Connect with a database administrator account:

```
mysql --user=root --password
```

Create the database and a dedicated application account. Replace the password placeholder before running this SQL, and use the same secret in the Cacti configuration. The database account is separate from the operating-system account that runs the poller.

```
CREATE DATABASE cacti CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'cactiuser'@'localhost'
  IDENTIFIED BY 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
GRANT ALL PRIVILEGES ON cacti.* TO 'cactiuser'@'localhost';
GRANT SELECT ON mysql.time_zone_name TO 'cactiuser'@'localhost';
EXIT;
```

Use an authorized database administrator to grant access. Do not edit `mysql.tables_priv` as a substitute for an authorized `GRANT` operation. Ensure the server's timezone tables are populated using the procedure for the installed MySQL or MariaDB server; granting access does not populate them.

Import the schema into the new database using the appropriate client:

```
mysql --user=root --password --default-character-set=utf8mb4 cacti \
  < /var/www/html/cacti/cacti.sql
```

The `--password` option prompts for the password. Use `mariadb` in place of `mysql` where appropriate for the installed server.

Configure access and complete installation

Copy the supplied configuration example:

```
cp /var/www/html/cacti/include/config.php.dist \
  /var/www/html/cacti/include/config.php
```

Edit the copied file to match the account created above:

```
$database_type     = 'mysql';
$database_default  = 'cacti';
$database_hostname = 'localhost';
$database_username = 'cactiuser';
$database_password = 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
$database_port     = '3306';
$url_path          = '/cacti/';
```

Identify the actual web and poller service accounts. Give them the required read/execute access to application files and external tools, and write access to the runtime paths they use, including RRD, log and cache paths. They need not be the same account. Do not assume a database username is also a local OS account or make the whole application tree writable.

During installation and package imports, grant the performing identity the additional access required by the installer, including query/script resources and the location used to initialize the CSRF secret. Remove temporary setup permissions afterward. Preserve read access to the generated secret; do not create an empty secret file. See the platform-specific installation guides for their account and permission examples.

Restrict installer access to the administrator and use HTTPS or a protected local connection before entering credentials. A freshly imported development schema uses application username `admin` and initial password `admin`. Open the configured Cacti URL, sign in and choose a unique administrator password when prompted. These are separate from the database credentials. Complete the installer's dependency, database, path and permission checks before allowing general access.

Schedule collection after installation

Use either cron or `cactid.service`, not both. The following examples assume an existing local user and group named `cacti` with the required polling permissions; replace them with the actual service identity for your system. Do not start polling before completing the web installer.

For a system cron file such as `/etc/cron.d/cacti`, use an absolute PHP CLI path and include the OS username:

```
*/5 * * * * cacti /usr/bin/php /var/www/html/cacti/poller.php >/dev/null 2>&1
```

The five-minute launch interval must match **Cron/Daemon Interval** under the poller settings. This is separate from individual data-source polling intervals. The redirection above works with `/bin/sh`; `&>` is Bash-specific.

The supplied systemd unit uses `Type=forking`. Confirm that the selected PHP CLI provides `pcntl_fork` before using it; otherwise use cron or install the appropriate PCNTL support. For the example CLI path, this should report `bool(true)`:

```
/usr/bin/php -r 'var_dump(function_exists("pcntl_fork"));'
```

For systemd, copy the supplied unit and edit the installed copy:

```
cp /var/www/html/cacti/service/cactid.service \
  /etc/systemd/system/cactid.service
```

Replace the unit's existing `User`, `Group`, `ExecStart` and `EnvironmentFile` settings with values for the actual deployment, retaining its other settings. For example:

```
User=cacti
Group=cacti
ExecStart=/usr/bin/php /var/www/html/cacti/cactid.php
EnvironmentFile=-/etc/default/cactid
```

The `-` allows the optional environment file to be absent. Disable the Cacti-specific cron entry before enabling the daemon, then load the edited unit:

```
systemctl daemon-reload
systemctl enable cactid.service
systemctl start cactid.service
systemctl status cactid.service
```

The unit manages the local daemon; it does not establish high availability. Verify scheduler execution, collector status, logs and recent graph data under the configured account.

(Optional) Install and Configure Spine

Spine is a very fast data collection engine, written in C. It is an optional replacement for cmd.php. If you decide to use it, you will have to install it explicitly. It does not come with cacti itself.

The easiest way is to install Spine using rpm or ports. You will find packages for Spine at the main cacti site or from your distribution.

To compile Spine, download it to any location of your liking. Then, issue from the downloaded directory following commands

```
./bootstrap
```

After `bootstrap` succeeds, follow the release's build instructions to configure, compile and install Spine.

Assuming, you've managed to install spine correctly, you will have to configure it. Use the installed `spine.conf.dist` as the example, and configure the actual Spine configuration path in Cacti. Use the same database account and secret as the Cacti configuration, and restrict file access to the poller account and administrators.

```
DB_Host     localhost
DB_Database cacti
DB_User     cactiuser
DB_Pass     REPLACE_WITH_UNIQUE_DATABASE_PASSWORD
DB_Port     3306
```

Considerations when using Proxies in front of Cacti (Cacti 1.2.23+)

A header allowlist does not authenticate a proxy. Trust forwarded client-IP headers only when a trusted proxy replaces those headers and untrusted clients cannot bypass it to reach Cacti directly. Leave forwarding disabled otherwise. In `include/config.php`, select only the headers your proxy sets. For a deployment using `X-Forwarded-For`, the setting is:

```
$proxy_headers = ['X-Forwarded-For'];
```

Copyright (c) 2004-2026 The Cacti Group

Installing on Enterprise Linux with Apache

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

This source-install example covers the unreleased Cacti 1.3 development branch on RHEL, Rocky Linux, or AlmaLinux. Read Requirements first. Run administrative commands as root and adapt paths and accounts to your deployment. For an existing installation, use Upgrading Cacti. For package-managed Cacti, follow the package maintainer's procedure instead of overlaying source files.

Choose repositories and PHP

For a new, fully updated RHEL 9 installation with the stream available, PHP 8.3 is an example meeting the development minimum:

```
dnf module list php
dnf module install php:8.3
```

This is a fresh-install example, not a stream-switching procedure for an existing server. Check the streams supplied by your distribution. EL 9 and EL 10 do not share identical repositories or module availability. If the required PHP version is unavailable, choose a maintained distribution or repository that provides it before continuing.

Install the web and database packages

This example uses MariaDB. MySQL installations should use the package and service names for their selected MySQL repository. Review the database version against Cacti's requirements before installation.

```
dnf install -y httpd php-fpm php-cli php-mysqlnd php-gd php-gmp \
  php-intl php-ldap php-mbstring php-xml php-process php-snmp \
  mariadb-server rrdtool net-snmp-utils cronie
php -v
php -m
```

Additional packages can be required by the chosen source revision. Check Requirements, including Composer dependencies where applicable. Do not mix extensions from different PHP versions or repositories.

Configure `date.timezone` in the PHP configuration used by both CLI and FPM. Check `php --ini` for CLI and the installer for web PHP. Review the FPM pool at `/etc/php-fpm.d/www.conf`; its account needs access to Cacti's runtime paths. Keep code owned by the deployment administrator.

Start the database:

```
systemctl enable --now mariadb
```

Follow the installed database vendor's initialization and account-security procedure. Do not set a root password on the shell command line. Configure only variables accepted by that database version; use Cacti's installer recommendations rather than a fixed tuning block. Keep InnoDB doublewrite protection enabled.

Configure Apache

Use the distribution's Apache/PHP-FPM integration. Check that PHP requests are executed by FPM rather than served as text. This example exposes only `/cacti/`; replace the hostname and certificate paths before use. Save it in `/etc/httpd/conf.d/cacti.conf`:

```
<VirtualHost *:443>
    ServerName cacti.example.com
    SSLEngine On
    SSLCertificateFile /etc/pki/tls/certs/cacti.crt
    SSLCertificateKeyFile /etc/pki/tls/private/cacti.key
    Alias /cacti /var/www/html/cacti

    <Directory /var/www/html/cacti>
        Options -Indexes
        AllowOverride All
        Require all granted
        DirectoryIndex index.php
    </Directory>
</VirtualHost>
```

Install `mod_ssl` if not already present. `AllowOverride All` lets Cacti's shipped `.htaccess` restrictions protect private paths; if your policy disables overrides, reproduce those restrictions in the server configuration. Do not grant general access to private directories as a troubleshooting step.

```
dnf install -y mod_ssl
httpd -t
systemctl enable --now php-fpm httpd
```

Use your certificate-management procedure and review any packaged default TLS virtual host for conflicts. Restrict access to administrators during setup.

Install and configure Cacti

Use General Installing Instructions to obtain the intended source, create a new database and dedicated application account, load timezone data, import `cacti.sql`, and configure `include/config.php`. The examples there use `/var/www/html/cacti`; use that same path here.

Complete the dependency and filesystem-permission checks before opening the installer. The database account and operating-system service accounts are separate identities. Grant runtime write access to the actual FPM and poller accounts, not automatically to the nginx master process or the database user. Keep executable code and credentials protected during normal operation.

Open `https://cacti.example.com/cacti/` and complete the installer. Configure one scheduler after installation: either a Cacti-specific cron entry or the supplied `cactid` systemd service, using the procedure in the general guide. Do not run both for the same collector. The five-minute cron example is:

```
*/5 * * * * apache /usr/bin/php /PATH/TO/cacti/poller.php >/dev/null 2>&1
```

Replace `/PATH/TO/cacti` and `apache` with the installed path and chosen poller account. Enable `crond` if using cron. Verify a manual poll under that account, then check scheduled runs, logs, collector status, and fresh graph samples.

SELinux and containers

Keep SELinux enforcing. For a custom installation path, use persistent file contexts and allow writes only on Cacti's runtime paths. Review audit denials and authorize the specific network and executable access needed by FPM, RRDtool, SNMP tools, and the collector. Do not use `setenforce 0` as the installation procedure. See the RHEL SELinux guide.

For LXC or Proxmox, retain an unprivileged container and its confinement where possible. Diagnose the specific ICMP/socket permission or mapped-file access failure. Adjust only the required capability or policy; do not make the entire container privileged or set an unconfined AppArmor profile merely to run Cacti.

Optional Spine and proxies

Follow Spine for the optional compiled collector. Select a compatible source or package and the development headers for your OS. Repository names differ: do not apply Rocky/AlmaLinux `crb` commands to RHEL without checking its repository configuration. Configure the Spine database connection with the same application credentials and restrict access to its configuration file.

When using a reverse proxy, see the proxy configuration in the general guide. A header allowlist alone does not authenticate a proxy; prevent clients from bypassing the trusted proxy and require it to replace forwarded client-IP headers.

Copyright (c) 2004-2026 The Cacti Group

Installing Cacti Development on Ubuntu or Debian with Apache

For nginx with PHP-FPM on Ubuntu Server 26.04 LTS or 26.10, use the Ubuntu LEMP guide.

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

This is a source-install example for Cacti development (1.3), not a tested distribution matrix. Use a maintained distribution and check its PHP version against Requirements before installing packages. Run the administrative commands below as root. The example uses `/var/www/html/cacti` and the default Debian/Ubuntu web account, `www-data`; adjust both if your web-server setup differs.

Install the required packages

The unversioned PHP package names select the distribution's configured PHP version. Confirm that it is PHP 8.3 or later in the PHP 8 series before continuing. Use a distribution or configured repository that supplies the required version; do not mix extensions from different PHP versions.

```
apt-get update
apt-get install -y apache2 libapache2-mod-php mariadb-server rrdtool snmp git cron \
  composer php-cli php-curl php-sqlite3 php-mysql php-snmp php-xml php-mbstring php-gd php-gmp php-zip php-ldap
php-intl
systemctl enable --now apache2 mariadb cron
```

If using PHP-FPM instead of the Apache PHP module, configure Apache's FPM integration and use the installed FPM pool's account. FPM service names are versioned on Debian/Ubuntu; inspect them with `systemctl list-unit-files 'php*-fpm.service'` and restart the actual unit after configuration changes. Do not assume a service named `php-fpm` exists.

Download the Cacti source

This example checks out the unreleased `develop` branch. Record the commit you deploy; do not treat a moving development branch as a stable release.

```
git clone --branch develop --depth 1 https://github.com/Cacti/cacti.git cacti
mv cacti /var/www/html/cacti
cd /var/www/html/cacti
composer install --no-dev
composer check-platform-reqs --no-dev
```

Do not overwrite an existing installation with these fresh-install commands.

Create the database

Choose a unique database password and replace the example placeholder in both the SQL and `include/config.php`. Connect using your database administrator's authentication method; the example below prompts for a password.

```
mysql -u root -p
```

```
CREATE DATABASE cacti CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'cactiuser'@'localhost' IDENTIFIED BY 'replace-with-a-unique-password';
GRANT ALL PRIVILEGES ON cacti.* TO 'cactiuser'@'localhost';
```

```
GRANT SELECT ON mysql.time_zone_name TO 'cactiuser'@'localhost';
EXIT;
```

Load the system timezone data and the schema into this new database. A MariaDB installation may provide `mariadb-tzinfo-to-sql` as the timezone utility's name.

```
mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -u root -p mysql
mysql -u root -p cacti < /var/www/html/cacti/cacti.sql
```

Configure Cacti and filesystem permissions

Copy the distributed configuration example; do not rename or delete it.

```
cp /var/www/html/cacti/include/config.php.dist /var/www/html/cacti/include/config.php
```

Edit the database fields to match the account and database created above:

```
$database_type     = 'mysql';
$database_default  = 'cacti';
$database_hostname = 'localhost';
$database_username = 'cactiuser';
$database_password = 'replace-with-a-unique-password';
$database_port     = '3306';
```

Keep application code owned by the administrator. The web and poller accounts must be able to read the configuration and write the runtime directories:

```
chown root:www-data /var/www/html/cacti/include/config.php
chmod 640 /var/www/html/cacti/include/config.php
chown -R www-data:www-data /var/www/html/cacti/rra /var/www/html/cacti/log /var/www/html/cacti/cache
chmod -R u+rwX,go-rwx /var/www/html/cacti/rra /var/www/html/cacti/log /var/www/html/cacti/cache
```

During installation or package import, the installer also checks write access to `resource/snmp_queries`, `resource/script_server`, `resource/script_queries` and `scripts`, and the location used to create the CSRF secret. Grant only the additional access requested by the installer and remove that temporary access afterwards. Do not make the entire application tree world-writable.

Restrict installer access to the administrator. Configure HTTPS or use a protected local connection before entering credentials, then browse to `/cacti/`. For a freshly imported development schema, sign in with application username `admin` and initial password `admin`, then choose a unique administrator password when prompted. These are separate from the database credentials. Complete the web installer before starting scheduled polling or allowing general access.

Schedule polling

Choose either cron or `cactid.service`, not both. Complete the web installer before starting the selected scheduler. The example cron interval is five minutes; match the **Cron/Daemon Interval** in Settings > Poller. This is separate from the data-source polling interval, which can be shorter.

Cron

Create `/etc/cron.d/cacti` as root, with mode `644`, and include this line:

```
*/5 * * * * www-data /usr/bin/php /var/www/html/cacti/poller.php >/dev/null 2>&1
```

This is a system cron file, so the username field is required. Use an absolute PHP CLI path and POSIX shell redirection; Debian cron uses `/bin/sh` by default. See Debian crontab format.

systemd

The supplied unit uses `Type=forking`, so the selected PHP CLI must provide `pcntl_fork`. Check `/usr/bin/php -r 'var_dump(function_exists("pcntl_fork"));'` for `bool(true)` before using this example. If fork support is unavailable, use cron or install the appropriate PCNTL support for that PHP CLI.

Copy the supplied service and edit the installed copy:

```
cp /var/www/html/cacti/service/cactid.service /etc/systemd/system/cactid.service
```

Replace its existing `User`, `Group`, `EnvironmentFile` and `ExecStart` entries with the following, retaining the other service settings:

```
User=www-data
Group=www-data
EnvironmentFile=-/etc/default/cactid
ExecStart=/usr/bin/php /var/www/html/cacti/cactid.php
```

The leading `-` makes the optional environment file unnecessary if you do not use one. After completing the installer, load and start the service:

```
systemctl daemon-reload
systemctl enable cactid.service
systemctl start cactid.service
systemctl status cactid.service
```

The service manages the local polling daemon; it does not by itself provide high availability. Verify the poller log and newly created graphs after starting the selected scheduler.

Considerations when using Proxies in front of Cacti (Cacti 1.2.23+)

A header allowlist does not authenticate a proxy. Trust forwarded client-IP headers only when a trusted proxy replaces those headers and untrusted clients cannot bypass it to reach Cacti directly. Leave forwarding disabled otherwise. In `include/config.php`, select only the headers your proxy sets. For a deployment using `X-Forwarded-For`, the setting is:

```
$proxy_headers = ['X-Forwarded-For'];
```

Copyright (c) 2004-2026 The Cacti Group

Installing Under Windows

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

These instructions describe a manual installation of Cacti development (1.3) on Windows. Use the requirements for your Cacti release and supported Windows builds of PHP, your database server and RRDtool. Community Windows installers are also discussed in the Cacti forums; check their included versions and maintenance status before using them.

The examples use `C:\inetpub\wwwroot\cacti` for Cacti, `C:\php` for PHP, `C:\rrdtool` for RRDtool and `C:\spine` for optional Spine. Substitute your actual paths throughout. Use a dedicated Windows account for the poller and a separate web-server identity. Install software and configure services as an administrator; the poller does not normally need administrator privileges.

Development packaging prerequisite

The current development dependency manifest requires PHP `^8.3`, including `ext-pcntl` and `ext-posix` unconditionally. Native Windows PHP cannot supply those two extensions. A source checkout therefore cannot currently install its Composer dependencies on native Windows. A bundled artifact must also pass its generated runtime platform check. Stop if it rejects Windows and use a compatible release or a Unix installation. Do not bypass platform checks or edit generated vendor files to force an incompatible installation.

The remaining instructions apply only to an artifact that passes its platform checks. Read the PHP PCNTL platform restriction and the selected revision's dependency manifest before choosing an artifact.

Install the software

1. Choose IIS or Apache and install only the web server you intend to use on the selected address and port.
2. Extract the Cacti ZIP distribution into the chosen web directory.
3. Install PHP and its required runtime libraries using the PHP Windows installation instructions. Match the architecture and build type to your web-server integration. Use a Non-Thread-Safe build for IIS FastCGI; an Apache module requires a compatible Thread-Safe build. A path without spaces, such as `C:\php`, simplifies tool configuration.
4. Install a Windows build of RRDtool and verify its runtime dependencies. If using Spine, install a compatible Spine release and the dependencies supplied or required by that build. The PHP poller can be used without Spine.
5. Install MySQL or MariaDB. Use the server's supported Windows configuration and service-management instructions. Start with its documented defaults, then review the Cacti installer's checks for your actual server and workload.

   Do not copy a tuning block from a different database version or operating system. MySQL 8 removed `NO_AUTO_CREATE_USER`, `innodb_file_format` and `innodb_large_prefix`; these settings can prevent startup. Do not disable InnoDB doublewrite protection or reduce commit durability as an installation shortcut. Size global and per-connection buffers for the available memory and expected concurrency.
6. Install Net-SNMP command-line tools if your collection configuration needs them, for example into `C:\usr`. Cacti uses these tools for SNMPv3 requests from the web interface and PHP poller; PHP's SNMP extension alone does not provide that collection path. Verify the installed executable paths instead of assuming a particular short filename or installation directory.

Configure PHP

Copy `php.ini-production` from the PHP distribution to `C:\php\php.ini`. Set the timezone for your installation and enable the required extensions. For current PHP Windows builds, an example configuration is:

```
extension_dir = "C:\php\ext"
extension=curl
extension=pdo_mysql
extension=sqlite3
extension=gd
extension=gmp
extension=intl
extension=ldap
extension=mbstring
extension=openssl
extension=sockets
extension=com_dotnet
; Enable if using PHP's SNMP extension:
extension=snmp

date.timezone = America/Detroit
file_uploads = On
```

Use only extensions supplied for the selected PHP build and install their required DLL dependencies according to PHP's instructions. Some modules are built in. The Cacti installer checks additional modules, including SimpleXML, XML and zlib, in both the web and command-line environments. `pdo_mysql` is the database driver Cacti uses; the removed `php_mysql.dll` is not a substitute. Set memory and execution limits using the installer recommendations for your installation rather than copying values sized for another server.

Check the command-line configuration before configuring the web server:

```
C:\php\php.exe --ini
C:\php\php.exe -m
```

Set `PHPRC` to `C:\php` for the web-server and poller environments if needed to select this configuration. An absolute path to `php.exe` avoids depending on `PATH` for scheduled tasks. Ensure required DLLs are discoverable by both identities; do not copy PHP DLLs into Windows system directories.

If loading external MIBs, set `MIBDIRS` to their actual directory and allow the collecting account to update the Net-SNMP index/cache there if needed. This is separate from permission to modify executable files.

After changing PHP configuration, recycle the application pool or restart the selected web-server service. Confirm the Cacti installer reports the expected configuration and modules for both web PHP and CLI PHP.

Configure the web server

IIS with FastCGI

Follow the PHP IIS instructions and Microsoft's FastCGI configuration reference.

1. Enable the IIS CGI role service, which supplies FastCGI support.
2. Create the Cacti site/application and a dedicated application pool. Match the pool architecture to PHP. In Handler Mappings, map `*.php` through `FastCgiModule` to `C:\php\php-cgi.exe`, not `php.exe`. Use the corresponding FastCGI application entry and allow script execution for the application. PHP 8 does not provide `php8isapi.dll`; do not configure an ISAPI filter.
3. Add `index.php` to the site's Default Document list. Apply the PHP CGI settings required by the PHP IIS guide, including `cgi.force_redirect=0` for IIS. Configure authentication/impersonation deliberately so the account accessing Cacti files is known.

4. Configure an HTTPS binding and restrict access during installation. Recycle the affected application pool after configuration changes.

Apache

Follow the PHP Apache instructions for a matching Apache/PHP build. For Apache 2.4 with a compatible PHP 8 Thread-Safe module, the relevant configuration is:

```
LoadModule php_module "C:/php/php8apache2_4.dll"
<FilesMatch "\.php$">
    SetHandler application/x-httpd-php
</FilesMatch>
DirectoryIndex index.php index.html
PHPIniDir "C:/php"
```

Check Apache's configuration before restarting its service. If using FastCGI instead of the Apache module, follow that integration's PHP instructions and use the appropriate PHP build.

File permissions

Identify the effective web identity and the scheduled-task account before setting ACLs. For IIS, this may be `IIS AppPool\Cacti` for a pool named `Cacti`, or another account selected by authentication and FastCGI impersonation. See Microsoft's application-pool identity guidance. Do not assume the account is always `IUSR`.

Give the web and poller identities read access to Cacti's code and configuration, and read/execute access to PHP, RRDtool, SNMP and optional Spine programs and their dependencies. If PHP requires `cmd.exe` to launch a command, allow only the necessary read/execute access. Do not grant either identity write access to Windows system executables or to tool installation directories.

The poller needs Modify permission on `rra` and `log` and on the cache subdirectories it uses. The web identity also needs Modify permission on `log`, its configured temporary/session directories and the Cacti cache subdirectories used by features such as real-time graphs. Use the installer's permission checks to identify the required paths, including `cache/boost`, `cache/mibcache`, `cache/realtime` and `cache/spikekill`. Grant access only to the identities and paths that need it, accounting for inherited ACLs.

During installation or signed-package imports, Cacti also needs to write queries and scripts under `resource/snmp_queries`, `resource/script_server`, `resource/script_queries` and `scripts`. Grant temporary Modify access to the identity performing that operation, then remove it when finished. The installer also needs to initialize a nonempty CSRF secret, by default `include/vendor/csrf/csrf-secret.php`. Allow creation in its containing directory during setup, then retain read access to the generated secret and remove the temporary directory write permission. Do not create an empty placeholder secret.

Edit `include/config.php` as an administrator. Keep it readable by the web and poller identities but not writable by them for ordinary operation. If a specific installation operation needs to update it, grant temporary write access for that operation and remove it afterward.

Create the database

The following commands are for **Command Prompt**, not PowerShell. Use the actual path to `mysql.exe` or the equivalent MariaDB client if it is not on `PATH`. The `--password` option prompts for the database administrator's password; do not put that password on the command line.

```
mysql --user=root --password
```

Create an empty database and a dedicated application account. Replace `REPLACE_WITH_UNIQUE_DATABASE_PASSWORD` with a unique secret before executing this SQL. The example account and connection use TCP loopback consistently.

```
CREATE DATABASE cacti CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'cacti_user'@'127.0.0.1'
  IDENTIFIED BY 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
GRANT ALL PRIVILEGES ON cacti.* TO 'cacti_user'@'127.0.0.1';
GRANT SELECT ON mysql.time_zone_name TO 'cacti_user'@'127.0.0.1';
```

Exit the SQL client and import the schema from the same Cacti release you extracted:

```
mysql --user=root --password --default-character-set=utf8mb4 cacti < "C:\inetpub\wwwroot\cacti\cacti.sql"
```

Ensure the server's time-zone tables are populated using its documented Windows procedure. Granting access to an empty `mysql.time_zone_name` table does not populate it. Resolve the database and time-zone checks reported by the Cacti installer before completing installation.

Optional Cygwin and Spine configuration

If your Spine or RRDtool build depends on Cygwin, install the runtime packages required by that build using the official Cygwin installer. For a source build, follow the release's build instructions and select its current compiler/development dependencies. Do not replace or delete DLLs shipped with a binary distribution without checking that distribution's requirements. Avoid mixing incompatible Cygwin runtimes.

For Spine, copy `spine.conf.dist` to `spine.conf` and set the connection details to match Cacti. Replace the password placeholder with the same application secret used above, not a sample password:

```
DB_Host     127.0.0.1
DB_Database cacti
DB_User     cacti_user
DB_Pass     REPLACE_WITH_UNIQUE_DATABASE_PASSWORD
DB_Port     3306
```

Restrict access to `spine.conf` to administrators and the poller account. Verify Spine from that account before selecting it as the poller in Cacti.

Finish the Cacti installation

1. Copy `include/config.php.dist` to `include/config.php` and set its database connection. Replace the password placeholder with your application secret:

   ```
   $database_default  = 'cacti';
   $database_hostname = '127.0.0.1';
   $database_username = 'cacti_user';
   $database_password = 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
   $database_port     = '3306';
   $url_path          = '/cacti/';
   ```

   Adjust `$url_path` if Cacti is served at another URL path.

2. Open the Cacti URL through HTTPS. For a freshly imported development schema, sign in with application username `admin` and initial password `admin`, then choose a unique administrator password when prompted. These are separate from the database credentials. Complete the installer and resolve its dependency, database, path and permission checks before allowing general access.

3. In **Console > Configuration > Settings > Paths**, verify the executable paths against your installation. For the example layout:

Setting	Example
PHP binary	`C:/php/php.exe`
RRDtool	`C:/rrdtool/rrdtool.exe`
SNMP tools, if used	`C:/usr/bin/snmpget.exe`, `snmpwalk.exe`, `snmpbulkwalk.exe`, `snmpgetnext.exe`
Cacti log	`C:/inetpub/wwwroot/cacti/log/cacti.log`
Spine, if used	`C:/spine/spine.exe`

Use the CLI PHP executable here, not the IIS FastCGI executable. Verify the separate Spine configuration path if selecting Spine.

4. Create devices and graphs deliberately, or configure discovery and automation rules for the intended network. Installing Cacti does not establish that every discovered device exposes all measurements in its selected template.

Schedule and verify the poller

Log on as the intended poller account and test the same executable and paths that the task will use:

```
C:\php\php.exe -q C:\inetpub\wwwroot\cacti\poller.php
```

Check `log/cacti.log`, data-source polling and graph updates. RRD files are created for configured data sources; an installation without graphs need not create them. Resolve permission and configuration errors before scheduling.

Open **Task Scheduler** and choose **Create Task**:

1. Select the dedicated poller account and **Run whether user is logged on or not**, with the account rights needed for batch logon. Do not enable **Run with highest privileges** unless a documented local requirement needs it.
2. Add a daily trigger that repeats every **5 minutes** indefinitely for the default five-minute scheduler configuration. If changing Cacti's scheduler interval, adjust this trigger to match it; the scheduler interval and data source polling intervals are separate settings.
3. Add a **Start a program** action with these separate fields:
    - **Program/script:** `C:\php\php.exe`
    - **Add arguments:** `-q "C:\inetpub\wwwroot\cacti\poller.php"`
    - **Start in:** `C:\inetpub\wwwroot\cacti`
4. Under **Settings**, choose **Do not start a new instance** if the task is already running. Review any task execution limit against the configured polling cycle; do not terminate a healthy poller prematurely.

Run the task manually, then verify subsequent scheduled runs, Task Scheduler's result/history, Cacti's poller statistics and graph freshness. Recheck these under the task account after changing PHP, paths, credentials or ACLs.

References

- Cacti development (1.3) PHP dependency checks
- Cacti development (1.3) installation permission checks
- PHP Windows installation, IIS and Apache
- Microsoft FastCGI configuration and application-pool identities
- MySQL 8 removed features
- Microsoft Task Scheduler

Copyright (c) 2004-2026 The Cacti Group

Upgrading Cacti Under UNIX Compatible Operating Systems

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

This procedure covers a source installation upgrading to Cacti development (1.3). For a package-managed installation, use the package maintainer's upgrade procedure and configuration-file handling. See the separate guides for Windows and FreeBSD.

Review the release notes and requirements first. Schedule time to back up and restore the installation. Test the upgrade on an isolated copy of the database and files before changing production. Keep dependency upgrades separate unless a prerequisite requires them.

Stop writers before backing up

Turn off **Data Collection Enabled** under the poller settings. Also suspend the actual scheduler on each affected collector: disable only the Cacti cron entry, or stop its installed `cactid` service. Preserve the scheduler settings so they can be restored later. Do not disable all system cron jobs.

Wait for existing poller, Spine, script-server, Boost and other background jobs to finish. Prevent web requests and plugin jobs from changing the installation during the backup and upgrade. The application polling switch alone does not stop every running process or administrative writer.

If collectors use `rrdcached`, drain its pending writes using the procedure for that deployment before taking the RRD snapshot. Keep writers stopped until the database and file backups are both complete. Coordinate this on remote collectors as well as the main server.

Back up the database and complete installation

Record the installed Cacti, Spine, PHP and database versions, installation paths, service accounts, scheduler entries and remote-collector configuration. Store backups outside the web root with access restricted to administrators.

Back up the Cacti database with a client compatible with the database server. For example, with the application stopped and a backup account with the required privileges:

```
umask 077
BACKUP_DIR=/srv/backups/cacti-before-1.3-dev
mkdir "$BACKUP_DIR"
mysqldump --user=backup_user --password --lock-tables --quick \
  --routines --events --triggers cacti \
  > "$BACKUP_DIR/cacti.sql"
```

Adjust the database name, account and paths before running these examples. Use `mariadb-dump` instead of `mysqldump` where appropriate. Enter the password at the prompt. Stop if a command fails; an output file alone does not prove a successful dump. The account needs the privileges required by the installed dump client, including table locks for the example above.

`--single-transaction` provides a consistent snapshot for transactional tables; it does not make nontransactional tables consistent. Inspect the actual storage engines and choose the backup method accordingly. Keep all application writers stopped for the corresponding file snapshot. See the database backup documentation for the option and privilege details.

Back up the complete installation, including the RRD files. For an example installation at `/var/www/html/cacti`:

```
tar -czpf "$BACKUP_DIR/cacti-files.tar.gz" -C /var/www/html cacti
tar -tzf "$BACKUP_DIR/cacti-files.tar.gz"
```

Preserve `include/config.php`, plugins, custom scripts, query resources and other local configuration. Include CSRF secret material, including an alternate `$path_csrf_secret` from the configuration. Back up separately any RRD, configuration or resource paths outside the installation tree. A tar archive normally preserves a symlink rather than including its external target. Retain the permissions and ownership needed for restoration.

Save the database user/grant definitions and any connection certificates needed to recreate access. These are not included in a dump of the Cacti database alone. Include site or plugin SQL routines and events, as in the example, and account for their additional backup privileges.

Back up remote-collector databases and local files where they hold state. Verify dump completion and archive contents, then rehearse restoration on an isolated system. Excluding `*.rrd` would leave monitoring history outside the recovery set.

Check development dependencies

Development source requires PHP 8.3 or later in the PHP 8 series, with the extensions declared by the selected revision. Install locked dependencies in the staging directory with `composer install --no-dev`, then run `composer check-platform-reqs --no-dev`. Preserve this complete dependency tree when deploying the staged application. Do not run `composer update` against a live installation. A complete release bundle can already include dependencies; verify its platform checks instead of assuming that every archive is complete.

Install the staged files

Obtain the intended development revision and stage it outside the live installation. Record its commit and verify its provenance. If using a release archive instead, follow that release's requirements and verification steps. Keep the existing installation and its backup until the upgrade has been checked.

Copy the contents of the prepared staging directory into the existing installation, not the directory itself. For example:

```
STAGING_DIR=/var/tmp/cacti-1.3-dev
CACTI_DIR=/var/www/html/cacti
cp -a "$STAGING_DIR"/. "$CACTI_DIR"/
```

Confirm that the new files are directly under the installation root, such as `$CACTI_DIR/poller.php`, rather than a nested `cacti-1.3-dev` directory. Preserve the existing `include/config.php` and review new options in `include/config.php.dist`. Reconcile local code changes and plugin compatibility with the new files; copying the release is not a substitute for reviewing those changes.

Check permissions using the accounts that actually run the web server, poller and maintenance jobs. Those accounts need not be identical. Give the poller the required access to RRD and log paths, and check configured cache, resource and script paths. Preserve access needed by installation and package imports. Do not make the entire application tree writable as a permissions workaround.

Run and verify the upgrade

Allow an administrator to open the Cacti URL and follow the database-upgrade workflow. Review errors before proceeding. The release also provides `cli/upgrade_database.php`; consult its `--help` output if using the CLI workflow. Do not manually advance the database version or force a starting version merely to bypass an upgrade check.

Confirm the installed version, database-upgrade result, credentials, paths, plugin state and collector configuration. Check remote collectors individually and follow the applicable synchronization procedure; do not assume that two polling cycles prove their files and databases have upgraded successfully.

Restore web access and the saved scheduler configuration only after the upgrade checks succeed, then re-enable collection. Use either the configured cron path or the daemon for a collector, not both. Verify logs, data freshness and representative RRD updates after normal polling resumes.

Recovery

If the upgrade fails, keep writers stopped while deciding whether to retry or restore. Replacing PHP files alone does not undo a database migration. Restore the matching pre-upgrade database, application files, RRD data and external configuration or secret material, including affected remote-collector state. Verify the restored installation before re-enabling its schedulers. Restoring an earlier backup discards changes collected after that backup.

Copyright (c) 2004-2026 The Cacti Group

Upgrading Cacti under Windows

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

These instructions cover upgrading a source/archive installation to Cacti 1.3-dev. For a managed Windows installation, also follow its package or installer upgrade procedure; do not overwrite package-managed files with an unrelated archive. Review the release notes and requirements for the target Cacti version first. An upgrade of Cacti does not automatically require upgrading Windows, PHP or the database.

The current source manifest requires `ext-pcntl` and `ext-posix`, which native Windows PHP cannot provide. Read the Windows installation prerequisite before scheduling downtime. Stop if the selected artifact rejects Windows.

Check development dependencies

Development source requires PHP 8.3 or later in the PHP 8 series, with the extensions declared by the selected revision. Install locked dependencies in the staging directory with `composer install --no-dev`, then run `composer check-platform-reqs --no-dev`. Preserve this complete dependency tree when deploying the staged application. Do not run `composer update` against a live installation. A complete release bundle can already include dependencies; verify its platform checks instead of assuming that every archive is complete.

Pause collection and take a recoverable backup

1. In `Console > Configuration > Settings > Poller`, clear **Data Collection Enabled** and save. Disable the actual Windows Scheduled Task that launches `poller.php`, or stop the service that manages your poller. Record the task or service settings so you can restore them. Use Task Scheduler to disable future starts; this alone does not finish an already running poll.

2. Wait for running pollers, Spine processes, script servers and background writers to finish. Include remote collectors and plugin jobs. Restrict web access to the upgrade administrator so configuration changes cannot race the backup. Flush pending RRD-cache writes before copying RRD files and confirm that writers have stopped. Do not start a manual poll during this maintenance window.

3. Create a protected backup directory outside the web root. From Command Prompt, use the dump utility matching your database server. Replace the executable path, account, database name and output path in this example. Use the actual database host, port and TLS options for your installation. The account needs the backup privileges required by the installed server; `--password` prompts rather than putting a password in the command.

   ```
   "C:\path\to\mysqldump.exe" --user=YOUR_BACKUP_USER --password ^
     --lock-tables --quick --routines --events --triggers ^
     --databases cacti --result-file="D:\CactiBackup\cacti-YYYYMMDD.sql"
   ```

   With all Cacti writers paused, this dump example uses table locks to cover transactional and nontransactional tables in the Cacti database. A transaction-only dump does not protect nontransactional tables. Use a new output filename and verify successful completion: a failed dump may leave a partial file. Do not proceed until the backup can be restored in a separate test environment.

4. Back up the existing Cacti tree, including hidden files, using a separate destination outside the web root. For example, Robocopy can preserve file permissions as well as contents when run with the required privileges:

   ```
   robocopy "C:\inetpub\wwwroot\cacti" "D:\CactiBackup\cacti-YYYYMMDD" /E /COPYALL /SL /XJ /R:2 /W:2
   ```

   Inspect its report; exit codes of 8 or greater indicate copy failures. Confirm that the backup includes `include/config.php`, RRDs, custom scripts, query resources, plugins and secret files, including any alternate `path_csrf_secret`. Also save configured paths outside this tree, the Spine configuration, scheduler settings, web-

server configuration, database grants and required certificates/keys. Record symbolic links and junctions and back up their external targets separately; the example copies symbolic links and excludes junctions. Preserve permissions and record the installed versions. Keep this file backup and database dump together as one recovery set.

Install and verify the upgrade

1. Extract the target Cacti release into a staging directory. Keep the existing tree and verified backup available. Deploy the release files using your existing installation layout, preserving the working `include/config.php` and local resources. Compare the new `include/config.php.dist` for changes; do not replace existing credentials, collector IDs, paths or secrets with example defaults. Check plugin and custom-script compatibility before restoring or enabling them.
2. Verify file permissions for the web server and poller accounts. Open the existing Cacti URL as an administrator and complete the database upgrade. Review installer checks and logs. Use version-appropriate database settings from Requirements, rather than applying fixed global SQL tuning commands.
3. For remote collectors, keep their schedulers paused while updating their local application files and configuration and completing the applicable upgrade and synchronization. Verify each collector's reported version, connectivity and logs; completion is not guaranteed after two poll cycles. Check the matching Spine version where it is used.
4. Re-enable **Data Collection Enabled** and restore the recorded scheduler or service only after the upgrade succeeds. Check a complete poll cycle, collector status, new RRD samples and representative graphs. If recovery is necessary, stop writers again and restore the matching application files, database, RRDs and configuration together; copying old PHP files over an upgraded database is not a rollback.

Copyright (c) 2004-2026 The Cacti Group

Upgrading Cacti Under FreeBSD

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

These instructions cover the FreeBSD upgrade procedure. This manual describes Cacti development (1.3), while the FreeBSD port currently supplies Cacti 1.2.31. Use the 1.2.x manual for release-specific requirements when upgrading the packaged release. Determine whether Cacti is maintained by `pkg`, by ports or from a source archive. Keep that installation method for the upgrade; do not overlay an archive onto package-managed files. Review the target Cacti release notes, requirements and package or port notes before proceeding. Do not make an unrelated operating-system, PHP or database major-version upgrade part of this procedure without a separate compatibility plan.

Pause collection and back up

1. Clear **Data Collection Enabled** under `Console > Configuration > Settings

   > Poller`. Disable the Cacti-specific cron entry or stop its configured poller service on every collector. Record what you change; do not stop the system-wide cron service merely to pause Cacti. Wait for pollers, Spine, script servers and other background writers to finish. Restrict web access to the upgrade administrator and pause plugin jobs that write to Cacti. Flush pending RRD-cache writes and confirm that the database and RRD files remain unchanged before taking the backup.

2. Use the dump client appropriate to the installed MySQL or MariaDB server. The following example requires an existing, protected backup directory and a database account with the server's required backup privileges. Replace the account, database name and filename for your installation. Use its actual database host, port and TLS options.

   ```
   mysqldump --user=YOUR_BACKUP_USER --password \
     --lock-tables --quick --routines --events --triggers \
     --databases cacti --result-file=/protected/backup/cacti-YYYYMMDD.sql
   ```

   The password is prompted. With all Cacti writers paused, this dump method uses table locks for the Cacti database, including nontransactional tables. A transaction-only dump does not cover those tables. Verify the command succeeded and test restoring the dump separately; a nonempty file alone is not proof of a complete backup. Use a new filename rather than overwriting a known good backup.

3. Back up the Cacti application files, RRDs and local configuration together with the database dump. Preserve ownership, permissions and hidden files. Include `include/config.php`, custom scripts, query resources, plugins, secret files, including any alternate `path_csrf_secret`, and configured paths outside the application tree. Record symbolic links and back up their targets separately. Also record the Spine configuration, cron/service settings, web-server settings, database grants, certificates/keys and installed versions. Package layouts may keep configuration and data outside `/usr/local/share/cacti`; inspect the installed paths rather than backing up that directory alone. Include `/var/db/cacti/rra` and `/var/log/cacti` when those are the configured package paths, and protect the backup directory from other users because the configuration contains credentials.

Upgrade using the existing installation method

For a binary package installation, review the proposed package transaction before applying it. The FreeBSD Handbook describes package upgrades. Run the following with appropriate administrator privileges:

Discover the installed flavor first:

```
pkg info -o -x '^cacti'
pkg update
```

For an installed `cacti-php84` package, for example:

```
pkg upgrade -n cacti-php84
pkg upgrade cacti-php84
pkg info -D cacti-php84
```

Replace `cacti-php84` with the installed package name. Keep the PHP flavor unless a separate migration is intended. If Spine is installed, review its upgrade and verify its version remains compatible with Cacti. See the FreeBSD installation section for package paths and the `net-mgmt/cacti-spine` port.

Check the proposed version and dependency changes before accepting the actual upgrade. A package repository normally supplies a release rather than development source. Stop if that is not your approved target and select an appropriate repository or installation method. Do not delete Cacti first or assume that upgrading one package cannot affect its dependencies.

For a ports-managed installation, follow the handbook's ports upgrade procedure and applicable `/usr/ports/UPDATING` entries. Use the existing ports management method; do not switch between ports and binary packages without reviewing their options and dependencies. A source installation instead follows Upgrading Cacti.

Preserve the working configuration and merge required changes from the new configuration example. Keep existing database credentials, collector IDs, paths and secrets. Review package-generated configuration changes and restore local resources only after checking compatibility. Recreate cache or data directories only where the installed configuration requires them, with access for the actual service account; do not assume a universal account or apply recursive ownership changes to the whole installation.

Check development dependencies

Development source requires PHP 8.3 or later in the PHP 8 series, with the extensions declared by the selected revision. Install locked dependencies in the staging directory with `composer install --no-dev`, then run `composer check-platform-reqs --no-dev`. Preserve this complete dependency tree when deploying the staged application. Do not run `composer update` against a live installation. A complete release bundle can already include dependencies; verify its platform checks instead of assuming that every archive is complete.

Complete the application upgrade

Open the existing Cacti URL as an administrator and complete its database upgrade. Review the installer and logs before resuming collection. Upgrade remote collectors' local files and configuration through their installation method, complete the applicable upgrade/synchronization, and verify their versions, connectivity and logs. Do not rely on a fixed number of poll cycles for completion. Check the matching Spine version on collectors using Spine.

After successful checks, re-enable **Data Collection Enabled** and restore the recorded Cacti cron entry or service. Verify a complete poll cycle, recent RRD samples and representative graphs on every collector. If recovery is needed, stop all writers again and restore the matching files, database, RRDs and configuration as one set; restoring old application files alone does not undo a database upgrade.

Copyright (c) 2004-2026 The Cacti Group

Hardening a Cacti Installation

This page covers OS-level hardening controls for a production Cacti host. The goal is defense in depth: even if a vulnerability in Cacti's PHP code is exploited, the attacker's reach should be limited.

Why These Controls Matter

A recurring class of vulnerability in Cacti involves writing an arbitrary file into the `resource/` or `scripts/` tree and then reaching that file over the web. The package-import advisory (GHSA-7cmj-g5qc-pj88, CVE-2024-25641) is a concrete example: a crafted package archive could write `resource/test.php` and execute it as a web request. The controls below close or limit that path at the OS and web-server layers, independent of the PHP fix.

---

1. File Permissions

Cacti writes to three paths at runtime. Everything else should be owned by root (or a non-web user) and not writable by the web process.

Path	Purpose	Recommended mode
`rra/`	RRDtool data files	`www-data:www-data 750`
`log/`	Cacti log files	`www-data:www-data 750`
`cache/`	Boost / poller cache	`www-data:www-data 750`
`resource/`	XML templates	`root:root 755` (read-only for web)
`scripts/`	Data-source scripts	`root:root 755` (read-only for web)
`lib/`, `include/`	PHP libraries	`root:root 755`

Apply once after installation:

```
# Writable paths (web user only)
chown -R www-data:www-data /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache
chmod -R 750              /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache

# Everything else read-only for the web user
chown -R root:root         /var/www/html/cacti
chmod -R 755               /var/www/html/cacti
# Re-apply writable exceptions after the recursive chmod
chown -R www-data:www-data /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache

# Protect the database credentials in config.php.  The recursive 755 above
# leaves it world-readable; restrict it to root and the web user.
chown root:www-data /var/www/html/cacti/include/config.php
chmod 640           /var/www/html/cacti/include/config.php
```

> **Note:** The `scripts/` and `resource/` paths may be written during template import or package import by a privileged CLI process running as root, not by the web process. If your workflow requires the web process to write there, use a strict code-review policy for any import and consider the compat profile instead.

---

2. Web Server — Block Direct Access to Scripts and Resources

PHP files inside `scripts/` and `resource/` must not be reachable via HTTP. These directories hold data-source scripts and XML templates; a web request to a `.php` file written there is the direct exploitation path.

Apache

`<DirectoryMatch>` is only valid in server config or `<VirtualHost>` context, not `.htaccess`. Add this inside your Cacti `<VirtualHost>`:

```
# Deny direct HTTP access to the script and resource trees.
# A PHP file written here via import cannot be reached over the web.
<DirectoryMatch "^/var/www/html/cacti/(scripts|resource)/">
    <FilesMatch "\.php$">
        Require all denied
    </FilesMatch>
</DirectoryMatch>
```

If you only have access to `.htaccess` (e.g. shared hosting), use a `FilesMatch` block inside each directory instead:

```
# scripts/.htaccess and resource/.htaccess
<FilesMatch "\.php$">
    Require all denied
</FilesMatch>
```

Or, more broadly, deny all non-XML/non-script files from direct access:

```
<Directory "/var/www/html/cacti/scripts">
    Require all denied
</Directory>

<Directory "/var/www/html/cacti/resource">
    <FilesMatch "\.php$">
        Require all denied
    </FilesMatch>
</Directory>
```

Nginx

Add inside your `server {}` block:

```
# Block PHP execution in scripts/ and resource/
location ~ ^/cacti/(scripts|resource)/.*\.php$ {
    return 403;
}
```

---

3. PHP-FPM Pool Hardening

PHP itself warns that php-fpm pools are not a true security boundary, but they provide useful knobs. Create a dedicated pool for Cacti (`/etc/php/8.x/fpm/pool.d/cacti.conf`):

```
[cacti]
user  = www-data
group = www-data

listen = /run/php/php-cacti.sock
listen.owner = www-data
listen.group = www-data
```

```
; Drop all inherited environment variables — pass only what Cacti needs.
clear_env = yes

; Only execute .php files — blocks .phtml, .php3, .phar, etc.
security.limit_extensions = .php

; Disable dangerous functions at the pool level (not overridable by ini_set).
; Cacti's web process shells out to RRDtool, fping and helper binaries via
; exec, shell_exec, system, popen and proc_open (see lib/rrd.php, lib/ping.php,
; lib/spikekill.php), so disabling those breaks graph rendering and diagnostics.
; Only passthru and pcntl_exec are unused by the web tier and safe to disable.
; For tighter control, constrain the process with open_basedir and a hardened
; PATH rather than removing the shell-exec family Cacti depends on.
php_admin_value[disable_functions] = passthru,pcntl_exec

; Restrict file operations to the Cacti tree.
; NOTE: open_basedir disables the PHP realpath cache.  Measure the
; performance impact before enabling in production.  It is an extra
; safety net, not a full security boundary (PHP documentation).
; php_admin_value[open_basedir] = /var/www/html/cacti:/tmp

; Suppress PHP version disclosure.
php_admin_flag[expose_php] = Off
```

Reload php-fpm after changes:

```
systemctl reload php8.x-fpm   # Debian/Ubuntu
systemctl reload php-fpm       # RHEL/Rocky/Alma
```

---

4. SELinux (RHEL / AlmaLinux / Rocky Linux)

SELinux ships enforcing on RHEL-family systems. These commands apply the correct file contexts so Apache can serve Cacti while limiting what the web process can write.

File Contexts

```
CACTI=/var/www/html/cacti

# Read-only web content (Apache can read, not write)
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}(/.*)?"
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}/scripts(/.*)?"
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}/resource(/.*)?"

# Read-write content for runtime paths only
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/rra(/.*)?"
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/log(/.*)?"
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/cache(/.*)?"

# Apply the contexts
restorecon -Rv "${CACTI}"
```

Database Network Access

If your database server is reached over TCP (not a local Unix socket), enable the SELinux boolean that permits it:

```
setsebool -P httpd_can_network_connect_db on
```

Poller Systemd Drop-in

The Cacti spine poller runs as a separate daemon and may need to connect outward for SNMP. Do not force it into the `httpd_t` domain: that confines spine to Apache's policy rather than freeing it, and outbound access in `httpd_t` is still gated by booleans. If SNMP polling is blocked by SELinux, enable the relevant boolean instead and leave the daemon in its own domain:

```
setsebool -P httpd_can_network_connect on
```

Run `systemctl daemon-reload && systemctl restart cactid` after changes.

> **Verify:** After applying contexts, confirm Apache can still serve pages and the poller can poll. Use `audit2why` on any AVC denials in `/var/log/audit/audit.log`.

---

5. AppArmor (Debian / Ubuntu)

Ubuntu Server ships AppArmor enforcing by default. Create a profile for the Cacti php-fpm pool at `/etc/apparmor.d/cacti-fpm`:

```
#include <tunables/global>

profile cacti-fpm /usr/sbin/php-fpm* {
    #include <abstractions/base>
    #include <abstractions/php>
    #include <abstractions/mysql>

    # Cacti tree — read-only by default
    /var/www/html/cacti/           r,
    /var/www/html/cacti/**         r,

    # Allow execution of Cacti PHP files
    /var/www/html/cacti/*.php      r,
    /var/www/html/cacti/**/*.php   r,

    # Runtime write paths
    /var/www/html/cacti/rra/**     rw,
    /var/www/html/cacti/log/**     rw,
    /var/www/html/cacti/cache/**   rw,

    # Helper binaries Cacti shells out to.  AppArmor is default-deny once
    # file rules are present, so these need explicit execute (ix) or graphs
    # and polling break under enforce mode.
    /usr/bin/rrdtool               ix,
    /usr/bin/fping                 ix,
    /usr/bin/ping                  ix,
    /usr/bin/snmpget               ix,
    /usr/bin/snmpbulkwalk          ix,
    /usr/bin/php                   ix,

    # Temp files
    /tmp/**                        rw,
    /var/tmp/**                    rw,

    # Deny write to scripts/ and resource/
    deny /var/www/html/cacti/scripts/**  w,
    deny /var/www/html/cacti/resource/** w,
}
```

Load and enforce the profile:

```
apparmor_parser -r /etc/apparmor.d/cacti-fpm
aa-enforce /etc/apparmor.d/cacti-fpm
```

Attach the Profile to the php-fpm Service

Add a drop-in at `/etc/systemd/system/php8.x-fpm.service.d/apparmor.conf`:

```
[Service]
AppArmorProfile=cacti-fpm
```

Reload and restart:

```
systemctl daemon-reload
systemctl restart php8.x-fpm
```

Use complain mode (`aa-complain`) first to test without blocking, then switch to enforce once logs show no legitimate denials:

```
# Test mode
aa-complain /etc/apparmor.d/cacti-fpm

# Enforce when ready
aa-enforce  /etc/apparmor.d/cacti-fpm
```

---

6. Verification Checklist

After applying all controls, confirm:

- [ ] `curl -s http://localhost/cacti/scripts/ss_webseer.php` returns 403
- [ ] `curl -s http://localhost/cacti/resource/` returns 403 for any `.php` inside
- [ ] The Cacti web UI loads and graphs render (rra/ is writable)
- [ ] The poller runs without errors (`tail -f /var/www/html/cacti/log/cacti.log`)
- [ ] On RHEL: no AVC denials in `ausearch -m avc -ts recent`
- [ ] On Debian/Ubuntu: no AppArmor denials in `journalctl -k | grep apparmor`

See Also

- Installing Under Ubuntu/Debian
- Installing Under Enterprise Linux (LAMP)
- Standards Security

Copyright (c) 2004-2026 The Cacti Group

Configuring SSL for Cacti

Enabling HTTPS for Cacti is done at the web server level. This page covers self-signed certificates for internal use and Let's Encrypt for public-facing servers.

> **Note**: If using multiple Data Collectors, all must have HTTPS enabled for the remote polling feature to work correctly.

Self-Signed Certificate (Internal Use)

Self-signed certificates are suitable for internal networks where you control the clients. Use ECDSA (secp384r1) for best performance, or RSA 4096 if your environment requires RSA.

ECDSA (recommended)

```
# RHEL/Rocky/AlmaLinux
dnf install -y mod_ssl

# Generate a 384-bit EC private key and self-signed certificate (valid 3 years)
openssl req -x509 -nodes -newkey ec -pkeyopt ec_paramgen_curve:P-384 \
    -keyout /etc/pki/tls/private/cacti.key \
    -out /etc/pki/tls/certs/cacti.crt \
    -days 1095 \
    -subj "/CN=cacti.example.com" \
    -addext "subjectAltName=DNS:cacti.example.com"

chmod 600 /etc/pki/tls/private/cacti.key
```

RSA (alternative)

```
# 4096-bit RSA — use when EC is not supported by your TLS client pool
openssl req -x509 -nodes -newkey rsa:4096 \
    -keyout /etc/pki/tls/private/cacti.key \
    -out /etc/pki/tls/certs/cacti.crt \
    -days 1095 \
    -subj "/CN=cacti.example.com" \
    -addext "subjectAltName=DNS:cacti.example.com"

chmod 600 /etc/pki/tls/private/cacti.key
```

> **Debian/Ubuntu paths**: the commands above use the RHEL layout (`/etc/pki/tls/...`). On Debian/Ubuntu, write the key and certificate to `/etc/ssl/private/cacti.key` and `/etc/ssl/certs/cacti.crt` instead, to match the Apache `SSLCertificateKeyFile`/`SSLCertificateFile` paths shown below.

> **Key size**: RSA 2048-bit is below NIST SP 800-131A Rev 2 guidance for long-term use. Use 4096-bit RSA or an ECDSA key.

Apache Configuration

Update `/etc/httpd/conf.d/ssl.conf` (RHEL/Rocky/AlmaLinux) or `/etc/apache2/sites-available/cacti-ssl.conf` (Debian/Ubuntu):

```
<VirtualHost *:443>
    ServerName cacti.example.com
    DocumentRoot /var/www/html/cacti

    SSLEngine on
    # RHEL/Rocky/AlmaLinux cert paths:
```

```
    #   SSLCertificateFile    /etc/pki/tls/certs/cacti.crt
    #   SSLCertificateKeyFile /etc/pki/tls/private/cacti.key
    # Debian/Ubuntu cert paths:
    SSLCertificateFile    /etc/ssl/certs/cacti.crt
    SSLCertificateKeyFile /etc/ssl/private/cacti.key

    # Require TLS 1.2 or 1.3; TLS 1.0 and 1.1 are prohibited (RFC 8996)
    SSLProtocol           -all +TLSv1.2 +TLSv1.3

    # TLS 1.2 cipher suites -- no RC4, 3DES, or export ciphers
    SSLCipherSuite        ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:\
ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:\
ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305
    SSLHonorCipherOrder   on

    # TLS 1.3 cipher suites (SSLCipherSuite does not apply to TLS 1.3)
    SSLOpenSSLConfCmd     Ciphersuites TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM_SHA256
</VirtualHost>

# Optional: redirect HTTP to HTTPS (uncomment to enable)
#<VirtualHost *:80>
#    ServerName cacti.example.com
#    Redirect permanent / https://cacti.example.com/
#</VirtualHost>
```

Restart Apache after editing:

```
apachectl configtest
systemctl restart httpd        # RHEL/Rocky/AlmaLinux
systemctl restart apache2      # Debian/Ubuntu
```

Let's Encrypt (Public-Facing Servers)

For servers reachable from the internet, use Let's Encrypt for a browser-trusted certificate at no cost.

RHEL/Rocky/AlmaLinux

```
# certbot ships in EPEL on RHEL/Rocky/AlmaLinux, not in BaseOS/AppStream
dnf install -y epel-release
dnf install -y certbot python3-certbot-apache
certbot --apache -d cacti.example.com
```

Debian/Ubuntu

```
apt-get install -y certbot python3-certbot-apache
certbot --apache -d cacti.example.com
```

Certbot configures Apache and sets up automatic renewal. Verify auto-renewal works:

```
certbot renew --dry-run
```

Verifying the Configuration

After restarting Apache, confirm TLS is working and that weak protocols are disabled:

```
# Check the certificate and protocol support
openssl s_client -connect cacti.example.com:443 -tls1_2 < /dev/null
openssl s_client -connect cacti.example.com:443 -tls1   < /dev/null  # should fail
```

For a comprehensive scan, use SSL Labs on public servers or `testssl.sh` on internal hosts:

```
# testssl.sh (download from https://testssl.sh/)
./testssl.sh cacti.example.com
```

Copyright (c) 2004-2026 The Cacti Group

Version Specific Release Notes

Summary

Below you will find the release note for all versions of Cacti 1.0.0 and above. Release notes for earlier Cacti versions have been excluded due to the large list of changes that have occurred over the years, and since Cacti 0.8.x is no longer supported, it's a good time to for the purposes of this document, move them aside.

Before that list, let's talks about Cacti 1.0.0 first.

Cacti 1.0.0

This was the initial release of Cacti 1.x. With the release of Cacti 1.0.0 many improvements and enhancements have been made. As part of ongoing efforts to improve Cacti almost 20 plugins were merged into the core of Cacti eliminating the need for the plugins. A major refresh of the interface has been started and will continue to occur as development on Cacti continues.

Plugins integrated into Cacti Core

The following plugins have been merged into the core Cacti code as of version 1.0.0:

Plugin	Description
snmpagent	An SNMP Agent extension, trap and notification generator
clog	Log viewers for administrators
settings	Core plugin providing email and DNS services
boost	Large system performance boost plugin
dsstats	Cacti data source statistics
watermark	Watermark graphs
ssl	Force https connection
ugroup	User groups support
domains	Multiple authentication domains
jqueryskin	User interface skinning
secpass	C3 level password and site security
logrotate	Log management
realtime	Realtime graphing
rrdclean	RRDfile maintenance
nectar	Email based graph reporting
aggregate	Templating, creation and management of aggregate graphs
autom8	Graph and Tree creation automation
discovery	Network Discovery and Device automation
spikekill	Removes spikes from Graphs
superlinks	Allows administrators to links to additional sites

ChangeLog for All Cacti 1.x Releases

The following shows the naming convention used for issues and features listed in the CHANGELOG for both cacti and it's plugins

Category	Description
issue[issue#]	Normal issues either experienced by a Developer or logged in GitHub and fixed
feature[issue#]	Features either implemented or logged in GitHub and addressed
security[issue#]	Security vulnerability either fixed or logged in GitHub and addressed
bug[bug#]	Feature of Bug reported in the old Cacti Mantis system that was addressed.

Below are a few examples of the version, type and description that can be found in the CHANGELOG

Version	Feature/Bug	Description
1.2.17	issue#3981	PHP ERROR: Call to undefined function api_data_source_cache_crc_update()
1.2.17	feature#4013	Remote Poller - reset avg/max polling time
1.2.16	issue#3704	When generating a report, the Cascade to Branches function does not as expected
1.2.16	issue#3859	When viewing graphs, automatic refresh so not always work as expected
1.2.16	feature	Update Chart.js to version 2.9.4
1.2.16	feature#3923	Add ability to hide the Graph Drilldown icons by datatecuk
1.2.16	feature#3943	Add hooks for plugins to show custom Graph Source and custom Template URL (List View)
1.2.13	security#3628	Lack of escaping on template import can lead to XSS exposure
1.2.13	issue#3517	When generating reports, function looping can occur resulting in 100% cpu usage

---

Copyright (c) 2004-2026 The Cacti Group

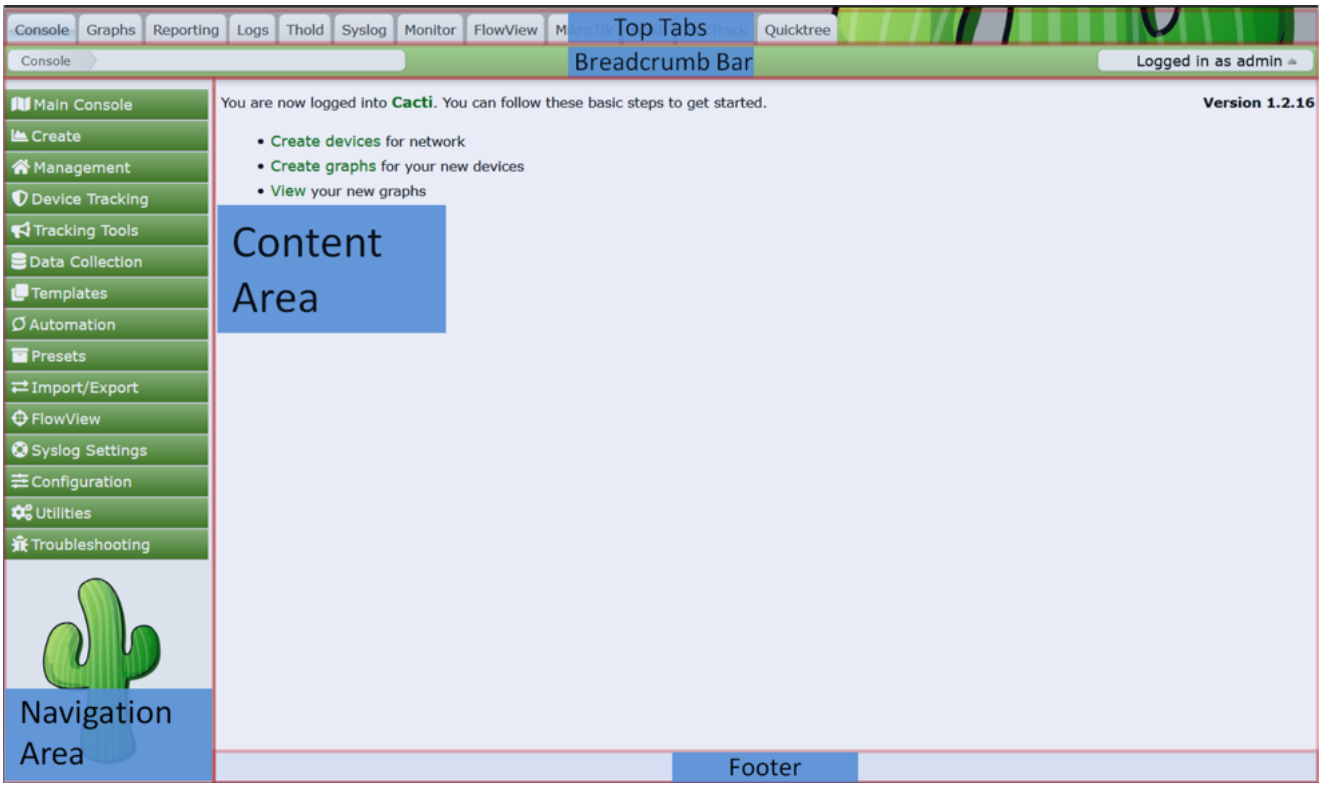
Navigating the Cacti User Interface

Menu names and available tabs depend on your Cacti version, permissions, theme, and installed plugins. The screenshots illustrate navigation and may show an earlier theme or release.

The Cacti User Interface is visually broken into multiple panels. Each major panel is designed to hold content. Depending on the Cacti Theme you use some of these panel may not be visible at the Theme Developers discretion. The common panels are:

- **Top Tabs** - Where you perform major navigation separate content
- **Navigation Area** - If the content in navigated to by the Top Tab has a menu, it will be found here.
- **Breadcrumb Bar** - This is where you can see where in the Cacti interface you are currently pointed to
- **Content Area** - This is where you present tables, charts, forms, etc. It is where your main Content resides
- **Footer** - This section is left for Theme Developers to insert content that belongs at the bottom of the screen.

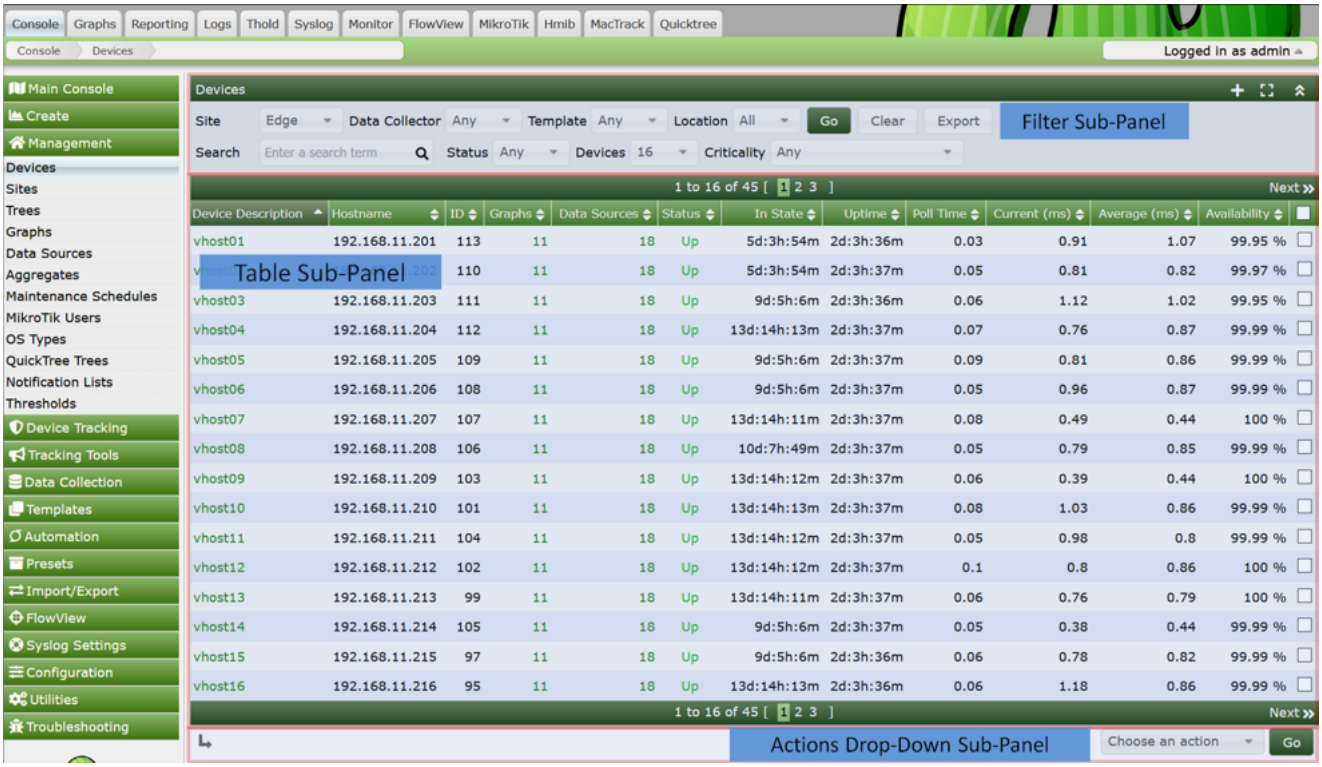
You can see the default Cacti layout with its various panels in the image below. You will note, in this `Modern` Theme, the Theme Author has decided to dispense with the Footer.



In Cacti, when you click on the **Top Tab**, you will by default enter a completely different section of Cacti. Cacti's Top Tabs are designed to mimic browser tabs, this helps with users orienting themselves to the various sections of Cacti. When you have many **Plugins** installed as in the example above, you can see clearly the benefit of these navigation aids.

Inside each of these panels, a page can be broken into sub-panels. Two panels customarily broken into sub-panels include the *Navigation Area* and the *Content Area*.

In the example below, we show the **Device** page in Cacti calling out the various sub-panels.



Most of Cacti's pages are laid out in this fashion. However, what goes into the Cacti **Content Area** is completely under the **Plugin** authors control.

At the Theme developers discretion, all pages should include both the **Top Tab** and **Breadcrumb Bar**. Inside of the **Breadcrumb Bar** or **Top Tab** panels you should always see the *User Profile and Menu* on the right.

To use Cacti properly, you should first understand these sections. We will start by describing the Cacti *Console*.

Cacti Core Panels and Sub-Panels

- **Top Tabs**

  Cacti **Top Tabs** provide Cacti with multiple **Navigation Areas**. By default, Cacti includes four **Top Tabs**. They are *Console*, *Graphs*, *Log* and *Reports*.

- **Breadcrumbs**

  Breadcrumbs appear directly below the **Top Tabs**. Note that some Cacti *Themes* disable the **Breadcrumbs**. You can click on a **Breadcrumb** area to navigate to that area if desired.

- **Cacti Content Area**

  This is where the main page content will be displayed. It is directly below the **Breadcrumbs** or the **Top Tabs** with some Cacti *Themes*. They can include any HTML that the **Plugin Author** or Cacti Administrator desires in the case of **External Links**.

- **Navigation Menu**

  If you click on the Cacti *Console*, you will see an example **Navigation Menu**. These menus can appear on any Plugin based **Top Tab** page in addition to the Cacti *Console*.

- **Cacti Tables**

  These tables are where table based data is rendered in Cacti. **Cacti Tables** are presented using an arcane, though easy to use API.

- **Table Filters**

  Any **Cacti Table** can include a **Table Filters**. These filters can be used to limit the data returned to a **Cacti Table**.

- **Actions Dropdown**

  Any page that includes a **Cacti Table** will generally include an **Actions Dropdown**. These **Actions Dropdown** menus allow you to take action on a table row or rows.

- **User Profile and Menu**

  This is where a **Cacti User** can edit their profile, change their password, logout, or find links to other Cacti information and support.

Non-Administrative users, such as the Cacti Guest account *should* not have access to the Cacti *Console*. The Cacti Guest account should additionally not have access to their *User Profile* as that account is shared with many users.

The Cacti Graphs Top Tab

The Cacti Graphs **Top Tab** is where most Cacti **Graphs** are viewed. By default, the Cacti Graphs **Top Tab** includes three distinct views. They include:

- **Tree View**

  Allows Cacti Users to view **Graphs** in the form of hierarchical **Trees**. These **Trees** are generally constructed by the Cacti Administrator and are controlled either at the **User** or **User Group** level.

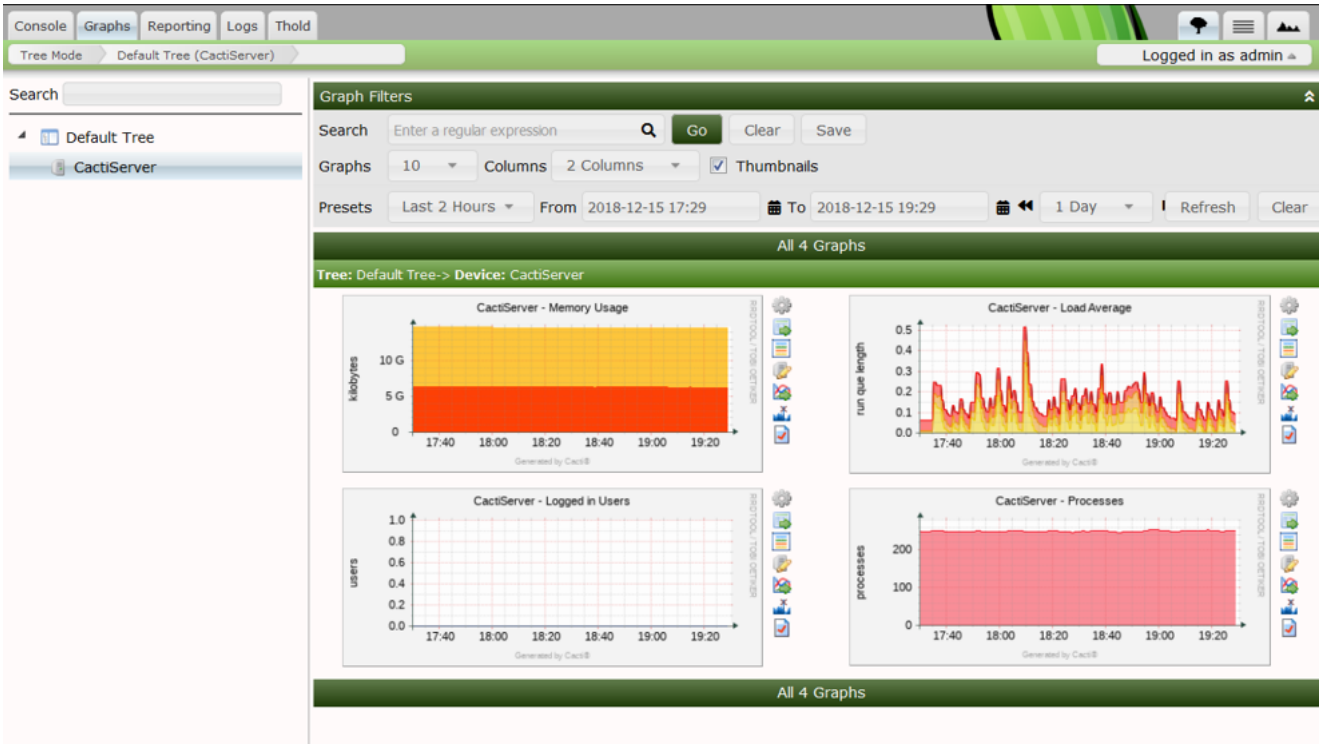
- **Preview View**

  The **Preview View** provides a view of all **Graphs** that a Cacti User has access to. **Table Filters** are provided to constrain the list of **Graphs** returned to the page.

- **List View**

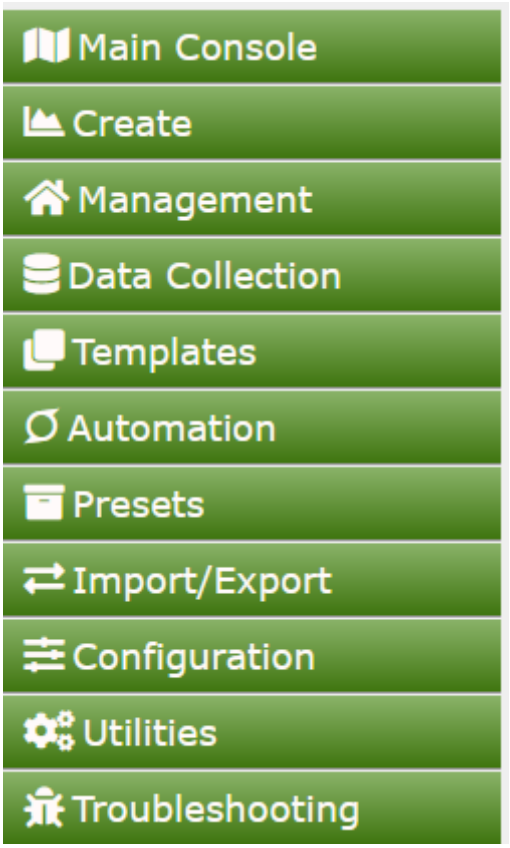
  The **List View** allows the Cacti user to Create their own **Preview Page** by allowing them to select graphs from various pages, and then finally view those pages from the **Preview View**.

In the example **Tree View** page below, you can see the **Tree Navigation Area** to the left, and in the **Cacti Content Area**, you can see the **Graphs** and a **Table Filter** area for constraining the list of **Graphs** returned. You can *Search* the Tree View from the *Search* area above the **Tree Navigation Area**.



The Cacti Console

In the image below, you can see a basic Cacti Console menu area. It is divided into separate sub-menus. We will describe the purpose of each next.



- **Main Console** - This sub-menu pick is fairly benign. It provides an open area. This screen feels like it needs more content if it's the *Main Console*. Fortunately, **Plugin** developers have solved this problem. The **intropage plugin** for example can fulfill that need.
- **Create** - This sub-menu allows you to create both **Devices** and **Graphs**. They are essentially shortcuts to other sub-menu picks.
- **Management** - This is where all core Cacti **Site**, **Graph**, **Device**, **Tree**, **Data Source**, and **Aggregate** non-templated objects reside. When you install Cacti **Plugins**, you will find they extend this sub-menu.
- **Data Collection** - This is where you define rules for **Data Collection** Examples include: **Data Collectors**, **Data Input Methods** and **Data Queries**
- **Templates** - This is where you find all of Cacti's various Templates outside of Automation. By default, You will find Templates for **Graphs**, **Data Sources**, **Devices**, **Aggregates**, and **Colors**.
- **Automation** - This sub-menu is where you find rules for automating **Device** Discovery, and Rules for creating **Devices**, **Graphs**, and **Trees**.
- **Presets** - This area contains meta objects that cross Template boundaries and are Global in nature. They include; **Data Source Profiles**, **CDEFs**, **VDEFs**, **GPrint Presets**, and **Colors**
- **Import Export** - This is how you can import and export various Cacti Template objects.
- **Configuration** - This is where you manage **Users**, **User Groups**, **Login Providers**, **Global Settings**, and **Plugins** on the development branch. Cacti 1.2 has **User Domains** instead.
- **Utilities** - This is where Cacti includes common utilities that can be use in the Web Portal without having to goto the Command line.
- **Troubleshooting** - There are some handy utilities here that help diagnose common problems with Cacti.

All of these objects types will be explained in subsequent sections of the Cacti documentation. For now, it's important just to know that these pages exist.

Copyright (c) 2004-2026 The Cacti Group

Principles of Operation

Cacti collects measurements, stores them in Round Robin Database (RRD) files, and uses RRDtool to render graphs. A MySQL or MariaDB database holds device configuration, templates, permissions, polling state, and intermediate results. RRD files hold the time-series measurements.

Devices and sites

A Device represents something Cacti monitors, such as a server, switch, sensor, or service. Its configuration identifies the responsible data collector, address, availability checks, and SNMP settings where needed. Site assignments organize devices by location; they do not create separate database or security boundaries.

Device Templates associate graph templates and data queries with a device. Automation can discover devices and apply rules for creating devices, graphs, and tree entries.

Data collectors and scheduling

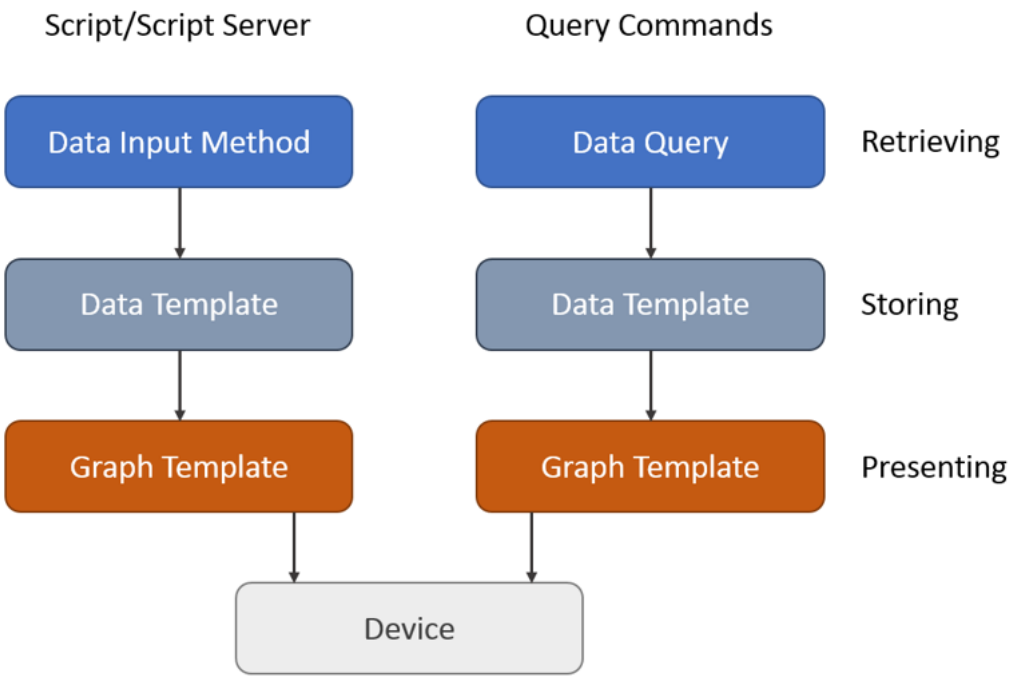
The main Data Collector runs on the central Cacti system. Remote collectors poll devices closer to their location and exchange configuration and results with the main system. During a loss of connectivity, a configured remote collector can retain results locally for later delivery. This does not replace backups or provide automatic failover of the main server.

An operating-system scheduler or the Cacti daemon starts `poller.php`. It coordinates collection and launches the configured engine: the PHP `cmd.php` poller or the C-based Spine poller. The scheduler interval, poller interval, and data-source profiles must agree; changing cron alone does not change an existing RRD file's sampling interval.

Data retrieval

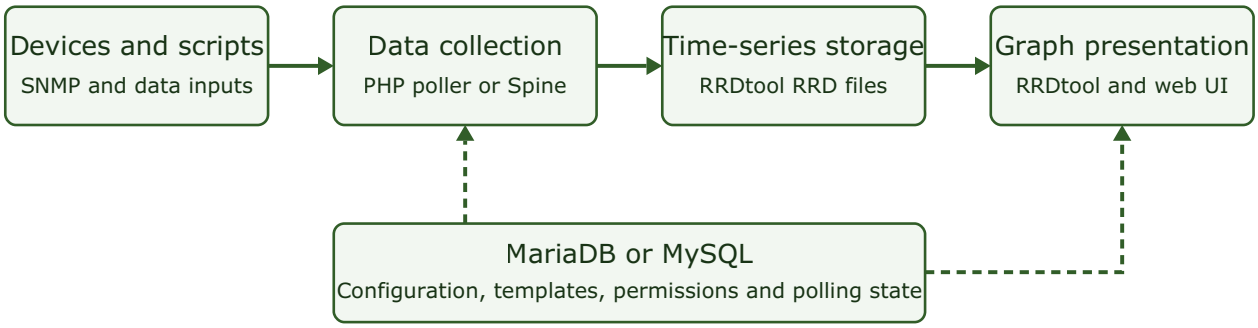
Cacti uses SNMP, external scripts, and resident PHP script-server functions. The available SNMP algorithms depend on the device, Net-SNMP, PHP, and Spine builds. Scripts execute with the poller's operating-system permissions, not with an interactive administrator's environment.

Data Input Methods describe commands and their input and output fields. Data Queries enumerate indexed objects, such as network interfaces or disk partitions. Query input fields describe those objects; output fields provide measurements to collect.



Collection and storage

Data Sources connect collection definitions to RRD files and the series stored in them. The poller writes results to database output tables. Cacti's update processing writes those measurements to RRD files, either during normal polling or through Boost when on-demand updating is enabled. Spine does not directly render graphs or replace the RRD update process.



Data Source Profiles define sampling, heartbeat, consolidation functions, and retention when files are created. RRD archives have bounded storage for their configured layout. Older measurements are consolidated or expire according to that layout; changing a graph's timespan does not recover detail already consolidated away.

An RRD file can contain multiple named series. Database data-source counts, poller item counts, graph counts, and RRD file counts therefore need not match.

Graphing

Graph Templates define presentation settings and graph items. A graph item can draw a series, calculate a value, or add legend text. Several graphs can reuse an existing data source without collecting it again. CDEFs, VDEFs, and GPRINT presets control calculations and formatting at graph-rendering time.

Users view permitted graphs through the tree, list, and preview views described in Viewing Graphs. Permissions and tree publication control which graphs a user can reach; adding a graph to a tree does not start polling.

Extensions

Plugins add features through Cacti's plugin API. Check each plugin's release compatibility, installation instructions, and permissions before enabling it. Template import and export share definitions; they do not transfer historical RRD measurements. See Plugin Management and Template Import.

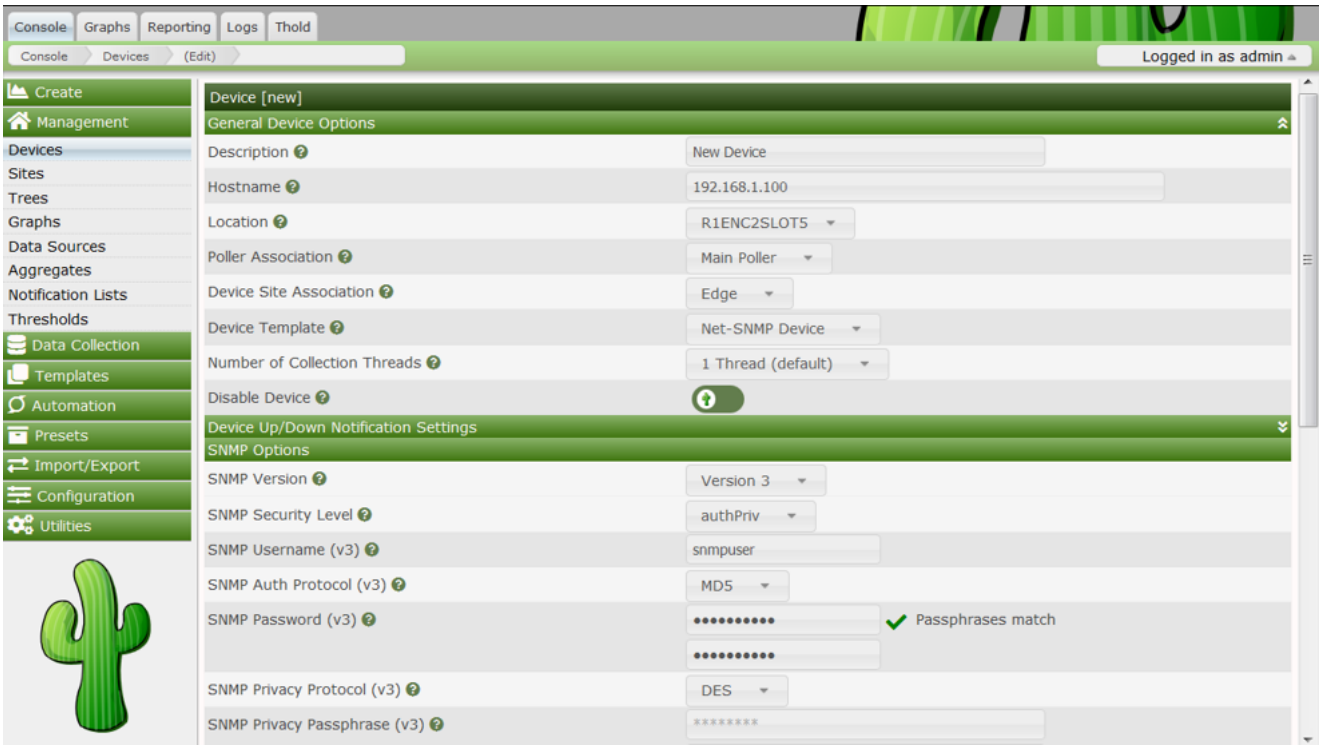
Copyright (c) 2004-2026 The Cacti Group

How to Graph Your Network

Create a device, select its measurements, and verify collection before adding its graphs to a tree. The manual workflow below is useful for a first device. Automation can perform these tasks for larger sets of devices once its discovery and graph rules are configured.

Creating a Device

Open **Console > Management > Devices** and click **Add**. Enter a description and address, choose the responsible data collector, and select a suitable Device Template. The template supplies associated graph templates and data queries; it does not restrict you to those defaults.



Device Field Definitions

- **Description** identifies the device in lists and default graph titles.
- **Hostname** is the device's address or resolvable DNS name.
- **Device Template** associates collection and graph definitions.
- **Poller Association** determines which collector reaches the device.
- **Site** groups devices by location.
- **Notes** records local information about the device.
- **Disable Device** stops collection while retaining its configuration.

Availability/Reachability Options

Choose a downed-device detection method appropriate to the device. Cacti can use SNMP, ping, both, either, or no availability test. A failed availability check can prevent measurements from being collected even when another protocol is reachable.

For ping checks, select the method, port where applicable, timeout, and retry count. ICMP permissions depend on the poller engine and operating system. Test connectivity from the collector assigned to the device.

SNMP Options

Configure the SNMP version and credentials accepted by the device:

- **SNMPv3** supports authentication and privacy. Prefer `authPriv` with mutually supported SHA and AES protocols where the device supports them.
- **SNMPv2c** supports 64-bit counters, but its community is not encrypted. Restrict access to trusted collectors and networks.
- **SNMPv1** is for legacy devices and does not support 64-bit counters.

Set the SNMP port, timeout, retries, and maximum OIDs per request. Start with values the device supports; increase request sizes only after verifying that responses are complete and collection remains reliable.

Security Options for SNMP V3

Choose the security level, username, authentication protocol and passphrase, privacy protocol and passphrase, and context or engine ID when required. The protocols offered depend on Cacti settings and the installed SNMP builds. The development defaults use SHA authentication and AES privacy. MD5 and DES are legacy algorithms and can be hidden by policy or unavailable in the build.

SNMP Retrieval Types

Cacti's PHP paths can use PHP's SNMP extension or external Net-SNMP tools. The extension supports SNMPv1, v2c, and v3; Cacti can select external tools for protocol combinations unavailable in the extension. Spine uses Net-SNMP through its C library. Verify the actual PHP, Net-SNMP, and Spine builds rather than assuming all offered algorithms are available on every platform.

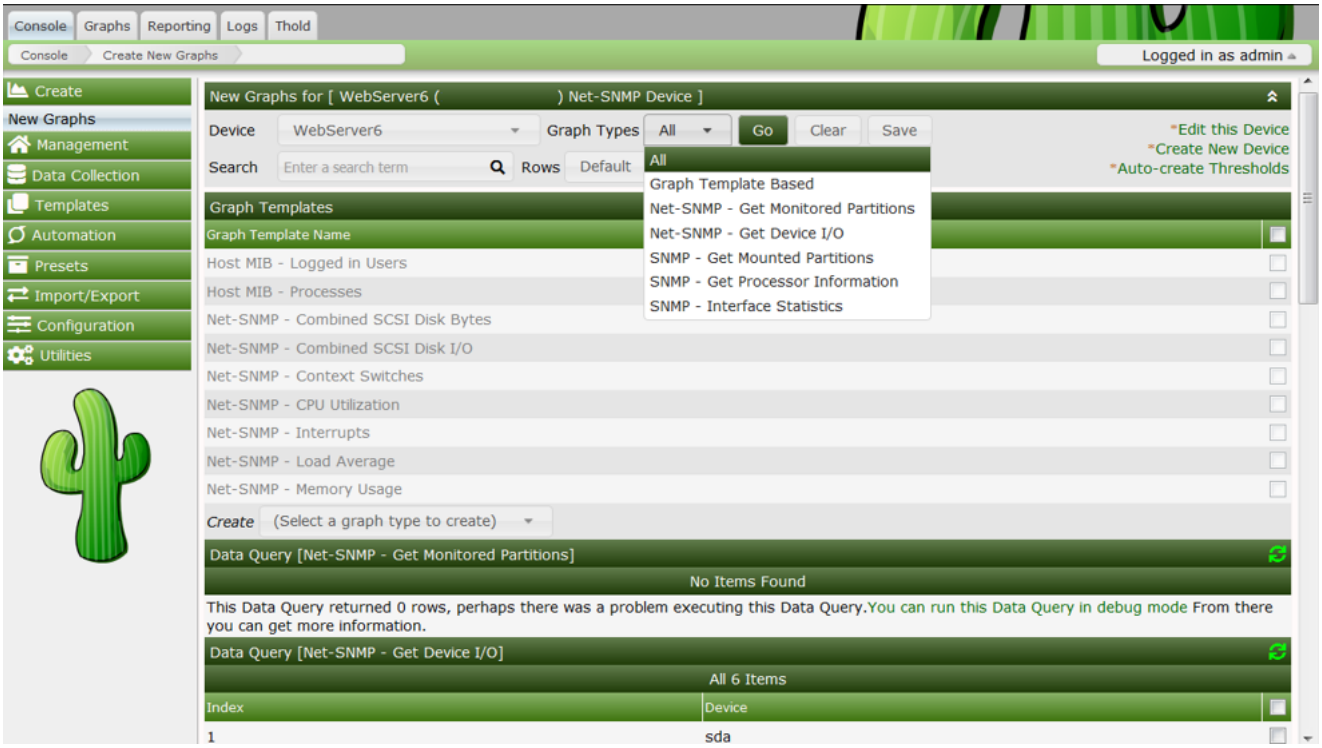
Verify device discovery

Save the device and check the SNMP information and any error shown on the edit page. Review **Associated Data Queries** and **Associated Graph Templates**. If a query has no rows, run its verbose query to check connectivity, OIDs, indexes, and credentials before creating graphs.

For interface traffic, prefer the 64-bit counter graph type when the device and selected SNMP version support it. See Data Queries for query-to-template mappings and Device Management for other availability and collection settings.

Creating the Graphs

Open **Console > Create > New Graphs** or use **Create Graphs for this Device** from the device page. Select the device, then choose the graph templates or query rows to graph. For indexed queries, choose the appropriate graph type before clicking **Create**.



Fill in any per-graph values requested by the templates and complete creation. Creating a graph defines collection; it does not immediately provide historical measurements. Allow scheduled polling to produce enough valid samples for the selected data-source type.

Verify and publish

1. Check the device's status and the Cacti log after polling.
2. Open the graph under **Console > Management > Graphs**. For blank graphs, inspect Data Source Debugging and the RRD last update.
3. Add the graph or device to a Tree, publish the tree, and grant the intended users access.
4. Confirm the result in Viewing Graphs with an account that has the intended permissions.

Copyright (c) 2004-2026 The Cacti Group

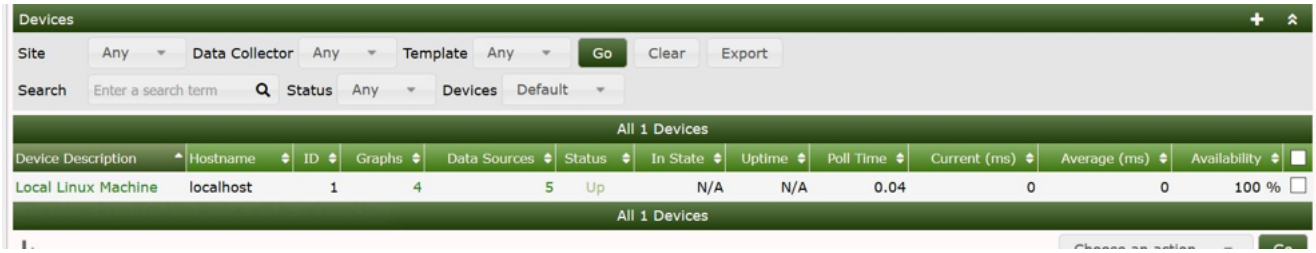
Device Management

This section will describe **Device** management in Cacti.

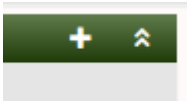
Adding a **Device** to Cacti can be done in a few different ways, either via the Web GUI, Cacti's Automation, or the Command Line Interface (CLI).

Web GUI Option

To add a device via the Web GUI first click on `Console > Management > Devices` and you will see the below device console window which will show existing devices if any



You will now select the + on the top right hand corner



Once you select the + otherwise known as the add device button you will see the below screen which will ask you for device specific information

Some of the most important information about the device will be required in this window which includes

- **Description** - The name that will appear on **Graphs** by default
- **IP/Hostname** - The DNS or IP address of the actual **Device**. IPv6 address insert into brackets (example: `[2001:db8::1]`)
- **Poller Association** - Defines which **Data Collector** is responsible for pulling information about the **Device**
- **Device Template** - Cisco, Net-SNMP, Linux, etc - The Cacti object that holds all the **Graph Templates** and **Data Queries** to be graphed
- **Site, Location** - Very important to performing Meta queries, or for Site level Graph organization on Cacti **Graph Trees**
- **Availability/Reachability** - Settings that describe **Device** timeouts and availability methods.
- **SNMP information** - SNMP Credentials for connecting to the **Device**
- **Device Notes** - Arbitrary unstructured information about the **Device**

Cacti requires this basic information to be able to monitor the device and once entered, click save on the bottom right corner. With the device created you will need to add graphs for the device by clicking **Create graphs for this device** on the top right hand corner.

Availability/Reachability Settings

Cacti prefers to use the Simple Network Management Protocol (SNMP) to communicate with **Devices**. Therefore, when creating a **Device**, you need to provide SNMP credentials to obtain information about the **Device** in order to collect data from it. Before Cacti will query the **Device** for data, it first verifies that the **Device** is up and responding. When doing so, you have several options. They include:

- **None** - Always assume the device is up. This is generally reserved for **Device** objects that do not have a state.
- **SNMP Uptime** - Query the SNMP Uptime Instance OID
- **Ping and SNMP Uptime** - Ping the device but also check the SNMP Uptime Instance OID
- **Ping** - Either ICMP, TCP at a port, or UDP as a port. Newer version has an additional method TCP Ping Closed. The device is considered UP even if the TCP ping returns closed
- **Ping or SNMP Uptime** - Only one needs to be working for Cacti to collect data
- **SNMP Desc** - Query the SNMP sysDescription in cases where the SNMP Uptime OID is not available
- **SNMP GetNext** - Query the first available OID in the OID tree for the **Device** Used for certain devices that have limited SNMP support.

SNMP Credentials

When providing the SNMP credentials, Cacti currently supports the following versions:

- **Version 1** - Rarely used any more. Reserved for very old hardware
- **Version 2c** - Supports 64-bit counters; community strings are not encrypted
- **Version 3** - Fully supported, including advanced settings such as SHA224+ or AES192+ with SNMPv3. Cacti automatically leverages the Net-SNMP binaries where needed, so there is no need to uninstall the php-snmp module.

When providing the SNMP Credentials, Cacti will warn you if you have provided incomplete information depending on the SNMP Version and SNMP Security Level you have specified.

Additional Important Options

There are some additional options to be aware of before starting to use Cacti. They include the following:

- **Device Threads** - If your device is far away, and can tolerate multiple threads querying information, you can increase this number to reduce the time it takes to collect all information.
- **Maximum OIDs Per Get Request** - Otherwise known as MaxOID's, this SNMP option will allow the SNMP client to gather more metrics per get request. Please keep in mind that the higher you make this number, the longer a SNMP respond may take. So, you have to be sensitive about the SNMP timeout as the number gets larger. Since, by default SNMP is generally collected over UDP, you will also be limited in the number of responses depending on how many routers or VPN's you traverse to reach a device. When traversing VPN connections, many VPN's limit the MTU to around 500 bytes, which will significantly limit how large the Max OID's can be. In some cases, it may be better to deploy a **Remote Data Collector** when your device is either far way from a latency perspective, or that you must traverse VPN's to communicate with.
- **External ID** - This field is normally used for Asset Tracking information for the **Device**, but it use is entirely up to the System Administrator.

Plugin Behavior

Many Cacti Plugins can and do add additional columns to the Device table in Cacti. Depending on the Plugin you have installed, you will find other information that you can provide about the device including things like:

- **Notification Settings** - Who to notify when the **Device** changes state
- **Criticality** - How important is the device
- **Failure and Recovery Counts** - How long till a device is treated as truly down.
- **Ping Thresholds** - What RTL is considered bad when reaching a device

Creating devices via CLI script

Run the CLI tool from the Cacti application root using the poller account and PHP runtime configured for that installation:

```
php cli/add_device.php --help
php cli/add_device.php --list-host-templates
```

Specify the device template, collector, availability method, and SNMP options that match the device. Omitted options can inherit the installation's defaults; there is no universal template or SNMP version implied by a minimal command.

For example, create an initially disabled device without SNMP, then complete its settings in the web interface:

```
php cli/add_device.php --description="Example device" --ip=192.0.2.15 \
  --template=0 --version=0 --avail=none --disable
```

Confirm the collector assignment, configure the required credentials and availability checks, create graphs, and enable the device when ready. Do not include real credentials in shared command output or shell history.

Copyright (c) 2004-2026 The Cacti Group

Graph Overview

Almost everything in Cacti is somehow related to a **Graph**. At any time, you can list all available **Graph** by clicking on `Console > Management > Graphs` menu pick. While it is possible to manually create graphs through this interface, new users should follow the instructions provided in the next chapter for creating **New Graphs** in Cacti.

For users who are familiar with RRDtool, you will immediately recognize that a **Graph** in Cacti is closely modeled after RRDtool's graphs. This makes sense since Cacti provides a user friendly interface to RRDtool without requiring users to understand how RRDtool works. With this in mind, every **Graph** in Cacti has certain settings and at least one **Graph Item** associated with it. While graph settings define the overall properties of a **Graph**, the **Graph Items** define the data that is to be represented on the **Graph**. So the **Graph Items** define which data to display and how it should displayed, and also define what should be displayed on the legend.

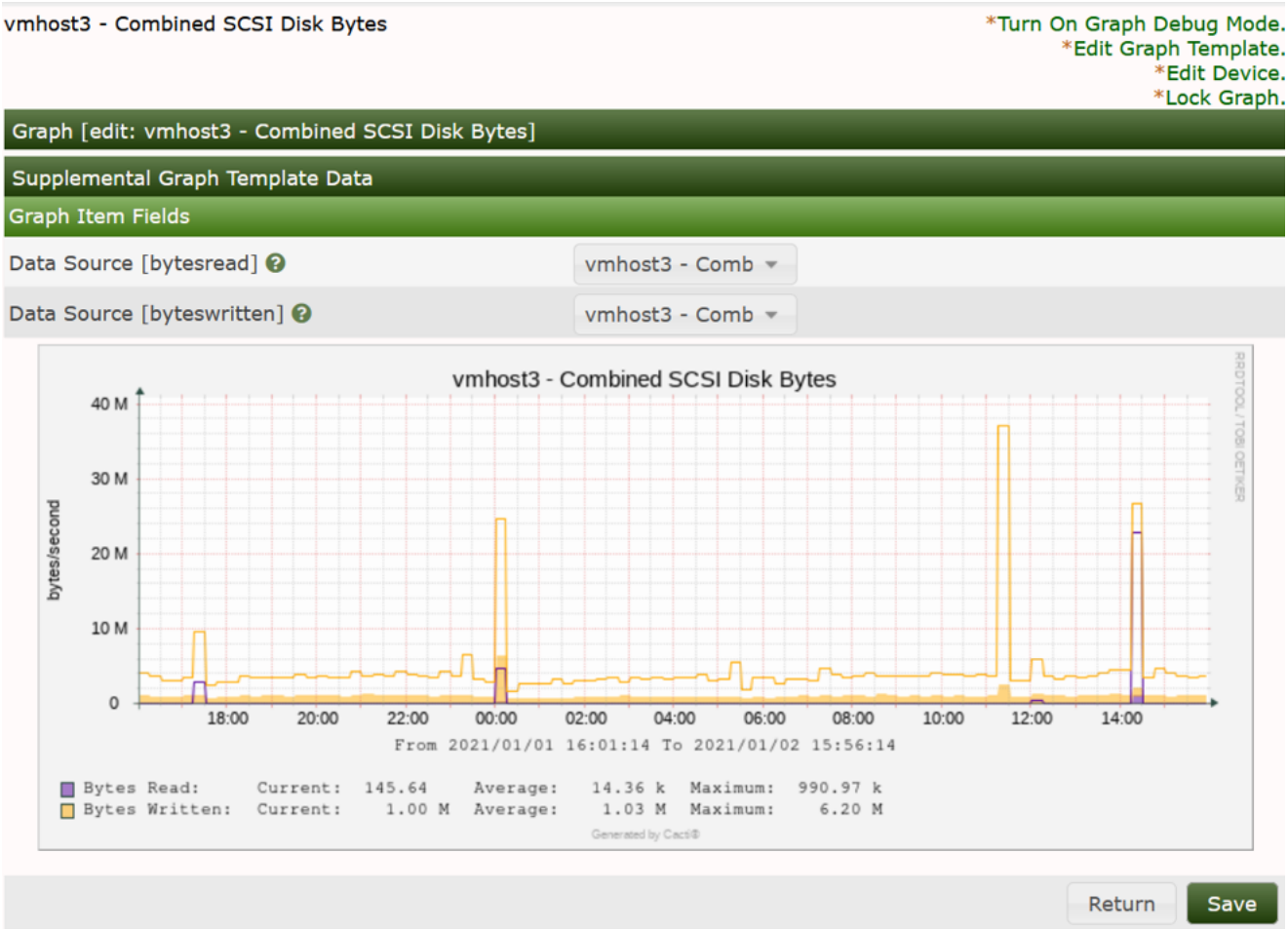
Each **Graph** and **Graph Item** has a set of parameters which control various aspects of the **Graph**. Fortunately through the use of **Graph Templates**, it is not necessary to understand the function of each field to create **Graphs** for your network. When you are ready to take on the task of creating your own **Graph Templates**, extensive field descriptions for both **Graphs** and **Graph Items** are provided in that section of the manual.

Below, you can see a simple version of the Graph Management interface found by going to `Console > Management > Graphs`

The screenshots below contain example devices and historical data from an earlier interface. Check menu labels against your installed version.

From this interface, you can see the Name, and ID of the **Graph**, the Source **Graph Template**, and it's size. At the bottom of the page, there is a list of Action that can be taken on **Graphs**. These actions are fairly extensive, and are also extended as you add **Plugins** to Cacti.

When you click on the Name of the **Graph**, you will be taken to a `Graph Edit` page as shown below.



In most cases, where the **Graph** is owned by a **Graph Template**, there is not much your can do on this page. However, in the case that a **Graph** is not managed by a template as in the image below, there are many more options you will have when working with the **Graph**. The downside of not leveraging a **Graph Template** for your **Graphs** is that you have to duplicate the work for each **Graph** created.

Multi Host Aggregate

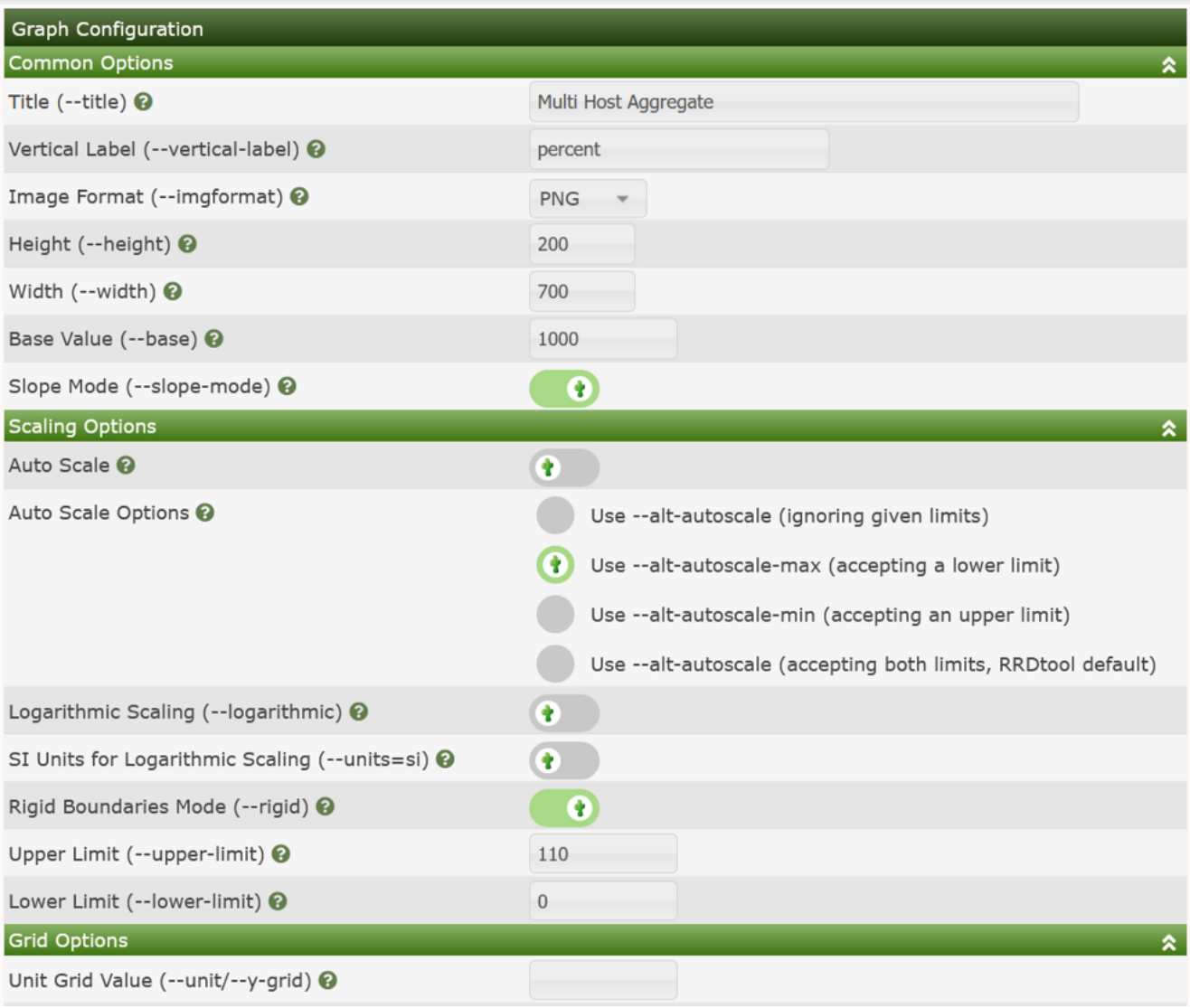
*Turn On Graph Debug Mode.
*Lock Graph.

Graph [edit: Multi Host Aggregate]

Graph Items [edit: Multi Host Aggregate]

Graph Item	Data Source	Graph Item Type	CF Type	Alpha %	Item Color
Item # 1	**vmhost0 - CPU Utilization - CPU0 (cpu): CPU#0**	**AREA**	**AVERAGE**	**50%**	**FFF200**
Item # 2	vmhost0 - CPU Utilization - CPU0 (cpu): Current:	GPRINT	LAST		
Item # 3	vmhost0 - CPU Utilization - CPU0 (cpu): Average:	GPRINT	AVERAGE		
Item # 4	vmhost0 - CPU Utilization - CPU0 (cpu): Maximum:<HR>	GPRINT	MAX		
Item # 5	**vmhost0 - CPU Utilization - CPU1 (cpu): CPU#1**	**AREA:STACK**	**AVERAGE**	**50%**	**FFD660**
Item # 6	vmhost0 - CPU Utilization - CPU1 (cpu): Current:	GPRINT	LAST		
Item # 7	vmhost0 - CPU Utilization - CPU1 (cpu): Average:	GPRINT	AVERAGE		
Item # 8	vmhost0 - CPU Utilization - CPU1 (cpu): Maximum:<HR>	GPRINT	MAX		
Item # 9	**vmhost0 - CPU Utilization - CPU2 (cpu): CPU#2**	**AREA:STACK**	**AVERAGE**	**50%**	**FFC73B**
Item # 10	vmhost0 - CPU Utilization - CPU2 (cpu): Current:	GPRINT	LAST		
Item # 11	vmhost0 - CPU Utilization - CPU2 (cpu): Average:	GPRINT	AVERAGE		
Item # 12	vmhost0 - CPU Utilization - CPU2 (cpu): Maximum:<HR>	GPRINT	MAX		
Item # 13	**vmhost0 - CPU Utilization - CPU3 (cpu): CPU#3**	**AREA:STACK**	**AVERAGE**	**50%**	**FFAB00**
Item # 14	vmhost0 - CPU Utilization - CPU3 (cpu): Current:	GPRINT	LAST		
Item # 15	vmhost0 - CPU Utilization - CPU3 (cpu): Average:	GPRINT	AVERAGE		
Item # 16	vmhost0 - CPU Utilization - CPU3 (cpu): Maximum:<HR>	GPRINT	MAX		
Item # 17	**vmhost0 - CPU Utilization - CPU4 (cpu): CPU#4**	**AREA:STACK**	**AVERAGE**	**50%**	**FF7D00**
Item # 18	vmhost0 - CPU Utilization - CPU4 (cpu): Current:	GPRINT	LAST		
Item # 19	vmhost0 - CPU Utilization - CPU4 (cpu): Average:	GPRINT	AVERAGE		
Item # 20	vmhost0 - CPU Utilization - CPU4 (cpu): Maximum:<HR>	GPRINT	MAX		

When viewing a Non-Templated **Graph**, you have complete control of the **Graphs** Canvas and all the **Graph Items** as you would in a **Graph Template**, and the Graph Configuration as shown below.



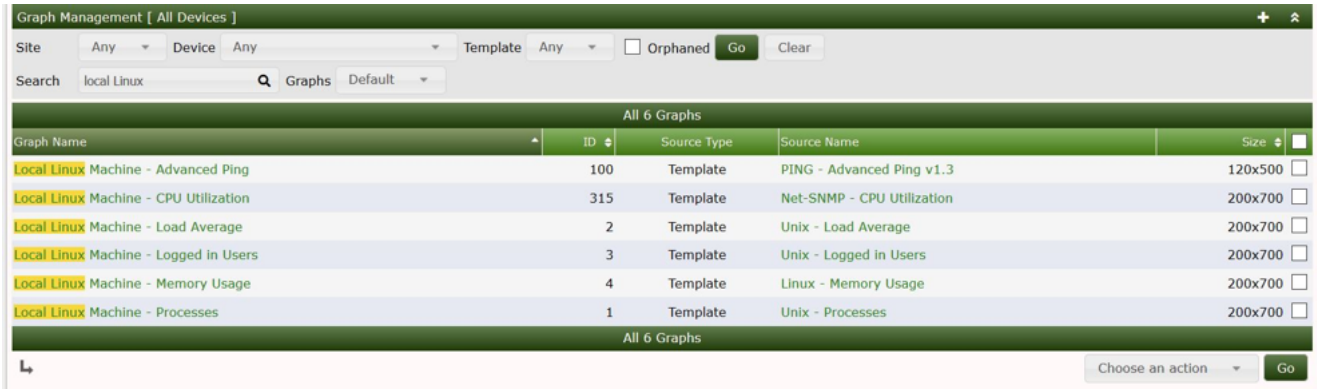
The settings in both the Canvas and Configuration components of the **Graph** will be covered in the **Graph Template** section of the manual and not covered here.

Copyright (c) 2004-2026 The Cacti Group

Graph Management

This section will describe **Graph** management in Cacti.

Cacti features a way to view the graphs per device via the console. This allows the administrator to view the graphs that are attached to a specific device. You can also search by type of graph. Below we search for graphs that are associated with the local linux server

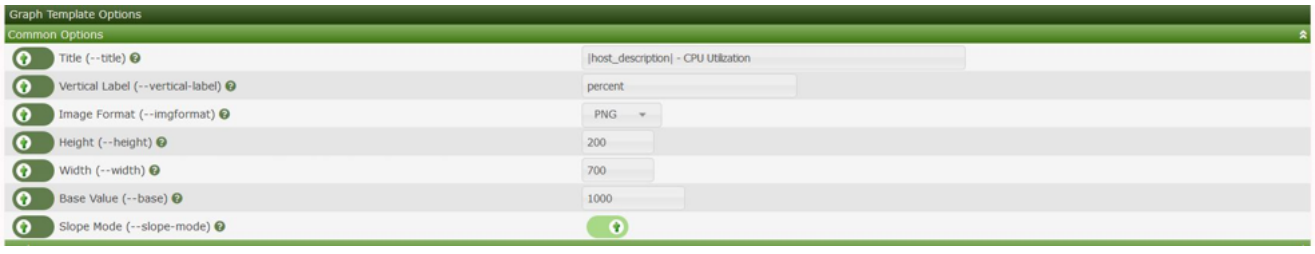


Clicking on one of the graphs in the list shows the below menu. From this menu you can enable debugging on the specific graph you can also change the template of the graph amongst other things

Modifying the graph template

An individual graph inherits managed settings from its graph template. Editing that graph does not update the template or other graphs. Fields marked **Use Per-Graph Value** in the template can be set independently on each graph.

To change shared settings, edit the Graph Template under **Console > Templates > Graphs**. Review the affected graphs first: template changes propagate to attached graphs for fields managed by that template.



Copyright (c) 2004-2026 The Cacti Group

Viewing Graphs

Background

Cacti, when it was first invented in 2001 by Ian Berry, his vision was to make it the fastest and easiest way to view and render Network Monitoring **Graphs** for people in the Network, Site, and Data Center Operations Space. As such, it's focus from the very beginning was on render just one thing, **Graphs**. So, as such, the second **Top Tab** on a Cacti system is the **Graphs** tab.

The Graphs Top Tab

The **Graphs Top Tab** has a few personalities. They include:

- **Tree View**

  Allows Cacti Users to view **Graphs** in the form of hierarchical **Trees**. These **Trees** are generally constructed by the Cacti Administrator and are controlled either at the **User** or **User Group** level.

- **Preview View**

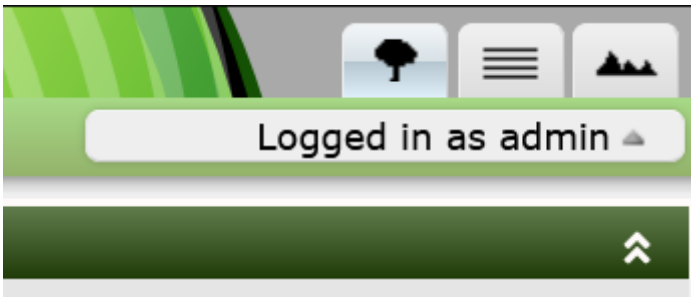
  The **Preview View** provides a view of all **Graphs** that a Cacti User has access to. **Table Filters** are provided to constrain the list of **Graphs** returned to the page.

- **List View**

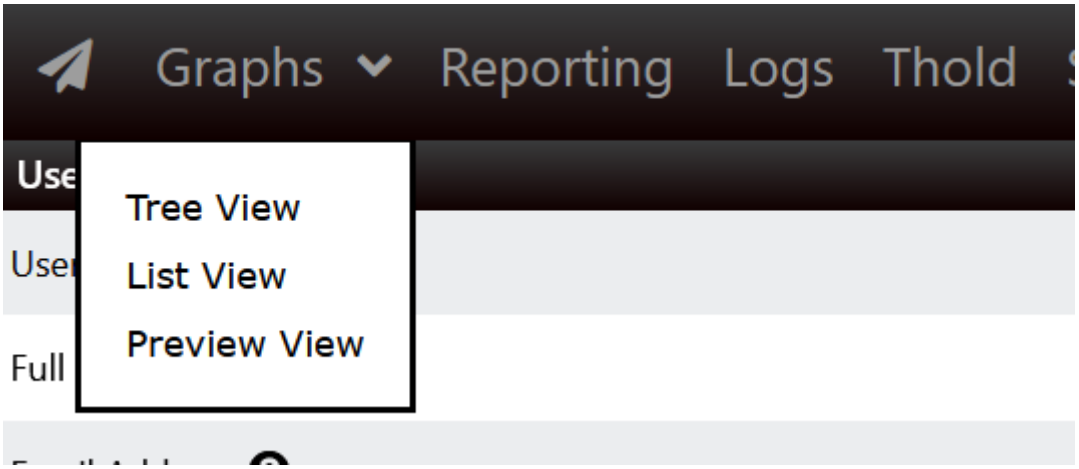
  The **List View** allows the Cacti user to Create their own **Preview Page** by allowing them to select graphs from various pages, and then finally view those pages from the **Preview View**.

The way these various personalities appear in Cacti is somewhat different depending on your Theme. Review the two images below, for how you navigate to the various **Graph View** modes.

In this first image, we see the way the **Graph View** options are displayed to the end user. This is the layout of the Classic, Modern and Dark themes.

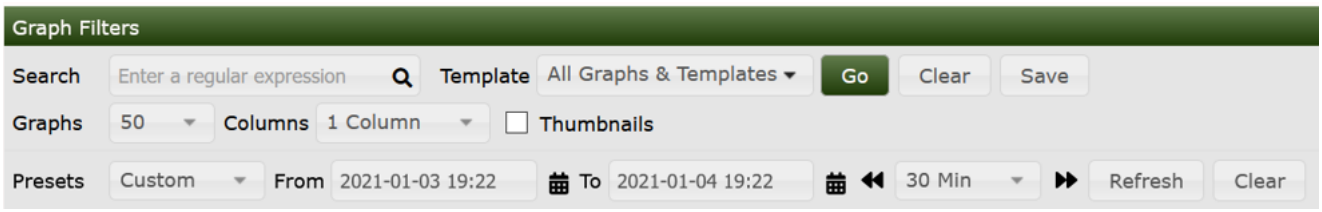


In this second image, you can see the way the **Graph View** page will appear for users of the Paw, Paper-Plane, and Sunrise themes.



Graph Manipulation and Options

Once you can view the **Graphs**, there will be a Filter Panel that will provide you multiple options to limit your view of them. Below, you can see an image of that Filter Panel.



From this sub-panel, you can do the following:

- Search by regular expression
- Select one to many, or all **Graph Templates** to view
- Specify the number of Graphs per Page
- Specify the number of columns to Display
- View either Legendless Thumbnails or full Graphs with their legends
- Set various Preset Time-spans, or select a Time Range
- Shift the Time either forward or backward by the Shift Time-span

Once you have the Filter where you like it from a Columns, Thumbnails, and Graphs per Page setting, you can press the `Save` button, if you have the correct permissions and save those defaults for your next login.

Depending again on permissions, to the right of the **Graphs**, you will find a number of action icons that allow you to operate on the **Graphs**

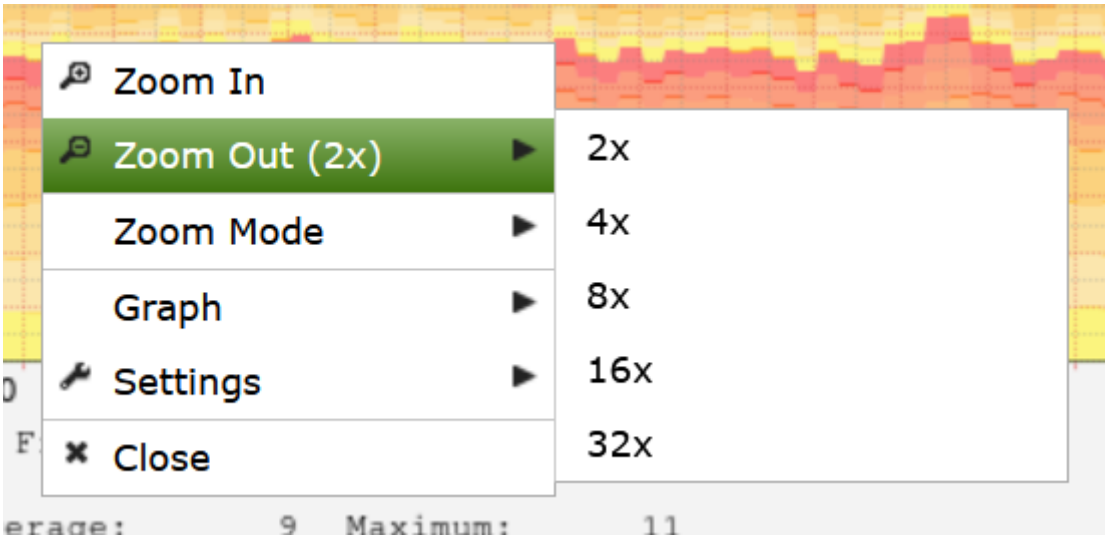
The image below shows what that might look like if you have the **Thold** and **QuickTree Plugins** installed.

From top to bottom, the Graph Action Icons do the following.

Name	Description
Graph Details	Allow you to perform precision zooming, view RRDtool debug information and perform CSV exports of your Graph Data.
CSV Export	Allows you to directly CSV Export your Graph Data
Realtime View	Allows you to view your Graphs at down to a 1 second granularity either in place, or in a pop-up Window
Spike Kill	Allow you to clear up Spikes and Gaps in your Graph
Create Threshold	Create a Threshold for the Graph
Add to QuickTree	Allow you to select Graphs to add to Trees by simply tooling around the Graphs page.

Graph Zooming

Cacti also has a powerful Graph Zoom interface built in. You can discover what it allows you to do by simply right clicking in the Graph area. When zooming, you will zoom all the **Graphs** on a page. It's quite powerful.



There are many options for the Zoom menu. Instead of explaining them here try them out for yourself.

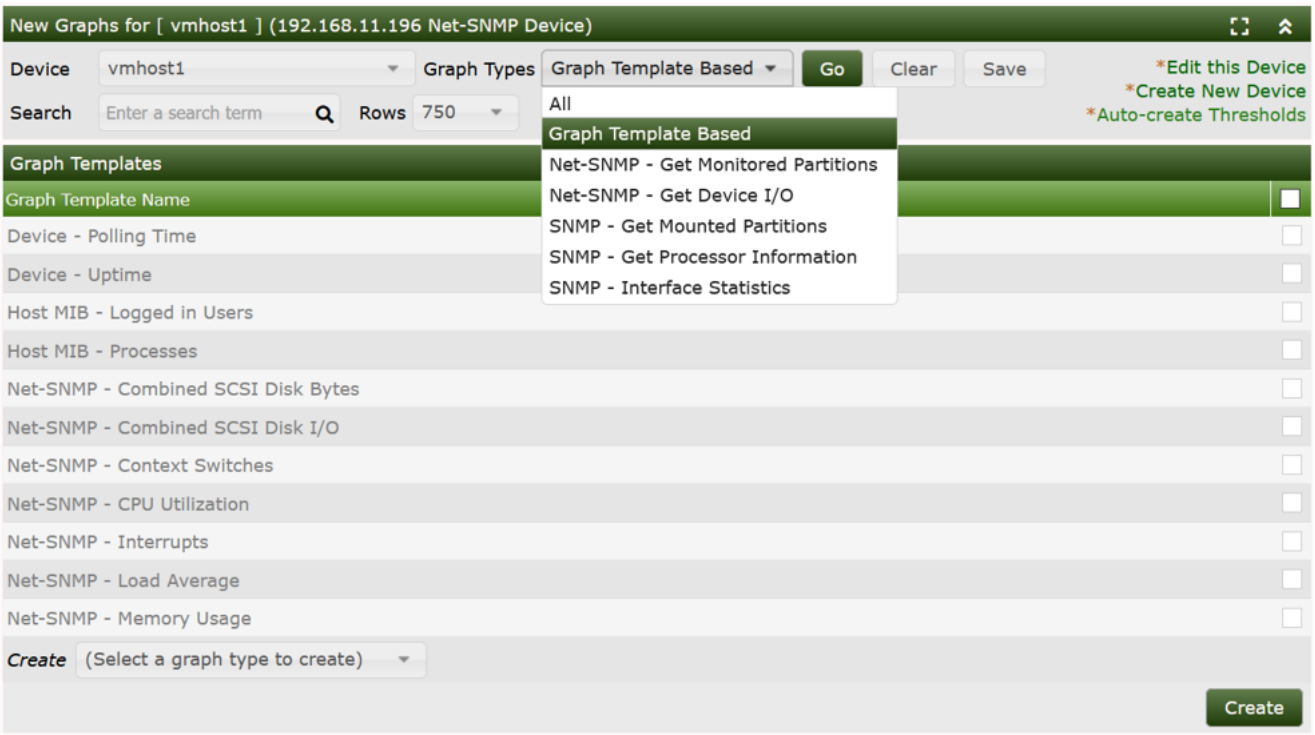
Copyright (c) 2004-2026 The Cacti Group

Creating a Graph for a Single SNMP OID

When dealing with SNMP-enabled **Devices**, there are often times when you want to graph the value of a single OID. This tutorial explains how to do this in Cacti. It also assumes that you have the "SNMP - Generic OID Template" **Graph Template** which is available in the **Generic SNMP Device Package**. If you do not see this **Graph Template** listed under **Graph Templates**, install the **Generic SNMP Device Package** using the `import_package.php` command line script located in Cacti's cli directory.

To start the process of creating a new graph for your OID, click go to `Console > Create > New Graphs`, and from there find the **Device** you would like to add the **Graph** to.

From the image below, you can see the **New Graphs** interface.



In this image, only the **Graph Template** based **Graph Templates** are displayed. Had the `Graph Types` been switched to any of the **Data Query** based `Graph Types` the interface would look a bit different.

From that Interface, to create a `Generic SNMP OID` based Graph, you need to first select it from the `Select a graph type to create` drop-down. Once you have selected it, you may press the `Create` button, at which point in time, you will see the image below.

From this interface, you must define various attributes for your resulting **Graph** as shown in the table below.

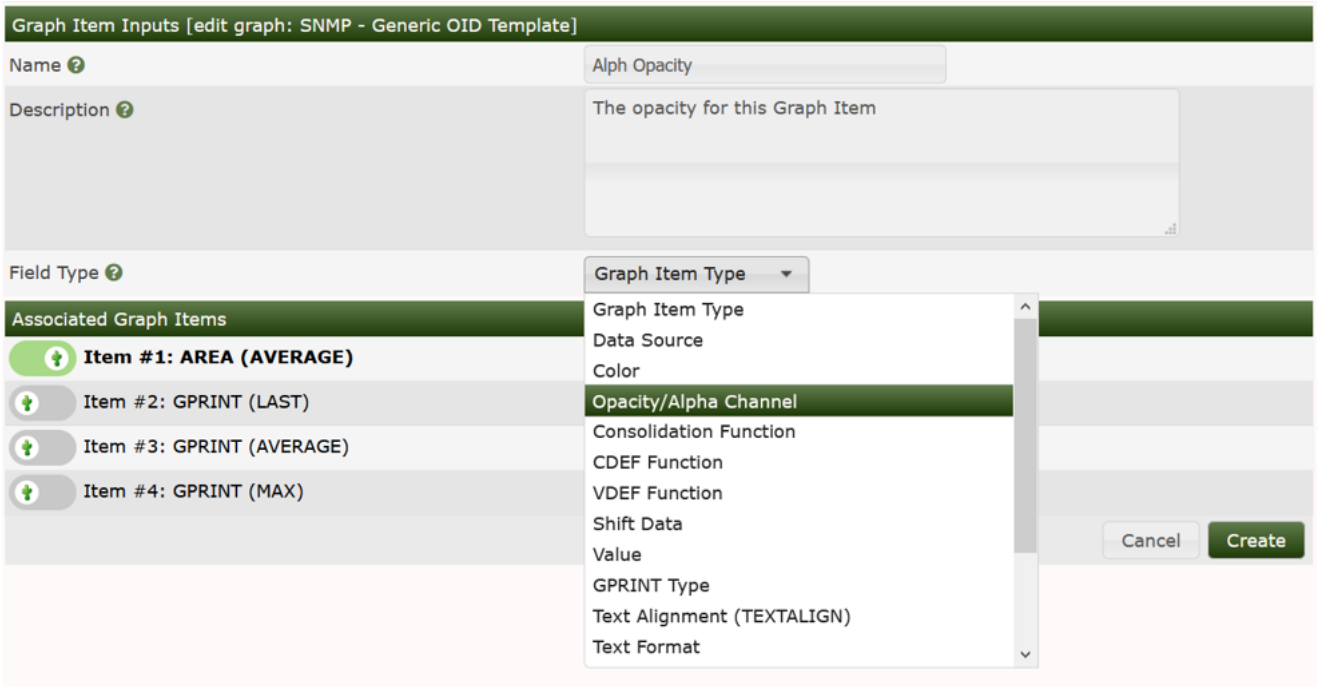
Table 10-1. Field Description: SNMP - Generic OID Template

Name	Description
(Graph) Title	The title that is to be used for the new graph. It is generally a good idea to keep `host_description` in the title, as to make the graph easier to identify later.
(Graph) Vertical Label	The text that will be printed along the y-axis of the graph. It is generally used to describe units, such as 'bytes' or 'percent'.
(Graph Items) Legend Color	The color that will be used to represent the data on the graph.
(Graph Items) Opacity/Alpha Channel	This optionally provides an opacity (not available with RRDtool-1.0.x) for a colored item.
(Graph Items) Legend Text	The text that will be used to describe the data on the graph legend.
(Data Source) Name	The title that is to be used for the new data source. It is generally a good idea to keep `host_description` in the title, as to make the data source easier to identify later.
(Data Source) Maximum Value [snmp_oid]	The maximum value that will be accepted from the OID. Make sure you choose a value that is reasonable for the data you are trying to graph because anything larger than the maximum will be ignored. If you are graphing a percentage, you should use '100' as the value should never exceed this.
(Data Source) Data Source Type [snmp_oid]	How the data from the OID should be stored by RRDtool and interpreted on the graph. If the value of the OID represents the actual data, you should use GAUGE for this field. If the OID value is a constantly incremented number, you should use COUNTER for this field. The two remaining field values, DERIVE and ABSOLUTE can be ignored in most situations.
(Custom Data) OID	The actual SNMP OID to graph. It is typically a good idea to enter the number OID here as opposed to using MIB names. For instance, to get the number of open files on a Netware server, you would use ".1.3.6.1.4.1.23.2.28.2.7.0" as the OID.

So, based upon your specific OID that you are graphing, You will need to fill out these fields before the new graph can be created properly.

When finished filling in values for these fields, click the Create button. Your new **Graph** can now be accessed through the `Console > Management > Graphs` menu pick.

> **NOTE**: When comparing the Table above, to the image, you will see that the `Opacity/Alpha Channel` pick is missing from the interface. To add it, you simply need > to edit the `Generic SNMP OID` > **Graph Template**, add a new **Graph Item Input** override to the Alpha/Opacity on the Graph Template as shown below. That topic will be covered in the **Graph Template** section of the manual though.



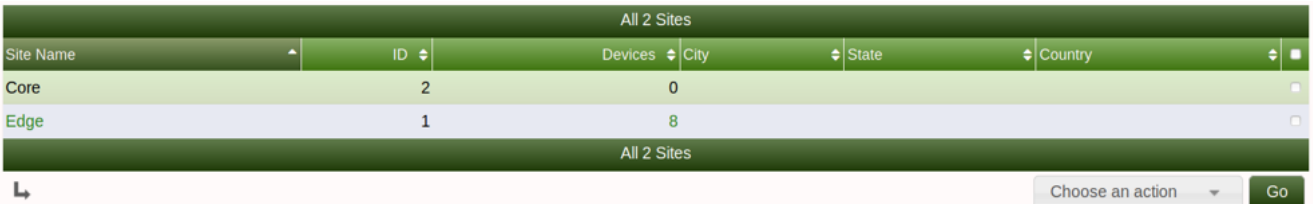
> **Important Note**:
>
> When graphing single OID's with Cacti, to ensure Spine and cmd.php will process single OID's correctly, they should be written using numeric format separated by dots, such as `1.3.6.1.4.1.9.9.97.`
>
> At this time, ASN's (named OID's) such as `enterprise.9.9.97` (Cisco Switch) or `enterprises.cisco.ciscoMgnt.ciscoSwitchEngineMIB` are not supported.

Copyright (c) 2004-2026 The Cacti Group

Site Management

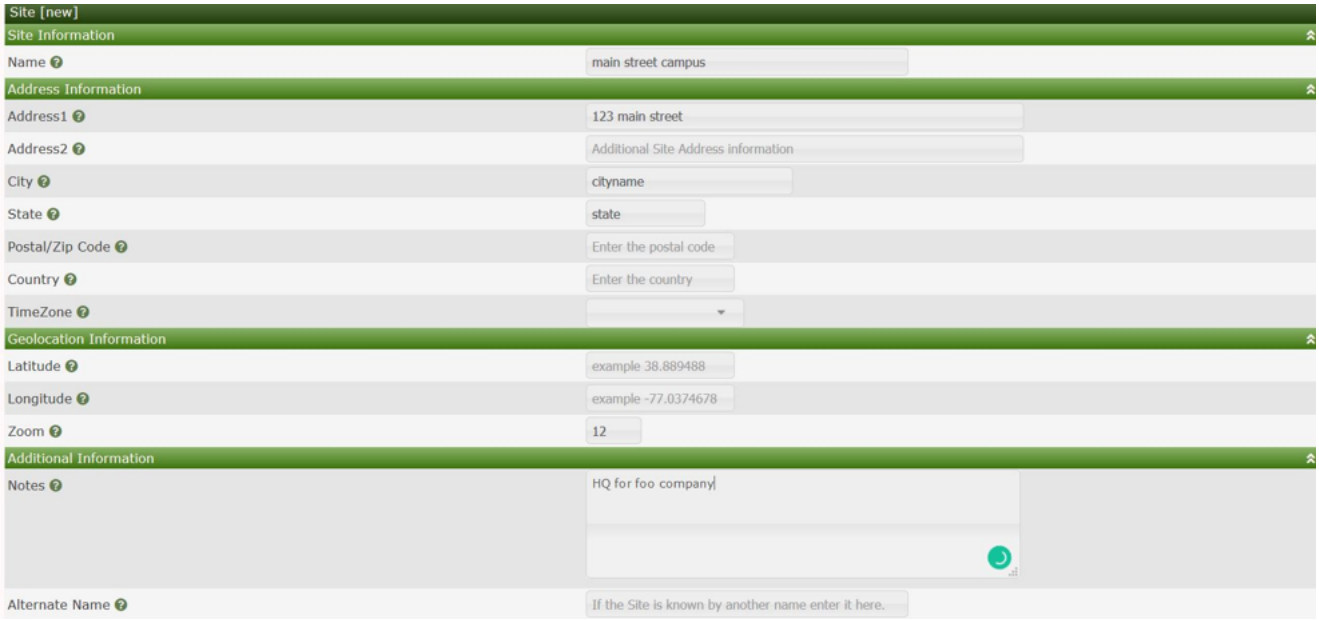
This section will describe **Site** management in Cacti.

Sites in Cacti can be used to separate different parts of your company with the respective location devices. For example, you can have a site called **123 main street** in which you can associate all of the devices that are in the physical location to the Cacti site. This could also be a customer site or data center location



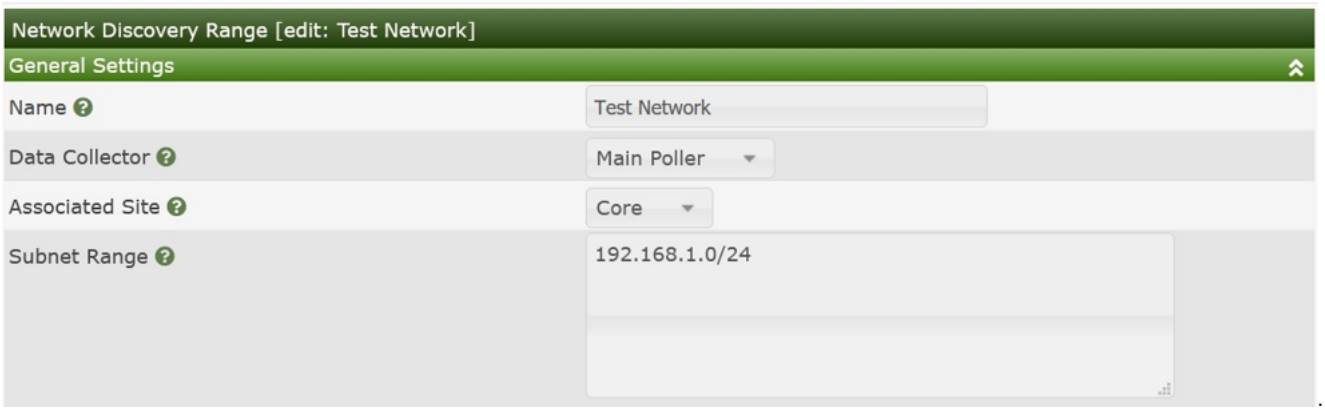
Below is an example of some of the attribute data you can enter for the site/location

Enter the appropriate information for the site and click **Create** in the lower right corner:



Once a site has been created, you can associate devices with it when adding or editing a device:

You can also associate the discovered devices via automation to a particular site.



Copyright (c) 2004-2026 The Cacti Group

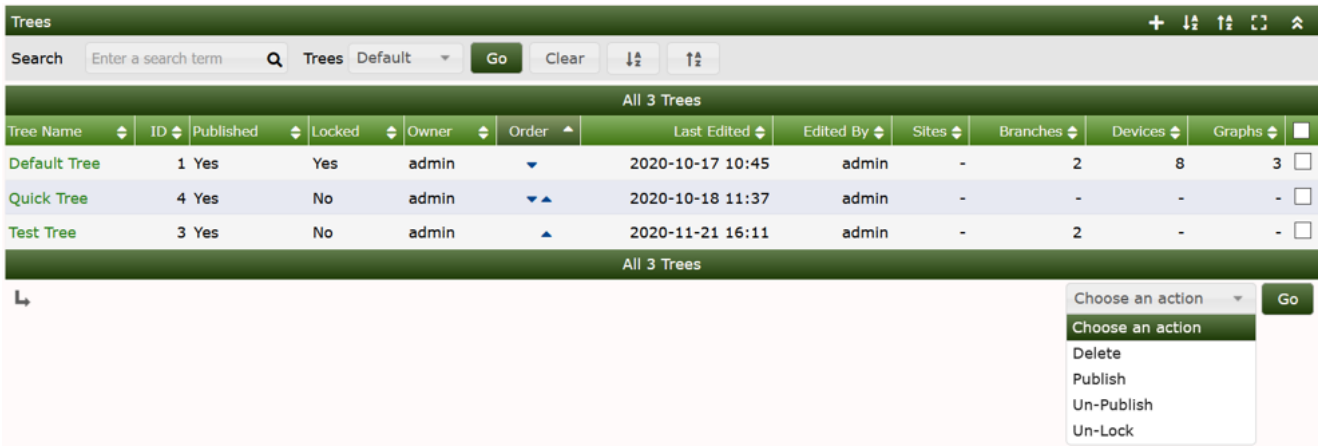
Cacti Trees

Trees

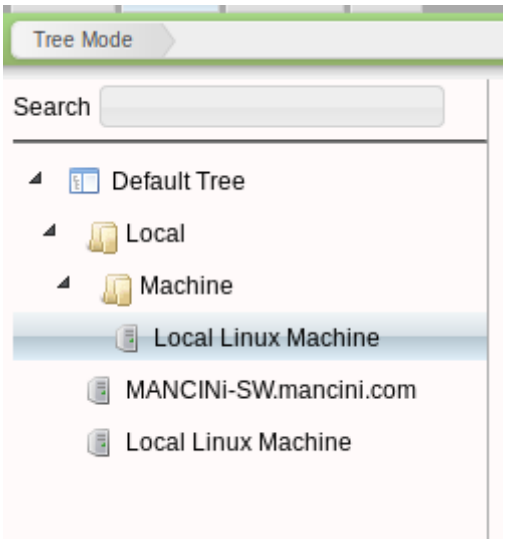
A **Tree** can be thought of as a hierarchical way of organizing your graphs. Each **Tree** consists of zero or more branches that contain leaf nodes such as **Graphs**, **Devices** and **Sites**. It's a very powerful way of organizing your **Graphs**.

Below we can see the current **Trees** we have setup on our Cacti server. To get to this screen click `Console > Management > Trees`.

From this page you can add or remove **Trees** as required.

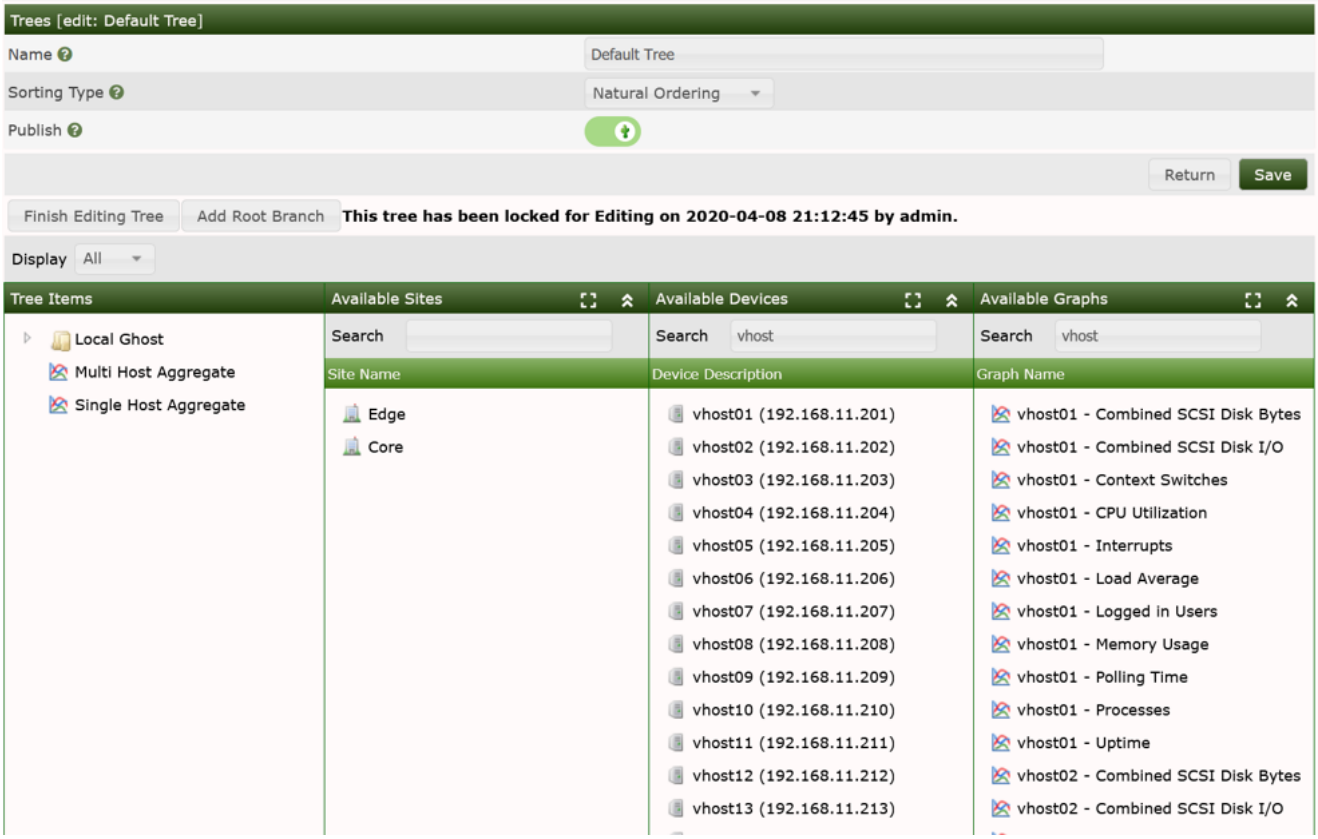


Below is how a **Tree** is displayed in **Graph View**. We can see the **Device** that is being monitored - clicking on this **Device** will result in seeing all of the **Graph** data generated for the **Device**.



Creating a Tree

To create a new tree simply click the Add button (+) on the top right hand corner and enter a name for your **Tree**. After the tree has been created you will see the below page where you are able to add **Devices** to the **Tree**.



To add devices to the new tree simply drag an available device to the tree and it will be added to the tree. Cacti currently supports four `Sort Types`, which can be either inherited, of left to the author to define inheritance and at what level. See the image below for a visual representation of how Tree Sorting is accomplished.

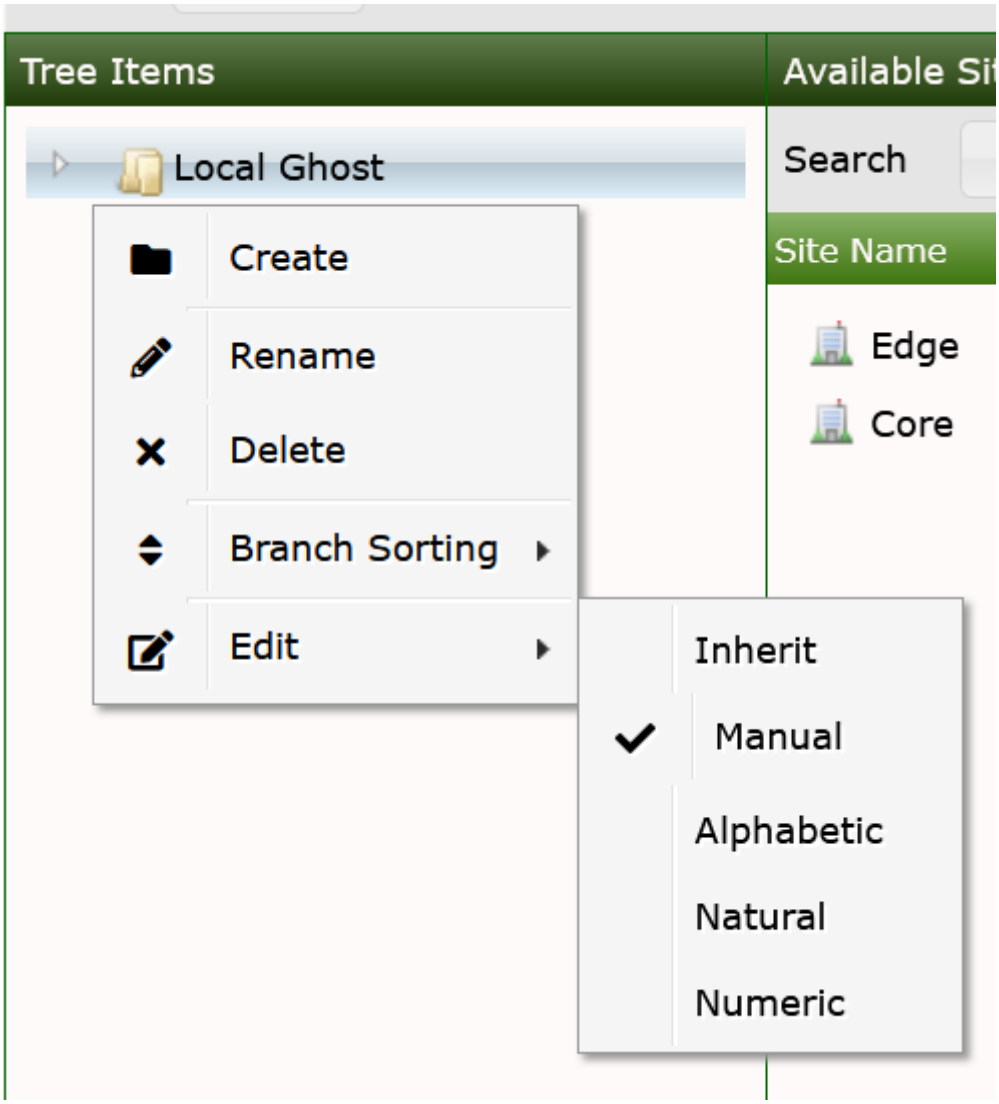


Table 8-1. Tree Sorting Type Definitions

Field	Value	Description
Name	Name of the tree entry.	The sort order of all trees themselves is always alphabetical
Sorting Type	Manual Ordering (No Sorting)	You may change the sequence at your will
Alphabetical Ordering	1, Ab, ab	All sub-trees are ordered alphabetically, unless specified otherwise (you may change sort options at sub-tree label)
Natural Ordering	ab1, ab2, ab7, ab10, ab20	N/A
Numeric Ordering	01, 02, 4, 04	Leading zeroes are not taken into account when ordering numerically

End users will not be able to view the **Tree** or it's Graphs until you publish it. To edit a **Tree**, you will be required to lock it for your use. The locking is designed to prevent multiple users from editing a **Tree** simultaneously.

When a Tree is locked, you can drag & drop the **Sites**, **Devices** and **Graphs** over to the Tree menu. To add a 'Root Branch', simply press the button to do so, once you have Root Branches, you may right click on the to create sub-branches on the Tree.

When you single click on a **Site**, the **Devices**, and **Graphs** associated with that **Site** should appear in their respective sections. You can also type into the Search fields above the various sections to drill into them. You can also shift-click and control-click on the objects within a section to drag & drop multiple objects at one time.

Don't forget to unlock your **Tree** before finishing your editing session.

Copyright (c) 2004-2026 The Cacti Group

Data Source Management

A Cacti data source connects a collection definition to an RRD file. Each file can contain more than one named RRDtool series, such as `traffic_in` and `traffic_out`. Graphs refer to those series; they do not each require a separate collection definition.

Counts and reuse

A switch with traffic graphs for 24 ports can have 24 Cacti data sources and 24 RRD files, each with inbound and outbound series. Additional graphs can reuse those sources. Reuse depends on the template and creation settings; creating another graph does not guarantee reuse in every case.

Poller statistics count collection items and processed RRD files separately. Do not treat `DataSources` in the poller log as a universal count of RRD files, graphs, or the rows shown on the Data Sources management page.

Check device-level counts under **Console > Management > Devices**:

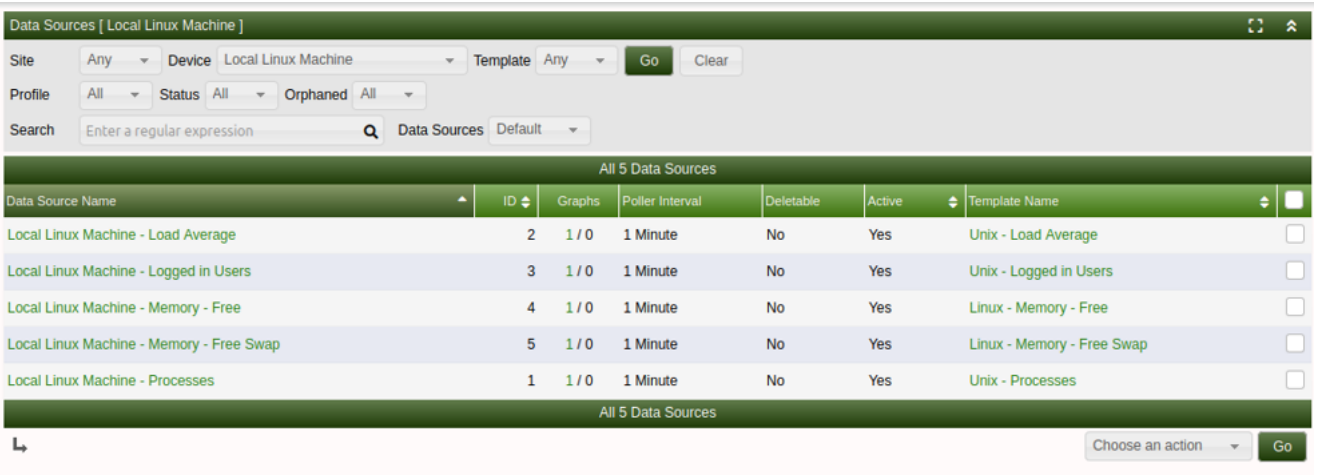
Storage and sampling

Storage depends on the number of series, sampling interval, archive rows, consolidation functions, and retention defined by the Data Source Profile. Use the profile's estimate and inspect representative files with `rrdtool info`; there is no single file size for all one-minute or five-minute data sources.

Changing a profile or template does not automatically convert an existing RRD file's layout. Plan and back up any migration of existing data separately from changes to future source definitions.

View data sources by device

Open **Console > Management > Data Sources** and filter by device or template. Inspect the data source's collection method, path, active state, and associated graphs before changing it.



The **Orphaned** filter identifies data sources without graph references. Review whether a plugin or external process still needs a source before removing it. RRD cleanup is controlled separately by the installation's Paths settings.

For missing values, use Data Source Debugging and the Cacti log. Check device reachability, poller ownership, file permissions, and the last update time before changing a template.

Copyright (c) 2004-2026 The Cacti Group

Aggregate Graphs

An **Aggregate Graph** combines series from existing graphs into one graph. Member graphs must have a compatible graph template so that corresponding items can be combined consistently. An aggregate uses their data sources; creating it does not start a separate collection cycle for those measurements.

Create an aggregate from a template

1. Create an Aggregate Template for the common graph template and choose its layout, totals and per-item color templates.
2. Under **Console > Management > Graphs**, select the compatible member graphs.
3. Choose **Create Aggregate from Template** in the Actions menu, select the aggregate template, and review the name and included members before saving.
4. Open the resulting graph and check units, stacking, member order and totals. It can be placed on a tree and viewed like other Cacti graphs.

Maintain members and propagation

Manage the aggregate through **Console > Management > Aggregates**. Review its member graphs when adding or removing measurements; membership determines the series contributing to the display and totals.

Propagation Enabled keeps the aggregate attached to its template's managed settings. Template saves update attached aggregates with propagation enabled. Disabling propagation allows independent settings and detaches template management; subsequent template edits no longer update that aggregate. Review this setting when an aggregate does not follow a template change.

Color Templates rotate colors across member graphs for the items to which they are assigned. A legend entry is not an additional member and does not advance that rotation.

Before a broad synchronization, back up the relevant definitions and verify one aggregate. Keep source graphs and data-source ownership distinct from aggregate membership when planning deletion or cleanup.

Copyright (c) 2004-2026 The Cacti Group

Automation Networks

This section will describe **Automation Networks** in Cacti.

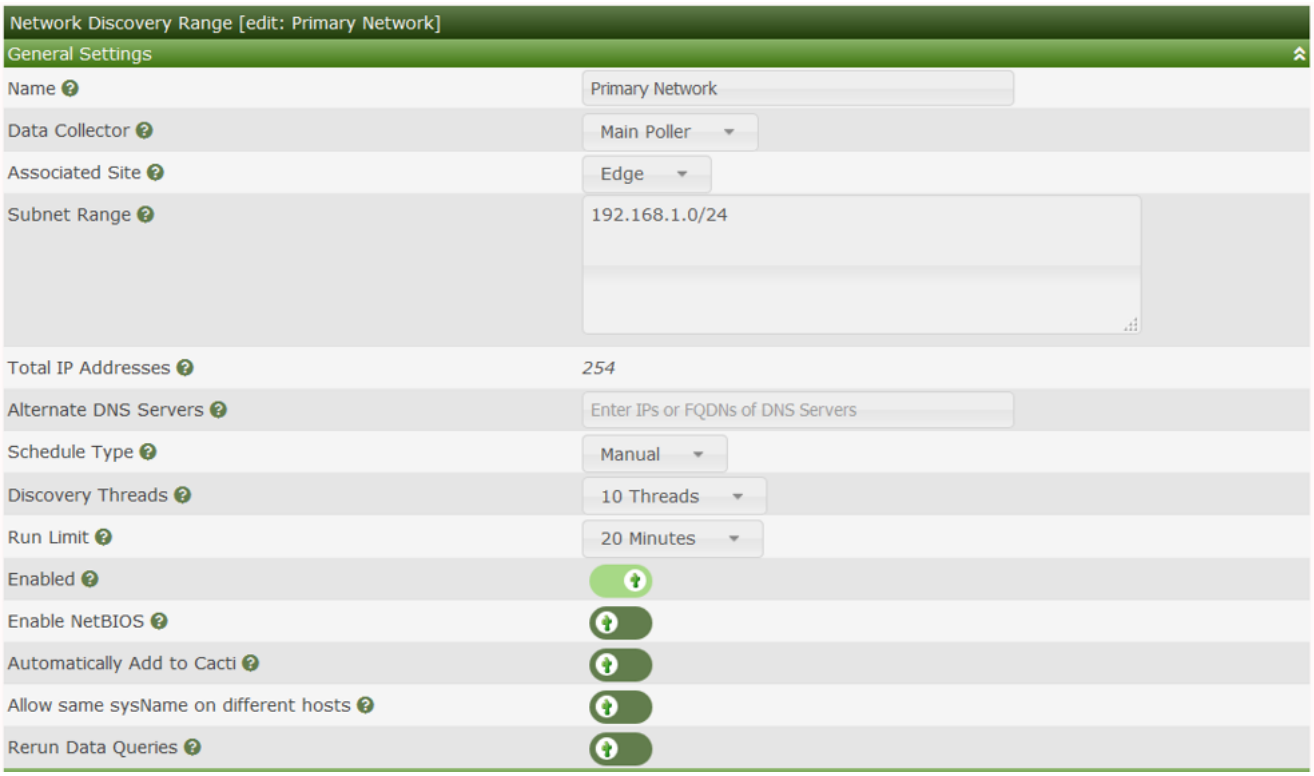
Adding a network to scan with built-in automation is easy. On the main console click Automation. Once on the below page click the + on the top right of the page.

You will now see the below page. If you want to scan 192.168.1.0/24, you would enter that in the subnet range textbox then enter the subnet in CIDR format.

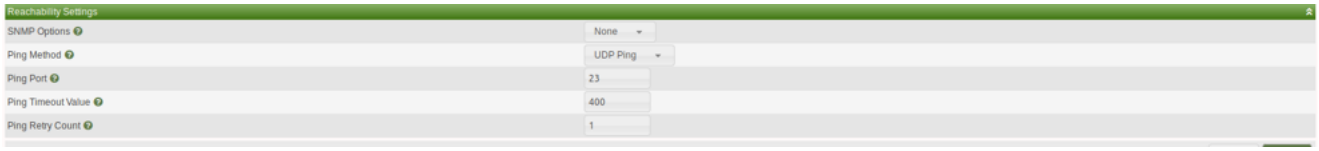
Other important options are

Option	Description
Schedule type	How often you want to scan this subnet for devices
Discovery threads	How many processes to spawn during the scan
Max Runtime	to prevent the scan from running indefinitely
Automatically add to Cacti	If a device is SNMP reachable and matches a rule from this subnet the device will be added
Netbios	Attempt to resolve the hostname by netbios

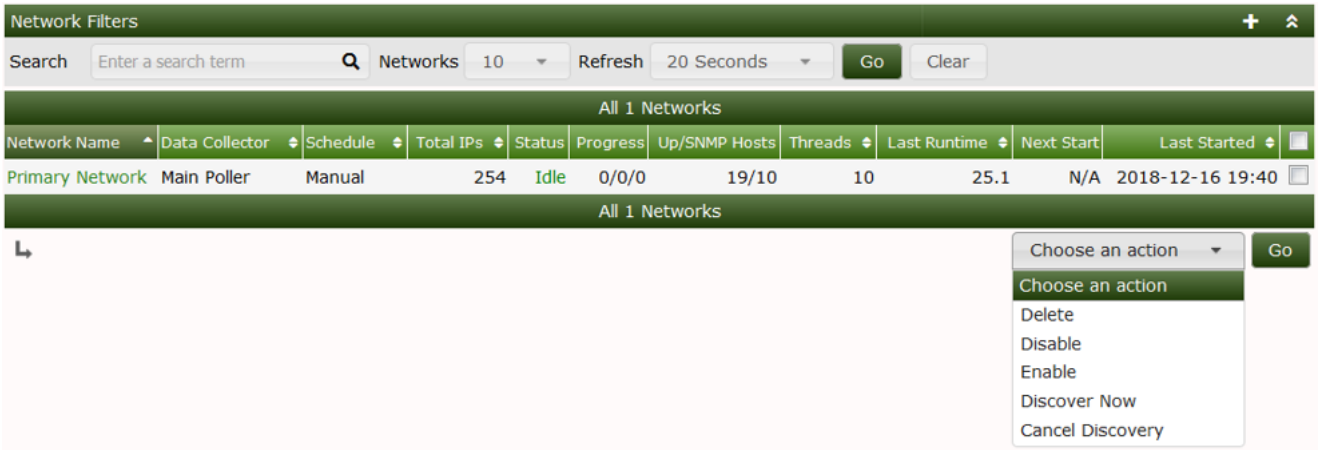
After you are done adding your network details, ensure you enable the rule and save.



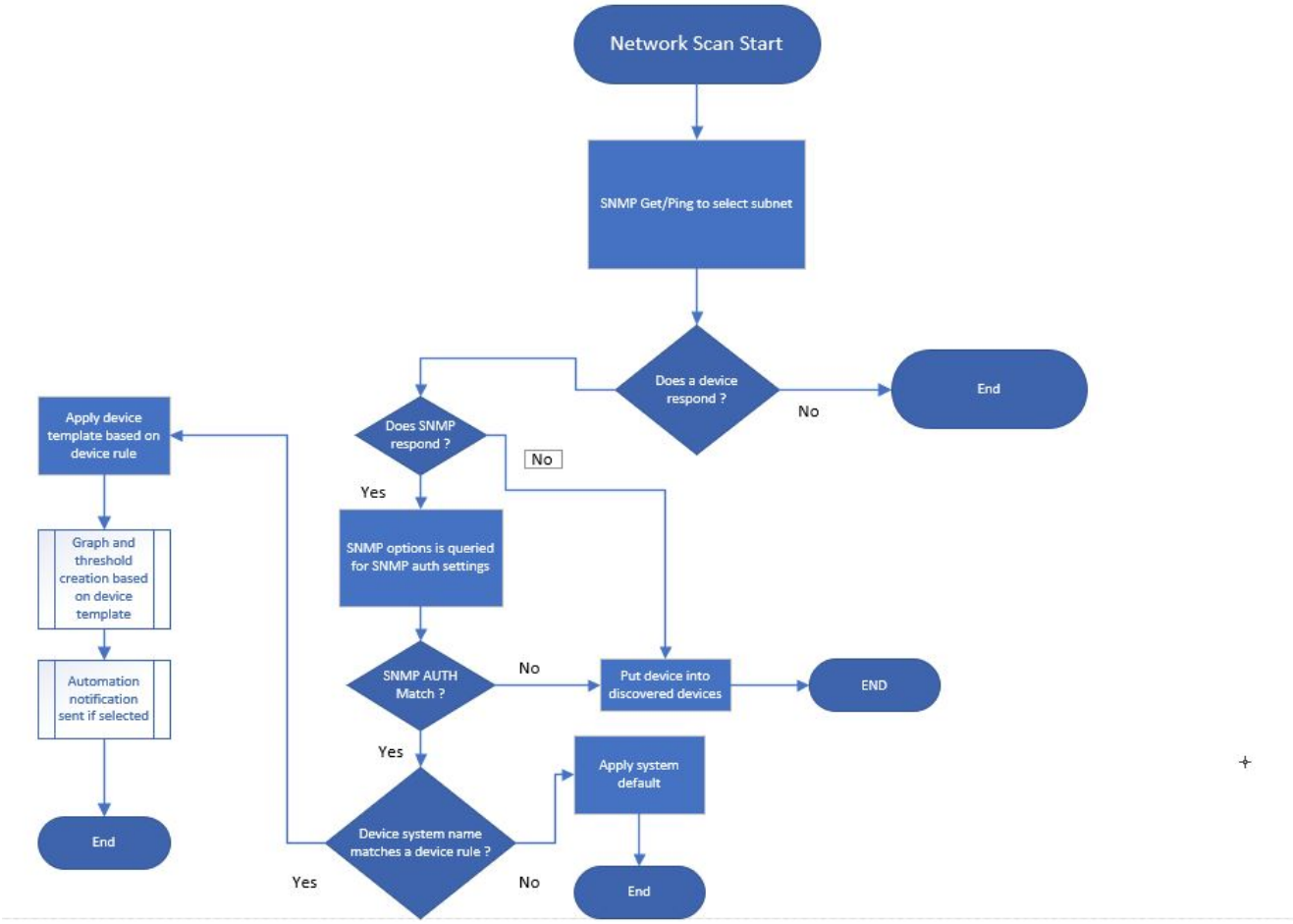
Ensure that if you have a SNMP rule for this part of the network to select the SNMP option set this will tie the network to the SNMP rules



To begin the scanner click on the bottom right drop down and select "Discover Now" this will start the discovery



Here is the flow of the automation scan



Copyright (c) 2004-2026 The Cacti Group

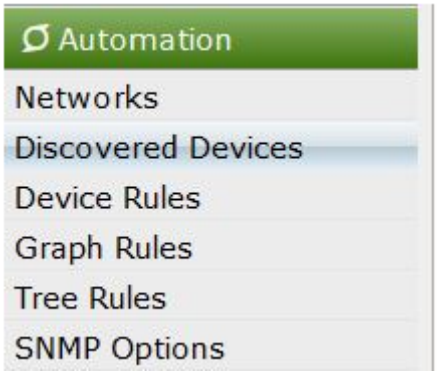
Automation Discovered Devices

After you initiate a scan from automation Cacti will compare devices found to the device rules and SNMP options you set.

If Cacti is unable to match the device to a device template or unable to find a proper SNMP credential.

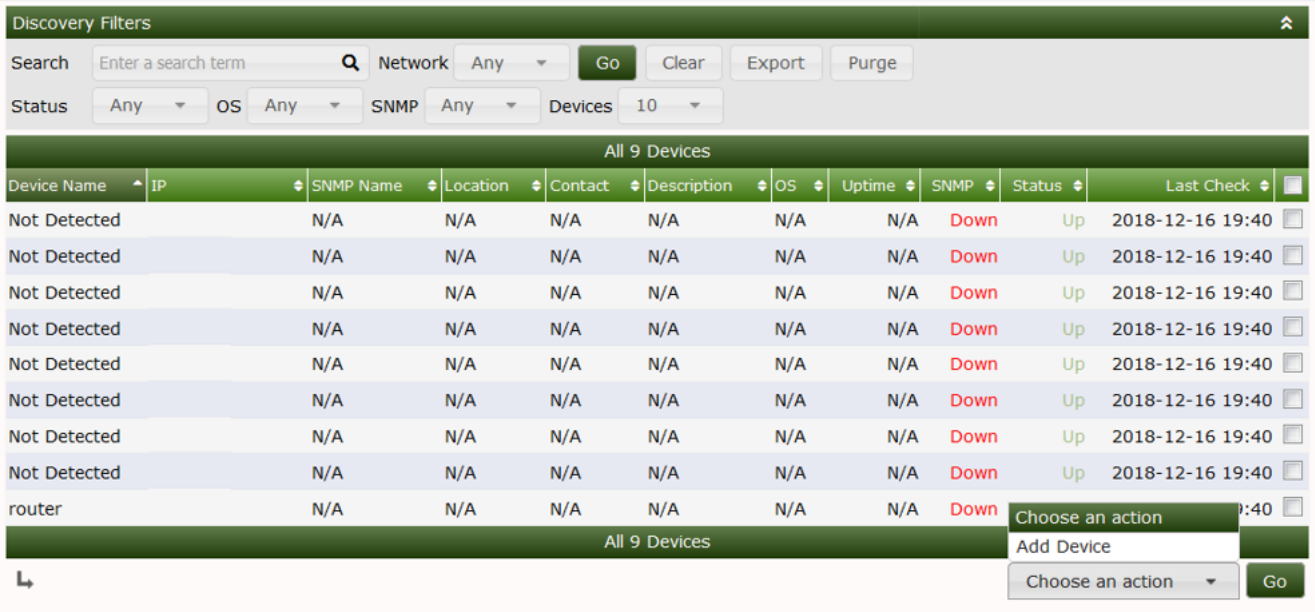
Also if you have the automatically add to Cacti set to off the devices will also be put in this section.

The Discovered Devices section can be found by navigating to the following area.



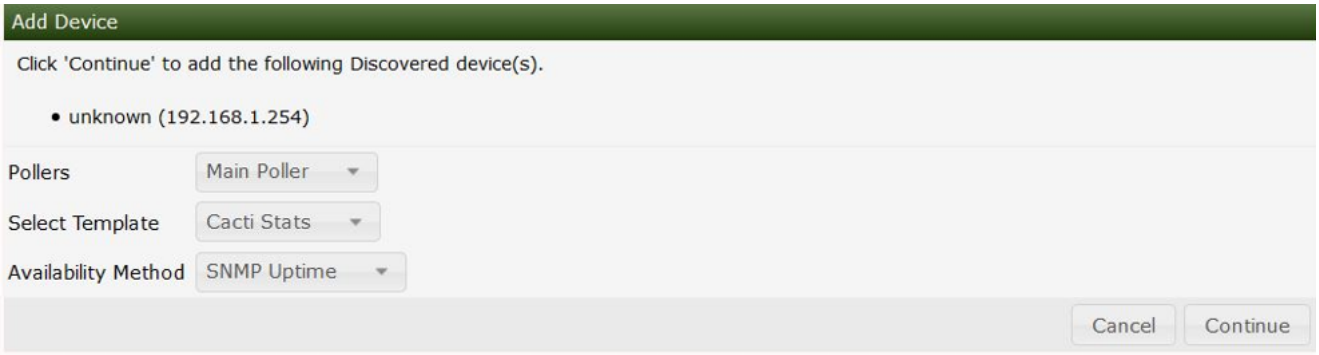
Below you can see devices that have been found during the scan that did not meet a match criteria.

You will see the IP that was scanned as well if available the resolved hostname either via DNS or netbios.



You can then select the device you are interested in adding click the checkbox next to the device

on the dropdown you select add device this will bring up the below menu. From this Menu you will be able to fill out the details of the new device.

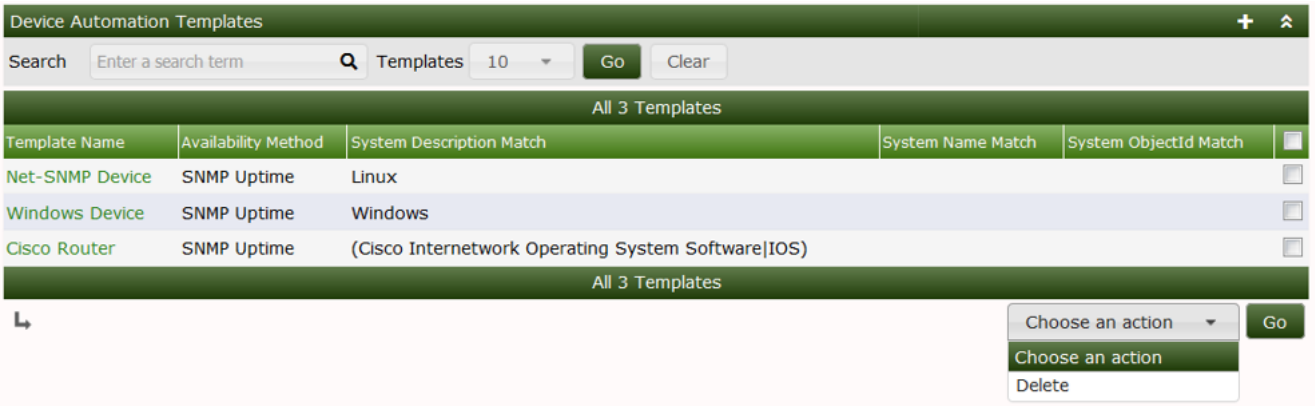


Copyright (c) 2004-2026 The Cacti Group

Automation Device Rules

This section will describe **Device Rules** in Cacti.

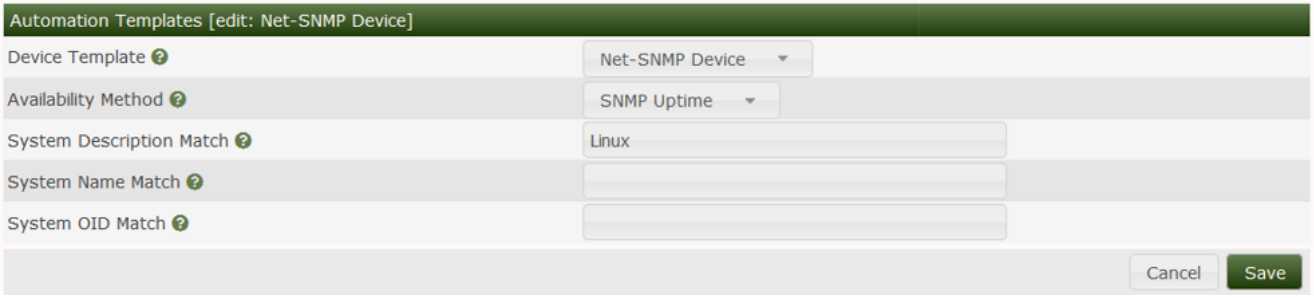
Cacti Automation relies on create device Rules these rules help the automation process figure out what type of device is being scanned. Based on the device rule REGEX the automation engine can apply the appropriate template to the device i.e if there is a REGEX for IOS. The automation engine may pick the Cisco router template for the device.



Below you will be able to choose a REGEX that will help identify the device the REGEX can be in any Perl supported format. You can select either to match **sysDescr**, the device's system description, or **sysName**, its administratively assigned name.

An example regex for Cisco would be (Cisco Internetwork Operating System Software|IOS) Use that pattern against **sysDescr**, where IOS software identifies itself.

Using this info automation would apply the appropriate device template you choose. If you see below we match for a system that matches Linux once a device is found with that system name it is assigned the Net-SNMP device template.



If the System/Device you are matching has a unique OID you can also match based on that.

Copyright (c) 2004-2026 The Cacti Group

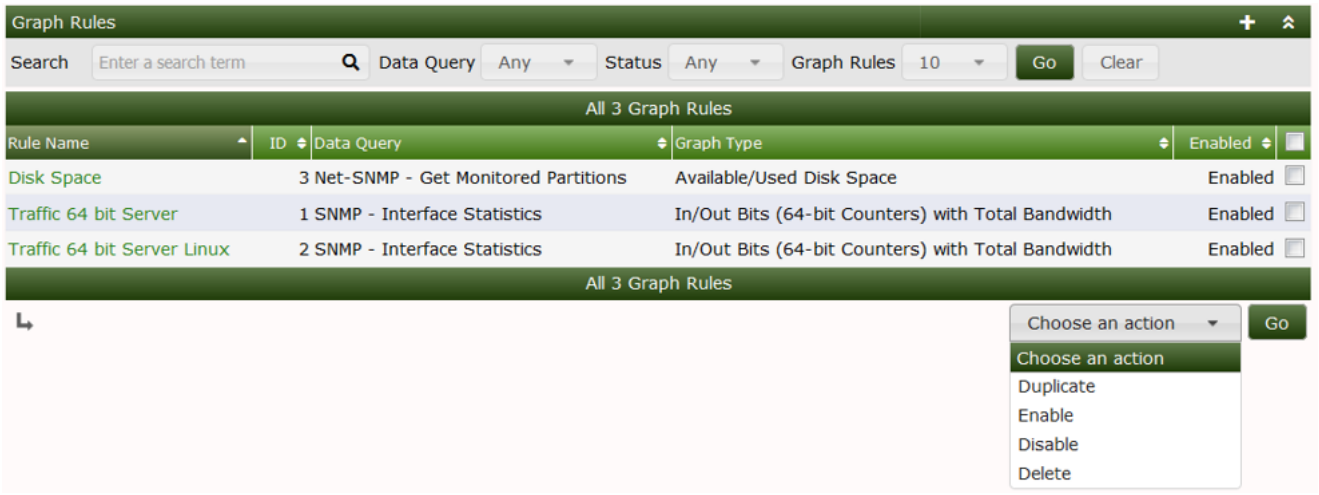
Automation Graph Rules

This section will describe **Graph Rules** in Cacti.

Graph rules are used by the Cacti automation process after a device has been identified and a template has been applied the next step is graph creation. Graph rules are processed from top to bottom.

The graph rule matches the graph template by looking at the data fields chosen that are available in the data query for example below we are creating graphs for interface traffic the data query has several fields available to match on.

You can see how these rules would take place by clicking the show matching objects link.



For example, filter interface query rows using `ifOperStatus` to select operationally up interfaces. Check the actual cached value and use **Show Matching Objects** to verify the rule. If both administrative and operational state must be up, also filter `ifAdminStatus`; `ifOperStatus` alone does not test both states.

A device selection Criteria is also important Cacti needs a way to find which device it needs to match. In the below example we are matching any device that has an interface that is up and is not a Windows server.

You can preview which devices this will apply to by clicking the Show matching devices link.

Rule Item [edit rule item for Device Match Rule: Traffic 64 bit Server]

Operation	
Field Name	H: snmp_sysDescr - varchar(300)
Operator	is not equal to
Matching Pattern	
Sequence	1

Cancel Save

Rule Item [edit rule item for Create Graph Rule: Traffic 64 bit Server]

Operation	AND
Field Name	ifIP - IP Address
Operator	is not empty
Matching Pattern	
Sequence	2

Cancel Save

This output shows us which devices match our devices rules this can help with tweaking rules as needed.

*Show Rule Details.
*Don't Show Matching Devices.
*Show Matching Objects.

Matching Devices

Search Enter a search term Type Any Status Any Devices Default Go Clear

All 9 Devices

Description	Hostname	Status	Device Template Name	ID	Graphs	Data Sources
CoreServer		Up	Net-SNMP Device	4	10	17
NasServer		Up	Net-SNMP Device	6	16	23
NetUPS		Up	Generic SNMP Device	10	2	2
WebServer1		Up	Net-SNMP Device	9	11	18
WebServer2		Up	Net-SNMP Device	8	10	17
WebServer3		Up	Net-SNMP Device	3	10	17
WebServer4		Up	Net-SNMP Device	2	10	17
WebServer5		Up	Net-SNMP Device	12	10	17
WebServer6		Up	Net-SNMP Device	11	10	17

All 9 Devices

This output shows which interfaces,drives,sensors etc will be graphed based on the graph rules.

*Show Rule Details.
*Show Matching Devices.
*Don't Show Matching Objects.

Matching Objects

Search Enter a search term Objects 10 Go Clear

Matching Objects [SNMP - Interface Statistics]

All 8 Matching Objects

Hostname	Device Status	Index	Status	Description	Name (IF-MIB)	Alias (IF-MIB)	Type	Speed	High Speed	Hardware Address
	Up	3	Up	eth0	eth0		ethernetCsmacd	1000000000	1000	
	Up	2	Up	eth1	eth1		ethernetCsmacd	100000000	100	
	Up	2	Up	eno1	eno1		ethernetCsmacd	1000000000	1000	
	Up	2	Up	eno1	eno1		ethernetCsmacd	1000000000	1000	
	Up	2	Up	eno1	eno1		ethernetCsmacd	1000000000	1000	
	Up	2	Up	eno1	eno1		ethernetCsmacd	1000000000	1000	
	Up	2	Up	eno1	eno1		ethernetCsmacd	1000000000	1000	
	Up	2	Up	eno1	eno1		ethernetCsmacd	1000000000	1000	

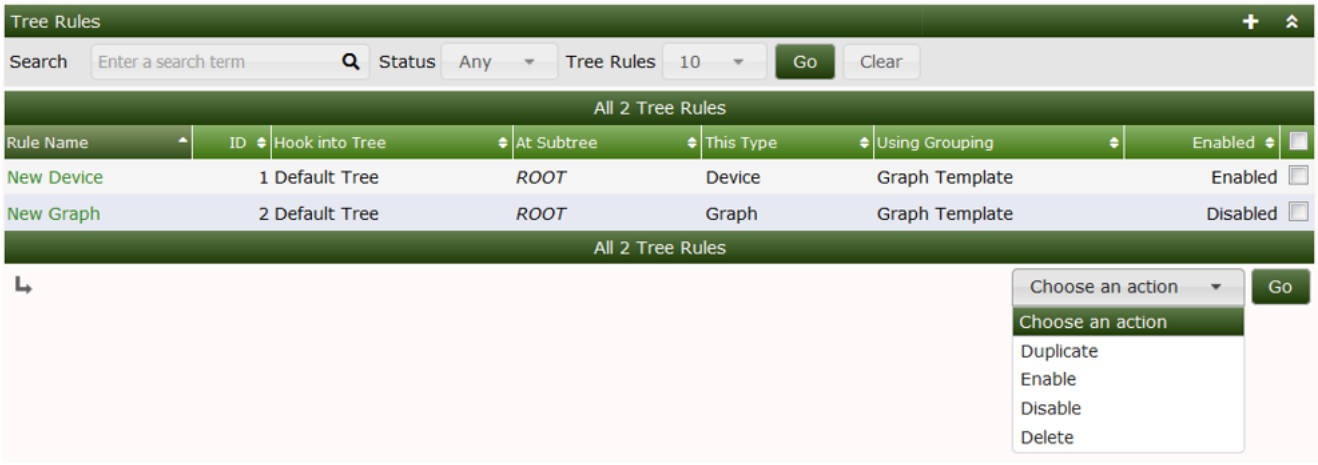
Copyright (c) 2004-2026 The Cacti Group

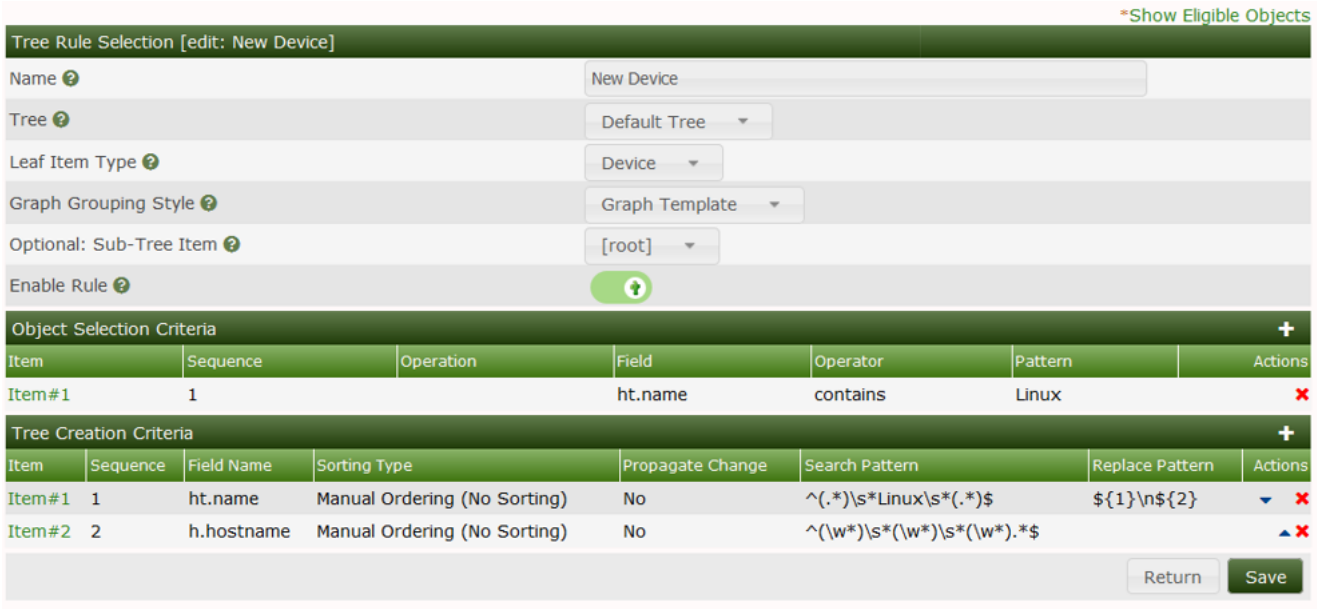
Automation Tree Rules

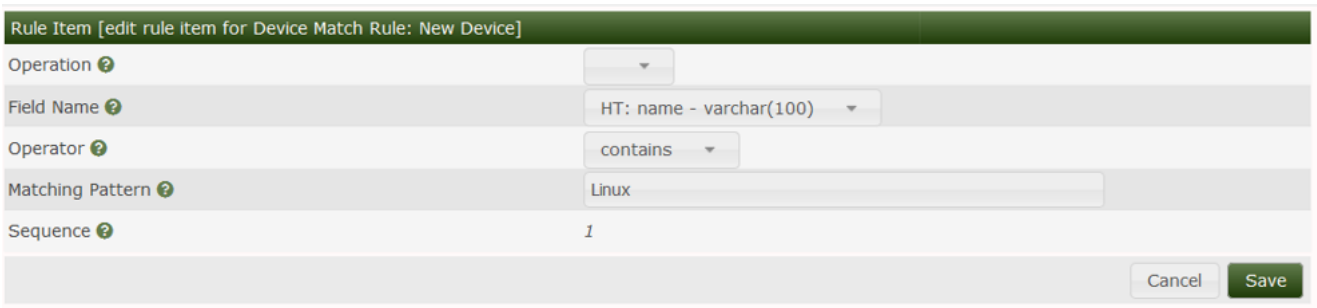
This section will describe **Tree Rules** in Cacti.

Tree rules allow Automation to bunch up devices based on matches on rules we come up with you can say bunch all Cisco devices on to a Cisco tree. Or if you have a site where all of the hostnames start with siteb you can have a site b tree etc see below for some examples.

To get started you will need to create a Tree rule on the top right click the + to start a new rule you are also able to duplicate rules to make it easier as well.

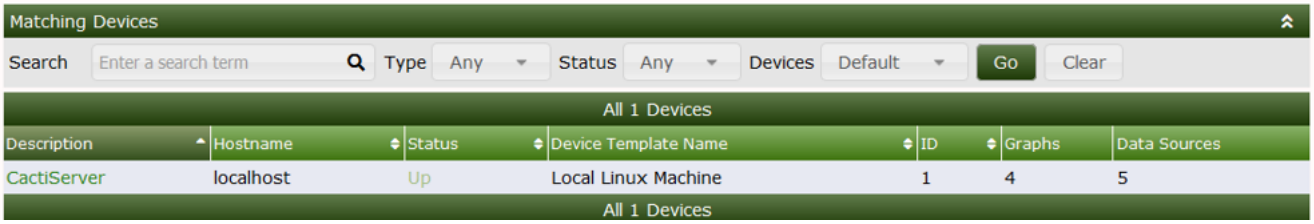




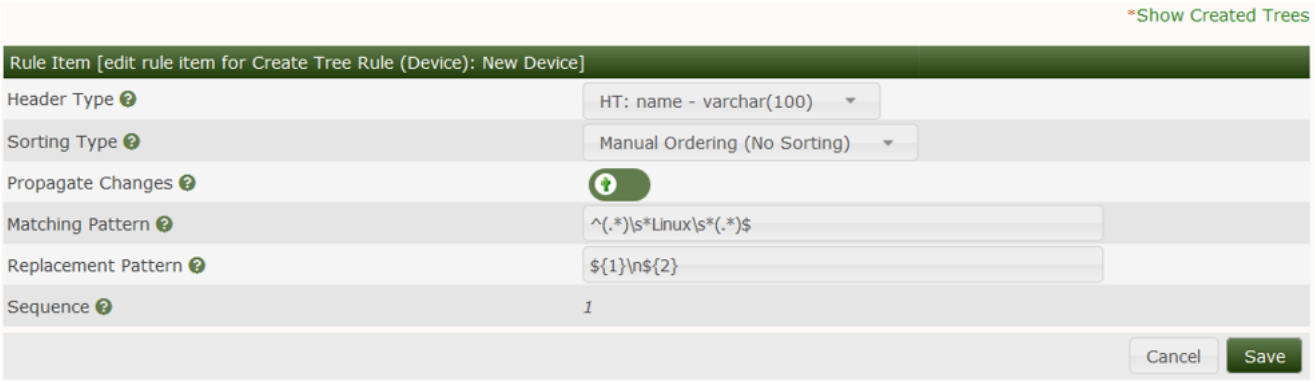


We need a way to tell automation what device we are looking for in the example above we are looking for a device with the hostname Linux this is done via a REGEX any perl based REGEX can be used.

To test this REGEX on existing devices you can click the show eligible Objects this will check the Devices currently in cacti for a match.



Once a device is a match it will be placed on the default tree. To change which tree the device will be placed on simply select the tree name from the dropdown list.



We also have the option to auto create a tree based on our rule in this case a new tree Named Linux will be created if it does not already exist.

Video Walkthrough

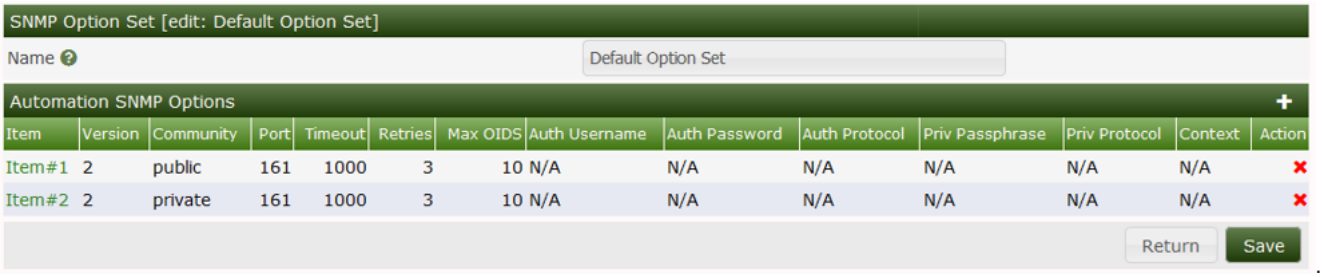
[Video Tutorial available online]

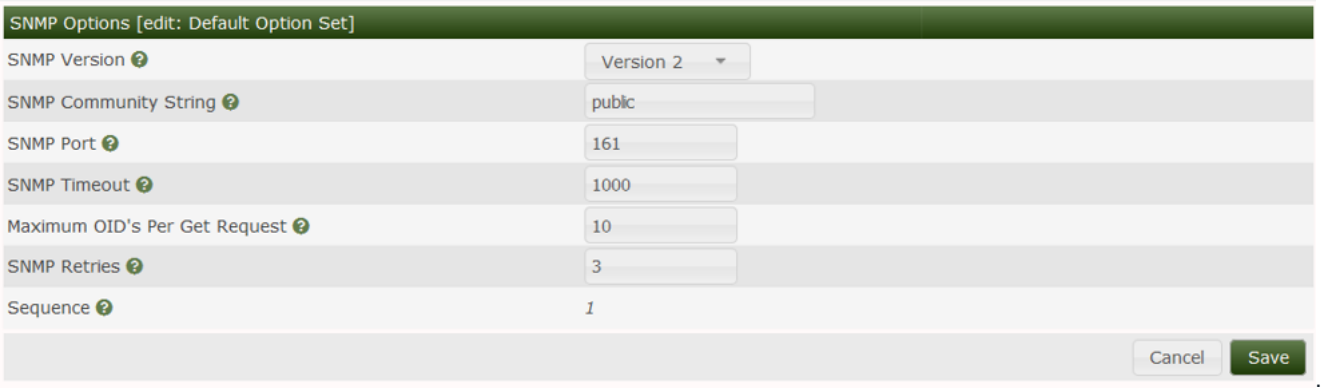
Copyright (c) 2004-2026 The Cacti Group

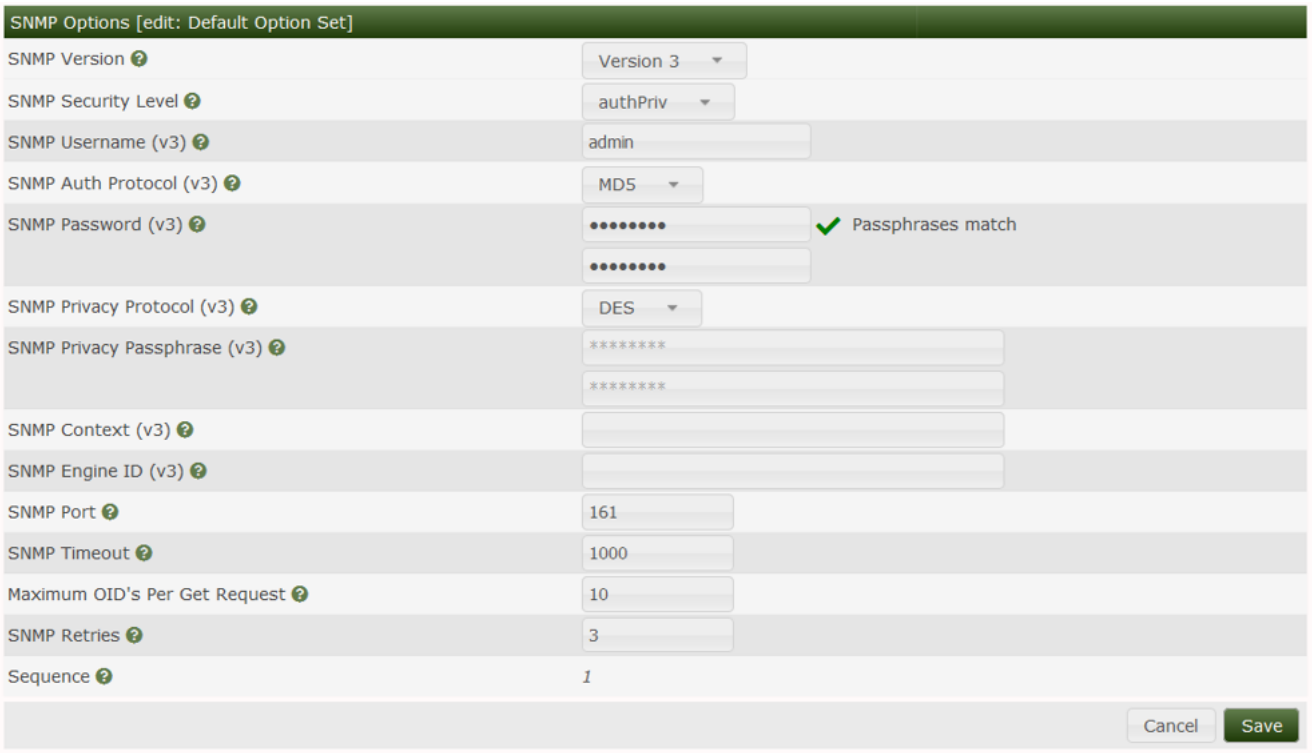
Automation SNMP Options

This section will describe **SNMP Options** in Cacti.

The SNMP Options page holds the SNMP strings and credentials that automation uses when it scans a network. Cacti supports SNMP v1, v2c and v3, so enter the details that match the devices you intend to poll.







Important: When performing a network scan ensure that you select the SNMP options that contain the SNMP strings you want to use

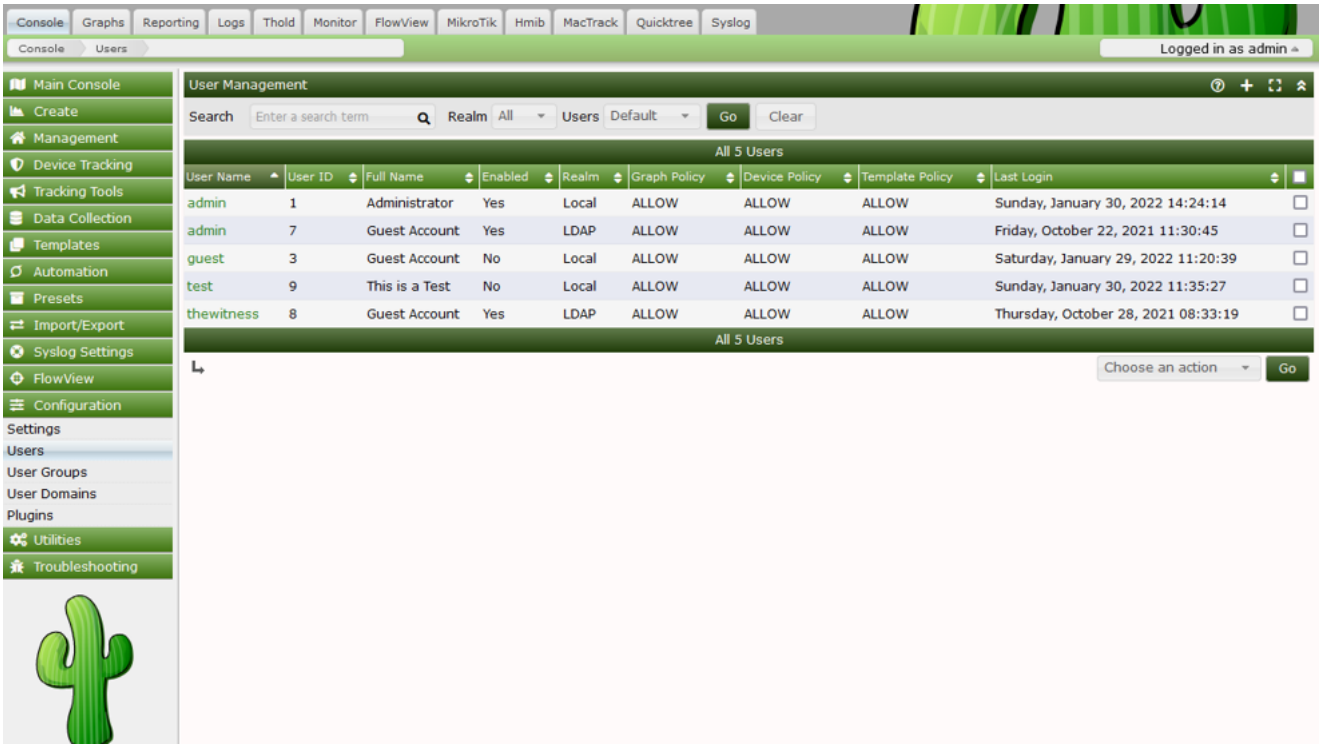
---

Copyright (c) 2004-2026 The Cacti Group

User Management

In addition to giving you the tools to create sophisticated **Graphs**, Cacti enables you to manage **User** and **User Group** access to those **Graphs** as well as various other areas of Cacti. Each **User** has a variety of settings and permissions. Some of these settings can be controlled by the individual **User** via editing their Profile, or controlled by the Cacti Administrator. A **Users** Cacti permissions are only modifiable by authorized Cacti Administrators.

Note: for a users to see new settings, they will have to log off and back in again to view.

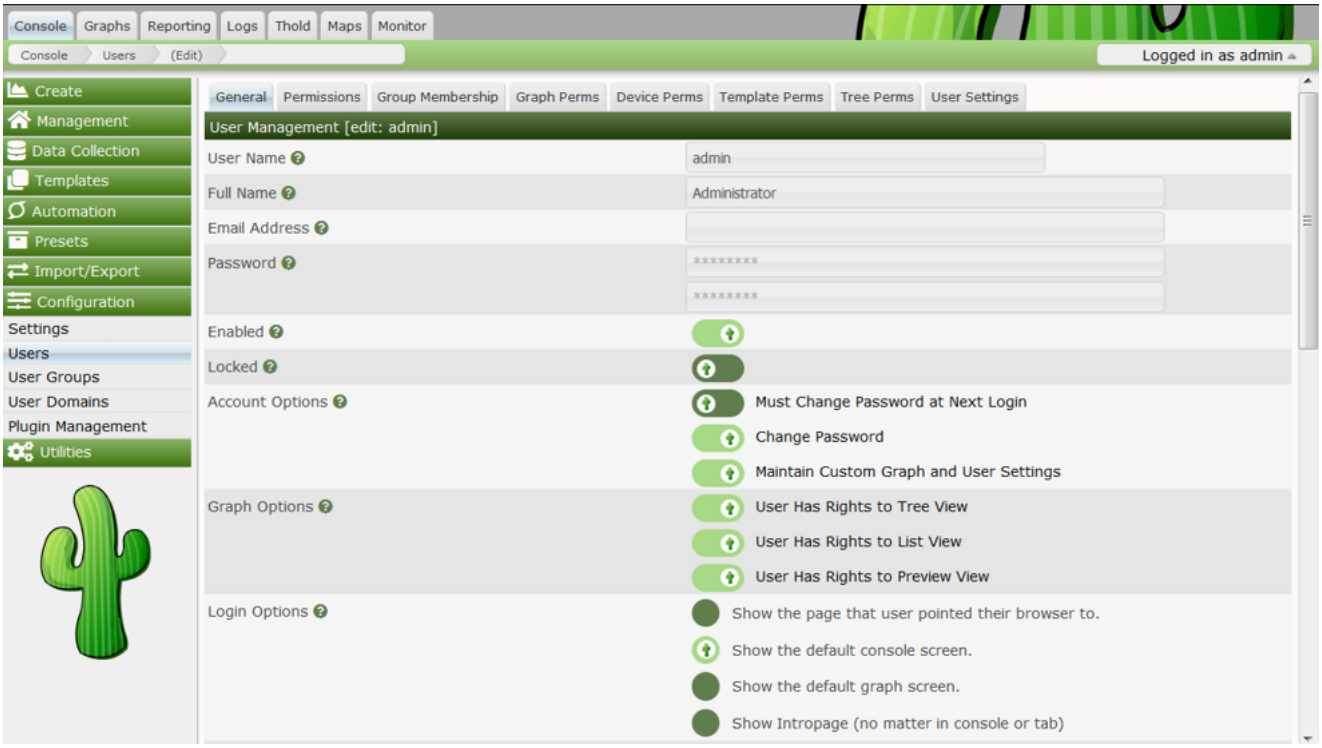


Out of the box, there are two users that come with every Cacti installation. The "admin" user - the main user that by default has access to see and change everything in Cacti. This is the user that you first login with in Cacti, and it is probably a good idea to keep around unless you know otherwise. The second user is the "guest" user, which controls which areas/graphs are allowed for unauthenticated users.

By default, this user only has rights to view, but not change graphs. This enables any unauthenticated user to visit 'graph_view.php' and view your graphs. This behavior can be changed by either changing the realm permissions for the "guest" user, or disabling the guest user altogether under Cacti Settings. By default in 0.8.7 and later, the "guest" user is not enabled, effectively disabling "guest" (Unauthenticated) access to Cacti.

Editing an existing User

To edit an existing user, open **Console > Configuration > Users**. Once at the user management screen, click username of the user you wish to edit. You will see a screen that looks similar to the image below.



At minimum, you must specify a User Name and a Password for each user. Each user field is described in more detail below. In addition to these fields, each user can have their own realm permissions", graph permissions, and graph settings. Each of these items are described in this section of the manual.

User Management Options

- ***User Name***

  This field contains the login name for the user. Use the account name expected by the selected login realm. The development editor permits up to 255 characters; external providers can impose their own identity requirements.

- ***Full Name***

  (Optional) You can use this field as a more descriptive identifier for the user. It is currently only used for display on the user management page.

- ***Password***

  Enter the password for the user twice, once in each text box. Keep in mind that passwords are case sensitive and the password will remain unchanged if the boxes are both left empty.

- ***Account Options***

  There are two account related options that can be set for each user.

  - ***User Must Change Password at Next Login***

    Force a password change immediately after the user logs in.

  - ***Allow this User to Keep Custom Graph Settings***

    Allows user to maintain their own custom graph viewing settings.

- ***Graph Options*** -

  Enables certain defined graph viewing options.

  - *User Has Rights to Tree View*

- *User Has Rights to List View*
- *User Has Rights to Preview View*

- ***Login Options***

  These options set what occurs after the user logs in.

  - ***Show the page that user pointed their browser to***

    Will point the user to whatever page they were heading to before being interrupted by the login page.

  - ***Show the default console screen***

    This option will always point the user to 'index.php' after a successful login.

  - ***Show the default graph screen***

    This option will point the user to default graph view after login.

- ***Authentication Realm***

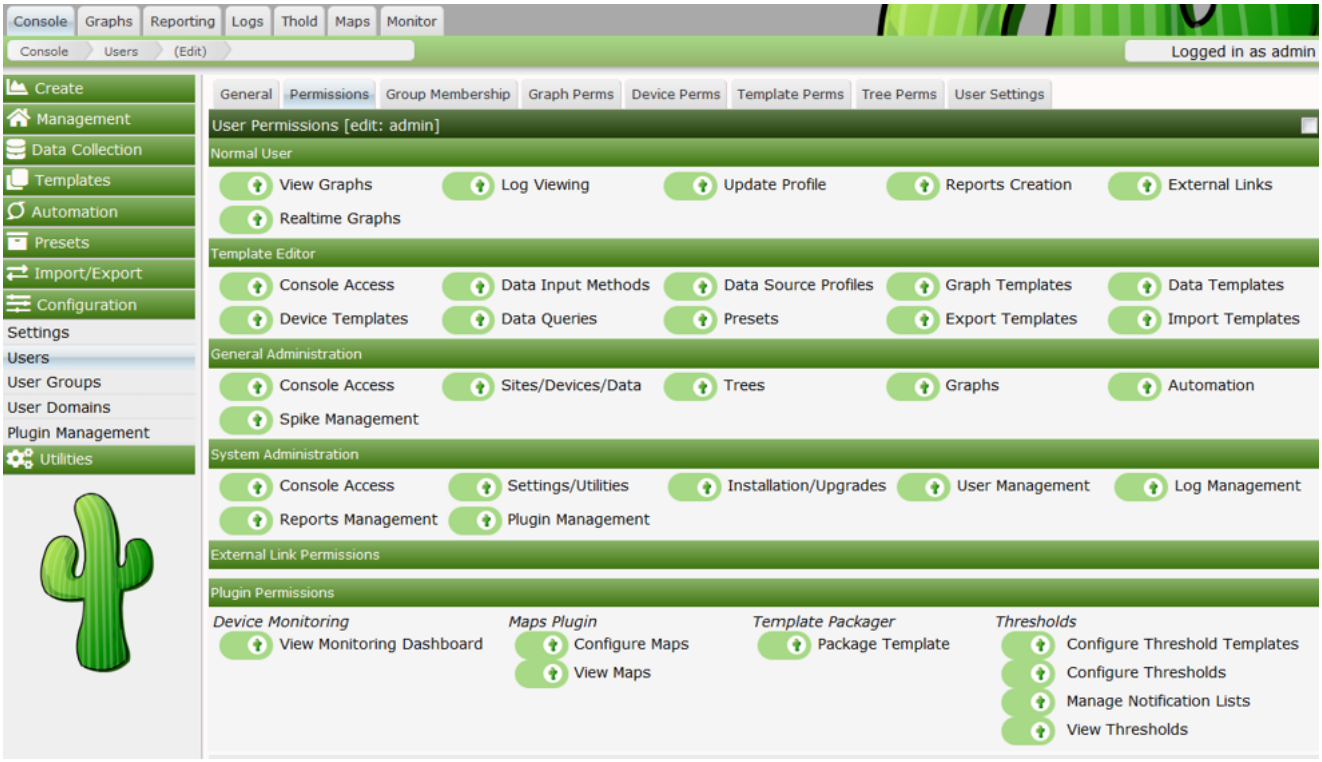
  This setting allows you to specify a user for use with `Local`, `Web Basic` or `LDAP` Authentication.

  Note: Template users must be Local.

User Interface Permissions

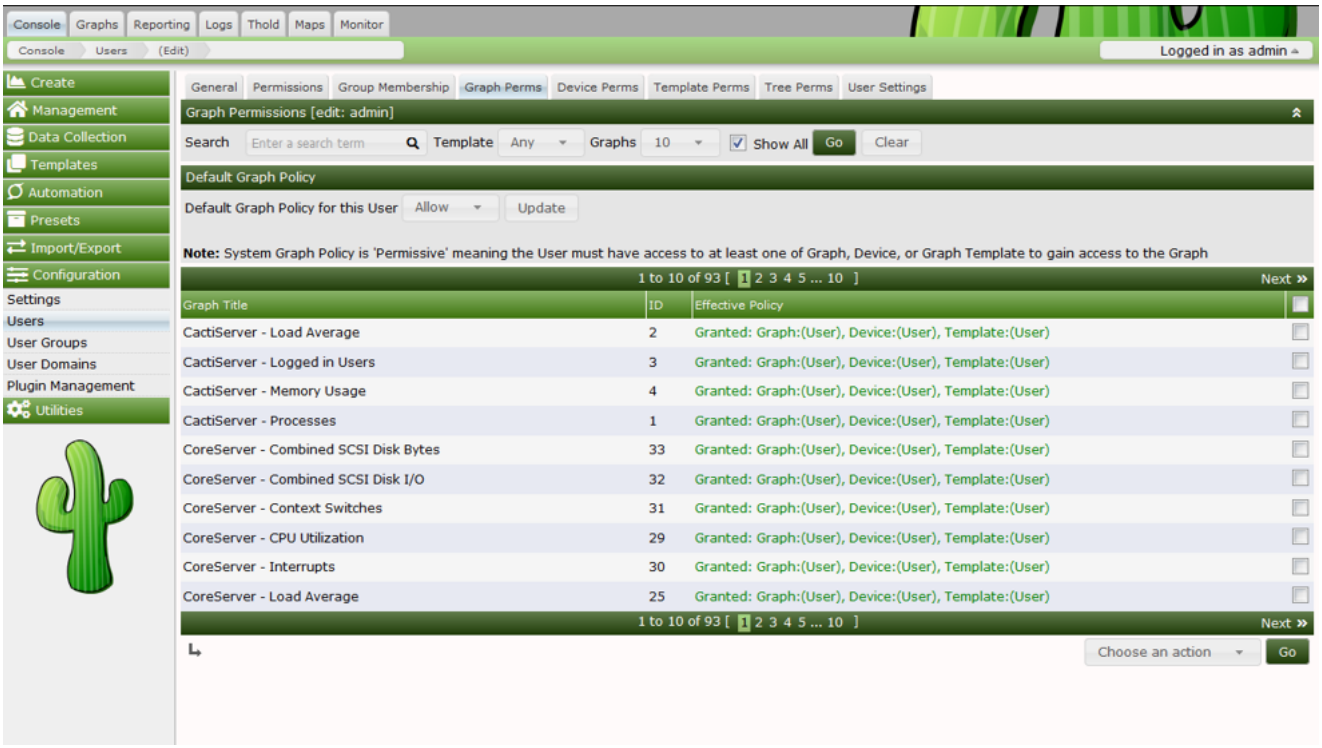
Realm permissions control which areas of Cacti a user can access. You can edit a user's realm permissions by selecting User Management and choosing the user you want to edit the permissions for. The Realm Permissions box will be displayed in the lower part of the screen. Each "realm" is a grouping that represents common tasks in Cacti, making it easier to fine tune each user's access.

If you want to create a user that can only view graphs, you should select the View Graphs realm and leave everything else unchecked. See the Graph Permissions section for more information about how to fine tune this even more on a per-graph basis. Conversely, if the user needs to access the console, they will need Console Access and any additional realms that you see fit.



Graph Permissions

Graph Permissions control which graphs a user is allowed to view. However, in general, these permissions do not apply to editing graphs. You can edit a **Users Graph Permissions** either by assigning that **User** to one to many **User Groups**, or by modifying their **Graph Permissions** directly.



Cacti supports four Graph Permission models on the development branch. There is the `Permissive` Graph permission which states that if a user has access either to the Graph, the Device or Graph Template, the user will have access to all the Graphs that match that class. For example, in the `Permissive` model, if you have access to the **Device**, you have access to all it's **Graphs**. If you have access to a **Graph Template**, you have access to every **Device Graph** that matches that **Graph Template**.

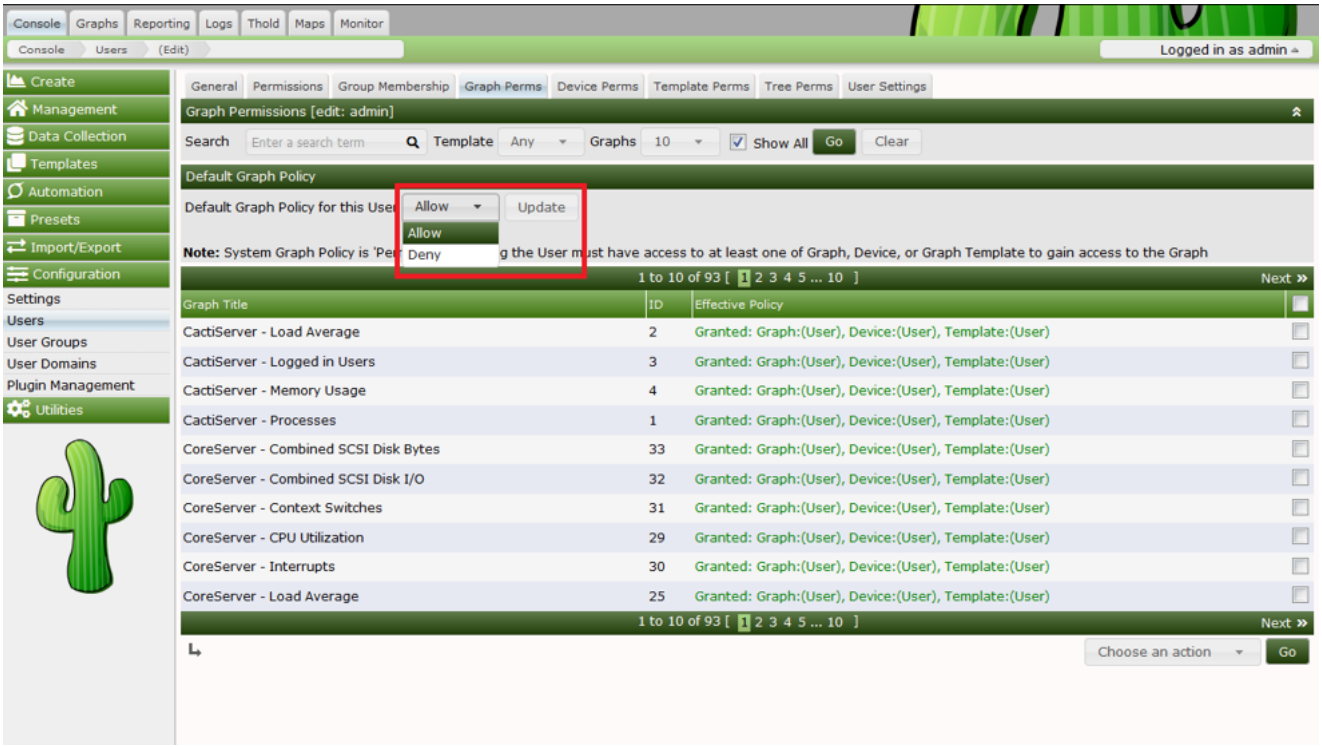
The second Graph Permission model is `Restrictive`. In the `Restrictive` Graph permission model, you must have access either to the **Graph** directly, or both the **Device** and **Graph Template**. Using this permission model requires the **Users** and **User Groups** have much more specific access if they want to gain access to various device Graphs.

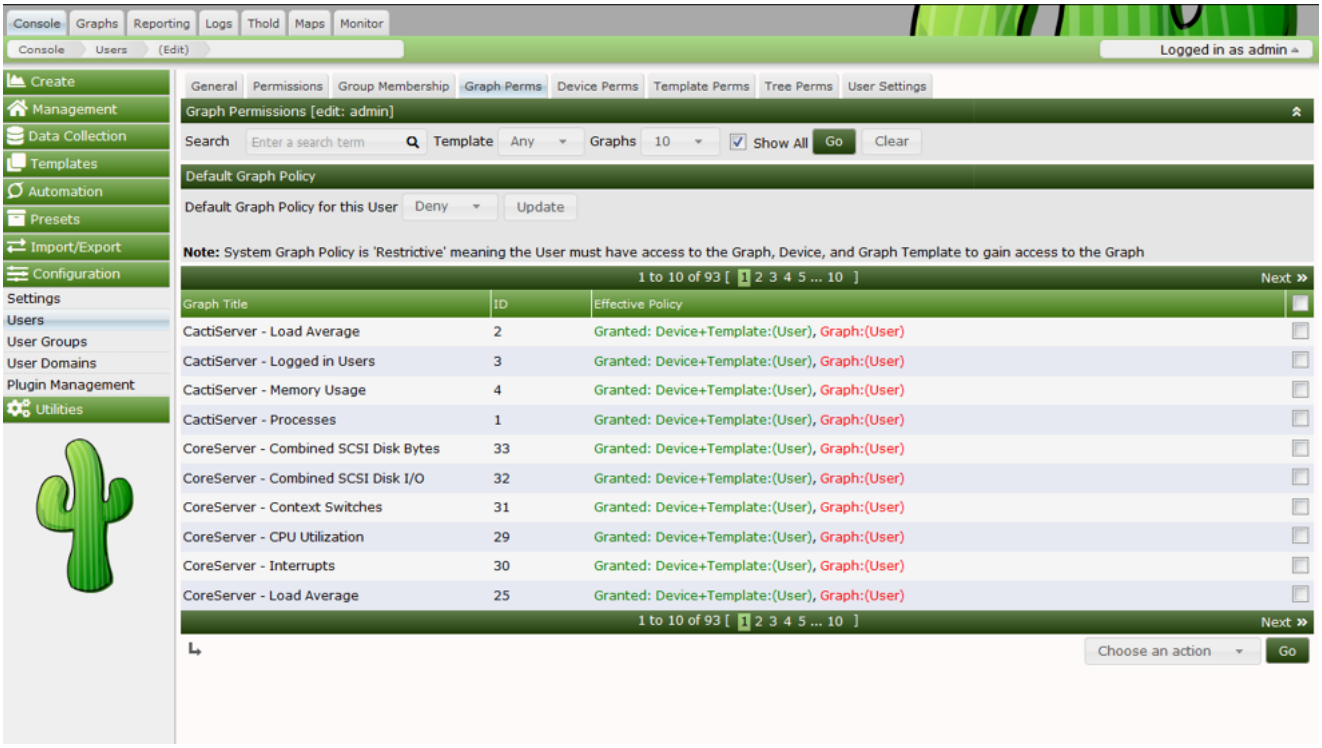
The development branch also offers `Device Based` and `Graph Template Based` methods. These use the corresponding device or graph-template policy, while retaining direct graph permissions. Check the selected method when diagnosing access that differs from the device or template checkbox alone.

In addition to the `Permissive` and `Restrictive` models, which can be set from **Console > Configuration > Settings > General**, a **User** or **User Group** is either `Allowed` or `Denied` all **Graphs**, **Devices** and **Graph Templates** through a setting called the **Default Policy** in each of the Permission Tabs. The **Graph Permissions** tab includes an `Effective Policy`, which combines all the **Users** various **User Group** and **User** permissions to provide an effective permission. If the **Graph** is ultimately permitted for the **User** to view, you will see for what reason from this Permission tab.

Note: even if a **User** is denied access to a **Device** or **Graph Template**, but has access to at least one **Graph** from that **Device** or **Graph Template**, they will see that **Device** or **Graph Template** name in areas of Cacti that display it.

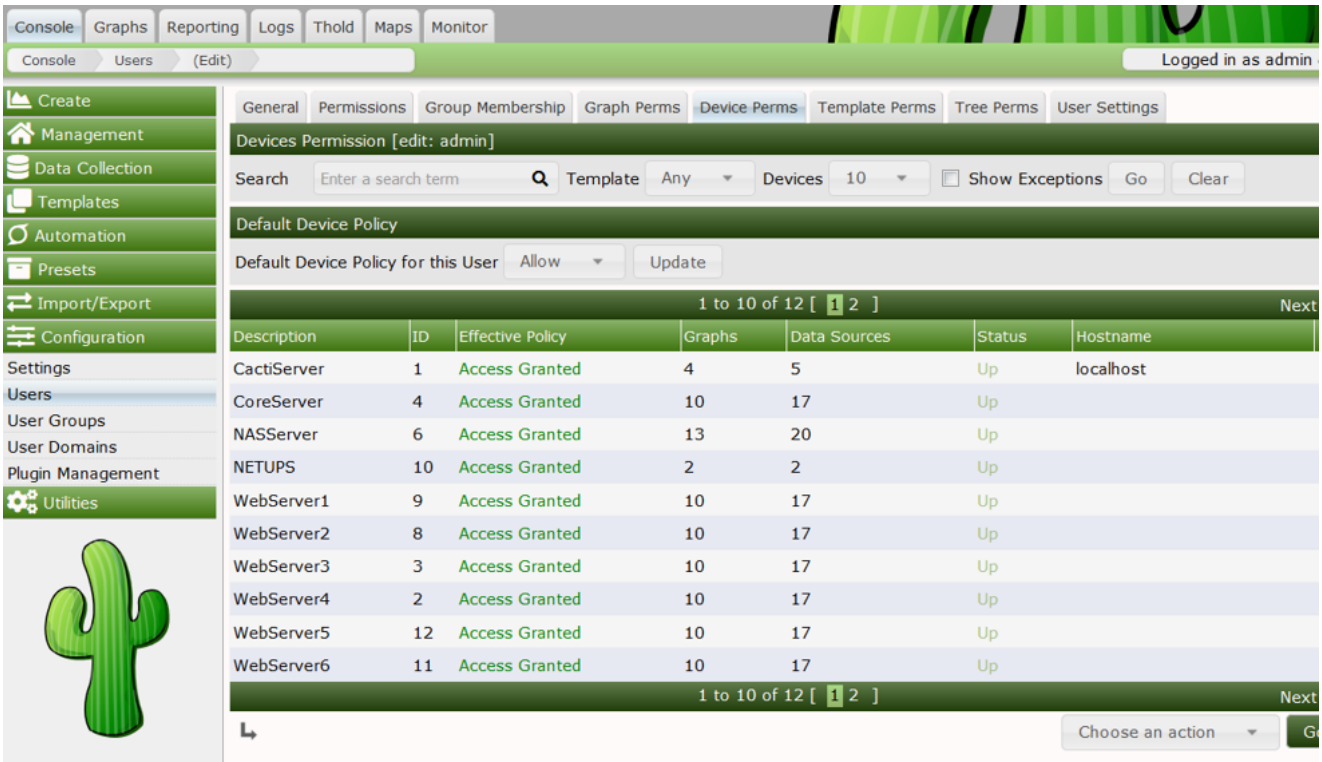
Default Permission Policies





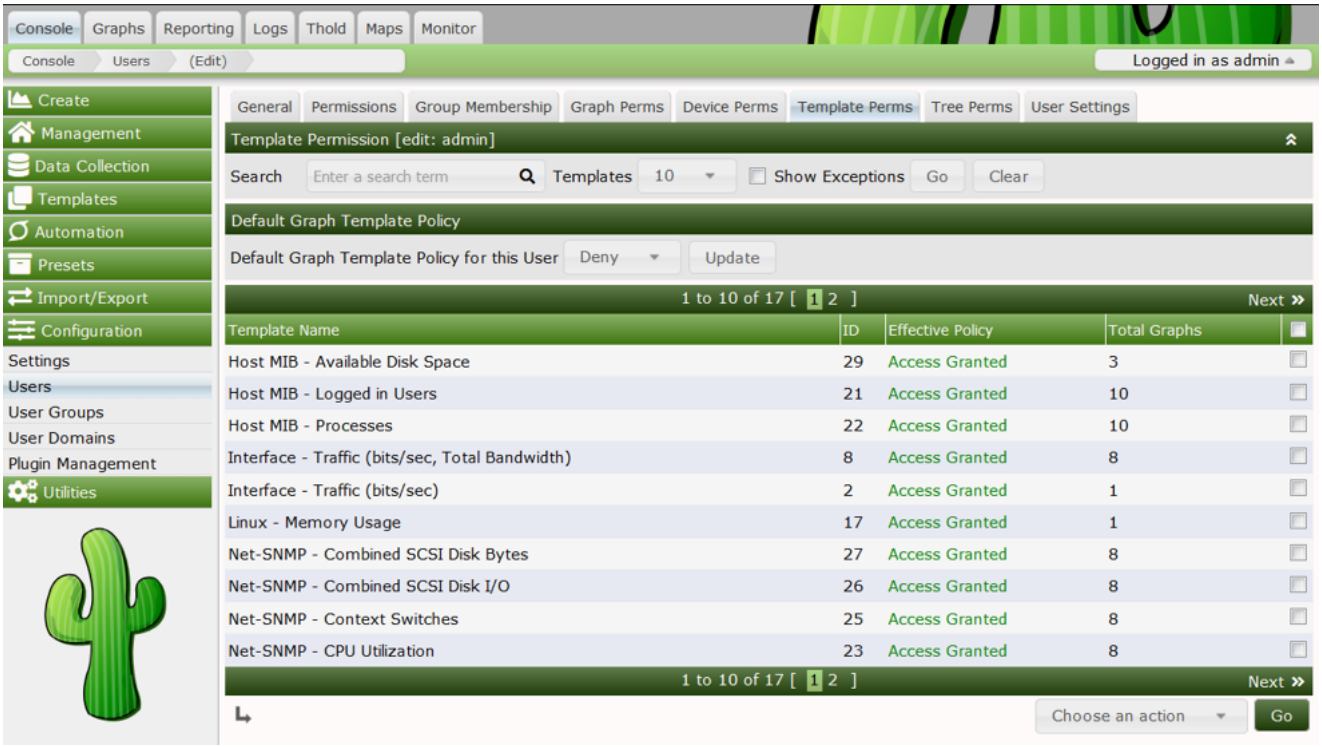
Device Permissions

Device Permissions allow you to `Grant` or `Revoke` access to specific **Devices**. If you wish to `Grant` or `Revoke` access to a **Device**, you can select it, and then choose the specified action from the drop down, and press the `Go` button to make that permission change. As mentioned above, revoking **Device** access does not necessarily block a **User** from seeing that **Device** from the User Interface.



Template Permissions

Template Permissions allow you to `Grant` or `Revoke` access to specific **Graph Templates**. If you wish to `Grant` or `Revoke` access to a **Graph Template**, you can select it, and then choose the specified action from the drop down, and press the `Go` button to make that permission change. As mentioned above, revoking **Graph Template** access does not necessarily block a **User** from seeing that **Graph Template** from the User Interface.

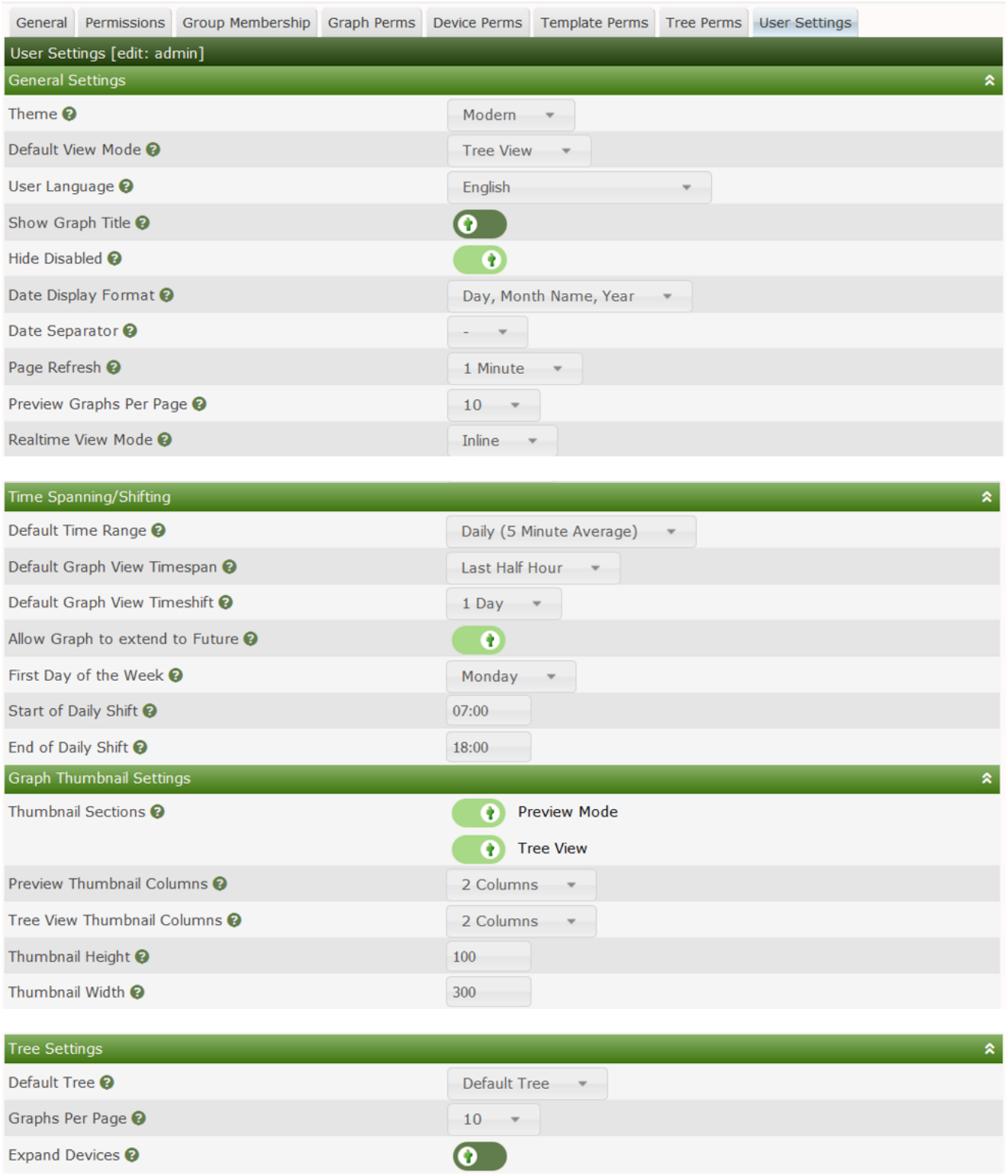


Tree Permissions

Cacti allows the Administrator to `Grant` and `Revoke` **Tree** permissions from this tab. The Cacti **Tree** permissions are a single level security object. However, when you browse a **Tree** to it's various branches, a user will only be allowed access to the Graphs and Devices that they have access to. Managing permissions to **Trees** is identical to **Graphs**, **Devices** and **Graph Templates** otherwise. Additionally, Cacti has will hide branches from a **User** when they have no access to any **Devices** or **Graphs** within it.

User Specific Settings

Cacti optionally allows **Users** to control certain **User Settings** such as their `Email Address` and `Full Name`. However, the Cacti Administrator also has the ability to view a **Users** settings and modify them for the **User** from the **User Settings** tab. This feature allows the Cacti Administrator to better support their **Users**. However, you can disable a **Users** ability to modify defaults from the `General` **User Management** tab. This is important for the `guest` account for example.

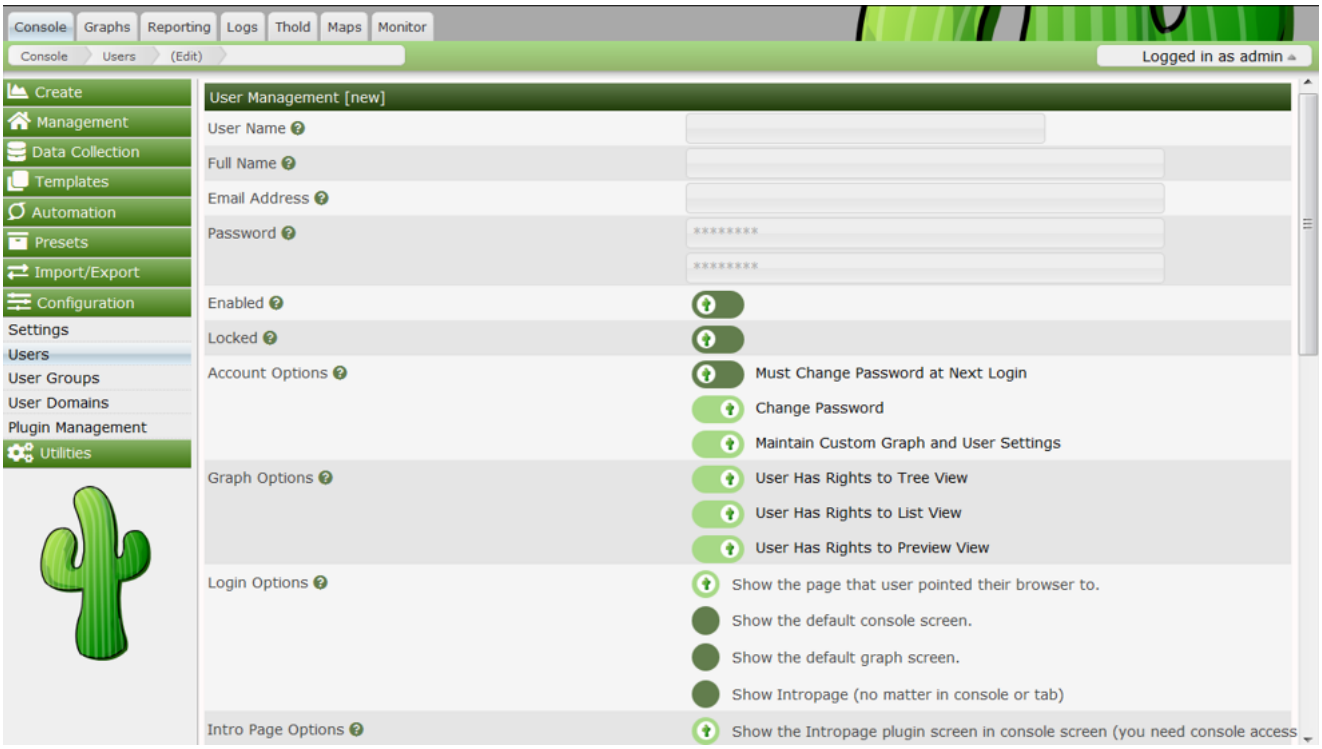


Caching of Permissions

Cacti caches permission results to reduce repeated work. After changing user or group access, test the effective permissions with the affected account in a new session. Include both allowed and denied objects in that check; do not infer access solely from a management-page checkbox or a previously loaded page.

Creating a New User

To create a new user, open **Console > Configuration > Users**. Once at the user management screen, click the plus (+) glyph.



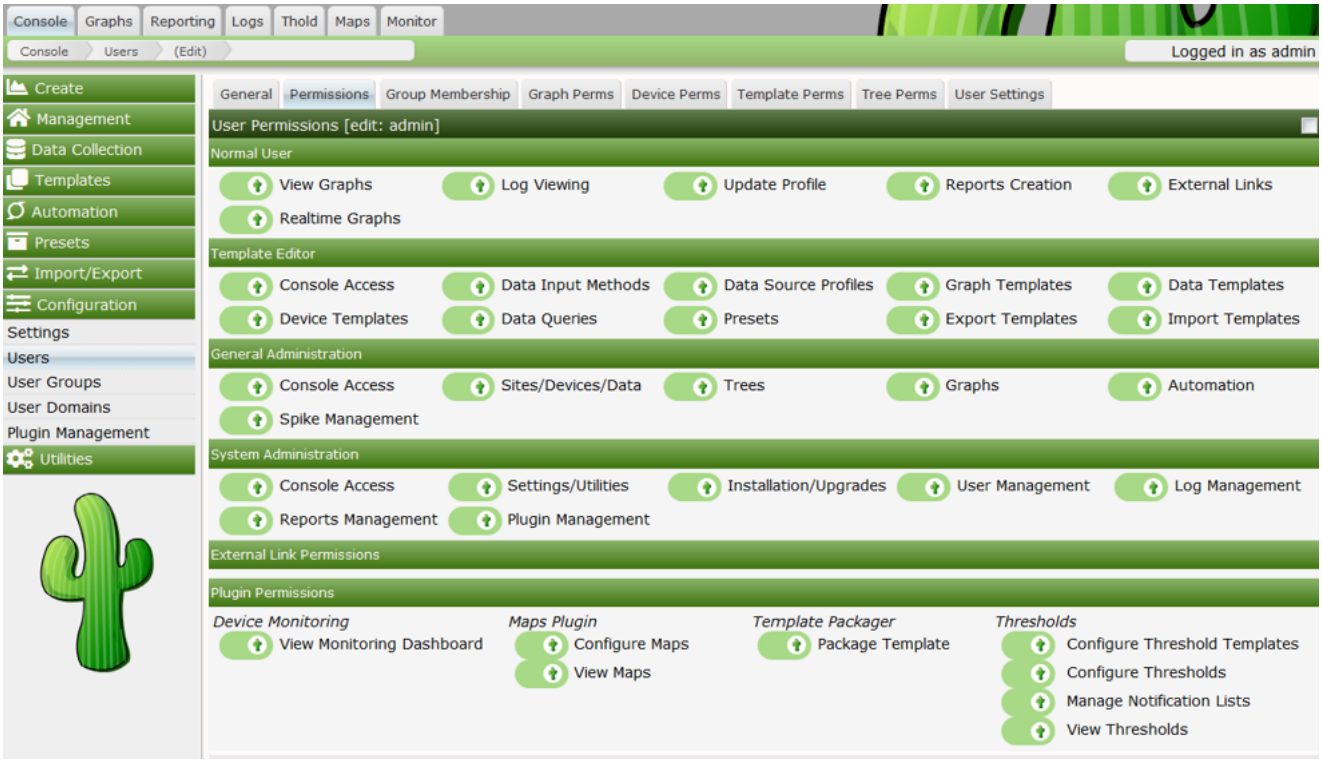
At minimum, you must specify a `User Name`, `Password` and `Login Realm` for each user. Each **User** field is self documented either below the field, or in the help bubble to the right of it. In addition to the fields on the `General` tab, settings can be controlled from the various sub-tabs when editing a user. Those sub tabs include:

General

From this sub-tab, you can modify general **User** attributes including Full Name, Email Address, Lockout status, Account Options, Graph Options, etc.

Permissions

From this sub-tab, you control what areas of Cacti a **User** has access to. Cacti has broken those access areas into various `Roles` to make the setting of permissions easier to understand. Note that **Plugin** permissions are still grouped by Plugin in the current version of Cacti. This will change over the next few releases to be more in line with the various Cacti Roles.

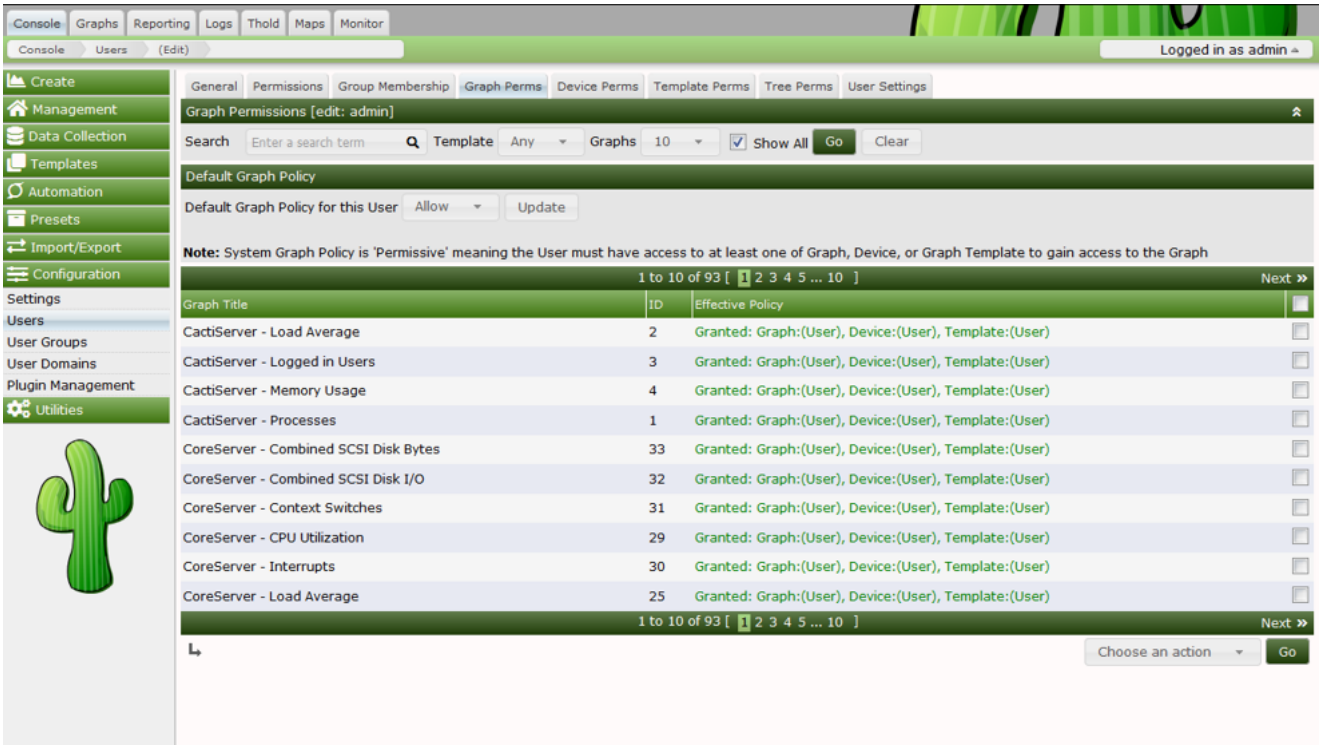


Group Membership

From this sub-tab, you can assign or remove users from various **User Groups**.

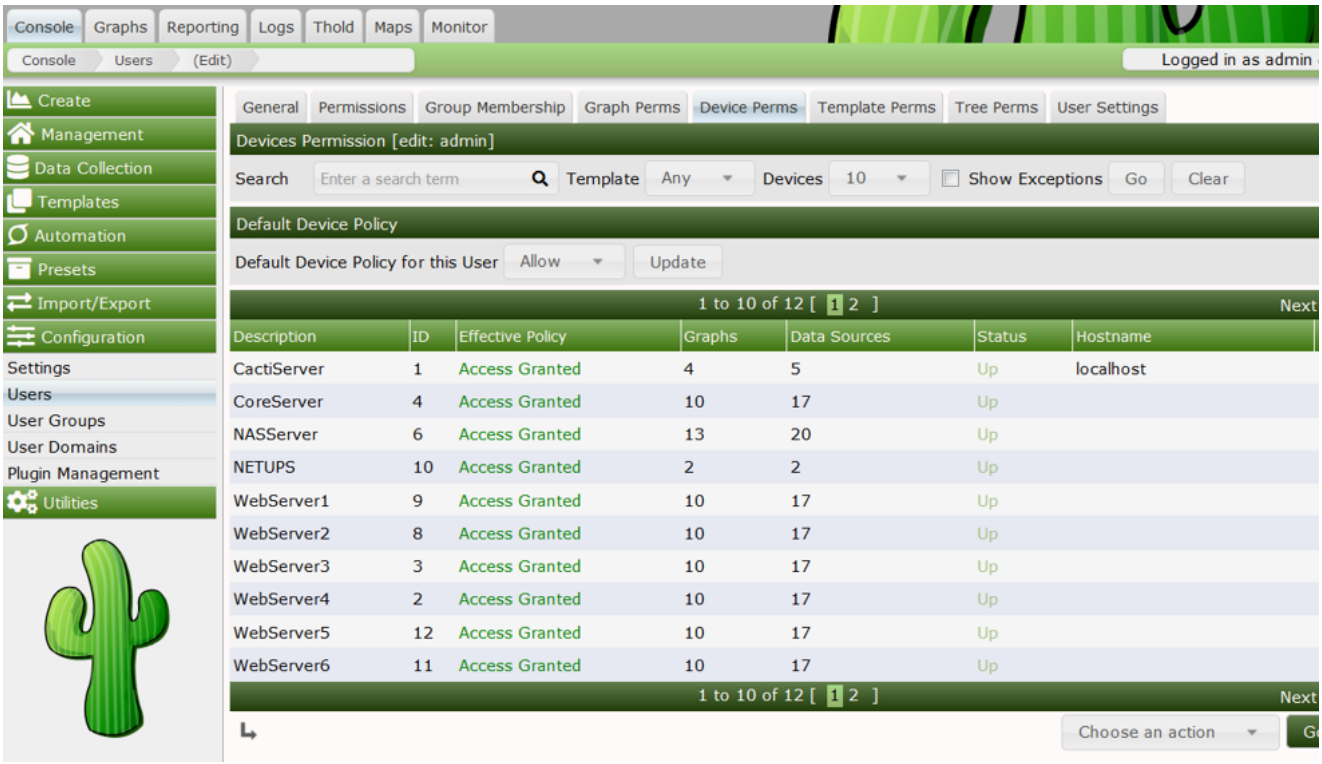
Graph Perms

From this sub-tab, you can view a users `Effective` Graph permissions and also `Grant` or `Revoke` access to **Graphs** for an individual **User**.



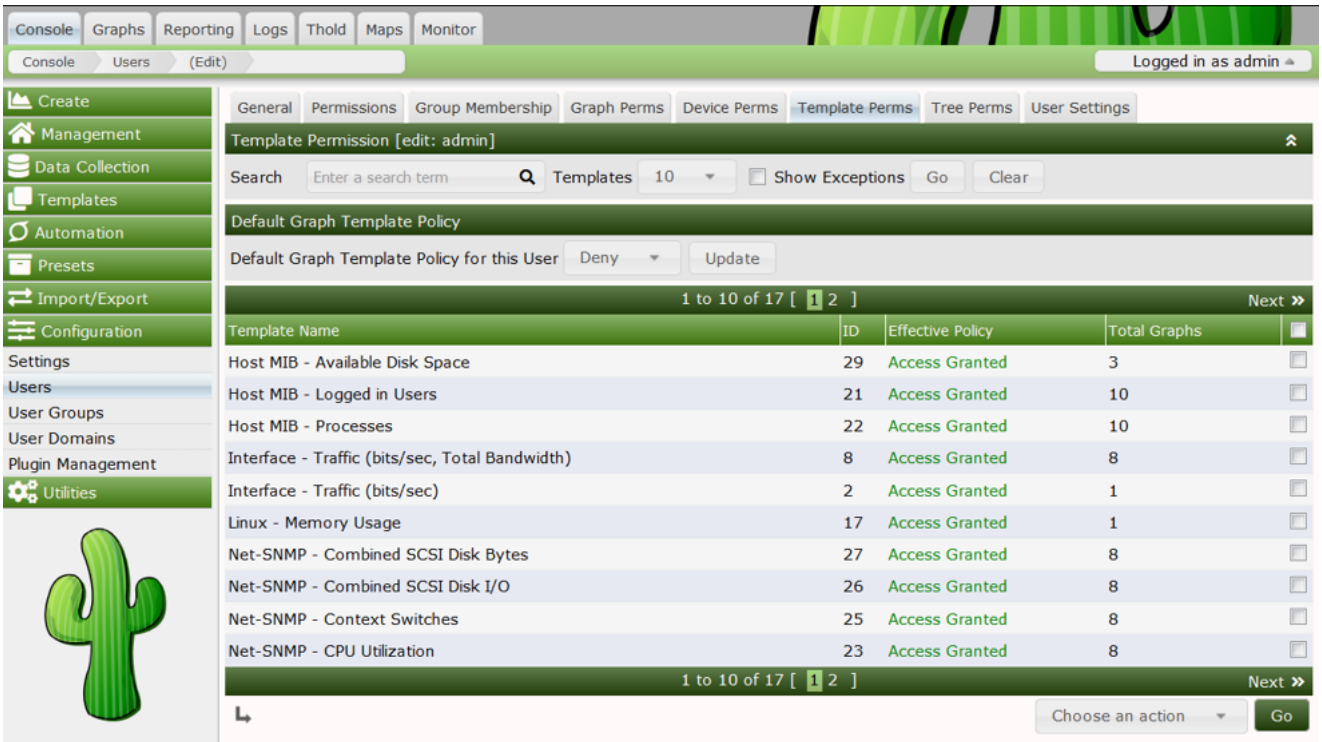
Device Perms

From this sub-tab, you can view and edit a users **Device** permissions. Note that removing a users permission to a **Device** can be overridden if they have access to either a **Graph** or, depending on the permission model you have chosen, a **Devices Graph Template**.



Template Perms

From this sub-tab, you can `Grant` or `Revoke` a **Users** access to various Cacti **Graph Templates**.

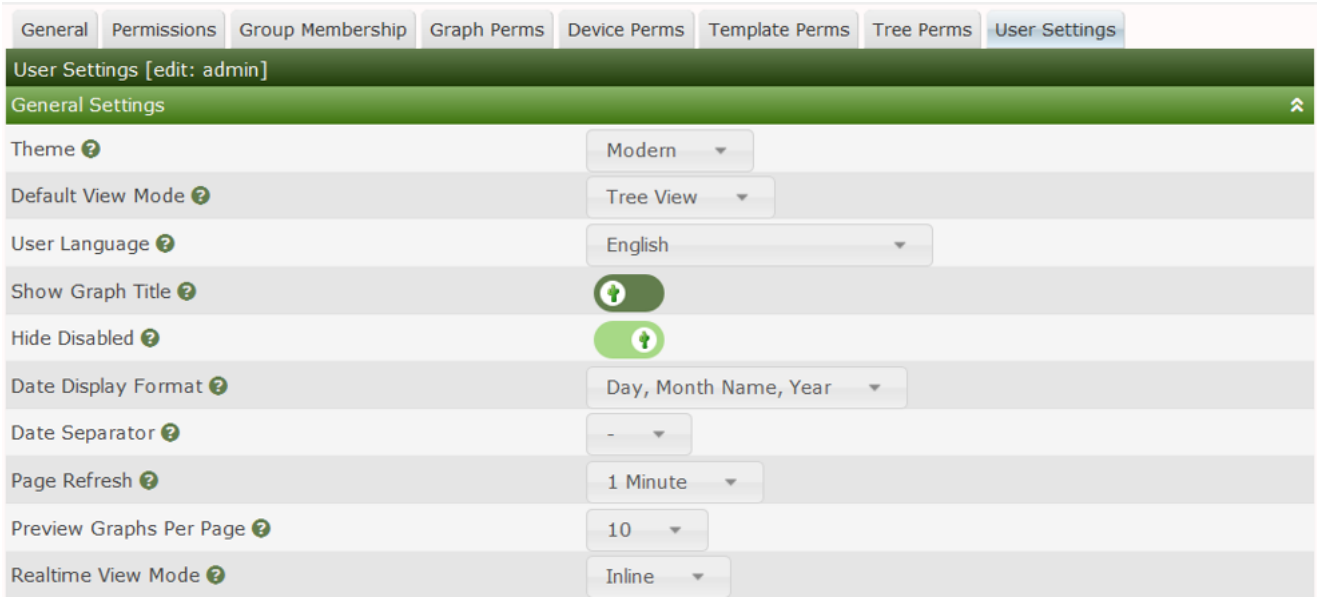


Tree Perms

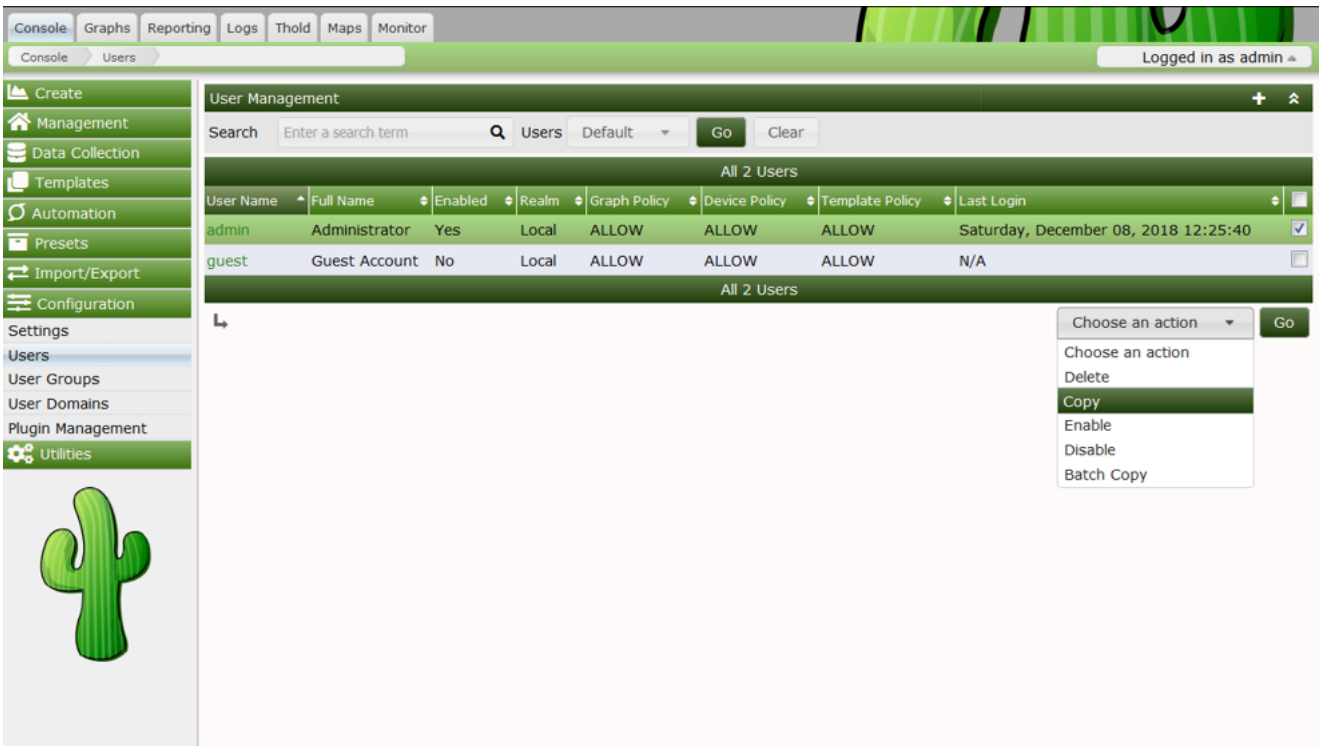
From this sub-tab, you can `Grant` or `Revoke` a **Users** access to the various Cacti **Trees**.

User Settings

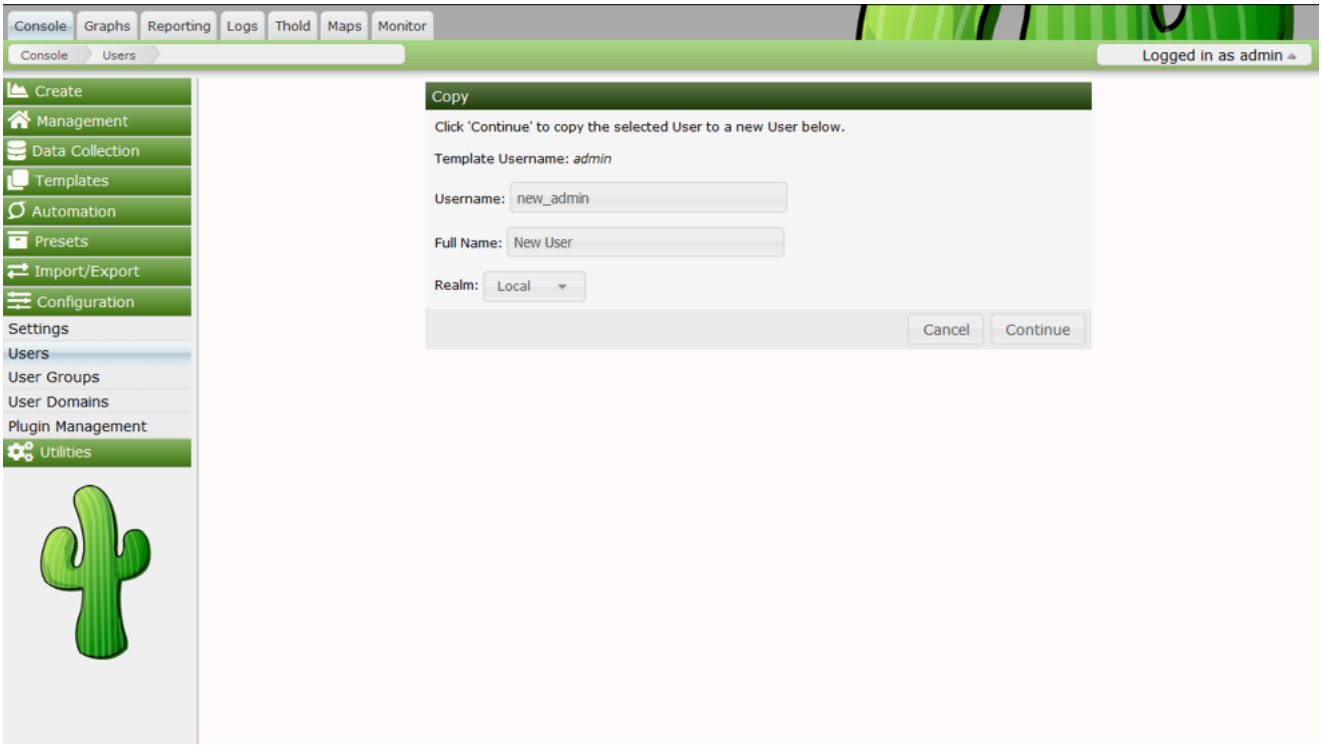
From this sub-tab, you can modify a **Users** settings to assist the **User** with their use of Cacti, or to set defaults for that **User**.



Copying a user

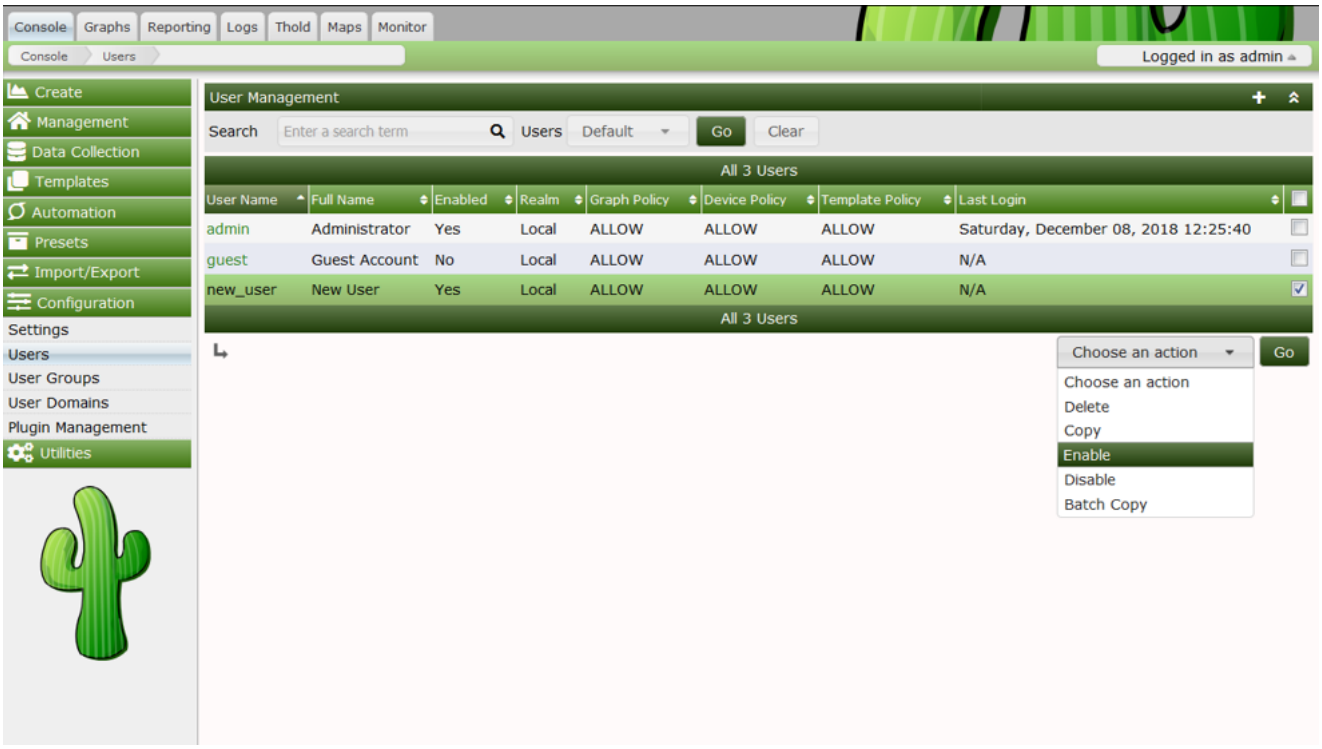


To copy a **User** to a new one, go to **Console > Configuration > Users**. Once at the **Users** interface, highlight the **User** you would like to copy from and select `Copy` from the `Choose an action` drop down selection box, click `Go` to continue. If you select multiple users, only the first selected **User** will be used as the source **User**.



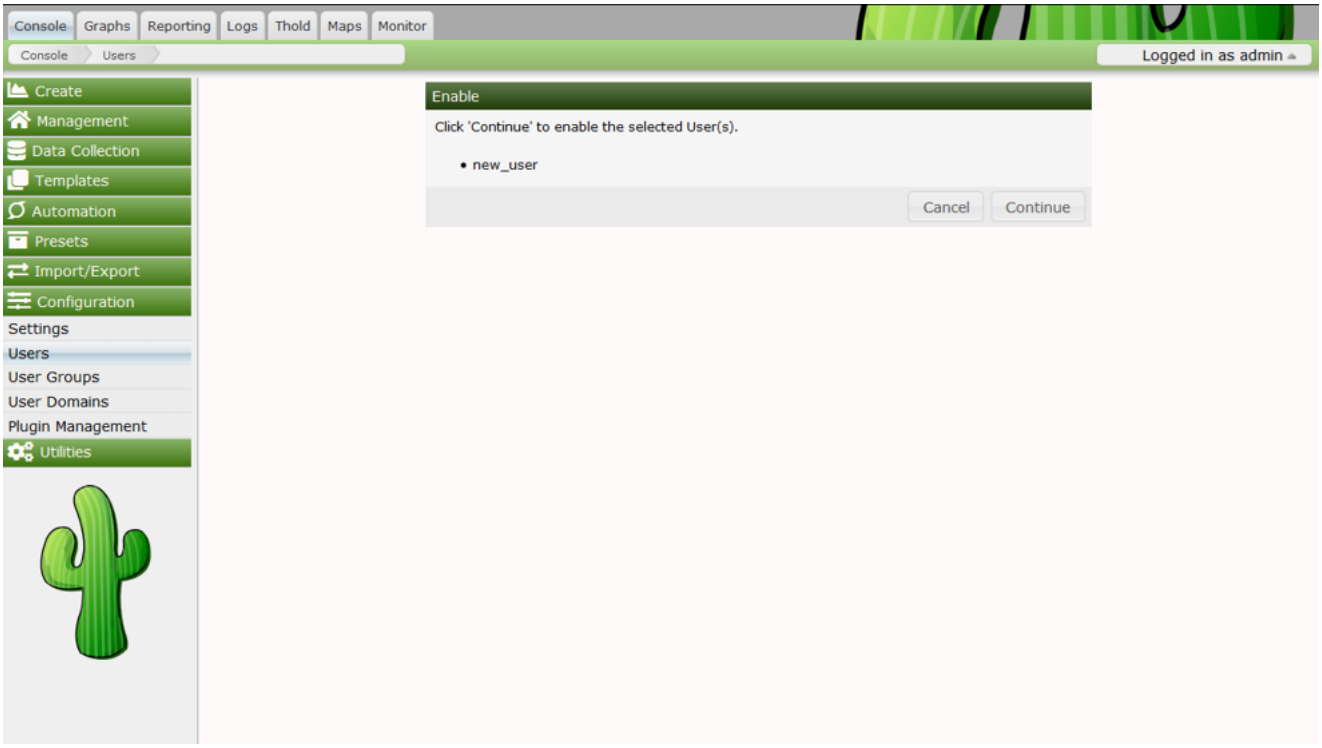
Specify the New Username, New Full Name and New Realm of the new user. Click `Continue` to copy the selected template user to the designated new user. An error will be returned if you attempt to copy over an existing user. If you would like to copy over an existing user, use Batch Copy.

Enable/Disable Users



To enable or disable a set of or a single user, open **Console > Configuration

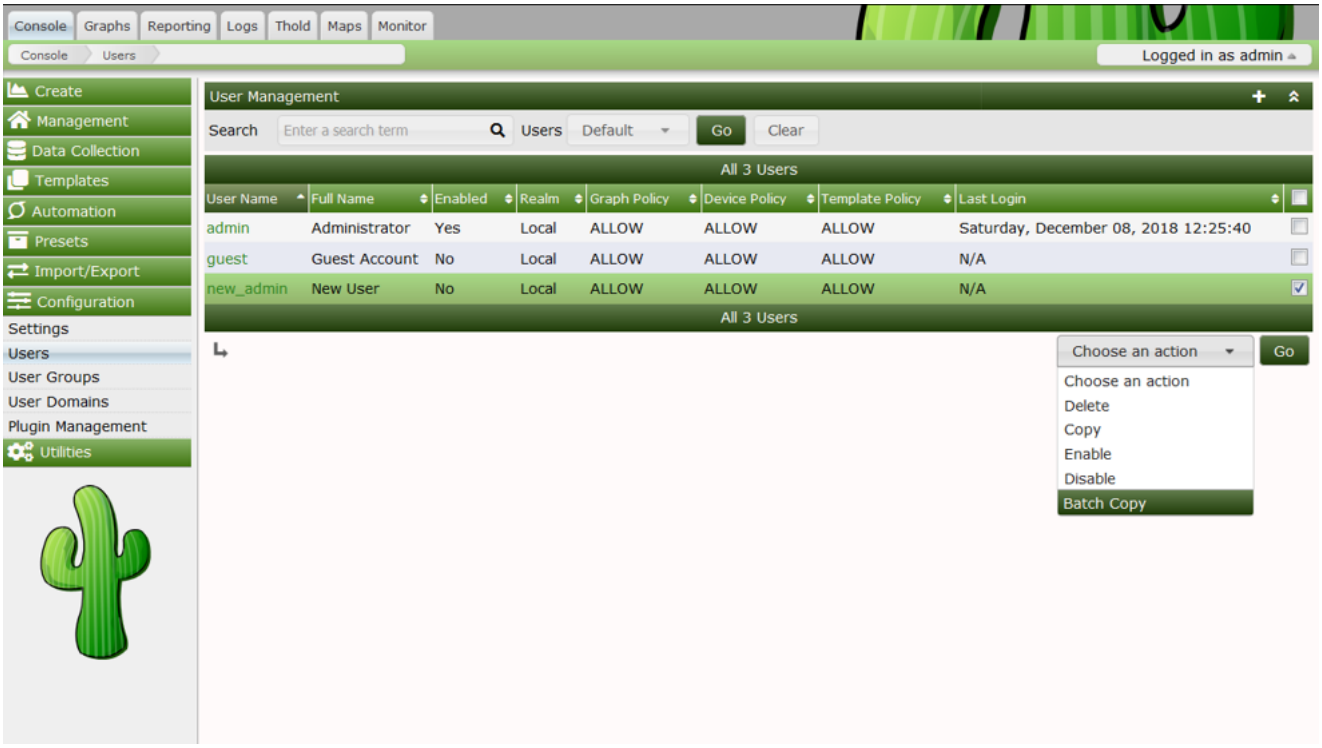
> Users**. Once at the **Users** interface, highlight the **User** you would like to `Enable` or `Disable` and select either `Enable` or `Disable` from the `Choose an action` drop down selection box, click `Go` to continue.



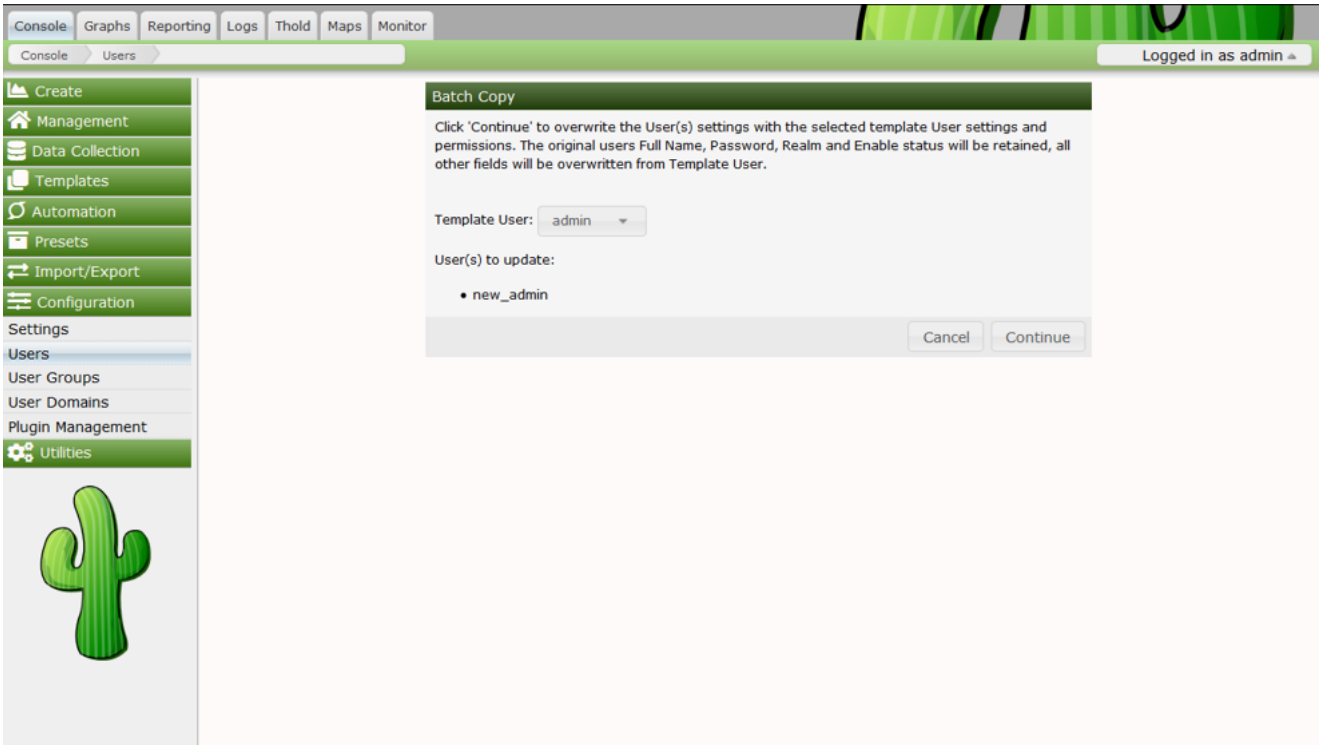
Confirm that you want to disable or enable the listed users. Click `Continue` to perform the action. Disabling your own account is not permitted.

Batch Copy Users

Batch Copy is a helpful utility that helps Cacti Administrators maintain users. It copies a template account's settings to selected users. For ongoing shared permissions, use User Groups.



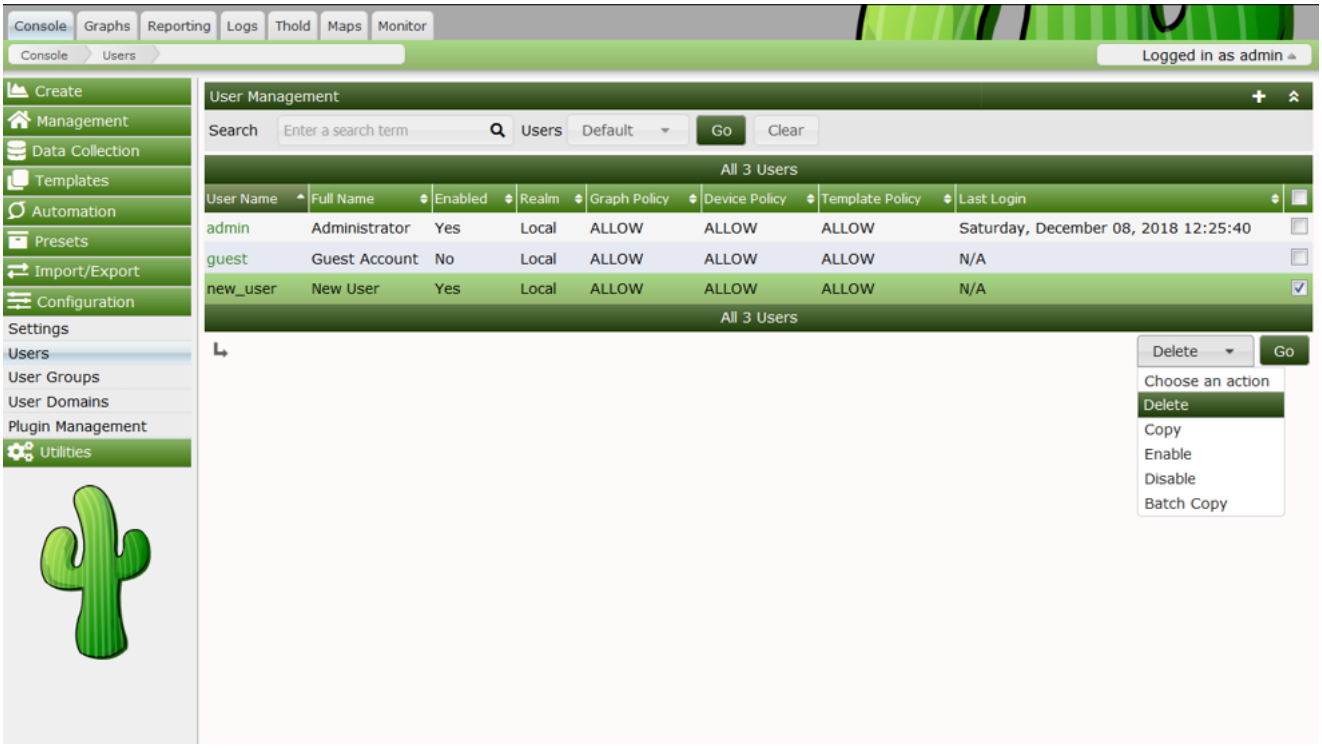
To Batch Copy a set of or a single user, select the **Users** interface under **Console > Configuration**. Once at the **Users** interface, highlight the **Users** you would like to batch copy information to and select `Batch Copy` from the `Choose and action` drop down selection box, click `Go` to continue.



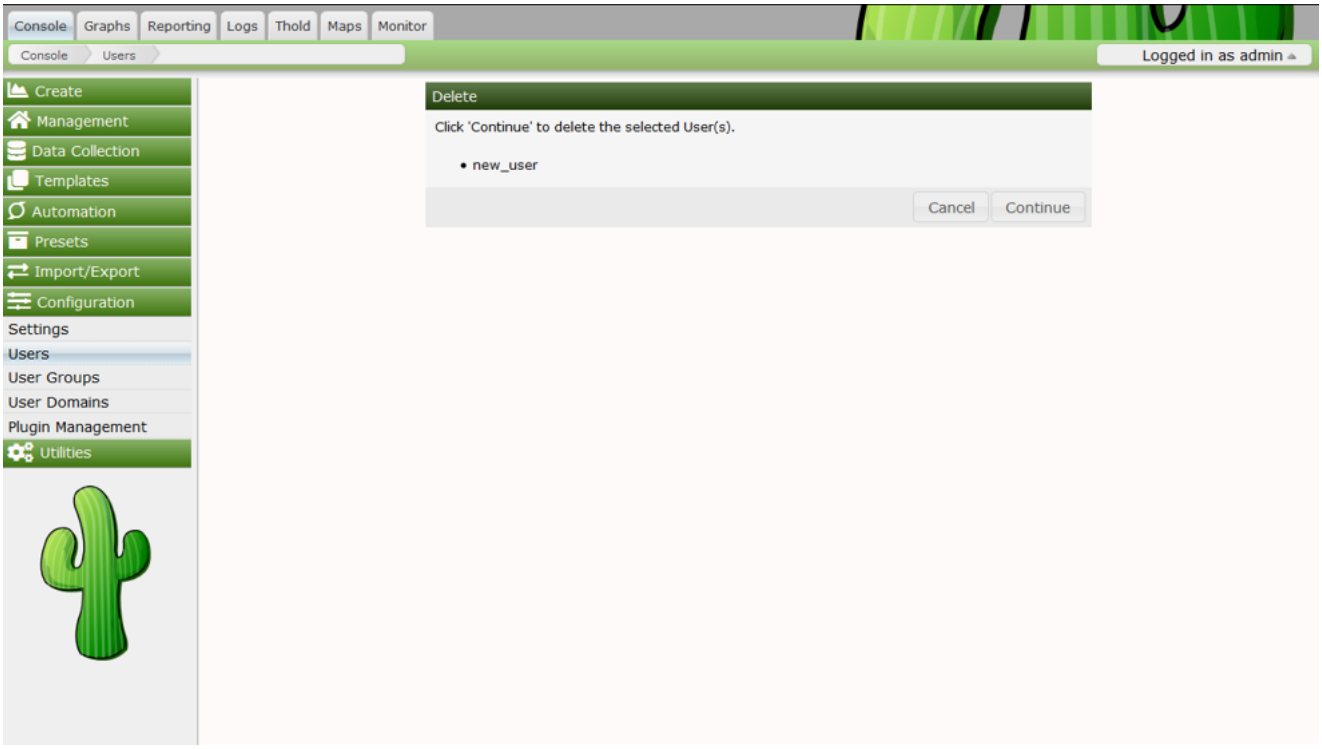
Select the `Template User` and confirm that you want to `Batch Copy` the listed **Users**. Click `Continue` to perform the action.

It is important to note that when using `Batch Copy` the original **Users** `Full Name`, `Password`, `Login Realm` and `Enable` status will be retained, all other fields will be overwritten with values from the selected `Template User`.

Delete Users



To delete users, select their rows under **Console > Configuration > Users**, choose **Delete** from the actions menu, and click **Go**.



Confirm that you want to delete the listed users. Click `Continue` to perform the action. Deleting your own account is not permitted.

Guest (Anonymous) Access

By default, Guest or Anonymous access is disabled in Cacti. Enabling "Guest" access is easy. But, you must understand that you should only allow the designated "Guest" user access to what you authorized. It is not suggested that you give the designated "Guest" user more than just access to graphs. To enable guest access, first, need a designated "Guest" user. Cacti by default comes with one already created that has access to only graphs.

It is important to note that the default guest user has access to all graphs on the system. Once you have created the "Guest" user, you need to tell Cacti what the "Guest" user is. This is done by updating the Authentication Settings and selecting the "Guest" user you have designated. The "Guest" user must be enabled and be set as the guest user for anonymous access to work.

Copyright (c) 2004-2026 The Cacti Group

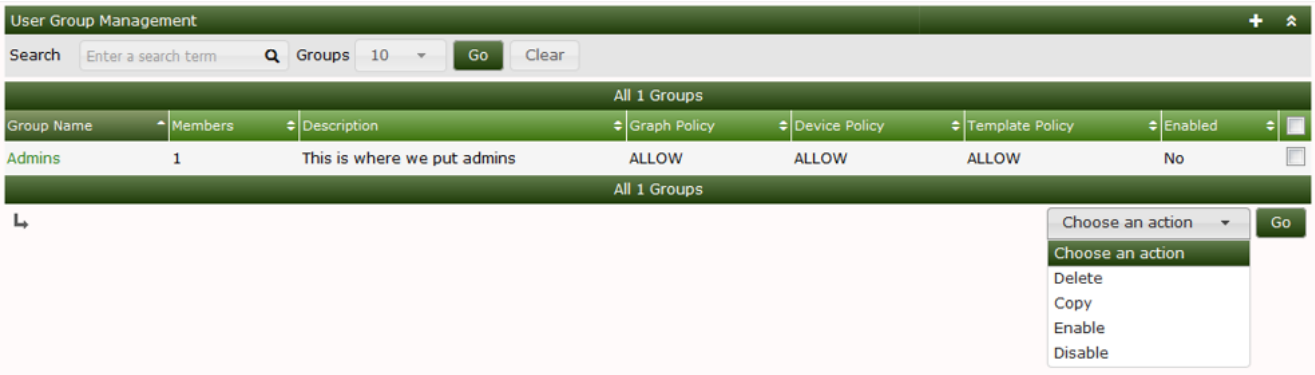
User Group Management

Cacti supports User Groups for shared permissions. On the development branch, Login Providers can also assign membership at login when automatic assignment rules are configured. User Groups closely mimic the normal user. You see things like **Graph** permissions, Roles and Realms, Default pages, and settings, it's all there.

The way that a Cacti User Group works is that it's combined with the Users permission, and then the resulting permission is the most permissive. So, if a User does not have a permission, but the User Group that they belong to does, the user then gains the permission.

There are other types of settings that the User Group can own the permission as well. There are only a few of these permissions however.

In the next series of images, we will walk through the User Group interface. In this first images, we can see the main User Group interface. From here, you can see all of the Groups defined on the system and the number of Users in the Group, if the User Group is enabled or disabled. Lastly, in the Action drop-down, you can see the actions that you can take with a User Group.

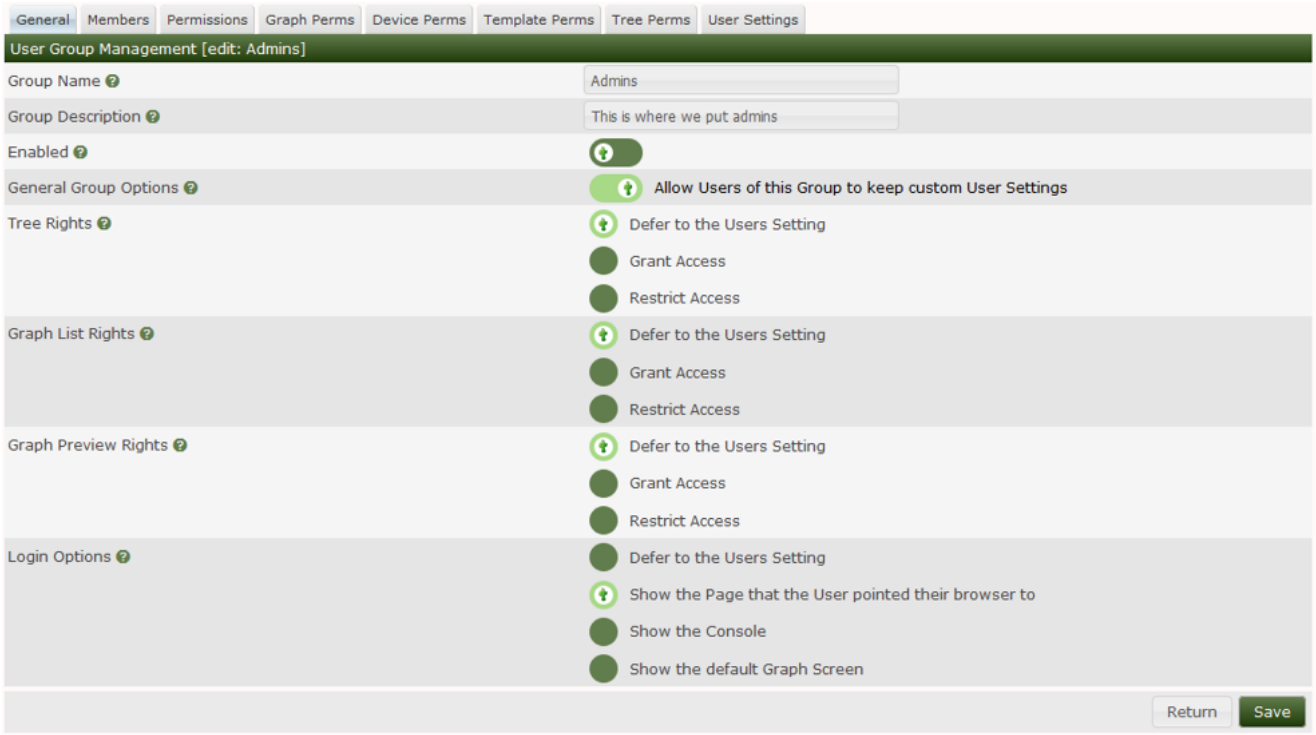


In this next image, you can see the User Group Edit page. The Edit page has multiple Sub-tabs. They include:

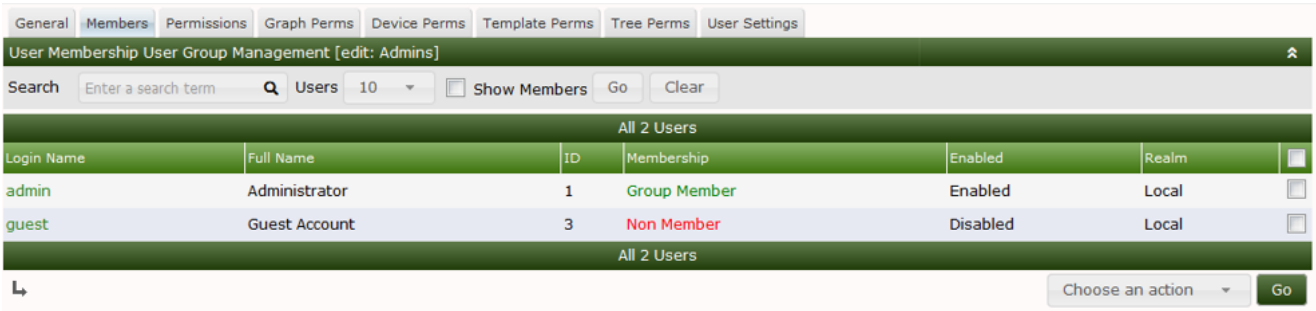
- **General** - General settings about the User Group
- **Members** - A list of User Group members
- **Permissions** - Roles and Realms that provide group member access to Cacti components
- **Graph Perms** - Effective Graph Permissions, rolling up Device and Template permissions
- **Device Perms** - User Group Device Permissions
- **Template Perms** - Graph Template Permissions
- **Tree Perms** - Tree Permissions
- **User Settings** - Settings for the User of the Group if they can not manage settings on their own

The first form elements are pretty self explanatory. The Tree, Graph List, and Preview rights speak to Cacti three Graph Viewing modes which are explained in the Viewing Graphs section of the Document.

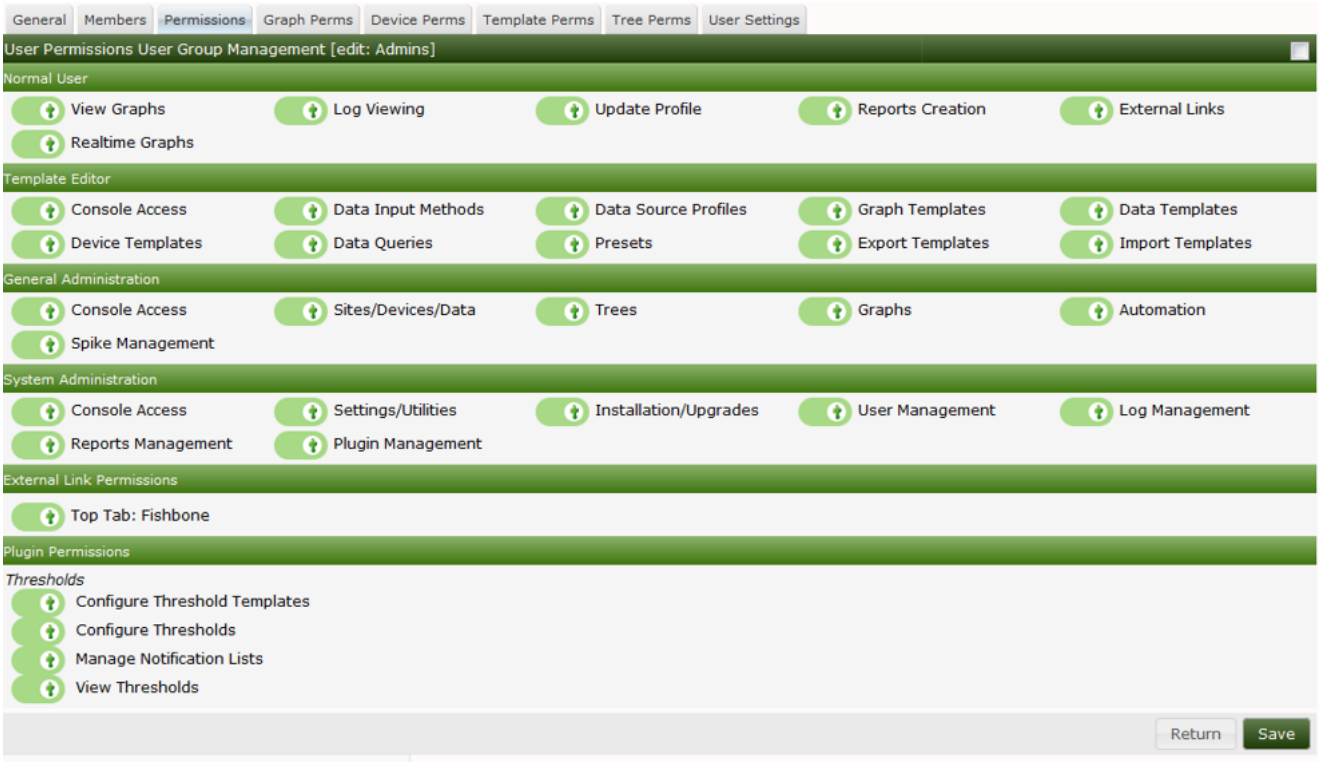
Lastly Login options, are how the user should be re-directed after logging. This is again, pretty self explanatory.



The next image shows the Member list. This one is pretty self explanatory. You can see all the users defined on the system and whether or not they below to the User Group, and the ability to add or remove them enmass.



The next image shows Cacti's Roles and Realms within those Roles. Cacti organizes it's Authorization Realms into Pre-defined Roles. Plugin Developers can create their own Roles as well. So, when you Extend Cacti using Plugins, you may find additional Roles.



In this last image on the page, you can see every **Graph** in the Cacti system and whether or not the User Group has permission to view it. This permission can granted by giving the Group permission to a **Device** or the **Graph Template** in the **Permissive** permission mode, or in the **Restrictive** permission mode, they have to have both **Device** and **Graph Template** permissions to gain access to the **Graph**. Finally, the Cacti Administrator can grant access to **Graphs** one at a time if they choose.

You can find the **Permission** and **Restrictive** permission setting on Cactis configuration page at `Console > Configuration > Settings > General`. We have not covered the **Device**, **Template**, **Tree** or **Settings** sub-tabs here. If you would like more information on them, see the User documentation.

Copyright (c) 2004-2026 The Cacti Group

Login Providers

Login Providers are part of the **unreleased Cacti 1.3 development series**. They replace the older User Domains management interface and support LDAP, Active Directory, SAML2, and OpenID Connect. For Cacti 1.2, see User Domains and Multiple LDAP/AD Domains.

Create a provider

1. Keep a working local administrator account and an authenticated session available while testing.
2. Open **Console > Configuration > Login Providers** and add a provider.
3. Enter a name and choose the **Provider Type**. Select a **User Template** with the permissions intended for newly created accounts.
4. Fill in the connection or identity-provider settings for that type.
5. Enable the provider and save it.
6. Under **Console > Configuration > Settings > Authentication**, select **Login Providers** as the authentication method and save.
7. Test a permitted account and a denied account in a new private browser session. Confirm the resulting Cacti permissions before wider use.

LDAP and Active Directory providers use credentials entered on the login page. SAML2 and OpenID Connect providers use redirect-based login buttons. The **Login Button Label** customizes the button text for those redirect providers. Local accounts remain available through the local login realm.

LDAP and Active Directory

Configure the server, ports, protocol version, encryption, and search or bind settings. Use the LDAP guide for the meaning of DN, search modes, and group membership checks. On the development branch, configure these values on the provider rather than in the older User Domains screen.

Use TLS with a trusted server certificate. If access should be restricted to a directory group, configure and test that membership requirement; merely naming a provider after an administrative group does not restrict access.

For one-time-password authentication, follow the server field's guidance: a space-delimited list can retry a consumed code against another server. Use a single resilient endpoint where directory failover happens below Cacti.

SAML2

Configure the service-provider entity ID and the identity provider's entity ID, single sign-on URL, and signing certificate. Configure signing options and service-provider certificate and private key as required by your identity provider. Exchange the Cacti service-provider metadata with the identity provider and register its assertion-consumer endpoint.

Map the username, full-name, and email attributes to those actually sent by the identity provider. Configure the group attribute and required group when restricting membership. Treat certificates, private keys, and identity claims as configuration that must be verified with the identity-provider administrator.

OpenID Connect

Register Cacti as a client with the identity provider. Enter its client ID, client secret, HTTPS discovery URL, and scopes. The implementation uses the authorization-code flow with PKCE. Discovery and the returned authentication endpoints must use HTTPS.

Set Cacti's **Server Base URL** to its externally reachable URL, including its installation path. The redirect URI is that base URL followed by `/login_sso.php?action=callback&realm=REALM`, where `REALM` is 1000 plus the saved provider ID. Register that exact URI with the identity provider.

Map the username, full-name, and email claims. Configure the group claim and required group if access must be limited to particular users.

Permissions, cookies, and troubleshooting

A successful identity-provider login does not grant unrestricted Cacti access. Review the template user, Cacti user groups, and resulting object permissions. Where automatic group assignment is configured, test both membership and removal cases. SAML2 and OpenID Connect assignment requires the corresponding group attribute or claim.

Persistent login requires HTTPS, the global **Support Authentication Cookies** setting, and the provider's **Allow "Remember Me" Cookies** option. Disable the provider option where every new session must return to the identity provider.

Use provider debug logging temporarily while diagnosing failures, then disable it. Check the Cacti log, callback URL, clock synchronization, certificate trust, claim mappings, and group restrictions. Remove sensitive information before sharing logs.

Copyright (c) 2004-2026 The Cacti Group

User Domain Management

This page describes **User Domains in Cacti 1.2**. The unreleased Cacti 1.3 development branch replaces this screen with Login Providers. User Domains are a core Cacti feature in 1.2, not a separate plugin.

User Domains in Cacti are a way of providing Custom Login experiences to differing groups of users and using LDAP or Active Directory Group Permissions to limit who can login using those customer permissions.

Once a user login in for the first time, their permissions will enherit the Template accounts permission settings including User Group's and their permissions.

So, as the name implies, you can have from one to many LDAP and Active Directory entries in the User Domains interface.

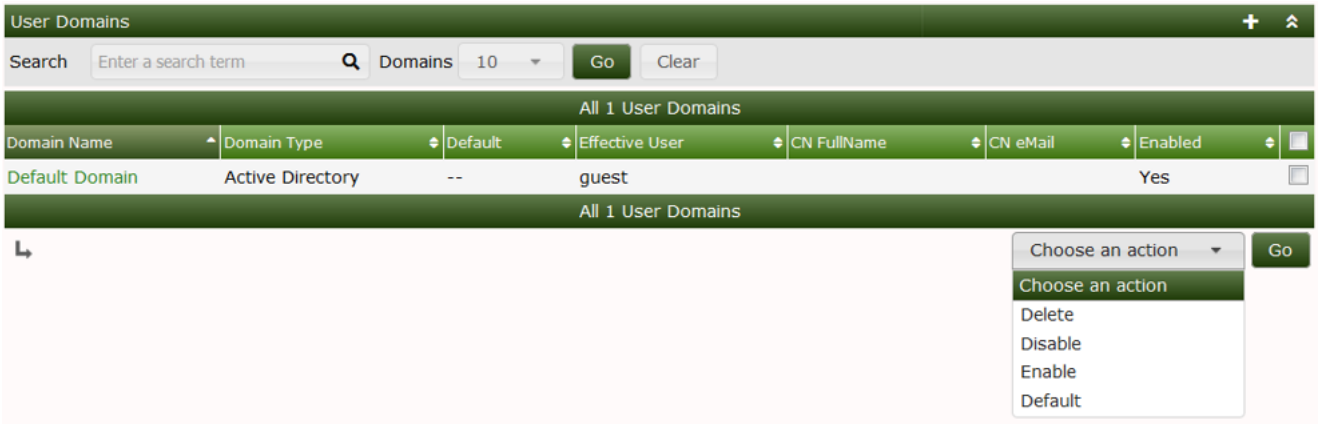
A use example would be for example 3 levels of customers who will have access to the system. For example lets say those three groups are:

- Administrators
- Managers
- Client Team

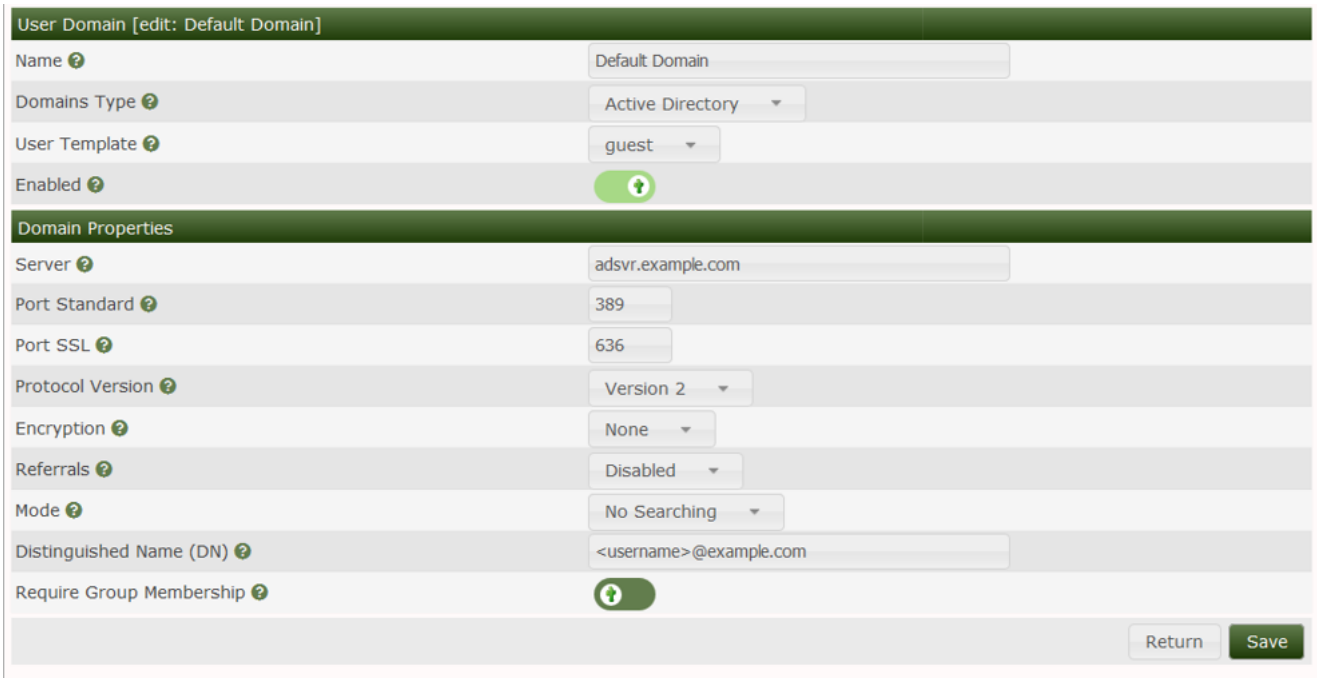
Each of these users groups are controlled by Active Directory there are AD User Groups that map one-to-one with these Groups. When you have these three Domains setup and enabled, when you goto the Cacti Login screen there will be a drop-down settings with the three Domains listed in the drop-down.

If a normal user tries logging as an Administrator, their login will be blocked and so on.

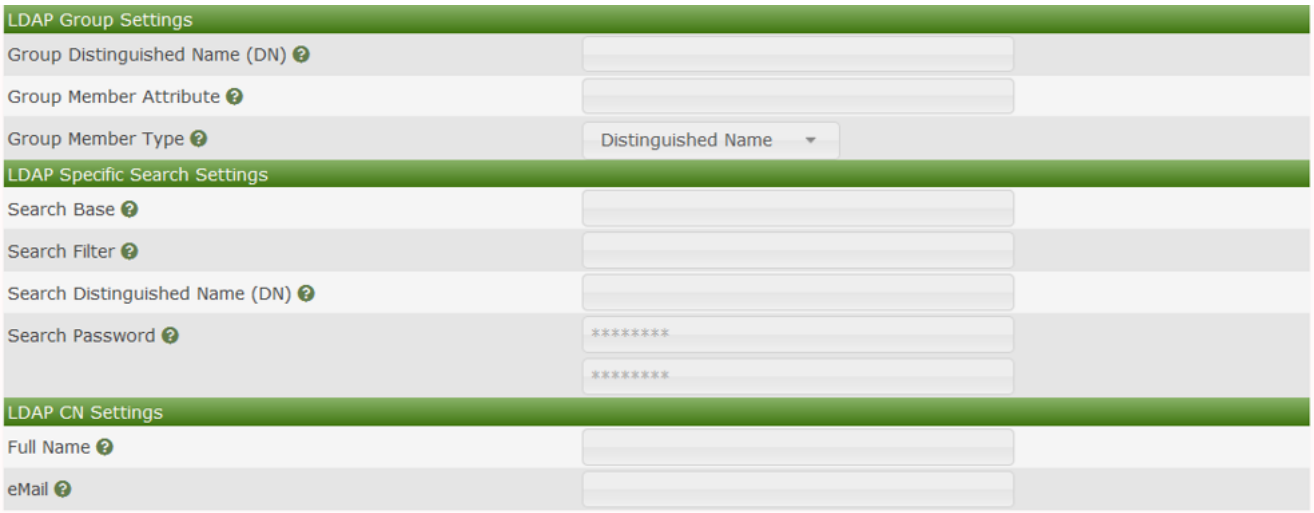
The first image shows the list of domains, which domain is primary, and the actions you can take on a Domain.



In this second screen, you can see the User Domain login screen where you can pick AD or LDAP, the credentials, importantly the Template User Account (That which holds base permissions), and the require group membership, which when you check it, you can then specify the AD and LDAP User Group paths.



In the last images, you can see the Group Settings information, and if you have the users Full Name and Email stored in LDAP or AD, you can pull that information and make it a part of the Cacti wrapper account when it's first created.



Copyright (c) 2004-2026 The Cacti Group

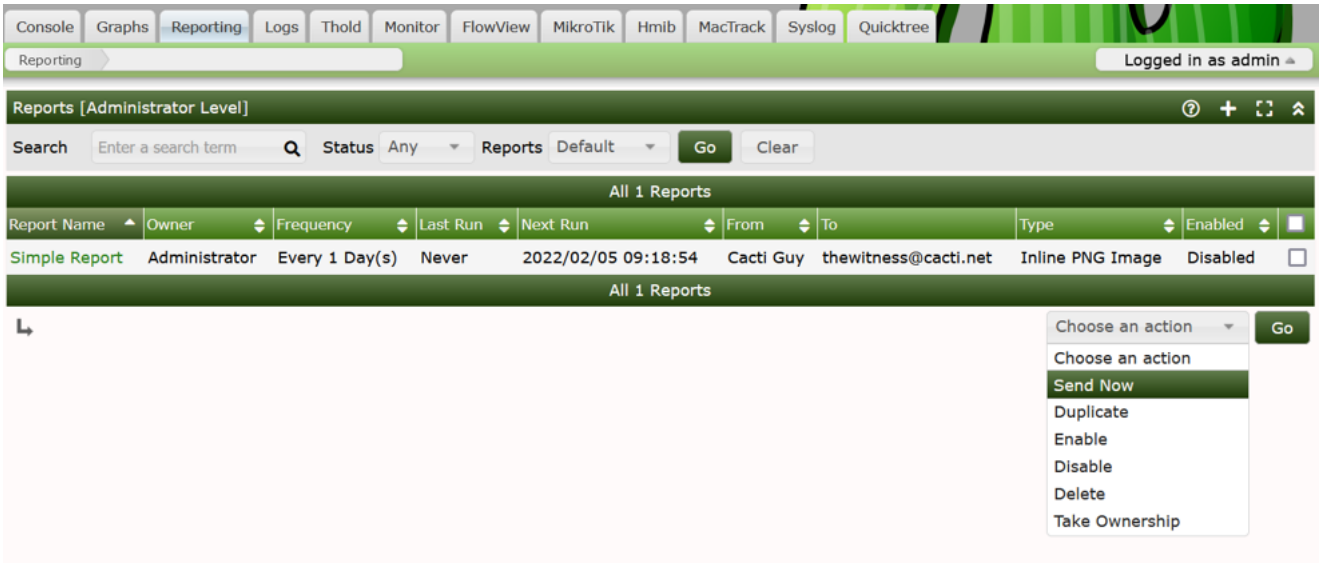
Reports Administration

This section will describe **Reports Administration** in Cacti.

The **Reports Administration** tab allows Cacti Administrators to administer all **Reports** in Cacti. The Administrator can change the ownership of or take ownership away from any user on the system, and can edit all **Reports**. The primary difference between the **Reports Administration** and the **Reports User** interfaces breaks down to simply what can be done and seen by the user.

The Web Interface

The administration of any **Report** begins on the main Reports tab in Cacti. A screen shot of the Reports tab is shown below.



When viewing any **Reports** from this interface, you will see the following columns:

- **Report Name** - The name of the Report.
- **Owner** - The Cacti account that has ownership of the Report.
- **Frequency** - How often the Report is to be sent to the distribution.
- **Last Run** - The last time that the Report was sent to the distribution.
- **Next Run** - The approximate next time that the report is to be sent. this is not an exact number. It does depend on the poller first running. So, depending on how long your poller runs for, this time could be off by a few minutes.
- **From** - The From Name portion of the From Email address
- **To** - The list of Emails that the reports is to be sent to.
- **Type** - The Image format and the type of embedding done in the Email.
- **Enabled** - Is the Report able to be sent on the Schedule.

The actions that you can take on an existing Report as an Administrator include:

- **Send Now** - Force send one or more **Reports** now
- **Duplicate** - Duplicate one or more **Reports**
- **Enable** - Enable one or more **Reports**
- **Disable** - Disable one or more **Reports**
- **Delete** - Delete one or more **Reports**
- **Take Ownership** - Take over ownership of one or more **Reports**

Report Editing

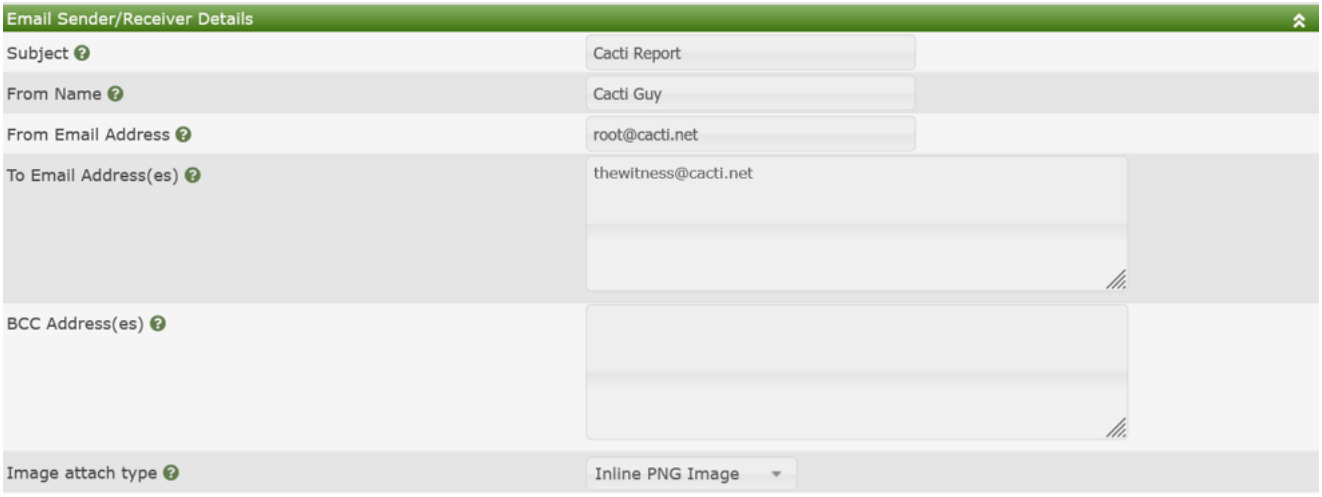
When you click the Add button on the right side of the **Reports** page, or Click the Report Name of an existing Report, you will be presented with the interface below.

This section includes:

- **Report Name** - The Name of the **Report**
- **Enable Report** - Is the **Report** Enabled
- **Use Custom Format HTML** - This is an indication that you wish to use a custom html/css document that you can use to customize the layout of your reports and add corporate branding for example.
- **Format File to Use** - The specific html/css file to use.
- **Graphs Linked** - Provide the recipient of the report a ling back to the actual graph in Cacti.
- **Graph Columns** - The number of Graph columns to use per row.
- **Graph Width** - The width of each Graph
- **Graph Height** - The height of each Graph
- **Thumbnails** - Should the Graphs be rendered in Thumbnail mode
- **Next Timestamp of Sending Mail Report** - The approximate next time to send the report
- **Report Interval** - The frequency to send the report. Options are every X Minute, Hours, Days, Weeks, Months on a Day of the Month, Months on the Day of the Week, and Years.
- **Interval Frequency** - How often to send, for example if you select Days, and the Interval is one, it means every day, if you select 2, it means every other day.

The **Email Frequency** section does not use the same layout as does our **Automation Networks**. Correcting that is a future project for the team.

The last section of the **Report Details** is the **Email Sender/Receiver** section as shown below.



From here you can set the following columns.

- **Subject** - The Email subject.
- **From Name** - The From Name on the Email. For example **Cacti Administrator**
- **From Email Address** - The From Email address. For example *noreply@mydomain.com*
- **To Email Address(es)** - A comma delimited list of Email addresses that should receive the Report.
- **BCC Address(es)** - An anonymized list of Email recipients
- **Image attach type** - This defines how the Cacti Graph images get attached to the Email. Maybe are for legacy purposes. Most users will use Inline PNG or Inline JPG.

Once you save a new **Report** additional Tabs will show up. Those tabs are:

- **Items** - The Graph's, Tree's, Text, Horizontal Rules, and Device's to place on the Report.
- **Preview** - See a Preview of your Report
- **Events** - Approximate Future Event Days for your Report to be sent.
  This helps interpret the idiosyncrasies of the scheduler design.

Copyright (c) 2004-2026 The Cacti Group

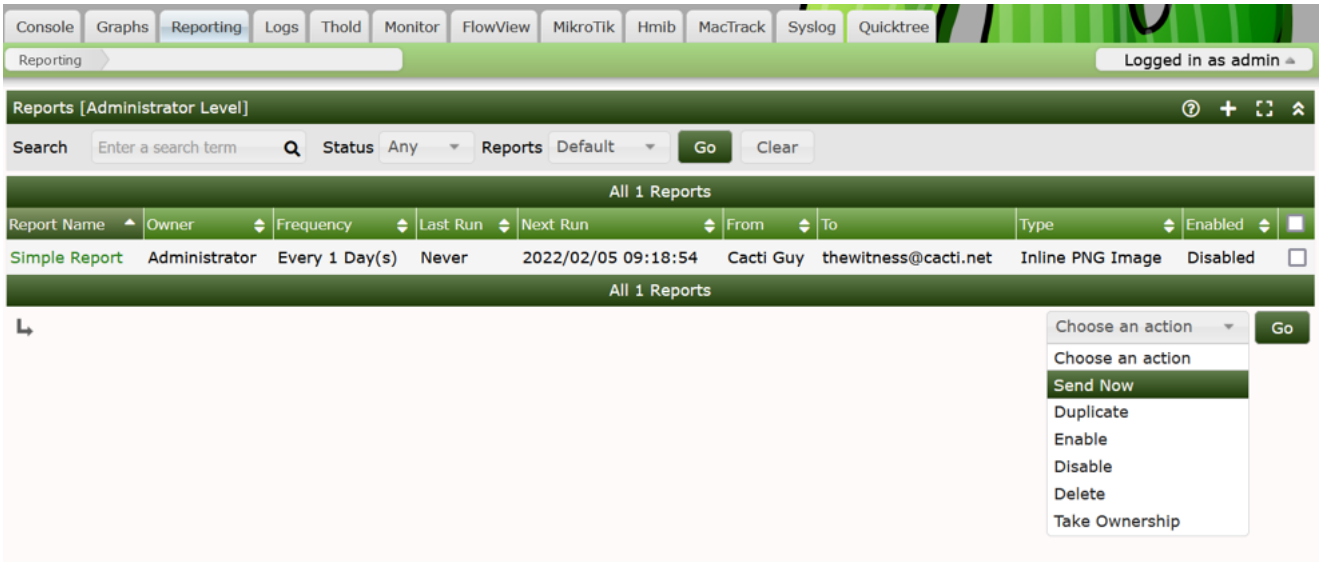
Reports User

This section will describe the user view to Cacti's **Reports** feature.

The **Reports User** interface allows you to see and manage your own reports. This is unlike the **Reports Administration** interface that allows you to manage others reports in addition to your own.

The Web Interface

The user of any **Report** begins on the main Reports tab in Cacti. A screen shot of the Reports tab is shown below.



When viewing any **Reports** from this interface, you will see the following columns:

- **Report Name** - The name of the Report.
- **Owner** - The Cacti account that has ownership of the Report.
- **Frequency** - How often the Report is to be sent to the distribution.
- **Last Run** - The last time that the Report was sent to the distribution.
- **Next Run** - The approximate next time that the report is to be sent. this is not an exact number. It does depend on the poller first running. So, depending on how long your poller runs for, this time could be off by a few minutes.
- **From** - The From Name portion of the From Email address
- **To** - The list of Emails that the reports is to be sent to.
- **Type** - The Image format and the type of embedding done in the Email.
- **Enabled** - Is the Report able to be sent on the Schedule.

The actions that you can take on an existing Report as an Administrator include:

- **Send Now** - Force send one or more **Reports** now
- **Duplicate** - Duplicate one or more **Reports**
- **Enable** - Enable one or more **Reports**
- **Disable** - Disable one or more **Reports**
- **Delete** - Delete one or more **Reports**

Report Editing

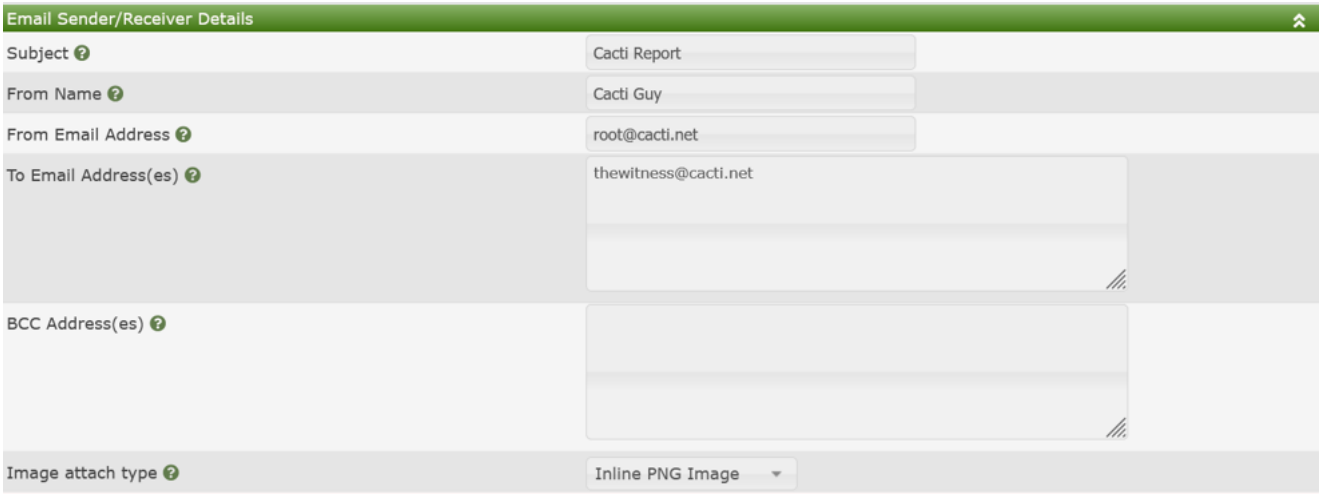
When you click the Add button on the right side of the **Reports** page, or Click on the Report Name of an existing Report, you will be presented with the interface below.

This section includes:

- **Report Name** - The Name of the **Report**
- **Enable Report** - Is the **Report** Enabled
- **Use Custom Format HTML** - This is an indication that you wish to use a custom html/css document that you can use to customize the layout of your reports and add corporate branding for example.
- **Format File to Use** - The specific html/css file to use.
- **Graphs Linked** - Provide the recipient of the report a ling back to the actual graph in Cacti.
- **Graph Columns** - The number of Graph columns to use per row.
- **Graph Width** - The width of each Graph
- **Graph Height** - The height of each Graph
- **Thumbnails** - Should the Graphs be rendered in Thumbnail mode
- **Next Timestamp of Sending Mail Report** - The approximate next time to send out the **Report**.
- **Report Interval** - The frequency to send the report. Options are every X Minute, Hours, Days, Weeks, Months on a Day of the Month, Months on the Day of the Week, and Years.
- **Interval Frequency** - How often to send, for example if you select Days, and the Interval is one, it means every day, if you select 2, it means every other day.

The **Email Frequency** section does not use the same layout as does our **Automation Networks**. Correcting that is a future project for the team.

The last section of the **Report Details** is the **Email Sender/Receiver** section as shown below.



From here you can set the following columns.

- **Subject** - The Email subject.
- **From Name** - The From Name on the Email. For example **Cacti Administrator**
- **From Email Address** - The From Email address. For example `noreply@mydomain.com`
- **To Email Address(es)** - A comma delimited list of Email addresses that should receive the Report.
- **BCC Address(es)** - An anonymized list of Email recipients
- **Image attach type** - This defines how the Cacti Graph images get attached to the Email. Maybe are for legacy purposes. Most users will use Inline PNG or Inline JPG.

Once you save a new **Report** additional Tabs will show up. Those tabs are:

- **Items** - The Graph's, Tree's, Text, Horizontal Rules, and Device's to place on the Report.
- **Preview** - See a Preview of your Report
- **Events** - Approximate Future Event Days for your Report to be sent. This helps interpret the idiosyncrasies of the scheduler design.

Copyright (c) 2004-2026 The Cacti Group

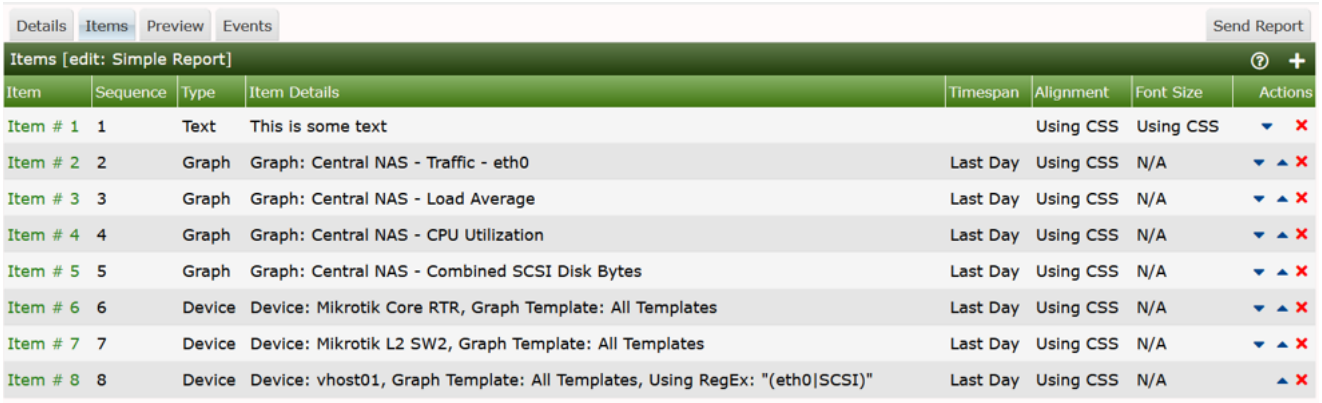
Report Items

This section will describe the **Report Edit Items** Sub-Tab.

From this Sub-Tab, you can add, remove, and if **Drag and Drop** is enabled, easily reorder any **Report Item**. **Report Items** include:

- **Trees** - Including one or more Branches, which include Devices as a Branch Type.
- **Graphs** - One or more Graphs from a Device.
- **Devices** - One or more Cacti Devices with Graphs whose titles may include a Regular Expression.
- **Text** - For example to sub-section a Report and add a Title
- **Horizontal Rule** - For separating sections of a **Report**.

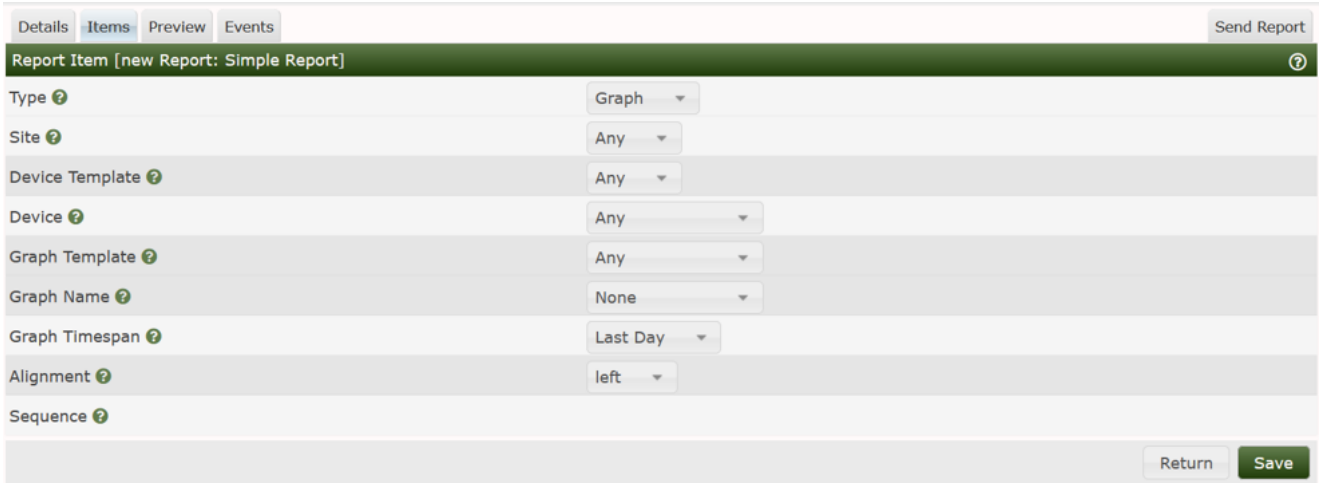
An example **Report Items** interface can be seen below.



From this image, you can see a combination of **Graphs**, **Devices**, and a **Device** showing only **Graphs** that include *eth0*, and *SCSI* in their titles indicating something about Interface Graphs and Disk Utilization Graphs.

Item Edit Page

When you either add a new Item by pressing the Add button on the right, Clicking on the Item # on the left, you will be presented a page that is personalized depending on the *Item Type*. When you add an item for the first time, you will see the image below.



Again, the user interface will change layout depending on the *Item Type* that you select which happens to be the first drop-down on that page.

The columns for a **Graph** Type **Report Item** include:

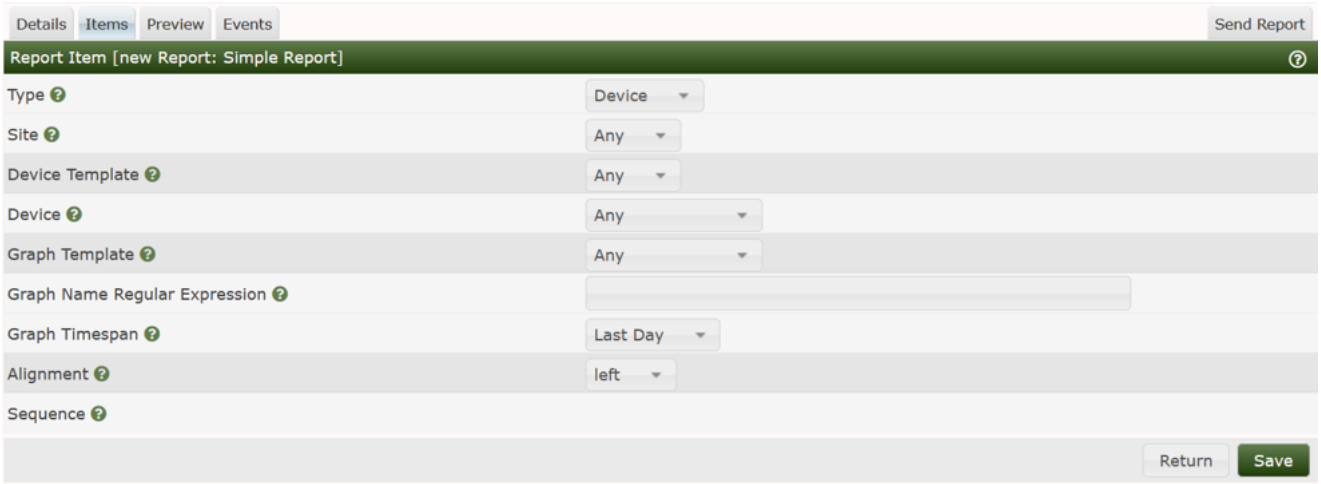
- **Site** - A filter that will show only Devices and Graphs from a specific site.
- **Device Template** - A filter that will only show Devices and Graphs from a specific Device Template
- **Device** - A filter that will show only Graphs from a specific Device.
- **Graph Template** - A filter that will show only Graphs from a specific Graph Template.

The options above are only Filter to reduce the Graph list when you actually select the **Graph Name**, it will apply those filters to narrow in on the Graphs to display in the drop-down. All of the filters above are *optional*.

The last sections are for the specific Graph in question including:

- **Graph Name** - The name of the Graph to include.
- **Graph Timespan** - The time range to show for the Graph.
- **Alignment** - Should the Graph be Center, Left or Right Aligned on the page.
- **Sequence** - The Graph Items Sequence number. This is a Read Only variable.

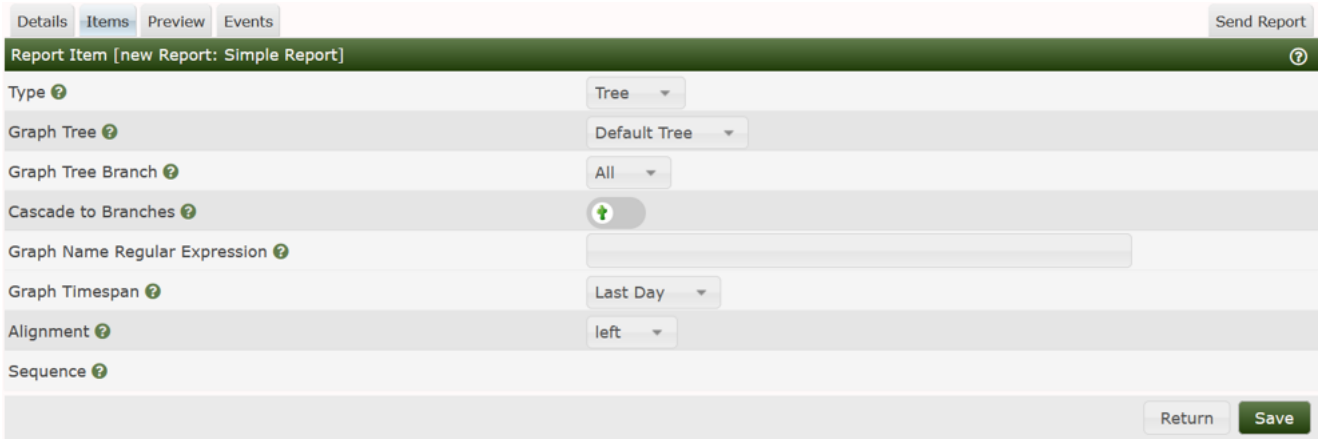
When, you change the **Report Item** to **Device** the layout of the page will change and you will see the form shown below.



In this case, the **Site**, and **Device Template** both remain Filters, the **Device** represents the **Device** to include on the **Report**. After that, the options below define what Graphs from the Device are to be included.

- **Graph Template** - Either Any Graph Template, or a Specific Graph Template.
- **Graph Name Regular Expression** - A regular expression that limits the **Graphs** to only those matching the Regular Expression.
- **Graph Timespan** - The time range to show for the Graph.
- **Alignment** - Should the Graph be Center, Left or Right Aligned on the page.
- **Sequence** - The Graph Items Sequence number. This is a Read Only variable.

The last **Report Item** that we will document here is the **Tree** type. The Text and Horizontal Rule types are pretty much self explanatory. The image below is the layout that will be shown when you change the **Report Item** to **Tree**.



In this case, there are no filter, just selections. They include:

- **Graph Tree** - The name of the Cacti Tree to include in the report.
- **Graph Tree Branch** - The specific Branch to include

if you are only wanting to include aportion of the **Tree**

- **Cascade to Branches** - Walk all the **Graph Trees** branches
- **Graph Name Regular Expression** - A MySQL/MariaDB compatible Regular

Expression of Graph Names to include in the Report.

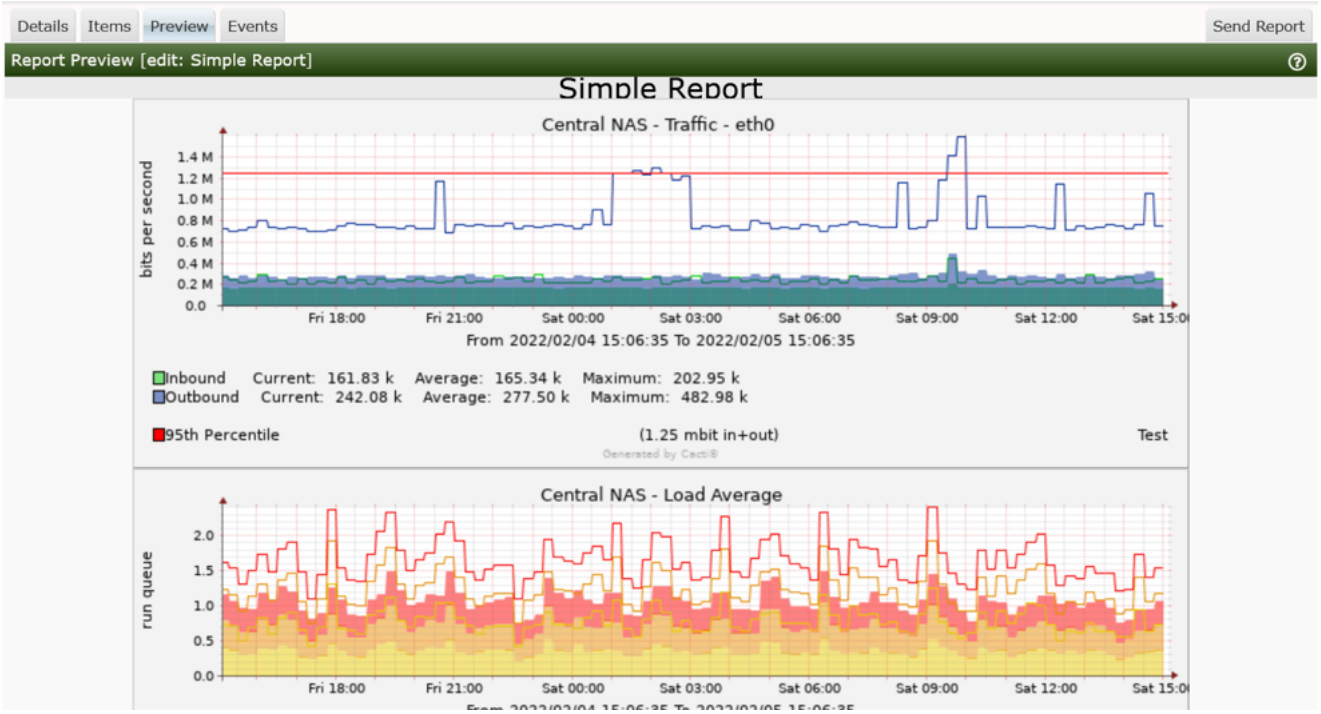
- **Graph Timespan** - The time range to show for the Graph.
- **Alignment** - Should the Graph be Center, Left or Right Aligned on the page.
- **Sequence** - The Graph Items Sequence number. This is a Read Only variable.

Copyright (c) 2004-2026 The Cacti Group

Report Preview

This section will describe the **Report Preview** Sub-Tab.

From this Sub-Tab, you will be able to preview the **Report** leveraging the **Format File** if you included one on the **Details** sub-tab. The rendering may actually be different depending on the Email client your recipient uses. So, it's best to send the reports to the recipients using the **Send Report** option in the upper right corner of the image below.



There is not too much more to speak about on this Sub-Tab.

Copyright (c) 2004-2026 The Cacti Group

Report Events

On this page, you can simply see on what dates and approximate times that the **Reports** scheduler will send future report. You can see from the image below this the report will be sent every day at approximately 09:18:30. The poller frequency or Data Collector interval is 30 seconds on this system.

Currently Cacti does not record past events in a Database table or store the published report in either PDF or other document format in a Database for historical reference.

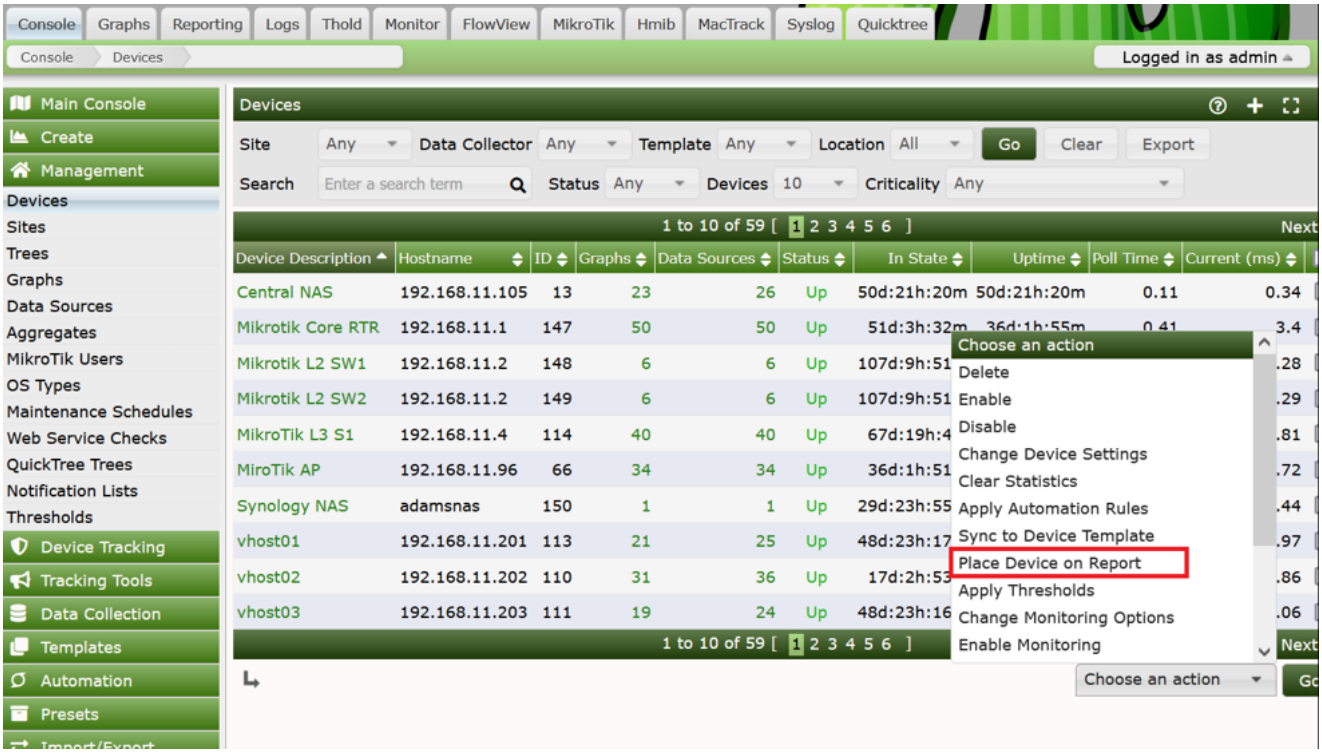
Copyright (c) 2004-2026 The Cacti Group

Other Report Management Options

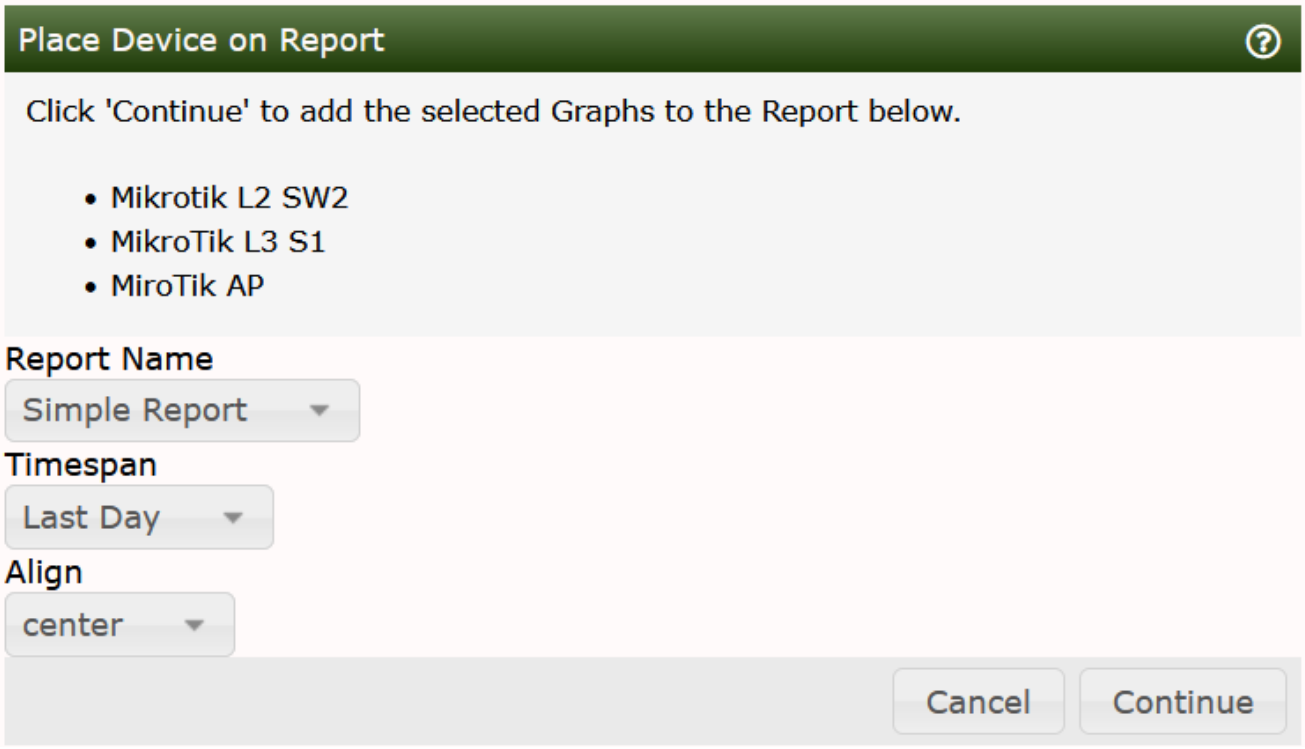
Not only can you add and remove **Report Items** from the **Reports** interface, but you can also add **Graphs**, **Devices**, and **Trees** to a **Report** from the Cacti Console under **Graphs**, **Devices** and **Trees** pages.

For each Management section, you will find a drop-down that looks similar to the Drop down below. Note that from this interface, you will not be able to add any object type more than once. If you need to add a **Device** more than once to more precisely choose filtering attributes, you will have to perform that from the Reports interface itself.

We have future plans to add an icon to the right of the Graph on the Graph Tree and Preview pages.



Regardless of the page type, when you select this option, upon clicking the *Go* button you will be presented with the following page.



From there, select your destination Report, Timespan, and Alignment.

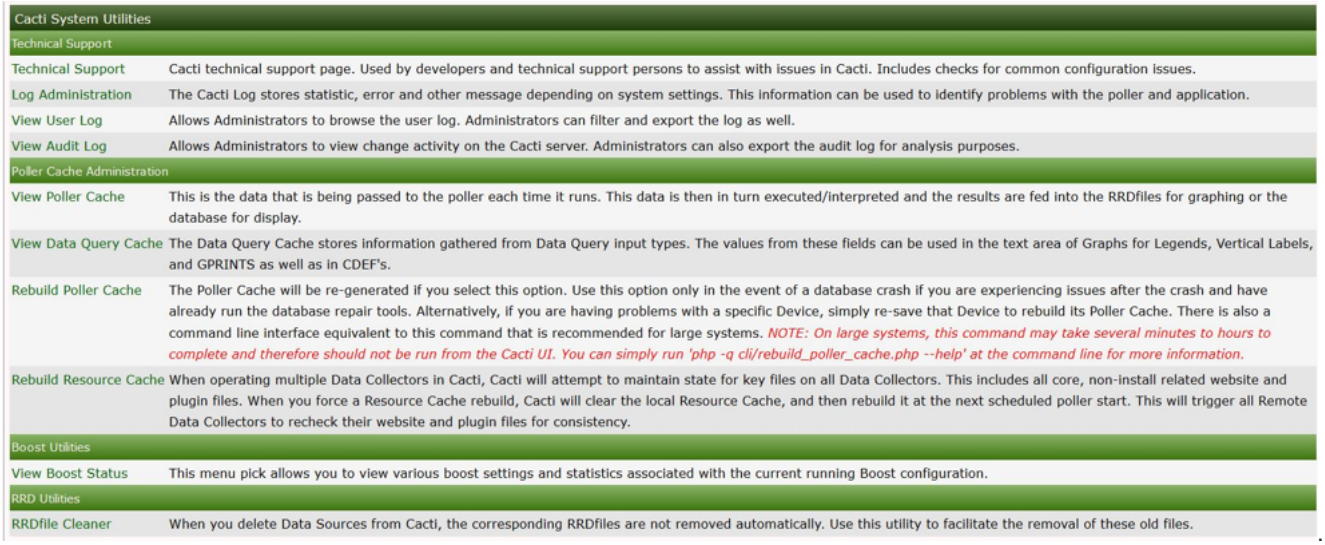
Copyright (c) 2004-2026 The Cacti Group

System Utilities

This section will describe **System Utilities** in Cacti.

The system utilities page in cacti contains tools used for administering cacti from this page you can do the following

- Rebuild poller cache.
- View technical support information.
- Adjust log settings.
- View the user log for audits.



Cacti System Utilities	
Technical Support	
Technical Support	Cacti technical support page. Used by developers and technical support persons to assist with issues in Cacti. Includes checks for common configuration issues.
Log Administration	The Cacti Log stores statistic, error and other message depending on system settings. This information can be used to identify problems with the poller and application.
View User Log	Allows Administrators to browse the user log. Administrators can filter and export the log as well.
View Audit Log	Allows Administrators to view change activity on the Cacti server. Administrators can also export the audit log for analysis purposes.
Poller Cache Administration	
View Poller Cache	This is the data that is being passed to the poller each time it runs. This data is then in turn executed/interpreted and the results are fed into the RRDfiles for graphing or the database for display.
View Data Query Cache	The Data Query Cache stores information gathered from Data Query input types. The values from these fields can be used in the text area of Graphs for Legends, Vertical Labels, and GPRINTS as well as in CDEF's.
Rebuild Poller Cache	The Poller Cache will be re-generated if you select this option. Use this option only in the event of a database crash if you are experiencing issues after the crash and have already run the database repair tools. Alternatively, if you are having problems with a specific Device, simply re-save that Device to rebuild its Poller Cache. There is also a command line interface equivalent to this command that is recommended for large systems. *NOTE: On large systems, this command may take several minutes to hours to complete and therefore should not be run from the Cacti UI. You can simply run 'php -q cli/rebuild_poller_cache.php --help' at the command line for more information.*
Rebuild Resource Cache	When operating multiple Data Collectors in Cacti, Cacti will attempt to maintain state for key files on all Data Collectors. This includes all core, non-install related website and plugin files. When you force a Resource Cache rebuild, Cacti will clear the local Resource Cache, and then rebuild it at the next scheduled poller start. This will trigger all Remote Data Collectors to recheck their website and plugin files for consistency.
Boost Utilities	
View Boost Status	This menu pick allows you to view various boost settings and statistics associated with the current running Boost configuration.
RRD Utilities	
RRDfile Cleaner	When you delete Data Sources from Cacti, the corresponding RRDfiles are not removed automatically. Use this utility to facilitate the removal of these old files.

Some of the most important tools are listed below

Rebuild poller cache

The Poller Cache will be re-generated if you select this option. Use this option only in the event of a database crash if you are experiencing issues after the crash and have already run the database repair tools. Alternatively, if you are having problems with a specific Device, simply re-save that Device to rebuild its Poller Cache. There is also a command line interface equivalent to this command that is recommended for large systems. NOTE: On large systems, this command may take several minutes to hours to complete and therefore should not be run from the Cacti UI. You can simply run 'php -q cli/rebuild_poller_cache.php --help' at the command line for more information.

Technical support

Cacti technical support page. Used by developers and technical support persons to assist with issues in Cacti. Includes checks for common configuration issues. There is also built-in checks that help you tune your Cacti installation.

For example in the technical support section under the summary, you will see a calculation for how many MySQL connections. You should have (Cacti 1.2.16+) you will also show poller information such as the last polling time process count and more detailed info that can be used for a quick view of how the system is configured.

Poller Information	
Interval	60
Type	SPINE 1.2.16 Copyright 2004-2020 by The Cacti Group
Items	Action[0]: 64 Action[1]: 8 Action[2]: 23 Total: 95
Concurrent Processes	Name: Main Poller, Procs: 1
Max Threads	Name: Main Poller, Threads: 1
PHP Servers	1
Minimum Connections:	Current: 151, Min Required: 102 Assumes 100 spare connections for Web page users and other various connections. The minimum required can vary greatly if there is heavy user Graph viewing activity. Each browser tab can use upto 10 connections depending on the browser.
Script Timeout	25
Max OID	10
Last Run Statistics	Time:1.3627 Method:spine Processes:1 Threads:1 Hosts:7 HostsPerProcess:7 DataSources:95 RRDsProcessed:54

You will also see all of the system variables such as allocated php memory as well as mysql variables.

MariaDB Tuning (/etc/my.cnf) - [Documentation] Note: Many changes below require a database restart

Variable	Current Value		Recommended Value	Comments
version	10.3.27-MariaDB-0+deb10u1-log	>=	5.6	MySQL 5.6+ and MariaDB 10.0+ are great releases, and are very good versions to choose. Make sure you run the very latest release though which fixes a long standing low level networking issue that was causing spine many issues with reliability.
collation_server	utf8mb4_general_ci	=	utf8mb4_unicode_ci	When using Cacti with languages other than English, it is important to use the utf8mb4_unicode_ci collation type as some characters take more than a single byte.
character_set_server	utf8mb4	=	utf8mb4	When using Cacti with languages other than English, it is important to use the utf8mb4 character set as some characters take more than a single byte.
character_set_client	utf8mb4	=	utf8mb4	When using Cacti with languages other than English, it is important to use the utf8mb4 character set as some characters take more than a single byte.
max_connections	151	>=	100	Depending on the number of logins and use of spine data collector, MariaDB will need many connections. The calculation for spine is: total_connections = total_processes * (total_threads + script_servers + 1), then you must leave headroom for user connections, which will change depending on the number of concurrent login accounts.
max_allowed_packet	1599488	>=	16777216	With Remote polling capabilities, large amounts of data will be synced from the main server to the remote pollers. Therefore, keep this value at or above 16M.
max_heap_table_size	70 M	>=	15.43 M	If using the Cacti Performance Booster and choosing a memory storage engine, you have to be careful to flush your Performance Booster buffer before the system runs out of memory table space. This is done two ways, first reducing the size of your output column to just the right size. This column is in the tables poller_output, and poller_output_boost. The second thing you can do is allocate more memory to memory tables. We have arbitrarily chosen a recommended value of 10% of system memory, but if you are using SSD disk drives, or have a smaller system, you may ignore this recommendation or choose a different storage engine. You may see the expected consumption of the

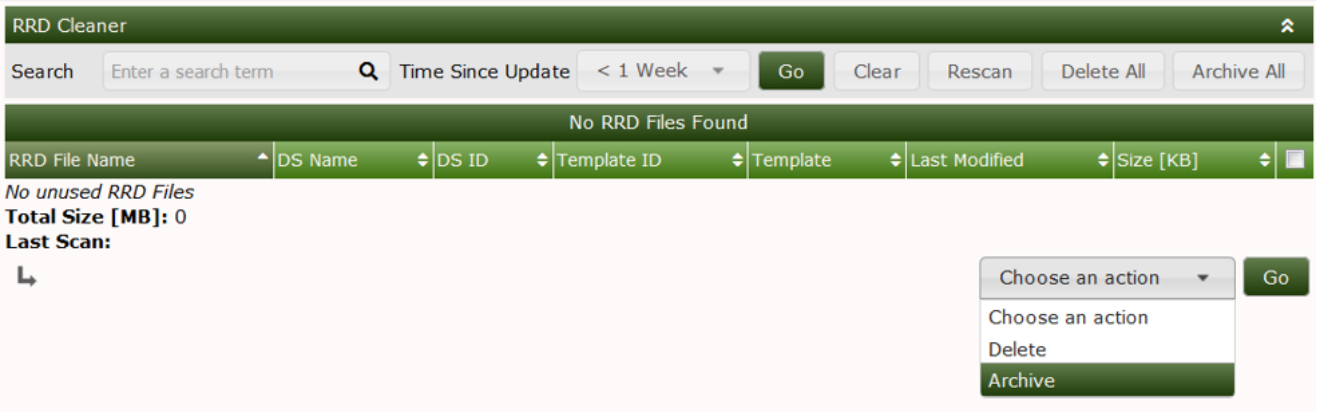
Any Variable appearing in red indicates a subpar setting and should be looked. the recommendations are dynamic in that they change when more system resources are detected so if you upgrade your server hardware be sure to check back to this page to get the most out of your new hardware.

RRD cleaner

`Console > utilities > System Utilities > RRDfile Cleaner`

When you delete Data Sources from Cacti, the corresponding RRDfiles are not removed automatically. Use this utility to facilitate the removal of these old files.

In system utilities you can manually start the RRD cleaning procedure You can either archive the selected RRD's to have them deleted in the next polling cycle



RRDfiles can also be schedule to automatically be removed this option can be configured in `Console > Configuration > Settings > Paths` by selecting the RRDfile autoclean selection.

RRDfile Checker

`Console > utilities > System Utilities > RRDfile Checker`

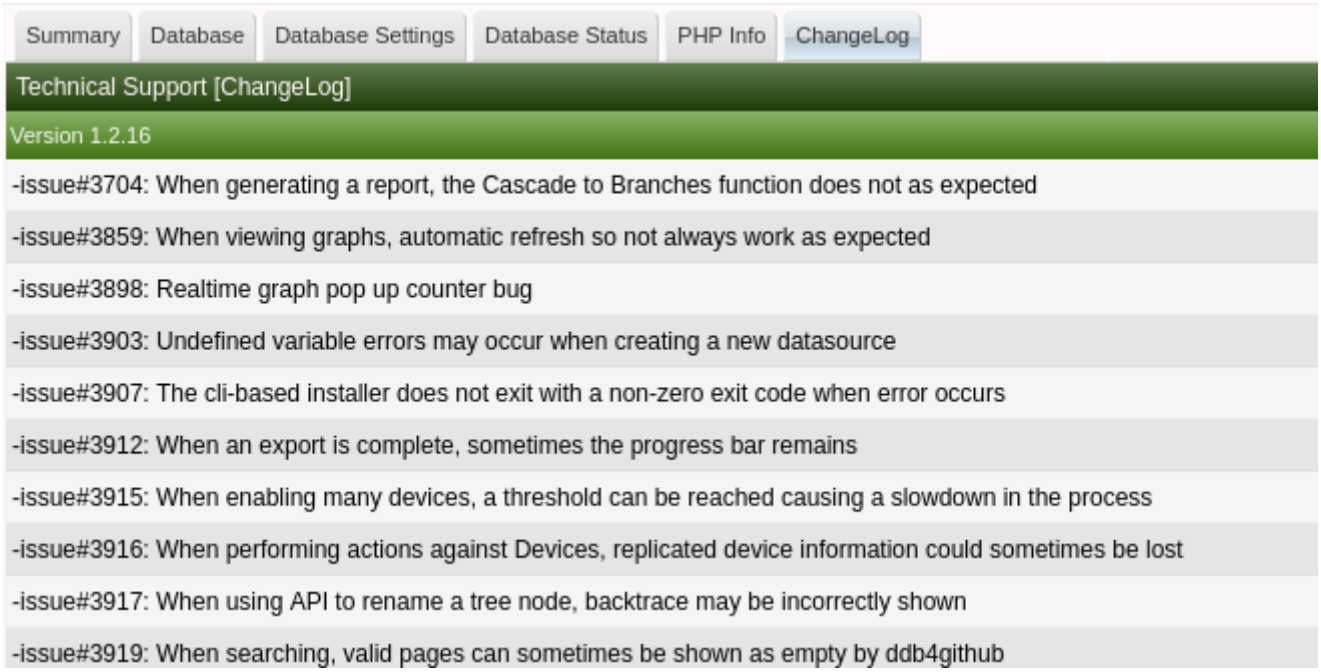
The RRDfile checker is a tool that scans all RRDfiles at a scheduled time and tests the stored data:

- Stale values for 1 or 24 hours
- Whether the file is updated regularly
- Incorrect file read/write access
- Whether the Database Data Source objects match the RRDfiles Data Sources

For very large Cacti installs the **RRDfile checker** can take quite a long time to complete. You can set number of concurrent parallel processes in `Console > Configuration > Settings > Data`

Viewing Change log from system utilities

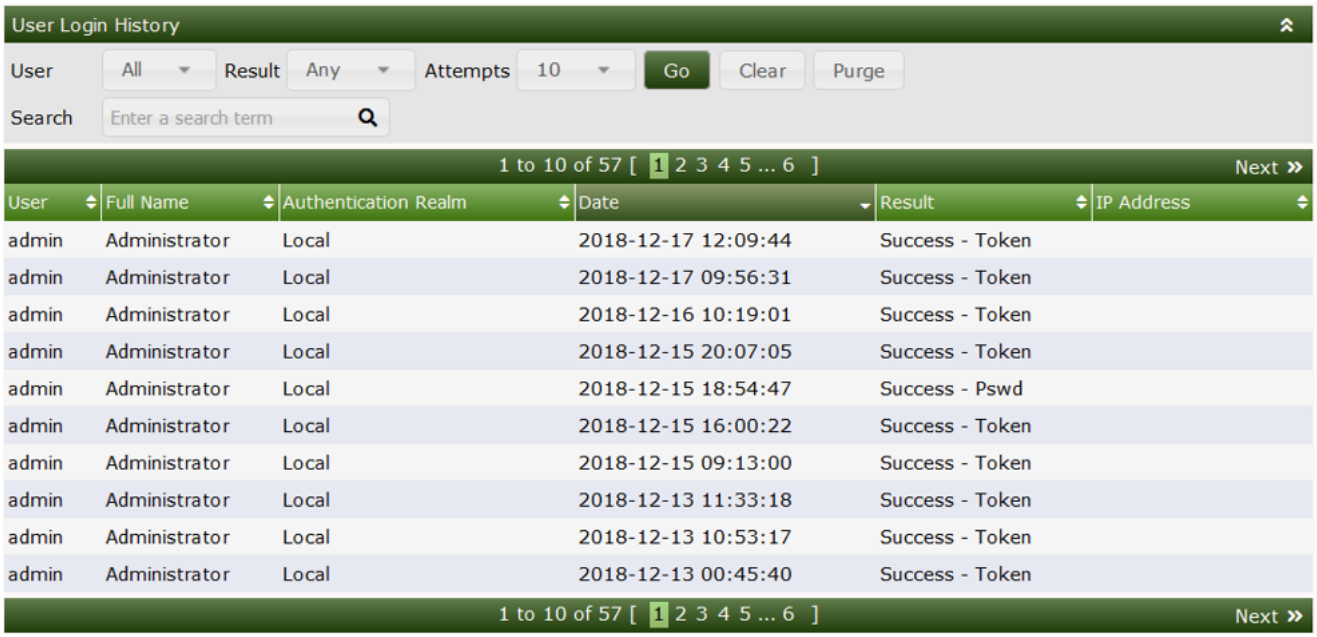
With Cacti 1.2.16+, you are now able to view the CHANGELOG information up to the version your server is currently running. This is available via the system utilities menu which is useful for seeing what fixes and occasional features are included in your version.



View user login history

`Console > utilities > System Utilities > View user Log`

In system utilities you are able to view a log of user logins both successful and failures The Log will also show what Client IP address the login originated from along with info if the login was based on password or token based login



View boost update status and stats

`Console > Utilities > System Utilities > View Boost Status`

In this view you can see statistics on how Boost is running you will be able to see Information such as when the last time boost started, How long it ran for, The average boost record size

Runtime Statistics	
Last Start Time:	2018-12-17 14:50:08
Last Run Duration:	0 seconds (0.00 percent of update frequency)
RRD Updates:	9922
Peak Poller Memory:	16.66 MBytes
Detailed Runtime Timers:	RRDUpdates:9922 TotalTime:0 get_records:0.01 results_cycle:0.35 rrd_filename_and_template:0.04 rrd_lastupdate:0.01 rrdupdate:0.05 delete:0.03
Max Poller Memory Allowed:	1024 MBytes
Run Time Configuration	
Update Frequency:	1 Hour
Next Start Time:	2018-12-17 15:50:08
Maximum Records:	1000000 Records
Maximum Allowed Runtime:	20 Minutes
Image Caching	
Image Caching Status:	Disabled
Cache Directory:	/var/www/html/cacti/cache/boost/
Cached Files:	0 Files
Cached Files Size:	0.00 Bytes

Viewing the Data Source Query Cache

`Console > Utilities > System Utilities > View Data Source Query Cache`

The Data Query Cache stores information gathered from Data Query input types. The values from these fields can be used in the text area of Graphs for Legends, Vertical Labels, and GPRINTS as well as in CDEF's.

Data Query Cache Items

Device: Any | Query Name: Any | Go | Clear

Search: Enter a search term | Rows: 10 | Include Index

1 to 10 of 1926 [1 2 3 4 5 ... 193] Next »

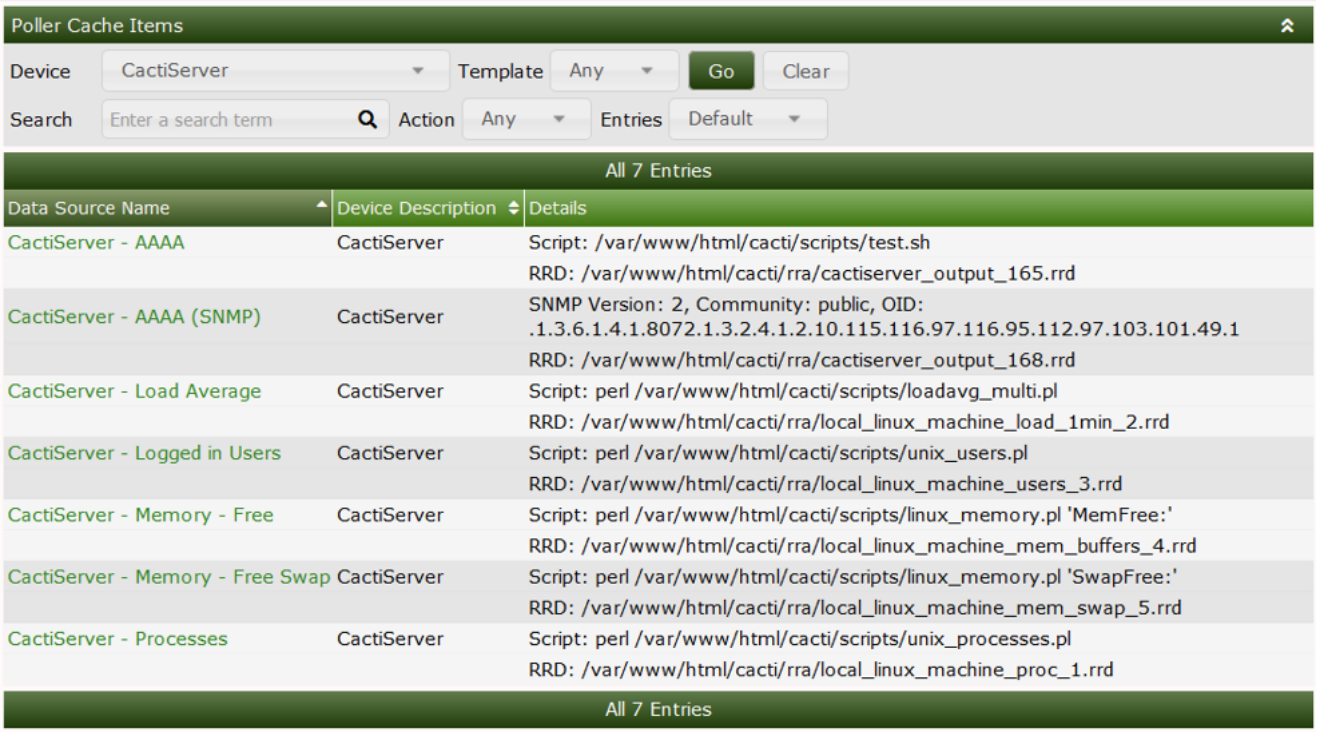
Device	Data Query Name	Index	Field Name	Field Value	OID
WebServer1	Example based on Interface XML	1	ifAlias		.1.3.6.1.2.1.31.1.1.1.18.1
WebServer1	Example based on Interface XML	2	ifAlias		.1.3.6.1.2.1.31.1.1.1.18.2
WebServer1	Example based on Interface XML	3	ifAlias		.1.3.6.1.2.1.31.1.1.1.18.3
WebServer1	Example based on Interface XML	4	ifAlias		.1.3.6.1.2.1.31.1.1.1.18.4
WebServer1	Example based on Interface XML	1	ifDescr	lo	.1.3.6.1.2.1.2.2.1.2.1
WebServer1	Example based on Interface XML	2	ifDescr	eno1	.1.3.6.1.2.1.2.2.1.2.2
WebServer1	Example based on Interface XML	3	ifDescr	virbr0	.1.3.6.1.2.1.2.2.1.2.3
WebServer1	Example based on Interface XML	4	ifDescr	virbr0-nic	.1.3.6.1.2.1.2.2.1.2.4
WebServer1	Example based on Interface XML	1	ifHighSpeed	10	.1.3.6.1.2.1.31.1.1.1.15.1
WebServer1	Example based on Interface XML	2	ifHighSpeed	1000	.1.3.6.1.2.1.31.1.1.1.15.2

1 to 10 of 1926 [1 2 3 4 5 ... 193] Next »

Viewing the Poller Cache

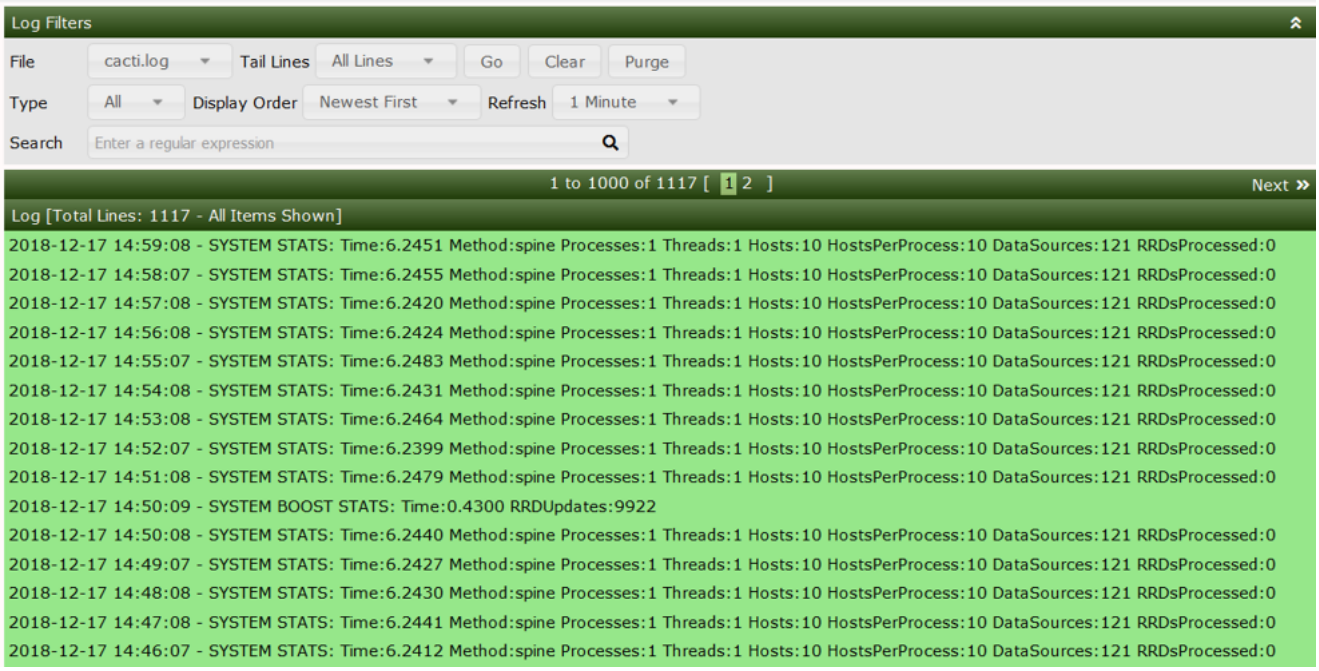
`Console > Utilities > System Utilities > View Poller Cache`

This is the data that is being passed to the poller each time it runs.



Viewing System log via System Utilities

You are able to view the system log via the System utilities page this will show the RAW log this slightly differs from the view log tab as the device names will show the device ID rather than the Device name this is the same view as you reading the log in the OS



Copyright (c) 2004-2026 The Cacti Group

Data Collectors

Data Collector Background

Cacti can support from one to many **Data Collectors**. There are two types of **Data Collectors** they are:

- **Main Data Collector** - This is essentially your core Cacti server and database. The **Main Data Collector** is also referred to as the **Primary Server**.
- **Remote Data Collector** - These Data Collectors are located in distant locations, or where reaching devices is blocked due to firewall or security policies. The **Remote Data Collectors** are also referred to as **Remote Pollers**.

Due to the design of the Cacti **Remote Data Collector**, someone at the remote site, can actually login to that **Data Collector** and interact with it as if their **Data Collector** was the **Main Data Collector**. Additionally, if for some reason the **Main Data Collector** becomes unavailable due to a WAN outage for example, the data for the **Devices** it manages will be cached locally until such time as the **Main Data Collector** is reachable again.

When connectivity returns, the remote collector forwards cached results to the main collector. This provides continuity during some network outages; it is not automatic failover of the main server, database, or RRD storage. Size local storage for the expected outage and monitor the delivery backlog.

Remote collection uses Cacti's built-in Boost functionality. Review its configuration before deploying additional collectors.

Main and remote systems exchange data over the configured web and database connections. Permit only the required hosts and configured ports, use HTTPS, and secure database connections according to the installation's policy. Confirm connectivity in both directions where configuration replication and remote-agent operations require it. Additional dependencies such as DNS, SNMP, and device availability checks have their own network requirements.

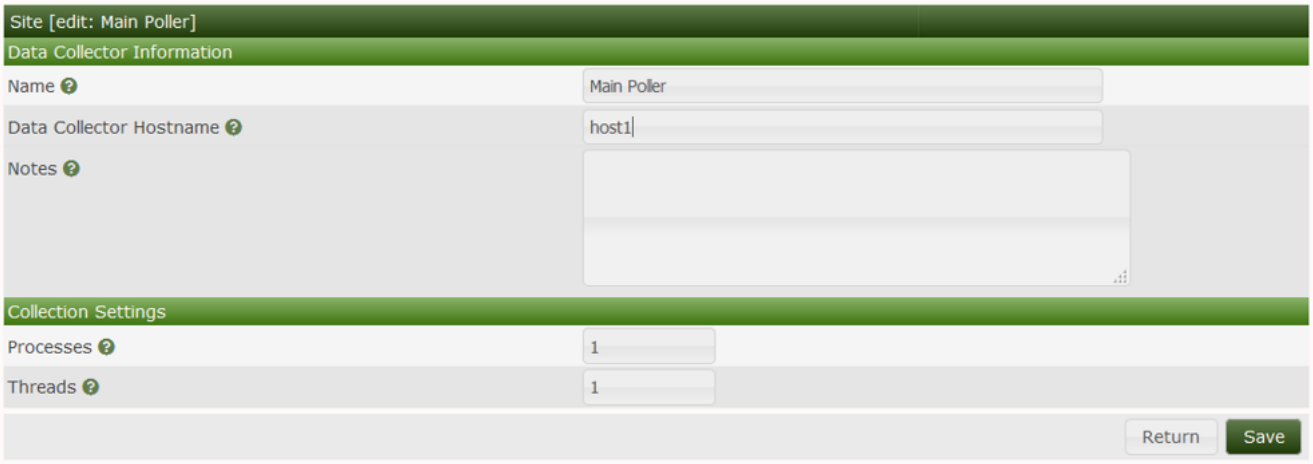
Data Collector User Interface

The image below shows the current online collector (aka pollers). On this page, we can see the current, average, and max data collection times, the **Data Collector** processes and threads used, the number of **Devices** as well as what those **Devices** are polling. The Actions drop-down allows your to Enable, Disable, and Delete **Remote Data Collectors**. There is also a `Full Sync` option there. The `Full Sync` option will replicate key Cacti tables to the selected **Remote Data Collectors** for things like authentication, global settings, etc.

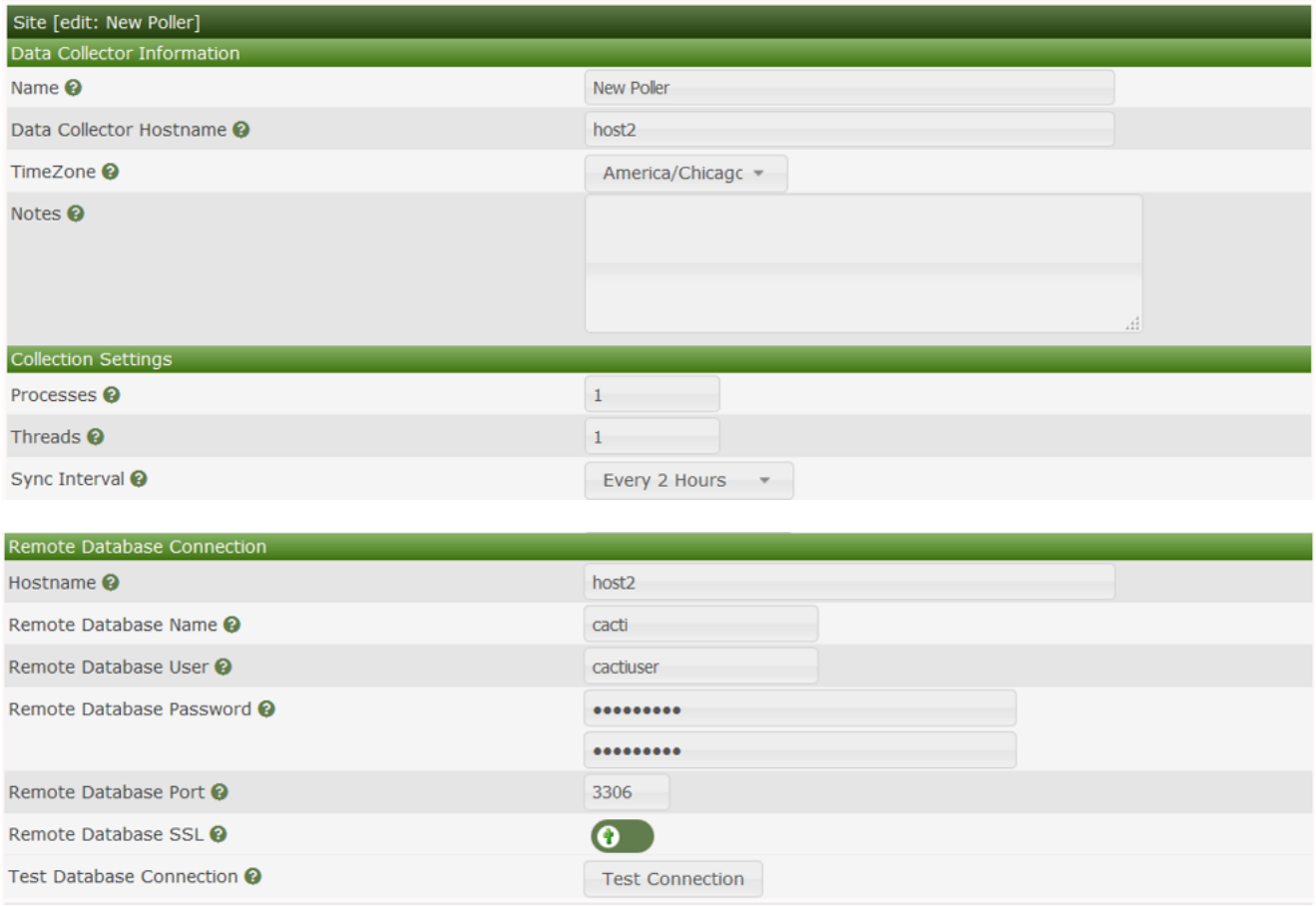
In the current Cacti design, you should not have to perform a `Full Sync` very often. It would mainly be used to push the user database and global settings to the remotes, after an outage if there were database changes during that outage.

The **Main Data Collector** resides on the central Cacti server. It also serves as the master **Data Collector** performing key maintenance operations for the entire system.

In the edit page below, you can see what options are available when editing the **Main Data Collector**. It is important that the hostname used is resolvable by the **Remote Data Collectors**.



When editing the **Remote Data Collector** in the images below, you can see that it shares many of the settings of the **Main Data Collector** with the addition of a `TimeZone` setting and MySQL/MariaDB credentials and a `Test Connection` button. Generally, these setting are only use during the initial setup of Cacti, and afterwards for diagnostics only.



Setup Main Data Collector to accept connections Remotes

Use the same Cacti version on the main and remote systems. Follow How To Setup Remote Pollers, and select **New Remote Poller** when installing the remote system.

Create a database account restricted to the remote collector's address, using the account name consistently for both grants. Run the following in an interactive database administration session after replacing the address and password placeholders:

```
CREATE USER 'cacti_remote'@'192.0.2.20' IDENTIFIED BY 'REPLACE_WITH_UNIQUE_PASSWORD';
GRANT ALL PRIVILEGES ON cacti.* TO 'cacti_remote'@'192.0.2.20';
GRANT SELECT ON mysql.time_zone_name TO 'cacti_remote'@'192.0.2.20';
```

Configure the database listener, firewall, TLS, and host restrictions for your network. Do not expose database access publicly or grant global privileges. Use separate accounts where collectors require different access policies.

On the remote system, `include/config.php` uses `$database_*` settings for its local database and `$rdatabase_*` settings for the main database. Populate the remote settings during installation using that release's `config.php.dist`. Verify the main hostname, database name, credentials, port, and TLS settings; do not copy the main database address into the remote's local database fields. Protect the configuration file because it contains credentials.

After installation, assign a test device to the collector, run a polling cycle, and verify both local collection and delivery to the main system. Confirm that graph timestamps advance before moving additional devices.

Copyright (c) 2004-2026 The Cacti Group

Spine

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

Spine is Cacti's C-based, multithreaded data collector. It replaces `cmd.php` as the collection engine; `poller.php` remains the scheduling entry point. Actual throughput depends on device latency, scripts, concurrency, database capacity, and the collection interval. Measure your installation before tuning.

Spine Architecture and Operating Principles

Spine reads polling definitions from the Cacti database, collects measurements using SNMP and scripts, and writes results to poller output tables. Cacti's poller or Boost processing updates the RRD files. Spine does not directly write historical RRD files or render graphs.

Spine can run several processes and multiple threads per process. It also maintains PHP script-server processes for methods configured to use `script_server.php`, avoiding a separate PHP startup for each such call.

Installing Spine

Use the Spine release corresponding to your Cacti release. For development checkouts, use compatible branches; do not mix a development poller with a maintenance installation without verifying compatibility.

Install a C compiler, make, Autoconf, Automake, Libtool, and the development headers and libraries required by that Spine version, including Net-SNMP and MySQL or MariaDB client libraries. Git checkouts also need the bootstrap tools. Use the package names and build repository appropriate to your distribution.

Download source from Cacti Spine releases or use your distribution's compatible package. From the extracted source:

```
./bootstrap
./configure --prefix=/usr/local/spine
make
sudo make install
```

Inspect `./configure --help` for library locations and optional build features. Package installations can use different paths and permission policies.

ICMP permissions

Run the poller as its dedicated service account. ICMP availability checks need socket permissions appropriate to the operating system and Spine version. Possible mechanisms include permitted ping sockets, a narrowly scoped raw socket capability, or the release's documented setuid installation.

The development branch can use unprivileged ICMP where the operating system permits it. When started setuid root, it opens the required sockets and drops root permanently before reading configuration. Older release behavior differs. Do not assume Spine ships setuid or repeatedly regains root while polling.

Follow the installed release's instructions and local security policy. Keep the executable owned by an administrative account and unwritable by the poller account. Test the chosen availability method as the actual poller user. SNMP collection does not itself require running the entire poller as root.

Configuring `spine.conf`

Create the configuration file from the installed template if one is not already provided by the package:

```
sudo cp /usr/local/spine/etc/spine.conf.dist /usr/local/spine/etc/spine.conf
```

Set the database host, database name, user, password, and port to match the collector's database. Use that release's `spine.conf.dist` as the reference. For example:

```
DB_Host         localhost
DB_Database     cacti
DB_User         cactiuser
DB_Pass         REPLACE_WITH_DATABASE_PASSWORD
DB_Port         3306
```

Use the `DB_UseSSL` and `DB_SSL_*` options when configuring the local database connection for TLS. Remote collectors also have `RDB_*` settings for the main database. Do not confuse local `DB_*` with remote `RDB_*` settings.

Restrict the configuration file to the accounts that need to read it, because it contains database credentials. One Linux example, with a poller in the `www-data` group, is:

```
sudo chown root:www-data /usr/local/spine/etc/spine.conf
sudo chmod 0640 /usr/local/spine/etc/spine.conf
```

Substitute the actual poller group. Do not change application code ownership to make a credential file readable.

Testing Spine from the Command Line

Use the poller account to run a database read-only test. The `-R` option suppresses database writes, but still polls devices and executes configured scripts. It is not a guarantee that those scripts have no side effects.

```
/usr/local/spine/bin/spine -R -V 3 -S
```

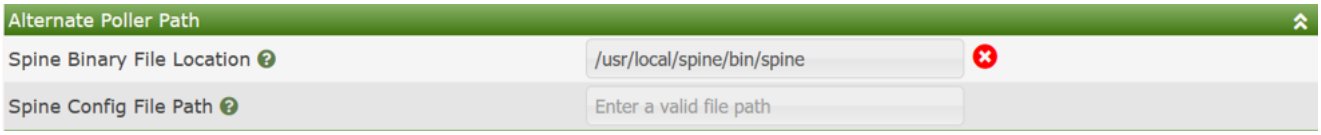
To restrict the test to one device, substitute its actual ID for `DEVICE_ID`:

```
/usr/local/spine/bin/spine -R -f DEVICE_ID -l DEVICE_ID -V 5
```

Confirm the configuration path, database connection, device availability, and returned values. Verbose output can expose sensitive operational information; redact it before sharing. Use the installed binary's `--help` for its options.

Activating Spine in Cacti

1. Open **Console > Configuration > Settings > Paths** and set **Spine Binary File Location** to the installed executable.
2. Set **Spine Config File Path** when using a non-default configuration path.
3. Open **Settings > Poller**, choose **spine** as the **Poller Type**, and save.
4. Leave the existing `poller.php` scheduler in place. Check the next scheduled cycle for errors and confirm that graph timestamps advance.



Tuning Spine for Optimal Performance

Change one setting at a time and compare elapsed polling time, missed samples, CPU use, database connections, and errors. More concurrency can overload a collector, database, or device.

- **Settings > Poller** contains global script-server counts and script timeout settings. Allow expected script execution time within the overall polling budget.
- **Data Collection > Data Collectors** sets processes and threads for each collector. Size them for that collector's workload and database capacity.
- **Management > Devices** provides device threads and maximum OIDs per request. Larger SNMP requests can reduce round trips, but can also exceed agent limits or fragment across the network.

There is no universal process-per-core ratio, maximum useful thread count, or OIDs-per-request value. Start with the release defaults and measured results.

Database Connection Sizing

Budget database connections across all collectors, poller processes, threads, script servers, web requests, and maintenance jobs. Remote collection can require connections to both local and main databases. Use Cacti's technical support information and observed peak connections to size `max_connections`.

A connection failure can indicate credentials, DNS, routing, TLS, account-host restrictions, or exhausted capacity. Check the database error before increasing limits, and account for the memory cost of additional connections.

Security Policies

SELinux or AppArmor can restrict execution, network connections, or configuration file access. Inspect policy denials and apply rules appropriate to the poller account and executable. Do not disable enforcement to diagnose a path or credential error. See Hardening.

Troubleshooting Spine

- **Configuration unreadable**: verify the configured path and read access as the poller user.
- **ICMP failure**: check the selected ping method, socket permissions, operating-system policy, and firewall. Do not assume SNMP must also fail.
- **Database failure**: verify the correct local or remote connection settings and inspect the database's error and connection count.
- **Missing samples**: inspect script output, timeouts, SNMP responses, and the configured collection interval.

A `SYSTEM STATS` log line containing `Method:spine` confirms the selected engine, not successful collection of every item. Check errors and actual RRD updates as well. See Data Source Debugging and The Cacti Log.

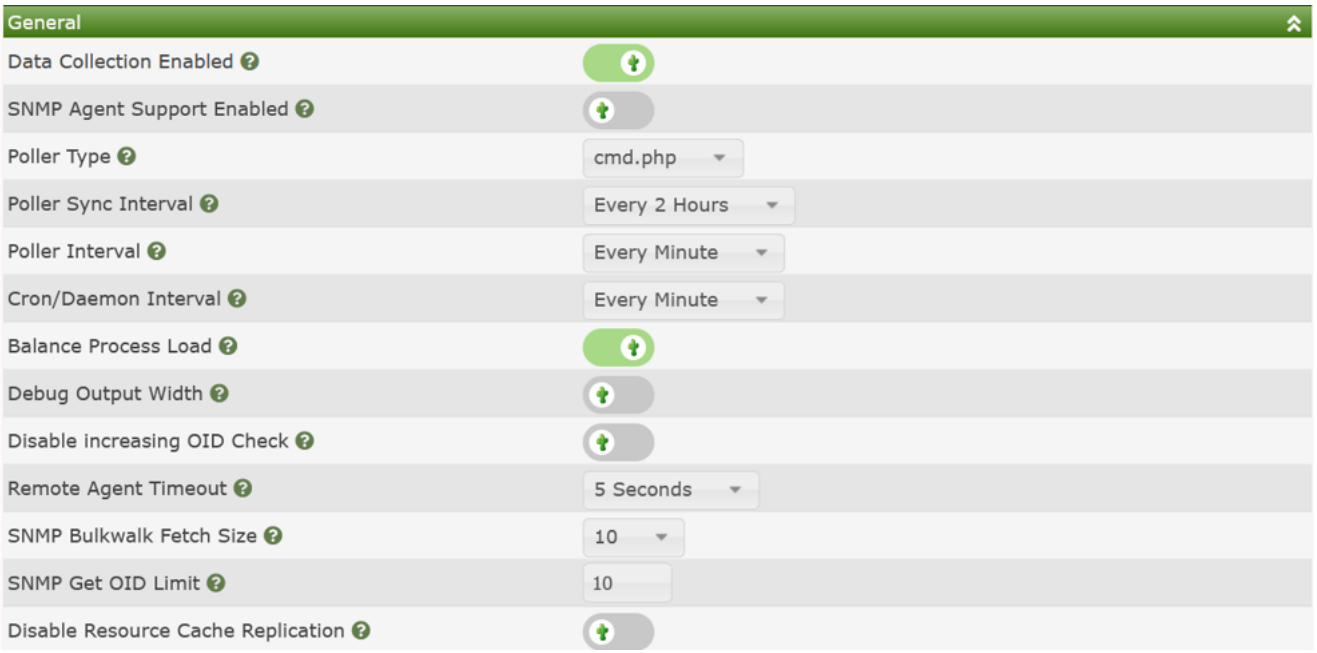
Copyright (c) 2004-2026 The Cacti Group

Poller Settings

The Poller sub-tab of the Cacti settings are where you go to control many aspects of **Data Collection** in Cacti, you control various base collection frequencies, communication timeouts, and other settings that impact how Cacti's **Data Collection** works.

General Settings

The **General Settings** are a broad collection of settings into multiple categories. The image below shows Cacti's **General Settings** sub-section.



Those settings include:

- **Data Collection Enabled** - This is a setting that essentially will override and disable all Cacti **Data Collection** from all of Cacti's **Data Collectors**. It's really intended to allow you to put Cacti in maintenance mode with a single click.
- **SNMP Agent Support Enabled** - The SNMP Agent has a measurable amount of overhead on larger systems. Therefore, if you are not using it, you should disable it. However, the SNMP Agent is a marvelous addition to any Cacti system that's participating in a multi-level Operations Framework.
- **Poller Type** - This is where you define either Cacti's built in poller `cmd.php`, or use the more robust poller `spine`. You will not see the `spine` drop-down option unless it has first been installed, and the `spine` binary path defined from `Console > Configuration > Settings > Paths` settings sub-tab.
- **Poller Sync Interval** - When creating a Cacti **Data Collector** this is the default `Full Sync` interval it will use. In the current Cacti design, a `Full Sync` should only be required when attempting to replicate local authentication or authorization data to **Remote Data Collectors**, of if you have made changes that would otherwise impact the behavior of a **Remote Data Collector** when it was offline.
- **Poller Interval** - This setting must align with the highest frequency **Data Source Profile** polling interval.

> **Note**: This only applies to the **Data Source Profiles** that are in use. If you only plan on polling every minute and therefore only using the `1 Minute` **Data Source Profile**, then your **Poller Interval** should be `1 Minute` as well.

- **Cron/Daemon Interval** - This is the frequency at which either the Cacti daemon (cactid.php), or the systems crontab/task-manager should relaunch every **Data Collectors** Poller process.

- **Balance Process Load** - This setting will sort the devices such that each concurrent Cacti process is polling the same number of **Data Sources**. Otherwise, Cacti will simply order the hosts by their `device ids`.
- **Debug Output Width** - This is a diagnostic settings that you would only enable from time to time if you wish to check if the output from your **Data Collector** scripts exceeded the output width in the various Poller tables. Since checking this have a very negative impact on polling performance, it's disabled by default.
- **Disable Decreasing OID Check** - This setting is for performing **Data Query** `Re-index` activities for tables whose OID's are not structured in an orderly way where the suffix for the OID increases in a known way. Though, in most cases enabling this setting will not be harmless. There may be some cases where it would slow the `Re-index` process.
- **Remote Agent Timeout** - When Cacti either needs information from a **Data Collector**, like `ping` latency from the collector to one of the **Devices** it's monitoring, or when a **Remote Data Collector** needs information from the main Cacti server, this is the timeout for receiving a handshake from that server. It's use to keep the Website from locking up in the case a **Data Collector** is not responding due to being down.
- **SNMP Bulkwalk Fetch Size** - This controls how many OID's are returned per UDP turn when obtaining `snmpwalk` data from an SNMP enabled **Device**.

> **Note**: This only works with SNMP v2 and v3. It is not supported on older SNMP v1 **Devices**.

- **SNMP Get OID Limit** - The default maximum number of SNMP Get OIDs to issue per `snmpget` request. For **Devices**, this setting is controlled at the **Device** level. You should only use this setting when using Cacti's SNMP API natively in your scripts and Plugins.
- **Disable Resource Cache Replication** - By default Cacti will continually replicate it's Website content with it's **Remote Data Collectors**. So, essentially, they are self upgradable. However there are some cases where customers wish to upgrade the Central system before allowing the remotes to self upgrade. In those cases, you would enable this feature.

Background Timeout Settings

The settings below limit how long Cacti background processes will be allowed to run before timing out and being killed the system and notifying the Cacti administrator. These background timeouts are required to keep Cacti performing a peak performance by not allowing certain tasks to re-spawn while they are still running causing system overload. Below is the image of those settings.

Those settings include:

- **Report Generation Timeout** - Some reports can take several minutes to complete. Adjust accordingly based upon your system size and the number of reports across all users.
- **Data Source Statistics Timeout** - Depending on the size of your Cacti system, some **Data Source Statistics** tasks can take quite a bit of time. This setting is for the periodic collection and not the Daily process.
- **Background Commands Timeout** - Cacti will schedule some commands to be run offline during normal operations. This timeout affects those commands. Some activities include some aspects of re-indexing, removing old RRDfiles, etc.

- **Maintenance Background Generation Timeout** - Cacti performs daily maintenance at midnight every day. This maintenance can include: rotating logs, purging old logs, calculating **Data Source** statistics, etc. This setting controls it maximum allowed runtime.
- **Spikekill Background Generation Timeout** - If you are using Cacti's prophylactic spike-killer process on your Data Queries, this will control it's maximum allowed run-time.

Data Collector Defaults

Cacti's **Data Collectors** include both concurrent process and thread settings. They are maintained at the **Data Collector** level due to the fact that not all **Data Collectors** are created equally. You may have one **Data Collector** that's collecting data from 1,000 hosts, and another that is collecting from 10. Maybe one is a Raspberry Pi and the other a two socket EPYC chip. You would not want the same settings for each.

These settings are only the defaults, when you fist create a **Data Collector**, once you save them, the setting stays with the individual **Data Collector**. Below, you can see an image of those settings.

Those settings include:

- **Data Collector Processes** - The default number of concurrent processes the **Data Collector** will spawn every cron or daemon cycle.
- **Threads per Process** - The default number of `spine` threads per Concurrent Poller Process above.

> **Note**: The Thread setting only applies if you are using the `spine` poller. It does not apply to the `cmd.php` poller.

Additional Spine Parameters

These settings are Global Spine poller settings. They apply to all **Data Collectors**. An image of these settings can be seen below:

Those settings include:

- **Invalid Data Logging** - By default spine will only log the number of errors it experienced for a **Device**. If you wish more granularity you can change this settings to either `Summary` or `Detailed` logging.
- **Number of PHP Script Servers** - Another level of Spine parallelism is it's ability to create a pool of memory resident PHP interpreters for running PHP scripts, making spine very powerful. Every concurrent process will have this many memory resident PHP interpreters pooled for running scripts.
- **Script and Script Server Timeout Value** - This is the timeout to stop run-on script from blocking poller performance. If a script exceeds this timeout, it will generate a log message.

Copyright (c) 2004-2026 The Cacti Group

PHP Script Server

The **PHP Script Server** is a memory resident PHP interpreter that is launched by each Cacti **Data Collector** or poller each polling cycle. If using `cmd.php`, there will be at most one **PHP Script Server** launched per poller process. If using `spine` poller, then up to 10 **PHP Script Servers** will be launched in a pool for the various `spine` threads to consume.

If you have a number of devices that must gather their data via a script, it's very important to utilize the **PHP Script Server**. This feature allows for the rapid execution of PHP based **Scripts** and **Data Queries** in Cacti thus reducing **Data Collector** polling times.

The reason that the **PHP Script Server** is so fast is that the PHP interpreter is started in memory one time, and for every **Data Source** polled, it's controlling function is interpreted only once. The resulting much faster script execution and reduced load average during the **Data Collectors** life cycle. Using the **PHP Script Server** process over the traditional exec() process nets roughly a 20+ fold speed improvement in Cacti's **Data Collection** process.

Since PHP is such a powerful language, this Cacti feature enables rapid collection of virtually any **Data Source** metric. It's an excellent choice for both SNMP and non-SNMP based data.

Using the Script Server

Cacti contains two sample **PHP Script Server** scripts for reference. They are for the collection of HostMib CPU and Disk Partition information. These two examples are based off the traditional POPEN version of the HostMib functions found in earlier versions of Cacti. Upgrading from these very old **Data Input Methods** to the **PHP Script Server** is not longer supported.

Migration of Existing Scripts and Data Queries to PHP Script Server

If you have **Scripts** or **Data Queries** that you wish to migrate, you must follow the steps below to migrate those scripts to the **PHP Script Server** format.

Script File Changes

First, if your script is written in another languages, say Perl, or Python, you need to convert it to PHP. Then, each PHP Script file must be changed to the new Script Server format. The changes are not dramatic, but required for the proper operation of the **PHP Script Server**. Note that you should use PHP internal functions for calling services rather than forking binaries within the PHP script as this will defeat the purpose of the **PHP Script Server**. Follow the steps below to complete the migration.

1. Copy your existing script to a new name. The name must begin `ss_` followed by your script name. The `ss_` identifies the script as being a script server variety of the a PHP script and is named that way to easily identify them in your install. For example, if you previously had a script called `get_mysql_stats.php`, it's new name would be `ss_get_mysql_stats.php`.
2. Edit the new PHP script and add the following required lines to the file, where `ss_myfunction` is the same as your filename.

```
<?php
$no_http_headers = true;

/* display No errors */
error_reporting(E_ERROR);

if (!isset($called_by_script_server)) {
    array_shift($_SERVER["argv"]);
```

```
    print call_user_func_array("ss_myfunction", $_SERVER["argv"]);
}
?>
```

3. What was originally just mainline code, must be replaced with a function name. For example, if your program previously contained the following three lines of code:

```
<?php
$a = 100;
$b = $a / 10;
print $b;
?>
```

   Would become:

```
function ss_myfunction() {
    $a = 100;
    $b = $a / 10;
    print $b;
}
```

4. If you have any additional functions declared within your script file, you must prefix them to make them unique among all functions. Our recommendation would be to prefix all functions with the name of the main function. For example if you have a function called "meme" you would rename it to `ss_myfunction_meme`. This guarantee's correct **PHP Script Server** functionality.

5. The last step is to change the function call that could have traditionally returned the value to the Cacti poller using the `print` function. You must change that line or lines in your code to utilize the `return` function instead. However, this does not apply to `print` statements that are not called from the Poller. For a simple script, this results in

```
function ss_myfunction() {
    $a = 100;
    $b = $a / 10;
    return $b;
}
```

   Be careful, when writing Script Server Data Queries. Use the `return` function for returning results of the GET operation. But use `print` for `index` and `query` operations, e.g.

```
if (($cmd == "index")) {
    ...
    print $some_index_data . PHP_EOL;
} elseif ($cmd == "query") {
    ...
    print $some_query_data . PHP_EOL;
} elseif ($cmd == "get") {
    ...
    return $some_get_data;
}
```

XML File Changes

If your **Data Sources** are based upon a **Data Query**, then you must also change your XML file contents and location. Please reference the XML files in the `<path_cacti>/resource/script_server` directory for the specifics related to your required modifications. However, you may also follow the instructions below:

1. Modify the `<script_path>` tag. Change it from:

```
<script_path>|path_php_binary| -q |path_cacti|/scripts/myfucntion.php</script_path>
```

   to simply the following:

```
<script_path>|path_cacti|/scripts/ss_myfunction.php</script_path>
```

2. Add the following two XML tags below the `<script_path>` tag. Replace `ss_myfunction` with your function name:

```
<script_function>ss_myfunction</script_function>
<script_server>php</script_server>
```

3. Save the XML file.

Data Query & Data Template Changes

Your **Data Queries** and **Data Templates** must be also modified. Although somewhat self explanatory by now, you must make the following changes:

1. If the **Data Template** is based upon a **Script** and not a **Data Query**, change it's **Data Input Method** to `Script Server`.
2. If the **Data Template** is based upon a **Data Query**, edit the **Data Query** and change it's **Data Input Method** to `Get Script Server Data (Indexed)` and change the XML file path to point to the new XML file in the `<path_cacti>/resources/script_server/` directory.
3. Your final step is to go to the **Console > System Utilities > Rebuild Poller Cache** to apply the new settings.

If your **PHP Script Server** script is operating correctly, you should now be migrated to the **PHP Script Server**.

Testing Your Script in the Script Server

To test your script in the script server, simply follow the instructions below. When you have finished you testing, simply type "quit" + `<CR>` at the **PHP Script Server** command line to exit the script server.

1. Start the script server - You can do this by typing the following command:

```
shell> php <path_cacti>/script_server.php
```

> **NOTE**: Due to a bug in Windows implementation of PHP, you must type the full path name to the `script_server.php` file.

2. Type in your command - Using the example from above, you would type in the following:

```
script server> /path_cacti/scripts/ss_myserver_file.php my_function argument1 argument2 ...
```

In the Windows environment, your example could be the following:

```
script server> c:\www\root\cacti\script\ss_myserver_file.php ss_myfunction argument1 argument2 ...
```

3. If your script and function is operating properly, you should get a result.

```
script server> c:\www\root\cacti\script\ss_myserver_file.php ss_myfunction argument1 argument2 ...
nameA:valueA nameB:valueB ...
```

4. To quit the script server, simply type `quit <CR>` at the command line.

> **Note**: If there are errors in your script, you must restart the script server before your retest your code.

Copyright (c) 2004-2026 The Cacti Group

Performance Boosting Settings

Summary

Cacti Performance Settings, formerly known as `boost` are available to support very large Cacti installations, and are required for supporting the multiple **Data Collector** architecture that Cacti affords.

Designed years ago, Boost's intent was to reduce the main data collector's cycle time by caching writes to disk, and those writes would be handled by an out of band process currently known as `poller_boost.php`.

Since these Large Sites had potentially well over one million **Data Sources**, there was previously no way to write all that data every 5 minutes. So, the buffering helped with maintaining a consistent run-time, and minimizing the user experience disruption from all that blocking Disk I/O.

In the modern Data Center, where we find an abundance of NVMe, Flash, and SSD's, the impact of all that I/O has been reduced. However, Boost now serves as a way to extend the life of your Flash media and therefore increase the availability of your Cacti server over its lifetime.

The Boost function in Cacti has several features that can be enabled or disabled on demand by the Cacti Administrator. They include:

- **On Demand RRD Updates** - Required in ALL multiple **Data Collector** installs and control the mass update of RRDfiles through periodic dumping of Cacti's Boost cache.
- **Image Caching** - caches PNG and SVG representations of the Cacti **Graphs** to reduce the overall CPU utilization of the Cacti server infrastructure.

Prerequisites

> **Recommendation**: For most installations, especially those using the **multiprocessing update** feature introduced in Cacti 1.2.17, or any deployment with multiple Data Collectors, use **InnoDB** for all Boost tables. The MEMORY engine can cause race conditions between concurrent processes competing for memory table locks, which leads to polling backlogs and data loss on busy systems. InnoDB with SSD or NVMe storage performs comparably and avoids these failure modes. MEMORY tables are also incompatible with MariaDB Galera and MySQL replication.

In its initial design the Boost process leveraged MySQL Memory Tables to increase overall performance, and to reduce writing data to disk. With improvements in InnoDB performance over the years combined with Flash storage, the need for MySQL Memory Tables has diminished. The sections below document MEMORY table sizing for sites that have an explicit reason to use it, but InnoDB is the recommended engine. The main boost data table (`poller_output_boost`) already ships as InnoDB; the `ALTER TABLE` examples below convert tables to MEMORY only if you have a specific reason to use that engine.

If you do wish to use Memory you have to pay close attention to the amount of data that will be cached in your design. You should periodically check that you have sufficient memory available to cache all the data in the Boost cache. The MySQL/MariaDB setting that controls the maximum size of a memory table is the `max_heap_table_size` setting in your `/etc/my.cnf.d/server.cnf` file.

To change the `max_heap_table_size` setting you require a restart of MySQL/MariaDB. So you won't want to make these changes often, as some systems have little time between mass updates to perform a restart. If you restart MySQL/MariaDB when the cache is full, you will lose those updates upon restart unless you first change the ENGINE of the Boost cache table to InnoDB or MyISAM before restarting.

When using Memory Tables you have to be cognizant of the maximum output width of the Cache table called `poller_output_boost`. By default, it's set to 512 bytes, which allows for very wide output from your **Data Collectors**. MySQL Memory Tables do not allow for variable sized columns in memory - which means if you write one byte to it, it will take 512 bytes. So, tuning this width is very important. That will be explained in more detail later on in this chapter.

Checking how your system is configured

To see how your system is configured, you can goto `Console > Utilities > System Utilities > View Boost Status` option, when you go there, you will see an image similar to that below.

From this image, we can tell a few things:

- Boost is Idle, or flushing the Cache
- The system has 1,120 Data Sources in total
- There are currently 58,710 Boost records cached in all tables

Additionally, you can tell

- Our Current Database Engine is InnoDB
- The size of the current table is 3.52 MBytes
- The average 'Bytes/Record' is 62
- We have unlimited Records that can be stored there

From that Data, we know if we were to use a Memory Table, at present it's width could be say 100 Bytes and we could cut memory use by 80%.

We can also estimate from this interface, how large our `max_heap_table_size` would ultimately have to be, keeping in mind that the `Current Boost Table Size` is misleading since even though each record can be in excess of 512 bytes, it's presently only using 62 bytes of total storage per record.

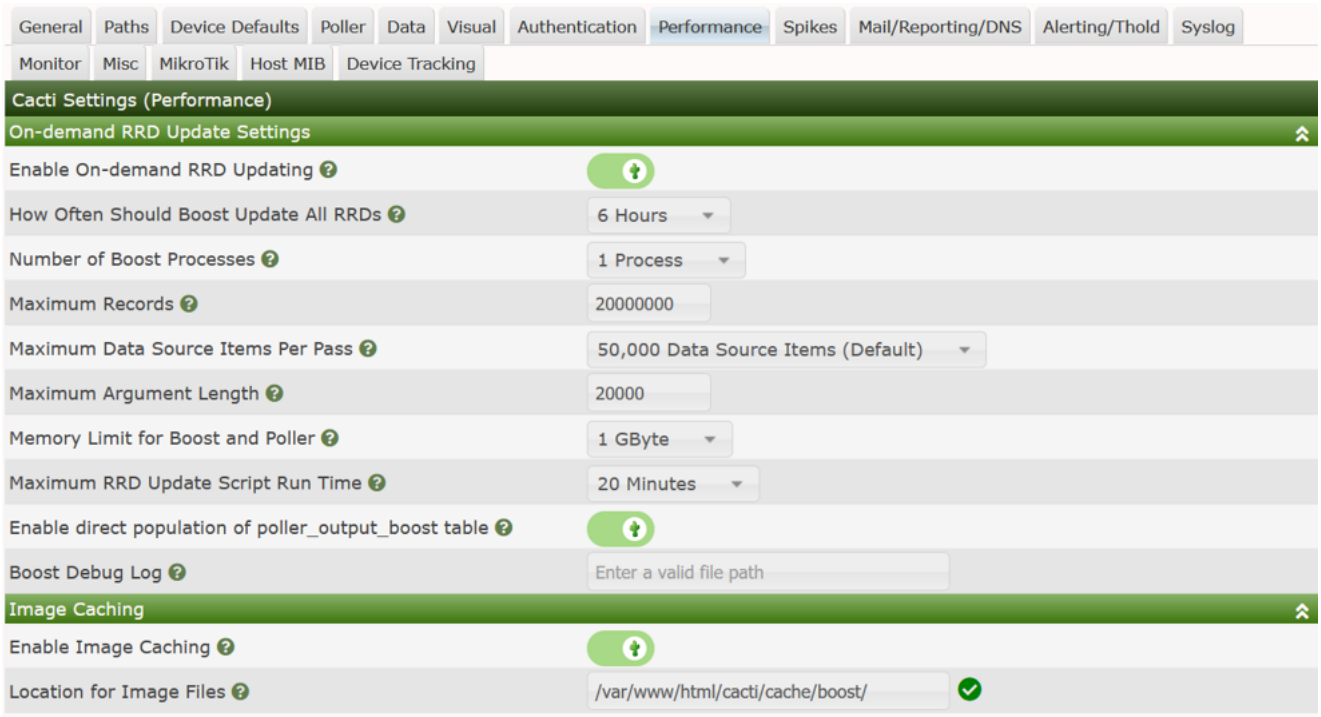
In the image below, you can see runtime statistics, update frequencies, and Image Caching settings, which are currently disabled.

Runtime Statistics	
Last Start Time:	2021-01-03 11:20:09
Last Run Duration:	11 seconds (0.00 percent of update frequency)
RRD Updates:	741,530
Peak Poller Memory:	58.74 MBytes
Detailed Runtime Timers:	RRDUpdates:741530 TotalTime:12 get_records:1.03 results_cycle:7.62 rrd_filename_and_template:0.33 rrd_lastupdate:0 rrdupdate:6.05 delete:2.33
Max Poller Memory Allowed:	1024 MBytes
Run Time Configuration	
Update Frequency:	6 Hours
Next Start Time:	2021-01-03 17:20:09
Maximum Records:	20,000,000 Records
Maximum Allowed Runtime:	20 Minutes
Image Caching	
Image Caching Status:	Disabled
Cache Directory:	/var/www/html/cacti/cache/boost/
Cached Files:	0 Files
Cached Files Size:	0 Bytes

From this setup, you can see that mass RRDfile updates are only happening roughly every 6 hours to minimize wear and tear on the SSD storage in use.

Enabling Boost

To find and Enable the Boost settings, you first go to `Console > Configuration > Settings > Performance`. From this interface, in the image below, you can setup Boost.



As an exercise, lets look at a fictitious setup and determine what the `max_heap_table_size` should be. Let's say, for example, that your Cacti systems **Poller Cache** contains 200k rows, and the maximum length of any rows output is 20 bytes, about the size of a 64bit counter value.

If you are running scripts, ones that return very long results, like the MySQL statistics plugin, you should consider carefully the next step.

In this case, we will assume that even though the largest value returned is 20 bytes, we will elect to maintain a maximum output column width of 50 bytes.

The important thing here is that memory tables store the full size of the column, even though the column type is `varchar()`. By default it's `varchar(512)`. Therefore, if your system only needs 50 bytes, you will have 90% waste in your `poller_output_boost` table.

The next step would be to modify the structure of your `poller_output` and `poller_output_boost` tables. You would do this by doing the following:

```
ALTER TABLE poller_output
    MODIFY COLUMN output varchar(50) NOT NULL DEFAULT '',
    ENGINE=MEMORY;

ALTER TABLE poller_output_boost
    MODIFY COLUMN output varchar(50) NOT NULL DEFAULT '',
    ENGINE=MEMORY;
```

As previously mentioned, its also important that the `poller_output` table is converted to memory, eliminate any disk I/O related to poller updates.

Now, you need to determine how many polling cycles will fit into your `poller_output_boost` table. In this case, when sizing the `poller_output_boost` table, you take the output column width and add 34 bytes per record. So, in this case, each data source result to be stored in the `poller_output_boost` table would take 84bytes.

Then, with the Maximum Heap Table size in hand, and let's say the default is 16MBytes for the largest table (the MySQL default), you can calculate the number of poller intervals that you can store without running out of MySQL memory. So, let's take our example:

```
200k Cache Entries x 84 Bytes Per Poller Cache Row = 1,680,000 Bytes per Poll
```

This means that your `poller_output_boost` can handle 10 Cacti polling cycles, or roughly 4/5 of an hour of poller data before it must be cleared by the system.

As all things go, you should not want to take your system to the edge. So, it is best to increase the `max_heap_table_size` of your system to ensure you have buffer in cases where say for example, your disks become degraded over time and the Boost cycle is extended as a result.

But let's look at the problem a different way. Let's say that you wish to perform major updates every 4 hours. With a 50% memory reserve, we should plan for 6 hours of updates. This way, we need to have size for 72 poller cycles in your `poller_output_boost` table. This means that the `max_heap_table_size` must be at least 131,040,000 Bytes.

So, let's call it a deal. Provided you have enough memory, and I expect that you do, you would edit the /etc/my.cnf.d/server.cnf and add/modify the following line:

```
max_heap_table_size=132M
```

Then, save the file, and restart MySQL. Once this is done, you are ready to "enable" Boost as described above.

Flushing the Boost Cache

> **Warning**: MEMORY tables are cleared on every MySQL/MariaDB restart. If you are using MEMORY storage, always flush the Boost Cache before taking the database offline. Failure to do so will result in loss of all buffered poller data since the last flush.

If you are planning on system maintenance and are using MEMORY storage in MySQL or MariaDB, flush your Boost Cache before the system is taken offline. Login to the Cacti system as root and run the following:

```
cd /var/www/html/cacti
php -q poller_boost.php --force
```

This operation will take several minutes to complete, at which time there will be additional records in the `poller_output_boost` table. So, it might be better that after the `poller_boost.php` process is complete, to modify the `poller_output_boost` table to InnoDB before reboot, and then convert it back to memory after they system maintenance is complete.

Copyright (c) 2004-2026 The Cacti Group

rrdproxy

Cacti RRD Proxy is a standalone proxy interface to Tobi Oetiker's great RRDtool.

The main focus of this project is to allow the relocation of RRDfiles, data replication, support of Cacti Boost and, if needed, the RRDcached daemon. It allows system administrators to split Cacti's access resources across machines without the need for NFS.

Due the fact that updates to files may contain sensitive data and to prevent undesired updates, RRDtool proxy utilizes high encrypted connections (RSA2048 and AES192) with continuously changing keys. This ensures a secured connection between the proxy and registered clients and/or other proxies.

A local service port is provided to allow administrators are able to access a separate command line interface which provides easy access to configure and debug the proxy.

Contribute

Get involved in development by participating in active development on GitHub.

Requirements

RRDproxy runs on any Unix-based operating system that can meet the minimum requirements, though it will perform better with the recommended (or higher).

Minimum Requirements

- PHP Version 7.1 or higher Installation must include and enable the following modules:
    - Sockets
    - Posix
    - PCntl
    - GMP
    - ZLIB
- RRDtool Version 1.5 or higher
- Cacti Version 1.2.7 or higher

Recommended Requirements

All minimum requirements must be met, though the following are more optional:

- PHP Version 7.3 or higher Include all minimum recommendation modules
- RRDtool 1.7 or higher
- Cacti 1.2.7

Usage

Before using the RRDproxy, you must install the system using the setup wizard which can be run at any point using the -w parameter.

```
php rrdtool-proxy.php --help

RRDtool Proxy v1.2.7
Copyright (C) 2004-2026 The Cacti Group
usage: rrdtool-proxy.php [--wizard] [-w] [--version] [-v]
Optional:
-v --version   - Display this help message
-w --wizard    - Start Configuration Wizard
```

When running through the wizard, the status of items will appear on the right hand side of the screen. When an item is successfully completed, the status of `[OK]` will be shown in green (when color is supported) as seen below:

```
Checking System Requirements...
[0.00001] test: operation system supported                              [OK]
[0.00007] test: php module 'sockets'                                    [OK]
[0.00011] test: php module 'posix'                                      [OK]
```

Should any item have an issue, the status will be displayed as something other than `[OK]` such as `[FAILED]`:

```
[0.40270] read: RRDproxy configuration file                          [FAILED]
```

Status Color | Description : --- | : --- Green | Success and without issue Yellow | Warnings and will be ignored, correct the problem if needed Red | Errors and will not be ignored, may prevent continuation

When the wziard prompts for the various settings that are available, you can accept the default value simply by pressing return to accept it. Below are a few of these settings and the default values that can be accepted:

Setting | Default | Comment : --- | : --- | : --- CLI System Prompt | rrdp | Max 8 chars IPv4 Listen Addr | 0.0.0.0 | Listen all addr Client Proxy Port | 40301 | Connections from a client Proxy port | 40302 | Connections from another proxy Dedicated CLI port | 40303 | Connections to the RRDproxy cli

Command Line Interface

Use '?' to get a list of all commands being supported by the proxy or hints about missing parameters.

```
telnet localhost 40303
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
rrdp>?
  enable          Turn on privileged commands
  help            Display help
  show            Show running system information
  quit            Close terminal session

rrdp>ena
rrdp#sh version

#     ___           _   _    __    __    ___     ___
#    / __\__ _  ___| |_(_)   /__\  /__\  /   \   / _ \_ __ _____  ___   _
#   / /  / _` |/ __| __| |  / \// / \// / /\ /  / /_)/ '__/ _ \ \/ / | | |
#  / /__| (_| | (__| |_| | / _  \/ _  \/ /_//  / ___/| | | (_) >  <| |_| |
#  \____/\__,_|\___|\__|_| \/ \_/\/ \_/___,'   \/    |_|  \___/_/\_\__, |
#                                                                   |___/

 RRDtool Proxy v1.2.7
 Copyright (C) 2004-2026 The Cacti Group
 rrdp uptime is 14 days, 0 hours, 4 minutes, 14 seconds
 Memory usage 0.18872917 % (2026464/1073741824 in bytes)
```

```
  a8:29:6a:b7:5d:be:c3:8a:be:13:7a:61:ee:0c:8b:d3
  Process ID: 13996
  Session usage (1/399)

  Server IP [192.168.10.10]
  Administration: [localhost       :40303]
  Replication:    [192.168.10.10   :40302]
  Clients:        [192.168.10.10   :40301]

rrdp#
```

Missing features

- Data replication has been completed to 95%, but is still not ready
- NET-SNMP pass persistent daemon and MIB needs to written to make proxy stats available through SNMP

Copyright (c) 2004-2026 The Cacti Group

Remote Poller Setup

Remote Pollers add complexity to your cacti setup while also massively extended the scalability of it. Remote Pollers allow system admins to distribute the load of polling between different servers as well as provide another source of polling close to a device.

For example you may have a device in Toronto but if your pollers are in New York the network latency may causes issues with your metrics having a remote poller in Toronto in this case will help with that issue.

Setup Instructions

The process of remote poller installation is similar to cacti installation. In the wizard, select the installation type for Remote poller. For the remote poller you must use the same version of cacti as the main poller installation.

With this setup the following layout is assumed

- Main server IP is 192.168.1.5
- Remote poller 1 server IP is 192.168.1.10
- Remote poller 2 servers Ip is 192.168.1.20

Database Configuration

> **Security**: The username `cacti` and password `cacti` shown in the examples below are placeholders. Use a strong, randomly generated password in production. Never use the literal string `cacti` as a database password on any reachable system.

Each server will have its own local database however the remote pollers will need to talk back to the main poller so we must allow the remote servers user account to connect back to the main pollers database via the network

Main server database config

```
CREATE USER 'cacti'@'192.168.1.10' IDENTIFIED BY 'strongpassword';
GRANT ALL PRIVILEGES ON cacti.* TO 'cacti'@'192.168.1.10';
CREATE USER 'cacti'@'192.168.1.20' IDENTIFIED BY 'strongpassword';
GRANT ALL PRIVILEGES ON cacti.* TO 'cacti'@'192.168.1.20';
FLUSH PRIVILEGES;
```

Remote poller 1 database configuration

```
CREATE USER 'cacti'@'192.168.1.5' IDENTIFIED BY 'strongpassword';
GRANT ALL PRIVILEGES ON cacti.* TO 'cacti'@'192.168.1.5';
FLUSH PRIVILEGES;
```

Remote poller 2 database configuration

```
CREATE USER 'cacti'@'192.168.1.5' IDENTIFIED BY 'strongpassword';
GRANT ALL PRIVILEGES ON cacti.* TO 'cacti'@'192.168.1.5';
FLUSH PRIVILEGES;
```

We have also let the main poller talk to the remote pollers to keep in sync

Spine configuration

When you have setup multiple pollers while using spine you will need to also configure the spine.conf file to connect to the remote database as well.

Be sure to remove the # next to the below entries

```
RDB_Host     192.168.1.5
RDB_Database cacti
RDB_User     cacti
RDB_Pass     strongpassword
RDB_Port     3306
```

If the network between pollers and the main server is not fully trusted, enable SSL for the remote database connection:

```
RDB_UseSSL 1
RDB_SSL_Key  /etc/cacti/ssl/client.key
RDB_SSL_Cert /etc/cacti/ssl/client.crt
RDB_SSL_CA   /etc/cacti/ssl/ca.crt
```

Spine poller and max connections

Since in a multi poller setup Spine is highly recommended you will need to follow the following calculation when that calculation would be:

```
connections = spine-processes x ( 1 + threads-per-process  + PHP servers )
```

Remote poller config.php setup

Now that we have set up the database connections we need to set up the cacti config.php file on the remote pollers to give them the database info they need to connect to the main poller we won't have to edit the main servers config.php file aside from the normal install procedure to give it access to its local database. Below is the portion of the config.php that we need to edit to allow the remote poller to talk to the main server be sure to remove the hash sign (#)

```
$rdatabase_type = 'mysql';
$rdatabase_default = 'cacti';
$rdatabase_hostname = '192.168.1.5';
$rdatabase_username = 'cacti';
$rdatabase_password = 'strongpassword';
$rdatabase_port = '3306';
$rdatabase_retries = 5;
$rdatabase_ssl = false;
$rdatabase_ssl_key = '';
$rdatabase_ssl_cert = '';
$rdatabase_ssl_ca = '';
```

Copyright (c) 2004-2026 The Cacti Group

Change poller interval from 5 minutes to 1 minute

This howto describes all necessary steps for convert your cacti to 1 minute poller interval. This cause one gap in graphs during the convert. Gap size depends on count of your rrd files.

Prerequisites and default values used in this howto

- Cacti installation directory is **/usr/local/share/cacti/**
- PHP binary is **/usr/local/bin/php**
- RRD files are in **/usr/local/share/cacti/rra**
- Poller is running by user **cacti**
- **Not structured RRDfile paths** (`Configuration->Settings-> Data` -> Structured RRDfile Paths). With structured path you will have more works with splice rrd files. You need change splice script.
- Crontab row looks like

```
*/5    *       *       *       *       cacti   /usr/local/bin/php /usr/local/share/cacti/poller.php 2>&1
```

Backup your Cacti

Before this operation, backup Cacti database and all data in rra folder

RRDCleaner (optional step)

You can run rrdcleaner and delete unused RRD files. It can decrease converting time. System Utilities

Stop poller

Disable your poller in Crontab or via GUI `Console -> Configuration -> Settings -> Poller tab -> Data Collection Enabled`

Prepare directory structure for RRD file convert

```
mkdir -p /tmp/cacti/rrd_old
mkdir -p /tmp/cacti/rrd_new
mkdir -p /tmp/cacti/rrd_fin
```

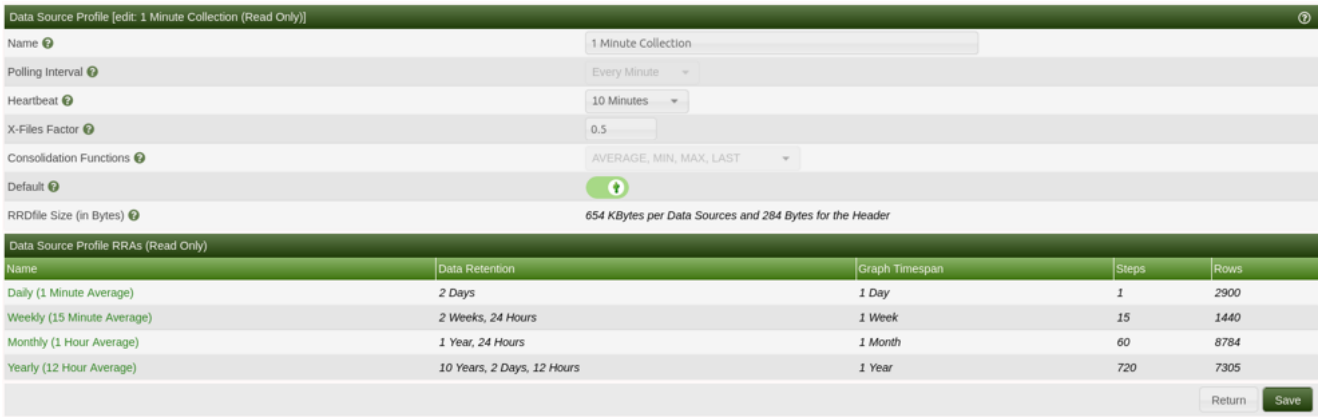
Make sure that you have enough space on temporary disk. You need more than twice of rra directory size.

Move rrd files

```
mv /usr/local/share/cacti/rra/* /tmp/cacti/rrd_old/
```

Check or create Presets -> Data profiles

Go to `Console -> Presets -> Data Profiles`. Here should be at least 5 Minute Collection profile (because you are using it until now) and 1 Minute Collection profile. If the second is missing, create it.



Remember Data profile ID - Edit your 1 minute collection again and in URL will be .../data_source_profiles.php?action=edit&**id=3**

Switch to 1 min

`Console -> Configuration -> Settings -> Poller tab`

- Poller interval = Every minute
- Cron/Daemon interval = Every minute

Update database records

Run these SQL queries:

```
UPDATE data_template_data SET rrd_step=60 WHERE rrd_step=300;
UPDATE data_template_rrd SET rrd_heartbeat=600;
```

Here replace X with ID from Step "Check or create Presets -> Data profiles":

```
UPDATE data_template_data SET data_source_profile_id=X;
```

Rebuild poller cache

`Console -> Utilities -> System utilities -> Rebuild Poller Cache`

Set 1 min Data Profile as default

`Console -> Presets -> Data Profiles`

Edit profile **1 min Collection**, checkbox default.

Change profile for all Data templates

`Console -> Templates -> Data Source`

Select all -> Change Profile

Run the poller once

New rrd files with correct 1 min profile will be created in **/usr/local/share/cacti/rra** directory. Cacti log shouldn't contain errors.

Move new rrd files to temporary folder

```
mv /usr/local/share/cacti/rra/* /tmp/cacti/rrd_new/
```

Run splice

You need run splice command for each rrd file. This could be time consuming, depends on number of files. You can use some tool for parallel tasks. The splice_rrd.php file is designed to allow two RRDfiles to be merged. This utility can effectively change the resolution/step of an RRDfile so long as the new RRDfile already has the correct step.

Basic script:

```
#!/bin/sh

cd /tmp/cacti/rrd_old/
for f in *.rrd
do
    echo $f
    /usr/local/bin/php /usr/local/share/cacti/cli/splice_rrd.php --oldrrd=/tmp/cacti/rrd_old/$f --newrrd=/tmp/
cacti/rrd_new/$f --finrrd=/tmp/cacti/rrd_fin/$f
done
```

Run script and wait. My experience - 5200 files took 100 minutes without parallelization. Script will be output sometning like:

```
fortigate_1_-_fg200e4q17911506_traffic_in_7317.rrd
NOTE: Using Native Arrays due to lack of SQLite.
NOTE: Using RRDtool Version 1.8.0
NOTE: RRDfile will be written to '/tmp/cacti/rrd_fin/fortigate_1_-_fg200e4q17911506_traffic_in_7317.rrd'
NOTE: Re-Importing '/tmp/fortigate_1_-_fg200e4q17911506_traffic_in_7317.dump.27482713' to '/tmp/cacti/rrd_fin/
fortigate_1_-_fg200e4q17911506_traffic_in_7317.rrd'
NOTE: Time:0.88, RUsage:18 MB
```

Move spliced files back to cacti

```
mv /tmp/cacti/rrd_fin/* /usr/local/share/cacti/rra/
```

File owner should be cacti

```
chown cacti /usr/local/share/cacti/rra/*
```

Enable poller in Cron again

Change your cron settings from 5 minutes to 1 minute and enable (uncomment). Old row:

```
#*/5    *       *       *       *       cacti   /usr/local/bin/php /usr/local/share/cacti/poller.php 2>&1
```

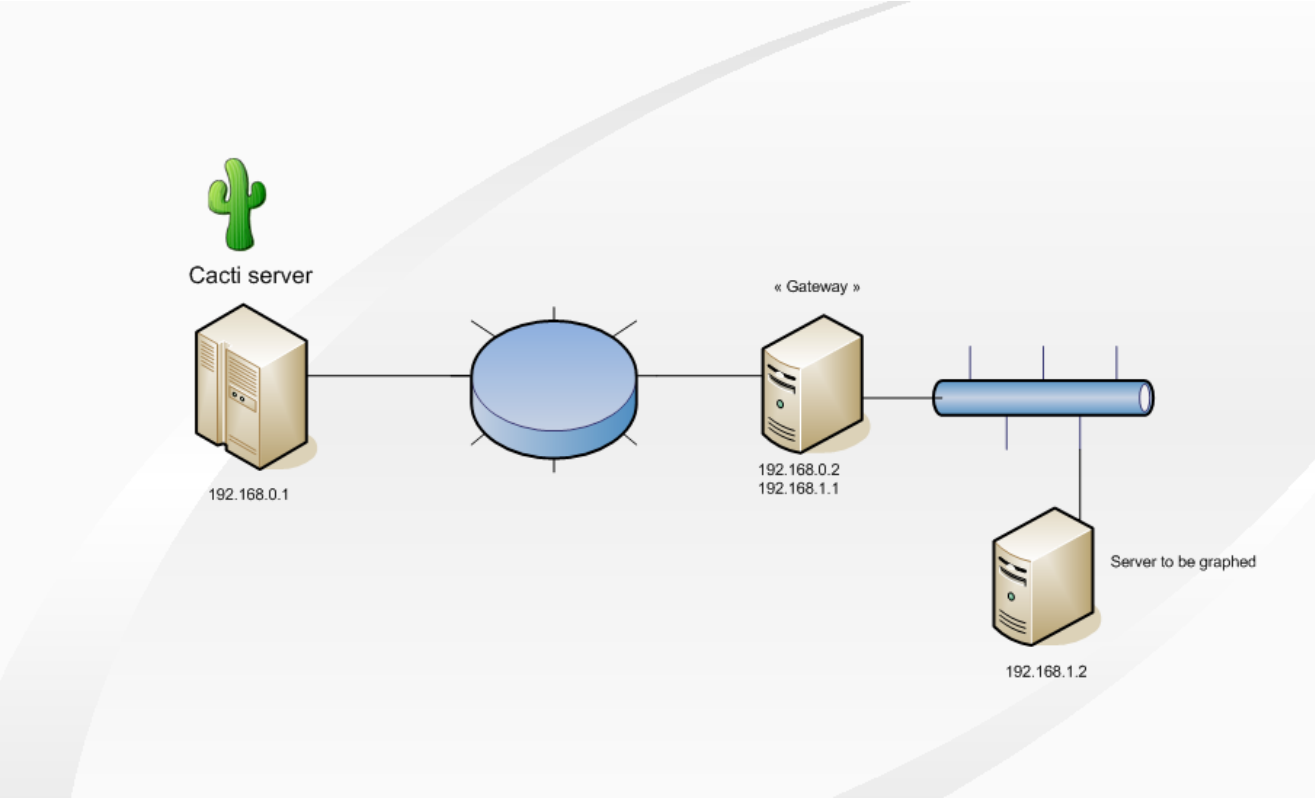
New row

```
*    *       *       *       *       cacti   /usr/local/bin/php /usr/local/share/cacti/poller.php 2>&1
```

How To Set Up SSH Tunnels to Graph a Remote Unix Server

Howto created by `fmangeant` at the Cacti Forum

This guide explains how to use SSH tunnels to graph a Unix server that is not directly reachable by your Cacti server.



In this example, the Cacti server can reach the Gateway, which can reach the target server.

> **Important — TCP vs UDP**: SSH port forwarding (`-L`) is TCP-only. Standard SNMP uses UDP port 161 and **cannot** be tunneled this way. This guide works only when the target's `snmpd` is explicitly configured to accept TCP transport (as shown below). It is **not** suitable for most network devices (routers, switches, etc.), which support UDP SNMP only. For UDP SNMP across untrusted networks, use a VPN instead.

Configuring the SSH tunnel

On the Gateway, create a `cactiuser` account:

```
# useradd -d /home/cactiuser -m cactiuser
```

Generate an SSH key pair (no passphrase, so the tunnel can start unattended). The modern recommendation is ed25519:

```
# su - cactiuser
$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/cactiuser/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/cactiuser/.ssh/id_ed25519.
Your public key has been saved in /home/cactiuser/.ssh/id_ed25519.pub.
```

Authorize the public key for login:

```
$ cd $HOME/.ssh
$ cp -p id_ed25519.pub authorized_keys
```

Create the SSH tunnel:

```
# su - cactiuser -c "ssh -f -N -g -L 192.168.0.2:10000:192.168.1.2:161 cactiuser@localhost"
```

This forwards all TCP traffic sent to `192.168.0.2:10000` on the Gateway to `192.168.1.2:161` on the target server.

Option summary:

```
-f  Go to background before executing the command
-N  Do not execute a remote command
-g  Allow remote hosts to connect to locally forwarded ports
-L  Forward the given local port to the given host and port on the remote side
```

Making the tunnel persistent (systemd)

On systemd-based hosts covered by the supported operating systems, `/etc/rc.local` is deprecated and disabled by default. Use a systemd service instead.

Create `/etc/systemd/system/cacti-ssh-tunnel.service`:

```
[Unit]
Description=SSH tunnel for Cacti SNMP polling
After=network.target

[Service]
User=cactiuser
ExecStart=/usr/bin/ssh -N -g -L 192.168.0.2:10000:192.168.1.2:161 cactiuser@localhost
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Enable and start it:

```
# systemctl daemon-reload
# systemctl enable cacti-ssh-tunnel
# systemctl start cacti-ssh-tunnel
```

Configuring Net-SNMP on the target server

By default, the Net-SNMP agent listens on **UDP** port 161. For this SSH tunnel approach you must configure it to listen on **TCP** port 161 instead.

In `snmpd.conf` on the target server:

```
agentaddress tcp:161
rocommunity mycommunity
```

For a more detailed `snmpd.conf` reference, see the Net-SNMP snmpd.conf man page.

Testing SNMP connectivity

From the Gateway host:

```
$ snmpwalk -v 1 -c mycommunity tcp:192.168.1.2 sysname
SNMPv2-MIB::sysName.0 = STRING: target_server
```

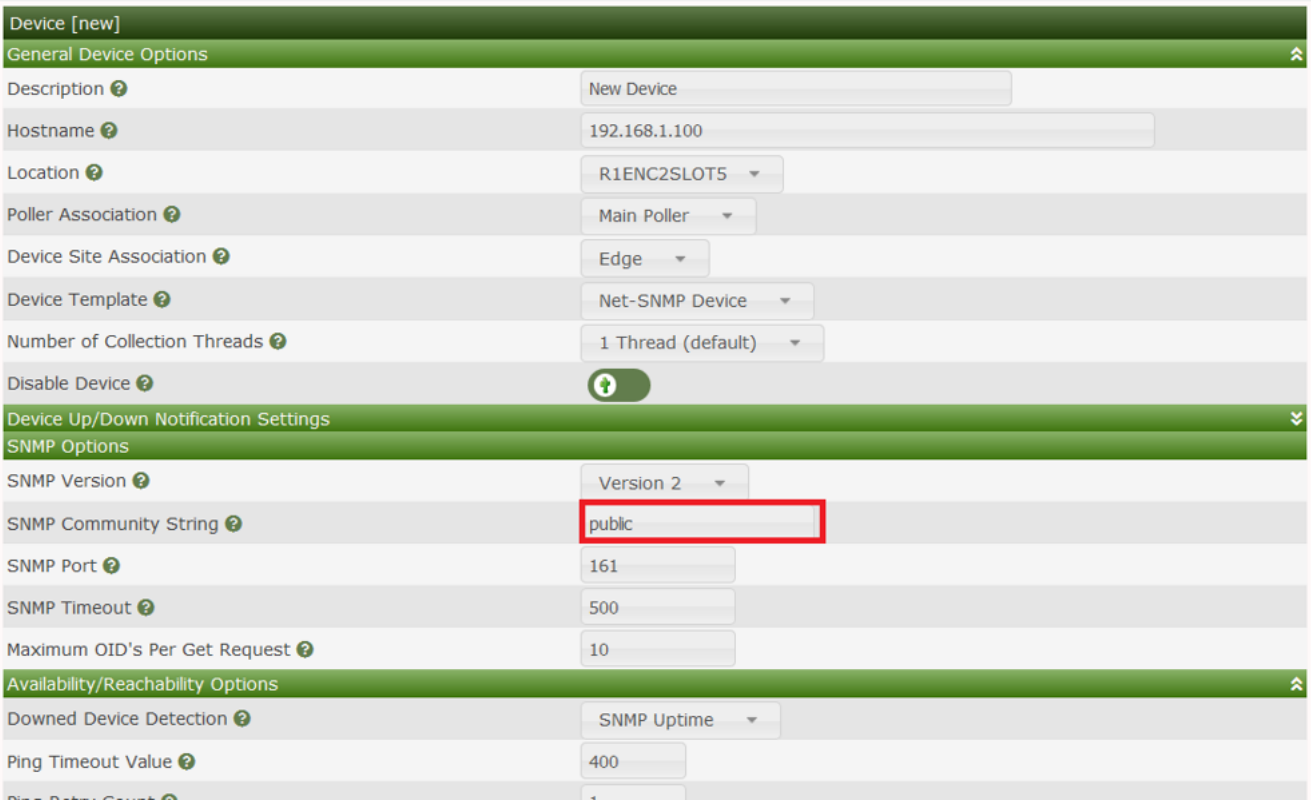
From the Cacti server:

```
$ snmpwalk -v 1 -c mycommunity tcp:192.168.0.2:10000 sysname
SNMPv2-MIB::sysName.0 = STRING: target_server
```

If these succeed, the host is ready to be added to Cacti. If not, review your firewall rules and verify `snmpd` is listening on TCP.

Adding the device to Cacti

In Cacti, create a new device as shown:



Your target server is now graphed by Cacti.

Copyright (c) 2004-2026 The Cacti Group

How To Extend snmpd with custom script/ data

Some SNMP servers can be extended with custom scripts or programs. The output of the running script can then be read using the OID and can therefore be easily used in Cacti. An example is the thermometer on the USB port. In combination with Graph a Single SNMP OID, this is a very simple way to display such data.

We will show the procedure for the very extensive Net-SNMP and FreeBSD BSNMPD.

Linux Net-SNMP

Simply add a line to `/etc/snmpd.conf` and restart snmpd:

```
extend test /bin/echo hello
```

The result will be here:

```
snmpwalk -v2c -c testing 127.0.0.1 nsExtendOutput1
NET-SNMP-EXTEND-MIB::nsExtendOutput1Line."test" = STRING: hello
NET-SNMP-EXTEND-MIB::nsExtendOutputFull."test" = STRING: hello
NET-SNMP-EXTEND-MIB::nsExtendOutNumLines."test" = INTEGER: 1
NET-SNMP-EXTEND-MIB::nsExtendResult."test" = INTEGER: 0
```

For Cacti you need a numeric OID. Use snmptranslate:

```
snmptranslate -On NET-SNMP-EXTEND-MIB::nsExtendOutput1Line."test"
```

The result is an OID that we can use:

```
.1.3.6.1.4.1.8072.1.3.2.3.1.1.4.116.101.115.116
```

FreeBSD BSNMPD

Install bsnmp-ucd:

```
pkg install bsnmp-ucd
```

Add these lines to `/etc/snmpd.config` and restart bsnmpd:

```
begemotSnmpdModulePath."ucd" = "/usr/local/lib/snmp_ucd.so"
%ucd
extNames.0 = "first command"
extCommand.0 = "/path/to/script.sh"
extNames.1 = "second command"
extCommand.1 = "/path/to/script.sh param1"
```

The name and result of the first command share the same index:

```
snmpwalk -v2c -c testing 127.0.0.1 .1.3.6.1.4.1.2021.8.1.2.0
.1.3.6.1.4.1.2021.8.1.2.0 = STRING: first command
```

```
snmpwalk -v2c -c testing 127.0.0.1 .1.3.6.1.4.1.2021.8.1.101.0
.1.3.6.1.4.1.2021.8.1.101.0 = STRING: 15
```

Copyright (c) 2004-2026 The Cacti Group

Templates

The real strength of Cacti is unleashed by using templates. There are five different types of templates with the basic Cacti installation:

- **Device Templates** - Define a Collection of **Graph Templates** and **Data Queries** that belong to a class of **Devices**
- **Graph Templates** - Define how to paint the Graph, it size, legend, left and right Axis and it Canvas
- **Data Templates** - Define how data is to be stored in the RRDfile and how often to collect that data
- **Aggregate Templates** - A special class of **Graph Template** that allows you to combine elements from multiple **Graph** of a common Template into one **Graph**
- **Color Templates** - A collection of color rotations to use for **Aggregate Graphs**

While it is perfectly fine to define all **Data Sources** and **Graphs** without using Templates at all, the burden of this approach is high.

In most installations, there are hundreds if not thousands of **Devices** of the same kind. And there are many similar **Graph Templates**, that span multiple **Device Templates**. For example: Network Traffic information is needed for almost every **Device**. Therefore, the parameters needed to create a traffic RRDfile are defined by the **Data Template**, in this case known as "Interface - Traffic". These definitions are used by all Traffic-related RRDfiles.

The same approach is used for defining **Graph Templates**. This is done only once. And all parameters defined within such a **Graph Template** are copied to all **Graphs** that are created using this Template.

The last type of classic Templates are the **Device Templates**. The purpose of **Device Templates** is to group all **Graph Templates** and **Data Queries** for a given **Device Type**. Example **Device Types** that you might find a **Device Template** for include:

- Cisco Router
- Net-SNMP Device
- F5 Load Balancer
- Windows Device
- Local Linux Device
- ...

So you will make up a **Device Template** e.g. for a specific type of router, switch, host and the like. By assigning the correct **Device Template** to each new **Device**, you'll never forget to create all needed **Graphs**.

There's no need to create all Templates on your own! Apart from the fact, that many common templates are provided out-of-the-box, there's a very simple mechanism to **Import Templates** and to **Export Templates** and more recently to **Import Packages**.

More recent additions to the Cacti family of Templates are the **Aggregate Template** and **Color Template**. The **Aggregate Template** allows you to define a **Graph Template** that can aggregate data from many **Graphs** sharing that Template.

The **Color Template** defines a list of colors that are used in a Round Robin fashion to change the colors seen on the resulting **Aggregate Graphs** to better assist the user in interpreting the resulting **Graph**.

In order to best follow the natural course of Template creation, the remainder of this chapter will explain Templating starting with **Data Templates**, then we will go from there to **Graph Templates**, and from there go to a walk-through of creating both a **Data Template** and **Graph Template**.

From there, we will explain how to create a **Graph** using that newly created set of Templates.

Data Templates

In Cacti, a **Data Template** provides a skeleton for an actual **Data Source**. If you have many **Data Sources** that share most of their characteristics, using a **Data Template** makes sense. Saving a template propagates eligible fields into attached Cacti data-source definitions, subject to their per-source settings. It does not automatically rewrite existing RRD files.

Changes to physical RRD structure need a separate migration or repair. RRDtool 1.5 and later can add a data source with `rrdtool tune DS:...`; current Cacti RRD comparison/repair code can generate those recommendations. Back up the RRD and inspect the proposed changes before applying them. Changing a sampling step or archive layout needs a migration suited to that change. See Data Source Profiles.

Creating a Data Template

To create a new **Data Template**, select **Data Source** under `Console > Templates` menu selection and then click Add, which in most cases will be the plus sign in the upper right area of the page.

The first thing you must do is give the template a name. This name has nothing to do with the **Data Source** name, but is what you will use to identify the template throughout Cacti.

Second, you will notice a list of **Data Source** / **Data Source Item** field names with Use Per-Data Source Value check-boxes next to each one. The nice thing about Templates in Cacti is that you can choose whether to template each field on a per-field basis. If you leave the check-box unchecked, every **Data Source** attached to the **Data Template** will inherit its value from the Template. If the check-box is checked, every **Data Source** attached to the **Data Template** will contain its own value for that particular field. When generating a real **Data Source**, you will be prompted to fill those non-templated fields.

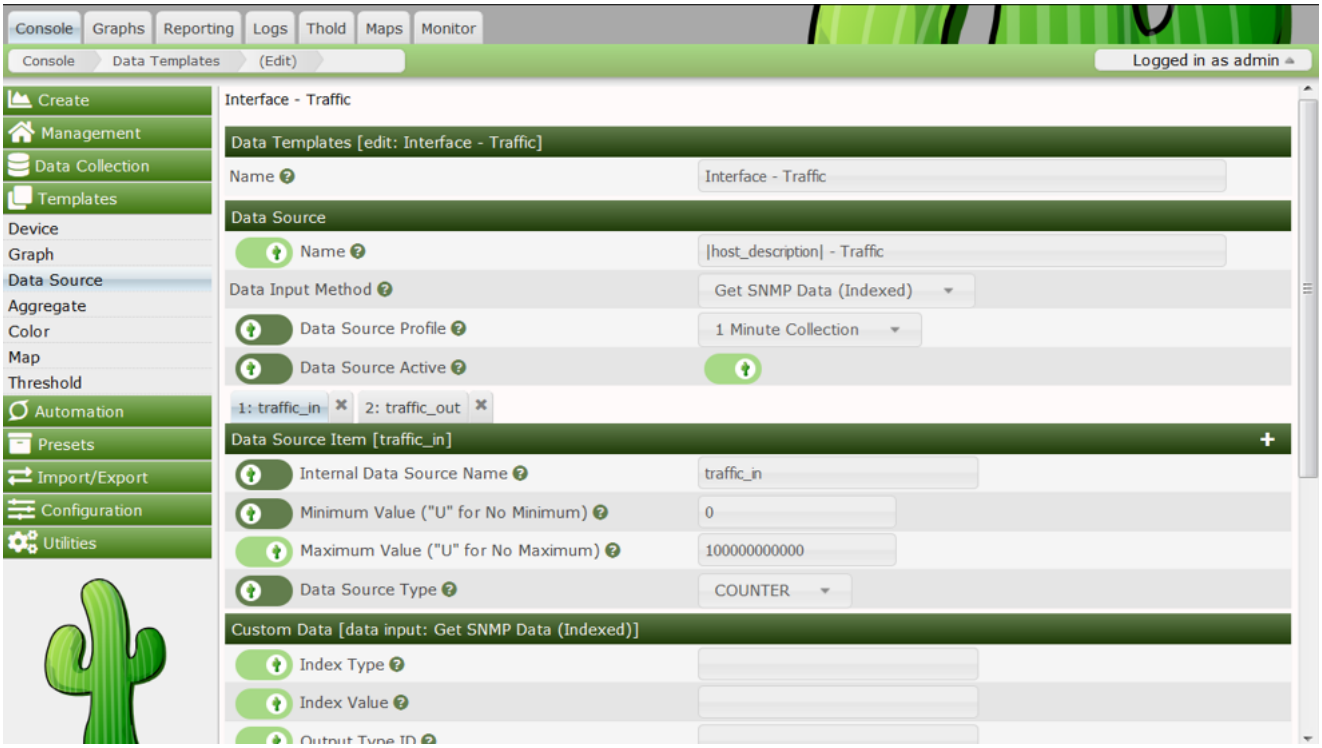
> **NOTE**: If you are using Cacti's **Automation** feature, there is presently no way for you to override these values. So, plan accordingly when engineering your Cacti deployment.

Table 13-1. Data Templates: Field Description of the Data Source Section

Name	Description
Name	The name of the **Data Source** that will be created by using this Template. You can use the keyword `host_description` enclosed by vertical bars in this field, which will be automatically substituted with the current host description of the **Device**. There are several replacement values available to you when using a Template.
Data Input Method	Here is where you tell Cacti how it is supposed to fetch data for this **Data Source**. There are several **Data Input Methods** that come with Cacti, and you can add your own by going to `Console > Data Collection > Data Input Methods`. If this RRDfile is being populated outside of Cacti, make sure to leave this field set to "None".
Data Source Profile	The **Data Source Profile** will define how often to collect data for the resulting **Data Source**, and how long to keep that data in the RRDfile.
Data Source Active	This is a quick and easy to tell Cacti to stop gathering data for this **Data Template**. The resulting **Data Source** can still be used on **Graphs**, but no data will be fed to it until it is made active again.

> **NOTE:** For most **Data Templates**, you should not have to check the Use Per-Data Source Value check-box for the name field. The Templating should handle this just fine. However, the Generic SNMP OID **Data Template** has this value checked to allow you to create Graphs and Data Sources on a piecemeal basis.

When you are finished filling in values for the **Data Template**, click Create and you will be presented with a screen similar to the image below.



Data Source Items

Just like a **Graph**, a **Data Source** can have more than one item. This is useful in situations where a script returns more than piece of data at one time and both need to appear on the same **Graph**. This also applies to **Data Queries**, so you can have a single **Data Template** that contains both Inbound and Outbound Traffic, rather than having to create a separate **Data Template** for each.

Table 13-2. Data source item fields

Name	Description
Internal Data Source Name	This is the name used by RRDtool to identify this particular data source within the RRDfile. The Cacti template editor accepts 1–19 letters, digits, or underscores. Some imported names can contain other RRDtool-valid characters, including hyphens, that this editor rejects. Use underscores for new templates; renaming an existing RRD data source requires a coordinated file and definition change.
Minimum Value	Here is where you specify the minimum value that is expected for this data source in the RRDfile. If a value lower than the minimum is given, it will be stored as Unknown (U).
Maximum Value	Here is where you specify the maximum value that is expected for this data source in the RRDfile. If a value higher than the maximum is given, it will be stored as Unknown (U). Note: It often makes sense to define a reasonable maximum value here to avoid spikes in case of a COUNTER wrap
Data Source Type	Choose a type supported by the collection method and installed RRDtool. See Data Source Templates for COUNTER, GAUGE, DERIVE, ABSOLUTE and version-dependent double-precision types, plus the current COMPUTE creation limitation.
Heartbeat	As defined by RRDtool: "The maximum amount of time that can pass before data is entered as "unknown". This field is usually '600' or 2 data gathering intervals".

Custom Data

Assuming you selected a **Data Input Method** on the previous screen, you should now be presented with a **Custom Data** form. It will show a single line for every single parameter required for that very **Data Input Method**. This is how the **Data Source** connects to the **Data Input Method** to provide all run time parameters.

Each **Custom Data** field is per-field Templatable as all of the other **Data Source** fields are. Even if you select the Use Per-Data Source Value check-box, it might be useful to specify a value that will be used as an "initial value" for any **Data Source** using the **Data template**.

Applying Data Templates to Data Sources

> **NOTE:** When changing parameters of a **Data Template**, existing RRDfiles will not be changed automatically. If this is required, you will have to apply `rrdtool tune` command to any related RRDfile manually.

Applying a **Data Template** to a **Data Source** is a very simple process. The first thing you must do is select the **Data Source** you want to apply the template to under **Data Sources**. Under the **Data Template** Actions drop-down, select the **Data Template** that you want to apply to the **Data Source** and click Save. Though this should be very rare in modern Cacti instances, it was fairly common before Cacti's Templating system matured.

Once a Template is applied to a Data Source, you will notice that you can only change values for the fields that you checked Use Per-Data Source Value for.

> **NOTE:** Eligible template and item fields propagate to attached database definitions unless configured per data source. This database propagation is separate from changing an existing RRD file. Inspect the actual RRD structure and use a backed-up repair or migration when its fields no longer match.

How To: Working with Templates

If you goto the following article How to Work with Templates. There you will find a step by step guide on how to work with variout Cacti Templates in general.

Graph Templates

In Cacti, a **Graph Template** provides a skeleton for an actual **Graph**. If you have many **Graphs** that share most of their characteristics, using a **Graph Template** makes perfect sense. After a **Graph** is attached to a particular **Graph Template**, all changes made to the **Graph Template** will propagate out to all of its **Graphs**, unless Use Per-Graph Value has been checked.

Creating a Graph Template

To create a new **Graph Template**, select **Graph** under `Console > Templates` menu selection, then click Add normally the plus sign in the upper right corner of the **Graph Template** interface.

The first thing you must do is give the **Graph Template** a name. This name has nothing to do with the graph title, but is what you will use to identify the **Graph Template** throughout Cacti. Second, you will notice a list of graph field names with Use Per-Graph Value check-boxes next to each one. The nice thing about Templates in Cacti is that you can choose whether to template each field on a per-field basis. If you leave the check-box unchecked, every **Graph** owned by the **Graph Template** will inherit its value from the Template. If the check-box is checked, every **Graph** owned by **Graph Template** will have its own value for that particular field.

Table 13-3. Field Description: Graph Templates

Name	Description
Title	The title of the **Graph** within Cacti and the title that will be printed on the actual **Graph** itself. **NOTE**: You can use the keyword `host_description`

Name	Description
	enclosed by vertical bars in this field, which will be automatically substituted with the current host description.
Image Format	Choose whether you would prefer to output your graph images in PNG or SVG.
Height	The height of the Graph Canvas in pixels. This is not the height of the overall Graph, just the Graph Canvas.
Width	The width of the Graph Canvas in pixels
Base Value	Should be set to 1024 for memory values and 1000 for traffic measurements.
Slope Mode	RRDtool graphs are composed of stair case curves by default. This is in line with the way RRDtool calculates its data. Some people favor a more "organic" look for their graphs.
Auto Scale	Enable auto scale for the graph. This option must be check to use the next two options. Upper/Lower limit values are ignored when using auto-scale since these boundaries are determined automatically.
Auto Scale Options	Choose whether you would like to use --alt-autoscale (ignoring given limits), --alt-autoscale-max (accepting a lower limit), --alt-autoscale-min (accepting an upper limit.
Logarithmic Scaling	Choose if you want logarithmic y-axis scaling.
Scientific Units for Logarithmic Scaling	For linear graphs, scientific units (in magnitudes of k=kilo, M=mega, ...) is default. But for logarithmic graphs, exponential notation is default. Choose if you want logarithmic y-axis scaling and scientific units.
Rigid Boundaries Mode	From the RRDtool manual "Normally rrdgraph will automatically expand the lower and upper limit if the graph contains a value outside the valid range. With this option you can disable this behavior".
Auto Padding	If you have ever created RRDtool based graphs manually, you may have realized how annoying it can be to get text to line up properly. With this option Cacti will do its best to make the columns on your graph legend line up. This option works best when your graph legend has a consistent number of columns.
Allow Graph Export	If you choose to use Cacti's static HTML/image export, you can choose whether you want each individual graph to be exported.
Upper Limit	The maximum value that will be displayed on the y-axis. This value is ignored is auto-scaling is turned on.
Lower Limit	The minimum value that will be displayed on the y-axis. This value is ignored is auto-scaling is turned on.
Base Value	Whether you want to base the y-axis labels on 1000 or 1024. This field will typically be set to 1024 for memory and 1000 for traffic measurements.
Unit Grid Value	Sets the unit value for the y-axis (--y-grid). You should probably keep the unit short as to keep it readable.
Unit Exponent Value	Sets the 10^e scaling of the y-axis. Valid values for this field are between -18 and 18. For example, you could use 3 to display everything in 'k' (kilo) or -6 to display everything in 'u' (micro).
Vertical Label	The text to print on the left edge of the graph. Usually is the units the data on the graph is measured in.

Auto-scaling Options Explained

Relative to Auto-scale Options, the RRDtool graph manual says:

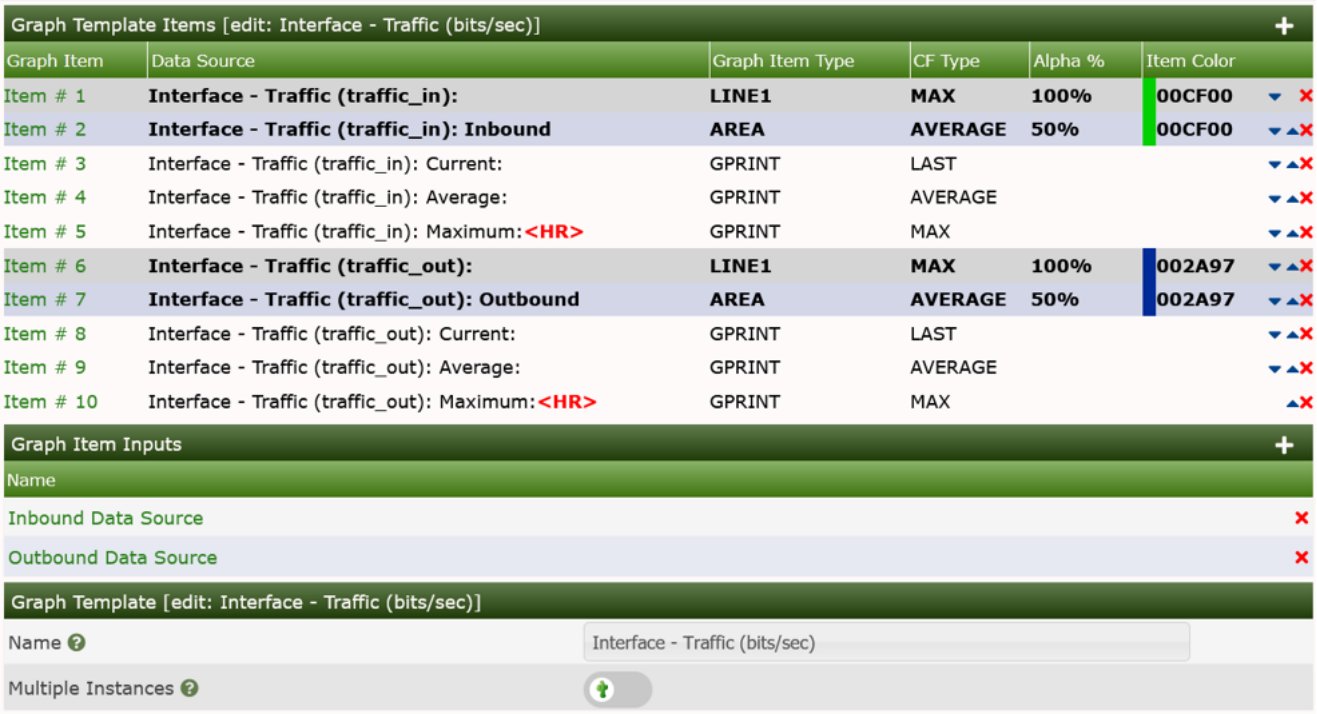
- Limits `[-u|--upper-limit value] [-l|--lower-limit value] [-r|--rigid]` By default the graph will be auto-scaling so that it will adjust the y-axis to the range of the data. You can change this behavior by explicitly setting the limits. The displayed y-axis will then range at least from lower-limit to upper-limit. Auto-scaling will still permit those boundaries to be stretched unless the rigid option is set.
- `[-A|--alt-autoscale]` Sometimes the default algorithm for selecting the y-axis scale is not satisfactory. Normally the scale is selected from a predefined set of ranges and this fails miserably when you need to graph something like "260 + 0.001 * sin(x)". This option calculates the minimum and maximum y-axis from the actual minimum and maximum data values. Our example would display slightly less than "260-0.001" to slightly more than "260+0.001".
- `[-J|--alt-autoscale-min]` Where "--alt-autoscale" will modify both the absolute maximum AND minimum values, this option will only affect the minimum value. The maximum value, if not defined on the command line, will be 0. This option can be useful when graphing router traffic when the WAN line uses compression, and thus the throughput may be higher than the WAN line speed.
- `[-M|--alt-autoscale-max]` Where "--alt-autoscale" will modify both the absolute maximum AND minimum values, this option will only affect the maximum value. The minimum value, if not defined on the command line, will be 0. This option can be useful when graphing router traffic when the WAN line uses compression, and thus the throughput may be higher than the WAN line speed.

When you are finished filling in values for the **Graph Template**, click Create and you will be presented with a page similar to the image below. The example below is from a completed Traffic Graph Template.

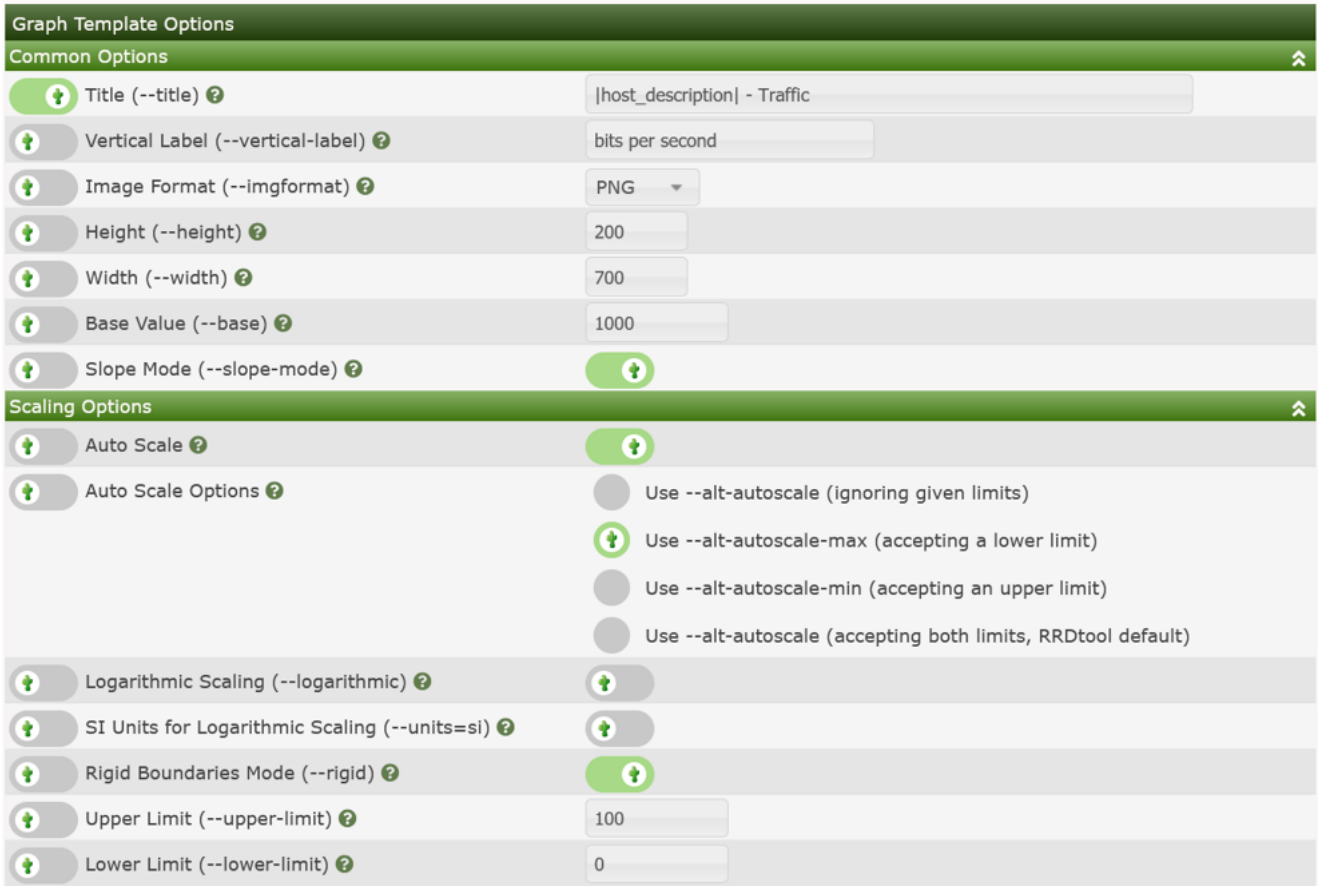
In this first image, you can see what elements will be painted on the **Graph** Canvas. They are represented by **LINEX**, **AREA** and **STACK** elements. The **Graph Items** listed as **GPRINT** and **COMMENT** appear in the Legend Area of the **Graph**.

The **Graph Input Items** are the Cacti names for the Data Sources that appear on the **Graph** as well as other items that can be substituted during **Graph** Creation.

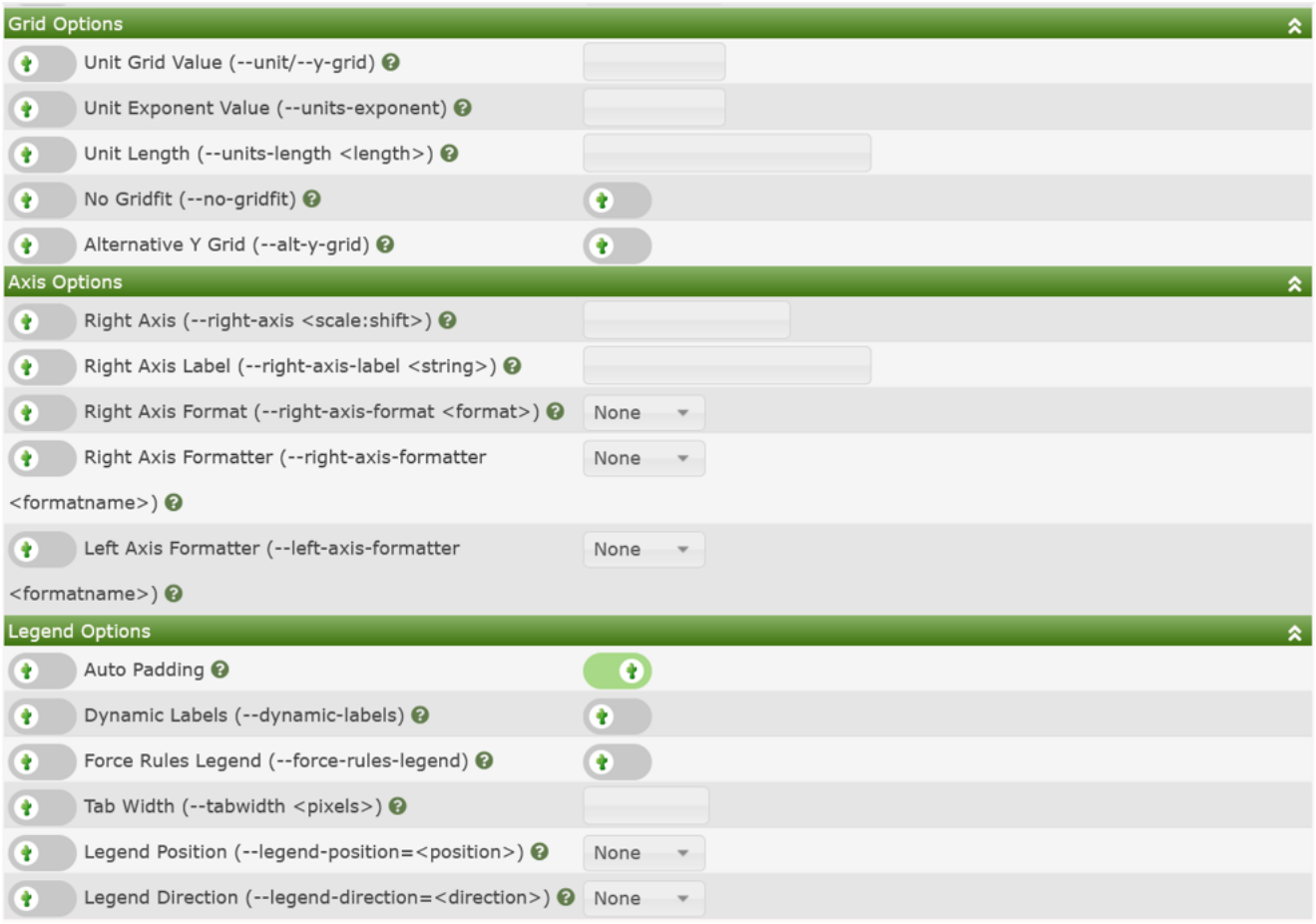
The `Multiple Instances` field is a parameter to let Cacti know that it should allow you to create more than one **Graph** per **Device** using the **Graph Template**.



The **Common Options** are options that most every **Graph Template** included. The **Scaling Options** are also very common, but related to **Graph** scaling only.



The **Grid Options**, **Axis Options**, and **Legend Options** are must less common but are available in Cacti should you choose to use them.



Graph Items

The first thing you should do is create **Graph Items** for a **Graph Template**, just like for a stand alone **Graph**. One difference you will notice is that the **Data Sources** drop-down will contain a list of **Data Template** items rather than **Data Source Items**. It is important that Cacti can make this association here, so that Cacti doesn't have to make unnecessary assumptions later.

Table 13-4. Field Description: Graph Template Items

Name	Description
Data Source	If this graph item is to represent some sort of data, you must select a your data source here. Keep in mind that not all graph items have a data source. Graph item types such as COMMENT, VRULE, and HRULE typically do not.
Color	Depending on the graph item type, you can select a color for the graph item. This field only applies to the graph item types AREA, STACK, LINE1, LINE2, and LINE3.
Opacity/Alpha Channel	For a colored graph item, you may optionally select an opacity (alpha channel). This option is not available with RRDtool-1.0.x.
Graph Item Type	This field is important because it defines what kind of graph item this is. Types such as AREA, STACK, LINE1, LINE2, and LINE3 are used to represent data on the graph, while COMMENT and GPRINT are used put on the legend. The LEGEND type is specific to Cacti only and can be used if you want to Cacti to automatically create a GPRINT-LAST/GPRINT-AVERAGE/GPRINT-MAXIMUM graph item for you. Note: You must always put an AREA item before using STACK or your graph will not render.
Consolidation Function	This tells RRDtool which consolidation function to use when representing this data on the graph. You will typically use AVERAGE for most things on the graph area, and LAST/MAXIMUM as well for GPRINT items.
CDEF Function	

Name	Description
	If you want to apply a CDEF function to the graph item, select one here. Check out the CDEF section of the manual for more information.
Value	This field is only used with the HRULE/VRULE graph item types. Type any valid integer to draw the line at for HRULE or the time of the day HH:MM for VRULE.
GPRINT Type	If this item is a GPRINT, you can choose how you want the number to be formatted. You can add your own in the GPRINT Presets section of Cacti.
Text Format	You can enter text to be displayed on the legend here. This field is applicable for all graph item types except for the virtual LEGEND type.
Hard Return	Check this box to force graph items onto the next line.

Graph Item Inputs

After creating **Graph Items** for your **Graph Template**, you will need to create some **Graph Item Inputs**. **Graph Item Inputs** are unique to **Graph Templates** because of the large number of items they sometimes contain. **Graph Item Inputs** enable you to take one **Graph Item** field, and associate it with multiple **Graph Items**. **Graph Item Input** fields are generally created automatically as you add **Graph Items** associated with a **Data Source**, like a LINE or AREA or GPRINT.

To create a new **Graph Item Input** manually, click Add on the right of the **Graph Item Inputs** form. There are various fields that must be filled in for every graph item input:

Table 13-5. Field Description: Graph Template Items

Name	Description
Name	This will be the name used to identify the **Graph Item Input** on both the **Graph Template** and **Graph** edit pages.
Description	(Optional) This description will be displayed on the **Graph** edit page of any **Graph** using this **Graph Template** if specified.
Field Type	You must choose the field that you are going to associate with one or more **Graph Item**.
Associated Graph Items	Choose one or more **Graph Items** to associate with the field selected for "Field Type". When the user specifies a value for the field, it will be applied to all of the items you select here.

Applying Graph Templates to Graphs

Applying a **Graph Template** to an existing **Graph** is a very simple process, though very uncommon in a modern Cacti install. The first thing you must do is select the **Graph** you want to apply the template to under **Graph Management**. Under the **Graph Template** Action drop-down, select the **Graph Template** that you want to apply to the **Graph** and click Save. If this is a new **Graph** or the **Graph** and **Graph Template** contain an equal number of **Graph Items**, the **Graph Template** will be automatically applied. If number of **Graph Items** varies from the **Graph** to the target **Graph Template**, you will be prompted with a warning that your **Graph** will be changed, and not always for the better.

Once the **Graph Template** is applied to the existing **Graph**, you will notice that you can only change values for the fields that you checked Use Per-Graph Value for. You will also notice a new box, called **Graph Item Inputs**. This is where you can specify values for the **Graph Item Inputs** that you defined in the **Graph Template**. The values specified here will be applied to each **Graph Item** tied to the **Graph Item Input**.

> **NOTE:** Any time a change is made to the **Graph Template**, it will be automatically propagated to the **Graphs** associated to it.

Device Templates

Device Templates in Cacti, often times referred to as Host Templates serve as a way to group a set of common **Graph Templates** and **Data Queries** to **Devices** of a certain class, say a Cisco Router or Linux host using Net-SNMP. It really accelerates the deployment of a Cacti system across and enterprise of Devices.

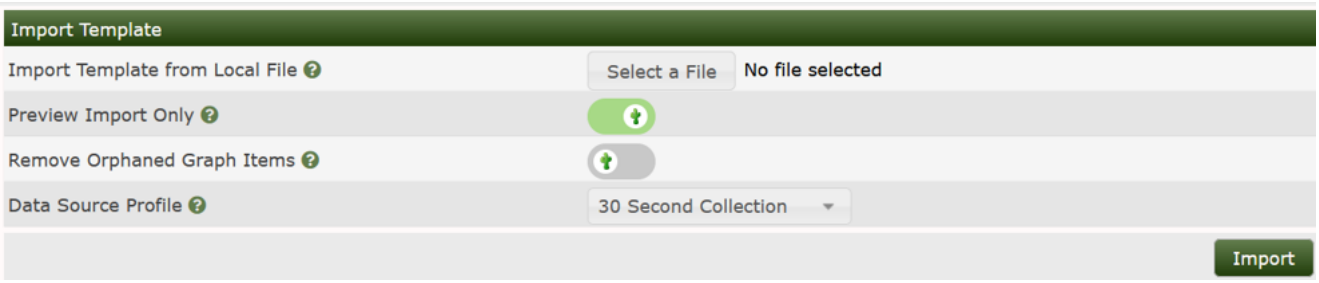
Adding a Device Template

To create a new **Device Template** in Cacti, select the **Device** option under the `Console > Templates` menu selection. Once on that page, click Add on the right. Type a unique name for the **Device Template** and click the Create button. You will be redirected back to the edit page with the `Associated Graph Templates` and `Associated Data Queries` forms empty. These two forms allow you to associate **Graph Templates** and/or **Data Queries** with the **Device Template**. Simply select something from the drop-down menu and click Add to associate it with your **Device Template**.

In the case below, we show the Net-SNMP **Device Template**. Note the variety of Graph Templates and Data Queries available for this Class of **Devices**.

Import Templates

Assume, you're searching for a specific set of templates to monitor a special type of device. Apart from designing templates from scratch, there's a good chance to find a solution in the Scripts and Templates Forum. The set of templates is usually provided as a single XML file holding all required definitions for a data template and a graph template. Depending on the goal of the original author, he/she may have provided a **Device Template** as well as part of this XML file. Note that after you **Import Templates**, you may also have to copy a Resource XML file to the appropriate <path_cacti>/resource directory, and a script to the <path_cacti>/scripts directory. However, also note that in the current version of Cacti, **Packages** are now importable as well. Signed **Packages** can include those files, subject to the import selection and permissions. See Import Packages; plain template XML and signed `.xml.gz` packages use different import workflows.

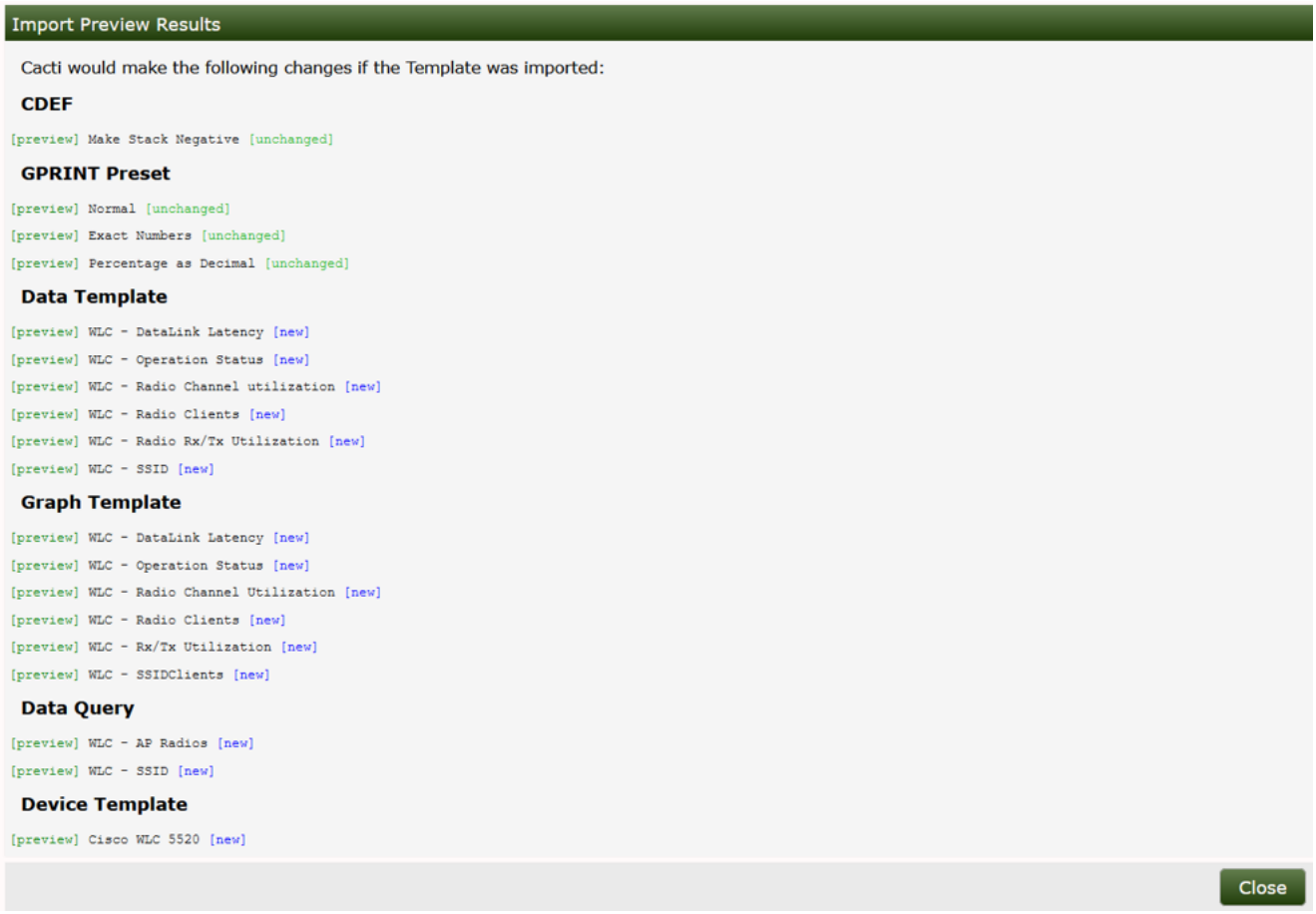


If the XML file was downloaded, simply select that file from the Downloads location on your local system. Cacti no longer supports pasting the XML into the browser window.

Please pay attention to the `Data Source Profile` field. By default, cacti will attempt to use your default **Data Source Profile**. However, you can change that at import time to use an alternate **Data Source Profile** or even the one from the **Template** itself, if it includes one.

Review the automatically generated preview before importing. Use **Remove Orphaned Graph Items** only when you intend to delete items absent from the incoming template, including those on attached graphs. It can remove local customizations; it is not a general requirement for every update. See Template Import for selection and override details.

In the image below, you can see the output of a `Preview` of the Cisco WLC 5520 **Device Template**. Preview shows proposed template changes. Import compatibility repair can still update system data-input metadata during preview; it is not a strictly read-only audit.

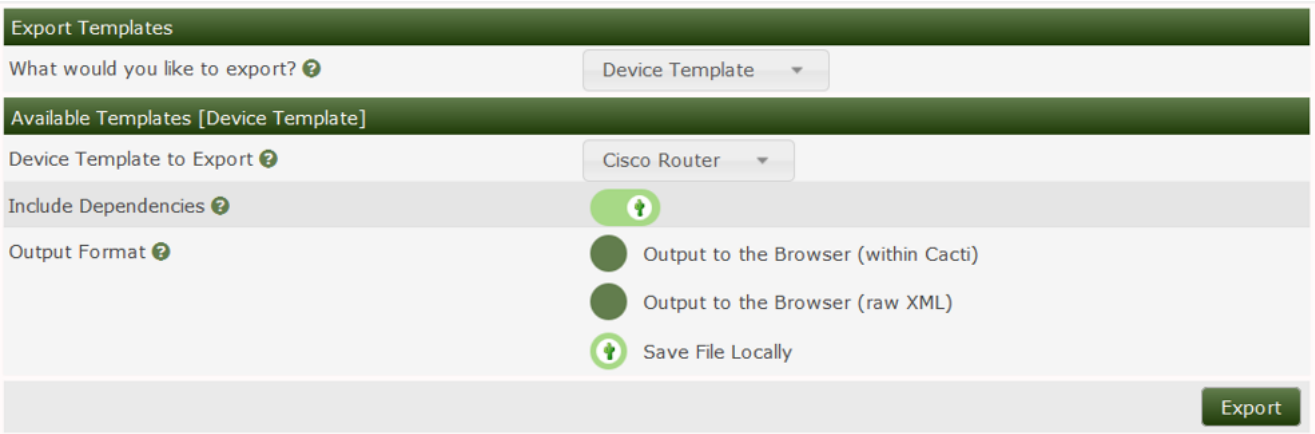


As mentioned previously, for a single OID based template, **Importing** the **Template** is all that will be required. However, for a script based template, the author will provide the script that has to be downloaded to the Cacti `./scripts` directory and for a **Data Query** based **Template**, a second XML file holding the Data Query definitions will have to be downloaded to the appropriate directory under the `./resource` directory.

Cacti checks the XML object's hash compatibility code against the installed version's mapping. Several releases can share a code, so it does not identify an exact exporter release. Unknown or newer codes can be rejected. See Determine a Template Version before troubleshooting compatibility; do not change hash prefixes to force an import.

Export Templates

Now that you know how to import, you may want to know in which way to export as well. Selecting the Export Templates gives



You may select to export a graph template, a data template, a **Device Template** or a **Data Query**. When selecting `Include Dependencies`, e.g. a **Device Template** will include all referred templates (**Graph Template**, **Data Template**, if defined, **Data Query**). Output may be written to the browser or to a file for uploading.

Copyright (c) 2004-2026 The Cacti Group

How To Work with Templates

We will follow the following process in this example:

- **Create Data Template** - Defines how we store the data in the RRDfile
- **Create Graph Template** - Defines how we graph the data in the RRDfile
- **Device Template** - Defines which Graphs to associate with a Device Type

Data Template Creation

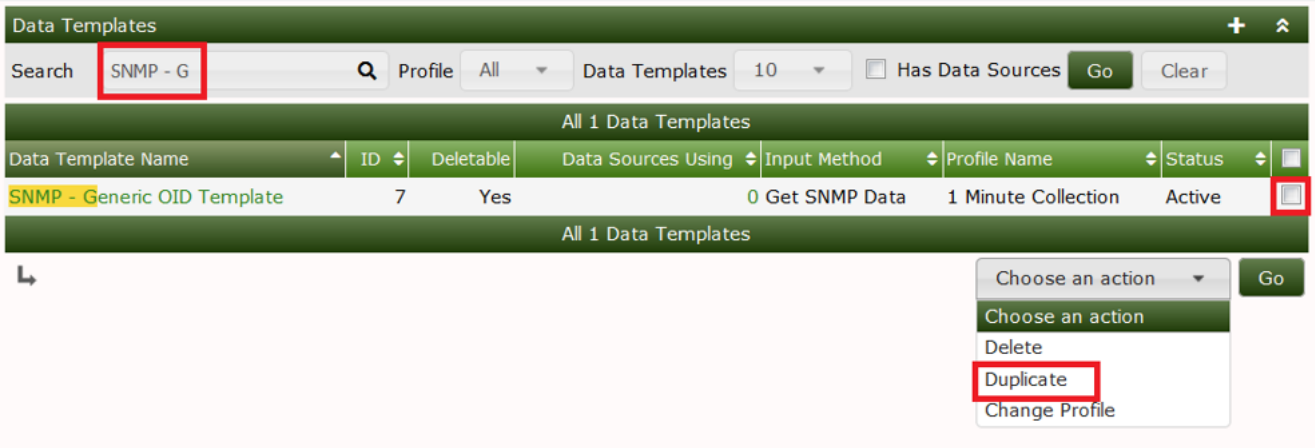
For this task, let's stick to SNMP stuff. For you to be able to reproduce this example, I've chosen the **UDP** information of the IP MIB.

```
snmpwalk -c <community string> -v1 <device> udp
UDP-MIB::udpInDatagrams.0 = Counter32: 7675
UDP-MIB::udpNoPorts.0 = Counter32: 128
UDP-MIB::udpInErrors.0 = Counter32: 0
UDP-MIB::udpOutDatagrams.0 = Counter32: 8406
```

As cacti does not use the MIBs but pure ASN.1 OIDs, let's search the OID used as `udpInDatagrams`:

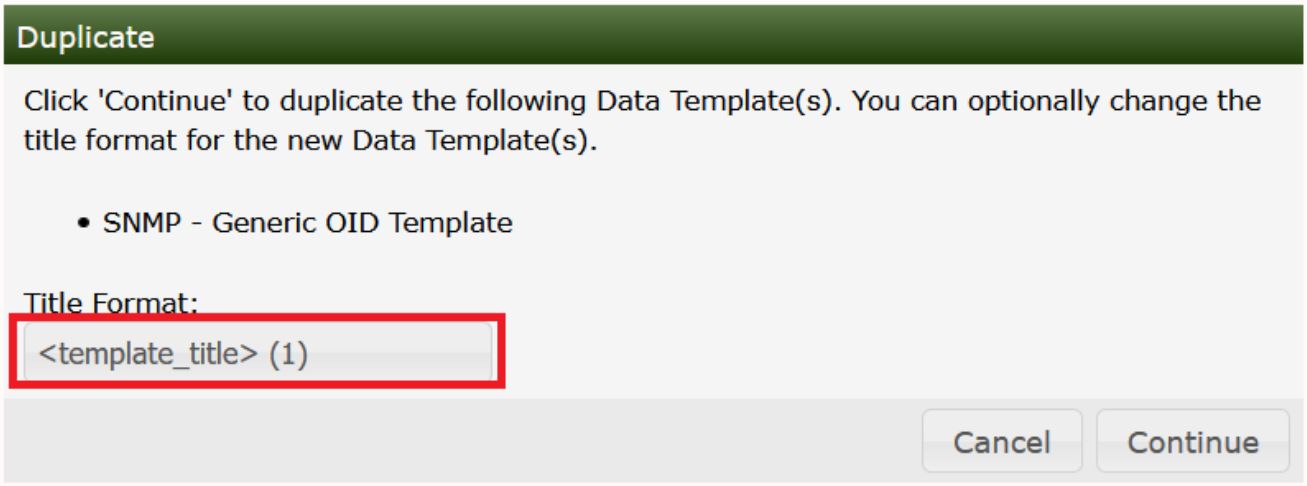
```
snmpwalk -c <community string> -v1 -On <device> udp
.1.3.6.1.2.1.7.1.0 = Counter32: 7778
.1.3.6.1.2.1.7.2.0 = Counter32: 129
.1.3.6.1.2.1.7.3.0 = Counter32: 0
.1.3.6.1.2.1.7.4.0 = Counter32: 8514
```

The needed OID is .1.3.6.1.2.1.7.1.0. Now learn how to enter this into a new Cacti Data Template: Please proceed to Data Templates and filter for SNMP. Check the SNMP - Generic OID Template

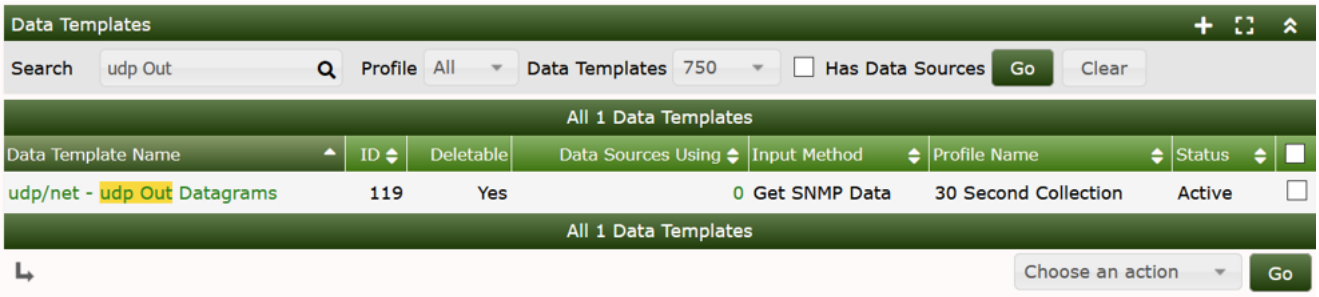


After clicking Go, you're prompted with a new page to enter `udp/net - udp In Datagrams` as the `Title Format` for the new **Data Template**:

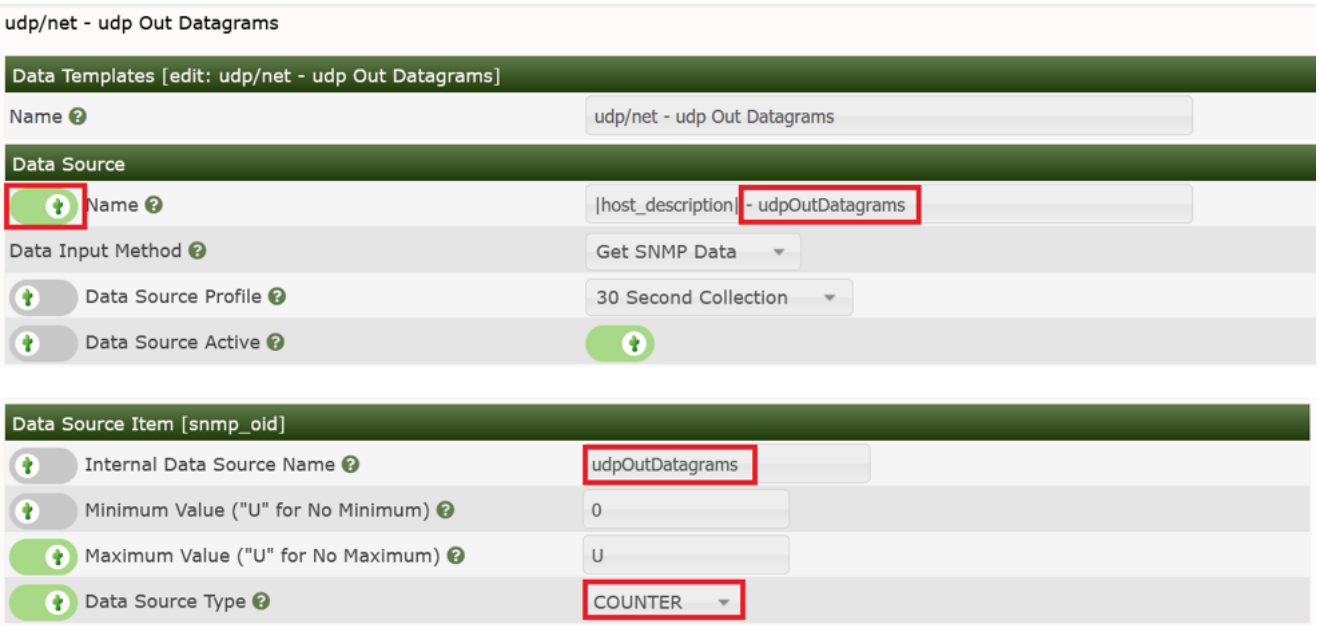
The screenshots are from an older example; follow the names and OIDs in this text for each copy.



Due to the filter defined above, you won't see the new Template at once, so please enter `udp In` as a new filter to find:



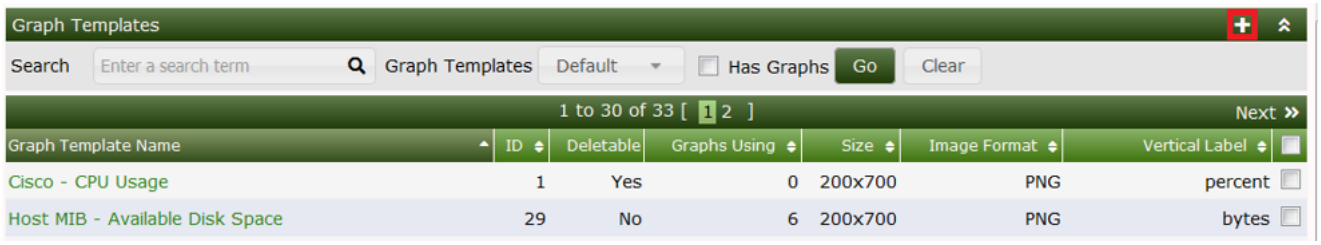
Edit the copy for `udpInDatagrams`, using OID `.1.3.6.1.2.1.7.1.0`, a recognizable internal name such as `udp_in`, and the `COUNTER` data-source type. The historical screenshots show the field locations and mix older labels. Use the name/OID pair above for the inbound copy, then `udpOutDatagrams` and `.1.3.6.1.2.1.7.4.0` for the second, outbound template:



Set **Maximum Value** to `U` for no upper limit, as shown in the screenshot. Do not use `0`: with the illustrated minimum of `0`, Cacti rejects that range. Use a finite upper limit only when it represents a valid bound for the metric. And you saw the OID `.1.3.6.1.2.1.7.1.0` from above, didn't you? Please copy another one for OID `.1.3.6.1.2.1.7.4.0`, using the description **udpOutDatagrams**

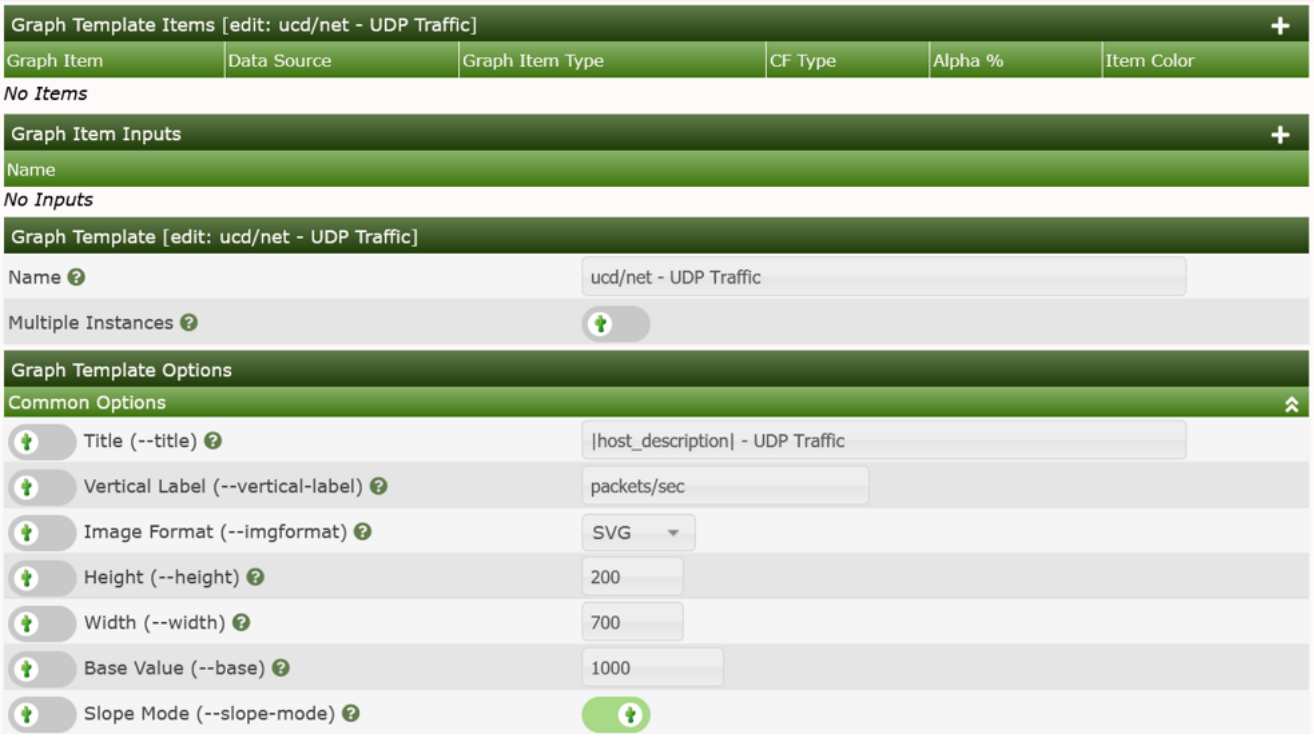
Graph Template Creation

Now let's generate the **Graph Template** for those already generated Data Templates. Please go to **Graph Templates** and **Add** a new one:

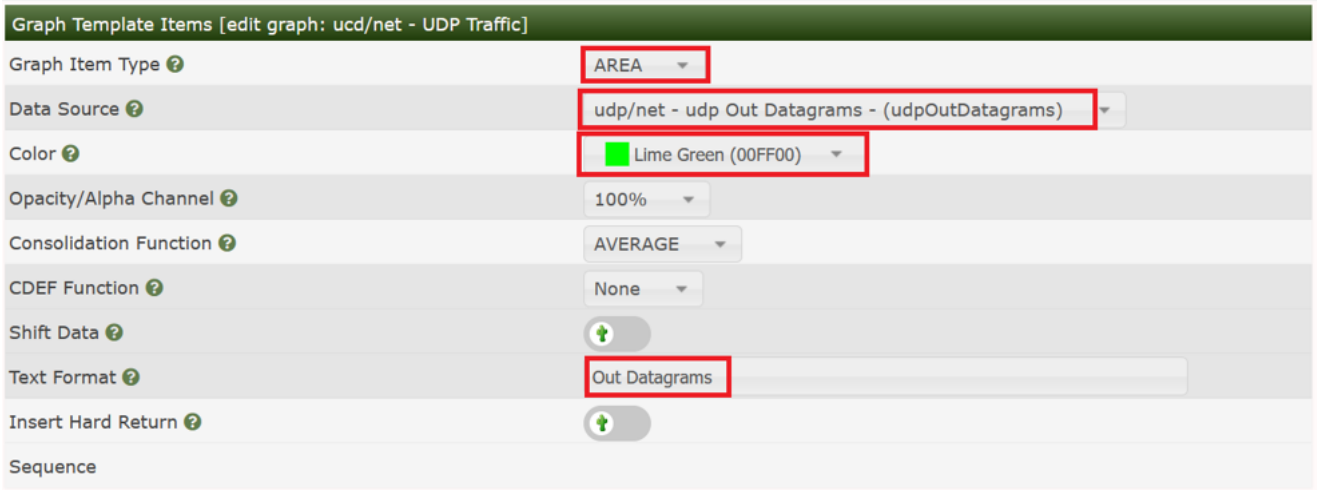


Now you have to fill in some global parameters:

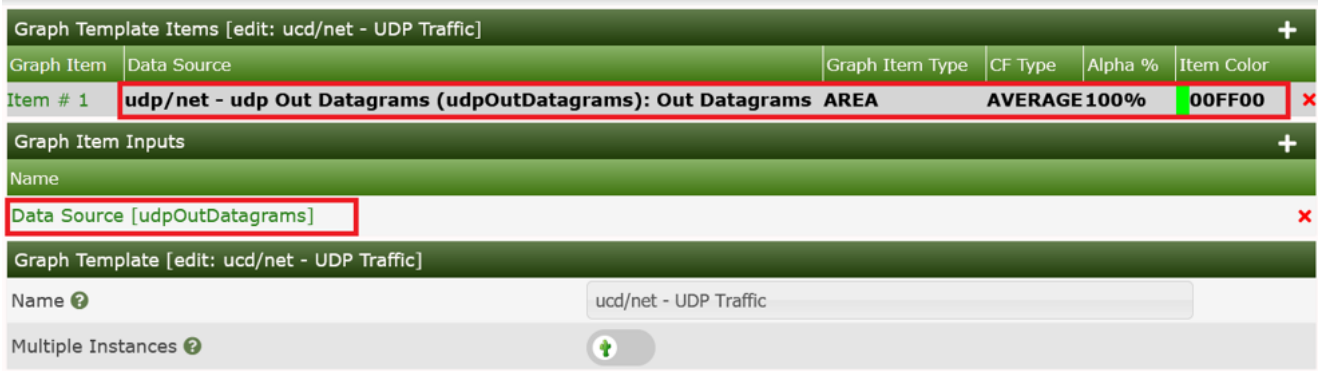
and **Create** to see:



Now let's add some **Graph Template Graph Items**. They will specify, which **Data Sources** defined by some **Data Template** should be displayed on the **Graph**. Please click Add as shown on the last image:

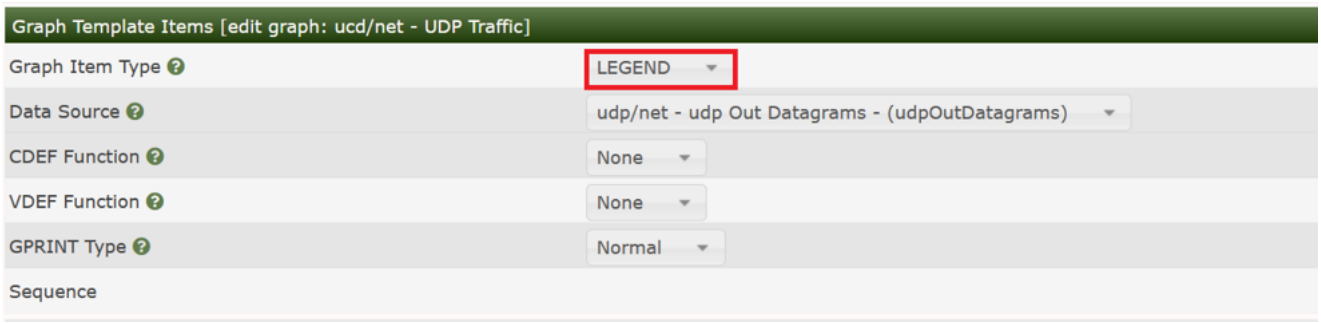


Now click Save to see the resulting image below. Not that both the **Graph Item** and the **Graph Item Input** have been created.

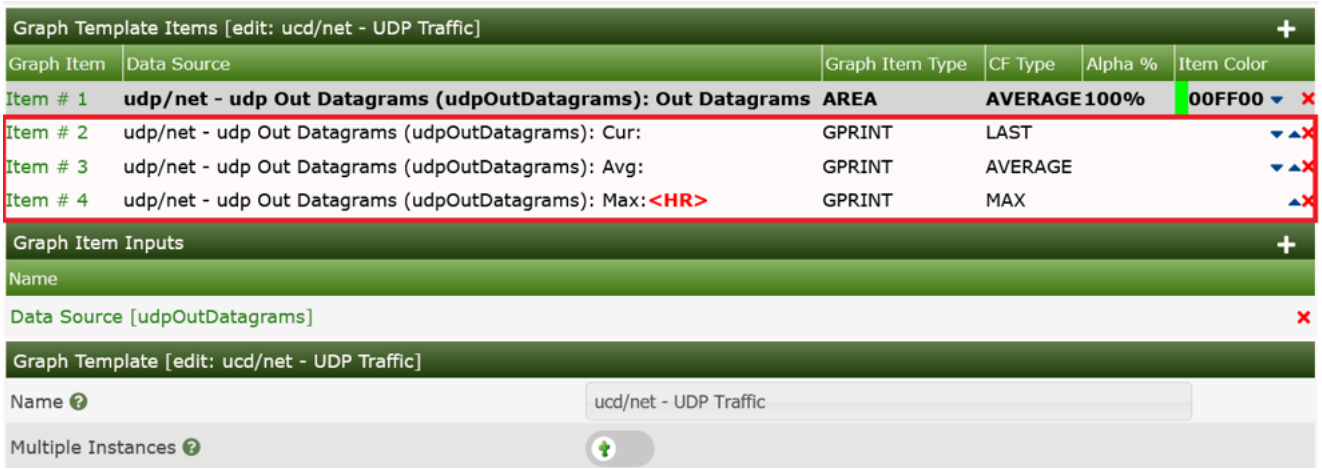


Next, we have to add a Legend in order for people viewing the **Graph** in the future to obtain numeric data about the **Graph** contents. We will create a very simple one, though Graph Legends can be a very complex subject.

To add the Graph Legend, simply press the Add button again and fill out the **Graph Item** as shown in the image below. You should note that for this second **Graph Item**, it remembered the previous **Data Source**.



Press Save to see three legend items created in one step!



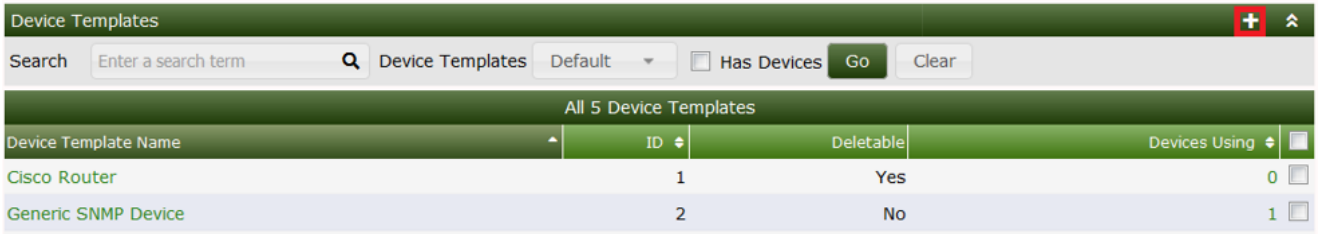
Now we could have continued to add the UDP Datagrams In by adding a second RRDfile Data Source to our **Data Template**, but we will skip that for this exercise and continue now that our **Graph Template** is reasonably completed.

At this point, since we only added **Graph Items** to the **Graph Template** there is no need to resave it at this point. We can simply continue to the next step.

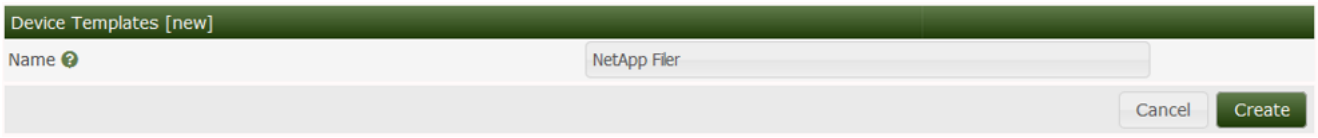
Now, you may add this new **Graph Template** to any **Devices** that responds to the udp OID we specified in the **Data Template**. But in this case, please wait a moment. Let's first proceed to the **Device Template** and use this new **Graph Template** for our first own **Device Template**.

Device Template Creation

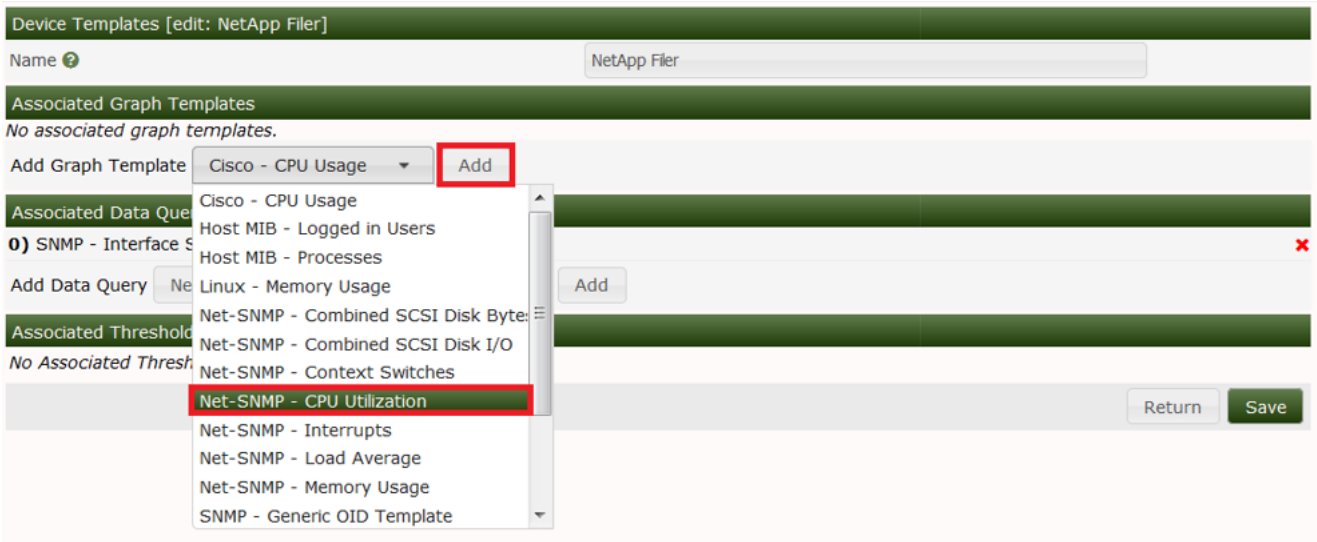
The next task is to create a new **Device Template**. So, select **Device** under `Console > Templates` and press the Add button as in previous examples. When you do that, you will be presented with an page that resembles the one below.



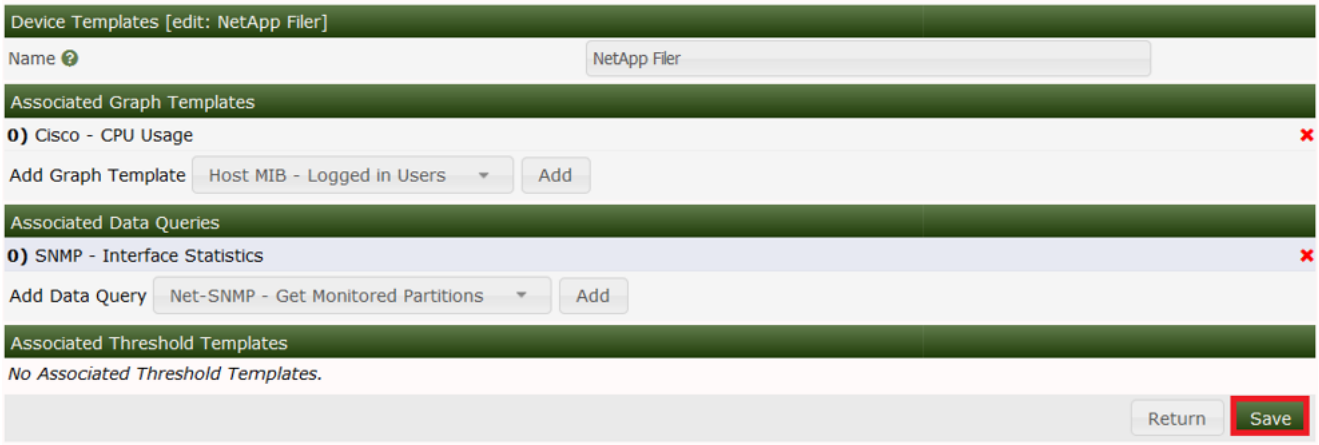
and fill in the name of this new Template:



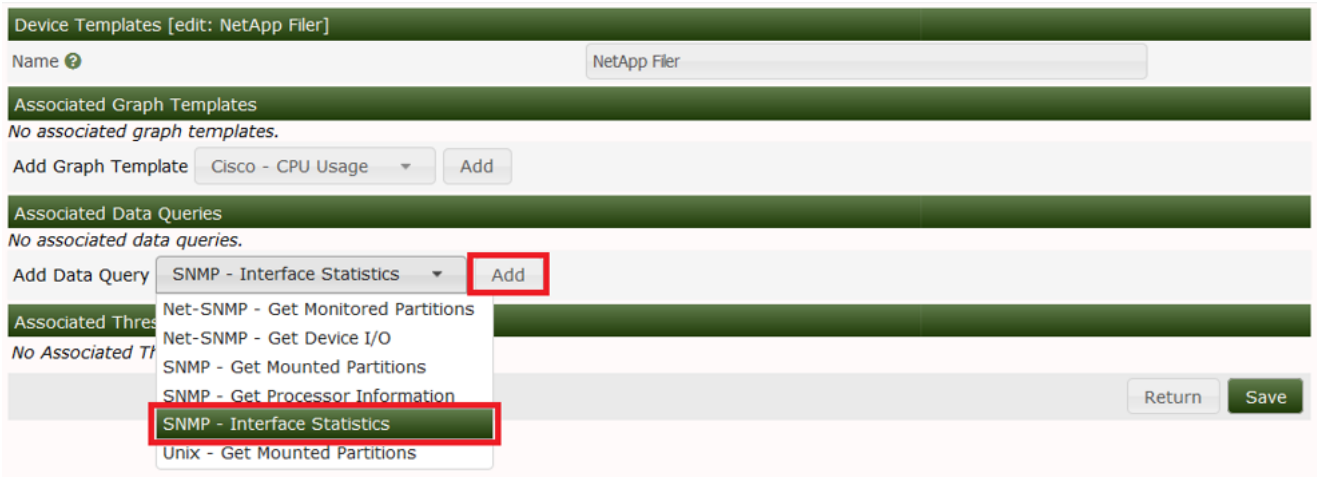
Now you'll find two sections added. First, let's deal with **Associated Graph Templates**. The Add **Graph template** select drop-down contains all existing **Graph Templates**. In the images, you will see that we are creating a **Device Template** for a NetApp Filer and adding some common Graph Templates and Data Queries from the Cacti built-in collection.



and Add it:



Next, let's add the Data Query already selected above:



Keep in mind, this is one of the more simple Template examples. Topics such as **Data Queries**, **Data Input Methods**, and **Script Server** add a few steps to the Template creation process.

Copyright (c) 2004-2026 The Cacti Group

Device Templates

Device Templates are Cacti objects that allow you to define classes of **Devices** that include one or more **Graph Templates**, **Data Queries** and other **Plugin** related object types.

The purpose of **Device Templates** is to simplify the **Automation** process by pre-defining the **Graphs** that should be created for every **Device** that is added to Cacti. They work in conjunction with **Automation Templates** so that when Cacti discovers a **Network** it knows what **Graphs** to create for each **Device**.

The **Device Templates** main screen looks like the image below:

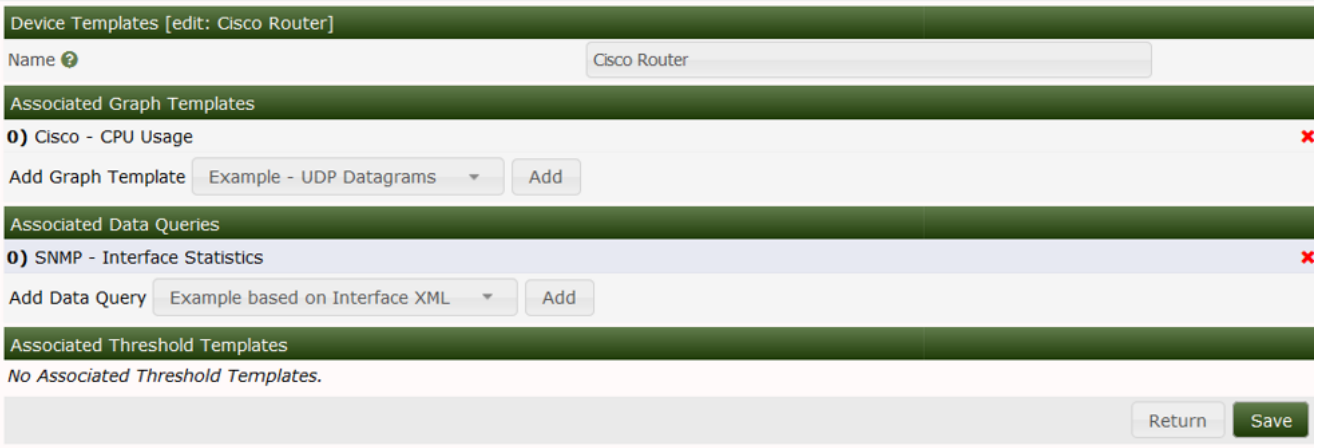
From this page, you can see the title of each **Device Template**, and its ID, which is important for the Cacti CLI scripts. You can see if the **Device Template** can be removed, and the number of **Devices** using the **Device Template**. Templates that are used by **Devices** can not be removed and therefore if you attempt to remove one of these Templates, you will receive an error message.

From the drop down there are the following options:

Option	Description
Delete	Remove the **Device Template** if it's *Deletable*
Duplicate	Make an exact copy of the **Device Template**.
Sync Devices	Update all **Devices** using this **Device Template** with the latest definition, adding, but not removing **Graph Templates**, and **Data Queries**.
Archive	Store a copy of the current **Device Template** definition with an optional *Archive Note*. Archives are listed on the *Archives* tab of the same page.
Download	Download the **Device Template** as a Cacti **Package**.

Archive and *Download* were added in Cacti 1.3; Cacti 1.2.x offers only the first three options. They appear only once the package signing keys exist, which `cli/genkey.php` creates.

When editing a **Device Template**, you will see the page as displayed below. From this page, you can add and remove **Graph Templates**, **Data Queries**, and other **Plugin** objects. In the image below, you can see that the *Cisco Router* has one **Graph Template**, *Cisco - CPU Usage*, and one **Data Query**, *SNMP - Interface Statistics*. There are no **Threshold Templates** defined on the system, so there is no way to select one.



To add a **Graph Template** or **Data Query** to the **Device Template**, simply select it from the drop down, and press the *Add* button. There is no need to *Save* afterwards. To remove one of these items, simply press the x glyph to the right of the desired **Graph Template** or **Data Query**.

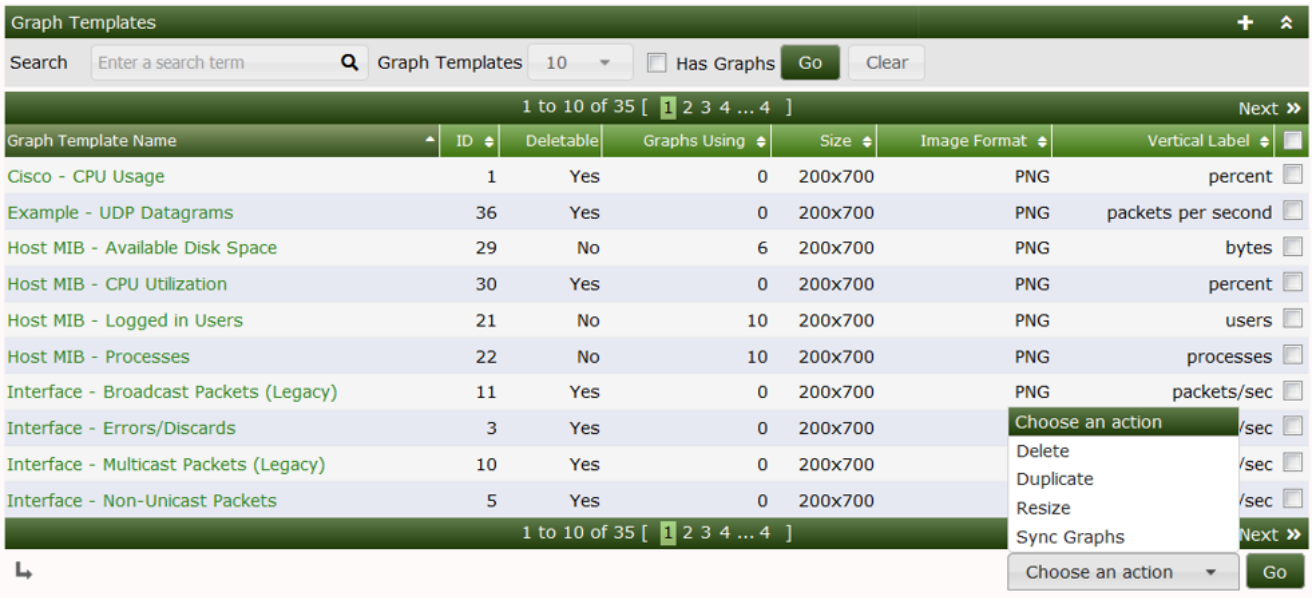
Copyright (c) 2004-2026 The Cacti Group

Graph Templates

Graph Templates are Cacti objects that allows you to define how RRDtool is to render a Cacti **Graph**. Most RRDtool options are supported including CDEF's and VDEF's, Left and Right Axis, Ticks and Dashes, multiple Auto Scaling, Grid, and Legend Options.

The purpose of **Graph Templates** is to simplify the **Automation** process by pre-defining the layout of **Graphs** for various metrics that are to be monitored in Cacti. When used in Conjunction with Cacti's **Graph Rules**, you can automatically create just about any **Data Query** based **Graph** during Cacti's **Network Discovery** process.

The **Graph Templates** main screen looks like the image below:



The list shows template IDs, the number of attached graphs, default size, image format and vertical label. IDs are useful for CLI operations. Deletability is based on referencing graph instances, not simply on a device-template association. A template with attached graphs is shown as non-deletable.

The Actions menu contains:

Table 1. Graph template actions

Action	Effect
Delete	Remove a deletable template.
Duplicate	Create a separate copy of the template.
Change Settings	Apply selected common settings, such as graph dimensions. Review the confirmation form before applying them.
Full Sync Graphs	Reconcile attached graphs with the selected template's full definition. Use when quick synchronization is insufficient.
Quick Sync Graphs	Repair graph-item differences with less work than a full synchronization. Try this first for item mismatches.

Synchronization changes existing graph definitions. Back up relevant database objects and review local graph inputs before applying a fleet-wide change.

The editor groups its fields into these sections:

Table 2. Graph template sections

Section	Purpose
Graph Template Items	Ordered drawing and legend items.
Graph Item Inputs	Fields applied to one or more items; data-source inputs are normally created automatically.
Graph Template	Name, multiple-instance policy and data-source testing.
Common Options	Title, vertical label, image format, dimensions, base and slope mode.
Scaling Options	Automatic scaling and optional upper/lower bounds.
Grid Options	Canvas grid rendering.
Axis Options	Right-axis conversion and formatting.
Legend Options	Legend placement and formatting.

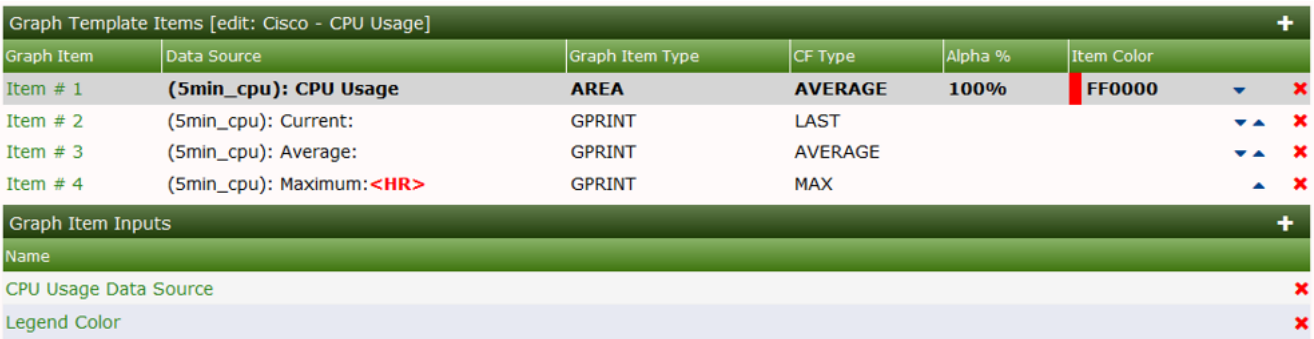
Each of these sections are displayed below for reference. For more information on how to use these options, please visit the RRDtool Website.

The *Graph Items* make up what is draw within the canvas of the Graph. There are several *Graph Item* types including:

Table 3. Common graph item types

Type	Purpose
AREA	Fill a plotted area.
AREA:STACK	Stack an area on a preceding plotted item.
COMMENT	Print legend text, including supported replacement variables.
GPRINT	Print a value with a GPRINT preset and applicable CDEF or VDEF.
GPRINT:AVERAGE / LAST / MAX / MIN	Print a value using the selected consolidation function.
HRULE	Draw a horizontal rule using the configured value.
LEGEND	Create Current, Average and Maximum GPRINT items.
LEGEND_CAMM	Create Current, Average, Minimum and Maximum GPRINT items.
LINE1 / LINE2 / LINE3	Draw a line of the selected width.
LINE:STACK	Stack a line on a preceding plotted item.
TEXTALIGN	Set alignment for subsequent text.
VRULE	Draw a vertical rule at a time position.

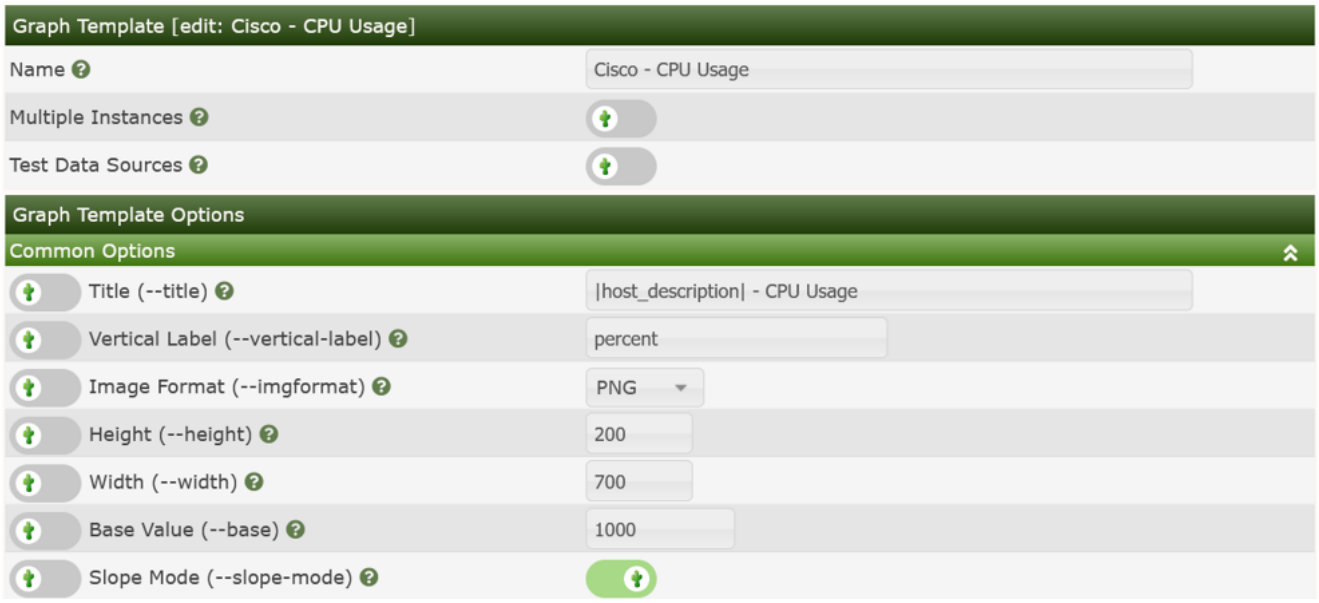
The canvas is painted from the first *Graph Item* till the last with each successive *Graph Item* rendered on top of the previous. Please keep this in mind when creating a **Graph Template**.



Graph Template - General and Common Options

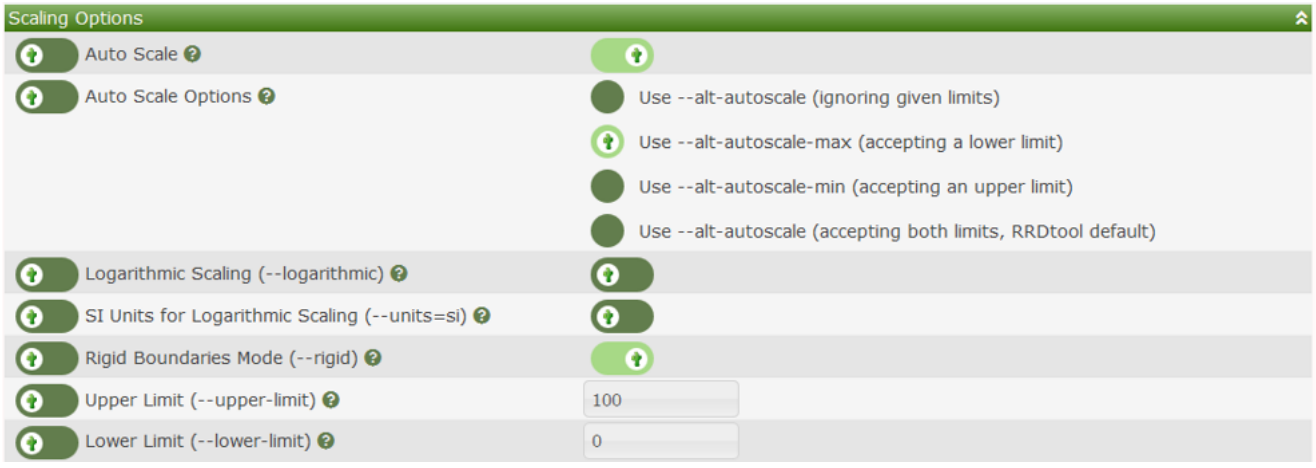
- **Name** - Identifies the template in Cacti; it is separate from the graph's title pattern. A prefix such as `Net-SNMP` can help organize related templates.
- **Class** - Categorizes the template using the available graph-template classes.
- **Version** - The author's template version. This is separate from the Cacti XML hash compatibility code.
- **Multiple Instances** - Allow more than one graph of this template on a device through the New Graphs interface.
- **Test Data Sources** - Require a valid data-source response before creating a graph. This helps a generic device template select applicable measurements. Long-running scripts can increase graph-creation time when testing is enabled.

Common options set the title pattern, vertical label, dimensions and image format. Use base `1000` for decimal scaling, such as traffic, and `1024` for binary scaling, such as memory. Decimal MB and binary MiB are different unit conventions; MiB means mebibytes. The base controls display scaling, not a conversion of stored measurements. **Slope Mode** smooths the rendered lines.



Graph Template - Scaling Options

The *Scaling Options* allow the Administrator to apply either *Rigid* or *Auto Scaling* settings to the resulting *Graph*. These options are fairly self explanatory. However, you can always view the RRDtool Documentation online for more information.

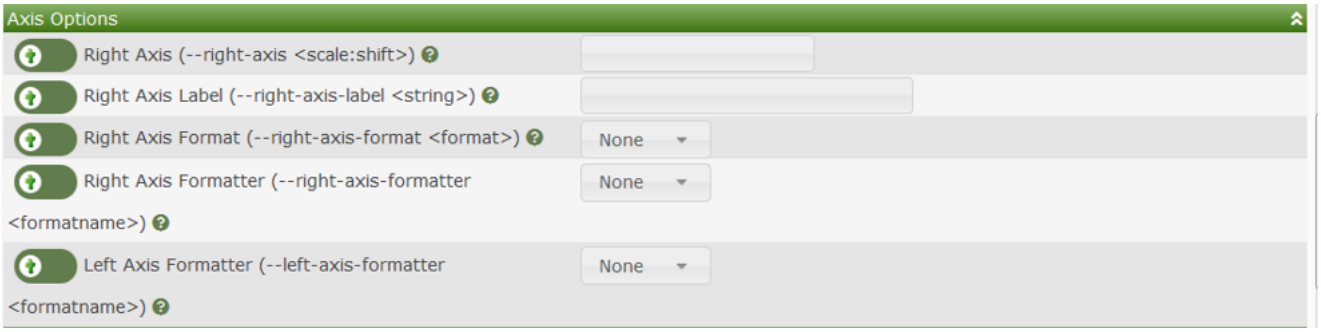


Graph Template - Grid Options

The *Grid Options* are rarely necessary unless you have specific requirements to render the resulting **Graphs** with some exotic unit.

Graph Template - Axis Options

The *Axis Options* allow you to define a *Right Axis* and optional *Formatters*.



Graph Template - Legend Options

The *Legend* options allow you to specify how the *Legend* should be placed on the resulting **Graph** modern RRDtool has several options that were not available in Cacti prior to Cacti 1.0.

Copyright (c) 2004-2026 The Cacti Group

Data Source Templates

Data sources serve an important role in Cacti data sources are data points collected from the devices you monitor in Cacti.

Data sources take the data that has been collected by the poller.

The Data source then holds the value to be presented to the graph.

The Data source template is then associated with a graph template.

These data sources can be for example incoming traffic on an interface. Cacti allows you to create data source templates to associate to data queries the template allows you to describe what type of data Cacti should expect.

To create a Data source you will need to select the proper data input there are several options but the most popular ones would be GET SNMP data or GET SNMP data (indexed.)

You would use Get SNMP data (indexed) for interfaces for example as Cacti would need to query the device to find the interface index. You would use Get SNMP data and put in a single OID for an OID that won't change for example CPU usage generally does not have an index and the OID won't change

You will also need to tell the template what the data source type is. The most common ones being Gauge and Counter. These data types come from the RRDtool its self. Below are the supported data types and their uses. For more information see RRD Tool Create Function

GAUGE

GAUGE is for things like temperatures or number of people in a room or the value of a RedHat share.

COUNTER

COUNTER is for continuous incrementing counters like the ifInOctets counter in a router. The COUNTER data source assumes that the counter never decreases, except when a counter overflows. The update function takes the overflow into account.

The counter is stored as a per-second rate. When the counter overflows, RRDtool checks if the overflow happened at the 32bit or 64bit border and acts accordingly by adding an appropriate value to the result.

DCOUNTER

Available when the configured RRDtool version is 1.5 or later.

DCOUNTER the same as COUNTER, but for quantities expressed as double-precision floating point number. It could be used to track quantities that increment by non-integer numbers, i.e. number of seconds that some routine has taken to run, total weight processed by some technology equipment etc.

The only substantial difference is that DCOUNTER can either be upward counting or downward counting, but not both at the same time. The current direction is detected automatically on the second non-undefined counter update and any further change in the direction is considered a reset. The new direction is determined and locked in by the second update after reset and its difference to the value at reset.

DERIVE

DERIVE will store the derivative of the line going from the last to the current value of the data source. This can be useful for gauges, for example, to measure the rate of people entering or leaving a room. Internally, derive works exactly like COUNTER but without overflow checks. So if your counter does not reset at 32 or 64 bit you might want to use DERIVE and combine it with a MIN value of 0.

DDERIVE

Available when the configured RRDtool version is 1.5 or later.

DDERIVE the same as DERIVE, but for quantities expressed as double-precision floating point number.

> **NOTE**: on COUNTER vs DERIVE
>
> by Don Baarda don.baarda@baesystems.com
>
> If you cannot tolerate ever mistaking the occasional counter reset for a legitimate counter wrap, and would prefer "Unknowns" for all legitimate counter wraps and resets, always use DERIVE with min=0. Otherwise, using COUNTER with a suitable max will return correct values for all legitimate counter wraps, mark some counter resets as "Unknown", but can mistake some counter resets for a legitimate counter wrap.
>
> For a 5 minute step and 32-bit counter, the probability of mistaking a counter resetfor a legitimate wrap is arguably about 0.8% per 1Mbps of maximum bandwidth. Note that this equates to 80% for 100Mbps interfaces, so for high bandwidth interfaces and a 32bit counter, DERIVE with min=0 is probably preferable. If you are using a 64bit counter, just about any max setting will eliminate the possibility of mistaking a reset for a counter wrap.

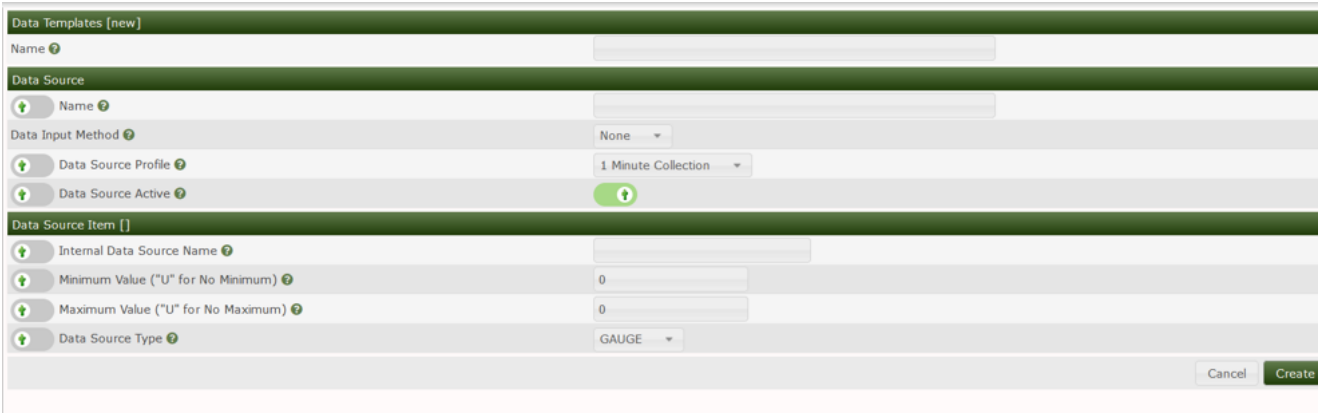
ABSOLUTE

ABSOLUTE is for counters which get reset upon reading. This is used for fast counters which tend to overflow. So instead of reading them normally you reset them after every read to make sure you have a maximum time available before the next overflow. Another usage is for things you count like number of messages since the last update.

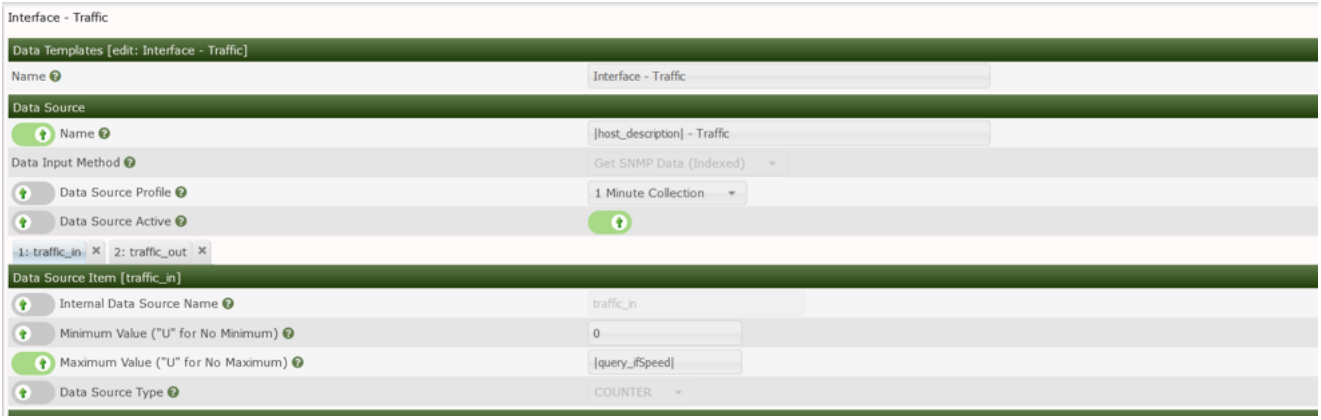
COMPUTE

RRDtool supports computed data sources using an RPN expression derived from other data sources in the same file. That is distinct from a graph-time CDEF.

The current Cacti type list includes `COMPUTE`, but the ordinary data-template editor has no expression field and the normal RRD creation path emits the heartbeat/minimum/maximum form used by collected data sources. That form is not a valid COMPUTE definition. Do not select it for a new data source through this workflow. Use a CDEF for graph-time calculations, or an explicitly managed external RRD workflow when persistent computed data is required. Advanced RRD/XML helpers do not supply the missing template editor and creation support.



You are also able to use Variables to set the name for the data source. The available Variables can be seen on the Variables documentation page.



What is also an important setting is the internal data source name. This will be used to associate the data source with the graph template. The template editor accepts 1–19 letters, digits, or underscores. Use a recognizable name, such as `traffic_in`. Do not rename an existing item without also planning the matching RRD file change; a metadata edit alone does not rename the stored series.

Copyright (c) 2004-2026 The Cacti Group

Aggregate Templates

Aggregate Overview

Aggregate Templates are a special form of a **Graph Template**. They allow you to easily create **Graphs** that combine data from multiple common **Graphs** from multiple **Devices**, and allow you to easily manage the resulting **Aggregate Graphs** to add and remove elements from other common **Graphs**.

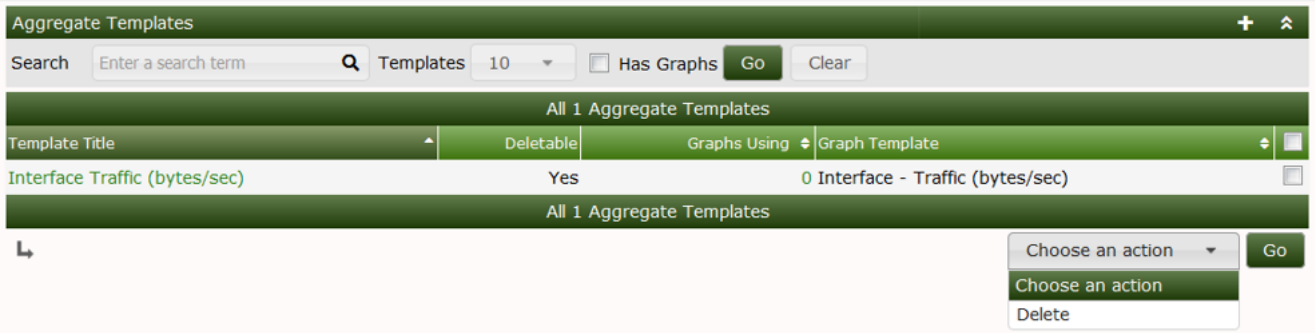
To create **Aggregate Graphs** that are managed through a Template, you first must create the **Aggregate Template**. Then from the Cacti **Graphs** page, you can select the **Graphs** that you want used as part of the **Aggregate**. You then select `Create Aggregate from Template` from the Cacti Actions drop-down.

Once you have created your **Aggregate Graphs**, they behave like any other Cacti Graph. They can be a part of a Tree, zoomed, etc. They have an added bonus - you can add and remove **Graphs** from aggregate in a very controlled way, reducing the effort to maintain them during their life-cycle.

If you wish to change settings for **Graphs** managed by the **Aggregate Template**, simply make the changes in the **Aggregate Template**, and they will cascade to attached **Aggregate Graphs** with **Propagation Enabled**. An aggregate whose propagation has been disabled no longer follows these template settings. See Aggregate Graphs for membership and propagation.

Aggregate Template Interface

The images below shows an Aggregate Template for Traffic.



When you edit the **Aggregate Template**, you are presented with an interface that allows you to define the Graph Canvas as well as it's formatting. You should experiment until you find the mechanism that works best to suite your needs.

The `Aggregate Template Prefix` setting allows you to provide a pattern to be applied to the **Aggregate Graph** legend items. Any `host`, `query` or `input` references can be used in the `Prefix` section in order to uniquely identify the **Graph Item**.

There are several `Graph Types` transformations that deal with how `AREA`, `LINEX` and `STACK` items are handled in the resulting **Aggregate Graph**, they include:

- **Keep Graph Types** - No transformation will occur. All `AREA`, `LINE`, and `STACKS` will be unchanged.
- **Keep Type and Stack** - Which means the types will be preserved, but all data will be stacked instead of simply LINEX it will be transformed to LINEX:STACK
- **Convert to Area/STACK Graph** - All `LINEX` will be converted to `AREA` and stacked.
- **Convert to LINE1** - All **Graph Items** will be converted to `LINE1`
- **Convert to LINE2** - All **Graph Items** will be converted to `LINE2`
- **Convert to LINE3** - All **Graph Items** will be converted to `LINE3`
- **Convert to LINE1/Stack** - All **Graph Items** will be converted to `LINE1` and stacked.
- **Convert to LINE2/Stack** - All **Graph Items** will be converted to `LINE2` and stacked.

- **Convert to LINE3/Stack** - All **Graph Items** will be converted to `LINE3` and stacked.

The `Totaling` setting has multiple values. They include:

- **No Totals** - There will be no Summation of Data in the Legend
- **Print All Legend Items** - Means that all Legend Items selected will be included with a Total
- **Print Totaling Legend Items Only** - This option means that the Legend will total all the Legend items into a single Legend.

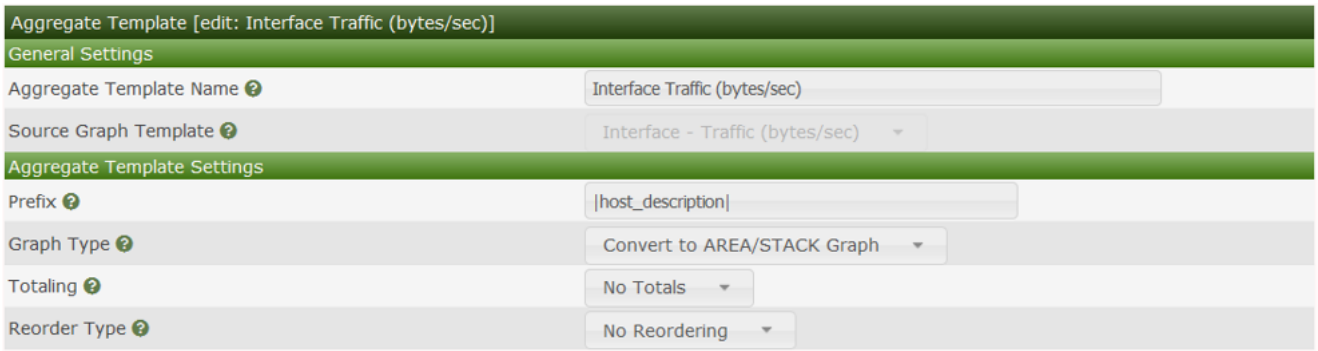
The `Total Type` - Will create groupings of common elements on the **Graph**, and reset stacking rules when a change in the common element occurs. The options include:

- **Total Similar Data Sources** - Means that you will Total legend items and **Data Sources** through similar **Data Source** names, for example `traffic_out` and `traffic_in`.
- **Total All Data Sources** - It means that you will sum the values for all **Data Sources** regardless of their **Data Source** name.

When using `Total Type`, you are provided an option to additionally prefix your legends using the `Prefix for GPRINT Totals` with a text value. The default works in most cases.

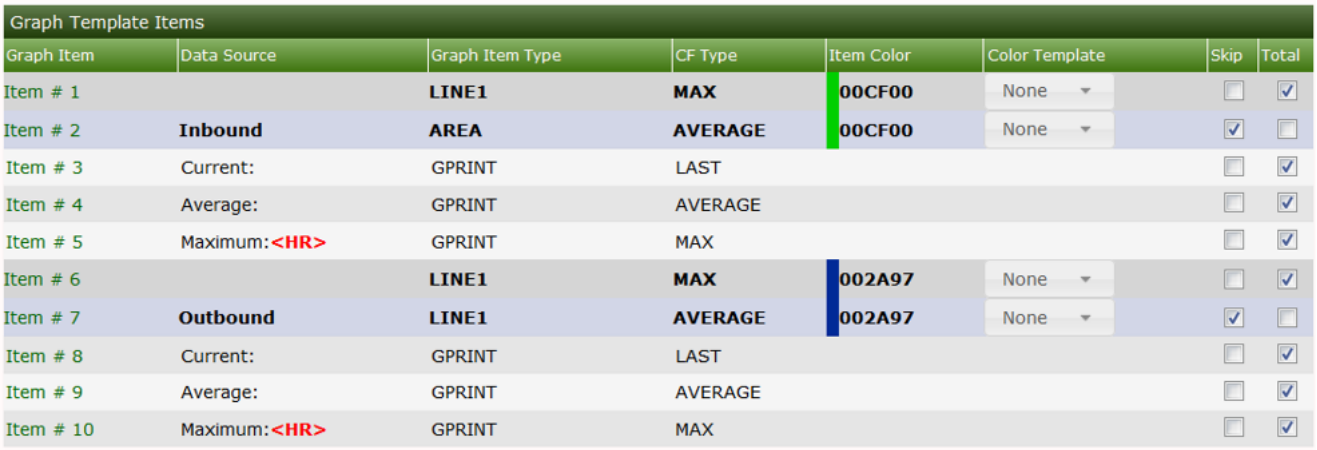
The `Reorder Type` will re-order the **Data Sources** within their respective grouping on the Graph so that they are ordered in a common way, in alphabetic order. The options include:

- **No Reordering** - Don't make any changes in ordering.
- **Data Source, Graph** - Order by **Data Source** name and then by **Graph** name
- **Graph, Data Source** - Order by **Graph** name and then by **Data Source** name
- **Base Graph Order** - Focus on the **Graph** name only



The `Graph Template Items` section allows you to either Skip or Total (aka include) the **Graph Items** in the resulting **Aggregate Graphs**. When you think about how a resulting **Aggregate** graph will look, there are some elements that simply will not result in a good looking **Graph**. In those cases you will want to remove them from the resulting **Aggregate Graphs**.

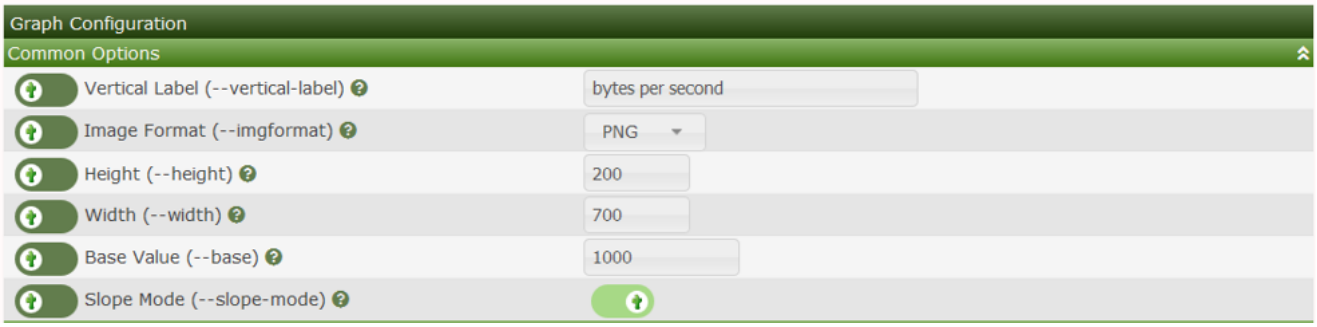
The `Color Template` option allows you to use differing Color rotations when displaying elements on the resulting **Aggregate Graphs**. **Color Templates** can be added and removed from the **Color Templates** menu pick under `Console > Templates`.



Graph Template Items

Graph Item	Data Source	Graph Item Type	CF Type	Item Color	Color Template	Skip	Total
Item # 1		**LINE1**	**MAX**	**00CF00**	None	☐	☑
Item # 2	**Inbound**	**AREA**	**AVERAGE**	**00CF00**	None	☑	☐
Item # 3	Current:	GPRINT	LAST			☐	☑
Item # 4	Average:	GPRINT	AVERAGE			☐	☑
Item # 5	Maximum:<HR>	GPRINT	MAX			☐	☑
Item # 6		**LINE1**	**MAX**	**002A97**	None	☐	☑
Item # 7	**Outbound**	**LINE1**	**AVERAGE**	**002A97**	None	☑	☐
Item # 8	Current:	GPRINT	LAST			☐	☑
Item # 9	Average:	GPRINT	AVERAGE			☐	☑
Item # 10	Maximum:<HR>	GPRINT	MAX			☐	☑

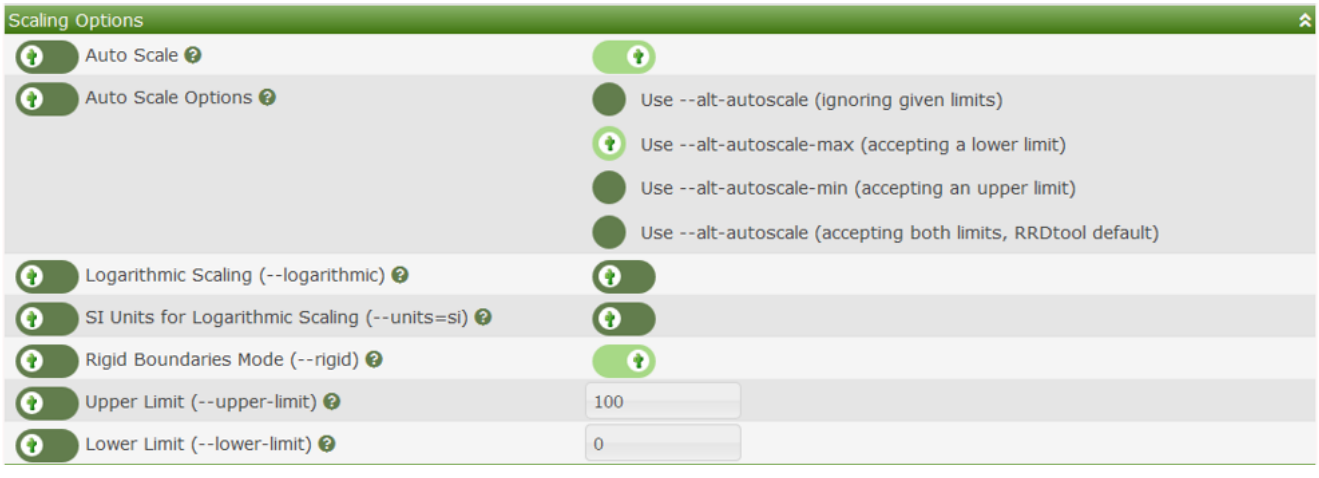
The several sections allow you to override any of the common **Graph Template** elements from the resulting **Aggregate Graph**. We will not explain those options here, only let you know that you can override them in your resulting **Aggregate Graph**.



Graph Configuration

Common Options

Vertical Label (--vertical-label)	bytes per second
Image Format (--imgformat)	PNG
Height (--height)	200
Width (--width)	700
Base Value (--base)	1000
Slope Mode (--slope-mode)	

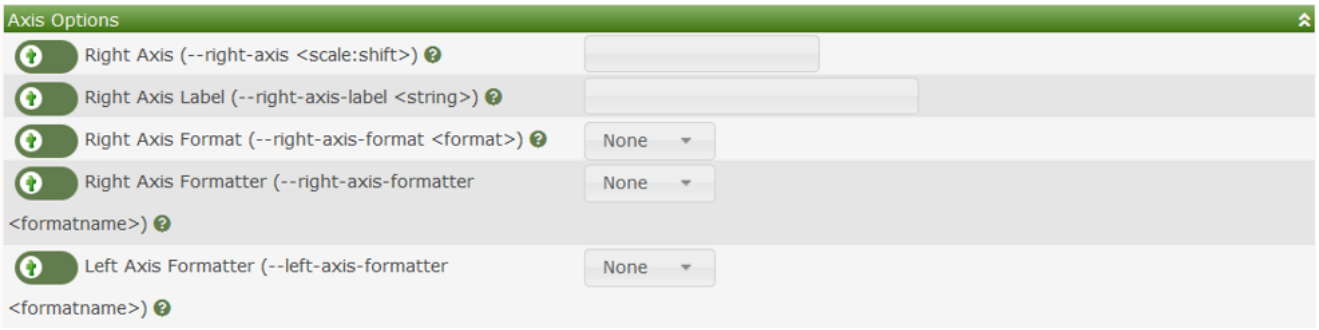


Scaling Options

Auto Scale	
Auto Scale Options	Use --alt-autoscale (ignoring given limits) Use --alt-autoscale-max (accepting a lower limit) Use --alt-autoscale-min (accepting an upper limit) Use --alt-autoscale (accepting both limits, RRDtool default)
Logarithmic Scaling (--logarithmic)	
SI Units for Logarithmic Scaling (--units=si)	
Rigid Boundaries Mode (--rigid)	
Upper Limit (--upper-limit)	100
Lower Limit (--lower-limit)	0

Grid Options

Unit Grid Value (--unit/--y-grid)	
Unit Exponent Value (--units-exponent)	
Unit Length (--units-length <length>)	
No Gridfit (--no-gridfit)	
Alternative Y Grid (--alt-y-grid)	



Summary

There are several combinations of options that you can use when working with an **Aggregate Template**. Some of these options will results in horrible and unexpected outcomes, so you will have to experiment until you come up with a desirable **Aggregate Template**.

Copyright (c) 2004-2026 The Cacti Group

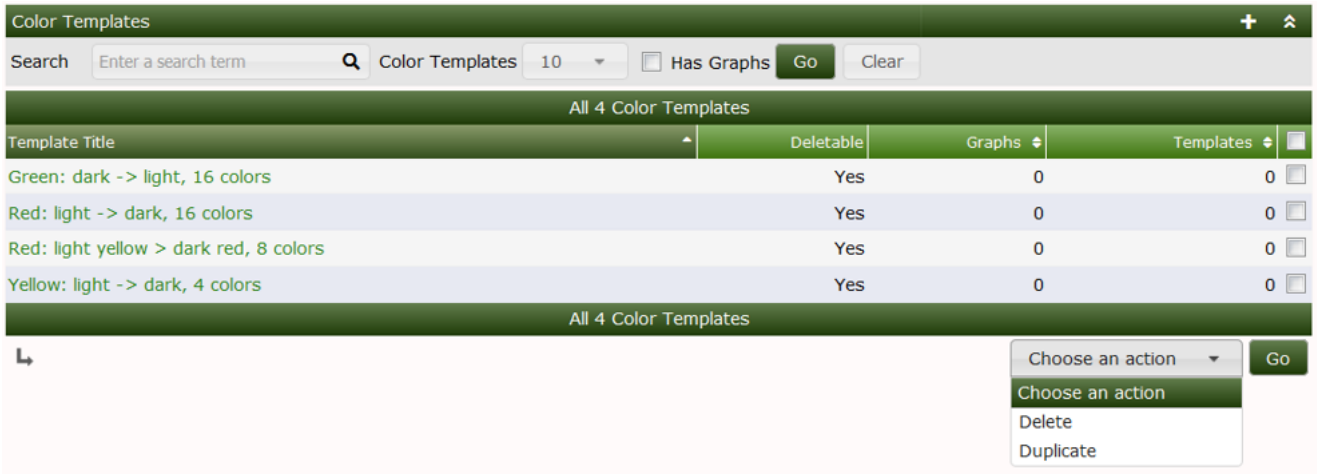
Color Templates

Color Templates define a list of Colors to be used for **Aggregate Graphs** in Cacti. As you add **Graphs** to an **Aggregate Graph**, you need to distinguish one **Graph** from the next within the **Aggregate Graph**. These **Color Templates** are a list of colors that will be looped through in Round Robin fashion to render the **Aggregate Graph**.

Color selection rotates across member graphs for each item to which a color template is assigned. It uses the member-graph index modulo the number of colors; it does not advance once for every graph item or legend entry.

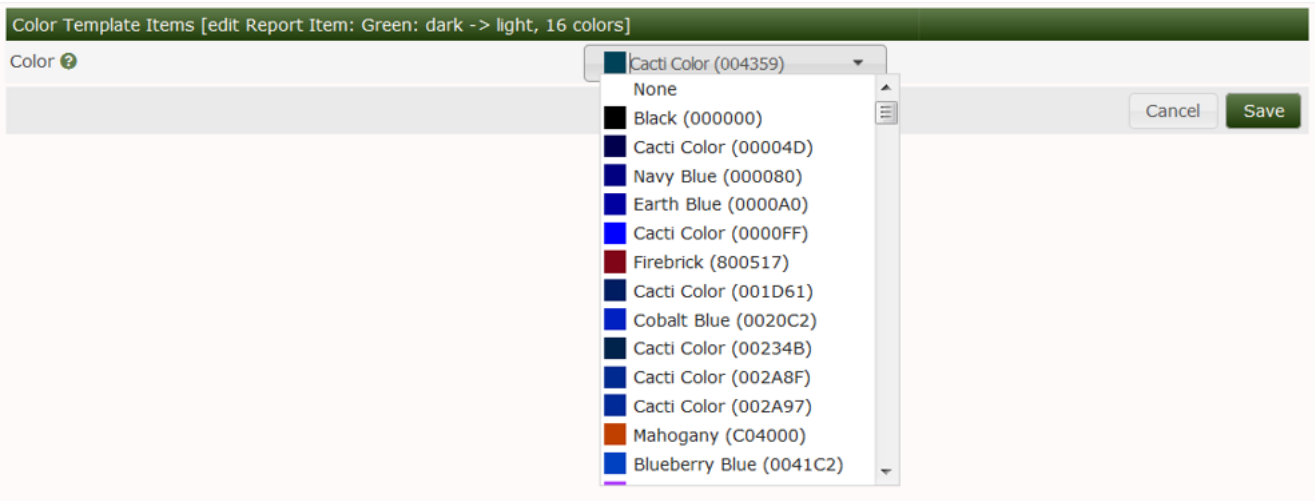
For example, assign an eight-color template to one plotted series shared by 16 member graphs. Each color is selected twice for that series. Additional series can have their own assigned color templates, while uncolored legend items do not add steps to this rotation.

Below, you can see the four standard **Color Templates**. From the Actions drop down you can *Delete*, *Duplicate*, or *Sync Aggregates* for the selected **Color Templates**. As with other Cacti objects, you will not be allowed to *Delete* a **Color Template** in use.



In the image below, you can see the **Color Template** edit screen. This simple screen allows you to add, remove and re-order colors in the list.

Shown in the image below, only Cacti **Colors** are allowed to be selected for Aggregate **Color Templates**. The **Color** drop down can be typed into if you wish to search through the list of approximately 340 legacy and *Named Colors*.



Copyright (c) 2004-2026 The Cacti Group

Data Input Methods

Data Input Methods allow Cacti to retrieve data to insert into RRDfiles based upon a mapping controlled by **Data Templates** and their corresponding **Data Sources**. These resulting **Data Templates** and **Data Sources** can then be used to create **Graph Templates** and **Graphs**.

Cacti includes a number of build in **Data Input Methods** for **SNMP** data and for **Script**, **Script Server** and **SNMP Data Queries**.

Outside of the built in **Data Input Methods**, the Cacti Administrator can create virtually any **Data Input Method** based upon a **Script**, or a PHP **Script Server** script. The **Script** based **Data Input Method** allows Cacti to virtually collect data from anywhere, though the built in **SNMP** and **Script Server** methods provide the greatest scalability in Cacti. Both **Data Queries**, and PHP **Script Server** topics will be covered in later sections of the documentation.

Creating a Data Input Method

To create a new **Data Input Method**, from the Cacti Console, select **Data Collection > Data Input Methods**. Once on that screen, click the plus (+) glyph on the right which will allow you to add a new **Data Input Method**. You will be presented with a few fields to populate on the following screen.

Table 11-1. Field Description: Data Input Methods

Name	Description
Name	Give the data query a name that you will use to identify it
Input Type	Select the type of **Data Input Method** you are trying to create
Input String	This field is only used when the Input Type is set to **Script/Command**

The `Name` specified will be used throughout Cacti to identify the human readable name given to the **Data Input Method**. Careful consideration should be made to help uniquely identify the **Data Source**. Having very similar names can lead to confusion when utilizing them as your Cacti system grows.

Valid options for `Input Type` are **Script/Command**, and **Script Server**. As mentioned previously, Cacti provides built in **Data Input Methods** for SNMP data gathering and for SNMP, **Script**, and **Script Server** based **Data Queries**. Though present in the Cacti database, they are hidden from user view.

When the type is set to `Script/Command`, the `Input String` specifies the full path to the script including any per **Data Source** input variables. **Data Source** input variables must be enclosed in greater than and less than characters. For instance, if you are passing an IP address to a script, your input string might look something like: `/path/to/script.pl <ip>` When the user creates a **Data Source** based on this **Data Input Method**, they will be prompted for an IP address to pass onto the script.

When you are finished filling in all necessary fields, click the Create button to continue. Upon saving the new **Data Input Method**, you will be presented with two new sections to complete. Those sections will instruct Cacti what to pass to the Script, known as `Input Fields` otherwise known as `Input Parameters` and how to handle the output data, which we refer to as `Output Fields`.

The `Input Fields` box is used to define any fields that require information from the user or from various data within the Cacti Database such as the hostname, ip address, host id, etc. Any input fields referenced to in the input string must be defined here.

The `Output Fields` box is used to define each field that you expect back from the script and will be eventually stored in both the database and RRDfiles.

*All **Data Input Methods** must have at least one output field defined*, but may have more than one depending on the type.

Data Input Fields

To define a new field, click the plus sign (+) next to the input or output field boxes. You will be presented with some or all of the fields below depending on whether you are adding an input or output field.

Table 11-2. Field Description: Data Input Fields

Name	Description
Field/Field Name	You will be presented a drop down list of the unused braced input fields from the command
Friendly Name	Enter a more descriptive name for this field
Regular Expression Match (Input Only)	Enter a valid regular expression as to how to modify the output
Allow Empty Input (Input Only)	Can the input value of this field blank or not
Special Type Code (Input Only)	Pull the input data from the Cacti database and don't prompt the user for this input value
Update RRDfile (Output Only)	Checked if you intend this output data to be stored in an RRDfile

The `Field Name` must contain no spaces or other non-alphanumeric characters (except '-' or '_').

If you want to enforce a certain regular expression pattern when the user enters a value into `Regular Expression Match (Input Only)` for this Data Input Field, use a Perl-compatible regular expression (PCRE), as required by PHP's `preg_*` functions.

If the Data Input Field needs to reference another field internally, you can enter this into the `Special Type Code`. For instance, if your field requires an IP address from the user, you can enter 'management_ip' here and Cacti will fill this field in with the current IP address of the selected host.

Table 11-3. Special Type Codes

Field Name	Description
hostname	The hostname
management_ip	The ip
snmp_community	The SNMP community
snmp_username	The SNMP username
snmp_password	The SNMPv3 authentication password

If you enable the `Update RRDfile`, Cacti will insert the return value from this field into the RRDfile. This box needs to be checked for at least one output field per data input source, but can be left blank to have Cacti store the value only in the database instead.

When you are finished filling in all necessary fields, click the Create button to continue. You will be redirected back to the **Data Input Method** edit page. From here you can continue to add additional fields, or click Save on this screen when finished.

Making Your Scripts Work With Cacti

The simplest way to extend Cacti's data gathering functionality is through external scripts. Cacti comes with a number of scripts out of the box which are located in the `scripts/` directory. These scripts are used by the **Data Input Methods** that are present in a new installation of Cacti.

To have Cacti call an external script to gather data you must create a new **Data Input Method**, making sure to specify Script/Command for the Input Type field. See the previous section, Creating a Data Input Method for more information about how to create a **Data Input Method**. To gather data using your **Data Input Method**, Cacti simply executes the shell command specified in the Input String field. Because of this, you can have Cacti run any shell command or call any script which can be written in almost any language.

What Cacti is concerned with is the output of the script. When you define your **Data Input Method**, you are required to define one or more output fields. The number of output fields that you define here is important to your script's output. For a **Data Input Method** with only one output field, your script should output its value in the following format:

```
<numeric value>
```

So if I wrote a script that outputs the number of running processes, its output might look like the following:

```
67
```

Data Input Methods with more than one output field are handled a bit differently when writing scripts. Scripts that output more than one value should be formatted like the following:

```
<fieldname_1>:<value_1> <fieldname_2>:<value_2> ... <fieldname_n>:<value_n>
```

If you wrote a script that outputs the 1, 5, and 15 minute load average of a Unix machine and in Cacti named the output fields '1min', '5min', and '15min', the output of the script should look like the following:

```
1min:0.40 5min:0.32 15min:0.01
```

One last thing to keep in mind when writing scripts for Cacti is that they will be executed as the user the data gatherer runs as. Sometimes a script may work correctly when executed as root, but fails due to permissions problems when executed as a less privileged user.

Walkthrough

You can find a detailed example of how to create a complete Graph from simple command output in the following example How To Create a Data Input Method.

Copyright (c) 2004-2026 The Cacti Group

Data Input Methods - From Script Output to Complete Graph

In this How-To we will take you through the process of creating a script and then from the scripts output generate a complete **Graph Template** that you can apply to your Cacti system. You should be able to reproduce this process regardless of the script type in question.

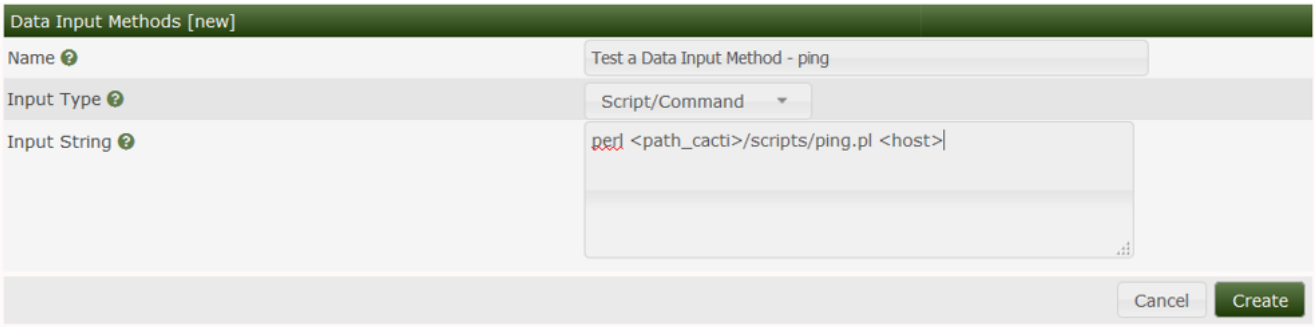
Data Input Method returning a single value

Lets start with a simple script that takes a hostname or IP address as input parameter returning a single value.

`<path_cacti>/scripts/ping.pl`:

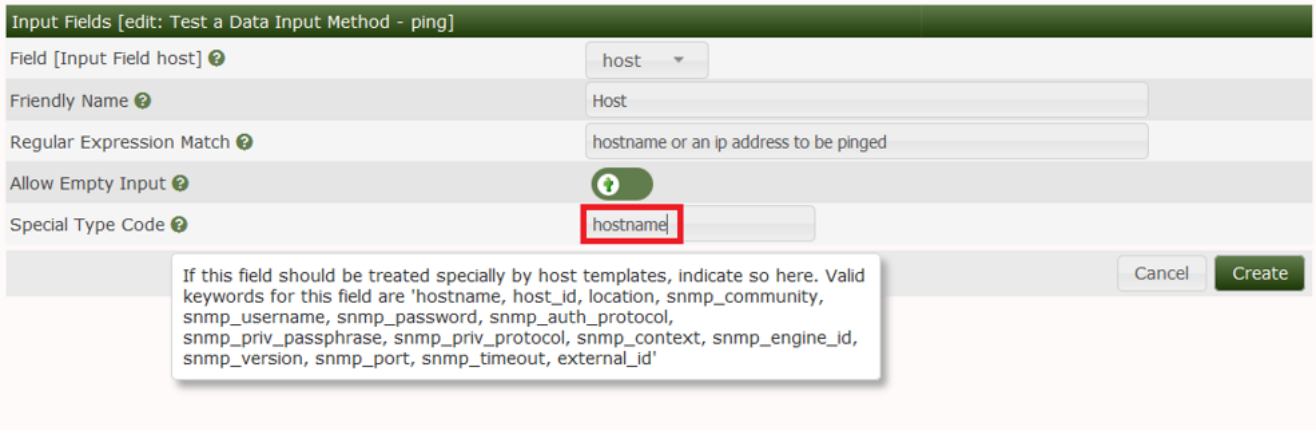
```
#!/usr/bin/perl

# take care for tcp:hostname or TCP:ip@
$host = $ARGV[0];
$host =~ s/tcp:/$1/gis;

# old Linux version use "icmp_seq"
# newer use "icmp_req" instead
open(PROCESS, "ping -c 1 $host | grep 'icmp_[s|r]eq' | grep time |");
$ping = <PROCESS>;
close(PROCESS);
$ping =~ m/(.*time=)(.*) (ms|usec)/;

if ($2 == "") {
    print "U";              # avoid cacti errors, but do not fake RRDtool stats
}elsif ($3 eq "usec") {
    print $2/1000;  # re-calculate in units of "ms"
}else{
    print $2;
}
```

To define this script as a **Data Input Method** to Cacti go to **Data Input Methods** and click **Add**. You should see:



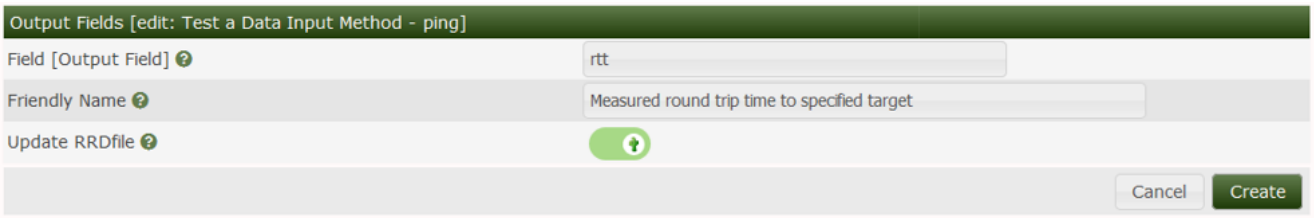
Please fill in **Name**, select **Script/Command** as Input Type and provide the command that should be used to retrieve the data. You may use `<path_cacti>` as a symbolical name for the path_to_your_Cacti_installation. Those commands will be executed from crontab; so pay attention to providing full path to binaries if required (e.g. `/usr/bin/perl` instead of `perl`). Enter all **Input Parameters** in `<>` brackets. Click create to see:

Now lets define the **Input Fields**. Click **Add** as given above to see:

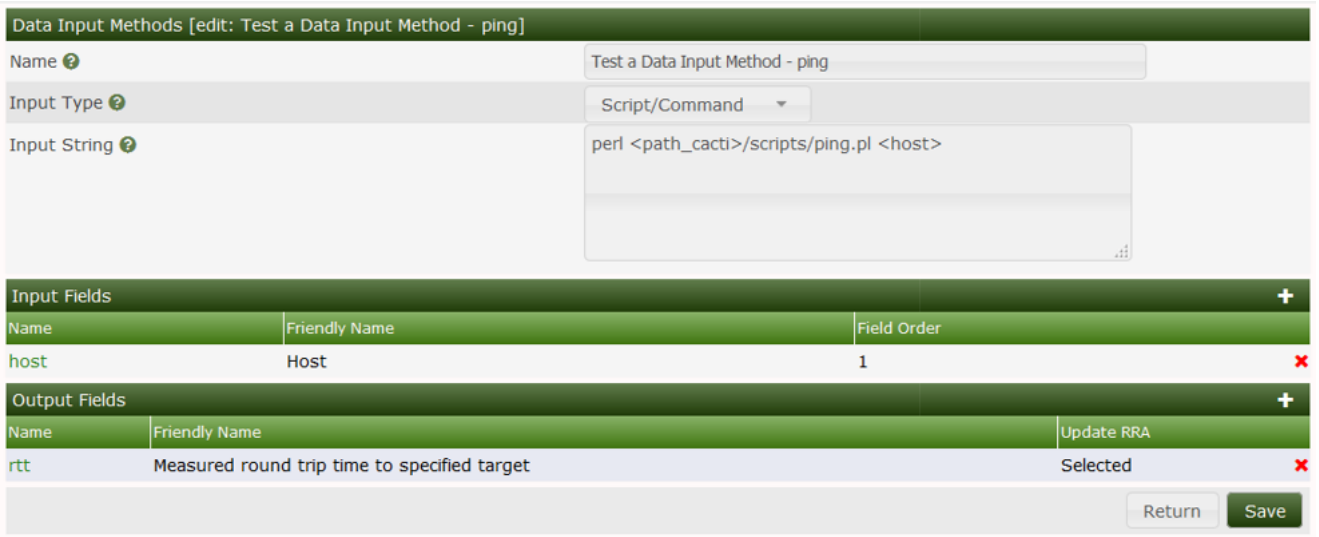


The drop down **Field [Input]** contains one single value only. This is taken from the **Input String** `<host>` above. Fill **Friendly Name** to serve your needs. The **Special Type** Code allows you to provide parameters from the current Device to be queried. In this case, the **hostname** will be taken from the current device. Click create to see:

At least, define the **Output Fields**. Again, click **Add** as described above:



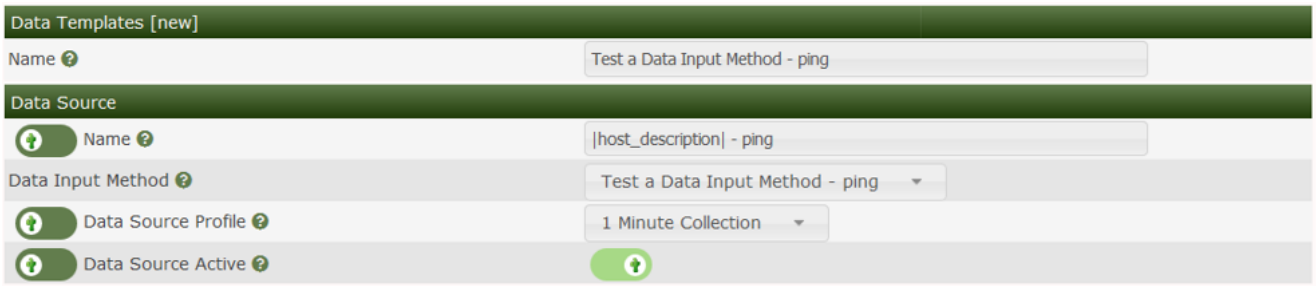
Provide a short **Field [Output]** name and a more meaningful **Friendly Name**. As you will want to save those data, select **Update RRDfile**. Create to see:



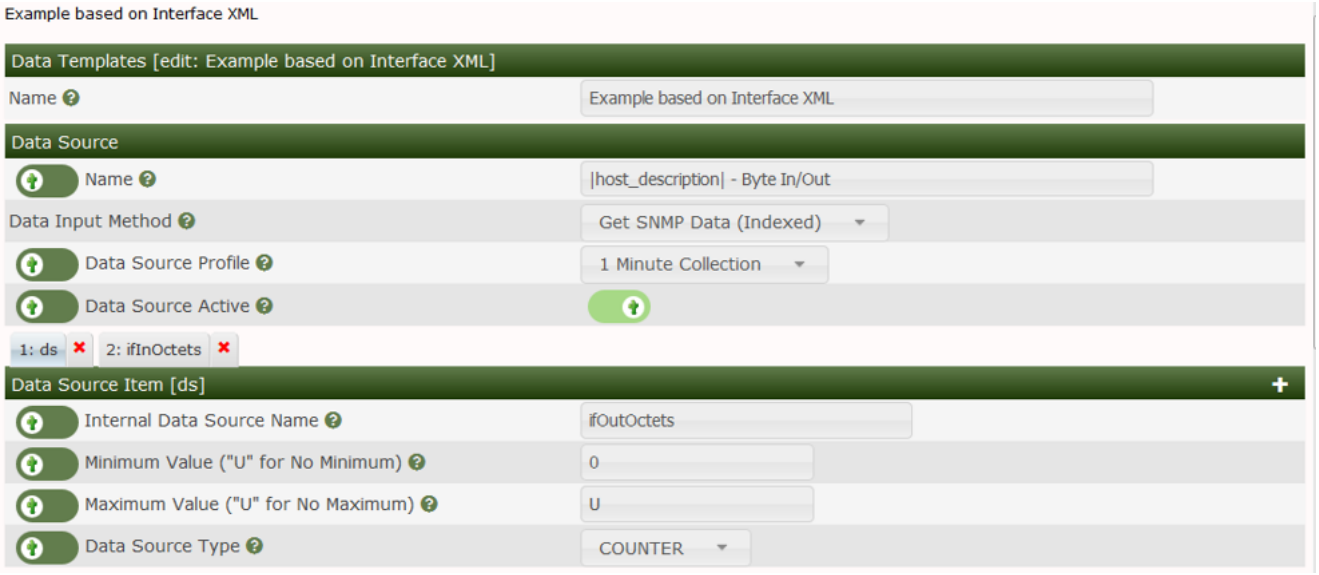
Click **Save** and you're done.

Creating the Data Template

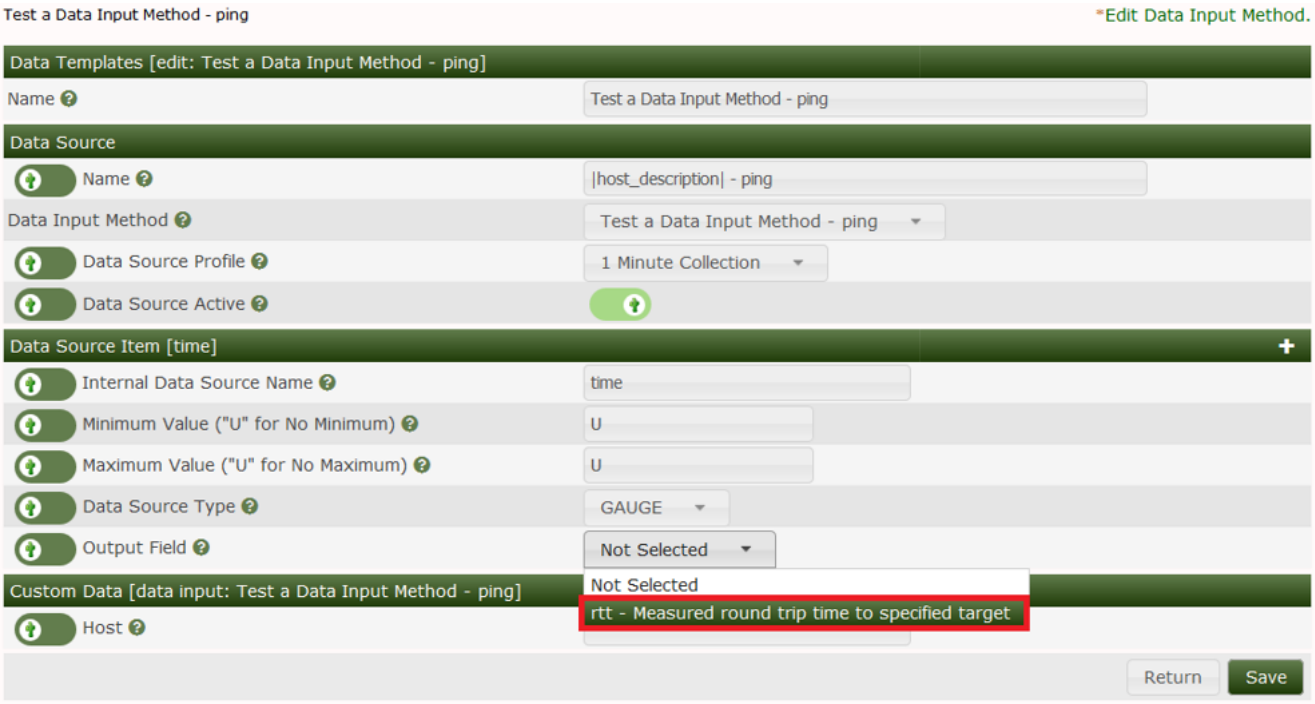
Now you want to tell Cacti, how to store the data retrieved from this script. Please go to **Data Templates** and click Add. You should see:



Fill in the **Data Templates Name** with a reasonable text. This name will be used to find this Template among others. Then fill in the **Data Source Name**. This is the name given to the host-specific **Data Source**. The variable **|host_description|** is taken from the actual **Device**. This is to distinguish **Data Sources** for different devices. The **Data Input Method** is a drop down containing all known scripts and the like. Select the **Data Input Method** you just created. The **Associated RRA's** is filled by default. At the moment there's no need to change this. The lower part of the screen looks like:



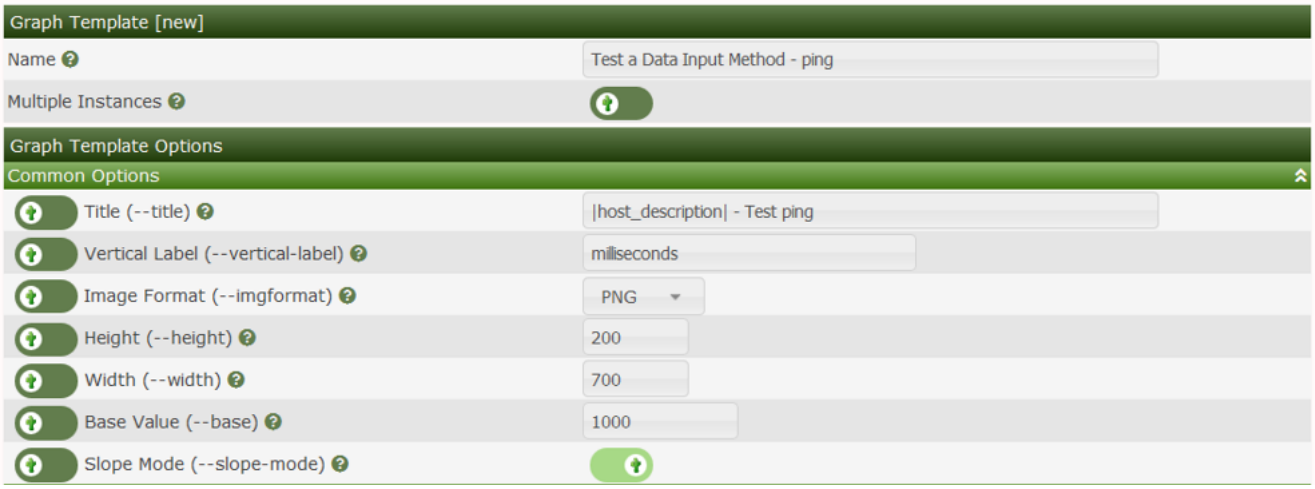
The **Internal Data Source Name** may be defined at your wish. There's no need to use the same name as the Output Field of the **Data Input Method**, but it may look nicer. Click create to see:



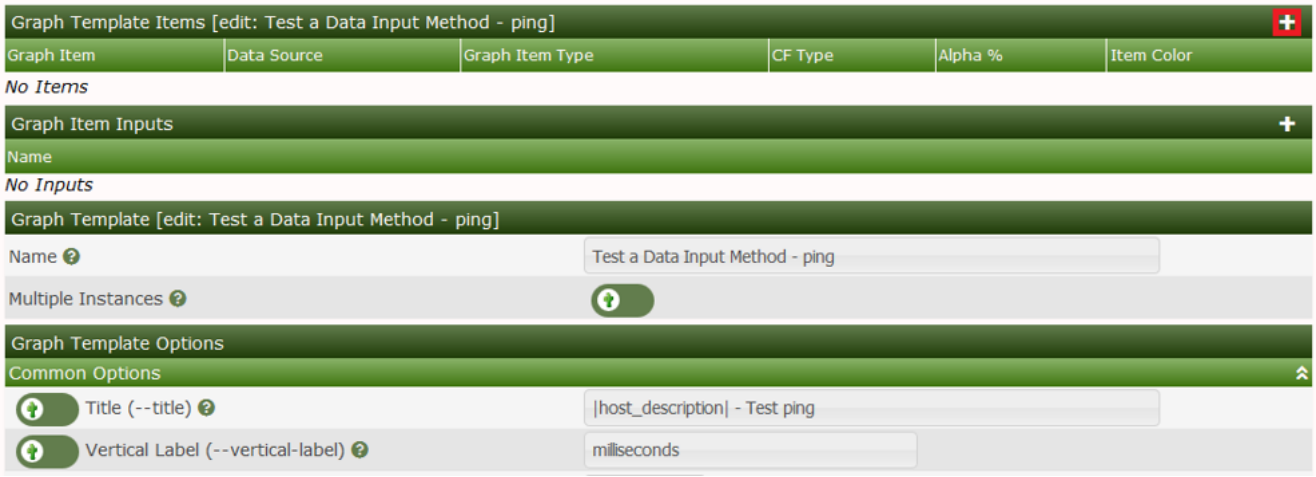
Notice the new drop down **Output Field**. As there is only one Output Field defined by our **Data Input Method**, you'll see only this. Here's how to connect the **Data Source Name** (used in the RRDfile) to the Output Field of the Script. Click **Save** and you're done.

Creating the Graph Template

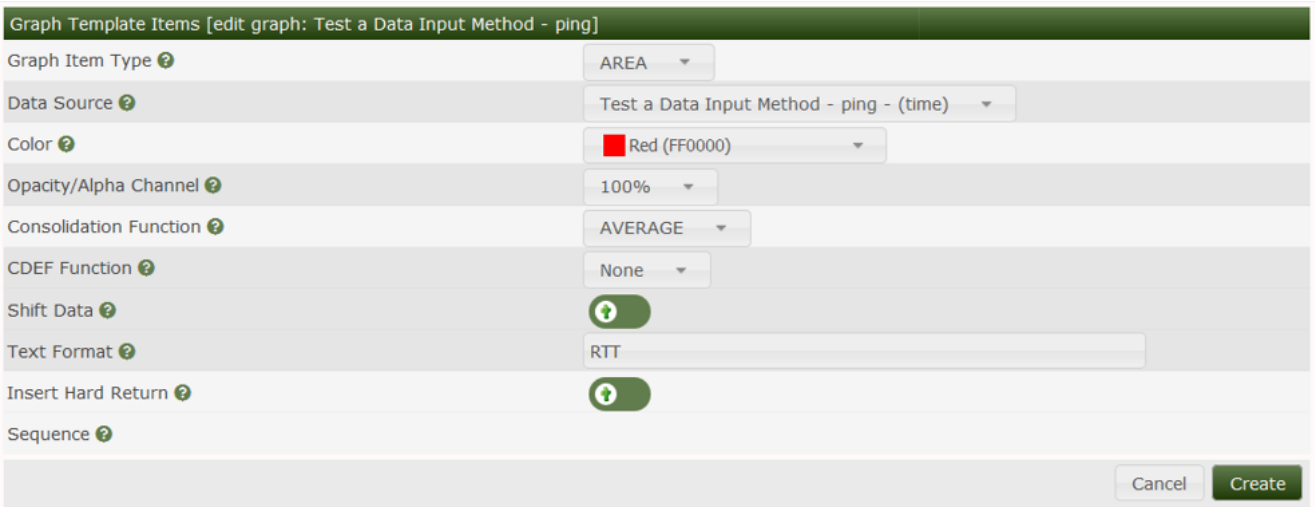
Now you want to tell Cacti, how to present the data retrieved from this script. Please go to **Graph Templates** and click **Add**. You should see:



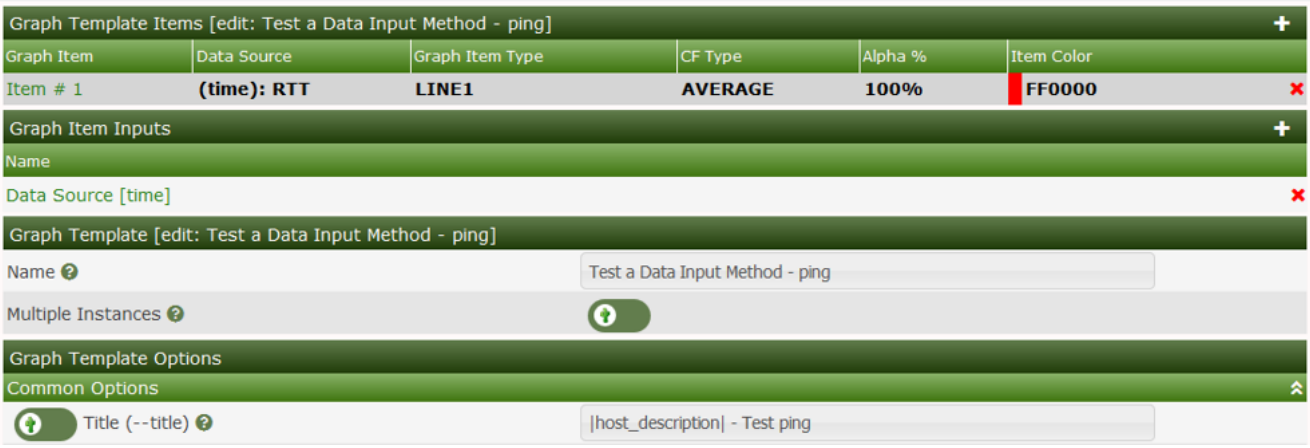
Fill in **Name** and **Title**. The variable `|host_description|` will again be filled from the Device's definition when generating the **Graph**. Keep the rest as is and **Create**. See:



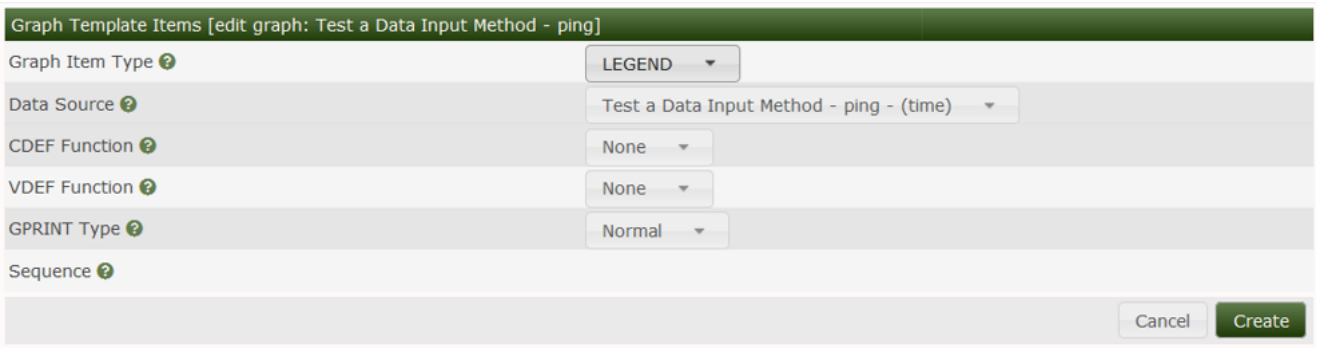
Now click the plus (+) glyph to **Add** the first item to be shown on the **Graphs**:



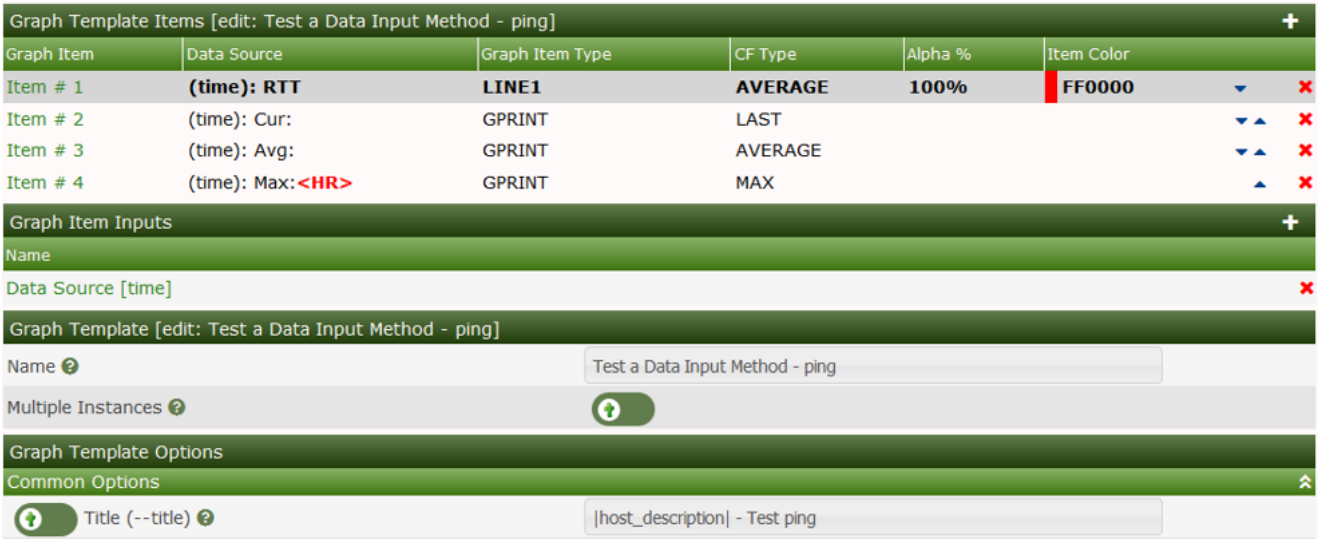
Select the correct **Data Source** from the drop down, fill in a color of your liking and select AREA as a **Graph Item Type**. You want to fill in a **Text Format** that will be shown underneath the **Graph** as a legend. Again, **Create**:



Notice, that not only an entry was made under **Graph Template Items**, but under **Graph Item Inputs** as well. Don't bother with that now. Lets fill some more nice legends, see:



Notice, that the **Data Source** is filled in automagically. Select LEGEND as **Graph Item Type** (it is not really a **Graph Item Type** in RRDtool-speak, but a nice time-saver), and click **Create** to see:



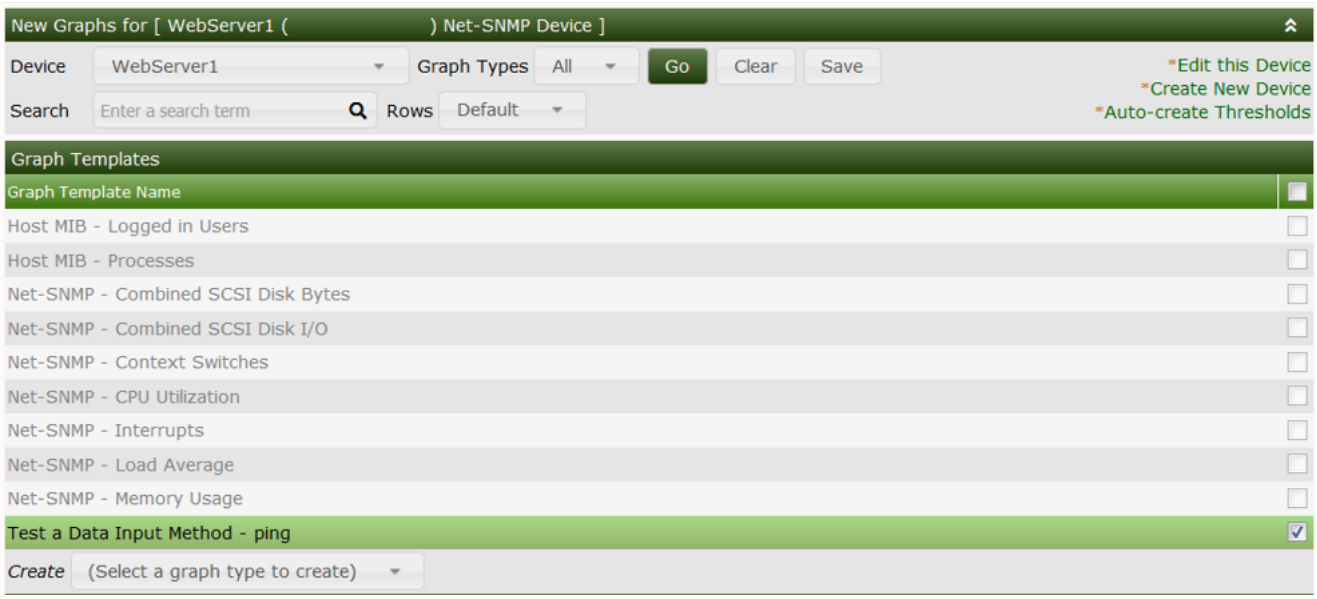
Wow! Three items filled with one action!

Apply the Graph Template to a Device

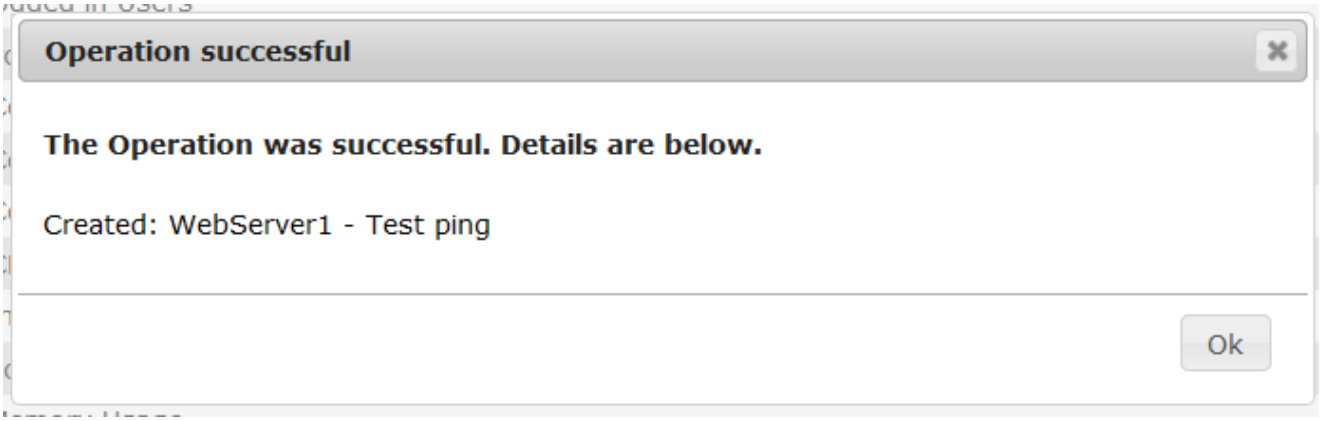
Now go to the **Devices** and select the one of your choice. See the **Associated Graph Templates** in the middle of this page:

Select your newly created **Graph Template** from the Add **Graph Template** drop down. Click the plus (+) glyph to see:

By default, the **Graph Template** is added and shown as **Is Being Graphed**. The reason for this is that Cacti now automatically creates **Graphs** automatically, unless disabled. If the **Graph Template** shows **Not Being Graphed**, you can create the **Graph** by pressing the **Create Graphs** at the top of the **Devices** page. Click this link to see:



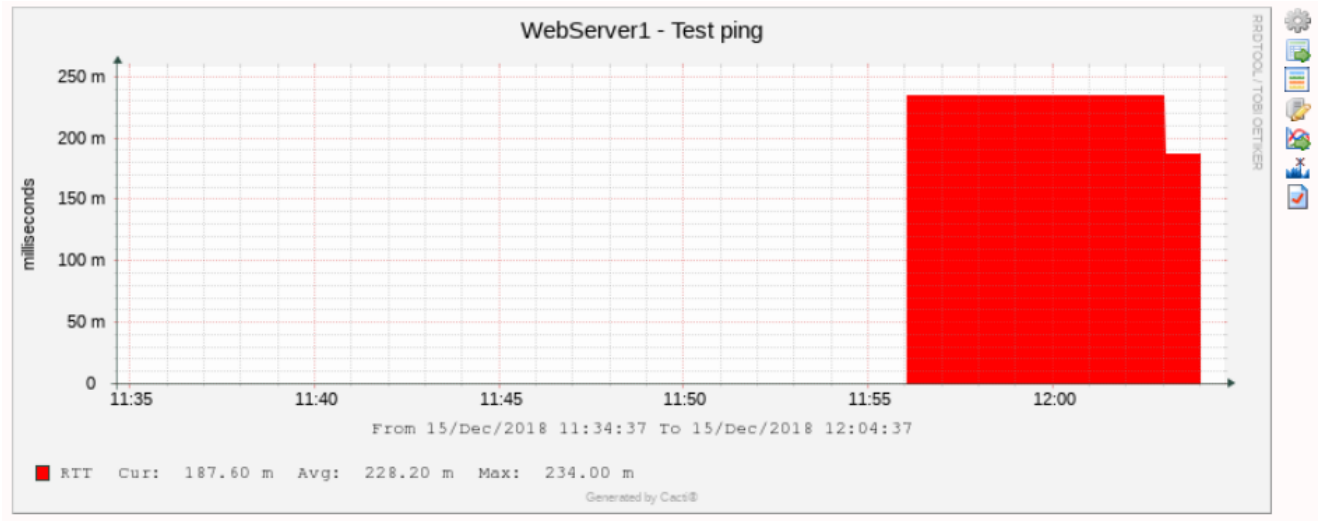
Check the box that belongs to the new template and Create. See the results:



This will automatically

- Create the needed **Graph Description** from the **Graph Template**. As you may notice from the success message, this **Graph** takes the hosts name in it: **router - Test ping** (router is the hosts name of this example).
- Create the needed **Data Source Description** from the **Data Template**. Again, you will find the Hosts name replaced for |host_description|
- Create the needed RRDfile with definitions from the **Data Template**. The name of this file is derived from the Host and the **Data Template** in conjunction with an auto-incrementing number.
- Create an entry to the poller_item to instruct Cacti to gather data on each polling cycle.

You'll have to wait at least two polling cycles to find data in the **Graph**. Find your **Graph** by going to **Graph Management**, filtering for your host and selecting the appropriate **Graph** (there are other methods as well). This may look like:



Walkthrough: Script with more Output Parameters

The script below will be implemented in perl. Please note that any supported language is supported using Cacti from direct command execution to scripts written in perl, python, php, bash, etc.

```
#!/usr/bin/perl -w

# ----------------------------------------------------
# ARGV[0] = &lt;hostname&gt;     required
# ARGV[1] = &lt;snmp port&gt;    required
# ARGV[2] = &lt;community&gt;    required
# ARGV[3] = &lt;version&gt;      required
# ----------------------------------------------------
use Net::SNMP;
```

```
# verify input parameters
my $in_hostname         = $ARGV[0] if defined $ARGV[0];
my $in_port             = $ARGV[1] if defined $ARGV[1];
my $in_community        = $ARGV[2] if defined $ARGV[2];
my $in_version          = $ARGV[3] if defined $ARGV[3];

# usage notes
if (
    ( ! defined $in_hostname ) ||
    ( ! defined $in_port ) ||
    ( ! defined $in_community ) ||
    ( ! defined $in_version )
) {
    print "usage:\n\n
        $0 &lt;host&gt; &lt;port&gt; &lt;community&gt; &lt;version&gt;\n\n";
    exit;
}

# list all OIDs to be queried
my $udpInDatagrams      = ".1.3.6.1.2.1.7.1.0";
my $udpOutDatagrams     = ".1.3.6.1.2.1.7.4.0";

# get information via SNMP
# create session object
my ($session, $error) = Net::SNMP->session(
    -hostname      => $in_hostname,
    -port          => $in_port,
    -version       => $in_version,
    -community     => $in_community,
    # add more parameters if there's a need for them:
    #   [-localaddr     => $localaddr,]
    #   [-localport     => $localport,]
    #   [-nonblocking   => $boolean,]
    #   [-domain        => $domain,]
    #   [-timeout       => $seconds,]
    #   [-retries       => $count,]
    #   [-maxmsgsize    => $octets,]
    #   [-translate     => $translate,]
    #   [-debug         => $bitmask,]
    #   [-username      => $username,]    # v3
    #   [-authkey       => $authkey,]     # v3
    #   [-authpassword  => $authpasswd,]  # v3
    #   [-authprotocol  => $authproto,]   # v3
    #   [-privkey       => $privkey,]     # v3
    #   [-privpassword  => $privpasswd,]  # v3
    #   [-privprotocol  => $privproto,]   # v3
);

# on error: exit
if (!defined($session)) {
    printf("ERROR: %s.\n", $error);
    exit 1;
}

# perform get requests for all wanted OIDs
my $result = $session->get_request(
    -varbindlist      => [$udpInDatagrams, $udpOutDatagrams]
);

# on error: exit
if (!defined($result)) {
    printf("ERROR: %s.\n", $session->error);
    $session->close;
    exit 1;
}

# print results
printf("udpInDatagrams:%s udpOutDatagrams:%s", # <<< cacti requires this format!
    $result->{$udpInDatagrams},
    $result->{$udpOutDatagrams},
);

$session->close;
```

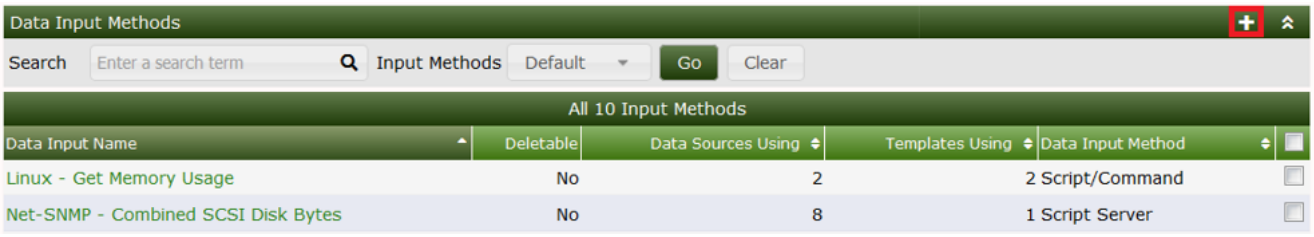
It should produce following output, when executed from command line:

```
[prompt]> perl udp_packets.pl localhost 161 public 1
udpInDatagrams:10121 udpOutDatagrams:11102
```

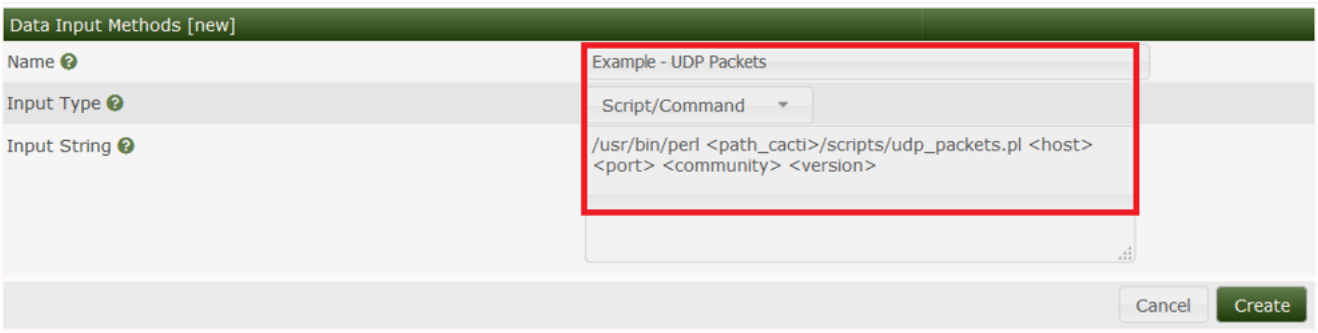
Where “public” would be replaced by your community string. Those input parameters again are controlled by the `Input Fields` in Cacti and the output from the script will be mapped to `Output Fields`. If you wish to return multiple output values, they must be space delimited name value pairs in the form of nameA:valueA nameB:valueB ...

The Data Input Method

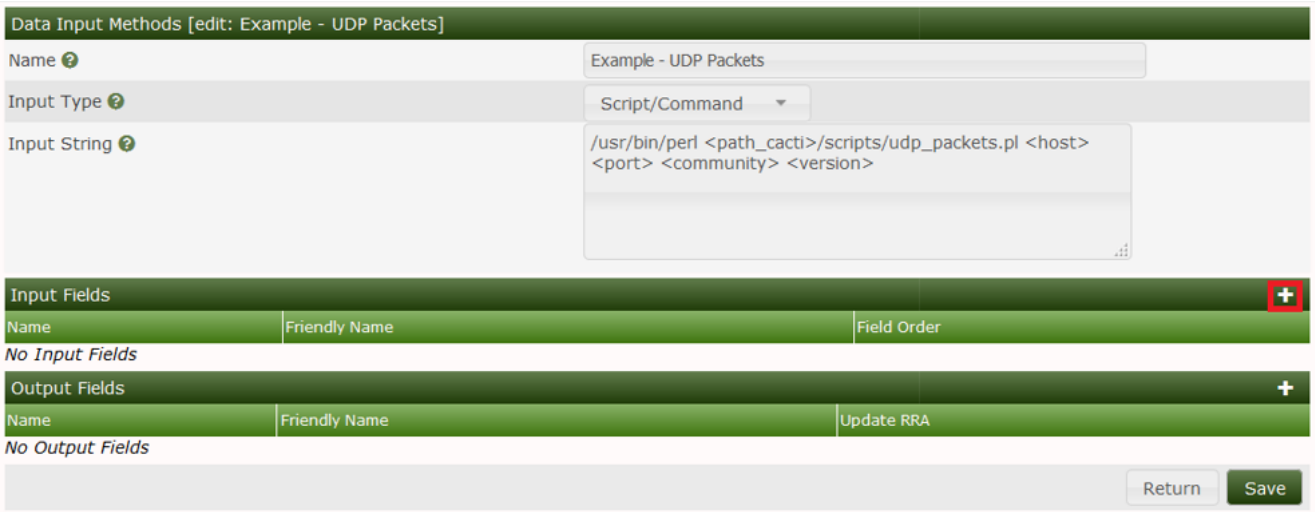
To define this script as a **Data Input Method** to Cacti go to **Data Input Methods** and click the plus (+) glyph.



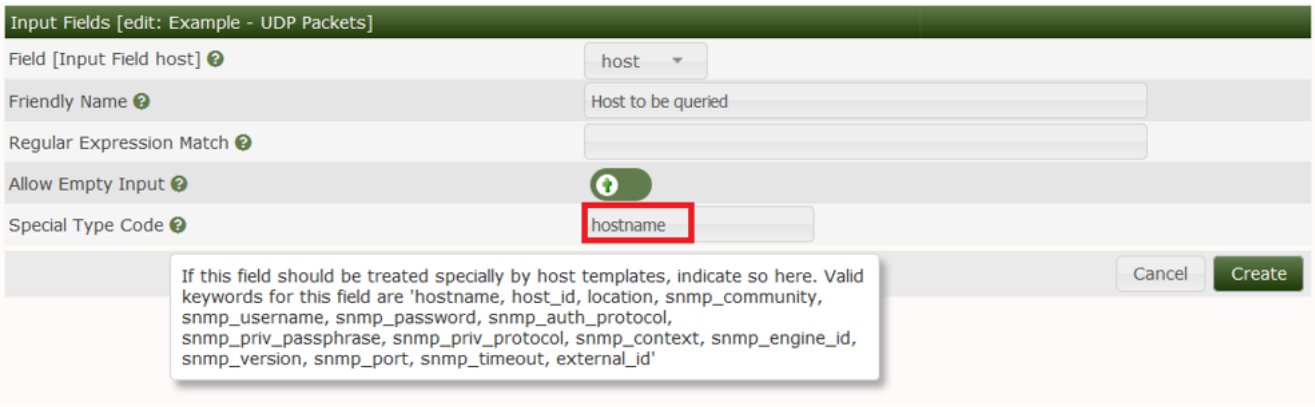
You should see:



Enter the name of the new **Data Input Method**, select **Script/Command** and type in the command to call the script. Please use the **full path to the command interpreter**. Instead of entering the specific parameters, type `<symbolic variable name>` for each parameter the script needs. **Save**:

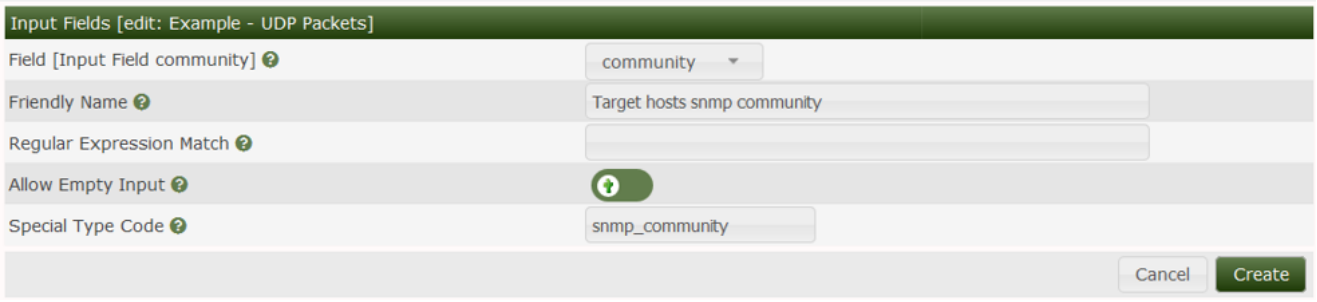


Now add each of the input parameters in the **Input Fields** section, one after the other. All of them are listed in sequence, starting with `<host>`:



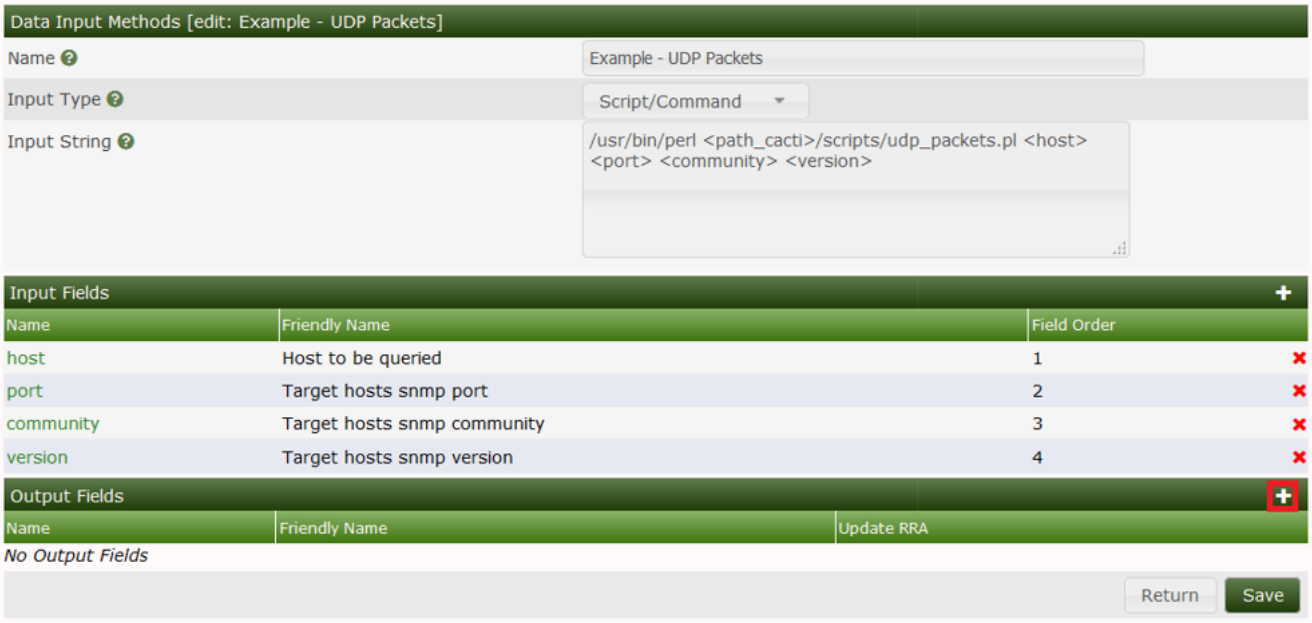
<port>

<community>

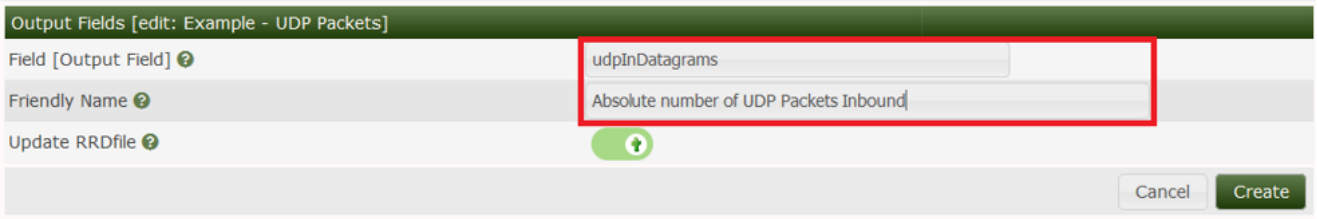


<version>

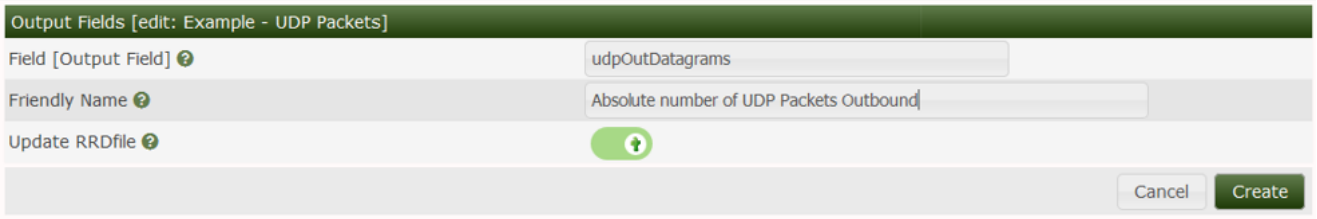
We've used some of Cacti builtin parameters. When applied to a host, those variables will be replaced by the hosts actual settings. Then, this command will be stored in the poller_command table. Now **Save** your work to see



After having entered all **Input Fields**, let's now turn to the **Output Fields**, respectively. Add the first one, udpInDatagrams:

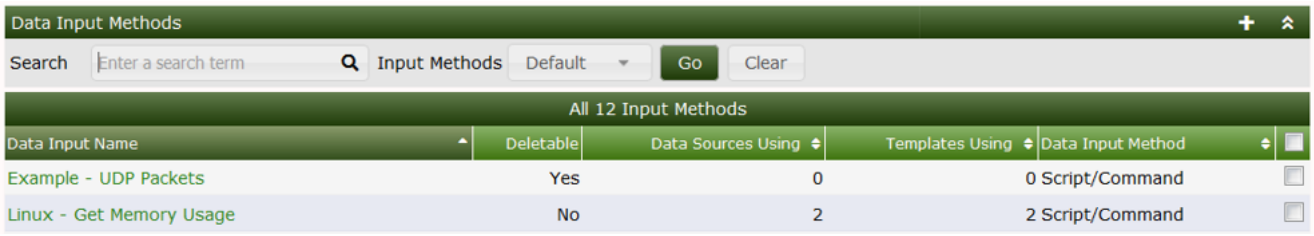


Now udpOutDatagrams:



Be careful to avoid typos. The strings entered here must **exactly** match those spitted out by the script. **Double check Output Fields**! Now, results should be like

Finally Save and be proud!

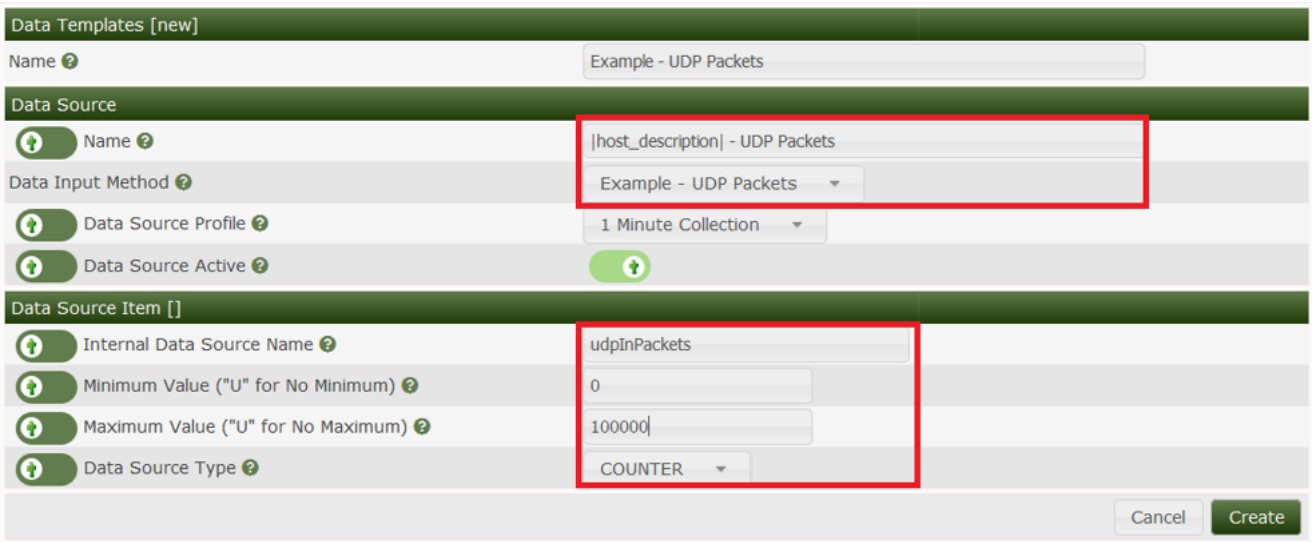


The Data Template

The previous step explained how to call the script that retrieves the data. Now it's time to tell Cacti, how to store them in RRDfiles. You will need a single **Data Template** only, even if two different output fields will be stored. RRD files are able to store more than one output fields; RRDtool's name for those is **Data Source**. So we will create

1. One single **Data Template** representing one RRDfile
2. Two output fields/data sources

The first step is quite the same as Create the **Data Template** for a simple **Data Input Method**. For sure, we provide a different name, **Example - UDP Packets**. Now, let's enter the first **Data Source**. Again, its like above. But we now provide the name of **udpInPackets**, enter a **Maximum value** of 100,000 and select the **Data Source Type** of COUNTER.



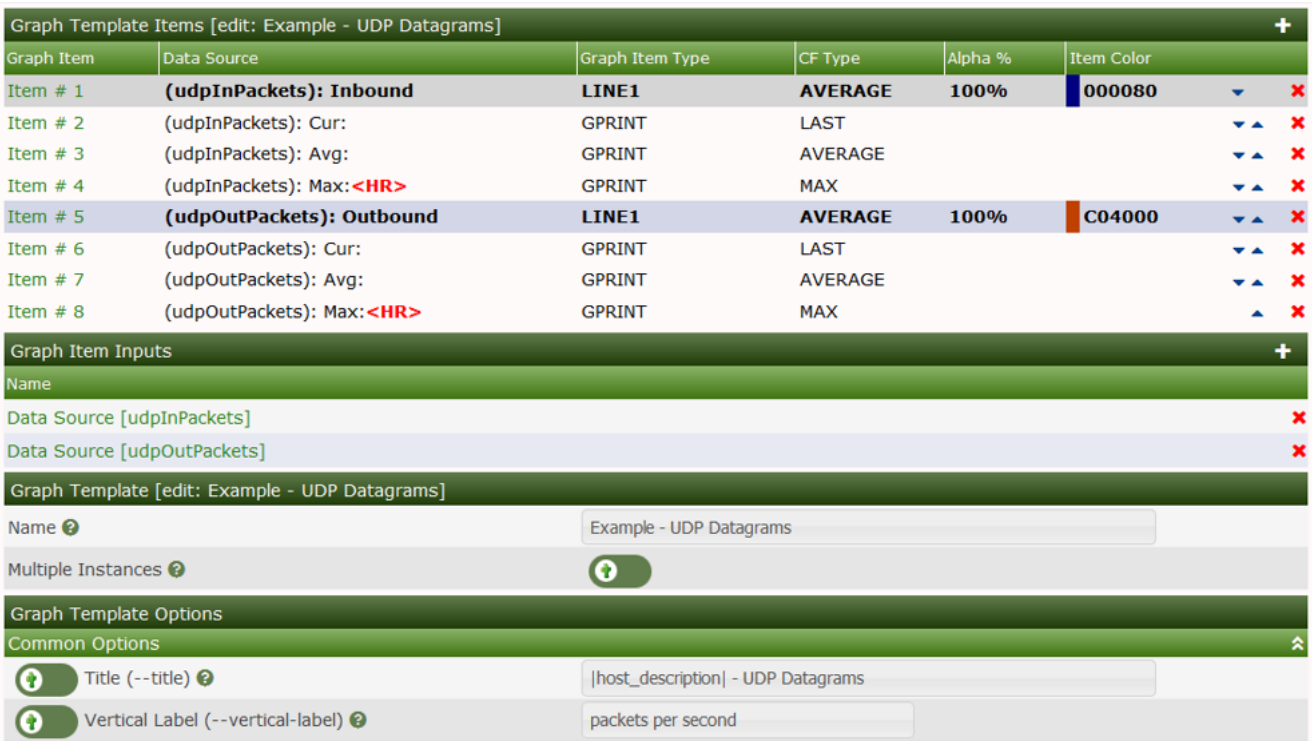
Then save and find with the exception that the *Output Field* will be set to *None Selected*. Therefore, before adding the second *Data Source Item*, make sure you select the *Output Field* of *udpInDatagrams* and press the Save button to create the association.

Then, add the second **Data Source** by pressing the plus (+) glyph and providing data for *udpOutPackets*. Pay attention to select the correct *Output Field* defined by the **Data Input Method** as we did with the *udpInDatagrams*.

To deactivate maximum checking, enter 'U', else the desired number. Do not forget to select the correct *Data Source Type* and the *Output Field* as before.

The Graph Template

Most of this task of creating a **Graph Template** has already been covered in our example. You will basically create a new **Graph Template**, provide a *Line* or an *Area Fill* followed by a *Legend* for both the *udpInDatagrams* and *udpOutDatagrams*. The result of those actions will be the image below:



After you have created your **Graph Template** you can proceed to Add the **Graph Template** to your **Device**, and either allow the automation to create the **Graph** for you, or to manually add it from the **New Graphs** menu pick.

Copyright (c) 2004-2026 The Cacti Group

Data Queries

Overview

Cacti **Data Queries** are not a replacement for **Data Input Methods** in Cacti. Instead they provide an easy way to extend Data Input methods to interpret multidimensional objects, or list data based upon an index, making the data easier to graph. The most common use of a **Data Query within Cacti is to retrieve a list of network interfaces via SNMP.

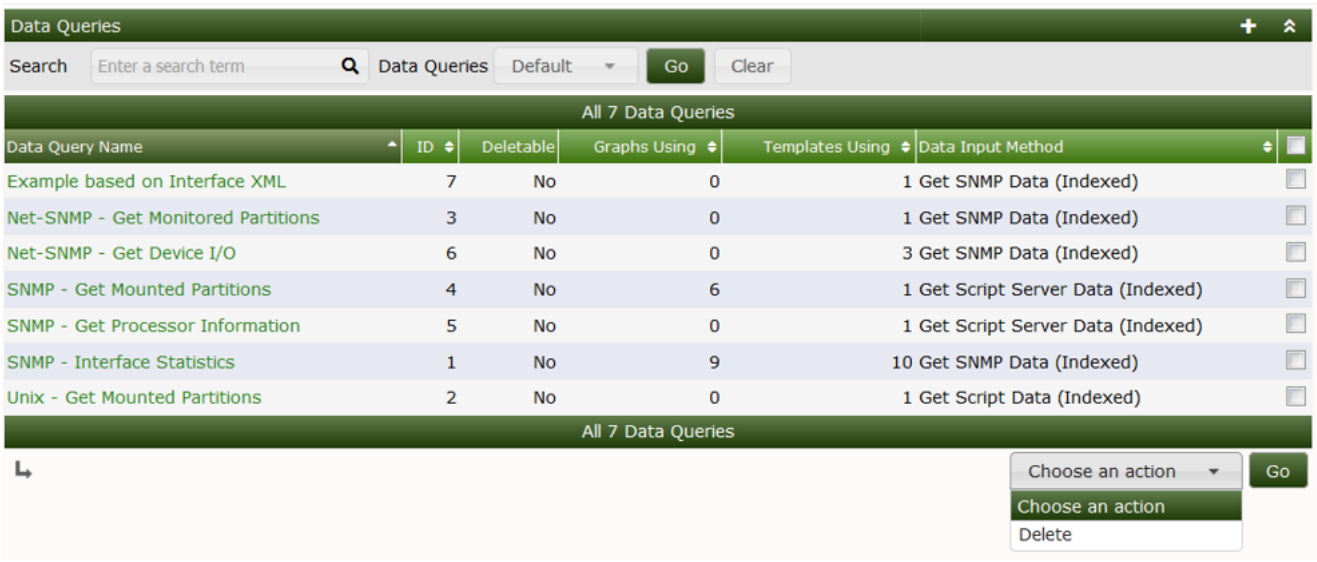
If you want to graph the traffic of a network interface, first Cacti must retrieve a list of interfaces on the host. Second, Cacti can use that information to create the necessary **Graphs** and **Data Sources**. **Data Queries** are only concerned with the first step of the process, that is obtaining a list of network interfaces and not creating the graphs/data sources for them. While listing network interfaces is a common use for **Data Queries**, they also have other uses such as listing partitions, processors, or even cards in a router.

One requirement for any **Data Query** in Cacti, is that it has some unique value that defines each row in the list. This concept follows that of a 'primary key' in SQL, and makes sure that each row in the list can be uniquely referenced. Examples of these index values are 'ifIndex' for SNMP network interfaces or the device name for partitions.

There are three types of **Data queries** that you will see referred to throughout Cacti. They are script queries, script server queries, and SNMP queries. Script, Script Server and SNMP queries are virtually identical in their functionality and only differ in how they obtain their information. Script and Script Server queries will call an external command or script and an SNMP query will make an SNMP call to retrieve a list of data.

User Interface

Below, you can see Cacti's default **Data Queries** interface. From the interface, you can see the Name, ID, Data Input Method, and other information about the Data Query.

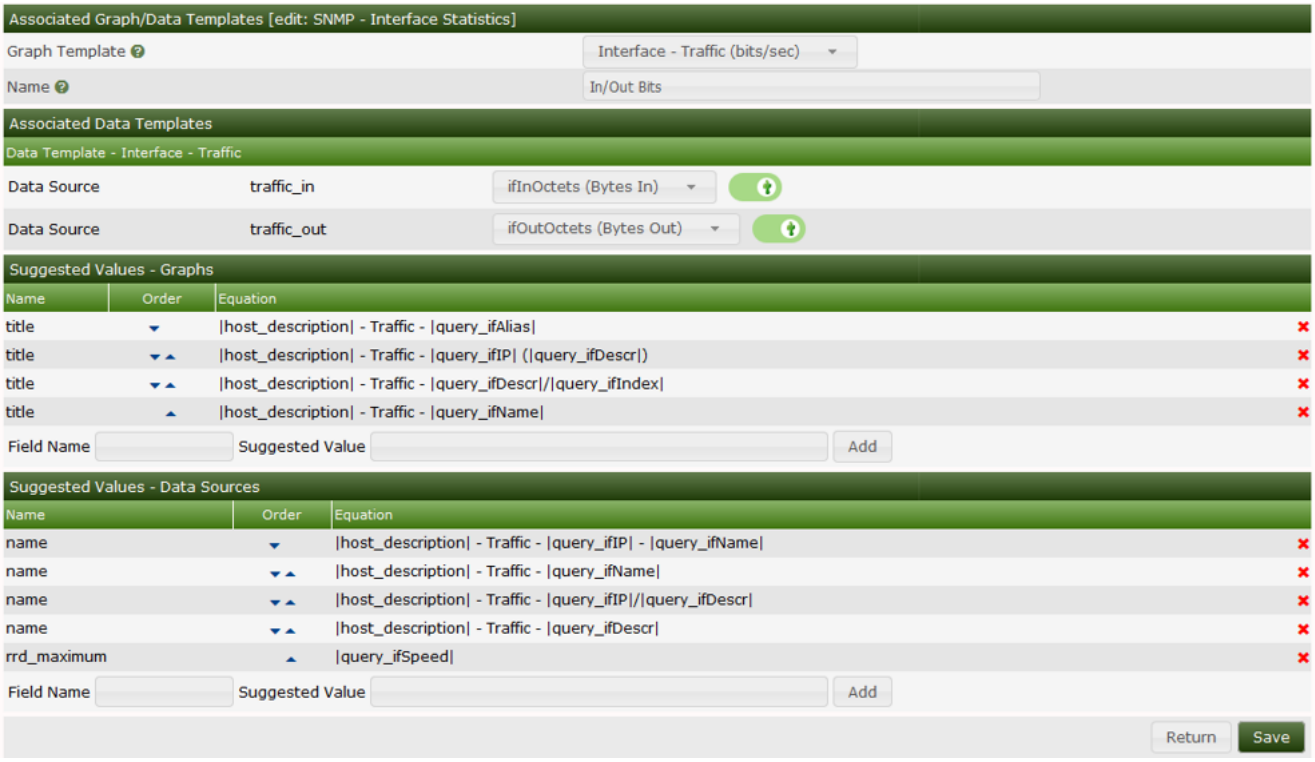


When you edit an existing Data Query, you will see an interface like the one below. In that interface, you will note that there is an XML file path included. The XML file is a key component of a Cacti **Data Query**. For Script based **Data Queries**, there will also be a script that must be called to gather information about the **Data Query**, the script name is included in the XML file.

In this Interface, you can see the Name, Description, XML Path, and **Data Input Method**. In the section below that, you have the `Associated Graph Templates` section which includes all the various **Graph Templates** that can be used for the **Data Query**.

Once a **Data Query Graph Template** is in use, it can not longer be removed from Cacti. This feature is there to prevent **Graphs** from becoming unrenderable if someone accidentally were to remove the **Graph Template** association.

When you click on any of the `Associated Graph Template` name, you will be taken to the interface below.



From here, you map the various matching **Data Template** RRDfile Data Source names to Data Source names that exist in the XML file. There can be many Data Source names in the XML file, but a Graph Template may only require a few of them, which explains the reason for having the `Associated Data Templates` section of the **Data Query**. To associate the XML field with the **Data Templates** RRDfile Data Source, you simply select the correct one, and then toggle on the checkbox to the right of the XML field Data Source name.

Below the `Associated Data Templates`, you will find two sections which are referred to as `Suggested Values` for both the **Data Source** and the **Graph** names. When creating **Graphs** and their associated **Data Sources** Cacti will use the first name in the `Suggested Values` pattern that fully replaces all the tags that are identified by opening and closing vertical bars. So, in the example above, when creating an Interface Graph, if that Interface does not have an ifAlias, Cacti will likely use the second `Suggested Value`, unless the interface does not have an IP Address, in which case it will use the third `Suggested Value`.

Cacti also allows `Suggested Values`, to replace both **Graph** and **Data Source** values, as in the case above, the "rrd_maximum" column is replaced by the Network Interfaces maximum speed, otherwise known as the "ifSpeed" from an SNMP perspective.

All data queries have two parts, the XML file and the definition within Cacti. An XML file must be created for each query, that defines where each piece of information is and how to retrieve it. This could be thought of as the actual query. The second part is a definition within Cacti, which tells Cacti where to find the XML file and associates the data query with one or more graph templates.

Creating a Data Query

Once you have created the XML file that defines your data query, you must add the data query within Cacti. Open **Console > Data Collection > Data Queries** and select **Add**. You will be prompted for some basic information about the data query, described in more detail below.

Table 12-1. Field Description: Data Queries

Name	Description
Name	Give the **Data Query** a name that you will use to identify it. This name will be used throughout Cacti when presented with a list of **Data Queries**.
Description (Optional)	Enter a more detailed description of the data query including the information it queries or additional requirements.
XML Path	Fill in the full path to the XML file that defines this query. You can optionally use the `path_cacti` variable that will be substituted with the full path to Cacti. On the next screen, Cacti will check to make sure that it can find the XML file.
Data Input Method	This is how you tell Cacti to handle the data it receives from the data query. Typically, you will select `Get SNMP Data (Indexed)` for an SNMP query and `Get Script Data (Indexed)` for a script query.

When you are finished filling in all necessary fields, click the Create button to continue. You will be redirected back to the same page, but this time with some additional information to fill in. If you receive a red warning that says 'XML File Does Not Exist', correct the value specified in the 'XML Path' field.

Associated Graph Templates

Every data query must have at least one graph template associated with it, and possibly more depending on the number of output fields specified in the XML file. This is where you get to choose what kind of graphs to generate from this query. For instance, the interface data query has multiple graph template associations, used to graph traffic, errors, or packets. To add a new graph template association, simply click Add at the right of the Associated Graph Templates box. You will be presented with a few fields to fill in:

Table 12-2. Field Description: Associated Graph Templates

Name	Description
Name	Give a name describing what kind of data you are trying to represent or graph. When the user creates a graph using this data query, they will see a list of graph template associations that they will have to choose from.
Graph Template	Choose the actual graph template that you want to make the association with.

When you are finished filling in these fields, click the Create button. You will be redirected back to the same page with some additional information to fill in. Cacti will make a list of each **Data Template** referenced to in your selected **Graph Template** and display them under the `Associated Data Templates` box. As previously mentioned, for each **Data Source** item listed, you must selected the **Data Query** output field that corresponds with it. *Do not forget to check the checkbox to the right of each selection, or your settings will not be saved.*

The `Suggested Values` form provides you a way to control field values of **Data Sources** and **Graphs** created using the **Data Query**. If you specify multiple `Suggested Values` for the same field, Cacti will evaluate them in order which you can control using the up or down arrow icons. For more information about valid field names and variables, read the section on `Suggested Values`.

When you are finished filling in all necessary fields on this form, click the Save button to return to the data queries edit screen. Repeat the steps under this heading as many times as necessary to represent all data in your XML file. When you are finished with this, you should be ready to start adding your data query to hosts.

SNMP Query XML Syntax

```
<query>
   <name>Get SNMP Interfaces</name>
   <description>Queries a host for a list of monitorable interfaces</description>
   <oid_uptime>.1.3.x.x.x</oid_uptime>
   <oid_index>.1.3.6.1.2.1.2.2.1.1</oid_index>
   <oid_index_parse>OID/REGEXP:.*.([0-9]{1,3}.[0-9]{1,3})$</oid_index_parse>
   <oid_num_indexes>.1.3.6.1.2.1.2.1.0</oid_num_indexes>
   <index_order>ifDescr:ifName:ifIndex</index_order>
   <index_order_type>numeric</index_order_type>
   <index_title_format>|chosen_order_field|</index_title_format>

   <fields>
      <ifIndex>
         <name>Index</name>
         <method>walk</method>
         <source>value</source>
         <direction>input</direction>
         <oid>.1.3.6.1.2.1.2.2.1.1</oid>
      </ifIndex>
   </fields>
</query>
```

Table 12-3. SNMP Query XML Settings Reference

Field	Description
name	(Optional) You can enter a "friendly name" for the SNMP Query here. It will not be used by Cacti, and is for identification only.
description	(Optional) You can enter a description for the SNMP Query here. It will not be used by Cacti, and is for identification only.
oid_uptime	If you have another OID that contains ticks (time), say for example a Java VM. Then, you can create a data query that specifies an alternate uptime OID. To implement this for a data query, simply add the oid_uptime XML parameter to your XML file. Then, if you select your re-index method to be Uptime Goes Backward, Cacti will use that OID to detect whether it is time to re-index the host instead of the standard SNMP OID for uptime.

Field	Description
oid_index	Every SNMP Query must have an OID that represents the index values for the query when walked. As described above, any data query in Cacti must contain a field that uniquely identifies each row returned by the query. In the example above, the oid_index points to the OID of ifIndex in the interface MIB. Note: Cacti is able to parse unique indexes from the OID itself. While the regular expression used for parsing the value from the OID is defined below, you must still specify an OID that can be walked by Cacti in order to obtain the list of OID's. Any OID defined for one of your input fields should work in this case. The values returned from the snmpwalk walk will be completely disregarded.
non_unique	If set to true, Cacti will allow the Data Query index to be non-unique, and it will not attempt to change the index_order if for some reason, the chosen index field is not unique.
oid_index_parse	This field should only be used if you are trying to parse the unique index from the OID itself. If this field is defined, to obtain a list of indexes, Cacti walks the OID provided in the oid_index field above. It then applies the regular expression provided in this field to the list of OID's that are returned. The matched substrings that remain become the list of indexes for this SNMP Query.
value_index_parse	This field uses regular expressions to restrict the list of data returned from the get or walk method. For example: `<oid_index>.1.3.6.1.2.1.2.2.1.8</oid_index> <value_index_parse>OID/REGEXP:1</value_index_parse>` will only show the list of up-state interfaces. Tip - When using this field it is also worth using oid_index_parse, example: `<oid_index_parse>OID/REGEXP:.\*\.([0-9]+)$</oid_index_parse>`
oid_num_indexes	An OID that can be queried to determine the total number of available indexes. If specified, this will be used to determine when to automatically re-cache this SNMP Query when it is attached to a device.
index_order	Cacti will attempt to find the best field to index off of based on whether each row in the query is unique and non-null. If specified, Cacti will perform this check on the fields listed here in the order specified. Only input fields can be specified and multiple fields should be delimited with a colon.
index_transient	If set to 'true', this field instructs Cacti not to stop attempting to Graph an index if it disappears from the host cache as a result of a re-index. This setting is for indexes that come and go, for example PPPoE statistics or user statistics which appear when a user log's in, but disappear a few hours after the connection is terminated.
index_order_type	For sorting purposes, specify whether the index is numeric or alphanumeric. numeric: The indexes in this SNMP Query are to be sorted numerically (ie. 1,2,3,10,20,31) alphabetic: The indexes in this SNMP Query are to be sorted alphabetically (1,10,2,20,3,31).
index_title_format	Specify the title format to use when representing an index to the user. Any input field name can be used as a variable if enclosed in pipes (|). The variable `|chosen_order_field|` will be substituted with the field chosen by Cacti to index off of (see index_order above).

Table 12-4. SNMP Query XML Field Settings Reference

Each field contained within the SNMP Query must be defined under the field tag as show in the example above. In addition, each defined field in the SNMP Query must have a unique name given to it. Do not use spaces or any non-alphanumeric characters, this name must be identifiable within Cacti.

Field	Description
name	Here you can specify a "friendly name" for the field. This name will be used by Cacti to help the user identify this field.
method	Tell Cacti how you want it to gather SNMP information for this field. get: The `get` method performs an snmpget for the OID specified for this field appended by the index values derived from `<oid_index>/ <oid_index_parse>`. walk: The `walk` method does a walk of the OID

Field	Description
	specified for this field. If all OIDs belong to the same SNMP table, both methods will return the same values, even though the 'walk' method is typically more efficient.
source	When Cacti obtains a list for this field, you need to tell it how to derive its value for each row. value: The 'value' option simply returns the result of the snmpget for each row. **OID/REGEXP** (regexp_match): The **OID/REGEXP** modifier can be used when you need to use a Perl-compatible regular expression to derive the value from the OID. The most common example of this is to retrieve the IP address of an interface, and can be seen in the 'interface.xml' file. **VALUE/REGEXP** (regexp_match): The **VALUE/REGEXP** modifier can be used to parse the value based on a regular expression, returning the first match. index: Simply use the value of the index for this row as the value. If the index is being parsed from the OID using the oid_index_parse field, you must specify index here. Omit the `<oid>...</oid>` field, then. **OID2HEX/REGEXP** (regexp_match): The **OID2HEX/REGEXP** modifier can be used to parse the OID to extract IPv6 addresses. This is useful for input type fields. It also supports the 'oid_index_parse' expression as well.
direction	input: Input values are the "known" values that you will use to derive the output values, this is where the "query" part of SNMP query comes in. When you create a graph based on an SNMP query, Cacti will prompt you to choose the input value to base the graph on. output: Output values are "unknown" values that are returned from the script. An SNMP query may return multiple statistics for a single index. For instance, a single interface could return bytes/sec in, errors, packets/sec, etc. A rule of thumb is that input fields contain semi-static data that is not graphable, while the output fields contain the data that will be graphed.
oid	You must specify the actual OID that corresponds with the field. Each value for this field can be obtained by doing an snmpget on `oid.(each)snmpindex`.
oid_rewrite_pattern	You can specify and **OID/REGEXP** string which will be replaced by the oid_rewrite_replacement string if found.
oid_rewrite_replacement	You must specify the actual OID that corresponds with the field. Each value for this field can be obtained by doing an snmpget on `oid.(each)snmpindex`.
oid_suffix	The suffix, if specified will be appended to the each oid from the walk or get response, prior to the oid_rewrite_pattern to oid_rewrite_replace regular expression replace. If the method is get, Cacti will issue an snmpget to obtain the resulting data.
rewrite_index	Modifies the snmp values using the rewrite_index expression.
output_format	Determines the output format of the column values. Options are 'ascii', 'hex'. If not specified, Cacti will allow SNMP to 'guess' the output format.

Script Query XML Syntax

```
<query>
   <name>Get Unix Mounted Partitions</name>
   <description>Queries a list of mounted partitions on a unix-based host  with the 'df' command.</description>
   <script_path>perl |path_cacti|/scripts/query_unix_partitions.pl</script_path>
   <arg_index>index</arg_index>
   <arg_query>query</arg_query>
   <arg_get>get</arg_get>
   <arg_num_indexes>num_indexes</arg_num_indexes>
   <output_delimeter>:</output_delimeter>
   <index_order>dskDevice:dskMount</index_order>
   <index_order_type>alphabetic</index_order_type>
   <index_title_format>|chosen_order_field|</index_title_format>

   <fields>
      <dskDevice>
         <name>Device Name</name>
         <direction>input</direction>
```

```
          <query_name>device</query_name>
       </dskDevice>
    </fields>
 </query>
```

Table 12-5. Script Query XML Settings

Field	Description
name	(Optional) You can enter a "friendly name" for the script query here. It will not be used by Cacti, and is for identification only.
description	(Optional) You can enter a description for the script query here. It will not be used by Cacti, and is for identification only.
script_path	Enter the complete path to the script or executable that is going to handle your script query. When in doubt, specify the pull path to all binaries referenced in this path, the query may not execute otherwise.
arg_index	Enter the argument that is to be passed to the script to retrieve a list of indexes.
arg_query	Enter the argument that is to be passed to the script to retrieve a list of values given a field name.
arg_get	Enter the argument that is to be passed to the script to retrieve a single value given a field name and index value.
arg_num_indexes	Enter the argument that is to be passed to the script to determine the total number of available indexes. If specified, this will be used to determine when to automatically re-cache this script query when it is attached to a device.
output_delimeter	Enter the one character delimiter that will be used to separate output values. This is only used when you "query" the script in which case it outputs 'index(delimiter)value'.
index_order	Cacti will attempt to find the best field to index off of based on whether each row in the query is unique and non-null. If specified, Cacti will perform this check on the fields listed here in the order specified. Only input fields can be specified; separate multiple fields with a colon, as in `ifDescr:ifName:ifIndex`.
index_transient	If set to 'true', this field instructs Cacti not to stop attempting to Graph an index if it disappears from the host cache as a result of a re-index. This setting is for indexes that come and go, for example PPPoE statistics or user statistics which appear when a user log's in, but disappear a few hours after the connection is terminated.
index_order_type	For sorting purposes, specify whether the index is numeric or alphanumeric. numeric: The indexes in this script query are to be sorted numerically (ie. 1,2,3,10,20,31) alphabetic: The indexes in this script query are to be sorted alphabetically (1,10,2,20,3,31).
index_title_format	Specify the title format to use when representing an index to the user. Any input field name can be used as a variable if enclosed in pipes (|). The variable |chosen_order_field| will be substituted with the field chosen by Cacti to index off of (see index_order above).

Table 12-6. Script Query XML Field Settings

Each field contained within the Script Query must be defined under the field tag as show in the example above. In addition, each defined field in the Script Query must have a unique name given to it. Do not use spaces or any non-alphanumeric characters, this name must be identifiable within Cacti.

Field	Description
name	Here you can specify a "friendly name" for the field. This name will be used by Cacti to help the user identify this field.
direction	input: Input values are the "known" values that you will use to derive the output values, this is where the "query" part of script query comes in. When

Field	Description
	you create a graph based on a script query, Cacti will prompt you to choose the input value to base the graph on. output: Output values are "unknown" values that are returned from the script. A script query may return multiple statistics for a single index. For instance, a single partition could return free disk space, total disk space, fragmentation percentage, etc. A rule of thumb is that input fields contain semi-static data that is not graphable, while the output fields contain the data that will be graphed.
query_name	Enter the name that Cacti must use when asking the script for information about this field. For instance, the following should return values: '(script_name) query (query_name)'.

Script Server XML Syntax

```
<query>
    <name>Get Host MIB CPUs</name>
    <script_path>|path_cacti|/scripts/ss_host_cpu.php</script_path>
    <script_function>ss_host_cpu</script_function>
    <script_server>php</script_server>
    <arg_prepend>|host_hostname| |host_id| |host_snmp_version|:|host_snmp_port|:|host_snmp_timeout|:|
host_ping_retries|:|host_max_oids|:|host_snmp_community|:|host_snmp_username|:|host_snmp_password|:|
host_snmp_auth_protocol|:|host_snmp_priv_passphrase|:|host_snmp_priv_protocol|:|host_snmp_context|</arg_prepend>
    <arg_index>index</arg_index>
    <arg_query>query</arg_query>
    <arg_get>get</arg_get>
    <arg_num_indexes>num_indexes</arg_num_indexes>
    <output_delimeter>!</output_delimeter>
    <index_order>hrProcessorFrwID</index_order>
    <index_order_type>numeric</index_order_type>
    <index_title_format>CPU#|chosen_order_field|</index_title_format>

    <fields>
        <hrProcessorFrwID>
            <name>Processor Index Number</name>
            <direction>input</direction>
            <query_name>index</query_name>
        </hrProcessorFrwID>

        <hrProcessorLoad>
            <name>Processor Usage</name>
            <direction>output</direction>
            <query_name>usage</query_name>
        </hrProcessorLoad>
    </fields>
</query>
```

Table 12-7. Script Server XML Settings

Field	Description
name	(Optional) You can enter a "friendly name" for the script query here. It will not be used by Cacti, and is for identification only.
description	(Optional) You can enter a description for the script query here. It will not be used by Cacti, and is for identification only.
script_path	Enter the complete path to the script or executable that is going to handle your script query. When in doubt, specify the pull path to all binaries referenced in this path, the query may not execute otherwise.
script_function	The function within the script in the case where the script can be used for multiple data collection functions. Most script server scripts are associated with only one Data Query, but it's possible to have one script perform the collection for multiple Data Queries.
arg_prepend	A list of device variables from the host to prepend after the script function when calling the script. This information can be used to identify the device that is being monitored or pass certain device information to the script.

Field	Description
arg_index	Enter the argument that is to be passed to the script to retrieve a list of indexes.
arg_query	Enter the argument that is to be passed to the script to retrieve a list of values given a field name.
arg_get	Enter the argument that is to be passed to the script to retrieve a single value given a field name and index value.
arg_num_indexes	Enter the argument that is to be passed to the script to determine the total number of available indexes. If specified, this will be used to determine when to automatically re-cache this script query when it is attached to a device.
output_delimeter	Enter the one character delimiter that will be used to separate output values. This is only used when you "query" the script in which case it outputs 'index(delimiter)value'.
index_order	Cacti will attempt to find the best field to index off of based on whether each row in the query is unique and non-null. If specified, Cacti will perform this check on the fields listed here in the order specified. Only input fields can be specified; separate multiple fields with a colon, as in `ifDescr:ifName:ifIndex`.
index_transient	If set to 'true', this field instructs Cacti not to stop attempting to Graph an index if it disappears from the host cache as a result of a re-index. This setting is for indexes that come and go, for example PPPoE statistics or user statistics which appear when a user log's in, but disappear a few hours after the connection is terminated.
index_order_type	For sorting purposes, specify whether the index is numeric or alphanumeric. numeric: The indexes in this script query are to be sorted numerically (ie. 1,2,3,10,20,31) alphabetic: The indexes in this script query are to be sorted alphabetically (1,10,2,20,3,31).
index_title_format	Specify the title format to use when representing an index to the user. Any input field name can be used as a variable if enclosed in pipes (|). The variable |chosen_order_field| will be substituted with the field chosen by Cacti to index off of (see index_order above).

Table 12-6. Script Server XML Field Settings

Each field contained within the Script Query must be defined under the field tag as show in the example above. In addition, each defined field in the Script Query must have a unique name given to it. Do not use spaces or any non-alphanumeric characters, this name must be identifiable within Cacti.

Field	Description
name	Here you can specify a "friendly name" for the field. This name will be used by Cacti to help the user identify this field.
direction	input: Input values are the "known" values that you will use to derive the output values, this is where the "query" part of script query comes in. When you create a graph based on a script query, Cacti will prompt you to choose the input value to base the graph on. output: Output values are "unknown" values that are returned from the script. A script query may return multiple statistics for a single index. For instance, a single partition could return free disk space, total disk space, fragmentation percentage, etc. A rule of thumb is that input fields contain semi-static data that is not graphable, while the output fields contain the data that will be graphed.
query_name	Enter the name that Cacti must use when asking the script for information about this field. For instance, the following should return values: '(script_name) query (query_name)'.

Copyright (c) 2004-2026 The Cacti Group

How To Work with Data Queries

During the past few weeks as I've made various SNMP Data Query templates I've ran into a few 'tricks' (which existing user scripts use) to make them work with most devices. It'd probably be beneficial to publish these in some manner on the documentation site so others don't have to stumble.

As you already know getting an index is the hardest thing users struggle with. I've ran across three scenarios and their solutions:

SNMP index is present

Basically mimic the interface.xml template which comes with Cacti `(cacti/resource/snmp_queries/interface.xml)`.

SNMPWALK:

```
Raw OIDs:
.1.3.6.1.2.1.2.2.1.1.1 = INTEGER: 1
.1.3.6.1.2.1.2.2.1.1.2 = INTEGER: 2
.1.3.6.1.2.1.2.1.0 = INTEGER: 2
```

```
Textual MIB names:
IF-MIB::ifIndex.1 = INTEGER: 1
IF-MIB::ifIndex.2 = INTEGER: 2
IF-MIB::ifNumber.0 = INTEGER: 2
```

In the SNMP XML file, specify the `<oid_index>` and optional `<oid_num_indexes>` fields. For example:

```
<interface>
    <name>Get SNMP Interfaces</name>
    <description>Queries a host for a list of monitorable interfaces</description>
    <oid_index>.1.3.6.1.2.1.2.2.1.1</oid_index>
    <oid_num_indexes>.1.3.6.1.2.1.2.1.0</oid_num_indexes>

    <fields>
        <ifIndex>
            <name>Index</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.2.2.1.1</oid>
        </ifIndex>
    </fields>
</interface>
```

A verbose query from inside Cacti shows the following:

```
+ Running data query [1].
+ Found type = '3' [snmp query].
+ Found data query XML file at 'C:/Inetpub/wwwroot/cacti/resource/snmp_queries/interface.xml'
+ XML file parsed ok.
+ Executing SNMP walk for list of indexes @ '.1.3.6.1.2.1.2.2.1.1'
+ Index found at OID: 'iso.3.6.1.2.1.2.2.1.1.1' value: '1'
+ Index found at OID: 'iso.3.6.1.2.1.2.2.1.1.2' value: '2'
+ Located input field 'ifIndex' [walk]
+ Executing SNMP walk for data @ '.1.3.6.1.2.1.2.2.1.1'
+ Found item [ifIndex='1'] index: 1 [from value]
+ Found item [ifIndex='2'] index: 2 [from value]
```

SNMP Index is a STRING

Some devices (like on a F5 BigIP) use a STRING to represent the Index due to the fact it is user configurable. In this case we need Cacti to parse out the variable length Index.

Sample of a STRING Index:

Raw OID:

```
.1.3.6.1.4.1.3375.2.2.10.2.3.1.1.5.76.88.87.69.66 = STRING: "LXWEB"
.1.3.6.1.4.1.3375.2.2.10.2.3.1.1.6.69.80.88.87.69.66 = STRING: "EPXWEB"
```

Textual MIB names:

```
F5-BIGIP-LOCAL-MIB::ltmVirtualServStatName."LXWEB" = STRING: LXWEB
F5-BIGIP-LOCAL-MIB::ltmVirtualServStatName."EPXWEB" = STRING: EPXWEB
```

Next we need to define a OID/REGEXP that will look for the start of the STRING and create the Index. In this case the regexp "^.{32}" matches the first 32 characters beginning at the start of the OID -- this is the length of the OID from the leading dot up to the start of the index. Then the capturing parens "(.*)" grab the remainder of the OID as a backreference. Use the following SNMP XML script:

```
<interface>
    <name>Get F5 Big-IP Load Balancer Virtual Server Statistics</name>
    <oid_index>.1.3.6.1.4.1.3375.2.2.10.2.3.1.1</oid_index>
        <oid_index_parse>OID/REGEXP:^.{32}(.*)</oid_index_parse>
        <index_order>vsIndex</index_order>
        <index_order_type>alphabetic</index_order_type>

        <fields>
        <vsIndex>
            <name>Index</name>
            <source>index</source>
            <direction>input</direction>
        </vsIndex>
    </fields>
</interface>
```

After creating the Data Query Data & Graph templates in Cacti a verbose query will spit out the following:

```
+ Running data query [20].
+ Found type = '3' [snmp query].
+ Found data query XML file at 'C:/Inetpub/wwwroot/cacti/resource/snmp_queries/f5_bigip_vs.xml'
+ XML file parsed ok.
+ Executing SNMP walk for list of indexes @ '.1.3.6.1.4.1.3375.2.2.10.2.3.1.1'
+ Index found at OID: 'enterprises.3375.2.2.10.2.3.1.1.5.76.88.87.69.66' value: 'LXWEB'
+ Index found at OID: 'enterprises.3375.2.2.10.2.3.1.1.6.69.80.88.87.69.66' value: 'EPXWEB'
+ index_parse at OID: 'enterprises.3375.2.2.10.2.3.1.1.5.76.88.87.69.66' results: '5.76.88.87.69.66'
+ index_parse at OID: 'enterprises.3375.2.2.10.2.3.1.1.6.69.80.88.87.69.66' results: '6.69.80.88.87.69.66'
+ Inserting index data for field 'vsIndex' [value='5.76.88.87.69.66']
+ Inserting index data for field 'vsIndex' [value='6.69.80.88.87.69.66']
```

SNMP Index does not exist

Some devices do not offer a traditional Index OID to query. Here we need to create/fake the index for Cacti. This is accomplished once again with a simple REGEX to split off the last OID (or middle, etc) to create the index for cacti.

SNMPWALK:

Raw OIDs:

```
.1.3.6.1.2.1.7.1.0 = Counter32: 69367
.1.3.6.1.2.1.7.2.0 = Counter32: 11
.1.3.6.1.2.1.7.3.0 = Counter32: 0
.1.3.6.1.2.1.7.4.0 = Counter32: 64392
```

Textual MIB name:

```
UDP-MIB::udpInDatagrams.0 = Counter32: 69458
UDP-MIB::udpNoPorts.0 = Counter32: 11
UDP-MIB::udpInErrors.0 = Counter32: 0
UDP-MIB::udpOutDatagrams.0 = Counter32: 64471
```

Consulting the MIB file for these OIDs, we know that the associated index will always be "0" for each OID. We'll want to split this off with the REGEX.

Create the following SNMP XML file:

```
<interface>
    <name>Get RFC1213 Stats</name>
    <description>Management Information Base for Network Management of TCP/IP-based internets</description>
    <oid_index>.1.3.6.1.2.1.7.2</oid_index>
    <oid_index_parse>OID/REGEXP:.*\.([0-9]{1,2})$</oid_index_parse>
    <index_order>Index</index_order>
    <index_order_type>alphabetic</index_order_type>

    <fields>
        <Index>
            <name>Index</name>
            <source>index</source>
            <direction>input</direction>
        </Index>
    </fields>
</interface>
```

Once all of the required templates have been created, a verbose query from Cacti results in the following:

```
+ Running data query [23].
+ Found type = '3' [snmp query].
+ Found data query XML file at 'C:/Inetpub/wwwroot/cacti/resource/snmp_queries/RFC1213.xml'
+ XML file parsed ok.
+ Executing SNMP walk for list of indexes @ '.1.3.6.1.2.1.7.2'
+ Index found at OID: 'iso.3.6.1.2.1.7.2.0' value: '11'
+ index_parse at OID: 'iso.3.6.1.2.1.7.2.0' results: '0'
+ Inserting index data for field 'Index' [value='0']
```

A note on Textual SNMP MIBS and SPINE

Please be aware the SPINE does not parse out textual Mibs. This means that if you were to use an OID that contained a string reference rather than numerical this will cause Graphs to fail as they will not be generated due to spine not receiving a value at each poll

Example of a bad OID reference for spine:

```
interfaces.mib.1.1.1.1.1
```

Copyright (c) 2004-2026 The Cacti Group

SNMP Data Queries

Walkthrough: A New SNMP Data Query

For SNMP Queries, you won't need to create a data retrieval script. Cacti will use SNMP to retrieve information. But cacti will need additional information on how the indexed data is structured. Think about a table (a MIB table in this case); you'll have to tell cacti about the table structure. This is done by defining an XML file. Basically, you have to define the index to tell cacti about the number of rows and about their unique index. This index is later used to access each rows data. Furthermore, you may define columns, that serve as **descriptive fields** to be shown in the selection table. The XML file knows them as

```
<direction>input</direction>
```

You will have to define those fields, that will be queried for the readings, e.g. ifInOctets, ifOutOctets, ifInErrors, …The XML file knows them as

```
<direction>output</direction>
```

Lets have an example: standard Interface MIB with the corresponding part of the `<path_cacti>/resources/snmp_queries/interfaces.xml` file are displayed using the following table:

OID	Name		Type	Value	cacti XML
.1.3.6.1.2.1.2.1.0	IF-MIB::ifNumber.0	=	INTEGER:	3	<oid_num_indexes> .1.3.6.1.2.1.2.1.0 </oid_num_indexes>

.1.3.6.1.2.1.2.2.1.1.1	IF-MIB::ifIndex.1	=	INTEGER:	1	<ifIndex> <name>Index</name> <method>walk</method> <source>value</source> <direction>input</direction> <oid>.1.3.6.1.2.1.2.2.1.1</oid> </ifIndex>
.1.3.6.1.2.1.2.2.1.1.2	IF-MIB::ifIndex.2	=	INTEGER:	2	
.1.3.6.1.2.1.2.2.1.1.3	IF-MIB::ifIndex.3	=	INTEGER:	3	

.1.3.6.1.2.1.2.2.1.2.1	IF-MIB::ifDescr.1	=	STRING:	lo	<ifDescr> <name>Description</name> <method>walk</method> <source>value</source> <direction>input</direction> <oid>.1.3.6.1.2.1.2.2.1.2</oid> </ifDescr>
.1.3.6.1.2.1.2.2.1.2.2	IF-MIB::ifDescr.2	=	STRING:	eth0	
.1.3.6.1.2.1.2.2.1.2.3	IF-MIB::ifDescr.3	=	STRING:	sit0	

.1.3.6.1.2.1.2.2.1.3.1	IF-MIB::ifType.1	=	INTEGER:	software Loopback (24)	<ifType> <name>Type</name> <method>walk</method> <source>value</source> <direction>input</direction> <oid>.1.3.6.1.2.1.2.2.1.3</oid> </ifType>
.1.3.6.1.2.1.2.2.1.3.2	IF-MIB::ifType.2	=	INTEGER:	ethernet Csmacd (6)	
.1.3.6.1.2.1.2.2.1.3.3	IF-MIB::ifType.3	=	INTEGER:	tunnel (131)	

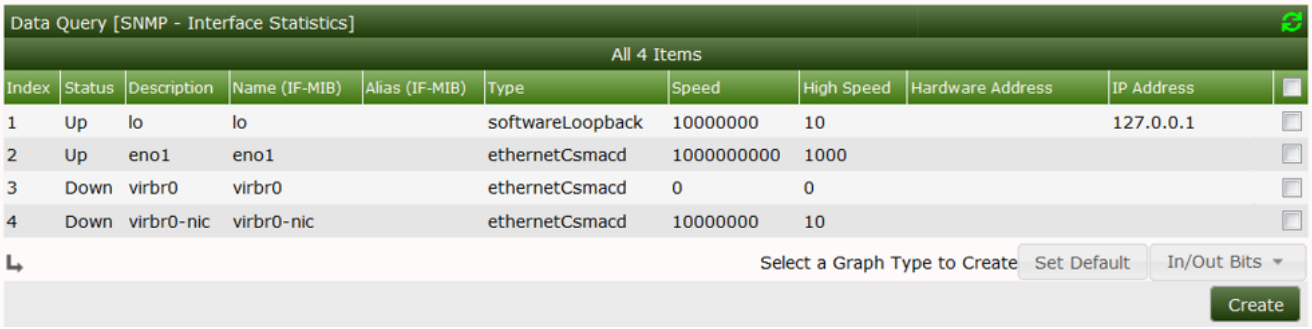
.1.3.6.1.2.1.2.2.1.5.1	IF-MIB::ifSpeed.1	=	Gauge32:	10000000	<ifSpeed> <name>Speed</name> <method>walk</method> <source>value</source> <direction>input</direction> <oid>.1.3.6.1.2.1.2.2.1.5</oid> </ifSpeed>
.1.3.6.1.2.1.2.2.1.5.2	IF-MIB::ifSpeed.2	=	Gauge32:	100000000	

.1.3.6.1.2.1.2.2.1.8.1	IF-MIB::ifOperStatus.1	=	INTEGER:	up(1)	<ifOperStatus> <name>Status</name> <method>walk</method> <source>value</source> <direction>input</direction> <oid>.1.3.6.1.2.1.2.2.1.8</oid> </ifOperStatus>
.1.3.6.1.2.1.2.2.1.8.2	IF-MIB::ifOperStatus.2	=	INTEGER:	up(1)	
.1.3.6.1.2.1.2.2.1.8.3	IF-MIB::ifOperStatus.3	=	INTEGER:	down(2)	

.1.3.6.1.2.1.2.2.1.10.1	IF-MIB::ifInOctets.1	=	Counter32:	18996826	<ifInOctets> <name>Bytes In</name> <method>walk</method> <source>value</source> <direction>output</direction> <oid>.1.3.6.1.2.1.2.2.1.10</oid> </ifInOctets>
.1.3.6.1.2.1.2.2.1.10.2	IF-MIB::ifInOctets.2	=	Counter32:	1229967	
.1.3.6.1.2.1.2.2.1.10.3	IF-MIB::ifInOctets.3	=	Counter32:	0	

.1.3.6.1.2.1.2.2.1.16.1	IF-MIB::ifOutOctets.1	=	Counter32:	18998670	<ifOutOctets> <name>Bytes Out</name> <method>walk</method> <source>value</source> <direction>output</direction> <oid>.1.3.6.1.2.1.2.2.1.16</oid> </ifOutOctets>
.1.3.6.1.2.1.2.2.1.16.2	IF-MIB::ifOutOctets.2	=	Counter32:	474292	
.1.3.6.1.2.1.2.2.1.16.3	IF-MIB::ifOutOctets.3	=	Counter32:	0	

and see the corresponding table structure when defining **New Graphs** for that **Device** (my laptop):



Now you can map

- Index: IF-MIB::ifIndex
- Status: IF-MIB::ifOperStatus
- Description: IF-MIB::ifDescr
- Type: IF-MIB::ifType
- Speed: IF-MIB::ifSpeed

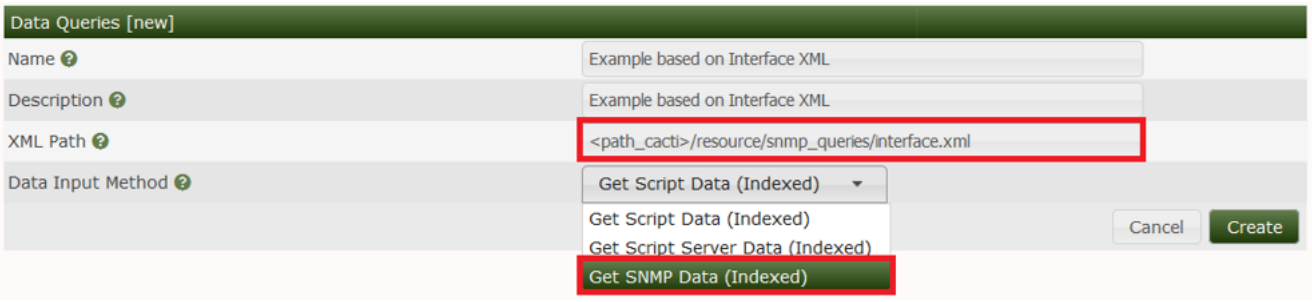
All those are `<direction>input</direction>` Parameters. They serve as descriptive information to each row to help you identify the proper interface to use. Those parameters of `<direction>output</direction>` can be compared to output parameters of a script (see ping.pl script above). These are the readings from the device. By selecting the appropriate row (the one grayed out had been selected by me), you tell cacti to retrieve data from the interface defined by this index. But how does cacti know, what output parameters it shall retrieve? See the Select a Graph type dropdown. It specifies a **Graph Template** defined for this Data Query. The Graph Template in turn references a **Data Template** which incorporates the needed output parameters as Data Sources. This works quite the same way as defined for a Data Input Method.

To sum up: the SNMP XML file is somehow a **replacement** for the **Data Input Method** described above to be used on indexed values. It tells cacti, what data it should retrieve (direction: output). To help you identifying the relevant indexes, the XML defines descriptive parameters (direction: input) to be displayed in the selection table.

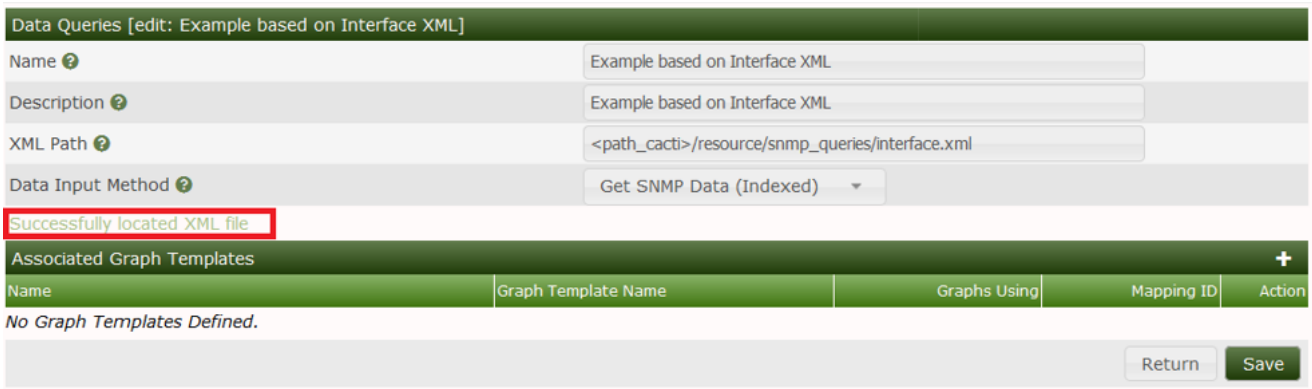
A walkthrough for this is given now. It is based on the already supplied interfaces.xml file.

Create a Data Query to tell cacti how to retrieve data

Go to **Data Queries** and click the plus (+) glyph to see:



Here, we are using the already existing interface.xml file. Select **Get SNMP Data (Indexed)** as **Data Input Method**. Press **Create** to see:

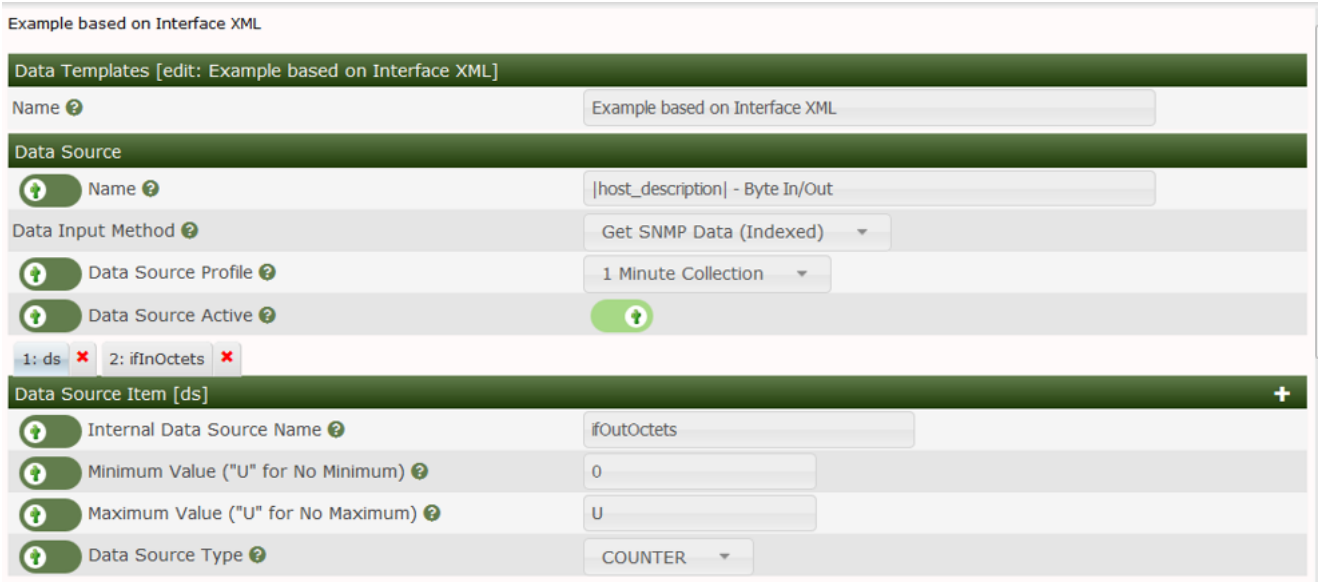


See, that cacti found the XML file. Don't bother with the Associated Graph Templates at the moment. The success message does not include checking of the XML file's content. Not lets proceed to the next definitions.

Create a Data Template to tell Cacti how to store data

This is the exact copy of the definitions made above. So I do not repeat everything here. **Data Input Method** must be selected as **Get SNMP Data (Indexed)**. As this data source is a COUNTER type, select this as the *Data Source Type*. But after saving the new Data Source definition, you may want to define a second Data Source to the same **Data Template**. To do so, select New from the *Data Source Item* heading to see:

The name of the internal RRDtool Data Source (ifOutOctets) is not replaced in the Tab until you save your work. By default, *Maximum Value* is set to 100. This is way too low for an interface. All readings above this value will be stored as NaN by RRDtool. To avoid this, set to 0 (no clipping) or to a reasonable value (e.g. interface speed). Don't forget to specify COUNTER! You will have noticed, that the name of the data source does not match the Name in the interface.xml. Don't worry, the solution to this is given later on. Before leaving, pay attention to the bottom of the page:



This is specific to indexed **Data Queries**. You will have to check the last three items to make indexing work. All other items should be left alone, there values will be taken from the appropriate device definitions. Now click **Save** and you're done with this step.

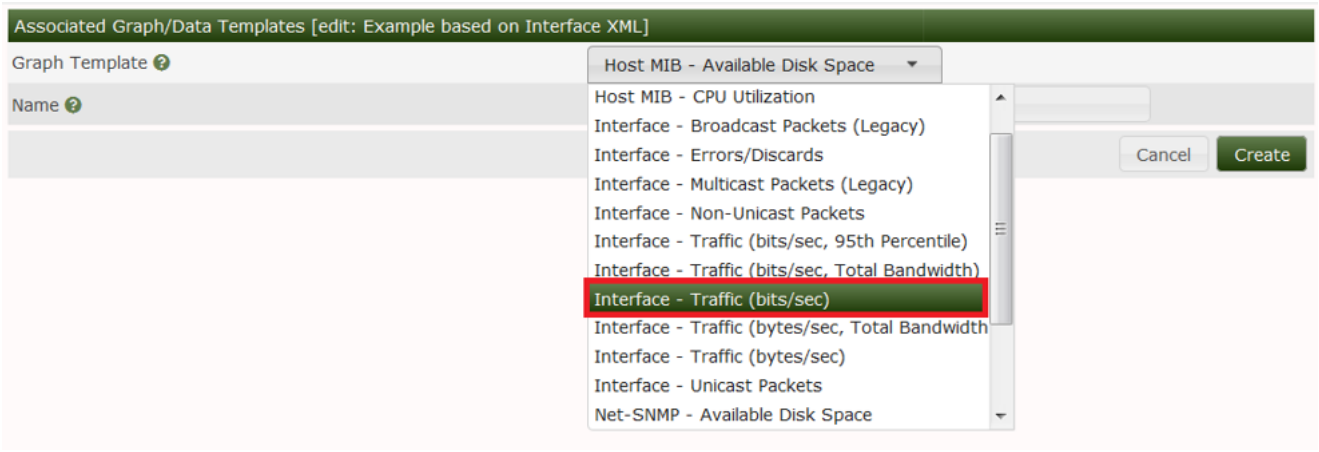
> Note that if you wish to hand Edit mappings of various indexed interfaces manually for each **Graph** in the future, you should check the following three fields *Index Type*, *Index Value* and *Output Type ID* as shown below. Do this for each **Data Template** *Data Source Items* in the **Data Template**.

Create a Graph Template to tell cacti how to present the data

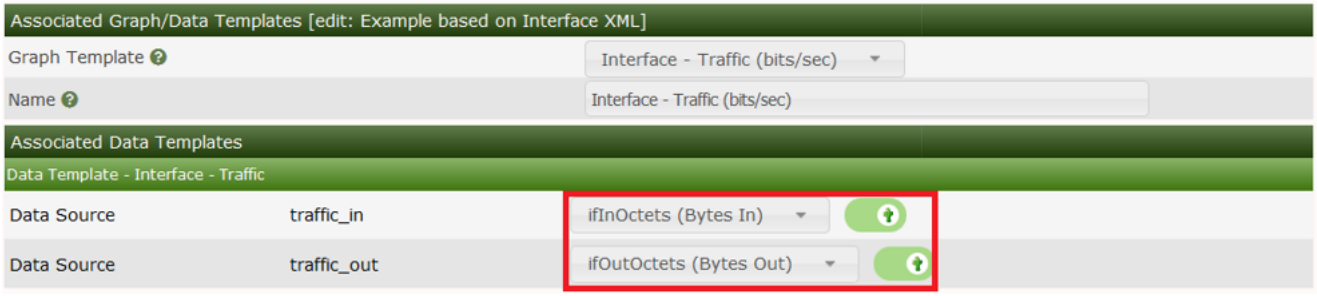
Since creating a new **Data Query** based upon an existing Data Query called *SNMP - Interface Statistics* we don't need to create any **Graph Templates**, we can use the ones created for the other **Data Query**.

Add Graph Template to the Data Query

Now it's time to re-visit our **Data Query**. Remember the **Associated Graph Template** we've left alone in the very first step? Now it will get a meaning. Go to **Data Queries** and select our new one. Then press the plus (+) glyph, and select *Interface - Traffic (bits/second)* from the drop down as shown below. Press **Create** to initialize the association of this **Graph Template** with out new **Data Query**.



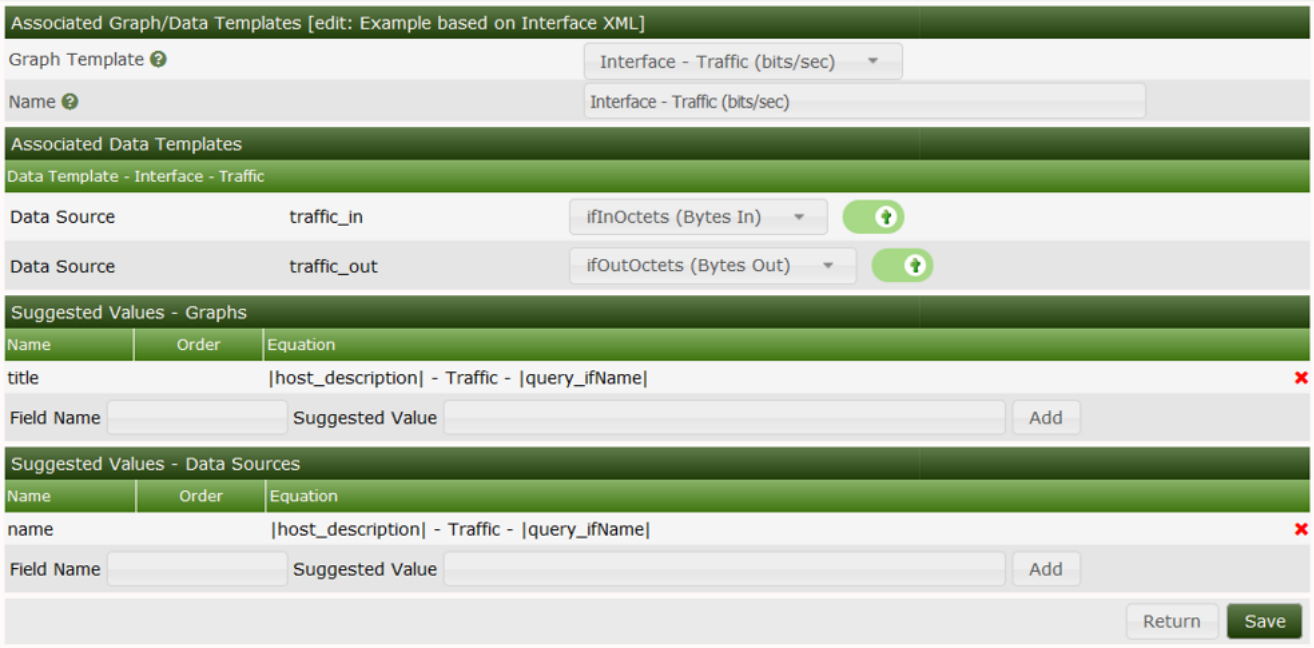
Once we press the **Create** button, we should get an image like the one below. From here, it's important that we associate the *Data Sources* to the XML file columns that retrieve the data using SNMP. Press the **Save** button to complete the mapping of *Data Sources* to the XML file *Data Source Output Values*.



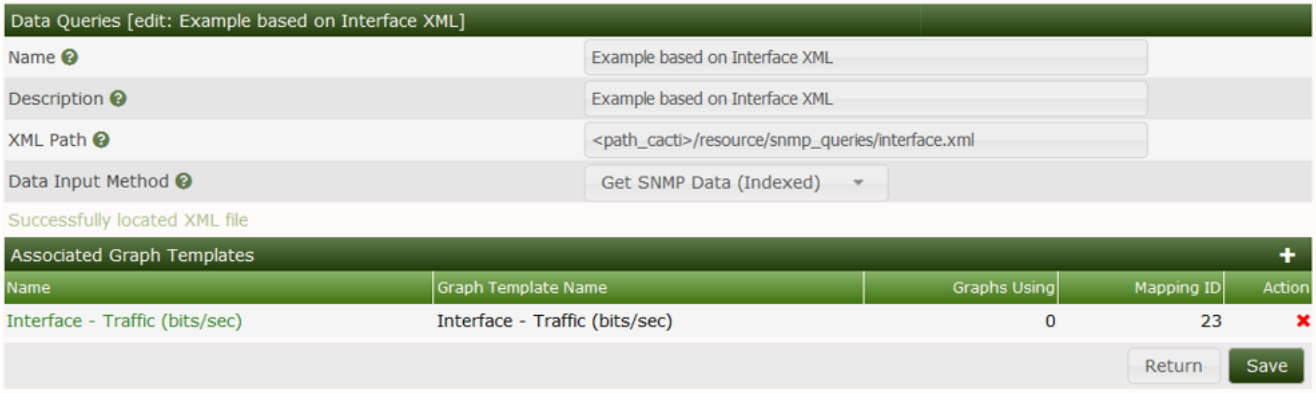
In addition, we need to setup any *Suggested Values* as you can see from the example below. *Suggested Values* are optional when the **Data Query** can have a multitude of suggested names based upon the output from the **Data Query**. For both the *Suggested Values - Graphs* and *Suggested Values - Data Sources*, we must include the name of the column, and the *Suggested Value* which can includes the special tag |*query_xmlcolumn*| as shown in the examples below.

Once the **Associated Graph Template** mappings have been completed, you should see the image below, where we have:

- Mapped the internal **Data Template** *Data Sources* to the XML file *Input Data Sources*
- Added any *Suggested Values* for the *Graph Title* or other **Graph** fields
- Added any *Suggested Values* for the *Data Source* name or other **Data Source** fields

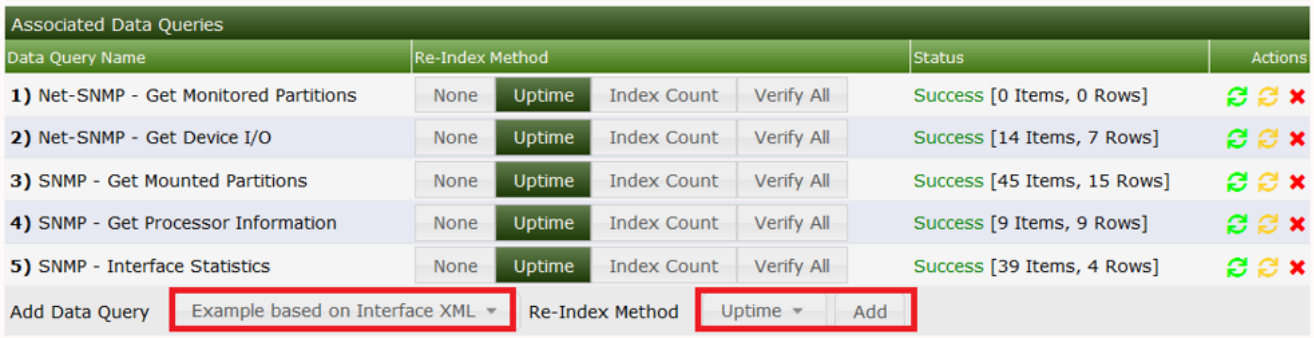


Once you have confirmed all the values, you can press **Return** to return to the **Data Query** page for our Example **Data Query** as shown below.

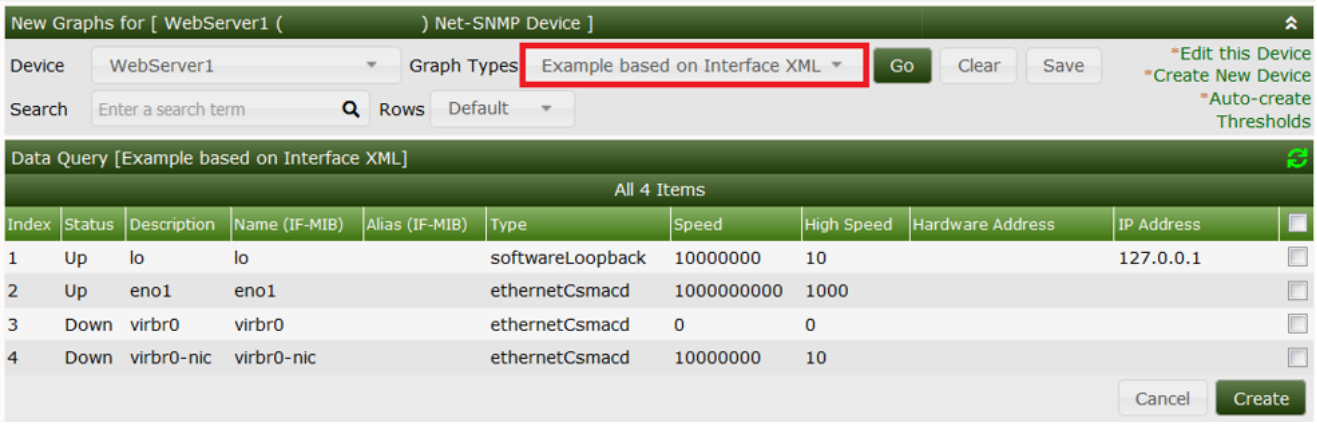


Apply the Data Query to your Device

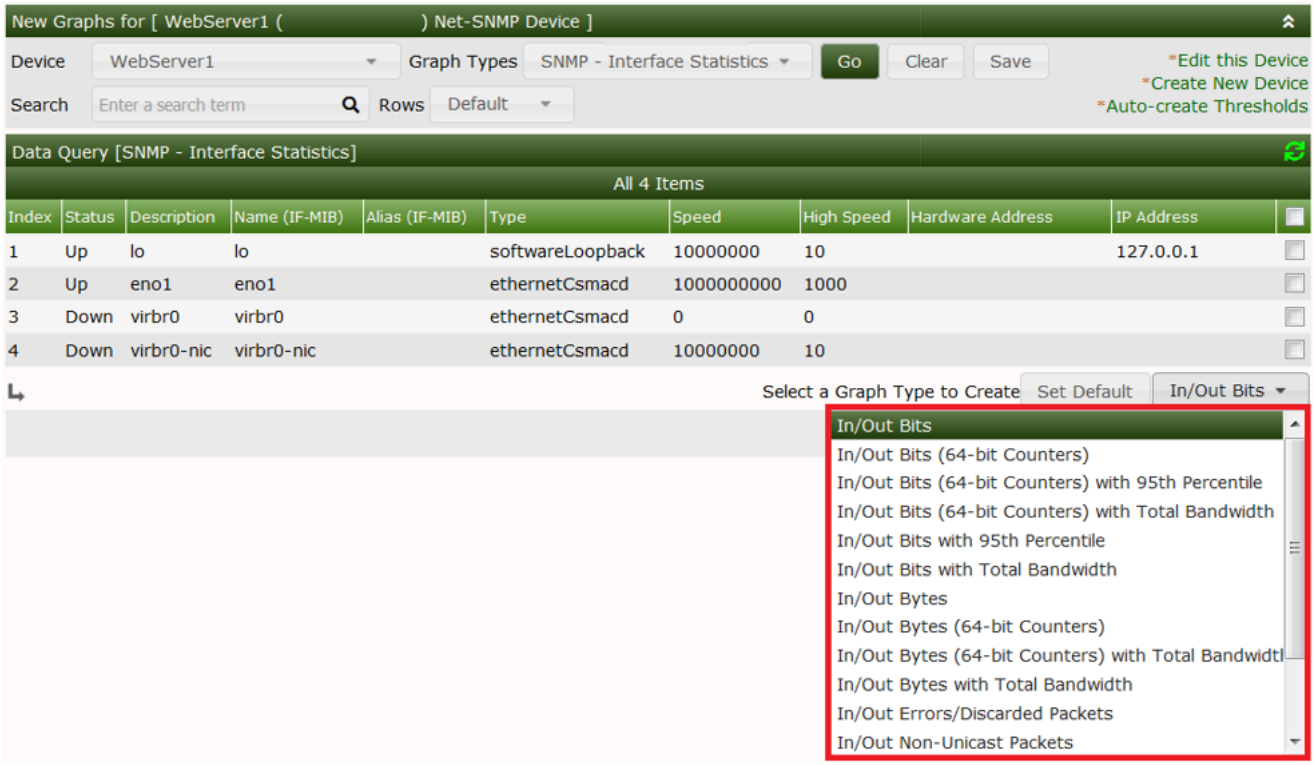
Now go to your **Device** to add the **Associated Data Query**:



Click **Add** and then **Create Graphs for this Host** to see:



Now select the wanted *Interfaces* and press **Create** to generate the Traffic Graph. As long as there's only one **Associated Graph Template** for that **Data Query**, there will be no *Associated Graph Type* drop down. Once you create more than one **Associated Graph Template** for the **Data Query**, you will see a drop down just above the **Create** button showing the available **Graph Templates** for the **Data Query** in question, like the *SNMP - Interface Statistics* as show below.



Pressing the *Set Default* button will set the selected *Graph Type* to be the default for this **Data Query**.

Copyright (c) 2004-2026 The Cacti Group

SNMP Data Query Walkthrough

This walkthrough will show you how to implement a new SNMP Data Query. Assuming, you know the SNMP table, the next steps show how to proceed.

Building Raw XML File

The starting point will be snmptable for a well know table of the HOSTS MIB:

```
snmptable -c <community> -v 1 <host> HOST-RESOURCES-MIB::hrStorageTable
SNMP table: HOST-RESOURCES-MIB::hrStorageTable

 hrStorageIndex               hrStorageType hrStorageDescr hrStorageAllocationUnits hrStorageSize hrStorageUsed
hrStorageAllocationFailures
          1     HOST-RESOURCES-TYPES::hrStorageOther Memory Buffers           1024 Bytes       1035356
59532               ?
          2       HOST-RESOURCES-TYPES::hrStorageRam    Real Memory           1024 Bytes       1035356
767448              ?
          3 HOST-RESOURCES-TYPES::hrStorageVirtualMemory     Swap Space           1024 Bytes       1048568
0               ?
          4     HOST-RESOURCES-TYPES::hrStorageFixedDisk          /           4096 Bytes       2209331
826154              ?
          5     HOST-RESOURCES-TYPES::hrStorageFixedDisk       /sys           4096 Bytes         0
0               ?
          6     HOST-RESOURCES-TYPES::hrStorageFixedDisk  /proc/bus/usb           4096 Bytes         0
0               ?
          7     HOST-RESOURCES-TYPES::hrStorageFixedDisk      /boot           1024 Bytes    102454
9029               ?
          8     HOST-RESOURCES-TYPES::hrStorageFixedDisk      /home           4096 Bytes    507988
446407              ?
          9     HOST-RESOURCES-TYPES::hrStorageFixedDisk     /usr/local           4096 Bytes    507988
17133               ?
         10     HOST-RESOURCES-TYPES::hrStorageFixedDisk       /var           4096 Bytes    507988
129429              ?
         11     HOST-RESOURCES-TYPES::hrStorageFixedDisk /var/lib/nfs/rpc_pipefs           4096 Bytes
0        0               ?**
```

This given, the first step will be the definition of an XML file based on those OIDs. So change to your `<path_cacti>/resources/snmp_queries` directory and create a file named **hrStorageTable.xml**. You may of course choose your own name, but for me it seems appropriate to take the name of the SNMP Table itself. Before doing so, it is necessary to identify the Index of that table. Without looking at the MIB file, simply perform

```
snmpwalk -c <community> -v 1 -On <host> HOST-RESOURCES-MIB::hrStorageTable|more
.1.3.6.1.2.1.25.2.3.1.1.1 = INTEGER: 1
.1.3.6.1.2.1.25.2.3.1.1.2 = INTEGER: 2
.1.3.6.1.2.1.25.2.3.1.1.3 = INTEGER: 3
.1.3.6.1.2.1.25.2.3.1.1.4 = INTEGER: 4
.1.3.6.1.2.1.25.2.3.1.1.5 = INTEGER: 5
.1.3.6.1.2.1.25.2.3.1.1.6 = INTEGER: 6
.1.3.6.1.2.1.25.2.3.1.1.7 = INTEGER: 7
.1.3.6.1.2.1.25.2.3.1.1.8 = INTEGER: 8
.1.3.6.1.2.1.25.2.3.1.1.9 = INTEGER: 9
.1.3.6.1.2.1.25.2.3.1.1.10 = INTEGER: 10
.1.3.6.1.2.1.25.2.3.1.1.11 = INTEGER: 11
.1.3.6.1.2.1.25.2.3.1.2.1 = OID: .1.3.6.1.2.1.25.2.1.1
.1.3.6.1.2.1.25.2.3.1.2.2 = OID: .1.3.6.1.2.1.25.2.1.2
```

The first index is `.1.3.6.1.2.1.25.2.3.1.1.1`, but the Index Base is `.1.3.6.1.2.1.25.2.3.1.1.` This OID is needed for the XML file:

```
<interface>
    <name>Get hrStoragedTable Information</name>
    <description>Get SNMP based Partition Information out of hrStorageTable</description>
    <index_order>hrStorageDescr:hrStorageIndex</index_order>
    <index_order_type>numeric</index_order_type>
    <oid_index>.1.3.6.1.2.1.25.2.3.1.1</oid_index>
```

```
    <fields>
        <hrStorageIndex>
            <name>Index</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.1</oid>
        </hrStorageIndex>
    </fields>
</interface>
```

Lets talk about the header elements

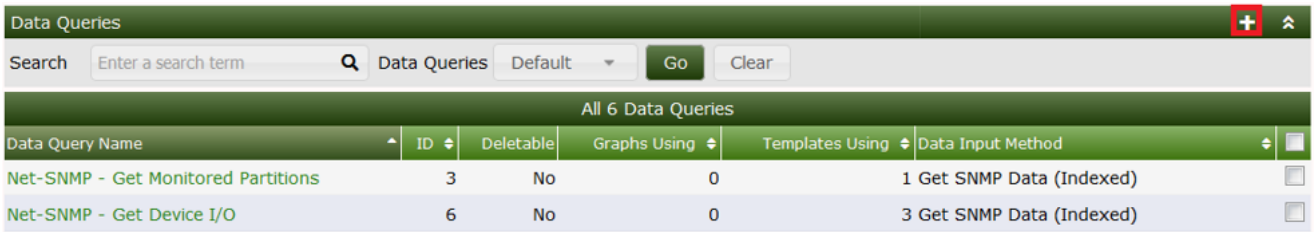
Field	Description
Name	Short Name; choose your own one if you want
Description	Long Name
index_order	tells us, which field shall be selected to index on; evaluated from left to right
index_order_type	numeric instead of alphabetic sorting
oid_index	the index of the table

There are more header elements, but for sake of simplification, we'll stick to that for now.

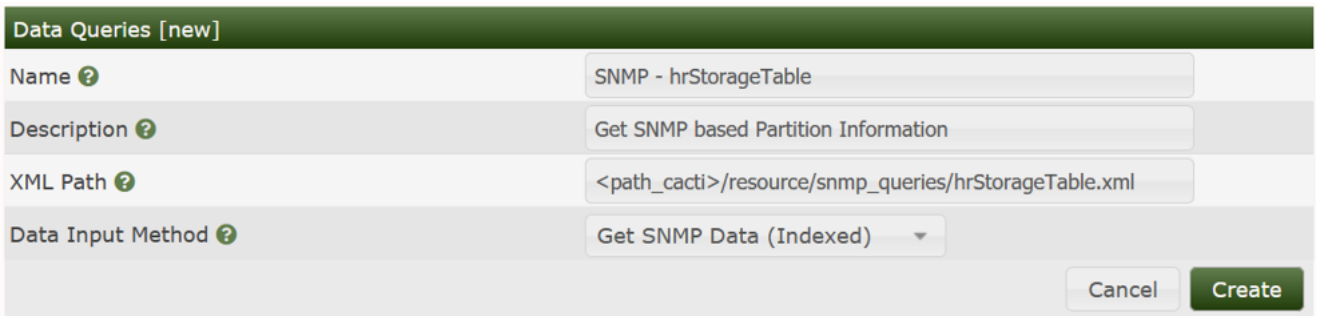
Lets turn to the fields. They correspond to the columns of the snmptable. For debugging purpose it is recommended to start with the Index field first. This will keep the XML as tiny as possible. The `<fields>` section contains one or more fields, each beginning with `<field_description>` and ending with `</field_description>`. It is recommended but not necessary to take the textual representation of the OID or an abbreviation of that.

Field	Description
Name	Short Name
Method	Walk or get (representing snmpwalk or snmpget to fetch the values)
Source	Value = take the value of that OID as the requested value. Sounds ugly, but there are more options that we won't need for the purpose of this example
Direction	Input (for values that may be printed as COMMENTs or the like). Output (for values that shall be graphed, e.g. COUNTERs or GAUGEs).
OID	the real OID as numeric representation

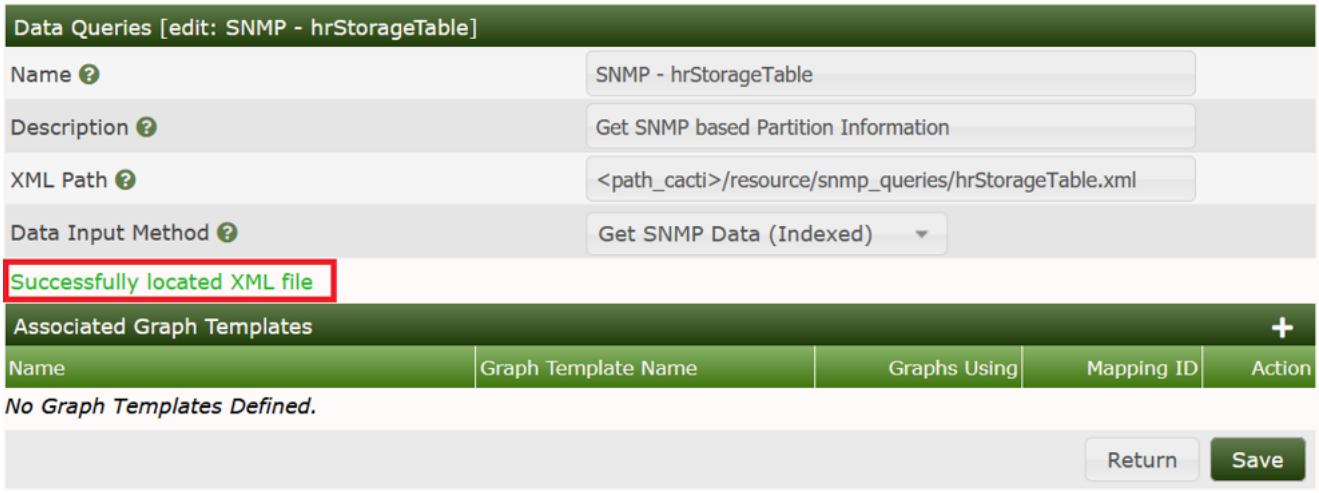
Now save this file and lets turn to cacti to implement this one. First, go to **Data Queries** to see



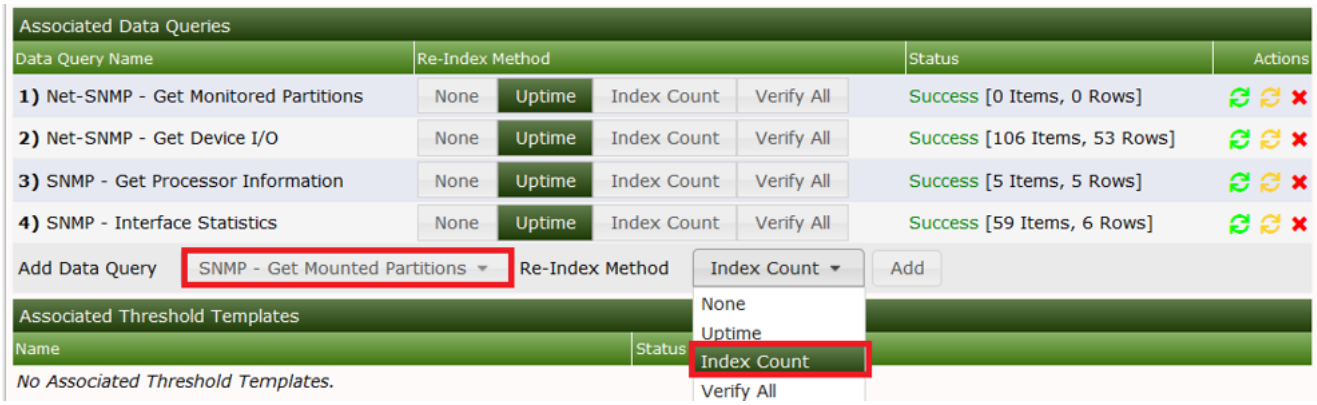
and **Add** a new one:



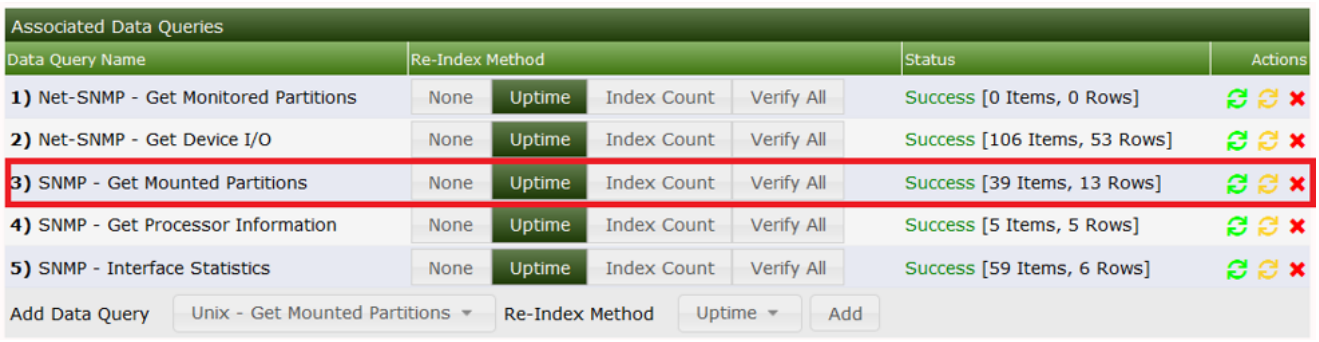
Fill in Short and Long Names at your wish. Enter the file name of the XML file and don't forget to choose **Get SNMP Data (indexed)**. **Create** to see



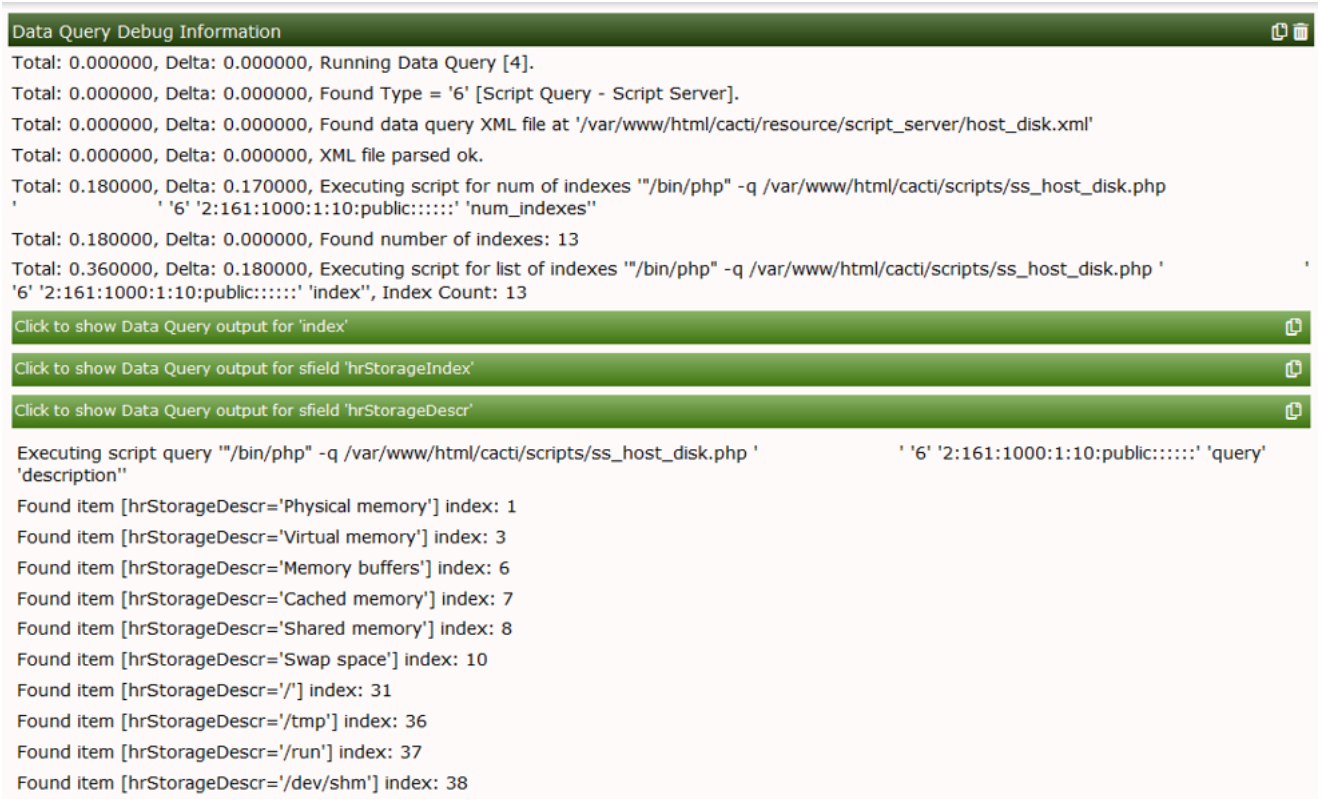
It has now **Successfully located XML file**. But this does not mean that there are no errors. So lets go on with that. Turn to the **Device** you want to query and add the new **Data Query** as shown:



Index Count Changed was chosen on purpose to tell cacti to re-index not only on reboot but each time the Index Count (e.g. number of partitions) changed. When done, see the results as



You'll notice, that on my laptop there are 11 indices = 11 partitions. So the XML worked up to now! To make this clear, select **Verbose Query** to see:



Insert all descriptive table columns

Now lets put all descriptive table columns into the SNMP Query XML file. This refers to

- hrStorageType
- hrStorageDescr
- hrStorageAllocationUnits

I like to take the XML field names from the snmptable output, but this is not a must.

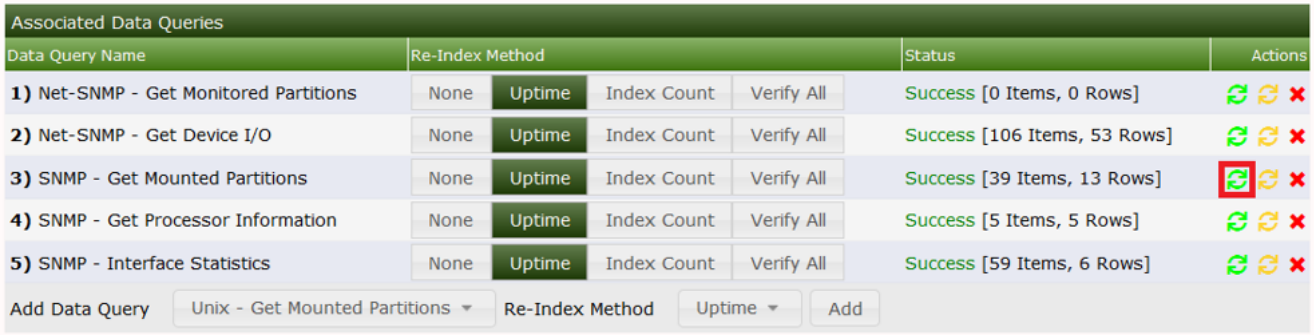
```
<interface>
    <name>Get hrStoragedTable Information</name>
    <description>Get SNMP based Partition Information out of hrStorageTable</description>
    <index_order>hrStorageDescr:hrStorageIndex</index_order>
    <index_order_type>numeric</index_order_type>
    <oid_index>.1.3.6.1.2.1.25.2.3.1.1</oid_index>

    <fields>
        <hrStorageIndex>
            <name>Index</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.1</oid>
        </hrStorageIndex>
        <hrStorageType>
            <name>Type</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.2</oid>

        </hrStorageType>
        <hrStorageDescr>
```

```
            <name>Description</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.3</oid>
        </hrStorageDescr>
        <hrStorageAllocationUnits>
            <name>Allocation Units (Bytes)</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.4</oid>
        </hrStorageAllocationUnits>
    </fields>
</interface>
```

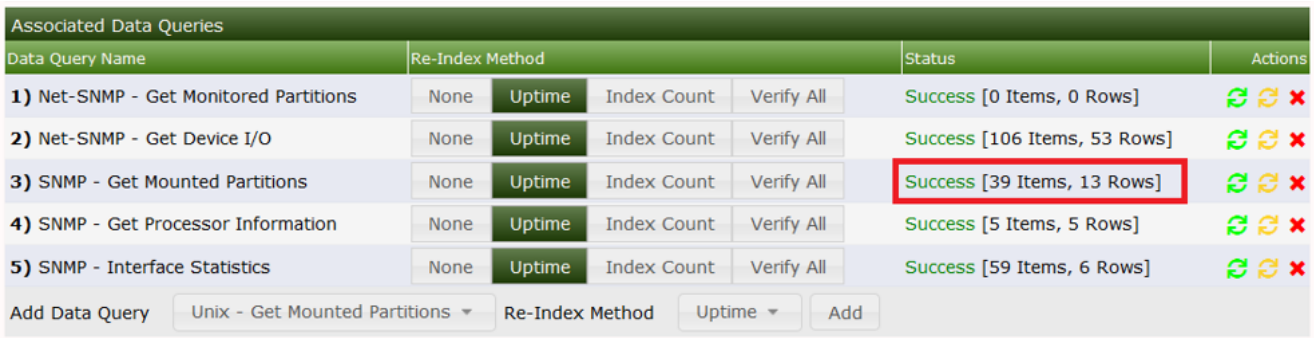
The `<name></name>` information will later show up as a column heading. Don't forget to provide the correct base OIDs. Remember, that the **Index** will always be appended to those OIDs, e.g. the first Description will be fetched from OID = `.1.3.6.1.2.1.25.2.3.1.3.1` (that is base OID = `.1.3.6.1.2.1.25.2.3.1.3` together with the appended index `.1` will form the complete OID `.1.3.6.1.2.1.25.2.3.1.3.1`.

Please notice, that all fields that will yield descriptive columns only take `<direction>input</direction>`

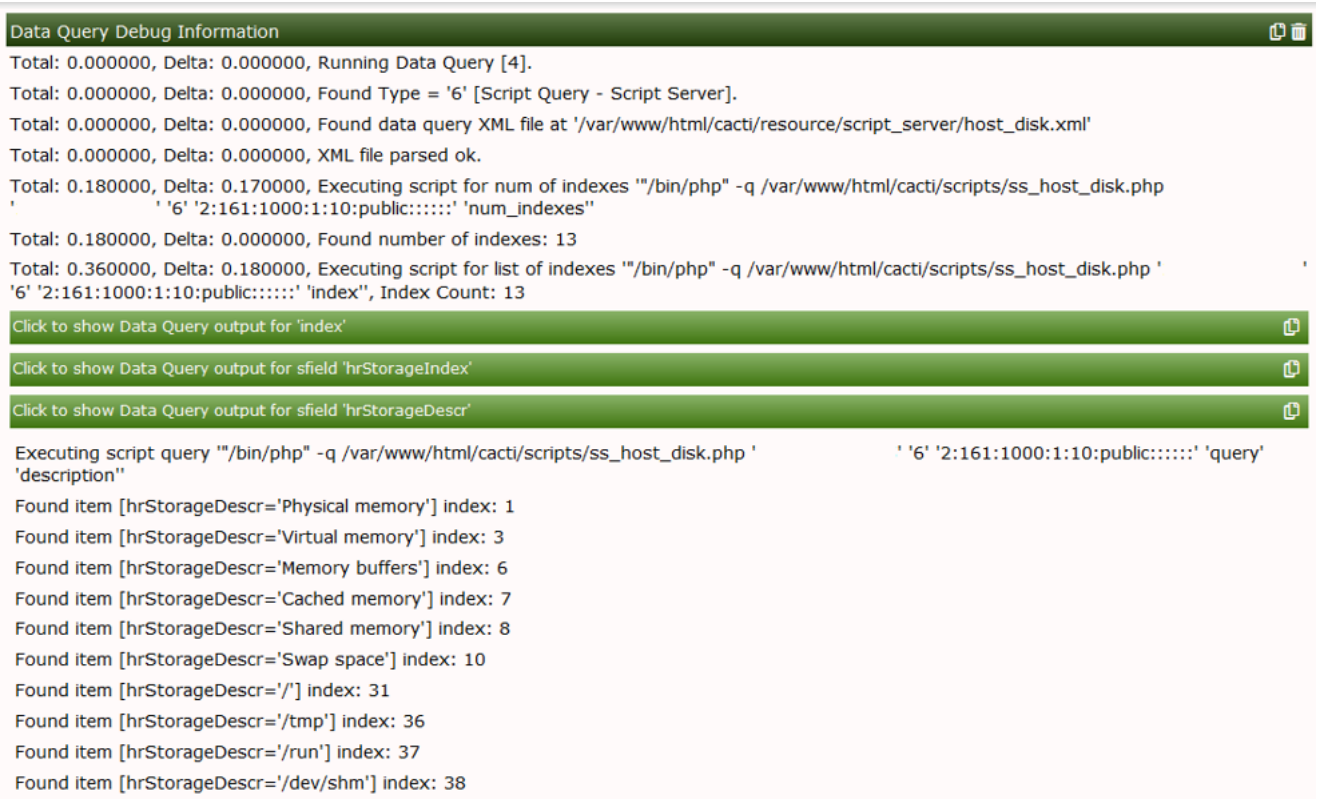
If you have completed your work, turn to the cacti web interface and select your host from the **Devices** list to see:



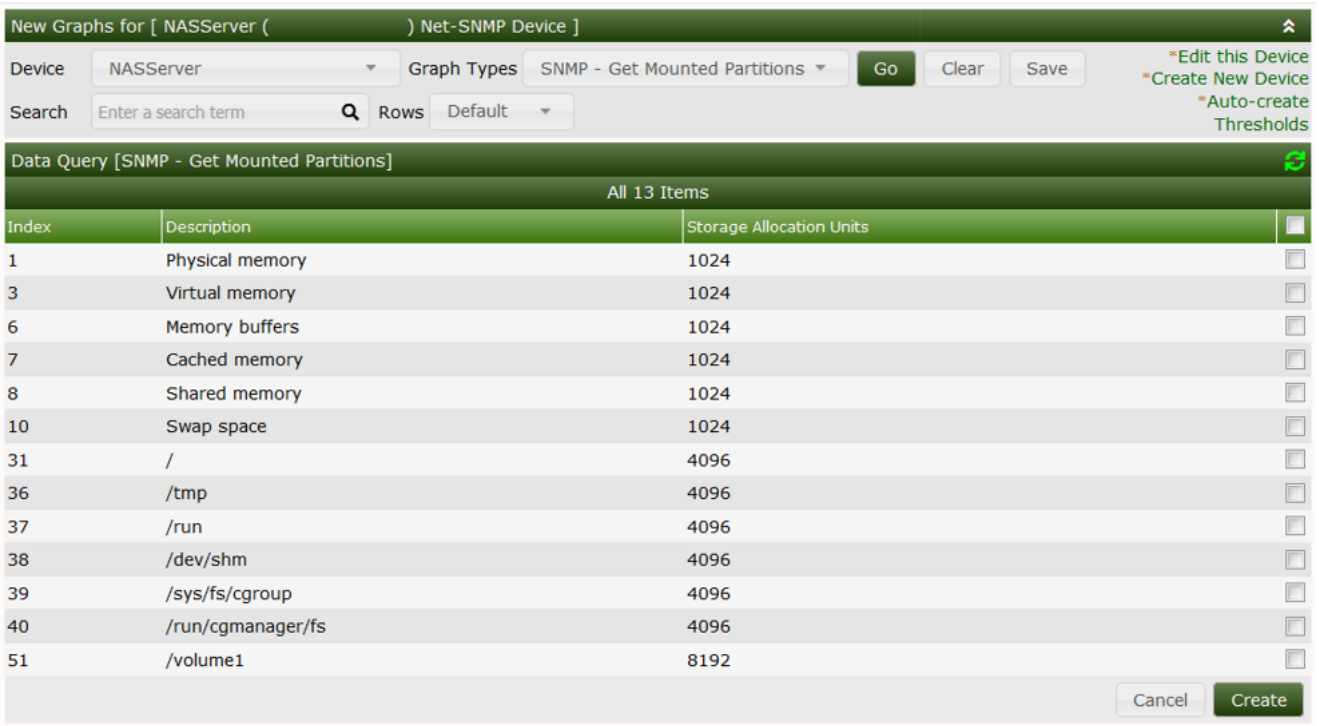
Select the little green circle next to our SNMP XML to update your last changes. Then you'll see something like:



When using **Verbose Query**, you'll now find



And clicking **Create Graphs for this host** will result in



You're not supposed to really create graphs at this moment, cause the XML is not yet complete. And you'll notice, that the second column does not present very useful information. So it may be omitted in later steps.

Getting the Output Values

Now lets modify the XML again. As said earlier, the second column is not very meaningful, so lets drop it. To get the output values, I appended the last two XML field descriptions, see:

```
<interface>
    <name>Get hrStoragedTable Information</name>
    <description>Get SNMP based Partition Information out of hrStorageTable</description>
    <index_order>hrStorageDescr:hrStorageIndex</index_order>
    <index_order_type>numeric</index_order_type>
    <oid_index>.1.3.6.1.2.1.25.2.3.1.1</oid_index>

    <fields>
        <hrStorageIndex>
            <name>Index</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.1</oid>
        </hrStorageIndex>
        <hrStorageDescr>
            <name>Description</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.3</oid>
        </hrStorageDescr>
        <hrStorageAllocationUnits>
            <name>Allocation Units (Bytes)</name>
            <method>walk</method>
            <source>value</source>
            <direction>input</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.4</oid>
        </hrStorageAllocationUnits>
        <hrStorageSize>
            <name>Total Size (Units)</name>
            <method>walk</method>
            <source>value</source>
            <direction>output</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.5</oid>
        </hrStorageSize>
        <hrStorageUsed>
            <name>Used Space (Units)</name>
            <method>walk</method>
            <source>value</source>
            <direction>output</direction>
            <oid>.1.3.6.1.2.1.25.2.3.1.6</oid>
        </hrStorageUsed>
    </fields>
</interface>
```

This works very much the same way as above.

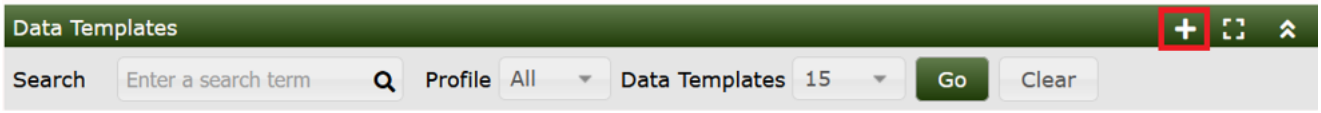
- Provide the fields hrStorageSize and hrStorageUsed
- Provide a useful name
- Don't forget to specify `<direction>output</direction>`
- Give the corresponding base OIDs

Now we may proceed as said above: Pressing the green circle runs that XML definitions against the host and updates the rows/columns. You will notice the “missing” second column only when Create Graphs for this Host is selected.

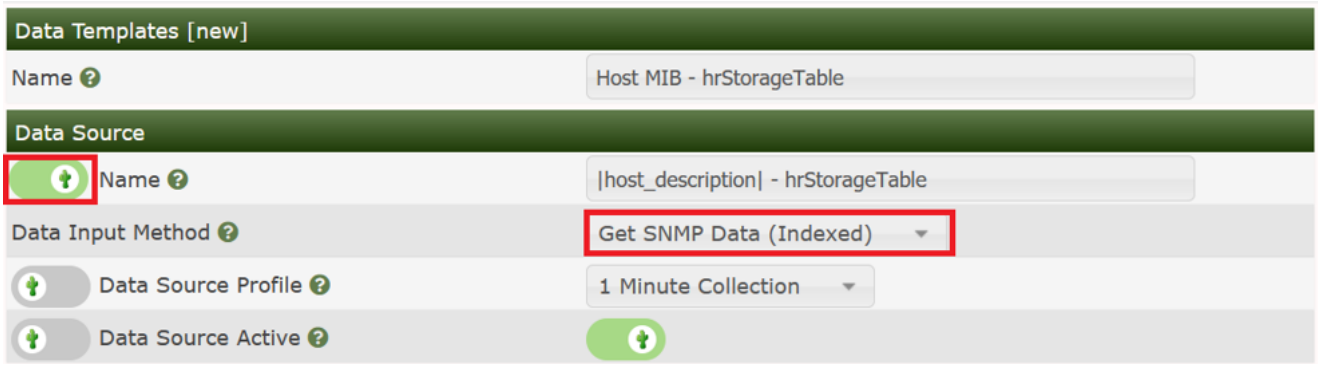
Note: Don't forget to set `<direction>output</direction>` for all variables/fields, that should be stored in RRDfiles and be graphed!. This is the mistake that occurs most often.

Defining the Data Template

The Data Template will define, how the data is retrieved by the XML Query is saved. For more information about the principles of operation, please see Common Tasks. Please go to **Data Templates** and **Add**:



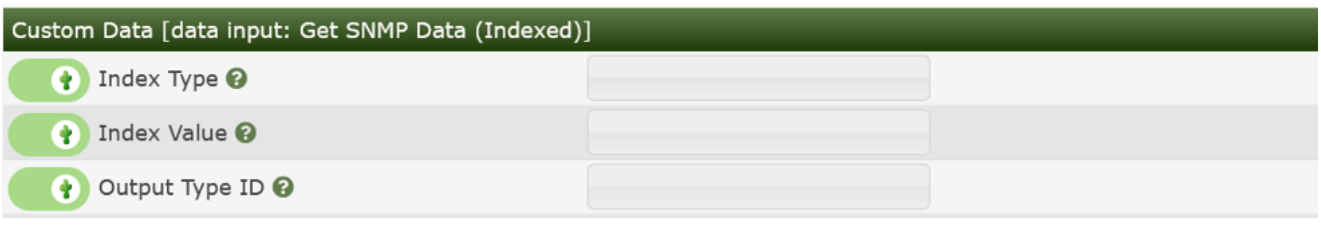
Define the **Name** of the Data Template. When defining the Name of the **Data Source**, do not forget to check the **Use Per-Data Source Value (Ignore this Value)** checkbox. This will come in useful later. Data Input Method will read **Get SNMP Data (Indexed)**. Select the **Data Source Profile** as usual:



Now to the **Data Source Items**. I like giving them the names of the MIB OIDs, see:

and **New** or the plus sign on the **Data Source Item** panel. Now enter the second **Data Source Item**:

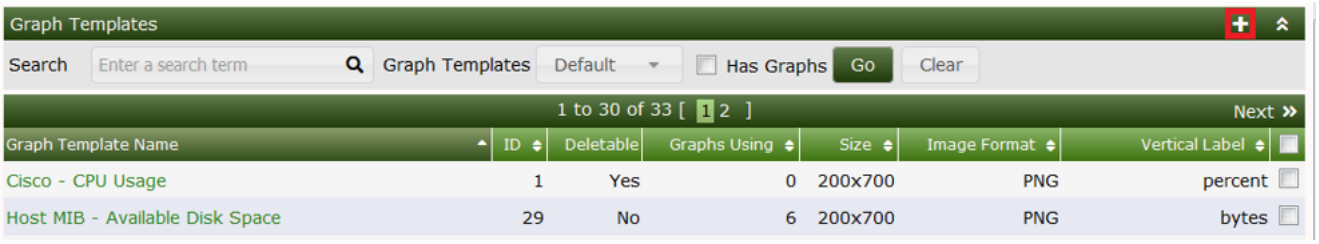
Please pay attention to setting the **Maximum Value** to 'U' (no limit). Else, all values exceeding the pre-defined as **NaN**. Now scroll down to the bottom of the page and check **Index Type, Index Value and Output Type Id**



Save and the **Data Template** is finished.

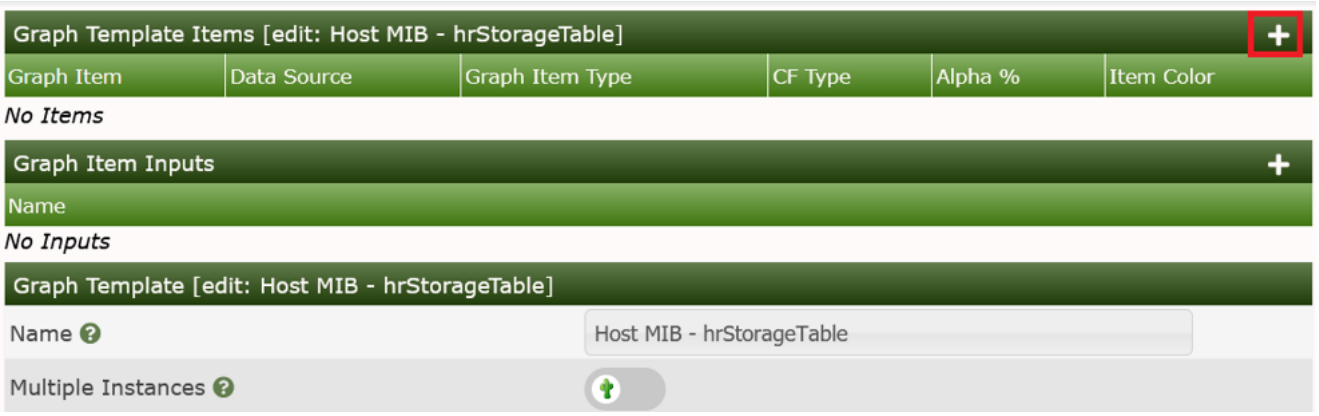
Defining the Graph Template

The Graph Template will define, how the data is presented. For more information about the principles of operation, please see Principles of Operation. Please go to Graph Templates and Add:

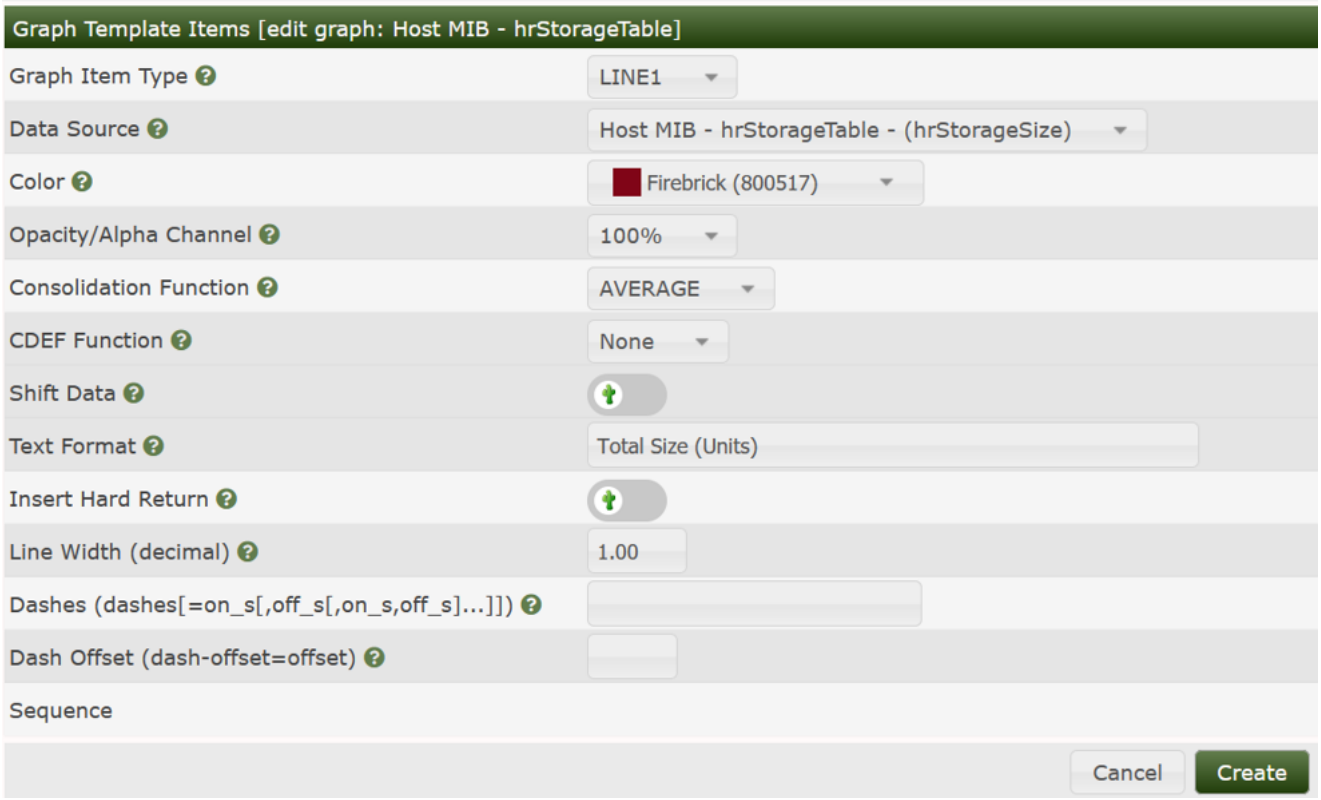


Fill in the header names and don't forget to check the **Use Per-Graph Value (Ignore this Value)** for the **Graph Template Title**:

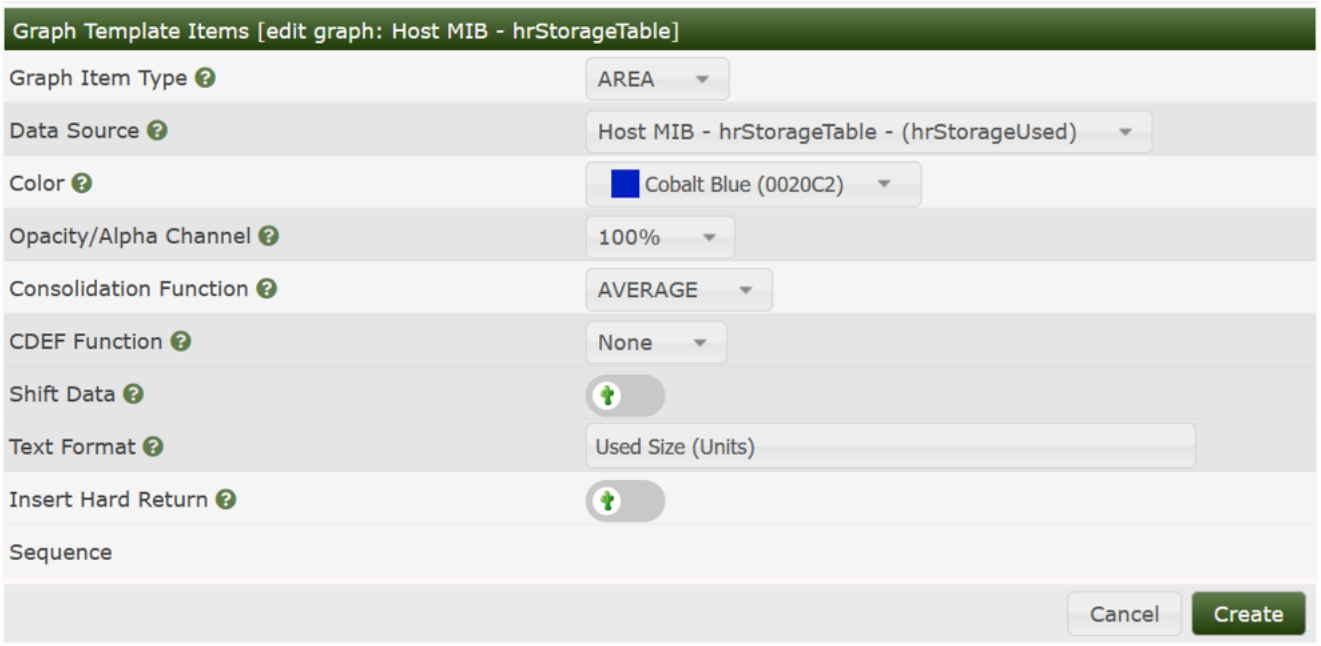
and **Create**.



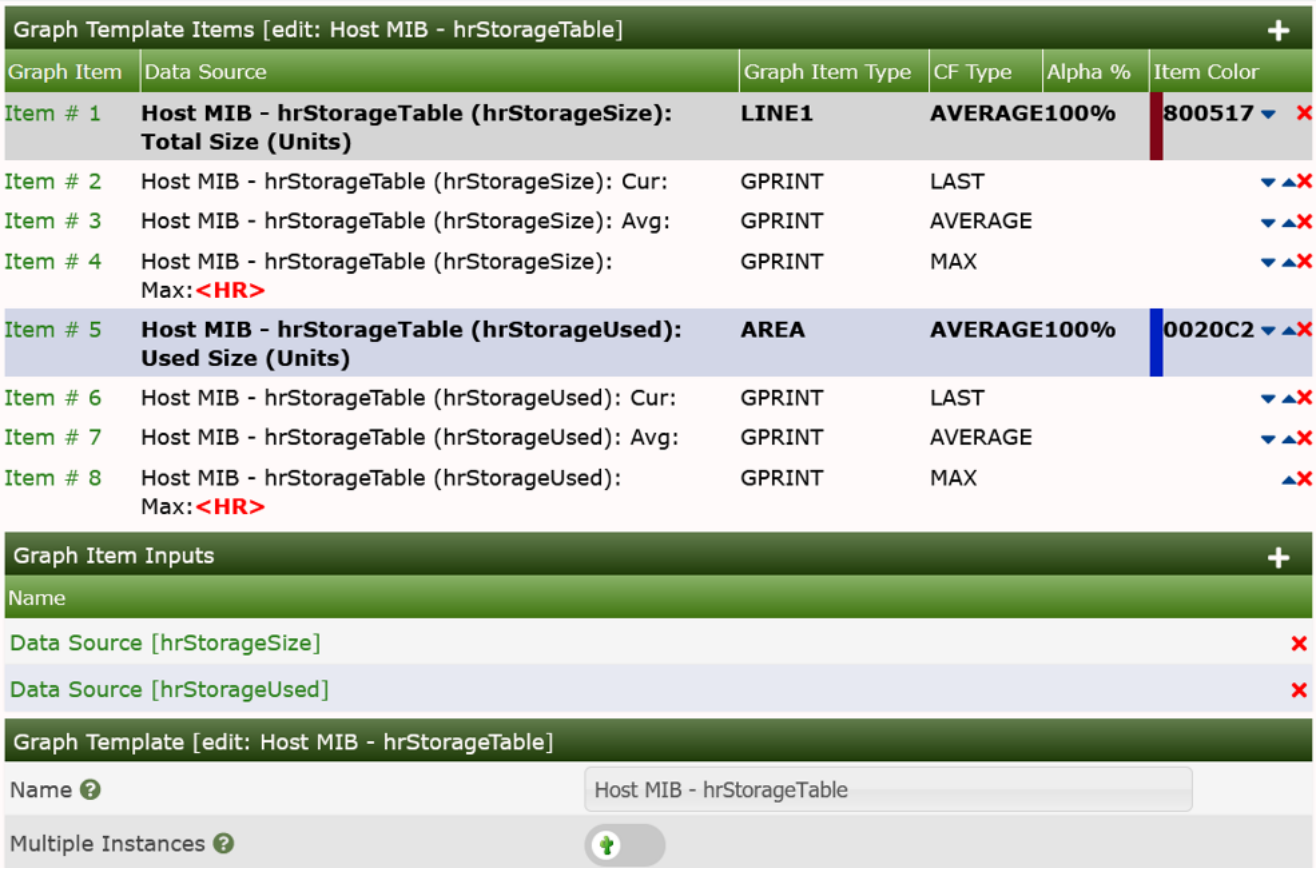
Now **Add** the first Graph Item as usual:



Add the Legend and the second Graph Item:

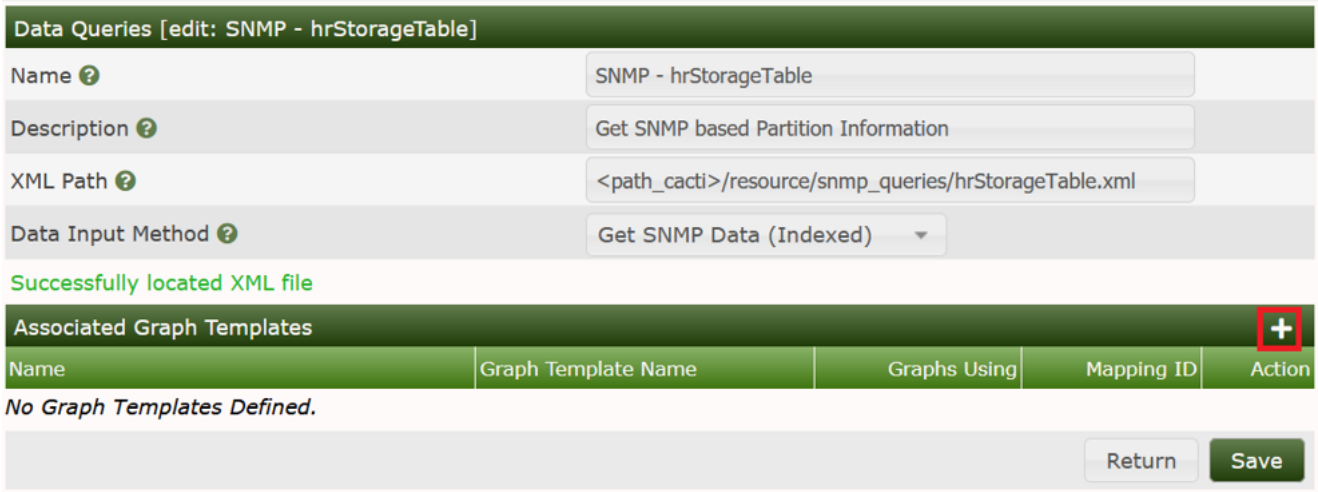


Again, add the Legend to end up with

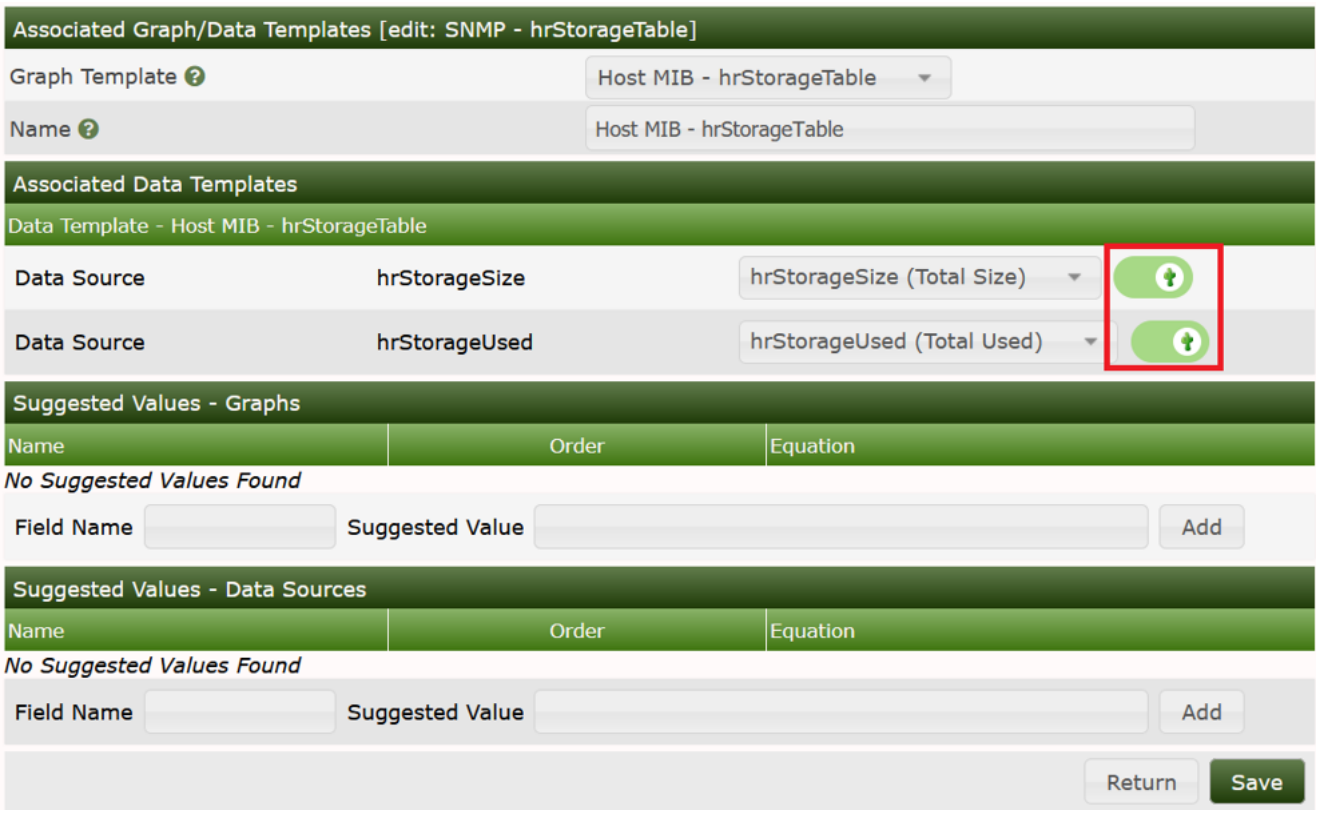


Revisiting The Data Query

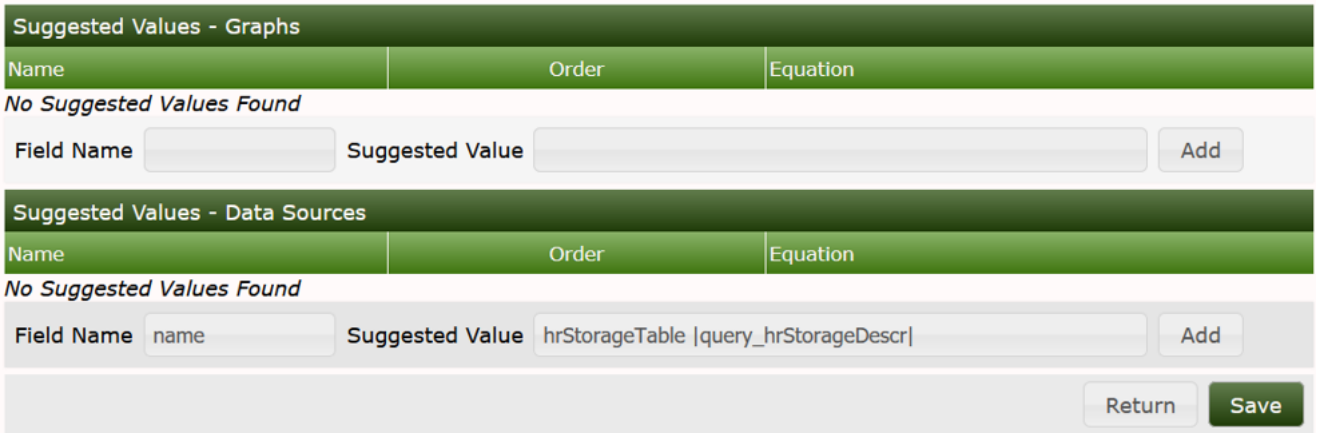
We'll now have to revisit the **Data Query**



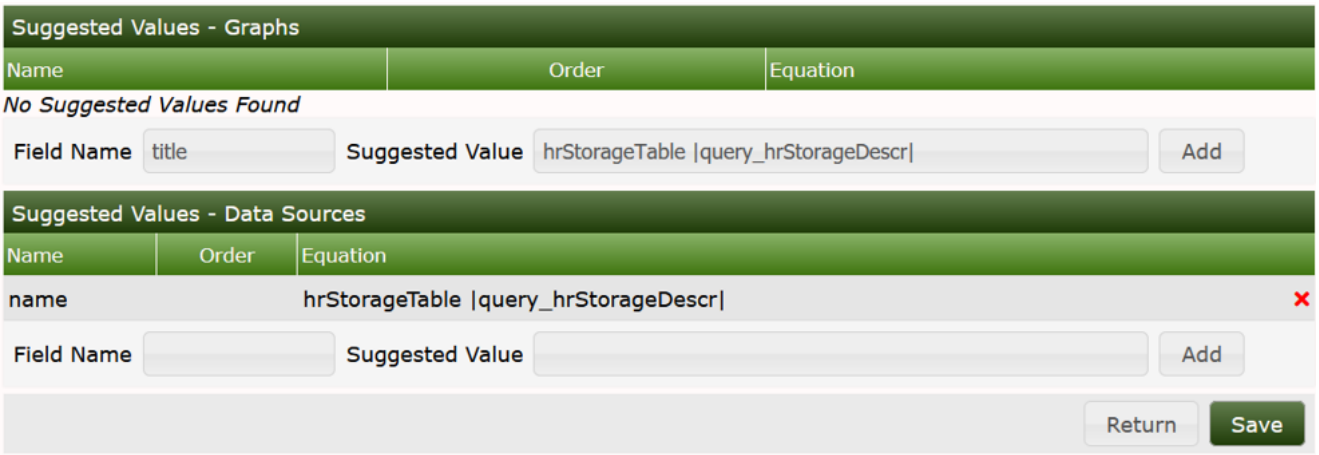
Create:



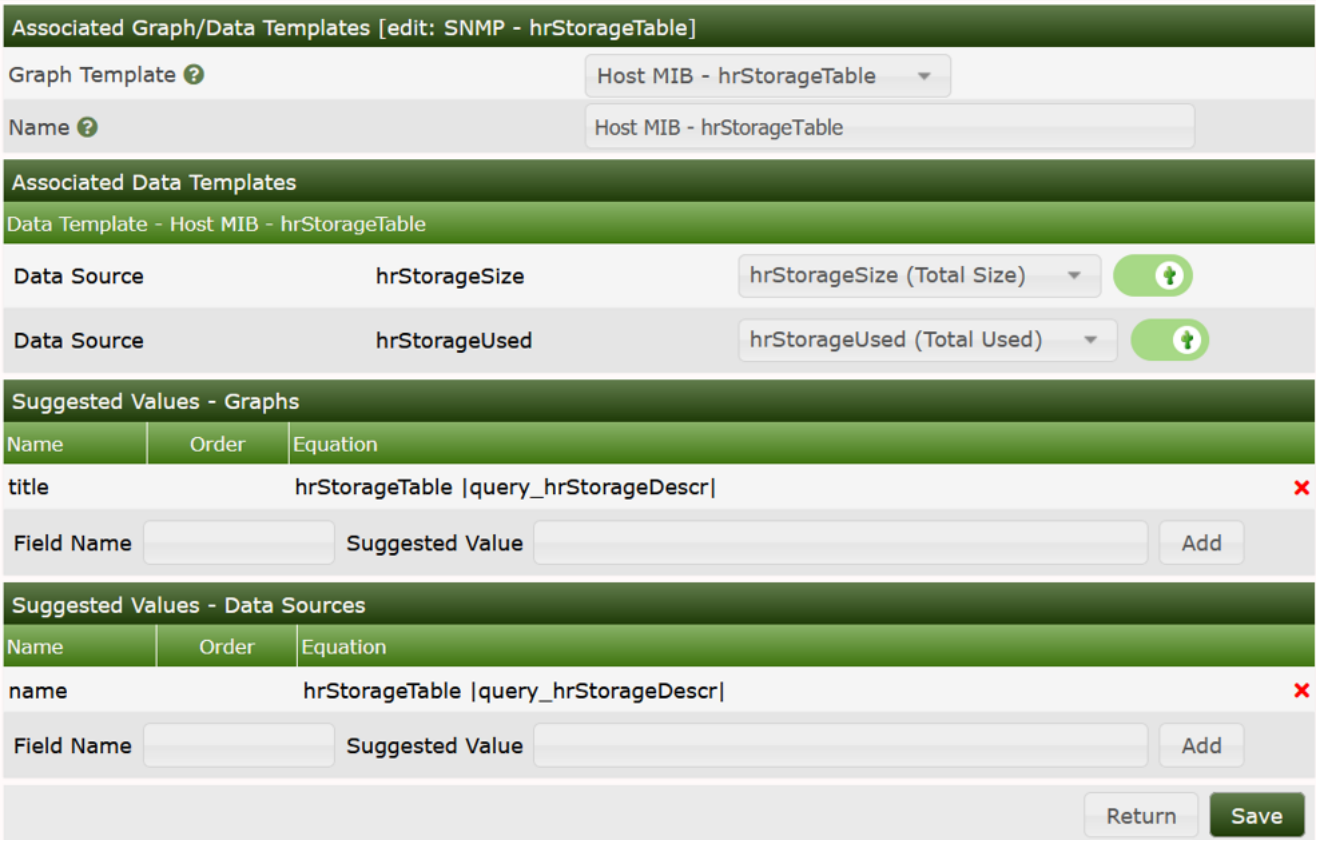
Select the correct **Data Sources** and check the boxes on the right. **Save**. Now fill in some useful **Suggested Values**, at first for the **Data Template**:



Now apply suggested values for the **Graph Template**:

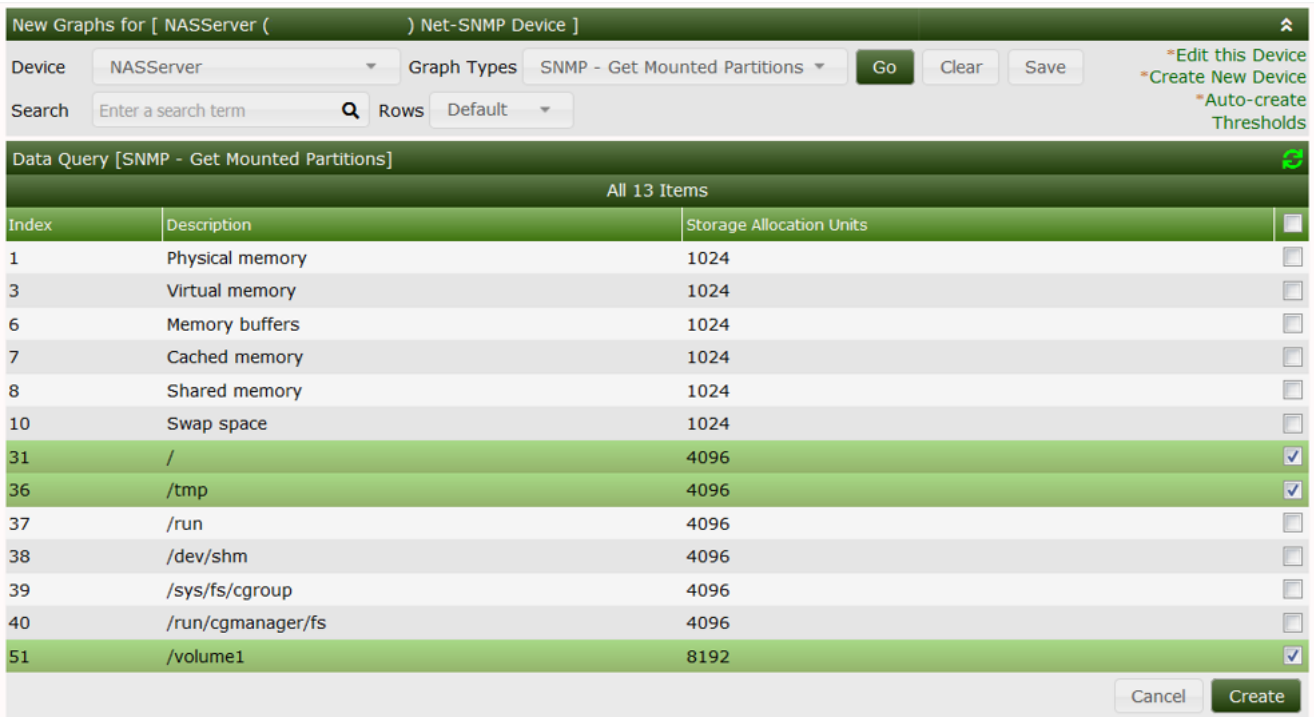


Now the **Data Query** is complete:

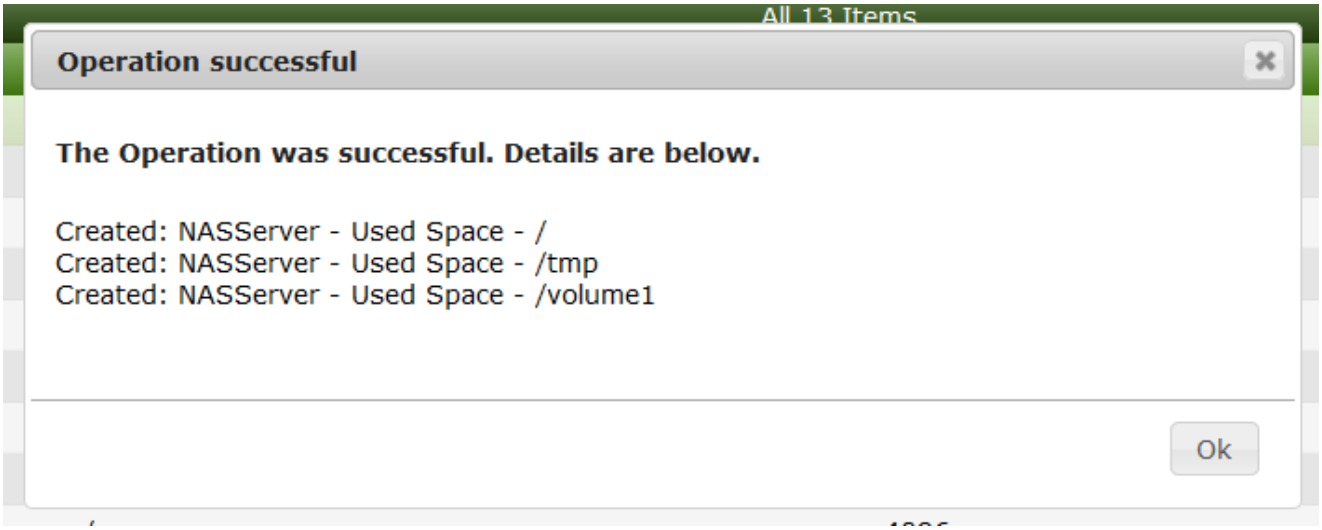


Create Graphs for this Host

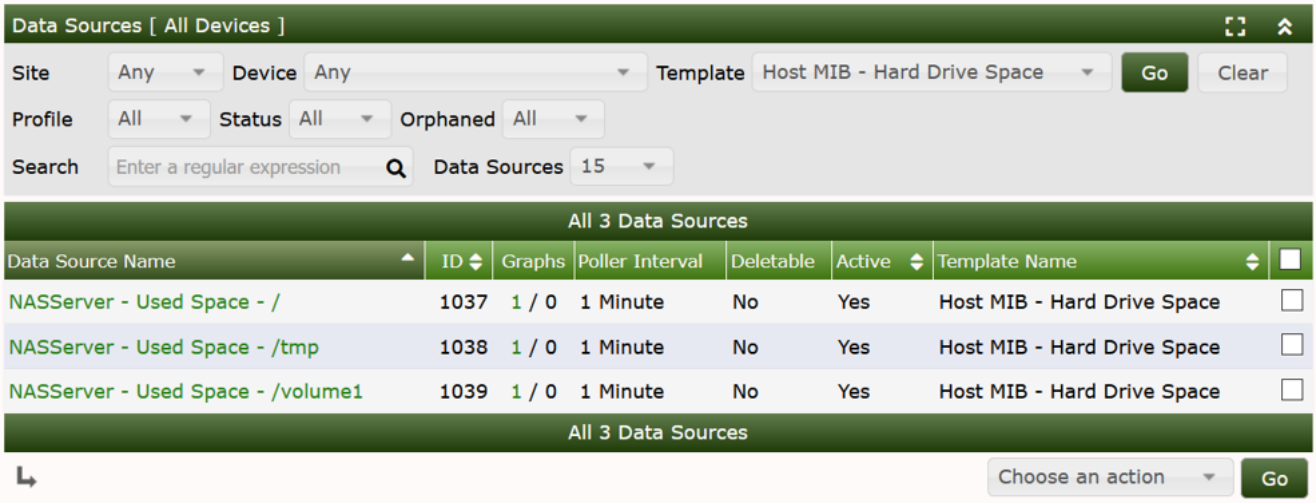
Now we're almost done. Everything's ready for use now. So go to your device and select **Create Graphs for this Host**. Select some of the partitions you're interested in:



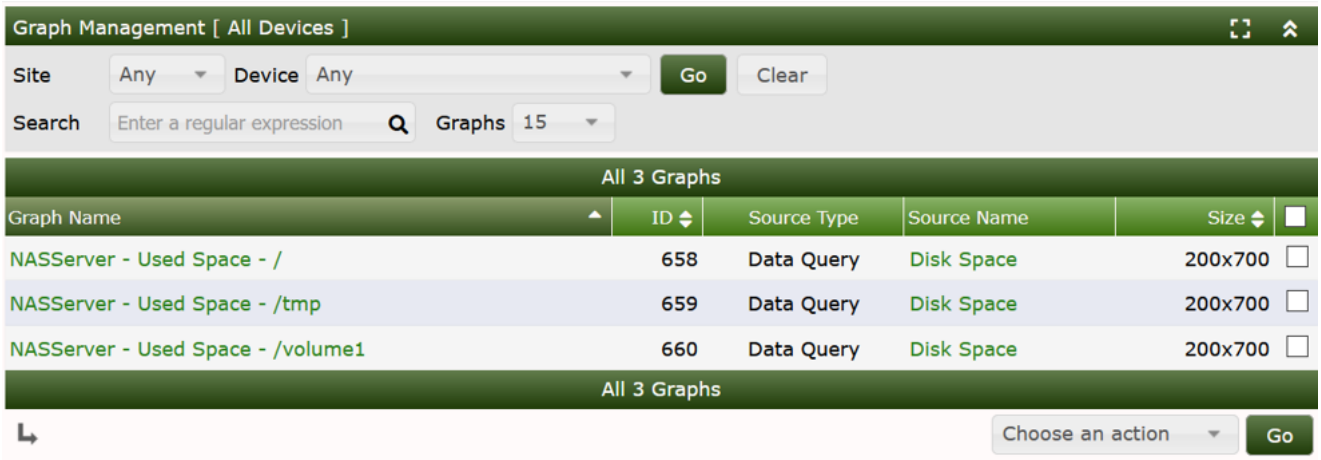
and **Create** to see:



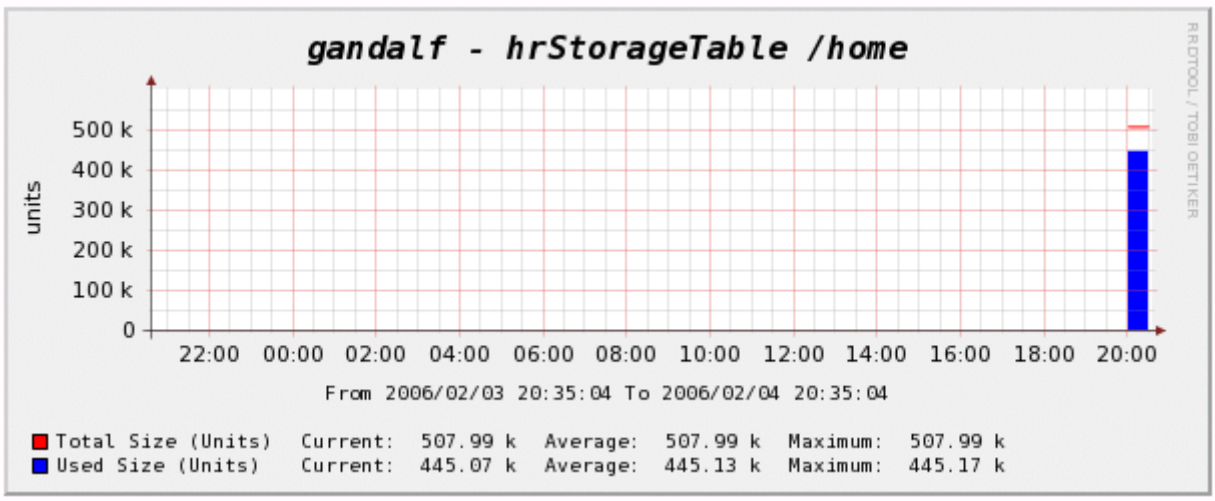
Lets visit the **Data Sources**:



As you can see, the **Suggested Values** of the Data Query defined the Name of the Data Template. So lets go to **Graph Management**:



to see the title defined by the **Suggested Values**. When turning to the **Graphs**, you may see something like



This might be the end of the show. While it should be enough to define some “easy” SNMP XML based Data Queries, there are some tricks and hints left to explain.

As you may have noticed, the quantities defined by this example are counted in Units, not Bytes. This is somewhat inconvenient but may be changed. Lets wait for the next Section...

VALUE/REGEXP in Action

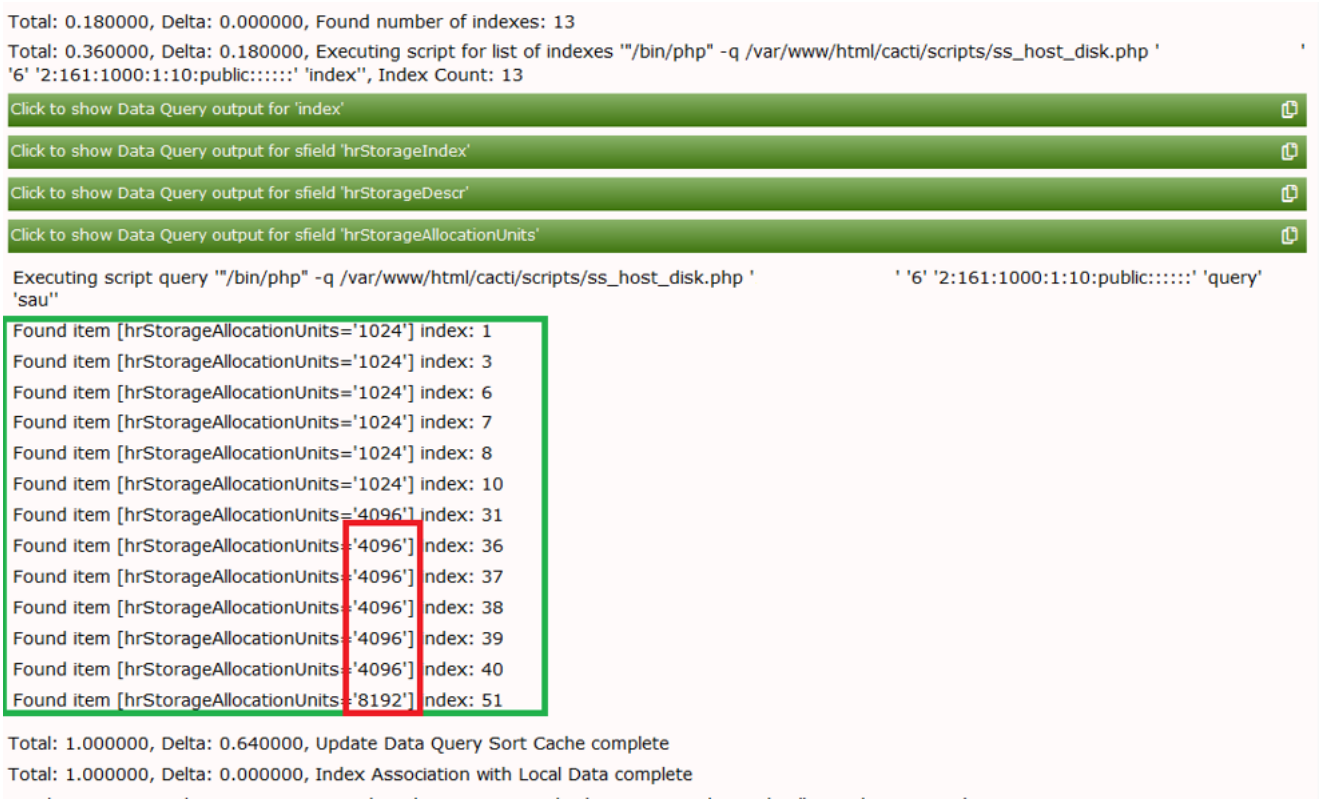
As said above, with the current XML size values are measured in **Units**. The current Unit Size is given by **hrStorageAllocationUnits**, but the reading of it is like **4096 Bytes**. To use this in any calculations, we must get rid of the string **Bytes**. This can be done by the **VALUE/REGEXP** Feature of Cacti's XML definitions. So please change

```
<hrStorageAllocationUnits>
   <name>Allocation Units (Bytes)</name>
   <method>walk</method>
   <source>value</source>
   <direction>input</direction>
   <oid>.1.3.6.1.2.1.25.2.3.1.4</oid>
</hrStorageAllocationUnits>
```

By

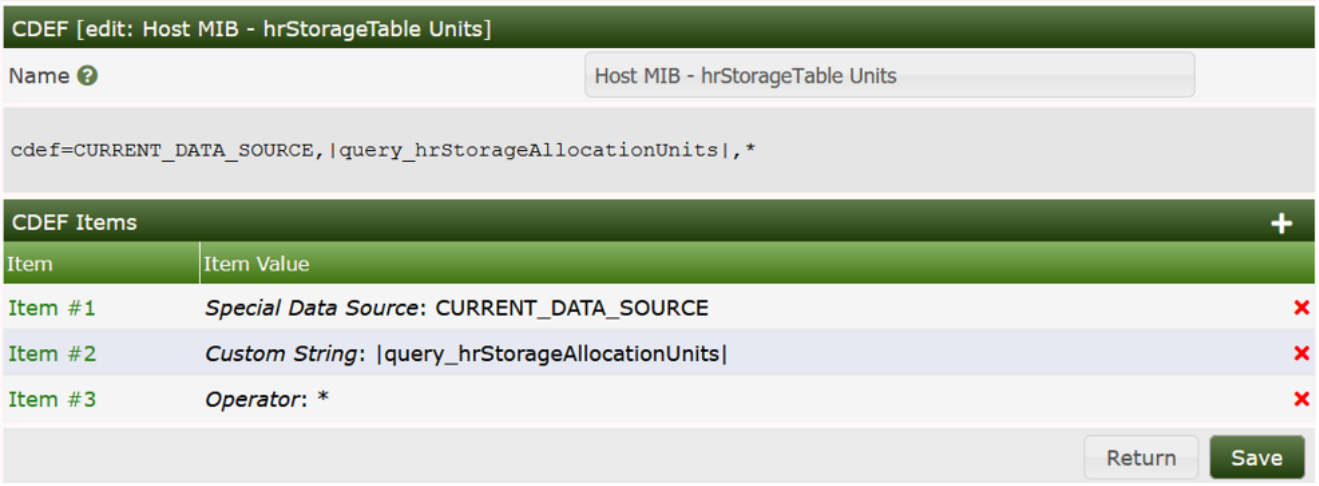
```
<hrStorageAllocationUnits>
   <name>Allocation Units (Bytes)</name>
   <method>walk</method>
   <source>VALUE/REGEXP:([0-9]*) Bytes</source>
   <direction>input</direction>
   <oid>.1.3.6.1.2.1.25.2.3.1.4</oid>
</hrStorageAllocationUnits>
```

To prove this, go to your device and again **Verbose Query** our Data Query to see:

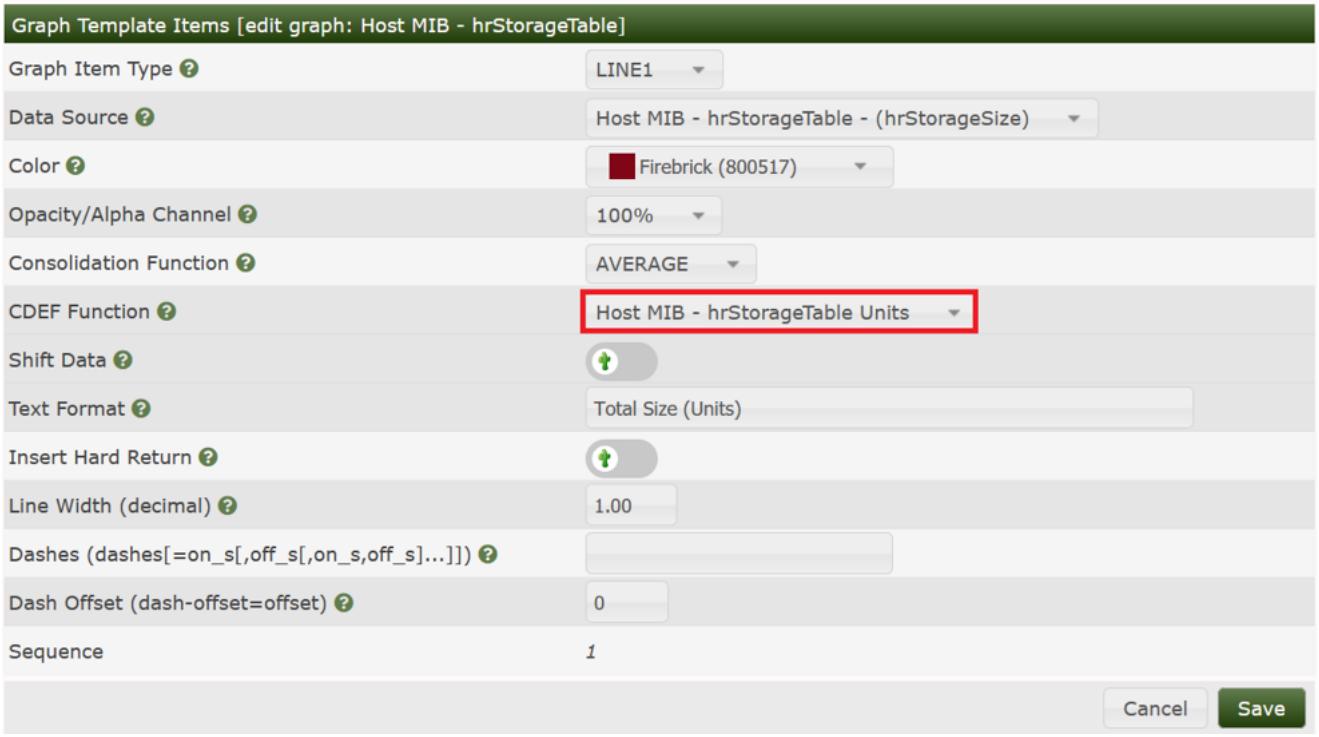


Now select **Create Graphs for this Host** and notice the change of the column **Allocation Units (Bytes)**. The string “Bytes” has gone:

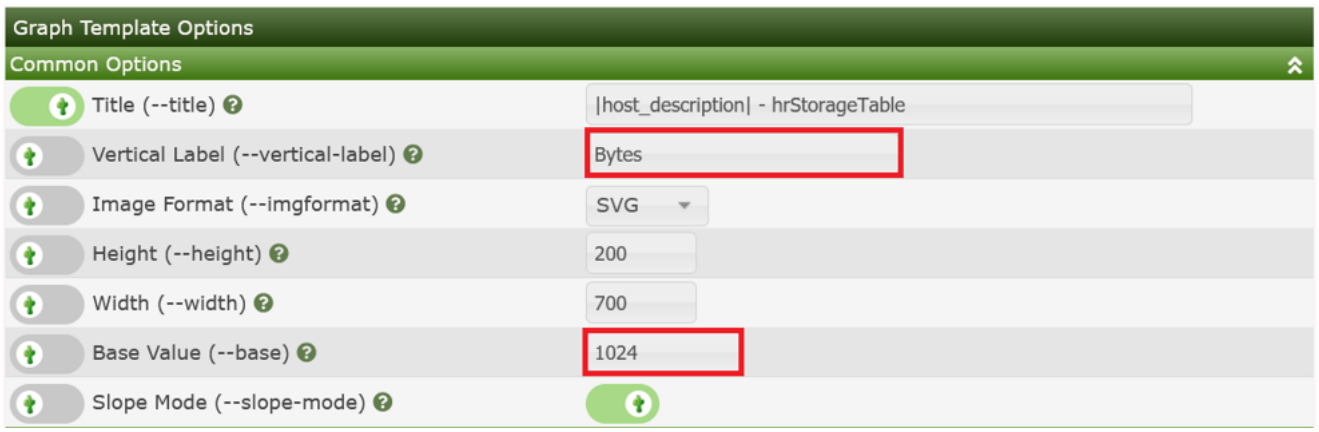
To use these values, we define a CDEF:



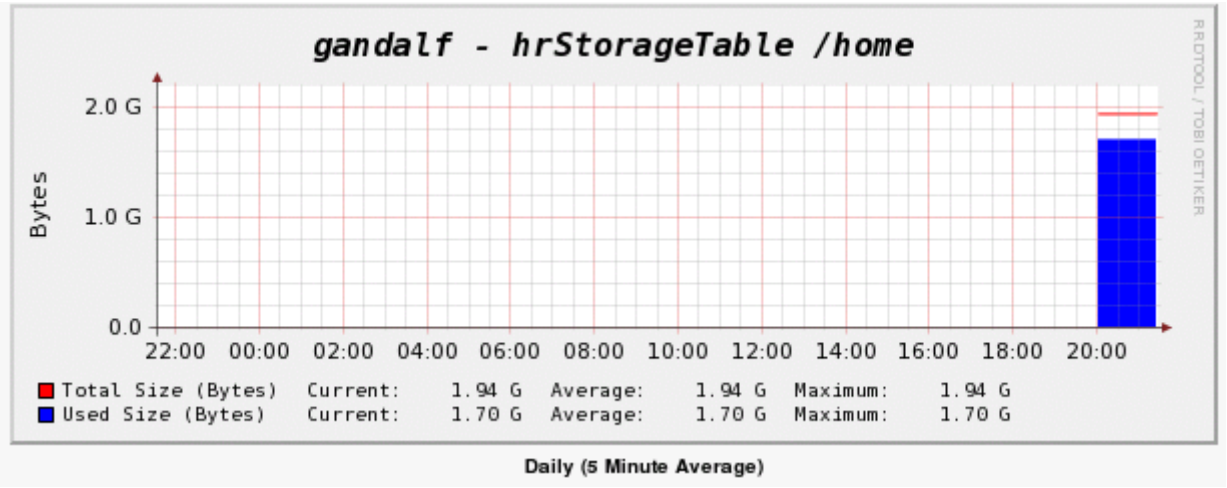
Notice, that with recent releases of cacti, it is possible to use `|query_*|` values within CDEFs. Finally, go to **Graph Templates** and use this CDEF with all **Graph Items**:



Change the **Base Value** to 1024 for Bytes → kBytes and the y-axis description to **Bytes**:



Now the Graph looks like



Please find the example resources below. Save `hrStorageTable.xml` into `./resource/snmp_queries` and import the Graph Template from `cacti_graph_template_host_mib_hrstoragetable.xml`.

Cacti Graph Template Host MIB hrStorageTable

hrStorageTable download and store into `resource/snmp_queries`.

Copyright (c) 2004-2026 The Cacti Group

Script Data Query Walkthrough

Goal of this HowTo will be to show the **principles of writing a Script Query**, including script, xml and all needed templates. Why should you create such a thing? Suppose, your target features some indexed readings, that are **not** available via SNMP but by some other method (e.g. wget/cgi, ssh, NRPE, …). Writing a Script Data Queries works very much the same way as SNMP Data Queries. But nevertheless, I'll take you through all of the steps now.

The example uses PHP. Why PHP? First, it's easier to copy stuff from already existing PHP scripts. Second, it would be possible to use cacti functions. It should be possible to imagine, how this works with other programming languages. Strictly speaking, I'm not that PHP expert. So be patient with me.

Please pay attention. This HowTo will not explain how to write a **Script Server Data Query** (yes, there is such a thing!). It would not introduce that many changes. But this will be left to some other HowTo.

Personally, my primary goal was to use an example, that all users should be able to copy to execute each and every step on its own. Unfortunately, there seems to be no example, that is common enough and interesting at the same time. So I'm sorry to announce, that this HowTo will show “Interface Traffic Data Gathering”. Yes, I know, this is not that new. And surely, it will not be as fast as pure SNMP. So, to my shame, I suppose that this will never make it into any production environment. But, again, this is not the primary goal.

Before starting the work, I feel encouraged to point out a drawback of this approach. Cacti will start a PHP instance, each time it has to fetch a value from the target device. This is not that fast, obviously. And it will not prosper from the performance boost when switching over from cmd.php to Spine. Of course, even Spine will need to start php! And that's exactly, where the thingy called **Script Server Data Query** drops in. But let's leave this for the next main chapter.

The code runs on Cacti 0.8.7 versions.

Basic Script

The starting point will be some very basic php script. Put it into `<path_cacti>/scripts/query_interface_traffic.php`. It will show interface indices only for the given target host. The script takes two parameters as input, the **hostname** of the target and the string **index**. You have to implement the **index method**, as OO programmers would say. In this case, there's an “if” clause to process index requests. Output is a list of indices, each one on a separate line.

```
<?php

# include some cacti files for ease of use
include(dirname(__FILE__) . '/../include/cli_check.php');
include(dirname(__FILE__) . '/../lib/snmp.php');

# define all OIDs we need for further processing
$oids = array(
    'index' => '.1.3.6.1.2.1.2.2.1.1',
);

$xml_delimiter  =  '!';

# all required input params
$hostname        = $_SERVER['argv'][1];        # hostname/IP@
$cmd             = $_SERVER['argv'][2];        # one of: index/query/get

# put your own community string here
$snmp_community  = 'public';           # community string
$snmp_version    = 1;               # snmp version
$snmp_port       = 161;             # snmp port
$snmp_timeout    = 500;             # snmp timeout
$snmp_retries    = 3;               # snmp retries
$max_oids        = 1;               # max oids for V2/V3 hosts
# required for SNMP V3
$snmp_auth_username       = '';
$snmp_auth_password       = '';
$snmp_auth_protocol       = '';
$snmp_priv_passphrase     = '';
$snmp_priv_protocol       = '';
```

```
$snmp_context            = '';

# ---------------------------------------------------------------------------
# main code starts here
# ---------------------------------------------------------------------------

# ---------------------------------------------------------------------------
# script MUST respond to index queries
#       the command for this is defined within the XML file as
#       <arg_index>index</arg_index>
#       you may replace the string 'index' both in the XML and here
# ---------------------------------------------------------------------------
#       php -q <script> <params> index
# will list all indices of the target values
# e.g. in case of interfaces
#      it has to respond with the list of interface indices
# ---------------------------------------------------------------------------
if ($cmd == 'index') {
    # retrieve all indices from target
    $return_arr = reindex(cacti_snmp_walk($hostname, $snmp_community,
    $oids['index'], $snmp_version, $snmp_auth_username,
    $snmp_auth_password, $snmp_auth_protocol, $snmp_priv_passphrase, $snmp_priv_protocol,
    $snmp_context, $snmp_port, $snmp_timeout, $snmp_retries, $max_oids, SNMP_POLLER));

    # and print each index as a separate line
    for ($i=0;($i<sizeof($return_arr));$i++) {
        print $return_arr[$i] . "\n";
    }
} else {
    print "Invalid use of script query, required parameters:\n\n";
    print "    <hostname> <cmd>\n";
}

function reindex($arr) {
    $return_arr = array();

    for ($i=0;($i<sizeof($arr));$i++) {
        $return_arr[$i] = $arr[$i]['value'];
    }

    return $return_arr;
}
```

It will be called like this

```
php -q query_interface_traffic.php <your target host> index
1
2
3
4
```

As you see, my `<target>` has 4 indices (interfaces).

Discussion: function_reindex

You may wonder why this function drops in. Well, lets have a look at `cacti_snmp_walk`. This function is part of cacti itself and eases the use of SNMP. That's why I call it here. But unfortunately, its output looks like:

```
Array
(
    [0] => Array
        (
            [oid] => 1.3.6.1.2.1.2.2.1.1.1
            [value] => 1
        )

    [1] => Array
        (
            [oid] => 1.3.6.1.2.1.2.2.1.1.2
            [value] => 2
        )

    [2] => Array
```

```
        (
            [oid] => 1.3.6.1.2.1.2.2.1.1.3
            [value] => 3
        )

    [3] => Array
        (
            [oid] => 1.3.6.1.2.1.2.2.1.1.4
            [value] => 4
        )

)
```

The values of interest are stored in `$return_arr[$i] = $arr[$i]["value"];`. The **function_reindex** gets them all.

XML File

This given, the first step will be the xml file defining how to access index values only. So change to your `<path_cacti>/resources/script_queries` directory and create a file named **ifTraffic.xml**. You may of course choose your own name.

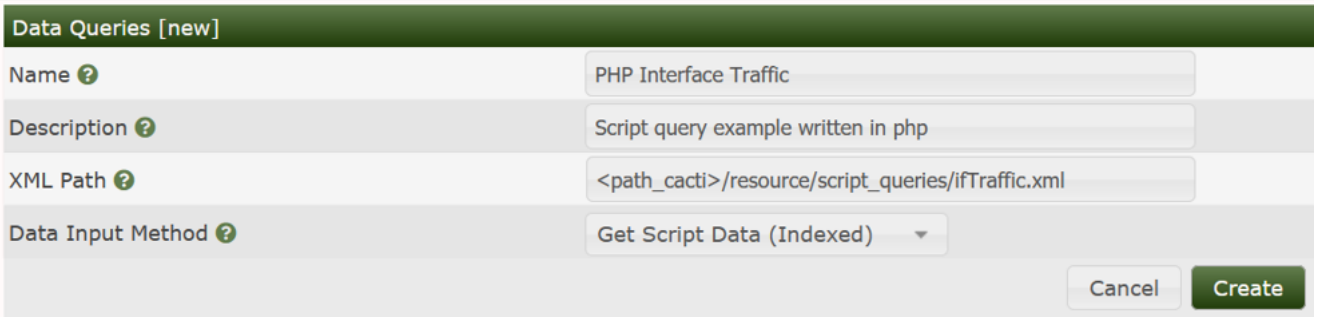
```
<interface>
    <name>Get Interface Traffic Information</name>
    <script_path>|path_php_binary| -q |path_cacti|/scripts/query_interface_traffic.php</script_path>
    <arg_prepend>|host_hostname|</arg_prepend>
    <arg_index>index</arg_index>

    <fields>
        <ifIndex>
            <name>Index</name>
            <direction>input</direction>
            <query_name>index</query_name>
        </ifIndex>
    </fields>
</interface>
```

Lets talk about the header elements

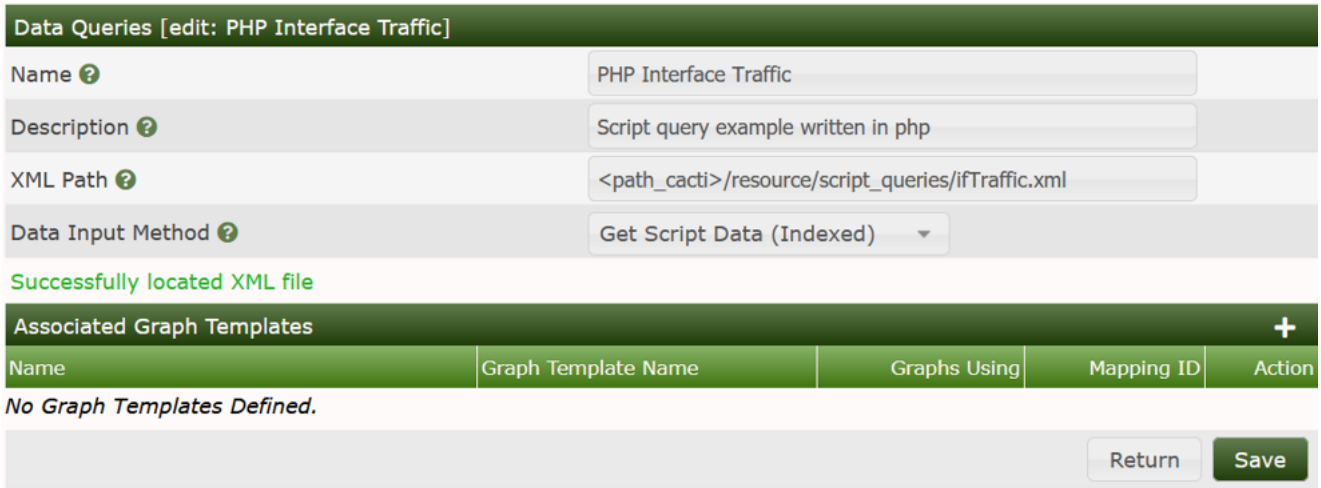
field	description
name	Short Name; chose your own one if you want
script_path	Whole command to execute the script from cli. `path_php_binary`is a cacti builtin variable for`/the/full/path/to/php`. `path_cacti` in turn gives the path of the current cacti installation directory.
arg_prepend	All arguments passed to the script go here. There are some builtin variables, again. `host_hostname` represents the hostname of the device this query will be associated to.
arg_index	The string given here will be passed just after all `<arg_prepend>` to the script for indexing requests. Up to now, this is the only method our script will answer to.
fields	All fields will be defined in this section. Up to now, only the index field is defined
name	The name of this very field
query_name	Name of this field when performing a query or a get request (will be shown later, don't worry now).
direction	**Description**
input	defines all fields that serve as a descriptive information to a specific table index. These values will not be graphed but may be printed in e.g.graph titles by means of `query\_<name>`
output	defines all fields that will yield a number that should be stored in some RRDfile

Now save this file and lets turn to cacti to implement this one. First, go to **Data Queries** to see

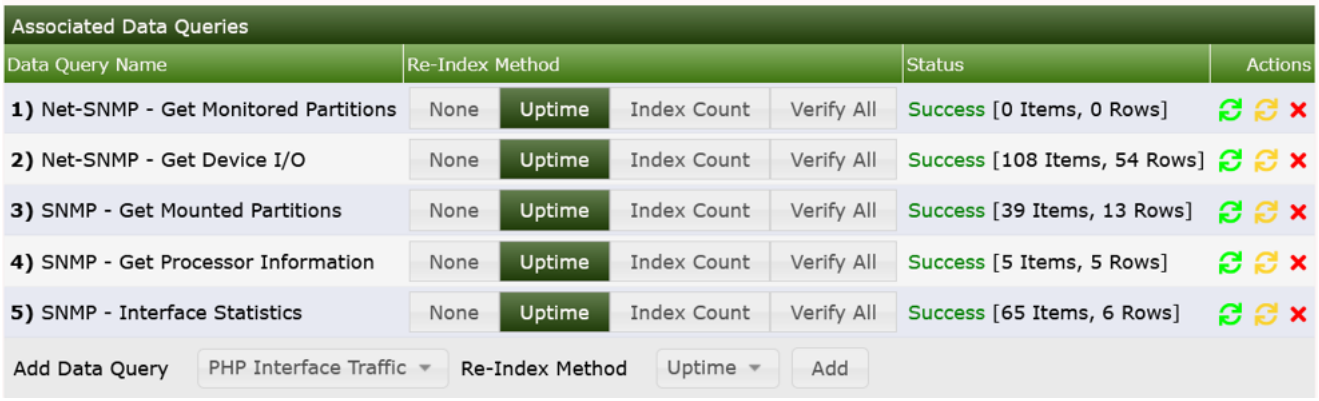
and **Add** a new one:



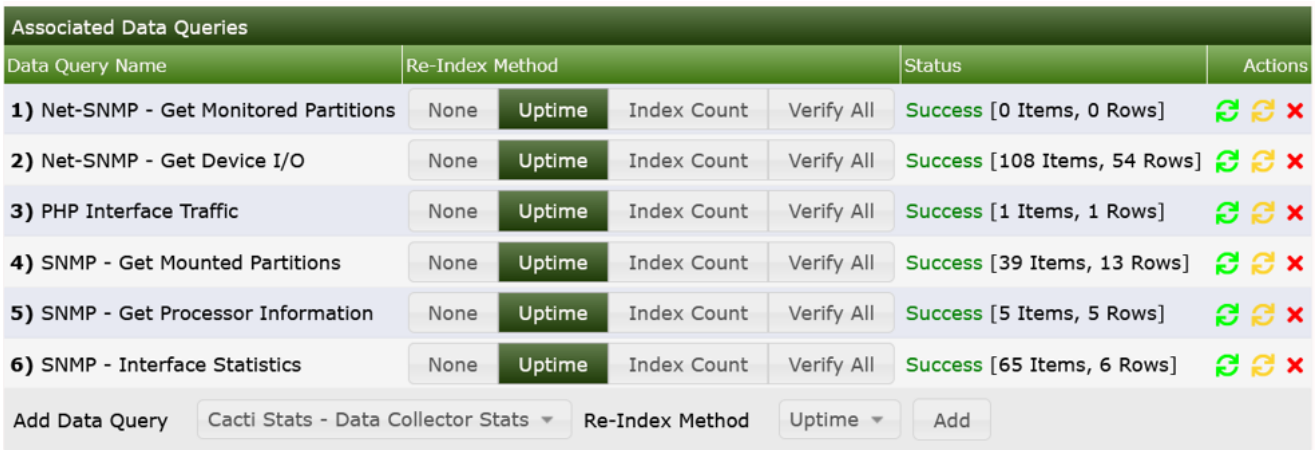
Fill in Short and Long Names at your wish. Enter the file name of the XML file and don't forget to choose **Get Script Data (indexed)**. **Create** to see



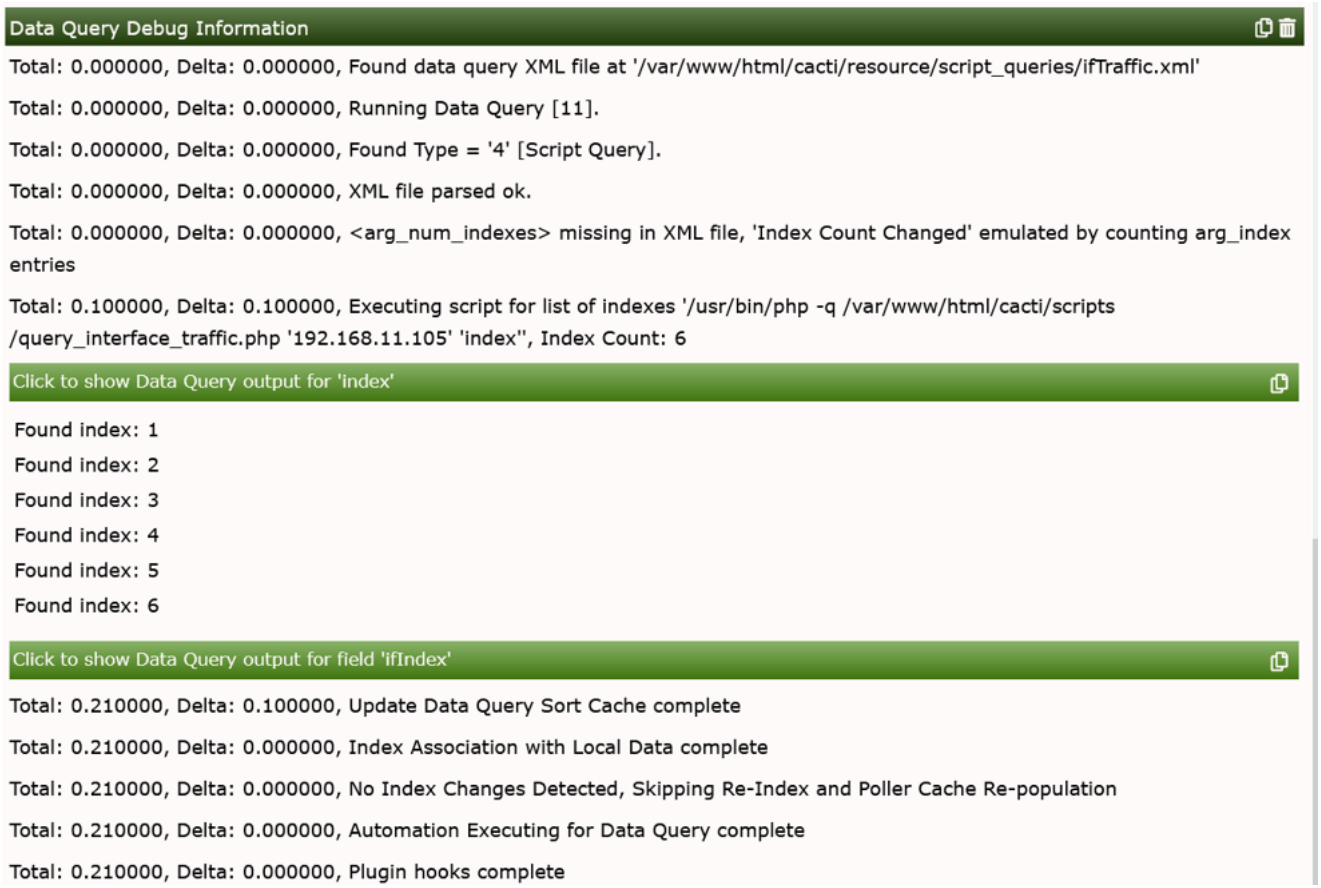
It has now **Successfully located XML file**. But this does not mean that there are no errors. So lets go on with that. Turn to the **Device** you want to query and add the new **Data Query** as shown:



Index Count Changed was chosen on purpose to tell cacti to re-index not only on reboot but each time the Index Count (e.g. number of interfaces) changed. When done, see the results as



To see your script at work, select **Verbose Query** to see:



Completing the Script

Now, lets improve our basic script. First, lets define all the variables (OIDs), this script should ask for.

```
<?php

/* do NOT run this script through a web browser */
if (!isset($_SERVER["argv"][0]) || isset($_SERVER['REQUEST_METHOD'])  || isset($_SERVER['REMOTE_ADDR'])) {
    die("<br><strong>This script is only meant to run at the command line.</strong>");
}

# deactivate http headers
$no_http_headers = true;
# include some cacti files for ease of use
include(dirname(__FILE__) . "/../include/global.php");
include(dirname(__FILE__) . "/../lib/snmp.php");

# define all OIDs we need for further processing
$oids = array(
    "index"         => ".1.3.6.1.2.1.2.2.1.1",
    "ifstatus"      => ".1.3.6.1.2.1.2.2.1.8",
    "ifdescription" => ".1.3.6.1.2.1.2.2.1.2",
    "ifname"        => ".1.3.6.1.2.1.31.1.1.1.1",
    "ifalias"       => ".1.3.6.1.2.1.31.1.1.1.18",
    "iftype"        => ".1.3.6.1.2.1.2.2.1.3",
    "ifspeed"       => ".1.3.6.1.2.1.2.2.1.5",
    "ifHWaddress"   => ".1.3.6.1.2.1.2.2.1.6",
    "ifInOctets"    => ".1.3.6.1.2.1.2.2.1.10",
    "ifOutOctets"   => ".1.3.6.1.2.1.2.2.1.16",
    );
$xml_delimiter          =  "!";
```

The next step removes all the builtin “magic strings” and replaces them by parameters. We'll have to change the XML template for that (see: `<arg_prepend>` later on). Cacti supports more SNMP parameters since version 0.8.7:

```
# all required input params
$hostname               = $_SERVER["argv"][1];
$snmp_community         = $_SERVER["argv"][2];
$snmp_version           = $_SERVER["argv"][3];
$snmp_port              = $_SERVER["argv"][4];
$snmp_timeout           = $_SERVER["argv"][5];
$max_oids               = $_SERVER["argv"][6];
# required for SNMP V3
$snmp_auth_username     = $_SERVER["argv"][7];
$snmp_auth_password     = $_SERVER["argv"][8];
$snmp_auth_protocol     = $_SERVER["argv"][9];
$snmp_priv_passphrase   = $_SERVER["argv"][10];
$snmp_priv_protocol     = $_SERVER["argv"][11];
$snmp_context           = $_SERVER["argv"][12];
$cmd                    = $_SERVER["argv"][13];
if (isset($_SERVER["argv"][14])) { $query_field = $_SERVER["argv"][14]; };
if (isset($_SERVER["argv"][15])) { $query_index = $_SERVER["argv"][15]; };

# get number of snmp retries from global settings
$snmp_retries   = read_config_option("snmp_retries");
```

The code responsible for the “index” option is left unchanged:

```
# ----------------------------------------------------------------------------
# script MUST respond to index queries
#       the command for this is defined within the XML file as
#       <arg_index>index</arg_index>
#       you may replace the string "index" both in the XML and here
# ----------------------------------------------------------------------------
#       php -q <script> <params> index
# will all indices of the target values
# e.g. in case of interfaces
#      it has to respond with the list of interface indices
# ----------------------------------------------------------------------------
if ($cmd == "index") {
    # retrieve all indices from target
    $return_arr = reindex(cacti_snmp_walk($hostname, $snmp_community,
    $oids["index"], $snmp_version, $snmp_auth_username,
    $snmp_auth_password, $snmp_auth_protocol, $snmp_priv_passphrase, $snmp_priv_protocol,
    $snmp_context, $snmp_port, $snmp_timeout, $snmp_retries, $max_oids, SNMP_POLLER));

    # and print each index as a separate line
    for ($i=0;($i<sizeof($return_arr));$i++) {
        print $return_arr[$i] . "\n";
    }
```

The new code implements the **query** function as follows

```
#
# ----------------------------------------------------------------------------
# script MUST respond to query requests
#       the command for this is defined within the XML file as
#       <arg_query>query</arg_query>
#       you may replace the string "query" both in the XML and here
# ----------------------------------------------------------------------------
#       php -q <script> <params> query <function>
# where <function> is a parameter that tells this script,
# which target value should be retrieved
# e.g. in case of interfaces, <function> = ifdescription
#      it has to respond with the list of
#      interface indices along with the description of the interface
# ----------------------------------------------------------------------------
} elseif ($cmd == "query" && isset($query_field)) {
    $arr_index = reindex(cacti_snmp_walk($hostname, $snmp_community,
        $oids["index"], $snmp_version, $snmp_auth_username,
        $snmp_auth_password, $snmp_auth_protocol, $snmp_priv_passphrase, $snmp_priv_protocol,
        $snmp_context, $snmp_port, $snmp_timeout, $snmp_retries, $max_oids, SNMP_POLLER));
        $arr = reindex(cacti_snmp_walk($hostname, $snmp_community,
        $oids[$query_field], $snmp_version, $snmp_auth_username,
        $snmp_auth_password, $snmp_auth_protocol, $snmp_priv_passphrase, $snmp_priv_protocol,
        $snmp_context, $snmp_port, $snmp_timeout, $snmp_retries, $max_oids, SNMP_POLLER));

    for ($i=0;($i<sizeof($arr_index));$i++) {
```

```
        print $arr_index[$i] . $xml_delimiter . $arr[$i] . "\n";
    }
```

Last option is the **get** function

```
#
# -----------------------------------------------------------------------
# script MUST respond to get requests
#       the command for this is defined within the XML file as
#       <arg_get>get</arg_get>
#       you may replace the string "get" both in the XML and here
# -----------------------------------------------------------------------
#       php -q <script> <params> get <function> <index>
# where <function> is a parameter that tells this script,
# which target value should be retrieved
# and   <index>    is the index that should be queried
# e.g. in case of interfaces, <function> = ifdescription
#                             <index>    = 1
#      it has to respond with
#      the description of the interface for interface #1
# -----------------------------------------------------------------------
} elseif ($cmd == "get" && isset($query_field) && isset($query_index)) {
    print (cacti_snmp_get($hostname, $snmp_community,
        $oids[$query_field] . ".$query_index", $snmp_version, $snmp_auth_username,
        $snmp_auth_password, $snmp_auth_protocol, $snmp_priv_passphrase, $snmp_priv_protocol,
        $snmp_context, $snmp_port, $snmp_timeout, $snmp_retries, $max_oids, SNMP_POLLER));
```

The rest of it is left unchanged. For sake of completeness, I repeat it here

```
# -----------------------------------------------------------------------
# -----------------------------------------------------------------------
} else {
    print "Invalid use of script query, required parameters:\n\n";
    print "    <hostname> <community> <version> <snmp_port> <timeout>
        <max_oids> <auth_user> <auth_passphrase> <auth_proto>
        <priv_passphrase> <priv_proto> <context> <cmd>\n";
}

function reindex($arr) {
    $return_arr = array();

    for ($i=0;($i<sizeof($arr));$i++) {
        $return_arr[$i] = $arr[$i]["value"];
    }

    return $return_arr;
}

?>
```

You may want to copy all those fragments together and replace the basic script. Now, lets have a try using the command line. The “index” option was already shown, but is repeated here

```
[me@gandalf scripts]$ php -q query_interface_traffic.php <target> <community> 1 161 500 "" "" "" "" "" "" ""
index
1
2
3
4
```

The Complete XML File

Of course, we now will have to complete the XML file given in Chapter II. Find it at `<path_cacti>/resources/script_queries/ifTraffic.xml`.

```
<interface>
    <name>Get Interface Traffic Information</name>
    <script_path>|path_php_binary| -q |path_cacti|/scripts/query_interface_traffic.php</script_path>
```

```
    <arg_prepend>|host_hostname| |host_snmp_community| |host_snmp_version| |host_snmp_port| |host_snmp_timeout| |
host_max_oids| "|host_snmp_username|" "|host_snmp_password|" "|host_snmp_auth_protocol|" "|
host_snmp_priv_passphrase|" "|host_snmp_priv_protocol|" "|host_snmp_context|"</arg_prepend>
    <arg_index>index</arg_index>
    <arg_query>query</arg_query>
    <arg_get>get</arg_get>
    <output_delimeter>!</output_delimeter>
    <index_order>ifIndex</index_order>
    <index_order_type>numeric</index_order_type>
    <index_title_format>|chosen_order_field|</index_title_format>
```

Let's discuss the changes

Field	Description
arg_prepend	some more parameters were added to provide all necessary values for the script. They are position-dependent. You may notice the strange tics I've added to e.g. host_snmp_username and host_snmp_password. If you're not using those SNMP V3 parameters, they must be quoted, else the script would fail because two parameters would be missing.
arg_query	The string passed to the query to perform query requests is given here. So you may modify it to your liking (in this case, the script has to be modified accordingly).
arg_get	Some as above for get requests
output_delimiter	The delimiter used for query requests to separate index and value
index_order (optional)	Cacti will attempt to find the best field to index off of based on whether each row in the query is unique and non-null. If specified, Cacti will perform this check on the fields listed here in the order specified. Only input fields can be specified and multiple fields should be delimited with a comma.
index_title_format (optional)	Specify the title format to use when representing an index to the user. Any input field name can be used as a variable if enclosed in pipes ( ). The variable `chosen_order_field` will be substituted with the field chosen by Cacti to index off of (see index_order above). Text constants are allowed as well
index_order_type (optional)	For sorting purposes, specify whether the index is numeric or alphanumeric.
Type	**Description**
numeric	The indexes in this script query are to be sorted numerically (ie. 1,2,3,10,20,31)
alphabetic	The indexes in this script query are to be sorted alphabetically (1,10,2,20,3,31)

Now lets turn to the fields section:

```
    ....
    <fields>
        <ifIndex>
            <name>Index</name>
            <direction>input</direction>
            <query_name>index</query_name>
        </ifIndex>

        <ifstatus>
            <name>Status</name>
            <direction>input</direction>
            <query_name>ifstatus</query_name>
        </ifstatus>

        <ifdescription>
            <name>Description</name>
            <direction>input</direction>
            <query_name>ifdescription</query_name>
        </ifdescription>

        <ifname>
            <name>Name</name>
```

```
            <direction>input</direction>
            <query_name>ifname</query_name>
        </ifname>

        <ifalias>
            <name>Alias</name>
            <direction>input</direction>
            <query_name>ifalias</query_name>
        </ifalias>

        <iftype>
            <name>Type</name>
            <direction>input</direction>
            <query_name>iftype</query_name>
        </iftype>

        <ifspeed>
            <name>Speed</name>
            <direction>input</direction>
            <query_name>ifspeed</query_name>
        </ifspeed>

        <ifHWaddress>
            <name>HWaddress</name>
            <direction>input</direction>
            <query_name>ifHWaddress</query_name>
        </ifHWaddress>

        <ifInOctets>
            <name>InOctets</name>
            <direction>output</direction>
            <query_name>ifInOctets</query_name>
        </ifInOctets>

        <ifOutOctets>
            <name>OutOctets</name>
            <direction>output</direction>
            <query_name>ifOutOctets</query_name>
        </ifOutOctets>
    </fields>
</interface>
```

These fields are related to the **OID** array of the script. **Attention**: The **query_name** strings must match the OID names **exactly!** Please notice, that all but the last two fields use **direction input**. All variables representing numeric values to be graphed must be defined as **direction output** instead.

Now, lets test the “query” option. The keyword “query” must be given along with the variable, that should be queried. The script now will scan all indices and report the contents of the given variable as follows:

```
[me@gandalf scripts]$ php -q query_interface_traffic.php <target> <community> 1 161 500 "" "" "" "" "" "" ""
query iftype
1!ethernetCsmacd(6)
2!0
3!0
4!ethernetCsmacd(6)
```

The output reports the index, followed by the chosen delimiter. Then, the content of the requested variable is printed

Last, the “get” option is shown. The keyword “get” is required, followed again by the variable (see above). Last needed option is the index, for which the “get” should be performed. Contrary to the “query” option, only one index is scanned. So the index number is not required and will not be printed.

```
[me@gandalf scripts]$ php -q query_interface_traffic.php <target> <community> 1 161 500 "" "" "" "" "" "" "" get
iftype 1
ethernetCsmacd(6)
```

The output is not followed by a “newline”!

See it at work

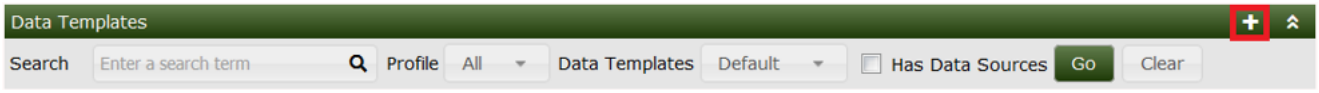
Lets return to the **Device** and perform a **Verbose Query** again. The result is as follows

```
+ Running data query [21].
+ Found type = '4 '[script query].
+ Found data query XML file at '/var/www/html/cacti/resource/script_queries/ifTraffic.xml'
+ XML file parsed ok.
+ Executing script for list of indexes '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php
router snmp-get 1 161 600 10 "admin" "admin" "MD5" "" "DES" "" index'
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query index'
+ Found item [ifIndex='1'] index: 1
+ Found item [ifIndex='2'] index: 2
+ Found item [ifIndex='3'] index: 3
+ Found item [ifIndex='4'] index: 4
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query ifstatus'
+ Found item [ifstatus='up(1)'] index: 1
+ Found item [ifstatus='up(1)'] index: 2
+ Found item [ifstatus='up(1)'] index: 3
+ Found item [ifstatus='up(1)'] index: 4
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query ifdescription'
+ Found item [ifdescription='Ethernet0'] index: 1
+ Found item [ifdescription=''] index: 2
+ Found item [ifdescription=''] index: 3
+ Found item [ifdescription='Ethernet1'] index: 4
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query ifname'
+ Found item [ifname=''] index: 1
+ Found item [ifname=''] index: 2
+ Found item [ifname=''] index: 3
+ Found item [ifname=''] index: 4
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query ifalias'
+ Found item [ifalias=''] index: 1
+ Found item [ifalias=''] index: 2
+ Found item [ifalias=''] index: 3
+ Found item [ifalias=''] index: 4
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query iftype'
+ Found item [iftype='ethernetCsmacd(6)'] index: 1
+ Found item [iftype='0'] index: 2
+ Found item [iftype='0'] index: 3
+ Found item [iftype='ethernetCsmacd(6)'] index: 4
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query ifspeed'
+ Found item [ifspeed='100000000'] index: 1
+ Found item [ifspeed='0'] index: 2
+ Found item [ifspeed='0'] index: 3
+ Found item [ifspeed='10000000'] index: 4
+ Executing script query '/usr/bin/php -q /var/www/html/cacti/scripts/query_interface_traffic.php router snmp-get
1 161 600 10 "admin" "admin" "MD5" "" "DES" "" query ifHWaddress'
+ Found item [ifHWaddress='00:30:30:2E:35:30:2E:37:46:2E:30:43:2E:30:30:2E:44:16:00:00:00:01:00'] index: 1
+ Found item [ifHWaddress=''] index: 2
+ Found item [ifHWaddress=''] index: 3
+ Found item [ifHWaddress=''] index: 4
+ Found data query XML file at '/var/www/html/cacti/resource/script_queries/ifTraffic.xml'
+ Found data query XML file at '/var/www/html/cacti/resource/script_queries/ifTraffic.xml'
+ Found data query XML file at '/var/www/html/cacti/resource/script_queries/ifTraffic.xml'
```

Of course, snmp_username and snmp_user_password and more may differ from your installation defaults. Read it carefully, and you'll notice, that all XML **fields** were scanned and the output shown. All? No, not all. The **direction output** fields are missing! But this is on purpose as those won't make sense as header fields but will be written to RRDfiles.

Create the Data Template

As usual, next step is to create the **Data Template**. Select that menu item and **Add**:

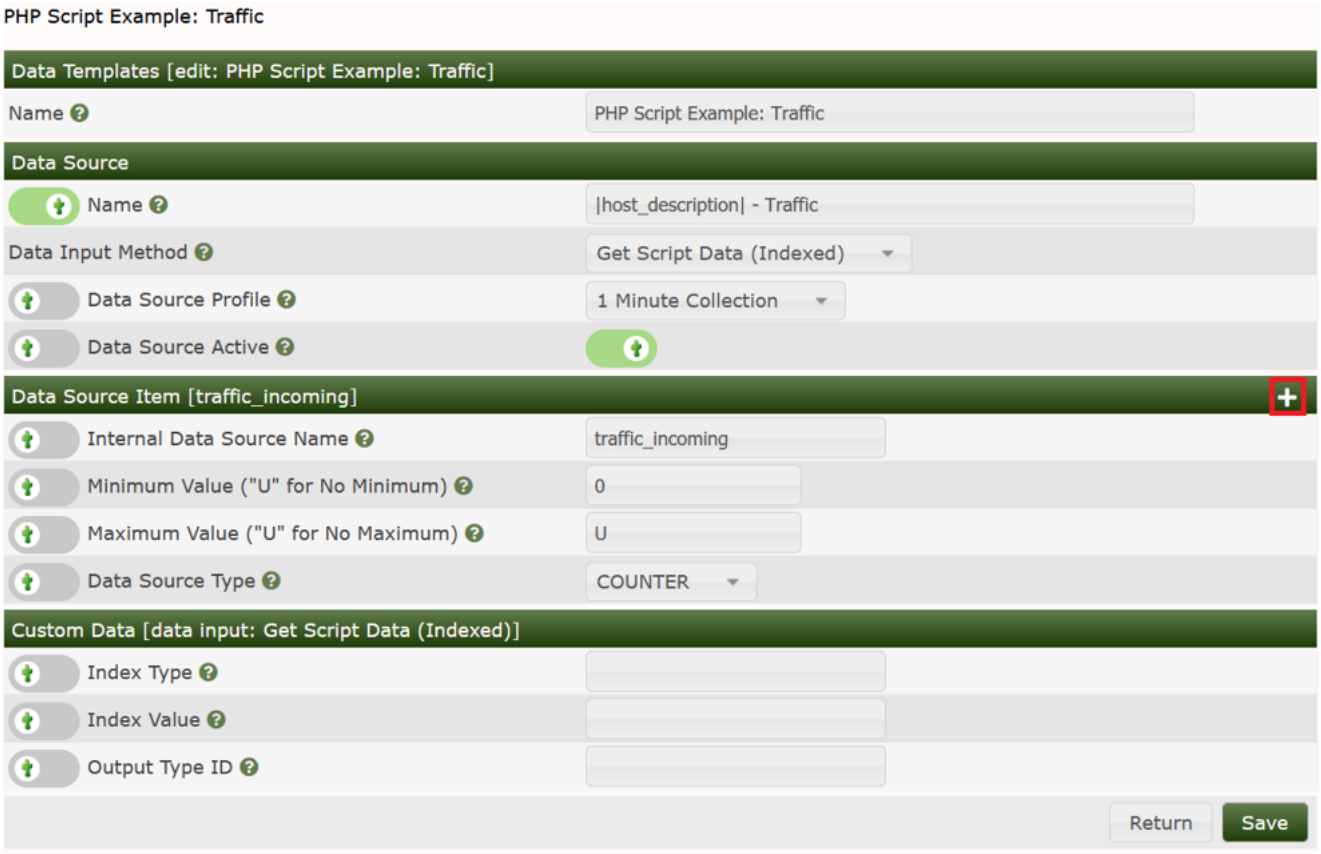


and find:

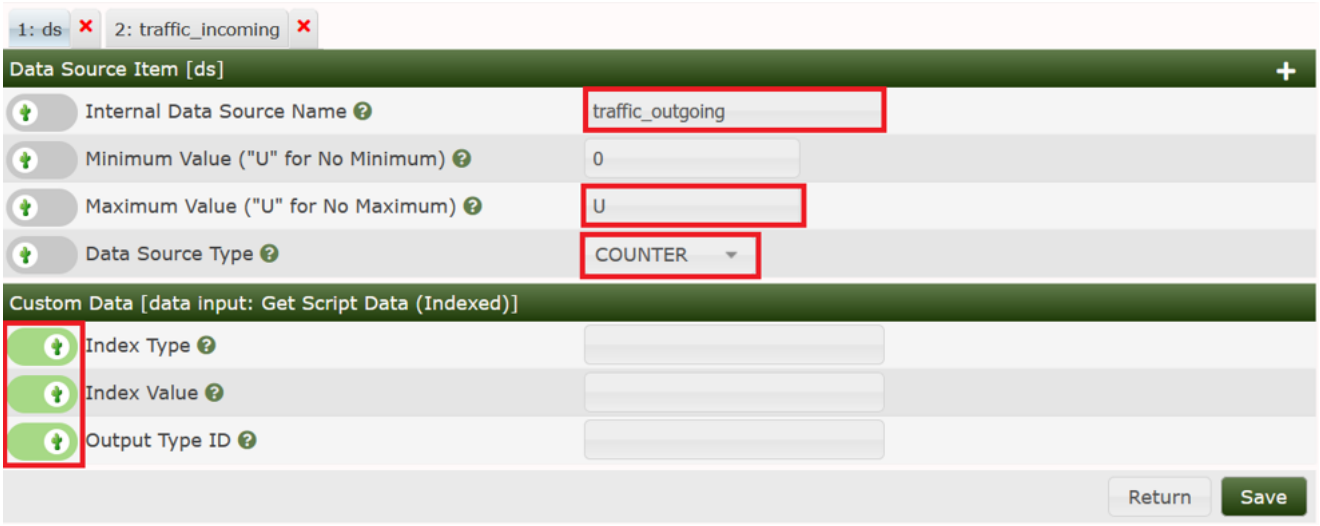
Fill in **Data Template Name**, **Data Source Name**, and, most important, select **Data Input Method** to read **Get Script Data (Indexed)**, and **Data Source Profile**.

When creating the **Data Template** and **Graph Template**, you SHOULD check the `Use Per Data Source Value` checkbox for name & title. When you first create graphs using the data query, it will use the `Suggested Values` to name the templates. But then if you ever edit the templates and leave the `Use Per Data Source Value` unchecked, then saving will overwrite all the data source and graph names.

Now, before you press the `Create` button also please proceed to the lower half of the form and complete the first RRDfile Data Source. Enter the **Internal Data Source Name**. You may select this name freely, though note your are limited to only 19 characters and spaces are not allowed. There's no need to match it to any of the XML field names. As the OID is a **COUNTER**, the **Data Source Type** must be selected appropriately. `Create`.



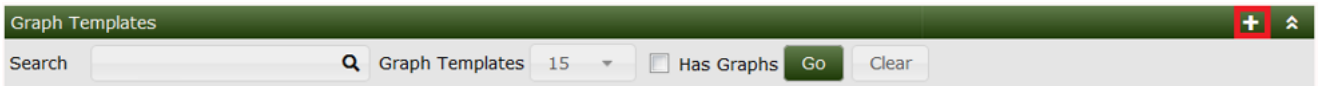
For the second data source item, please select **New**.



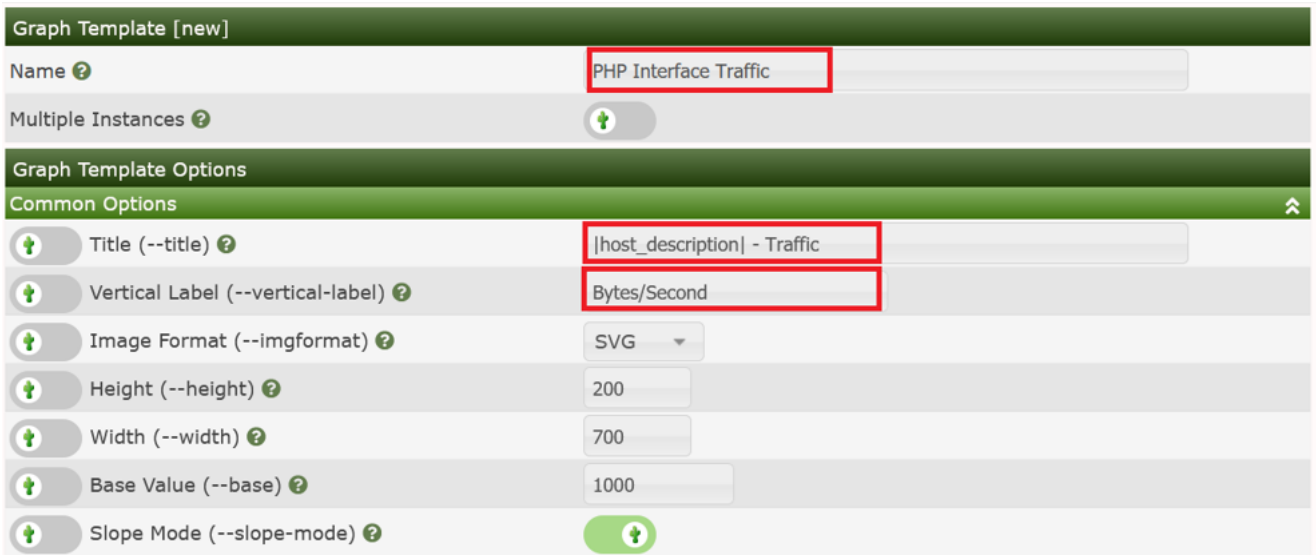
Again, fill in the Data Source Name. Pay attention to set the maximum value to 0 to avoid clipping it off during updating of the RRDfile. COUNTER has to be set as done above. **Important!** You have to select the marked Index fields! Now, save again and you're done.

Create the Graph Template

Now, its time for the **Graph Template**. Select this menu item and **Add**.

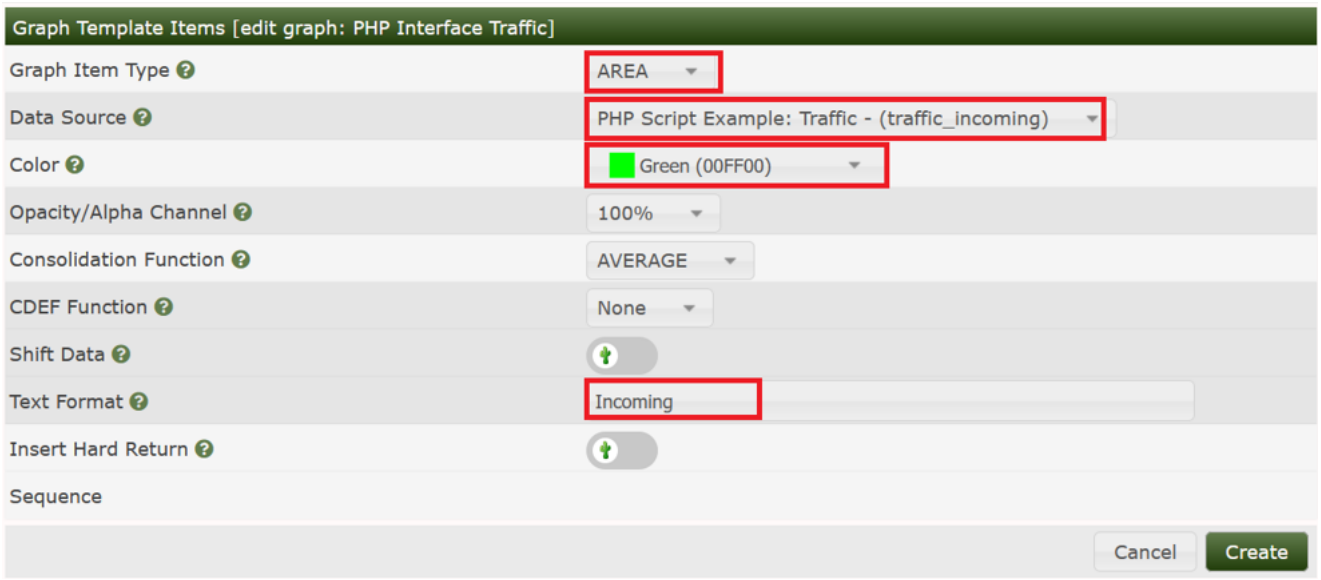


and fill in the values as usual:

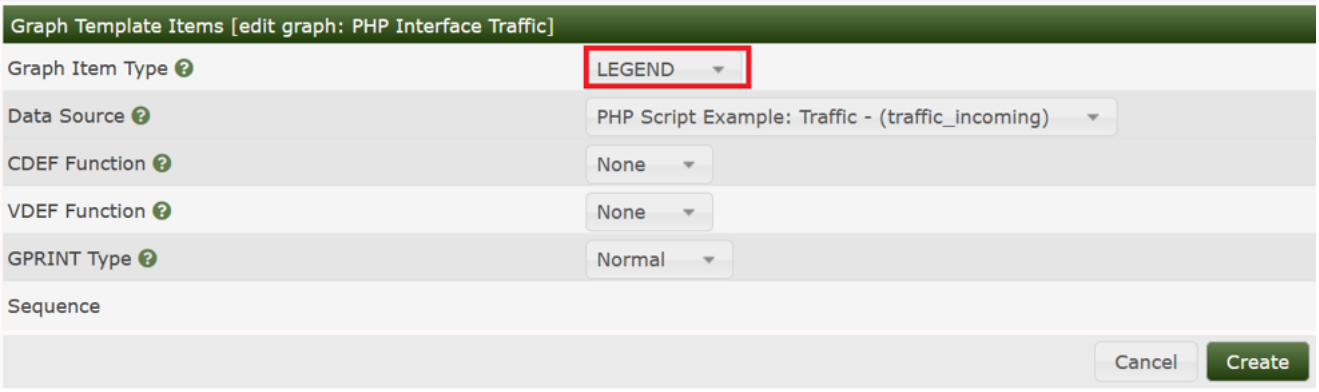


Now **Save**. Next, fill in the **Graph Items**

Select the Data Source from our Data Template, take the color and select AREA, enter some text

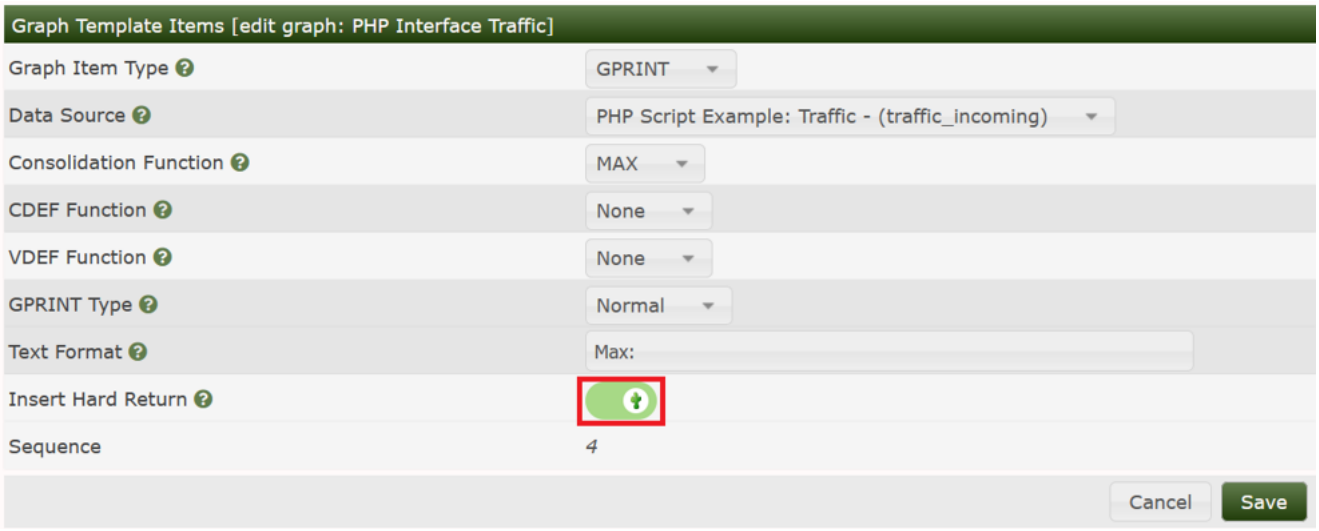


Save and add the next graph item. Now, we're going to use the “LEGEND” time saver again:



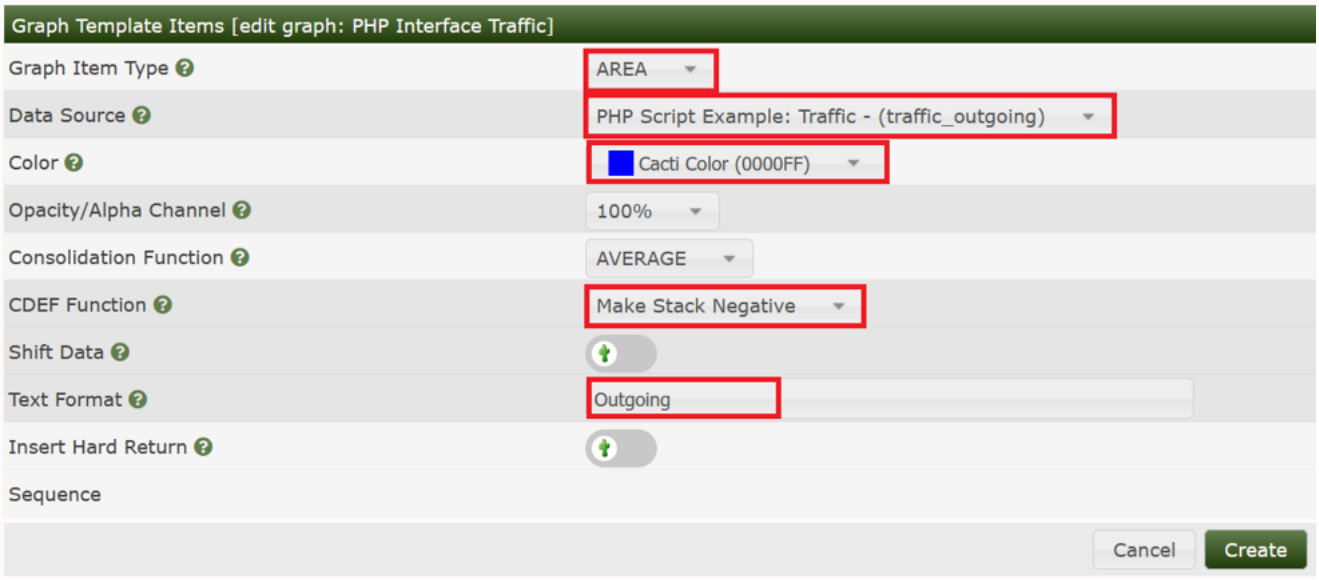
For the next step, it's necessary to remove the newline added with the last action. Please select the 4th item as follows

and remove the newline by deselecting the checkbox

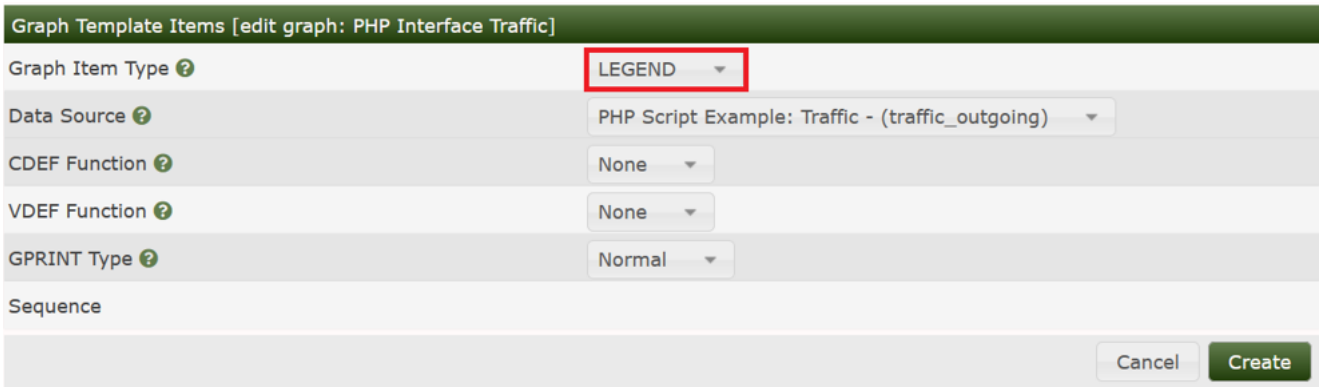


Now lets add the same data source again, but as a LINE1, MAXimum with a slightly changed color. Newline is checked this time

Pooh. Now lets apply the same procedure for the **Outgoing Traffic**. Personally, I love those outgoing stuff to be presented on the negative y-axis. So we'll have to apply some CDEF magic to some items. Lets see



Please pay attention when adding the “LEGEND” stuff. No CDEF to be applied in this case (else, legends will show negative values)



Again, select last legend item

Graph Template Items [edit: PHP Interface Traffic]

Graph Item	Data Source	Graph Item Type	CF Type	Alpha %	Item Color
Item # 1	**PHP Script Example: Traffic (traffic_incoming): Incoming**	**AREA**	**AVERAGE**	**100%**	**00FF00**
Item # 2	PHP Script Example: Traffic (traffic_incoming): Cur:	GPRINT	LAST		
Item # 3	PHP Script Example: Traffic (traffic_incoming): Avg:	GPRINT	AVERAGE		
Item # 4	PHP Script Example: Traffic (traffic_incoming): Max:	GPRINT	MAX		
Item # 5	**PHP Script Example: Traffic (traffic_incoming): Max<HR>**	**LINE1**	**MAX**	**100%**	**00BF47**
Item # 6	**PHP Script Example: Traffic (traffic_outgoing): Outgoing**	**AREA**	**AVERAGE**	**100%**	**0000FF**
Item # 7	PHP Script Example: Traffic (traffic_outgoing): Cur:	GPRINT	LAST		
Item # 8	PHP Script Example: Traffic (traffic_outgoing): Avg:	GPRINT	AVERAGE		
Item # 9	PHP Script Example: Traffic (traffic_outgoing): Max:<HR>	GPRINT	MAX		

to remove the newline

Graph Template Items [edit graph: PHP Interface Traffic]

Graph Item Type	GPRINT
Data Source	PHP Script Example: Traffic - (traffic_outgoing)
Consolidation Function	MAX
CDEF Function	None
VDEF Function	None
GPRINT Type	Normal
Text Format	Max:
Insert Hard Return	
Sequence	9

Cancel Save

and add a new LINE1, MAXimum, “Make Stack Negative” CDEF with some text and a newline

Graph Template Items [edit graph: PHP Interface Traffic]

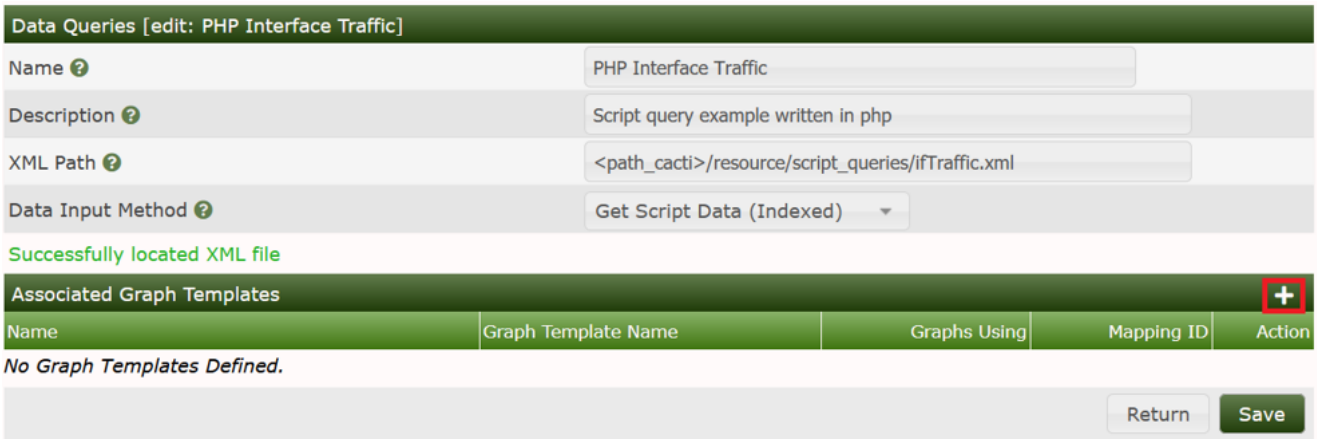
Graph Item Type	LINE1
Data Source	PHP Script Example: Traffic - (traffic_outgoing)
Color	Cacti Color (00004D)
Opacity/Alpha Channel	100%
Consolidation Function	MAX
CDEF Function	Make Stack Negative
Shift Data	
Text Format	Max
Insert Hard Return	
Line Width (decimal)	1.00
Dashes (dashes[=on_s[,off_s[,on_s,off_s]...]])	
Dash Offset (dash-offset=offset)	
Sequence	

Cancel Create

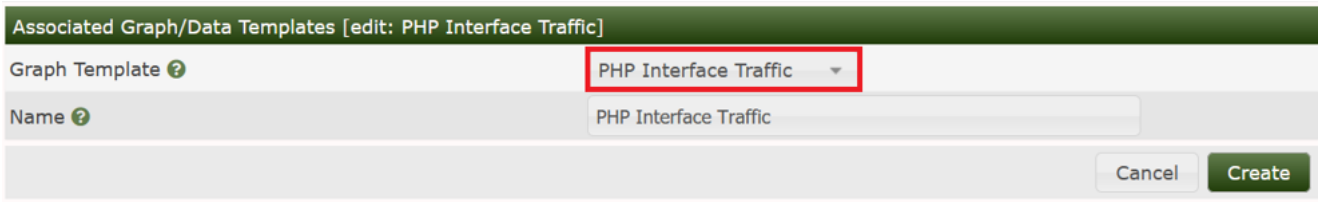
Hoping, you've got all those steps correctly, finally **Save** your work. Take a cup of coffee to get your brains free again, kiss your wife, hug your children and/or pet your dog; sequence is arbitrary.

Associate Graph Template with Data Query

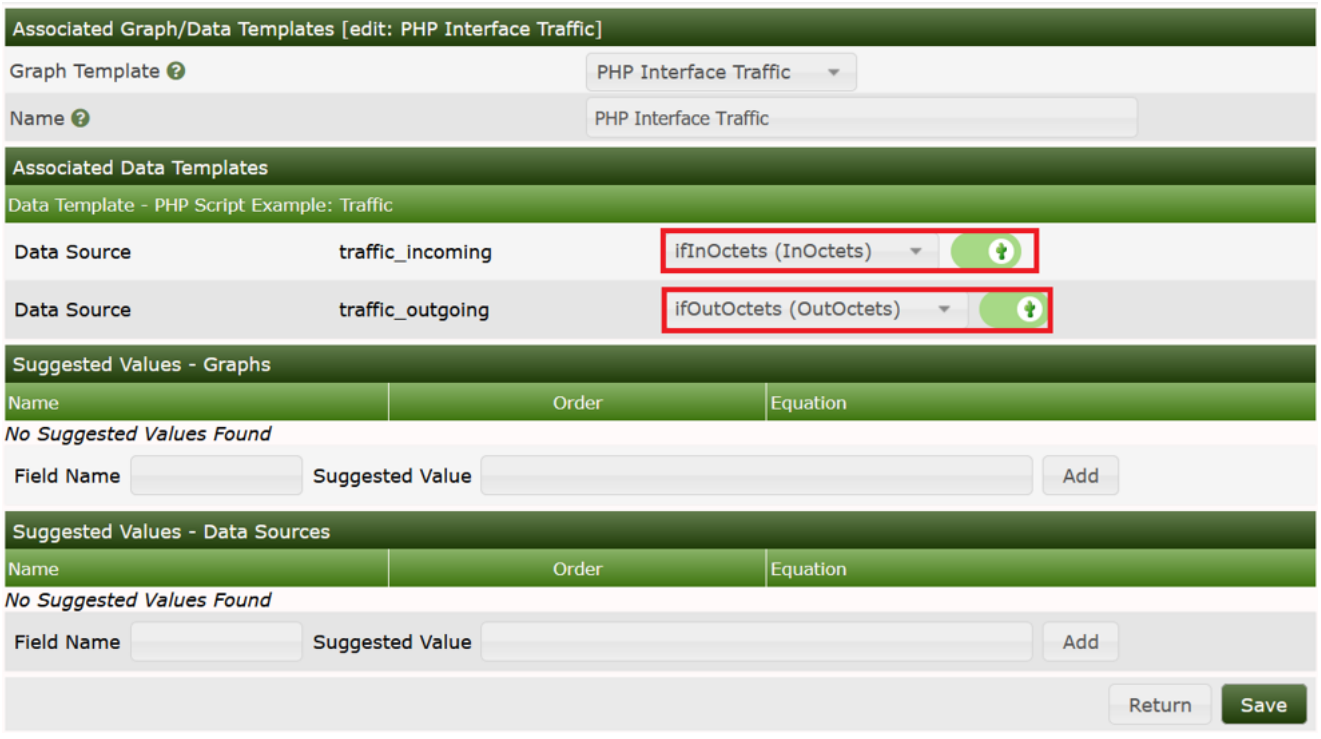
Huh, that sound complicated. Why would it be necessary to do so? Let me explain: You remember the **Data Template**, do you? The names of the data source item was chosen arbitrary. The Graph Items were associated with those data source items, but those in turn were not related to anything in the XML file. Not related? Not yet! So, let's revisit the **Data Query**. Remember the lower part on **Associated Graph Templates**. Click **Add**



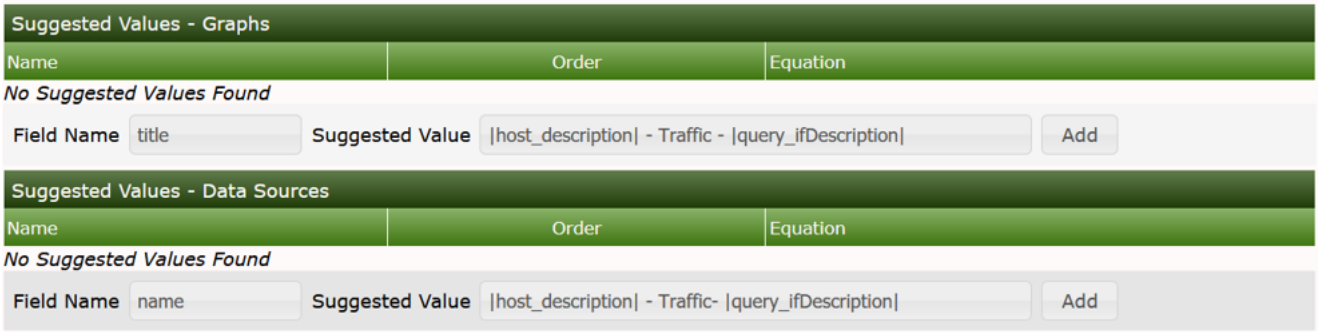
fill in a name for your choice and select the Graph Template that we have created in the last step.



Create to see



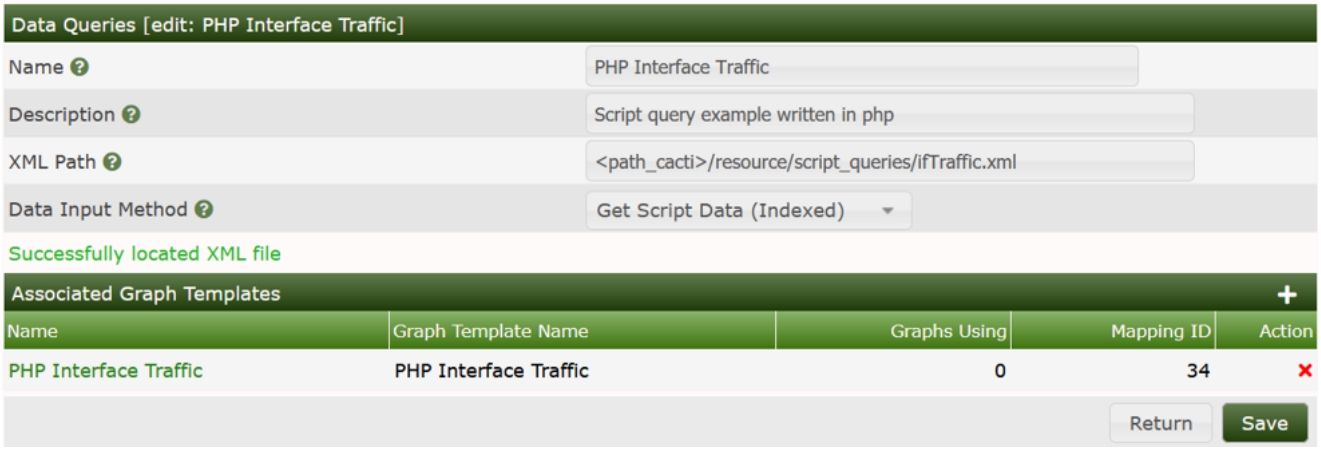
First, let's have a look at the upper half of the screen. The red box to the left show the **Internal Data Source Names** names taken from the **Data Template** that is associated with the **Graph template** we've just added. The red box to the middle has a dropdown for each data source item. The dropdown list contains all **output** fields taken from the XML file. In our case, there are only two of them. The red box to the right must be checked on each line, to make the association valid. Now, lets turn to the lower half of the screen, denoted **Suggested Values**



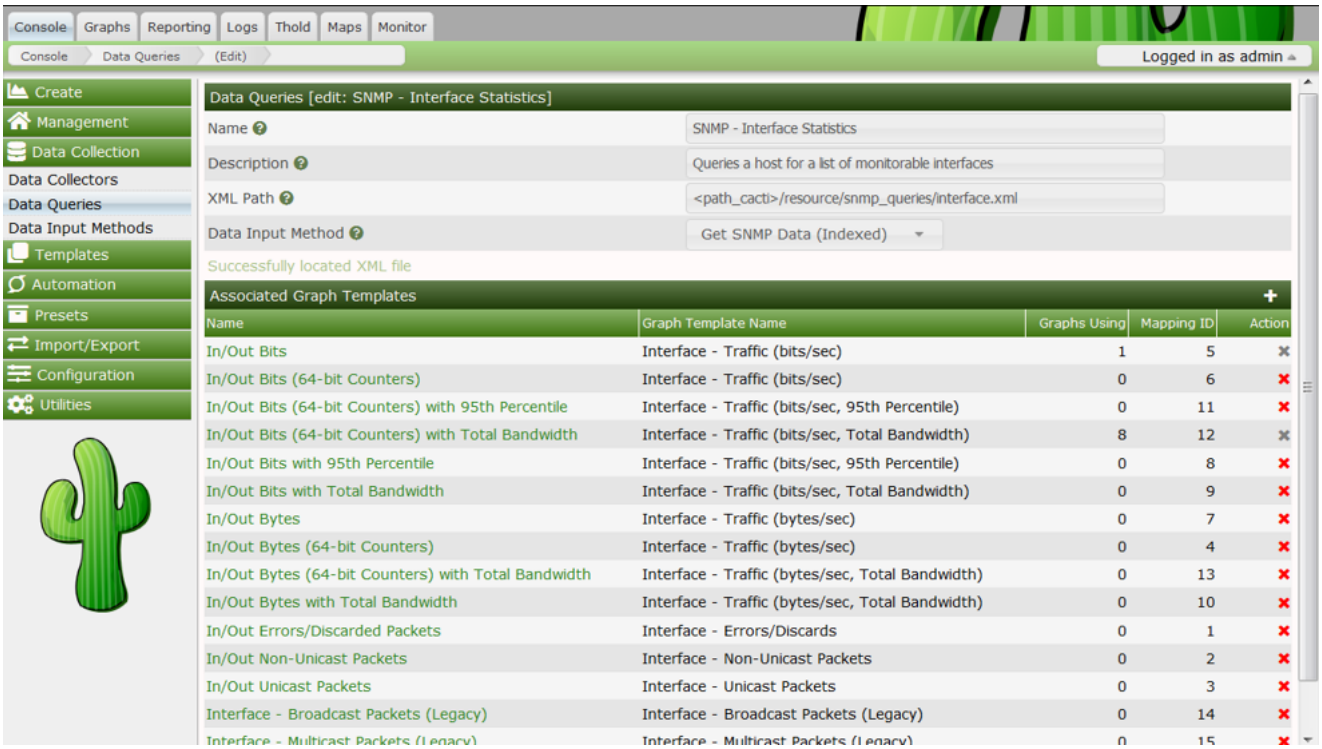
The example shows `|host_description| - Traffic - |query_ifdescription|` entered both for **name** of the Data Template and **title** of the Graph Template. Click **Add**, one by one

Notice the second **title** I've added here. If more than one entry is present, they are applied from top to bottom, until a match is found. Match means, that all variables present are filled. Of course, you may add more than one variable taken from the XML file. But pay attention, that not all devices will fill all those variables. So my router does, sigh. You may use all **input** variables listed in the XML file. A `<variable>` may be listed as `|query_<variable>|`, e.g. for ifalias write `|query_ifalias|` and so forth.

Click **Save**, and find the new Graph Template added to the list of **Associated Graph Templates**.



You may continue to add more **Graph Templates**, each of them may be related to other output field of the XML file. Find, as an example, lots of graph templates associated to the standard `Interface Statistics` Data Query to get an idea what I'm talking about.



Don't worry about the first two entries; they are home-made.

Creating the Images

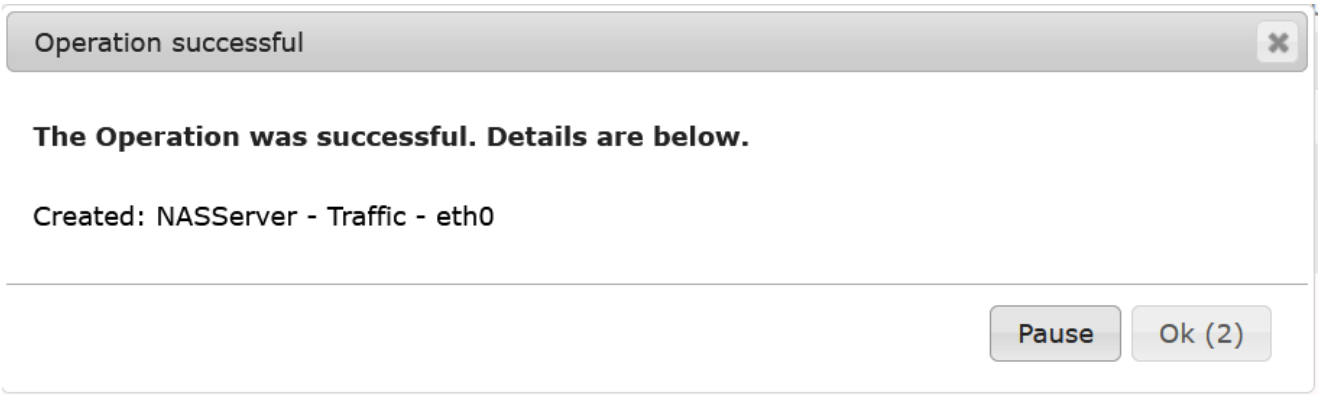
Now, let's return to the **Device**, that we've already have used for this Data Query. **Create Graphs for this Host**

to see

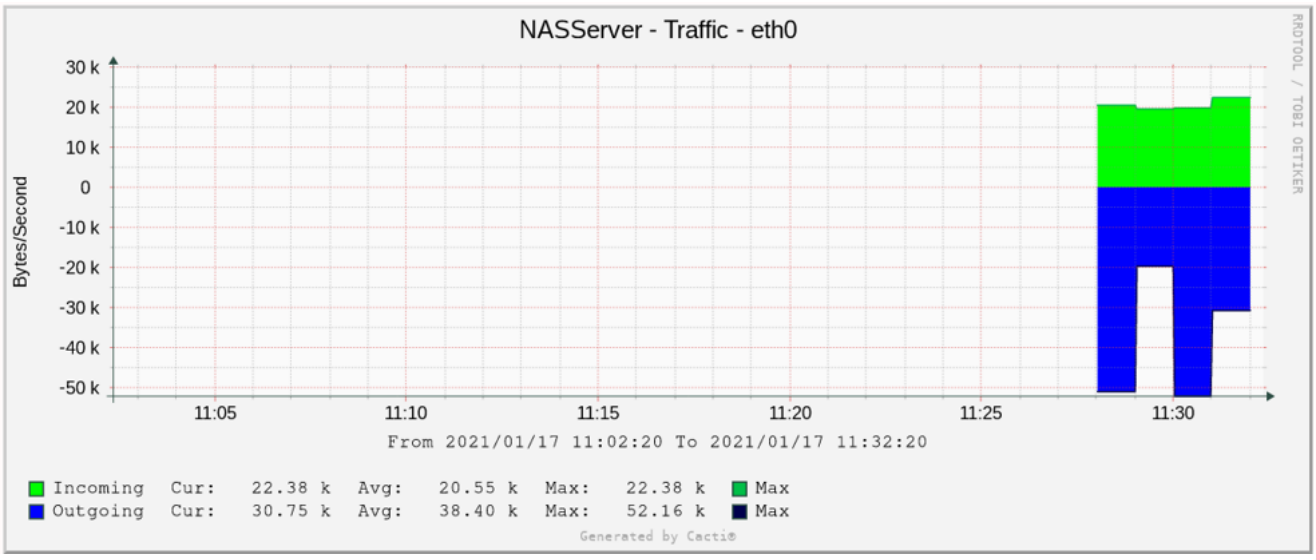
I've left the standard Interface Statistics in the screenshot. So you may compare both Queries. Our PHP Interface Traffic stuff has two more header items, Name and Alias. But all data seen equals the standard SNMP Data Query; not that bad, eh?

Now, select one item

and **Create**



You'll have to wait a bit, at least two polling cycles. Then, you may notice some data in your new graph. The next image shows both our new graph (the first one) and a standard interface traffic graph. The latter one holds more data in this example, don't worry about that.



Having a closer view, you may notice a difference in magnitude (y-axis). But please compare the units used. The first graph uses **Bytes**, the latter one uses **Bits**. For comparison, it would be necessary to multiply the first one with 8. This may be done using a CDEF Turn Bytes into Bits, applied to all items of the **Graph Template**. This task is left to you.

Please find the example resource, script, and Data Query below. Save `ifTraffic.xml` into `./resource/script_queries`, and `query_interface_traffic.php` into the `./scripts` directory and import the Data Query `cacti_data_query_php_interface_traffic.xml`.

Cacti Data Query for PHP Interface Traffic

ifTraffic.xml download and store into `resource/script_queries`.

query_interface_traffic.php download and store into `scripts`.

Copyright (c) 2004-2026 The Cacti Group

How To Determine a Cacti Template Version

Template XML object names combine an object-type code, a compatibility code, and an object identity. The compatibility code does not identify an exact exporting Cacti release, because several releases can share it.

For example:

```
hash_040018258d1c9487a6c58dd804f4a012007664
     04     object type
       0018 compatibility code
           258d1c9487a6c58dd804f4a012007664 object identity
```

The first two characters identify the object type. The next four are the compatibility code, and the remaining 32 hexadecimal characters identify the object. Preserve these identities when exporting an update to existing objects. The author's template or package version is a separate value.

Compatibility code families

Table 1. Selected code families

Code	Releases represented in the source mapping
`0018`	Cacti 0.8.7d, the legacy example above.
`0024`	Cacti 0.8.8, 0.8.8a and 0.8.8b.
`0025`	Cacti 0.8.8c through 0.8.8h.
`0100`	Cacti 1.0.x and 1.1.0 through 1.1.5.
`0101`	Cacti 1.1.6 through 1.2.2.
`0102`	Cacti 1.2.3 through 1.2.18.
`0103`	Cacti 1.2.19 and later 1.2.x entries in the mapping.
`0104`	Cacti 1.3.0 in the development mapping.

For the complete mapping, consult `include/global_arrays.php` in the selected Cacti release. Older `0000`–`0017` codes also appear there. The source mapping and version gate show the behavior inspected for this documentation branch.

For example, the gate for a Cacti 1.2.3 installation recognizes the same `0102` code used by 1.2.18. Passing this gate does not prove every dependency or imported field is usable; review the import preview and the template author's requirements.

Resolve an import error

An unknown code is absent from the installed mapping. A recognized code can also be newer than the installed version's code. These conditions differ from an invalid XML document, an unmet dependency, or a damaged export.

Check the full error in `cacti.log`, confirm the exporting release and required dependencies with the author, and use a compatible export or upgrade Cacti when required. Do not rewrite hash prefixes to bypass validation. See Template Import for the actual import workflow.

Copyright (c) 2004-2026 The Cacti Group

Import Packages

Use **Console > Import/Export > Import Packages** for signed `.xml.gz` packages on the development branch. Unlike Template Import, this workflow can install selected scripts and query resources as well as database objects. The repository and public-key screens described here are not Cacti 1.2 interfaces; use its installer or `cli/import_package.php` instead.

Select the source and preferences

1. Select an enabled **Package Location**, or **Local Package File** to upload a package from your computer. Configure missing locations through Repositories.
2. Review **Data Source Profile**, including **Create New from Template** when appropriate. Profile selection does not convert existing RRD files.
3. Review graph format/dimensions, **Remove Orphaned Graph Items**, and suggested value-pattern replacement. Orphan removal can delete local graph items; pattern replacement can affect names after reindexing or forced replacement.
4. Choose the local `.xml.gz` file or select the packages from the repository. Inspect the preview before submitting the import.

Verify the signer and changes

The importer validates package signatures and checks signer trust. Compare an unfamiliar signing identity with information obtained independently from the author before accepting it. The name or homepage inside a package is not independent proof. See Package Public Keys.

Inspect the template objects and file changes, including paths and whether existing files will be replaced. Import only the objects and files needed for the selected templates. Make a database and file backup before updating existing objects. Preview is not a strictly read-only audit: compatibility repair can update existing system data-input metadata in the import path.

If a signature, key, dependency, file permission or download check fails, resolve that failure before continuing. Do not modify a signed payload to work around verification; obtain a corrected package from the author.

Apply and verify

Submit **Import** after reviewing the selected templates, files and signer. Check the result messages and `cacti.log`; a downloaded package or completed preview does not establish that all requested objects/files were installed.

Confirm the imported templates and required scripts/query resources on the collector that will poll the device. Grant temporary resource-directory write access only where required by the operation, then restore intended permissions. Complete the package's external setup, such as SNMP-agent configuration or credentials. Create a test graph and verify its measurements before applying the template across a fleet.

For command-line use, run `php cli/import_package.php --help` from the installed Cacti directory and use the selected release's options. `--filename` selects a package and `--preview` requests a preview; CLI and web preferences are not interchangeable arguments.

Copyright (c) 2004-2026 The Cacti Group

Template Import

Import Templates imports ordinary template XML into Cacti. Templates can create new objects or update objects identified by matching hashes. Back up existing definitions before importing an update from a template author.

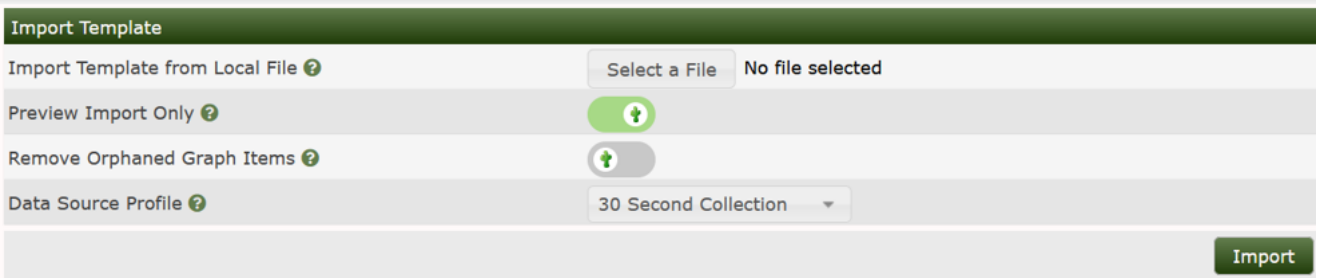
Template XML and signed packages

Signed `.xml.gz` packages also contain resource files. They use the separate Import Packages workflow; do not upload them here.

An ordinary XML import may require scripts under `scripts` and query definitions under `resource` to be installed separately. Follow the author's instructions and inspect the collection method and dependencies first.

Import template XML

1. Open **Console > Import/Export > Import Templates**.
2. Choose the **Data Source Profile** for the resulting definitions, or **Create New from Template** when the export includes a profile to import. Review graph format/dimension overrides and the update options before choosing the file.
3. Use **Select File** to choose the XML. File selection automatically requests a preview and displays the proposed objects and changes. There is no preview checkbox to untick and no need to upload the same file a second time.
4. Review new, updated, damaged and unchanged objects and missing dependencies. Select the objects you intend to import; the interface notes that selecting none imports all. Resolve errors before continuing.
5. Press **Import** to submit the selected file and options. Check the resulting messages and `cacti.log`, then verify the imported templates and required resources before creating graphs.



Preview is not a strictly read-only audit. Compatibility repair can update existing system data-input metadata during the import path, even in preview. Use an isolated copy when assessing an untrusted or disruptive update.

Update options

Remove Orphaned Graph Items deletes items absent from the incoming graph template, from both that template and its attached graphs. This can be useful when deliberately adopting a revised definition; it is not restricted to damaged templates and is not required for every update. Review orphaned items first, especially when retaining local customizations.

Replace Data Query Suggested Value Patterns replaces the query's suggested naming patterns. Graph and data-source names can change after reindexing or a forced suggested-value replacement. Review these changes separately from graph item deletion.

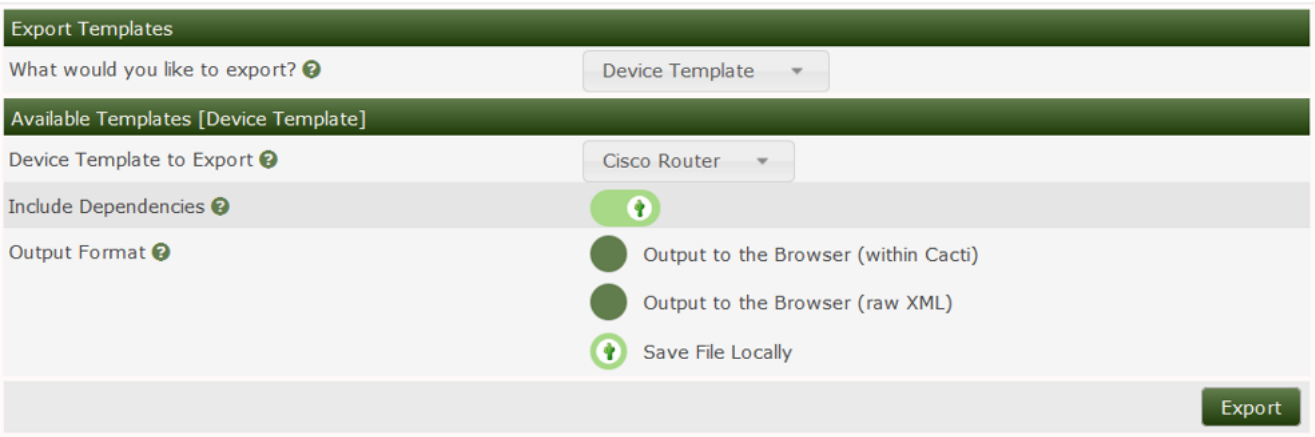
The selected profile and graph overrides affect imported definitions; importing a new profile does not automatically migrate the physical structure of existing RRD files. See Data Source Profiles.

For compatibility errors, see Determine a Template Version. For sharing complete definitions, see Template Export.

Copyright (c) 2004-2026 The Cacti Group

Template Exporting

Open **Console > Import/Export > Export Templates**, choose the object type, then select the template to export. Select **Include Dependencies** when the recipient needs its referenced objects. Choose the output option to display XML or download it as a file.



Each template component has a persistent hash identity. Import uses it to match objects across installations. The historical Linux example below illustrates the XML structure; it is not a current, self-contained import fixture. Generate a complete export from your own installation rather than copying omitted or legacy dependencies.

An export with dependencies can include:

1. The device template name and its identity
2. All the Graph Templates and their GUID's
3. All the Graph Template inputs and their GUID's
4. All the Data Templates and their GUID's
5. All the Data Input Method's and their GUID's
6. All the GPrint Presets and their GUID's
7. All the CDEF's and VDEF's and their GUID's
8. All the Data Source Profiles and their GUID's

Most fields use XML escaping. Data-input command strings can be base64-encoded; decode only fields defined that way by the format. For example, the path placeholder is stored as `<path_cacti>` in XML, not as an unclosed element.

Template export writes XML. Preserve object hashes when sharing an update: they identify the objects to update on import. The compatibility prefix is described in Determine a Template Version.

Select **Include Dependencies** when the recipient needs referenced templates, queries, profiles, CDEFs or VDEFs. Ordinary template XML does not bundle the external scripts or query XML files; install those separately or use a signed package workflow. Review the import preview on a separate test installation before distributing an update that changes existing objects.

```
<cacti>
        <hash_0201022d3e47f416738c2d22c87c40218cc55e>
                <name>Local Linux Machine</name>
                <graph_templates>hash_0001029fe8b4da353689d376b99b2ea526cc6b|
hash_000102fe5edd777a76d48fc48c11aded5211ef|hash_00010263610139d44d52b195cc375636653ebd|
hash_0001026992ed4df4b44f3d5595386b8298f0ec</graph_templates>
                <data_queries>hash_0401028ffa36c1864124b38bcda2ae9bd61f46</data_queries>
        </hash_0201022d3e47f416738c2d22c87c40218cc55e>
        <hash_0001029fe8b4da353689d376b99b2ea526cc6b>
                <name>Unix - Processes</name>
                <graph>
                        <t_title></t_title>
                        <title>|host_description| - Processes</title>
                        <t_vertical_label></t_vertical_label>
```

```
                        <vertical_label>processes</vertical_label>
                        <t_image_format_id></t_image_format_id>
                        <image_format_id>1</image_format_id>
                        <t_height></t_height>
                        <height>200</height>
                        <t_width></t_width>
                        <width>700</width>
                        <t_base_value></t_base_value>
                        <base_value>1000</base_value>
                        <t_slope_mode></t_slope_mode>
                        <slope_mode>on</slope_mode>
                        <t_auto_scale></t_auto_scale>
                        <auto_scale>on</auto_scale>
                        <t_auto_scale_opts></t_auto_scale_opts>
                        <auto_scale_opts>2</auto_scale_opts>
                        <t_auto_scale_log></t_auto_scale_log>
                        <auto_scale_log></auto_scale_log>
                        <t_scale_log_units></t_scale_log_units>
                        <scale_log_units></scale_log_units>
                        <t_auto_scale_rigid></t_auto_scale_rigid>
                        <auto_scale_rigid></auto_scale_rigid>
                        <t_upper_limit></t_upper_limit>
                        <upper_limit>100</upper_limit>
                        <t_lower_limit></t_lower_limit>
                        <lower_limit>0</lower_limit>
                        <t_unit_value></t_unit_value>
                        <unit_value></unit_value>
                        <t_unit_exponent_value></t_unit_exponent_value>
                        <unit_exponent_value></unit_exponent_value>
                        <t_unit_length></t_unit_length>
                        <unit_length></unit_length>
                        <t_no_gridfit></t_no_gridfit>
                        <no_gridfit></no_gridfit>
                        <t_alt_y_grid></t_alt_y_grid>
                        <alt_y_grid></alt_y_grid>
                        <t_right_axis></t_right_axis>
                        <right_axis></right_axis>
                        <t_right_axis_label></t_right_axis_label>
                        <right_axis_label></right_axis_label>
                        <t_right_axis_format></t_right_axis_format>
                        <right_axis_format>0</right_axis_format>
                        <t_right_axis_formatter></t_right_axis_formatter>
                        <right_axis_formatter>0</right_axis_formatter>
                        <t_left_axis_formatter></t_left_axis_formatter>
                        <left_axis_formatter>0</left_axis_formatter>
                        <t_auto_padding></t_auto_padding>
                        <auto_padding>on</auto_padding>
                        <t_dynamic_labels></t_dynamic_labels>
                        <dynamic_labels></dynamic_labels>
                        <t_force_rules_legend></t_force_rules_legend>
                        <force_rules_legend></force_rules_legend>
                        <t_tab_width></t_tab_width>
                        <tab_width>40</tab_width>
                        <t_legend_position></t_legend_position>
                        <legend_position>0</legend_position>
                        <t_legend_direction></t_legend_direction>
                        <legend_direction>0</legend_direction>
                </graph>
                <items>
                </items>
                <inputs>
                        <hash_090102592cedd465877bc61ab549df688b0b2a>
                                <name>Processes Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_000102d023fcaa39c9ffe41116492a116ea18e|
hash_00010210b6ba6d1f88815cfac5397d41c426a6|hash_000102eadd16a7d7cb09b2b0555ac3710196ef|
hash_0001025ae0cb568c7ca4ea3e6a0fbef39a7f76</items>
                        </hash_090102592cedd465877bc61ab549df688b0b2a>
                        <hash_0901021d51dbabb200fcea5c4b157129a75410>
                                <name>Legend Color</name>
                                <description></description>
                                <column_name>color_id</column_name>
                                <items>hash_000102d023fcaa39c9ffe41116492a116ea18e</items>
                        </hash_0901021d51dbabb200fcea5c4b157129a75410>
                </inputs>
        </hash_0001029fe8b4da353689d376b99b2ea526cc6b>
        <hash_000102fe5edd777a76d48fc48c11aded5211ef>
                <name>Unix - Load Average</name>
                <graph>
```

```
                        <t_title></t_title>
                        <title>|host_description| - Load Average</title>
                        <t_vertical_label></t_vertical_label>
                        <vertical_label>run que length</vertical_label>
                        <t_image_format_id></t_image_format_id>
                        <image_format_id>1</image_format_id>
                        <t_height></t_height>
                        <height>200</height>
                        <t_width></t_width>
                        <width>700</width>
                        <t_base_value></t_base_value>
                        <base_value>1000</base_value>
                        <t_slope_mode></t_slope_mode>
                        <slope_mode>on</slope_mode>
                        <t_auto_scale></t_auto_scale>
                        <auto_scale>on</auto_scale>
                        <t_auto_scale_opts></t_auto_scale_opts>
                        <auto_scale_opts>2</auto_scale_opts>
                        <t_auto_scale_log></t_auto_scale_log>
                        <auto_scale_log></auto_scale_log>
                        <t_scale_log_units></t_scale_log_units>
                        <scale_log_units></scale_log_units>
                        <t_auto_scale_rigid></t_auto_scale_rigid>
                        <auto_scale_rigid>on</auto_scale_rigid>
                        <t_upper_limit></t_upper_limit>
                        <upper_limit>100</upper_limit>
                        <t_lower_limit></t_lower_limit>
                        <lower_limit>0</lower_limit>
                        <t_unit_value></t_unit_value>
                        <unit_value></unit_value>
                        <t_unit_exponent_value></t_unit_exponent_value>
                        <unit_exponent_value>0</unit_exponent_value>
                        <t_unit_length></t_unit_length>
                        <unit_length></unit_length>
                        <t_no_gridfit></t_no_gridfit>
                        <no_gridfit></no_gridfit>
                        <t_alt_y_grid></t_alt_y_grid>
                        <alt_y_grid></alt_y_grid>
                        <t_right_axis></t_right_axis>
                        <right_axis></right_axis>
                        <t_right_axis_label></t_right_axis_label>
                        <right_axis_label></right_axis_label>
                        <t_right_axis_format></t_right_axis_format>
                        <right_axis_format>0</right_axis_format>
                        <t_right_axis_formatter></t_right_axis_formatter>
                        <right_axis_formatter>0</right_axis_formatter>
                        <t_left_axis_formatter></t_left_axis_formatter>
                        <left_axis_formatter>0</left_axis_formatter>
                        <t_auto_padding></t_auto_padding>
                        <auto_padding>on</auto_padding>
                        <t_dynamic_labels></t_dynamic_labels>
                        <dynamic_labels></dynamic_labels>
                        <t_force_rules_legend></t_force_rules_legend>
                        <force_rules_legend></force_rules_legend>
                        <t_tab_width></t_tab_width>
                        <tab_width>40</tab_width>
                        <t_legend_position></t_legend_position>
                        <legend_position>0</legend_position>
                        <t_legend_direction></t_legend_direction>
                        <legend_direction>0</legend_direction>
                </graph>
                <items>
                </items>
                <inputs>
                        <hash_0901028cb8ed3378abec21a1819ea52dfee6a3>
                                <name>1 Minute Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_0001023702a05e8729c3d48d162208dd072137|
hash_000102862aeaac11049c1d81ae654d66c97dfb|hash_000102124baf8d0260ff62a1e6d21d9c4a4e97|
hash_00010242b9183300db803dfcc0af1fd4b9f122|hash_000102cb9b5207c9f1f1009b813ef56f55702f</items>
                        </hash_0901028cb8ed3378abec21a1819ea52dfee6a3>
                        <hash_0901025dfcaf9fd771deb8c5430bce1562e371>
                                <name>5 Minute Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_000102ab258aac51f4a0160d43e0ba20b9b393|
hash_0001020984a4c622139c0d11b78608160b33e3|hash_00010264469091b7dc7d6b39d3754980fc4a43|
hash_0001028457ef82c0b87cae267536de4b667e09|hash_0001024e8ee8aa54c425e1ec17c83441fcc393</items>
                        </hash_0901025dfcaf9fd771deb8c5430bce1562e371>
```

```
                        <hash_0901026f3cc610315ee58bc8e0b1f272466324>
                                <name>15 Minute Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_0001022438af0fdd01e612291791acb29a711d|
hash_000102752e3ae4e9a82bc3c923b0b236a5ac67|hash_0001028060cd7596e4f6b0831bf73ef8ba531a|
hash_0001024a6c18ff005f2bcb751cdba638757566|hash_000102bfc93ecb2b6784c8525bc00d6448617b</items>
                        </hash_0901026f3cc610315ee58bc8e0b1f272466324>
                </inputs>
        </hash_000102fe5edd777a76d48fc48c11aded5211ef>
        <hash_00010263610139d44d52b195cc375636653ebd>
                <name>Unix - Logged in Users</name>
                <graph>
                        <t_title></t_title>
                        <title>|host_description| - Logged in Users</title>
                        <t_vertical_label></t_vertical_label>
                        <vertical_label>users</vertical_label>
                        <t_image_format_id></t_image_format_id>
                        <image_format_id>1</image_format_id>
                        <t_height></t_height>
                        <height>200</height>
                        <t_width></t_width>
                        <width>700</width>
                        <t_base_value></t_base_value>
                        <base_value>1000</base_value>
                        <t_slope_mode></t_slope_mode>
                        <slope_mode>on</slope_mode>
                        <t_auto_scale></t_auto_scale>
                        <auto_scale>on</auto_scale>
                        <t_auto_scale_opts></t_auto_scale_opts>
                        <auto_scale_opts>2</auto_scale_opts>
                        <t_auto_scale_log></t_auto_scale_log>
                        <auto_scale_log></auto_scale_log>
                        <t_scale_log_units></t_scale_log_units>
                        <scale_log_units></scale_log_units>
                        <t_auto_scale_rigid></t_auto_scale_rigid>
                        <auto_scale_rigid>on</auto_scale_rigid>
                        <t_upper_limit></t_upper_limit>
                        <upper_limit>100</upper_limit>
                        <t_lower_limit></t_lower_limit>
                        <lower_limit>0</lower_limit>
                        <t_unit_value></t_unit_value>
                        <unit_value></unit_value>
                        <t_unit_exponent_value></t_unit_exponent_value>
                        <unit_exponent_value></unit_exponent_value>
                        <t_unit_length></t_unit_length>
                        <unit_length></unit_length>
                        <t_no_gridfit></t_no_gridfit>
                        <no_gridfit></no_gridfit>
                        <t_alt_y_grid></t_alt_y_grid>
                        <alt_y_grid></alt_y_grid>
                        <t_right_axis></t_right_axis>
                        <right_axis></right_axis>
                        <t_right_axis_label></t_right_axis_label>
                        <right_axis_label></right_axis_label>
                        <t_right_axis_format></t_right_axis_format>
                        <right_axis_format>0</right_axis_format>
                        <t_right_axis_formatter></t_right_axis_formatter>
                        <right_axis_formatter></right_axis_formatter>
                        <t_left_axis_formatter></t_left_axis_formatter>
                        <left_axis_formatter></left_axis_formatter>
                        <t_auto_padding></t_auto_padding>
                        <auto_padding>on</auto_padding>
                        <t_dynamic_labels></t_dynamic_labels>
                        <dynamic_labels></dynamic_labels>
                        <t_force_rules_legend></t_force_rules_legend>
                        <force_rules_legend></force_rules_legend>
                        <t_tab_width></t_tab_width>
                        <tab_width>40</tab_width>
                        <t_legend_position></t_legend_position>
                        <legend_position></legend_position>
                        <t_legend_direction></t_legend_direction>
                        <legend_direction></legend_direction>
                </graph>
                <items>
                </items>
                <inputs>
                        <hash_090102b457a982bf46c6760e6ef5f5d06d41fb>
                                <name>Logged in Users Data Source</name>
                                <description></description>
```

```
                                <column_name>task_item_id</column_name>
                                <items>hash_000102ab8392b002d42ad2fa1ba35a99ea51fe|
hash_000102ad192b4b614505bbc36a8afd01d1781d|hash_0001023dcb58ff0357a88a9031cbf27b6f4c60|
hash_000102efbf534120dad14b8ffcec7d71961640|hash_000102a1b9a134b64c84366bedc7e8d4c3a5bb</items>
                        </hash_090102b457a982bf46c6760e6ef5f5d06d41fb>
                        <hash_090102bd4a57adf93c884815b25a8036b67f98>
                                <name>Legend Color</name>
                                <description></description>
                                <column_name>color_id</column_name>
                                <items>hash_000102ab8392b002d42ad2fa1ba35a99ea51fe</items>
                        </hash_090102bd4a57adf93c884815b25a8036b67f98>
                </inputs>
        </hash_00010263610139d44d52b195cc375636653ebd>
        <hash_0001026992ed4df4b44f3d5595386b8298f0ec>
                <name>Linux - Memory Usage</name>
                <graph>
                        <t_title></t_title>
                        <title>|host_description| - Memory Usage</title>
                        <t_vertical_label></t_vertical_label>
                        <vertical_label>bytes</vertical_label>
                        <t_image_format_id></t_image_format_id>
                        <image_format_id>1</image_format_id>
                        <t_height></t_height>
                        <height>200</height>
                        <t_width></t_width>
                        <width>700</width>
                        <t_base_value></t_base_value>
                        <base_value>1000</base_value>
                        <t_slope_mode></t_slope_mode>
                        <slope_mode>on</slope_mode>
                        <t_auto_scale></t_auto_scale>
                        <auto_scale>on</auto_scale>
                        <t_auto_scale_opts></t_auto_scale_opts>
                        <auto_scale_opts>2</auto_scale_opts>
                        <t_auto_scale_log></t_auto_scale_log>
                        <auto_scale_log></auto_scale_log>
                        <t_scale_log_units></t_scale_log_units>
                        <scale_log_units></scale_log_units>
                        <t_auto_scale_rigid></t_auto_scale_rigid>
                        <auto_scale_rigid>on</auto_scale_rigid>
                        <t_upper_limit></t_upper_limit>
                        <upper_limit>100</upper_limit>
                        <t_lower_limit></t_lower_limit>
                        <lower_limit>0</lower_limit>
                        <t_unit_value></t_unit_value>
                        <unit_value></unit_value>
                        <t_unit_exponent_value></t_unit_exponent_value>
                        <unit_exponent_value></unit_exponent_value>
                        <t_unit_length></t_unit_length>
                        <unit_length></unit_length>
                        <t_no_gridfit></t_no_gridfit>
                        <no_gridfit></no_gridfit>
                        <t_alt_y_grid></t_alt_y_grid>
                        <alt_y_grid></alt_y_grid>
                        <t_right_axis></t_right_axis>
                        <right_axis></right_axis>
                        <t_right_axis_label></t_right_axis_label>
                        <right_axis_label></right_axis_label>
                        <t_right_axis_format></t_right_axis_format>
                        <right_axis_format>0</right_axis_format>
                        <t_right_axis_formatter></t_right_axis_formatter>
                        <right_axis_formatter>0</right_axis_formatter>
                        <t_left_axis_formatter></t_left_axis_formatter>
                        <left_axis_formatter>0</left_axis_formatter>
                        <t_auto_padding></t_auto_padding>
                        <auto_padding>on</auto_padding>
                        <t_dynamic_labels></t_dynamic_labels>
                        <dynamic_labels></dynamic_labels>
                        <t_force_rules_legend></t_force_rules_legend>
                        <force_rules_legend></force_rules_legend>
                        <t_tab_width></t_tab_width>
                        <tab_width>40</tab_width>
                        <t_legend_position></t_legend_position>
                        <legend_position>0</legend_position>
                        <t_legend_direction></t_legend_direction>
                        <legend_direction>0</legend_direction>
                </graph>
                <items>
                </items>
                <inputs>
```

```
                        <hash_0901026273c71cdb7ed4ac525cdbcf6180918c>
                                <name>Free Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_000102404665179f589ad7ce64b458bf93a58d|
hash_00010264dd010cd56ccca790f2d640486a5bb7|hash_000102216032db321bbce61c28e318e1610c19|
hash_0001026b871e99f6f37c98004a43df0155f521</items>
                        </hash_0901026273c71cdb7ed4ac525cdbcf6180918c>
                        <hash_0901025e62dbea1db699f1bda04c5863e7864d>
                                <name>Swap Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_0001023e8ad5940852e6d4bf3cefa6680885b4|
hash_000102bb01f635dbb02625115a9640ba6ece90|hash_0001025f16039fc5cef2deaa4dbe10d12a90b8|
hash_000102fa62617953fa551b24d4109c52f6906e</items>
                        </hash_0901025e62dbea1db699f1bda04c5863e7864d>
                </inputs>
        </hash_0001026992ed4df4b44f3d5595386b8298f0ec>
        <hash_0001028e7c8a511652fe4a8e65c69f3d34779d>
                <name>Unix - Available Disk Space</name>
                <graph>
                        <t_title>on</t_title>
                        <title>|host_description| - Available Disk Space</title>
                        <t_vertical_label></t_vertical_label>
                        <vertical_label>bytes</vertical_label>
                        <t_image_format_id></t_image_format_id>
                        <image_format_id>1</image_format_id>
                        <t_height></t_height>
                        <height>200</height>
                        <t_width></t_width>
                        <width>700</width>
                        <t_base_value></t_base_value>
                        <base_value>1024</base_value>
                        <t_slope_mode></t_slope_mode>
                        <slope_mode>on</slope_mode>
                        <t_auto_scale></t_auto_scale>
                        <auto_scale>on</auto_scale>
                        <t_auto_scale_opts></t_auto_scale_opts>
                        <auto_scale_opts>2</auto_scale_opts>
                        <t_auto_scale_log></t_auto_scale_log>
                        <auto_scale_log></auto_scale_log>
                        <t_scale_log_units></t_scale_log_units>
                        <scale_log_units></scale_log_units>
                        <t_auto_scale_rigid></t_auto_scale_rigid>
                        <auto_scale_rigid>on</auto_scale_rigid>
                        <t_upper_limit></t_upper_limit>
                        <upper_limit>100</upper_limit>
                        <t_lower_limit></t_lower_limit>
                        <lower_limit>0</lower_limit>
                        <t_unit_value></t_unit_value>
                        <unit_value></unit_value>
                        <t_unit_exponent_value></t_unit_exponent_value>
                        <unit_exponent_value></unit_exponent_value>
                        <t_unit_length></t_unit_length>
                        <unit_length></unit_length>
                        <t_no_gridfit></t_no_gridfit>
                        <no_gridfit></no_gridfit>
                        <t_alt_y_grid></t_alt_y_grid>
                        <alt_y_grid></alt_y_grid>
                        <t_right_axis></t_right_axis>
                        <right_axis></right_axis>
                        <t_right_axis_label></t_right_axis_label>
                        <right_axis_label></right_axis_label>
                        <t_right_axis_format></t_right_axis_format>
                        <right_axis_format>0</right_axis_format>
                        <t_right_axis_formatter></t_right_axis_formatter>
                        <right_axis_formatter></right_axis_formatter>
                        <t_left_axis_formatter></t_left_axis_formatter>
                        <left_axis_formatter></left_axis_formatter>
                        <t_auto_padding></t_auto_padding>
                        <auto_padding>on</auto_padding>
                        <t_dynamic_labels></t_dynamic_labels>
                        <dynamic_labels></dynamic_labels>
                        <t_force_rules_legend></t_force_rules_legend>
                        <force_rules_legend></force_rules_legend>
                        <t_tab_width></t_tab_width>
                        <tab_width>40</tab_width>
                        <t_legend_position></t_legend_position>
                        <legend_position></legend_position>
                        <t_legend_direction></t_legend_direction>
```

```
                        <legend_direction></legend_direction>
                </graph>
                <items>
                </items>
                <inputs>
                        <hash_090102940beb0f0344e37f4c6cdfc17d2060bc>
                                <name>Available Disk Space Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_0001023343968ae34b1a88fa46213c3e070d95|
hash_00010243c7d573c7ae75f2ce7b8b5f43b8dc88|hash_00010283ae02d7a5034b683ee150a8245aa8a4|
hash_000102b7a591d11b24890b366e82d15f70f474|hash_000102e277320375d05534aa01971d9320fc2e</items>
                        </hash_090102940beb0f0344e37f4c6cdfc17d2060bc>
                        <hash_0901027b0674dd447a9badf0d11bec688028a8>
                                <name>Used Disk Space Data Source</name>
                                <description></description>
                                <column_name>task_item_id</column_name>
                                <items>hash_000102b3d537a4ecdabbcd43313ed3a9ba3243|
hash_000102b8bddc548722b2666a52beb9637890d8|hash_000102b09a9febbd898acec5a0af9296861c5b|
hash_000102b974496fa30d693fafdea2aba4eda440|hash_00010288cce764897070f936dccdc37d620ed9</items>
                        </hash_0901027b0674dd447a9badf0d11bec688028a8>
                </inputs>
        </hash_0001028e7c8a511652fe4a8e65c69f3d34779d>
        <hash_0401028ffa36c1864124b38bcda2ae9bd61f46>
                <name>Unix - Get Mounted Partitions</name>
                <description>Queries a list of mounted partitions on a unix-based host with the</description>
                <xml_path>&lt;path_cacti&gt;/resource/script_queries/unix_disk.xml</xml_path>
                <data_input_id>hash_03010280e9e4c4191a5da189ae26d0e237f015</data_input_id>
                <graphs>
                        <hash_110102a0b3e7b63c2e66f9e1ea24a16ff245fc>
                                <graph_template_id>hash_0001028e7c8a511652fe4a8e65c69f3d34779d</
graph_template_id>
                                <name>Disk Space</name>
                                <rrd>
                                        <item_000>
                                                <snmp_field_name>dskAvailable</snmp_field_name>
                                                <data_template_id>hash_010102e4ac6919d4f6f21ec5b281a1d6ac4d4e</
data_template_id>

<data_template_rrd_id>hash_080102e801aab0a4b1d242b1e2bd49004d48ab</data_template_rrd_id>
                                        </item_000>
                                        <item_001>
                                                <snmp_field_name>dskUsed</snmp_field_name>
                                                <data_template_id>hash_010102e4ac6919d4f6f21ec5b281a1d6ac4d4e</
data_template_id>

<data_template_rrd_id>hash_0801027b94166ab8c3e3374f44f13c8e36c109</data_template_rrd_id>
                                        </item_001>
                                </rrd>
                                <sv_graph>
                                        <hash_120102f21b23df740bc4a2d691d2d7b1b18dba>
                                                <field_name>title</field_name>
                                                <sequence>1</sequence>
                                                <text>|host_description| - Disk Space - |query_dskDevice|</text>
                                        </hash_120102f21b23df740bc4a2d691d2d7b1b18dba>
                                </sv_graph>
                                <sv_data_source>
                                        <hash_130102cb09784ba05e401a3f1450126ed1e395>
                                                <field_name>name</field_name>
                                                <data_template_id>hash_010102e4ac6919d4f6f21ec5b281a1d6ac4d4e</
data_template_id>
                                                <sequence>1</sequence>
                                                <text>|host_description| - Free Space - |query_dskDevice|</text>
                                        </hash_130102cb09784ba05e401a3f1450126ed1e395>
                                </sv_data_source>
                        </hash_110102a0b3e7b63c2e66f9e1ea24a16ff245fc>
                </graphs>
        </hash_0401028ffa36c1864124b38bcda2ae9bd61f46>
        <hash_03010280e9e4c4191a5da189ae26d0e237f015>
                <name>Get Script Data (Indexed)</name>
                <type_id>4</type_id>
                <input_string></input_string>
                <fields>
                        <hash_070102d39556ecad6166701bfb0e28c5a11108>
                                <name>Index Type</name>
                                <update_rra></update_rra>
                                <regexp_match></regexp_match>
                                <allow_nulls></allow_nulls>
                                <type_code>index_type</type_code>
                                <input_output>in</input_output>
```

```
                        <data_name>index_type</data_name>
                    </hash_070102d39556ecad6166701bfb0e28c5a11108>
                    <hash_0701023b7caa46eb809fc238de6ef18b6e10d5>
                        <name>Index Value</name>
                        <update_rra></update_rra>
                        <regexp_match></regexp_match>
                        <allow_nulls></allow_nulls>
                        <type_code>index_value</type_code>
                        <input_output>in</input_output>
                        <data_name>index_value</data_name>
                    </hash_0701023b7caa46eb809fc238de6ef18b6e10d5>
                    <hash_07010274af2e42dc12956c4817c2ef5d9983f9>
                        <name>Output Type ID</name>
                        <update_rra></update_rra>
                        <regexp_match></regexp_match>
                        <allow_nulls></allow_nulls>
                        <type_code>output_type</type_code>
                        <input_output>in</input_output>
                        <data_name>output_type</data_name>
                    </hash_07010274af2e42dc12956c4817c2ef5d9983f9>
                    <hash_0701028ae57f09f787656bf4ac541e8bd12537>
                        <name>Output Value</name>
                        <update_rra>on</update_rra>
                        <regexp_match></regexp_match>
                        <allow_nulls></allow_nulls>
                        <type_code></type_code>
                        <input_output>out</input_output>
                        <data_name>output</data_name>
                    </hash_0701028ae57f09f787656bf4ac541e8bd12537>
                </fields>
            </hash_03010280e9e4c4191a5da189ae26d0e237f015>
</cacti>
```

Copyright (c) 2004-2026 The Cacti Group

Create Packages

A Cacti package is a signed `.xml.gz` file containing template definitions and selected scripts or query resources. This development-branch interface is separate from ordinary XML export. Cacti 1.2 has installer and CLI package import support, but not these package-management screens.

Prepare the author and signing keys

Run `cli/genkey.php --help` with PHP from the Cacti installation to review the options for that version. For a new author identity, an example is:

```
php cli/genkey.php --generate \
  --author="Template Author" \
  --homepage="https://example.org/templates" \
  --email="templates@example.org" \
  --country=US --state=Example --org=Example --unit=Monitoring --days=365
```

Replace the example metadata with the author's details. The tool writes the signing material and author information into the configured PKI directory. `package.key` is private; `package.pem` is the certificate, `package.pub` is the public key, and `package.info` contains author metadata. Allow the packaging process to read what it needs, keep the private key inaccessible through HTTP, and back up the author identity securely. Do not publish the private key.

Reuse the existing identity when publishing an update. `--replace` replaces signing material; it is not an ordinary package-update step. Changing keys also changes the identity recipients must verify and trust. See Package Public Keys.

Build the package

1. Open **Console > Import/Export > Create Packages**.
2. Choose the object type and exact template to export. Select **Include Dependencies** when the recipient needs its referenced templates, input methods, queries, profiles, CDEFs and VDEFs.
3. Complete the description, license, version, class, tags and installation instructions. The package version is the author's version, not the Cacti XML hash compatibility code.
4. Inspect the dependent files and include the scripts/query resources required by the template. Check any dependencies that cannot be discovered from paths in the template; an external agent, credentials or device configuration is not supplied merely by including a script.
5. Submit the package and check for export errors. Download the resulting signed `.xml.gz` file.

The Device Templates screen can also **Download** a package or **Archive** its current definition with a note. Those actions appear when the package public key exists; successful packaging also needs the matching signing material and author information. An archive is a saved package definition, not a backup of collected RRD data.

Verify before distribution

Test the package in a separate Cacti installation using the recipient's release. Follow Import Packages, review the signer and selected files, and confirm the templates, scripts and queries are present. Then create a graph for a device exposing the required measurements and verify collected values. A valid signature does not establish device or firmware compatibility.

Distribute the package with its author version, required Cacti/runtime versions, external setup instructions and independently verifiable signer information. For repository distribution, see Package Repositories.

Copyright (c) 2004-2026 The Cacti Group

Package Repositories

A package repository supplies a `package.manifest` and package files to the **Import Packages** interface. On the development branch, manage locations under **Console > Import/Export > Repositories**. These management screens are separate from Cacti 1.2's installer and CLI package-import workflow.

Add a repository

Create a repository and give it a recognizable name. Choose its type:

Table 1. Repository types

Type	Location and additional settings
GitHub Based	The repository URL and an explicit branch. Use the optional API key only if access requires it.
File Based	The directory containing the manifest and packages on the Cacti server, not a path on your browser's computer.
Direct URL	The base URL serving the manifest and packages.

For a GitHub-based location, use a repository URL such as `https://github.com/Cacti/packages/` and a branch that actually contains the manifest. Do not enter a browser URL for an individual file. The importer looks for `package.manifest` at the selected repository location/branch.

For a direct URL, use the repository's base location so that the manifest and referenced packages resolve beneath it. Use HTTPS where available. For a file location, make the directory readable by the Cacti service account. Keep any repository API credentials private and out of exported templates or examples.

Enable the repository to make it available in **Import Packages**. Mark it as the default only if it should be selected initially. Save and inspect the reachability/manifest result; a reachable host without the manifest is not a usable repository.

Import and maintain

Open Import Packages, select the enabled location, and review available packages, signer identities and the proposed changes. Repository access and package-signature trust are separate decisions. Merely adding a URL does not establish that every package author should be trusted.

The Actions menu can enable, disable, delete, or select a default repository. Disabling a location stops offering it for imports; it does not remove templates or files previously imported from it. If a manifest or download becomes unavailable, check the configured branch/path, credentials and service-account access before retrying. Do not replace a failed signed download with an edited payload.

Package authors should use Create Packages and follow the repository's manifest format and publication procedure when adding a release. Record package versions and required external setup; a repository listing is not a device-certification matrix.

Copyright (c) 2004-2026 The Cacti Group

Package Public Keys

The development branch's **Console > Import/Export > Public Keys** screen manages accepted package-signing identities. A public key verifies a package signature; the corresponding private key remains with the author. Do not confuse these keys with SSH login keys, HTTPS server trust, or SNMP credentials.

Accept a signing identity

When importing an unfamiliar package, compare the signing key with independently obtained author information before accepting it. A package's embedded author name and homepage are claims made by that package, not independent verification. Cacti validates the signature and enforces the implementation's key requirements; a signature alone does not prove that included code is appropriate for your installation.

Acceptance stores the public identity for later imports. Review the accepted keys when an author rotates keys or an identity is no longer trusted. A valid signature made with a different key is a different signing identity even if the displayed author name is unchanged.

Remove or replace trust

Removing an accepted key changes the stored trust decision for future imports. It does not undo templates already imported, remove installed scripts, or revoke an author's access to another system. Cacti also recognizes its configured built-in signing identities; deleting an accepted-key row does not redefine those built-in identities.

If a package is invalid or fails the current key requirements, obtain a corrected release from its author. Do not weaken validation or edit the signed data to make an import pass.

Publish packages

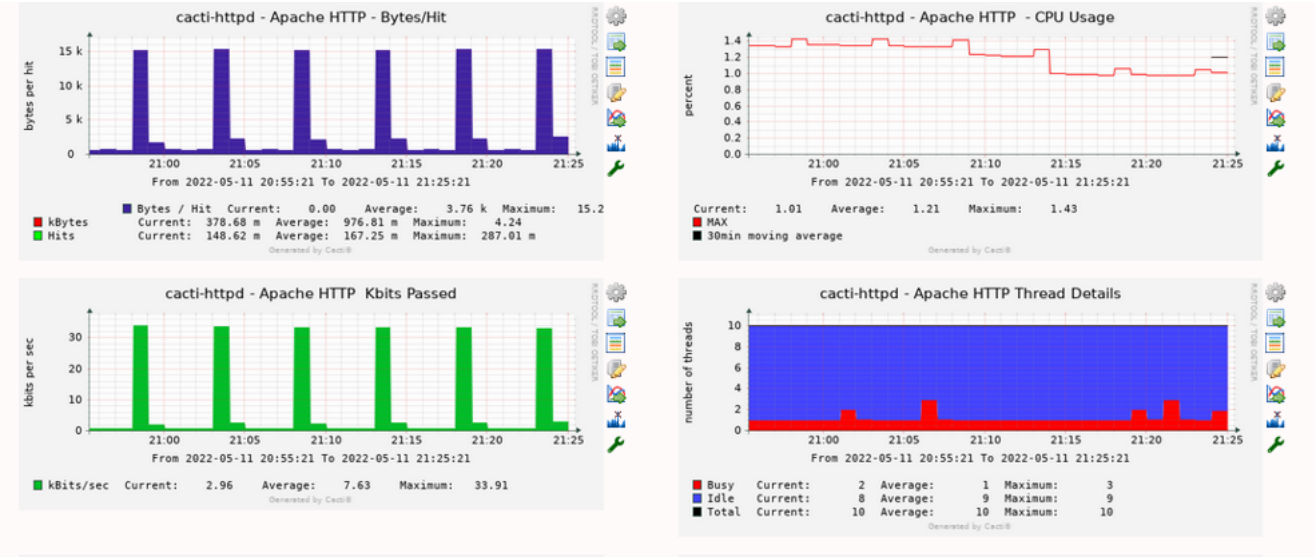
Authors generate signing material with `cli/genkey.php`; see Create Packages. Keep `package.key` private and protect the configured PKI directory from HTTP access. Preserve a secured backup so updates can retain the same identity. Use key replacement deliberately and tell recipients how to verify the new public identity independently.

This screen manages package trust on the development branch. It is not a configuration step for Cacti 1.2's ordinary XML importer. See Import Packages and Template Import for the different formats and workflows.

Copyright (c) 2004-2026 The Cacti Group

Apache Template Config

The Apache template collects server statistics over HTTP from `/server-status?auto`, using Apache's `mod_status` module. These examples allow local access. Check the mod_status documentation for the available statistics and use the endpoint on the server being monitored.



Enterprise Linux configuration

Check whether the installed Apache configuration already loads `mod_status`. Add the module load only if it is missing, then configure the status location in the site's Apache configuration, for example under `/etc/httpd/conf.d`:

```
LoadModule status_module modules/mod_status.so
ExtendedStatus On

<Location /server-status>
    SetHandler server-status
    Require local
</Location>
```

With administrator privileges, test the configuration before reloading a running service:

```
apachectl configtest && systemctl reload httpd
```

Debian configuration

Create `/etc/apache2/conf-available/80-server-status.conf` with:

```
ExtendedStatus On

<Location /server-status>
    SetHandler server-status
    Require local
</Location>
```

Enable the module and configuration with administrator privileges. Reload only if the configuration test succeeds:

```
a2enmod status
a2enconf 80-server-status
apache2ctl configtest && systemctl reload apache2
```

Verify the collection endpoint

Request `http://127.0.0.1/server-status?auto` locally and check for the expected machine-readable statistics. Then verify collection through Cacti under the account and from the host that actually runs the helper.

`Require local` rejects a collector on a different host. For remote collection, configure access for the intended collector addresses and network path rather than making server-status public. The package does not configure the target's Apache service or access controls for you.

Copyright (c) 2004-2026 The Cacti Group

ESXi/VMware Template

The bundled ESXi template polls the ESXi host through SNMP. Configure the host's SNMP agent and allow the Cacti collector to reach its polling port. Run ESXCLI as an authorized administrator through the ESXi Shell or SSH.

For an SNMPv2c configuration, inspect the existing settings first:

```
esxcli system snmp get
```

The following example uses UDP port 161. Replace `REPLACE_WITH_READ_COMMUNITY` with your chosen community before running it. The `--communities` option replaces the existing community list; preserve other required communities when changing an existing configuration.

```
esxcli system snmp set --port 161
esxcli system snmp set --communities 'REPLACE_WITH_READ_COMMUNITY'
esxcli system snmp set --enable true
```

These options are documented in Broadcom's ESXCLI command reference. They configure the agent; they do not prove that routing or firewall rules permit polling. Allow the intended collector to reach the host's SNMP port using the firewall procedure for your ESXi release.

Check the resulting configuration:

```
esxcli system snmp get
```

In Cacti's device settings, select the ESXi template and matching SNMP version, community and port. Verify the returned queries and graphs from the assigned collector. Use your ESXi release's SNMPv3 configuration procedure if required; the community example above does not configure SNMPv3.

Copyright (c) 2004-2026 The Cacti Group

Proxmox - external storage usage

The Linux template is used for Proxmox, it allows you to create disk usage graphs. However, this is not always possible with external storage.

You can check the usage of volumes on external storage with the `pvesm status` command.

Here are instructions on how to get this value into SNMP and use it in Cacti.

We will work with volumes named alletra-lvm and local

```
pvesm status
Name               Type     Status           Total            Used       Available        %
alletra-lvm         lvm     active     10737401856      1927315456      8810086400   17.95%
local               dir     active        98497780        23395264        70052968   23.75%
```

```
apt-get install sudo
```

Run `visudo` and add line

```
Debian-snmp ALL=(ALL) NOPASSWD: /usr/sbin/pvesm status
```

Create a systemd drop-in file to allow snmpd to call the monitoring script. Use a drop-in rather than editing the service file directly, so package updates do not overwrite your changes:

```
mkdir -p /etc/systemd/system/snmpd.service.d
cat > /etc/systemd/system/snmpd.service.d/proxmox-monitoring.conf << 'EOF'
[Service]
# Allow snmpd to execute the monitoring script and call sudo pvesm.
# PrivateTmp is disabled because some Proxmox storage backends access
# /tmp during status queries. Narrow this further if your storage
# backends do not require /tmp access.
PrivateTmp=false
EOF
systemctl daemon-reload
```

> **Note**: Avoid setting `ProtectSystem=off` or `ProtectHome=off` unless you have confirmed that pvesm requires broader filesystem access on your specific Proxmox version. The sudoers entry above already restricts the script to a single command, which limits the blast radius of a compromised snmpd process.

Create new shell script `/usr/local/bin/monitoring.sh`:

```
#!/bin/bash

STORAGE="$1"

/usr/bin/sudo /usr/sbin/pvesm status | /usr/bin/awk -v lv="$STORAGE" '$1==lv {gsub("%","",$NF); print $NF}'
```

Make it executable: `chmod 755 /usr/local/bin/monitoring.sh`

Add these lines to `/etc/snmp/snmpd.conf`. Alletra-lvm and local are from `pvesm status`:

```
extend-sh alletra-lvm /usr/local/bin/monitoring.sh alletra-lvm
extend-sh local /usr/local/bin/monitoring.sh local
```

Restart snmpd:

```
systemctl daemon-reload
systemctl restart snmpd
```

Now we can check if data are in snmp:

```
snmpwalk -v2c -c your_community -On  localhost .1.3.6.1.4.1.8072.1.3
.1.3.6.1.4.1.8072.1.3.2.1.0 = INTEGER: 2
.1.3.6.1.4.1.8072.1.3.2.2.1.2.5.108.111.99.97.108 = STRING: "/usr/local/bin/monitoring.sh"
.1.3.6.1.4.1.8072.1.3.2.2.1.2.11.97.108.108.101.116.114.97.45.108.118.109 = STRING: "/usr/local/bin/
monitoring.sh"
.1.3.6.1.4.1.8072.1.3.2.2.1.3.5.108.111.99.97.108 = STRING: "local"
.1.3.6.1.4.1.8072.1.3.2.2.1.3.11.97.108.108.101.116.114.97.45.108.118.109 = STRING: "alletra-lvm"
.1.3.6.1.4.1.8072.1.3.2.2.1.4.5.108.111.99.97.108 = ""
.1.3.6.1.4.1.8072.1.3.2.2.1.4.11.97.108.108.101.116.114.97.45.108.118.109 = ""
.1.3.6.1.4.1.8072.1.3.2.2.1.5.5.108.111.99.97.108 = INTEGER: 5
.1.3.6.1.4.1.8072.1.3.2.2.1.5.11.97.108.108.101.116.114.97.45.108.118.109 = INTEGER: 5
.1.3.6.1.4.1.8072.1.3.2.2.1.6.5.108.111.99.97.108 = INTEGER: 2
.1.3.6.1.4.1.8072.1.3.2.2.1.6.11.97.108.108.101.116.114.97.45.108.118.109 = INTEGER: 2
.1.3.6.1.4.1.8072.1.3.2.2.1.7.5.108.111.99.97.108 = INTEGER: 1
.1.3.6.1.4.1.8072.1.3.2.2.1.7.11.97.108.108.101.116.114.97.45.108.118.109 = INTEGER: 1
.1.3.6.1.4.1.8072.1.3.2.2.1.20.5.108.111.99.97.108 = INTEGER: 4
.1.3.6.1.4.1.8072.1.3.2.2.1.20.11.97.108.108.101.116.114.97.45.108.118.109 = INTEGER: 4
.1.3.6.1.4.1.8072.1.3.2.2.1.21.5.108.111.99.97.108 = INTEGER: 1
.1.3.6.1.4.1.8072.1.3.2.2.1.21.11.97.108.108.101.116.114.97.45.108.118.109 = INTEGER: 1
.1.3.6.1.4.1.8072.1.3.2.3.1.1.5.108.111.99.97.108 = STRING: "23.75"
.1.3.6.1.4.1.8072.1.3.2.3.1.1.11.97.108.108.101.116.114.97.45.108.118.109 = STRING: "17.95"
.1.3.6.1.4.1.8072.1.3.2.3.1.2.5.108.111.99.97.108 = STRING: "23.75"
.1.3.6.1.4.1.8072.1.3.2.3.1.2.11.97.108.108.101.116.114.97.45.108.118.109 = STRING: "17.95"
.1.3.6.1.4.1.8072.1.3.2.3.1.3.5.108.111.99.97.108 = INTEGER: 1
```

These OIDs contain requested information. OID depends on names used in snmpd.conf:

```
.1.3.6.1.4.1.8072.1.3.2.3.1.1.5.108.111.99.97.108 = STRING: "23.75"
.1.3.6.1.4.1.8072.1.3.2.3.1.1.11.97.108.108.101.116.114.97.45.108.118.109 = STRING: "17.95"
```

We can graph these OIDs with SNMP - generic OID template (Graph-a-Single-SNMP-OID.md)

Copyright (c) 2004-2026 The Cacti Group

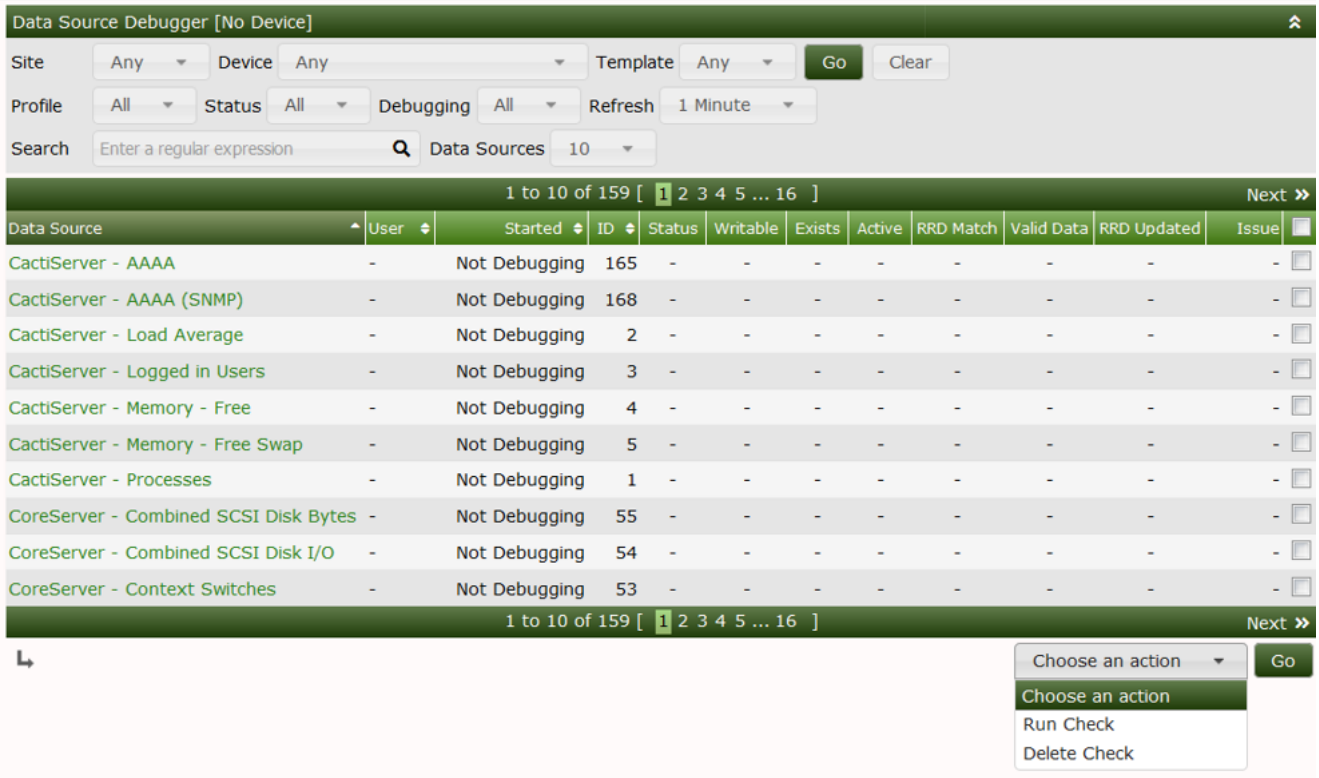
Data Source Debugging

If you are having issues with blank graphs and have done the basic checks such as permissions the Data source debug tool can help troubleshoot what information is coming into Cacti.

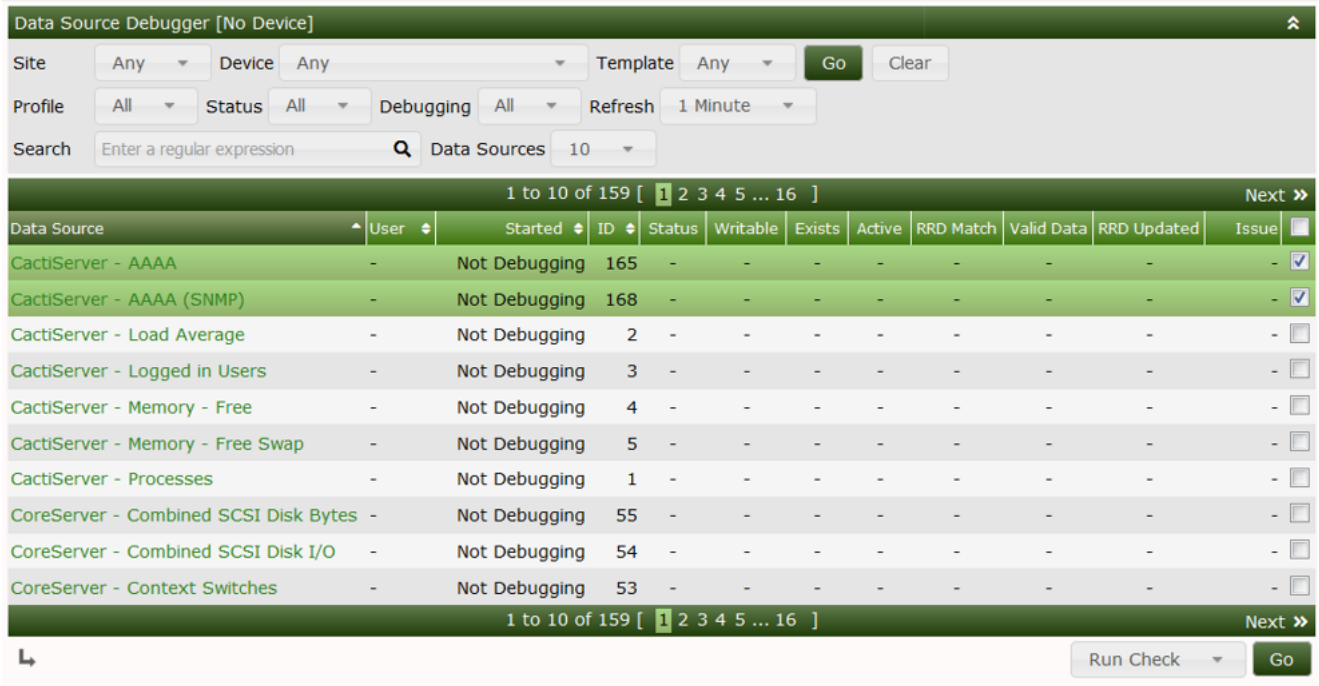
The debug tool will check the data source, graphs and permissions along incoming data and graph settings, such as step count, to ensure the definition matches the profile of the data source amongst other things.

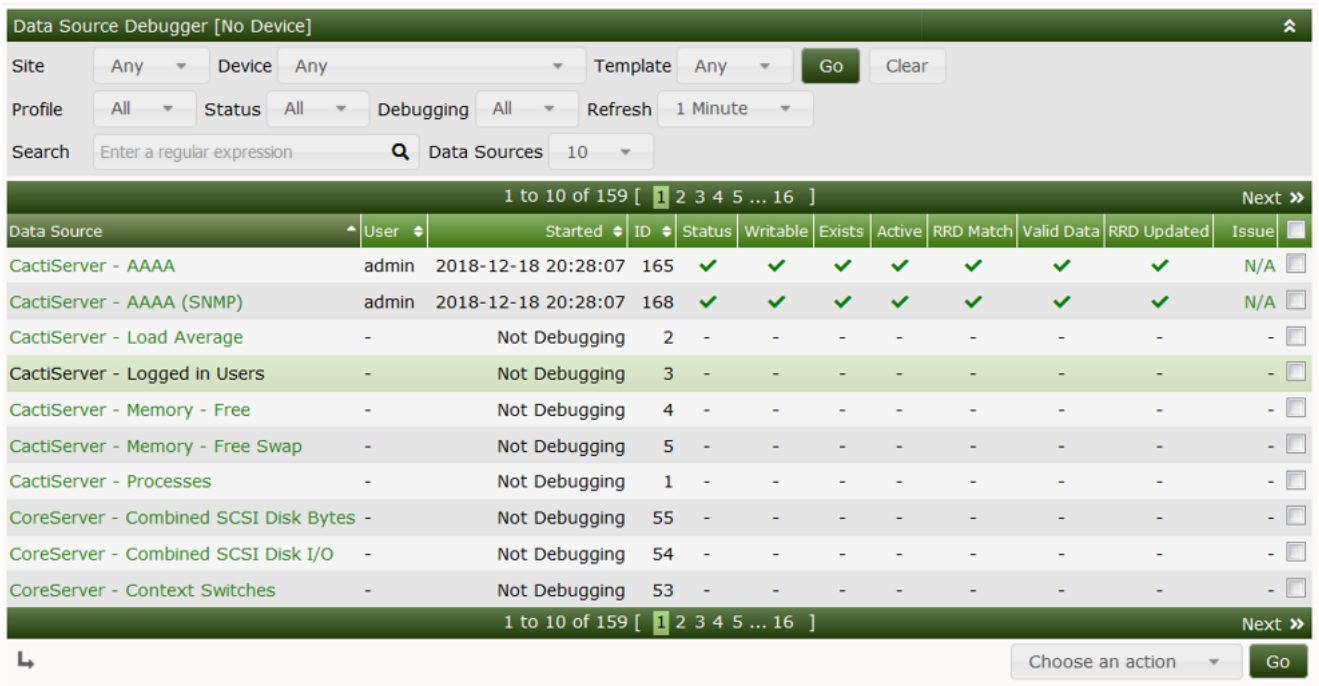
The tool allows you to run a debug on one or many data source. To request debug information, tick all applicable data sources you require information on, then select 'Run Check' in the actions drop down list. This will put all of the selected data sources into debug mode.

For a single data source, you can click the data source name which will also put it into debug mode.



You can see the status of each data source along with all of the checks run against it. Successful checks show a green checkmark, whilst failed checks shows a red X. Depending on the issue a recommendation will be provided for a failed check.





A summary of the checks as well as the properties of the data source can be displayed by clicking on the data source name. This is useful for a single page view of the data source and graph details.

Data Source Debugger		
Check	Value	Results
RRDfile Owner	apache:apache	-
Website runs as	apache	
Poller runs as	apache	
Is RRA Folder writeable by poller?	/var/www/html/cacti/rra	✔
Is RRDfile writeable by poller?	/var/www/html/cacti/rra/cactiserver_output_165.rrd	✔
Does the RRDfile Exist?	1	✔
Is the Data Source set as Active?	on	✔
Did the poller receive valid data?	output: -2.0	
Was the RRDfile updated?		Yes
First Check TimeStamp	1545182941	-
Second Check TimeStamp	1545183002	-
Were we able to convert the title?	CactiServer - AAAA	✔
Does the RRA Profile match the RRDfile structure?		✔
Issue		-

This page also shows what the last values where when last polled. This is also useful for troubleshooting as if the data comes in out of range of your Data Source template, or Graph template, this could point you into the right direction.

If the Web Server has write access to Cacti's RRA folder, if the Data Debug finds and issue that it knows how to correct. It will offer you a link to make that correction.

> **NOTE**: Devices that are down for long periods of time will report as having errors since the RRDfiles associated with them will not be updated as expected for with an available Device.

Copyright (c) 2004-2026 The Cacti Group

The Cacti Log Interface

Summary

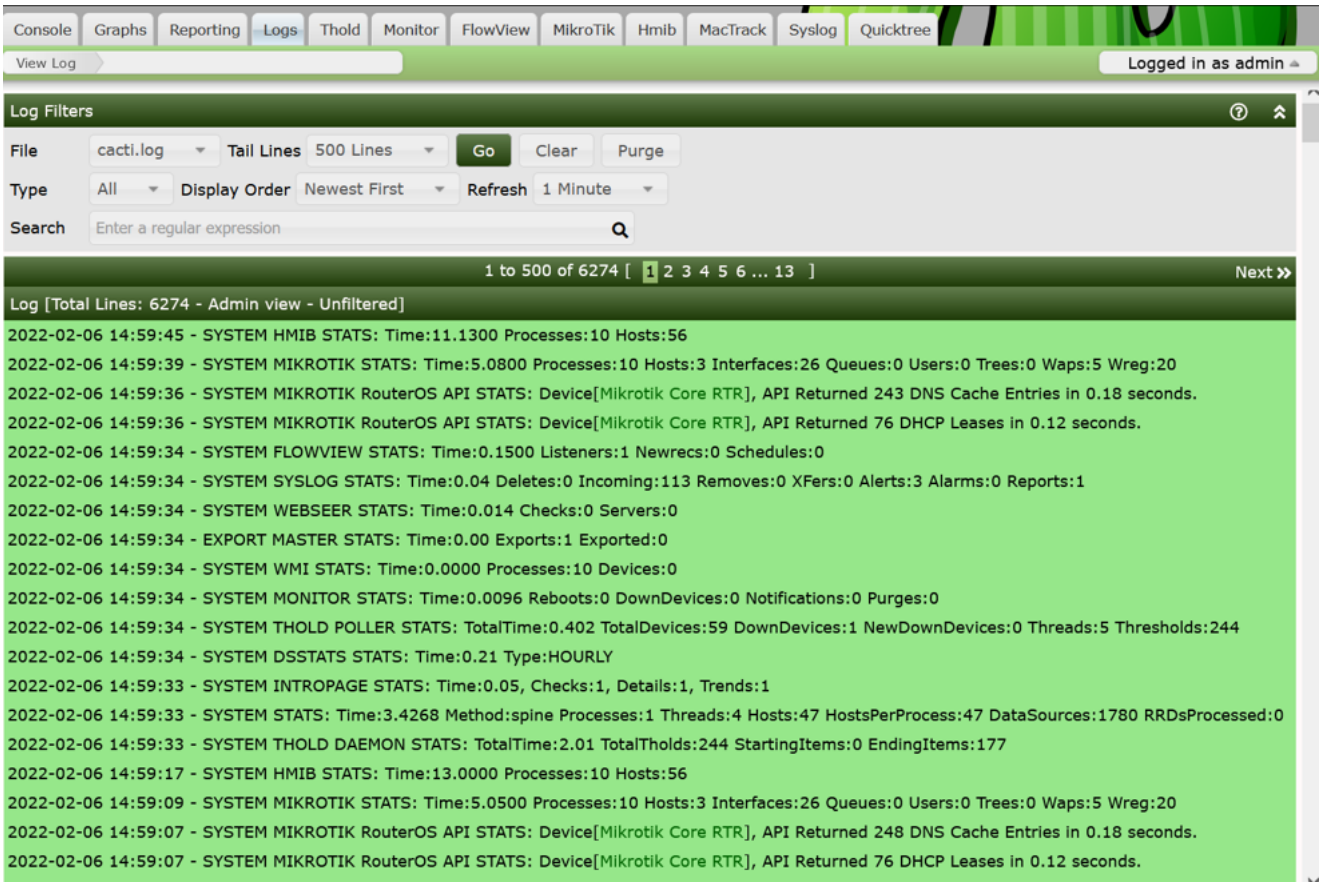
The Cacti Log is where all Cacti statistics, warnings, errors and other log messages are sent. Cacti can optionally log some information to your syslog or event log depending on your OS type.

The Cacti Log is by default ordered from newest event to oldest event, and log rotation follows **Settings > Paths** or the operating-system package configuration. Cacti can rotate logs daily, weekly, or monthly when its own rotation is enabled.

If you have the User Level permission, you can view any available Cacti Standard Output or Standard Error log. If you have the Administrative level access, you may additionally purge information from the logs.

As a Cacti Administrator, you have the ability to filter out some message types from users for example, one message type that people like to filter out is "AUTH" messages that tell you who is logging into the Cacti Web Site. You can edit these removal rules from the Cacti Settings page.

An image of the Cacti Log file can be seen below.



Cacti Log Filters

What we like to see in the Cacti Log is a lot of Green, which means good. By default, WARNINGS are yellow, and ERRORS are red. But there are other types of messages that highlight differently. Generally, you can filter for these message types. The filters include:

- **File** - The Cacti Log file to view. Rotated files are suffixed with the date they were rotated, using the pattern YYYY-MM-DD.
- **Tail Lines** - The number of lines to view from the selected Cacti Log.

- **Types** - The message types to view. These will be explained below.
- **Display Order** - The order to display the Log lines, either *Newest First* or *Oldest First*.
- **Refresh** - How often to Auto-refresh the interface.
- **Search** - A Perl compatible regular expression to search the log with.

The Cacti Log **Types** include the following:

- **Stats** - Anything that includes the word **STATS**
- **Warnings** - Anything that includes the word **WARNING**
- **Warnings++** - Anything of each **WARNING**, **ERROR**, **DEBUG**, and **SQL**. This one is good for debugging, and it includes the errors that matter most to the Cacti Administrator.
- **Errors** - Anything that includes the word **ERROR**
- **Errors++** - Anything of each **ERROR**, **DEBUG**, and **SQL**
- **Debug** - Anything that includes the word **DEBUG**
- **SQL Calls** - Anything that includes the word **SQL**
- **AutoM8** - Anything that includes the word **AUTOM8**. These messages come from Cacti automation of Devices, Graphs, and Trees
- **Non Stats** - Anything that does not include the word **STATS**.
- **Boost** - Anything that contains the word **BOOST**.
- **Device Up/Down** - Any device Up/Down notification, including those that come from the *Thold* or *Monitor* plugins.
- **Recaches** - Any Device Recache Events.
- **Security Issues** - Added in Cacti 1.3. Anything logged by Cacti's security auditing.
- **Threshold** - If the *Thold* plugin is installed, any of its threshold breach and restoration events.

Differences between User and Administrative Views

As previously mentioned, the Cacti log has two permission levels, the User Permission level will not include the *Purge* button, and can have some information hidden from public view whereas the Administrative view includes all Log messages and a *Purge* button.

Copyright (c) 2004-2026 The Cacti Group

Debugging

Use this procedure when graphs show gaps, unknown values (`NaN`), or unexpected zeros. Start with one affected device and data source, and identify the failing stage: device or script, collector, database buffering, RRD storage, or graph rendering. The commands and menu labels follow Cacti development (1.3).

Use Data Source Debugging for the guided checks. Record the device, data-source and collector IDs, the expected polling interval, and the last successful sample. A down device or an idle counter can legitimately produce a different result from a failed collector.

Check Cacti Log File

Read the configured Cacti log, including the affected remote collector's log. The log path can be outside the application directory; the FreeBSD package normally uses `/var/log/cacti`.

Look for timeouts, authentication failures, script errors, database errors and poller overruns around the missing samples. Preserve the original log. Enable additional logging only for the affected device where supported, or briefly for a controlled polling cycle, then restore the previous logging level. Redact credentials and sensitive device details before sharing diagnostics.

An SNMP timeout does not establish that bulk requests are the cause. Check reachability, SNMP version, credentials, access controls and response time first. If the agent fails larger requests, adjust that device's request limits and retest rather than lowering global limits for every device.

Check Basic Data Gathering

Run a data-input script as the actual poller account, using the configured interpreter, arguments and environment. Compare its output with the expected field names and numeric values in the [Data Input Method](Data-Input- Methods.md). For a script-server input, use the PHP Script Server procedure; executing a function-based script directly is not an equivalent test.

For SNMP, query the exact OID with the same version, context and access settings as Cacti. Configure secrets through the supported protected Net-SNMP configuration rather than pasting real credentials into shared commands or logs. Check both the device response and Cacti's configured timeout/retries. A successful ping alone does not confirm that SNMP is available.

Check Cacti's poller

Verify the responsible Data Collector, its selected polling engine, and **Data Collection Enabled**. Check the actual launcher and **Cron/Daemon Interval** as well as **Poller Interval**. FreeBSD uses cron or configured rc.d services; Linux systemd commands do not apply there.

If a manual collector test is needed, first pause the overlapping scheduled work and wait for existing workers to finish. Restrict the test to the affected device. The development PHP collector uses named ID arguments, as shown by its command help:

```
php -q cmd.php --first=42 --last=42 --debug
```

Replace `42` with the device ID and run from the Cacti directory as the poller account. This collects data; it is not a read-only inspection. Restore the original schedule after the test. For Spine, use the installed binary's `--help` and Spine instructions for its version and configuration.

Check MySQL updating

Check the collector log for failed database writes, connection errors or replication problems. Inspect the Poller Cache and confirm that the affected data source is assigned to the expected collector.

Cacti can buffer samples for later processing. With Boost or an RRD cache enabled, collection and disk updates can occur at different times. Check the relevant worker and pending-data state before concluding that a missing direct `rrdtool update` log entry means collection failed.

Do not replay captured SQL writes against the production database as a debug step; they can duplicate or overwrite application state.

Check RRDfile updating

Use the RRD path shown by the data source and its configured collector. Do not assume it is under the web application's `rra` directory. On a default FreeBSD package installation, RRDs are under `/var/db/cacti/rra`.

Check the file's timestamps, data-source names and last update:

```
rrdtool info /path/to/device.rrd
rrdtool lastupdate /path/to/device.rrd
```

Use the configured RRDtool binary and, when applicable, the configured RRD cache connection. Account for pending cache writes before comparing disk timestamps. Do not issue manual production `rrdtool update` commands to fill a gap.

Check RRDfile ownership

The poller or RRD-writing service must be able to traverse the parent directories and write the RRD file. The graph-rendering process needs the access required by the configured storage/cache arrangement. Check those actual service identities rather than assuming they are the same user.

Inspect the affected file and its parent directories. Correct the specific ownership or permissions that differ from the deployment's policy. Avoid blanket ownership changes to application code, credentials, or every RRD file. See the FreeBSD installation guidance for package defaults.

Check RRDfile numbers

Read recent stored values and metadata:

```
rrdtool fetch /path/to/device.rrd AVERAGE --start -1h
rrdtool info /path/to/device.rrd
```

Check data-source type, minimum, maximum, step and heartbeat against the Data Source Template and Data Profile. An out-of-range value, counter reset or gap beyond the heartbeat can produce unknown data even when collection succeeds.

Changing a template does not necessarily rewrite existing RRD definitions. Plan any RRD repair or tuning against the actual files, preserve a consistent backup, and verify the result before applying it to other data sources. Increasing heartbeat can permit interpolation; it does not recover observed measurements that were never collected.

Check `rrdtool graph` statement

In Graph Management, open the affected graph and enable **Graph Debug Mode**. Inspect the command and RRDtool response. Check the `DEF` paths, data-source names, consolidation functions, requested time range and graph permissions.

If reproducing the command as the web account, preserve its binary paths and environment. Replace a `graph -` or `graphv -` output destination with a writable temporary image path so binary image output does not obscure diagnostics.

Duplicate Poller Entries

Inspect Cacti-specific cron entries, service configuration and running workers on the responsible collector. A package may have supplied a schedule in addition to one added manually. Keep one intended launcher; do not disable the system-wide cron service to stop a single Cacti job.

After resolving duplicate launches, verify a complete polling cycle and fresh RRD samples. Preserve the original service and scheduling configuration for rollback. See Settings - Poller.

Not NaN, but 0 (zero) values

Distinguish a valid zero from a script failure converted to zero. Run the input under the poller account, check its exit status and output, and verify external command paths. A successful interactive command as root does not establish that the collector has the same permissions or environment.

For tools that require elevated access, use a narrowly scoped mechanism supported by that tool or its package. Do not make a script or general-purpose interpreter setuid as a troubleshooting shortcut.

Copyright (c) 2004-2026 The Cacti Group

Frequently Asked Questions

These answers follow Cacti's **develop** branch, the unreleased 1.3 series. Menu labels and available options can differ in released versions. For Cacti 1.2, use the 1.2.x manual and your installed version's code.

Using Cacti

Q: How do I create traffic graphs?

A: First, configure read access to the device's SNMP interface counters. Cacti's PHP SNMP functions use the PHP SNMP extension where supported and fall back to Net-SNMP command-line tools for other requests. Configure the installed binaries under **Console > Configuration > Settings > Paths**. Installing the PHP extension does not eliminate every need for these tools. Spine uses its own Net-SNMP support for polling.

1. Open **Console > Management > Devices** and add a device.
2. Enter a description and hostname or IP address, select a suitable **Device Template**, and configure the SNMP version and credentials. For SNMPv3, match the device's username, authentication, privacy, and context settings.
3. Save the device and check the SNMP information displayed on its edit page. Ensure **SNMP - Interface Statistics** is in **Associated Data Queries**. Add the query if the selected template did not supply it. Use **Verbose Query** if it returns no interfaces.
4. Select **Create New Graphs**, then select the interfaces in the interface statistics query and choose the desired graph type. Use a 64-bit counter graph where the device supports it, especially for high-speed interfaces.
5. Click **Create** and complete any additional fields presented. Allow polling to collect samples before expecting traffic data.

See How to Graph Your Network and Data Queries for the full workflow.

General

Q: The website displays PHP source instead of a login page. What is wrong?

A: The web server is serving PHP files without passing them to PHP. Configure its PHP handler, such as PHP-FPM or the Apache PHP module used by your installation. Having a working `php` command in a terminal does not confirm that the web server can execute PHP.

Keep the site inaccessible to other users until this is fixed: source files can contain database credentials. Check the web server and PHP service logs, then follow the installation guide for your operating system and web server.

Q: I have forgotten my local `admin` password. How do I reset it?

A: If another administrator can log in, use User Management to reset the local account. Otherwise, connect to the Cacti database with an account permitted to update `user_auth`:

```
mariadb -u root -p cacti
```

Replace `cacti` with your database name and `newpassword` with a unique temporary password, escaping any SQL quote characters in the value:

```
UPDATE user_auth
SET password = SHA2('newpassword', 256),
    password_change = 'on',
```

```
    must_change_password = 'on',
    lastchange = UNIX_TIMESTAMP()
WHERE username = 'admin' AND realm = 0;
```

The MySQL `mysql` client accepts the same connection arguments. Cacti accepts this SHA-256 bootstrap hash and re-hashes it with its current password algorithm after successful local authentication. The account must then change the temporary password. The reset timestamp also preserves password-expiration behavior. See the local login provider and password verification implementations.

Changing the password does not unlock or enable an account. If login still fails, check its status in User Management or inspect it in the database:

```
SELECT username, realm, enabled, locked, failed_attempts, lastfail
FROM user_auth
WHERE username = 'admin' AND realm = 0;
```

Have an administrator resolve an unintended lockout or disabled status. For LDAP or another external login provider, reset the password at that provider.

Monitoring

Q: Polling does not finish within the configured interval. What can I do?

A: Check the Cacti log for slow or unreachable devices, script timeouts, database errors, and runtime warnings. Compare poller statistics with the configured interval before changing concurrency.

Spine is the multithreaded C alternative to the PHP `cmd.php` collector. `poller.php` still launches the selected collector and coordinates the polling cycle. Install Spine, configure its database access and paths, then select it under **Settings > Poller > Poller Type**.

Tune **Processes** and Spine **Threads** for the affected entry under **Data Collectors**. Threads are per collector process, so consider the total concurrency and database connections. Increase gradually while measuring runtime, CPU, memory, database load, and storage I/O. More threads do not guarantee faster polling; there is no fixed item throughput or CPU ratio that applies to every installation. See Data Collectors.

Q: I changed a setting or template and some graphs stopped updating.

A: Use Data Debug and inspect the affected device's **View Poller Cache** entries. Confirm that the data sources still have valid polling commands, credentials, and OIDs.

If entries are missing or stale, use **Repopulate Poller Cache** on the device edit page or save that device again. For a targeted CLI rebuild, run this from the Cacti directory as the polling account, replacing `123` with the device ID:

```
php -q cli/rebuild_poller_cache.php --host-id=123
```

For a system-wide rebuild, use **Utilities > System Utilities > Rebuild Poller Cache**, or omit `--host-id` from the CLI command. Large installations should use the CLI. Rebuilding the cache does not repair a faulty template or restore lost RRD data. Back up the database, RRD files, and custom scripts and query XML before changing templates. See the cache rebuild utility.

Q: SNMP queries fail. Where should I start?

A: Test from the Data Collector assigned to the device, using the same SNMP version, credentials, port, and context as Cacti. A successful query from your workstation does not prove that the collector can reach the agent.

Check the agent's listening address, firewall rules, access controls, and supported authentication/privacy protocols. For Cacti's PHP paths, verify the PHP SNMP extension and the Net-SNMP binary paths. For Spine, also inspect its configuration and log messages. Use the device's **Verbose Query** output to distinguish an inaccessible agent from an empty or unsupported data query.

Q: How do I configure Net-SNMP's agent for Cacti?

A: Configure `snmpd` on the monitored system. `snmpconf -g basic_setup` can help generate a configuration; confirm which file the installed service actually reads. Paths differ between operating systems and packages.

Use read-only access and restrict requests to your Data Collectors. Prefer SNMPv3 authentication and privacy where supported. If you use SNMPv2c, this example restricts a community to one collector:

```
# Replace both placeholders with your own values.
rocommunity REPLACE_WITH_UNIQUE_COMMUNITY 192.0.2.10
```

Replace the example IP with the collector's source address as seen by the agent. Set matching credentials in Cacti and permit the configured SNMP UDP port through the firewall only from the collectors. Apply the configuration using your platform's service manager. On a systemd installation whose service is named `snmpd`, for example:

```
sudo systemctl restart snmpd
```

Use the Net-SNMP configuration manual for SNMPv3 user creation, views, and any disk-monitoring options required by your templates.

Q: I scheduled network discovery, but nothing is happening.

A: Open the **Log** tab and look for recent `SYSTEM` polling statistics, `AUTOM8` messages, and errors. Missing statistics can indicate a stopped poller, an early failure, or logging settings that suppress or redirect the messages. Check the active log destination and filters before concluding that polling has stopped.

Confirm **Settings > Poller > Data Collection Enabled** is checked. Verify that your chosen scheduler, such as cron or `cactid`, is running under the intended account and can execute PHP and access Cacti. Check its service or scheduler logs for startup and permission errors. Use one scheduler for each collector to avoid duplicate launches.

If polling works, inspect the enabled state, schedule, assigned collector, address range, and SNMP options of the network under **Automation > Networks**. Discovery is launched from the polling cycle and has its own scheduling and process checks. See Automation Networks, Discovered Devices, and the automation implementation.

Graphs

Q: I just installed Cacti and my graphs appear as broken images.

A: A graph cannot read its data until the corresponding RRD files exist. Confirm that polling has run successfully and inspect the Cacti log for RRD creation errors. The normal local storage directory is `rra/`, but use the actual data source path if you configured another storage layout or RRDproxy.

The polling account needs access to create and update those files. The web server needs access to render graphs and, when it performs RRD updates or maintenance, write access as well. See installation hardening for permissions.

To diagnose a polling failure, run from the Cacti directory as the configured polling account, while avoiding an overlapping scheduled run:

```
php -q poller.php --debug
```

Use the PHP CLI binary configured for your installation. If the files exist, open the graph under **Management > Graphs**, select **Turn On Graph Debug Mode**, and inspect the RRDtool command and response. Check the configured RRDtool path and the web server's access to the files named in that command.

Q: My graphs render, but they do not contain data.

A: Start with Data Debug to confirm that polling returns valid values. Inspect the log for SNMP, script, and RRD update errors. Check that the polling account can update the actual RRD path. A manual run as another user can leave files with unsuitable ownership or permissions.

Check the data source type, minimum, maximum, heartbeat, and polling interval. Values outside the RRD data source bounds become Unknown. For counters, the bounds apply to the calculated rate, so use the data source's units when choosing a maximum. A traffic graph may convert byte rates to bits for display. Counter-based sources also need successive samples before a rate is available. See Data Sources and Debugging.

Q: Long `ifAlias` values are truncated in graph titles. Can I change this?

A: Under **Settings > Visual > Graph/Data Source/Data Query Settings**, increase **Data Source Field Length** (`max_data_query_field_length`). Its development-branch default is **40**, not 15. Also check **Maximum Title Length**, whose default is **110**.

If existing names still contain the shortened text, use **Reapply Suggested Names** for the affected graphs or data sources. This regenerates names from their naming rules, so check custom names before applying it in bulk. See setting definitions and query substitution.

Q: A device rebooted and now there is a large spike on its traffic graph.

A: Counter resets or discontinuities can produce an implausible rate. Check the counter type, interface speed, and the maximum for the RRD data source. A realistic maximum lets RRDtool reject impossible calculated rates. Use the source's units, such as bytes per second, rather than the graph's displayed units.

For an existing spike, see Spikekill. Back up the affected RRD files before removing spikes because this changes stored history.

Q: A remote collector times out even though SNMP responses are fast.

A: SNMP is only one part of a remote collection cycle. Check the logs on both collectors and verify connectivity, latency, and database access between the remote collector and the main system. Also inspect replication/recovery messages, database load, script runtimes, and RRD processing.

The **Remote Agent Timeout** setting controls how long the central web server waits for remote device-information requests. It is not the polling cycle's runtime limit. Diagnose the timed-out operation before changing it. See Remote Poller Setup.

Q: RRDtool reports an unknown option or a malformed `COMMENT`. What next?

A: Check **Settings > Paths > RRDtool Binary Path** and **Settings > General > RRDtool Version** against the executable actually used by Cacti. A version mismatch can make Cacti generate unsupported options.

Enable Graph Debug Mode to inspect the failing command. A malformed `COMMENT` can also come from graph text or escaping; a version mismatch is not the only possible cause. Check the exact option or graph item named in the error.

Windows Related

Q: PHP reports `Cannot redeclare title_trim()`. How do I fix it?

A: This means PHP loaded more than one definition of the function. Check for a mixed installation, duplicate Cacti files, and custom PHP include or `auto_prepend_file` settings that load Cacti code twice. Check the PHP configuration used by the failing web or CLI process, since they can differ.

Do not add a Cacti directory to PHP's `include_path` as an installation step. Use a complete, consistent release and follow the Windows installation guide. The error alone does not establish that `include_path` is the cause.

Common Problems

MySQL server has gone away

This means the database connection was lost. Check the database error log and service status for restarts or crashes, then investigate network failures, connection timeouts, and oversized packets. Inspect relevant settings:

```
SHOW GLOBAL VARIABLES LIKE 'wait_timeout';
SHOW GLOBAL VARIABLES LIKE 'max_allowed_packet';
SHOW GLOBAL VARIABLES LIKE 'max_connections';
SHOW GLOBAL STATUS LIKE 'Threads_connected';
SHOW GLOBAL STATUS LIKE 'Max_used_connections';
```

Connection exhaustion normally reports **Too many connections**. Raise `max_connections` only when the logs and workload support that diagnosis and the server has capacity. Increasing it alone does not fix a lost connection. See the MySQL troubleshooting guide.

Cron is out of sync with the Poller Interval

A scheduling warning can indicate that the launcher cadence and Cacti's configuration disagree, or that launches were delayed or missed. Under **Console > Configuration > Settings > Poller**, check **Cron/Daemon Interval** and **Poller Interval**.

Cron/Daemon Interval is the launch frequency, with choices of 60 or 300 seconds. **Poller Interval** is the collection frequency; `poller.php` can perform multiple collection passes during a single launch. Changing the poller interval also requires rebuilding the poller cache; existing RRD profiles need separate consideration. See the interval-change guide.

For cron, the schedule must match **Cron/Daemon Interval**. For example, these are alternative entries in a system cron file. Replace the account and paths with your installation's values:

```
# Every five minutes (300 seconds)
*/5 * * * * cacti /usr/bin/php -q /path/to/cacti/poller.php

# Every minute (60 seconds)
* * * * * cacti /usr/bin/php -q /path/to/cacti/poller.php
```

Install only the matching entry. A per-user crontab omits the `cacti` account field. If using `cactid`, check its service configuration and disable the equivalent cron entry. Also check for runtime overruns and scheduler errors. See the daemon installation instructions.

Poller Runtime exceeded

```
Maximum runtime of ... seconds exceeded. Exiting.
```

This means the poller reached its runtime limit. Check earlier log messages and collector statistics to identify the slow or failing work:

- Spine cannot connect with its configured database credentials or cannot start from the configured binary/configuration paths.
- Unreachable devices, slow SNMP requests, or scripts consume the interval.
- Collector processes or Spine threads are insufficient for the workload, or excessive concurrency overloads the database or system.
- Database connection limits, CPU, memory, storage I/O, or remote connectivity constrain the cycle.

Resolve the measured bottleneck, then confirm subsequent cycles finish within the configured interval. See poller orchestration.

RRDtool reports Unknown Error

Edit the affected graph and select **Turn On Graph Debug Mode**. Inspect both **RRDtool Command** and **RRDtool Says**. If necessary, run the displayed command as the web server account, using the same executable, paths, and environment. On Linux, an administrator can use `sudo -u` to select that account.

Graph commands can write binary image data to standard output. When copying one that uses `rrdtool graph -` or `graphv -`, change the output destination from `-` to a writable temporary image path so diagnostics remain readable.

On Windows, reproduce the failure under the relevant application-pool or scheduled-task identity. These accounts can have different permissions, PHP configurations, and environment variables; an interactive administrator's successful command does not prove that the service account can run it.

Source References

Cacti behavior in this page was checked against development commit `883b334de`. The links below pin the relevant implementation so that setting names, defaults, and authentication behavior can be checked when updating this FAQ. Also see SNMP method selection, device actions, collector concurrency, and RRD creation and graph generation.

Copyright (c) 2004-2026 The Cacti Group

Known Issues

Use the issue tracker and release notes to check whether a problem affects **your installed version**. This page points to those maintained sources; it is not a complete list of open defects.

Check existing reports

- Open Cacti issues include reported problems and proposed improvements. An open report is not confirmation that every release is affected.
- Cacti releases describe published versions. Check the changelog included with your version and the target release of any linked fix. A fix merged into `develop` may not be available in a released maintenance version.
- Security advisories describe published vulnerabilities, affected versions, and available fixes.
- For a plugin problem, use the issue tracker and release notes of that Cacti plugin's repository.

Search both open and closed issues using the error message, affected feature, and version. Before applying a workaround, confirm that it matches your release and environment. Back up the database, RRD files, and configuration before an upgrade or a change that can alter stored data.

Troubleshoot or report a problem

For installation and usage questions, search the Cacti Forums. For missing measurements or blank graphs, start with data source debugging and the Cacti log.

If you find a reproducible defect, follow the contribution guide. Include the Cacti and Spine versions, operating system, PHP and database versions, steps to reproduce, expected behavior, and relevant logs with credentials and sensitive information removed.

Report suspected vulnerabilities privately using the Cacti security policy. Do not put credentials or exploit details in a public issue.

Copyright (c) 2004-2026 The Cacti Group

Spikekill

Warning

Spikekill is designed to overwrite specific data points in one or more RRD files. You should use great caution when running Spikekill, and you should always ensure you have proper backups from which to restore your RRDfiles if Spikekill performs a modification that you did not expect.

Overview

Spikekill is a tool used to remove spikes in a graph. Spikes can appear in a graph after a device reboots, or when you switch from 32-bit to 64-bit interface counters on a device. Spikekill works by statistically analyzing the data contained inside an RRDfile, and overwriting specific data points. It offers 4 methods by which to analyze the data and selectively overwrite data points:

1. *Standard Deviation*

   Calculates the standard deviation of the data, and overwrites data points that are *N* times higher than the standard deviation.

2. *Variance Average*

   Calculates the average value of the data, and overwrites data points that are *N* percent higher than the average.

3. *GapFill*

   Finds gaps in the data (missing data points) and also finds data points that are *N* percent higher than the average, and overwrites them, but only does so inside the specified time range.

4. *Float*

   Overwrites all data points that are within a specified time range.

How to use

GUI

Spikekill is easily run on a graph with a couple mouse clicks in the Cacti GUI:

1. On Cacti's main `Graph` tab, click the Spikekill icon next to a graph.
2. In the drop-down menu that appears, review your current settings by hovering on `Settings` and reviewing each item, making changes if desired.
3. Run Spikekill by choosing one of the four methods available in the drop-down menu. Spikekill runs immediately, and the graph may be modified, depending on your settings. The graph is refreshed when Spikekill is finished.

CLI

Spikekill also has a flexible command line interface. The following example will help you get started.

```
shell>php cli/removespikes.php --help
```

Settings

Spikekill requires certain values to successfully calculate which data points to overwrite. The following settings are customizable:

1. *Replacement Method*

   When Spikekill identifies data points to overwrite, the value of *Replacement Method* represents the data that will be written in place of each data point.

   When *Replacement Method* is set to `NaN`, running Spikekill using `GapFill` or `Float` methods will perform no modifications. Please be aware of Issue #3673, where 'Last Known Good' is always used, regardless of setting.

2. *Standard Deviations*

   When using the Standard Deviation method, the value of *Standard Deviations* is the coefficient which determines how many times above the standard deviation the data must be to be considered a spike.

3. *Variance Percentage*

   When using the Variance Average method, the value of *Variance Percentage* is the coefficient which determines how much higher than the average the data must be to be considered a spike.

4. *Variance Outliers*

   When using the Variance Average method, the highest *N* values and the lowest *N* values are considered *outliers*. These outliers are ignored when calculating the average of the data.

5. *Kills Per RRA*

   Spikekill will limit the number of data points that it overwrites to the value of *Kills Per RRA*. For reference, a single RRDfile can contain multiple *data sources* and multiple *archives*. Therefore, the total number of data points that could be overwritten in a single RRDfile will be determined by the formula:

   ```
   (number_of_RRAs) x (number_of_data_sources) x (Kills_Per_RRA)
   ```

   This only applies when using the `Standard Deviation` or `Variance Average` methods, and does not apply when using the `GapFill` or `Float` methods.

The above settings are maintatined individually for each Cacti user. The default values can be changed globally in the Cacti settings, found at Configuration > Settings > Spikes. A user can change her or his individual settings by selecting different choices in the Spikekill menu which appears next to a graph.

Detailed Operation

Independence between each RRA:DS pair

When Spikekill runs, it analyzes each data source (DS) of each round-robin archive (RRA) independently. During analysis, it calculates values for `average` and `standard deviation` for each RRA:DS pair.

To understand this better, consider the following example.

- You have a round-robin database (RRD) file that stores traffic levels on a router interface.
- You have two DSes, which are `traffic_in` and `traffic_out`.
- You also have selected three profiles, which are 1 minute for 7 days, 15 minutes for 5 weeks, and 1 hour for 3 years.
- Lastly, you have three consolidation functions (CF), which are `average`, `min`, and `max`.

In this example, you have 9 RRAs (3 profiles multiplied by 3 CFs). Therefore, Spikekill will calculate 18 averages and 18 standard deviation values (9 RRAs multiplied by 2 DSes). Data points for each RRA:DS pair will be compared against their respective `average` and `standard deviation` values.

Search order

Spikekill searches for spikes in the same order as `rrddump` exports data, which is sequentially by RRA, from oldest to newest data points within each RRA. Data points are overwritten if they exceed the maximum allowed threshold or if they do not reach the minimum required threshold.

Spikekill will cease overwriting data points for any individual RRA when it has reached the value of *Kills Per RRA*, counting all DSes of that RRA together. It will then begin the process again on the next RRA.

This means that for any given RRA, older spikes will always be overwritten before newer spikes. However, newer spikes in one RRA can still be overwritten before older spikes in a different RRA, since the thresholds are different per RRA:DS pair.

Copyright (c) 2004-2026 The Cacti Group

Settings

Cacti has numerous settings that can be changed from the Web Interface. Cacti, from it's inception desired to have as many settings as possible to be configurable without having to goto the Command Line to make them. While, for the most part this is still true, there are several settings that must be configured in the config.php which we will cover in a separate sub-section.

The standard Settings sub-tabs include:

- General - This is where Cacti holds a general collection of settings that every Cacti administrator should become familiar with.
- Paths - Mostly, this sub-tab holds all settings that the Cacti administrator has control of. It also includes some settings around RRDfile maintenance.
- Device Defaults - This is where you can establish presets that every new Cacti **Device** gets by default, through **Automation**, the CLI or though the Web Interface.
- Poller - Though most **Poller** settings have been moved to the **Data Collectors**, some are still managed at the system level including the poller type to use, script server and a number of additional settings.
- Data - This is where you can enable the RRDproxy and **Data Source Statistics** collection. The **Data Source Statistics** module allows for the saving of key metrics that are important in dash-boarding plugins in Cacti.
- Visual - This is where you can set some preset like **Graph** and **Data Source** title lengths, and drop-down sizes these Visual settings are global in nature.
- Authentication - This is where Cacti defines how **Users** are managed. The available methods depend on the Cacti version.

  They include:

  - Local Users - Authentication and Authorization are handled in Cacti. Passwords are stored as hashes.
  - Basic Users - Authentication outside of Cacti, for example SAML2, TACACS+, NIS, etc.). Authorization remains in Cacti
  - LDAP Users - Authentication outside of Cacti, either LDAP or Active Directory. Authorization remains in Cacti
  - Login Providers - LDAP, Active Directory, SAML2, and OpenID Connect on the unreleased development branch.
  - Multiple Domains - Cacti 1.2 directory domains with their own authorization templates.
- Performance - This is where Cacti's `Boost` functionality was moved to. Cacti's `Boost` module allows Cacti to scale to systems with over a million **Graphs** and **Data Sources**.
- Spikes - This is a built-in tool for killing spikes on graphs in a scheduled fashion. It also holds system level spike killing defaults.
- Mail/Reporting/DNS - This is where Cacti saves settings associated with these three services, for example Simple Mail Transfer Protocol (SMTP) settings used for general notification across all of Cacti.

In addition, to the base settings, Cacti Plugins can extend settings for their own purposes. When a Plugin has few settings that it uses, the Cacti Group has recommended a sub-tab called `Misc` for those Plugins for their purposes. Most Plugins conform to that guideline.

Copyright (c) 2004-2026 The Cacti Group

General Settings

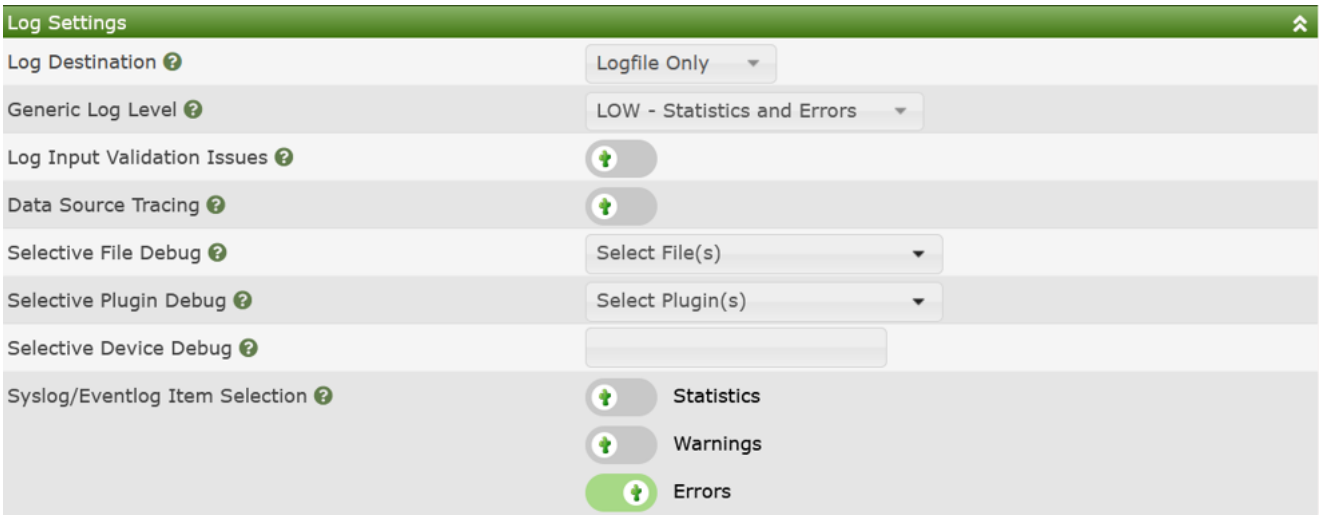
General Settings in Cacti consist of the following categories:

- **Log Settings** - This is where you setup Cacti's logging
- **Internationalization (i18n)** - Settings around how Cacti handles languages other than English
- **Other Settings** - Other not object settings in Cacti
- **Site Security** - Web Site Security settings such as `Content-Security-Policy`, use of external JavaScript API's for plugins like Google Maps, forcing https, etc.
- **Automation** - Miscellaneous settings around Cacti's Automation
- **Graph Template Defaults** - Defaults for **Graph Templates**

Each of these settings will be documented below.

Log Settings

Cacti has the ability to log at various levels for various files. It has the ability to log to both the standard cacti.log file, as well as sending some notifications to Syslog for forwarding to centralized logging servers. The image below show what this settings sub-tab section looks like:



Those settings include:

- **Log Destination** - Where to place Cacti log output. Options include:
  - **Logfile Only** - Only send log data to the Cacti log file
  - **Logfile and Syslog** - Send log data to both the Cacti log file and the System log facility
  - **Syslog Only** - Send select log data to just the System log facility

  This setting is quite self explanatory, when it comes to what type of messages are sent to the System log facility, see the `Syslog/Eventlog Item Selection` section below.

- **General Log Level** - This is the level of logging that Cacti will use. Options include:
  - **NONE** - Don't perform any logging, not recommended.
  - **LOW** - Statistics and Errors
  - **MEDIUM** - Statistics, Errors and Results
  - **HIGH** - Statistics, Errors, Results and Major I/O Events
  - **DEBUG** - Statistics, Errors, Results, I/O and Program Flow

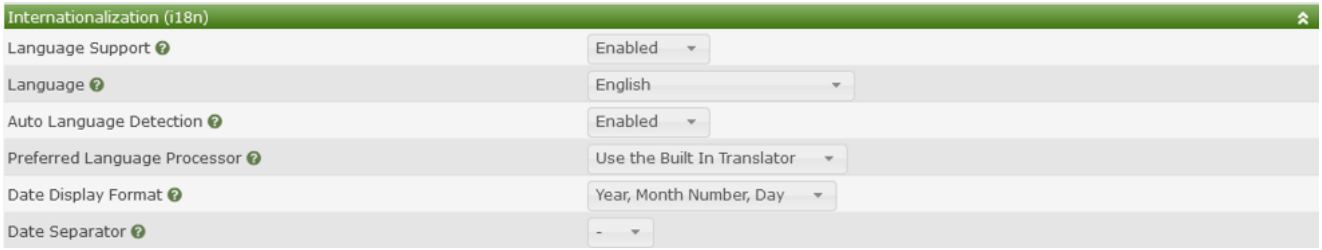
    - **DEVEL** - Developer level messages, mainly **SQL Queries**

  Select the log level that works best for your logging needs. Be advised that **DEBUG** generates a large amount of log data. **DEVEL** should only log **SQL Queries**.

- **Log Input Validation Issues** - This is a Developer option to log the use of request variables that have not been validated through Cacti's validation functions.
- **Data Source Tracing** - This is a debug options for logging of program flow around the creation of **Data Sources**.
- **Selective File Debug** - A Multi-select setting that allows you to place any Cacti file to be placed into debug mode. This prevents the cacti.log file from being flooded with messages as is common in a general debug context.
- **Selective Plugin Debug** - For Plugins, the same level of debug that is available for core Cacti files are available for Plugin files as well.
- **Device Debug** - This is a comma delimited list of Cacti `device_id` that debugging of **Data Collection** services will be enabled for.
- **Syslog/Eventlog Item Selection** - In order to not saturate the System logging facility, Cacti will only forward some events as defined by the various check-boxes.

Internationalization (i18n)

The image below shows system level Internationalization settings for Cacti, commonly referred to as i18n.



Those settings include:

- **Language Support** - Whether or not to allow selecting other languages for Cacti
- **Language** - The default language to use at login time, or by default when the user has not specified a language.
- **Auto Language Detection** - Whether or not Cacti should read the language from the browser and use that language by default.
- **Preferred Language Processor** - Some PHP distributions doe not come with the php-gettext module. In those cases, the various Linux distributions come with alternate GetText processors. You can select the one that works best for your environment here.
- **Date Display Format** - The default Date/Time display format.
- **Date Separator** - The default Data/Time separator character.

Other Settings

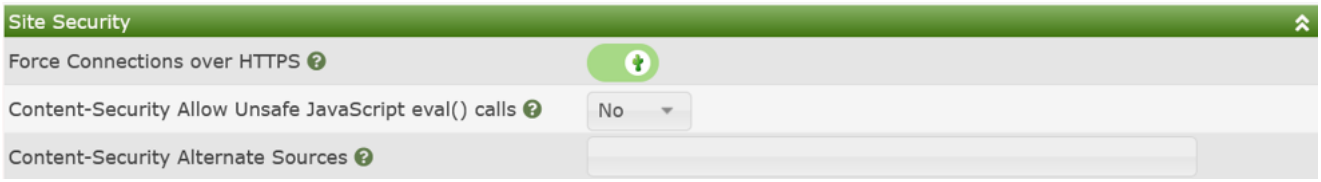
These are additional settings for Cacti that fall into various categories, but that done have enough entries to justify a separate section. The image below shows that sub-section.

Those settings include:

- **Has Graphs/Data Sources Checked** - If from the Console, you enter a pages that has the various **Has** options, if checked here, the default will be to check that setting.
- **RRDtool Version** - Selecting the correct RRDtool Version will unlock features expected in modern graphing frameworks that leverage SVG (Canvas) rendering.
- **Graph Permission Method** - The development branch offers Permissive, Restrictive, Device Based, and Graph Template Based policies. These combine direct graph access with device and template policies in different ways. See User Management and verify the affected user's effective permissions before changing the global method.
- **Graph/Data Source Creation Method** - If you wish to only create **Graphs** from **Graph Templates** choose the **Simple** method. The **Advanced** method should only be used by people who wish to define individual **Graphs** with their own settings.
- **Show Form/Setting Help Inline** - If this option is checked, you will be able to read all form/setting help inline. Otherwise, you will receive a pop-up to access form help.
- **Delete Verification** - For simple object removal, things that can be removed by clicking and 'X' on the page, whether or not you wish to confirm those deletes.
- **Data Source Preservation Preset** - Whether or not you wish to preserve **Data Sources** by default, or remove them when deleting **Graphs**. In all cases, Cacti will not allow the removal of a **Data Source** if there are **Graphs** using the **Data Source**. However, some **Data Sources** are managed externally to Cacti, and removing those **Data Sources** by default may not be desired.
- **Graph Auto Unlock** - If when editing a Graph you wish to always unlock it by default, select this option. Not that this does have performance implications that may make the User Interface slower on vary large systems where devices have tens of thousands of **Data Sources**.
- **Hide Cacti Dashboard** - This is an option used by some plugin developers who replace the default Cacti `Console` with their own contents, for example the Intropage plugin.
- **Enable Drag-N-Drop** - Certain Cacti pages allow Drag-N-Drop. If you with to use it, check that option.

Site Security

The image below shows Site Security settings for Cacti.



Those settings include:

- **Force Connections over HTTPS** - Even if the web site allows http, redirect their browsers to HTTPS.
- **Content-Security Allow Unsafe JavaScript eval() calls** - Allow plugins to use the JavaScript `eval()` function call, which is historically unsafe. However, some older plugins require it.
- **Content-Security Alternate Sources** - If for some reason your Cacti site requires content from a CDN or a third party site, you can include a space delimited list of URLs that will be able to be traversed for Cacti content.

Automation Settings

The image below shows general Automation Settings for Cacti.

Those settings include:

- **Enable Automatic Graph Creation** - If a Cacti **Device** has Non **Data Query Graph Templates** associated with it, create those **Graphs** automatically when adding a new **Device**. Additionally, if you have **Graph Rules** apply those at **Device** creation time and save time.
- **Enable Automatic Tree Item Creation** - If you have Cacti **Tree Rules** apply then at **Device** creation and save time.
- **Automation Notification To Email** - When Network Automation finishes, send a notification to this Email address.
- **Automation Notification From Name** - The `From Name` on the Automation Email.
- **Automation From Email** - The `From Email` such as noreply@cacti.net that will be used for Automation notifications.

Graph Template Defaults

This is where Cacti obtains defaults for **Graph Templates**.

- **Graph Template Image Format** - When creating a new **Graph Template** what `Image Format` should be used. There is either:

SVG - Scalable Vector Graphics, which retain detail when scaled. **PNG** - Portable Network Graphics, a raster image format.

- **Graph Template Height** - The height in pixels of the Graph Canvas for new **Graph Templates**
- **Graph Template Width** - The width in pixels of the Graph Canvas for new **Graph Templates**

Copyright (c) 2004-2026 The Cacti Group

Path Settings

Cacti's Path Settings include mainly the locations of core binaries that Cacti must call in order to operate correctly. Generally, these binaries are included in the Web Servers path, but being able to override these settings is important, especially for debugging issues. Cacti's Paths sub-tab is broken into the following sub-sections:

- **Required Tool Paths** - Paths that are essential for Cacti's operation
- **Logging** - Settings associated with Cacti's logging facilities including log rotation
- **Alternate Poller Path** - Locations for Cacti's spine binary and spine config file if not in the established default locations
- **RRD Cleaner** - Setting for Cacti's RRDfile disposal of removed RRDfile when Data Sources are removed.

When using Cacti's **Remote Data Collectors** or **Remote Pollers** the settings that refer for a `File Path`, or `Directory Path` will be local to the **Data Collector**. Therefore, you can have one set of paths for your Main Cacti site, and another set of paths for your Remote Poller, say in the case that they are on Windows.

Required Tools Paths

The image below shows Cacti's **Required Tool Paths** settings.

Those settings include:

snmpwalk Binary Path - This is the path to the `snmpwalk` binary.

snmpget Binary Path - This is the path to the `snmpget` binary.

snmpbulkwalk Binary Path - This is the path to the `snmpbulkwalk` binary. This binary path is essential if you use SNMPv2 of SNMPv3 versions where **Data Queries** perform bulkwalks by default to increase performance.

snmpgetnext Binary Path - This is the path to the `snmpgetnext` binary.

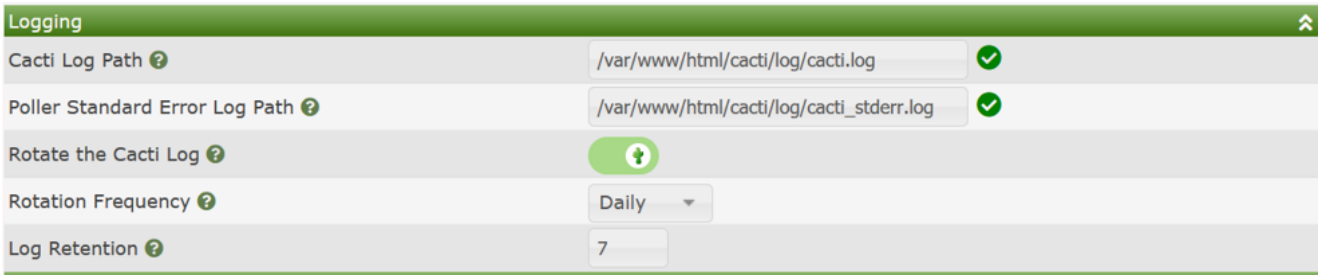
snmptrap Binary Path - This is the path to the `snmptrap` binary. This binary is important for SNMP Notification and Trap generation.

RRDtool Binary Path - The location of the RRDtool binary that matches your RRDtool version.

PHP Binary Path - The full path to the PHP CLI binary used by the poller, for example `/usr/bin/php`. Verify its version and extensions separately from the PHP runtime used by the web server.

Logging

The next set of settings define where Cacti's Standard Output and Standard Error files are to be placed, and how they should be rotated, and for how long they should be retained. You can see those settings from the image below.



Those settings include:

Cacti Log Path - The location of Cacti's Standard Output (stdout) log file.

Poller Standard Error Log Path - For Cacti's **Data Collector** functions only, the location of the Standard Error (stderr) log file.

Rotate the Cacti Log - Determines if Cacti or the package maintainer rotates the Cacti log.

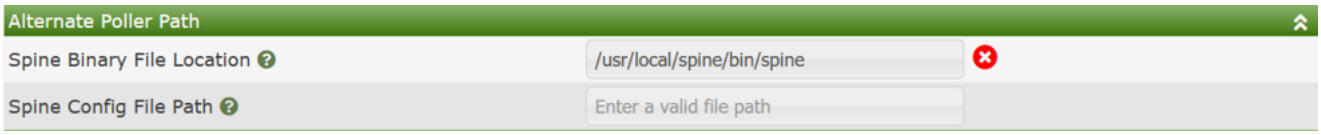
Rotation Frequency - Should the Log files be rotated `Daily` or `Weekly` or `Monthly`

Log Retention - How many generations of Log files should be maintained. Older files will be removed automatically.

> **NOTE**: Some package maintainers control log rotation and therefore the log rotation settings may be missing from certain version of Cacti if you install through a packaging tool such as `apt-get`, `yum` or `dnf`

Alternate Poller Path

This subsection holds information on how to configure Cacti's `spine` binary path. Since `spine` is written in C, it is compiled by the Cacti Administrator to match the system's environment. The primary benefit of `spine` is that it is multi-threaded and highly scalable, capable of polling tens of thousands of data sources per minute. The image below shows this subsection.

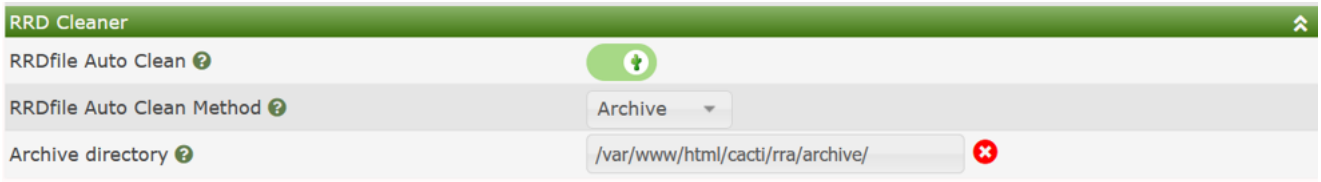


Those settings include:

- **Spine Binary File Location** - The path to the spine binary.
- **Spine Config File Path** - This setting is optional. If blank, Cacti will search the following locations for a `spine.conf` file.
    - **pwd** - The same directory as spine
    - **../etc/** - One sibling `etc` directory of the `spine` binary directory.
    - **/etc/** - The systems /etc/ directory.

RRD Cleaner

This setting defines how Cacti manages RRDfiles from **Data Sources** that have been removed. By default, Cacti does not manage these files, and leaves it to the Cacti Administrator to pro-actively manage them. The RRD Cleaner settings can be seen in the image below. Note that unless you enable `RRDfile Auto Clean`, you will not see the other settings by default.



Those settings include:

- **RRDfile Auto Clean** - Denotes that you wish Cacti to manage RRDfiles from removed **Data Sources** potentially automatically.
- **RRDfile Auto Clean Method** - This is the method to perform cleanup. Options include:
  - **Delete** - Simply remote the files once the **Data Source** is removed
  - **Archive** - Archive the RRDfile to an Archive directory once it's removed.
- **Archive directory** - The Archive directory to move old RRDfile to.

Copyright (c) 2004-2026 The Cacti Group

Device Default Settings

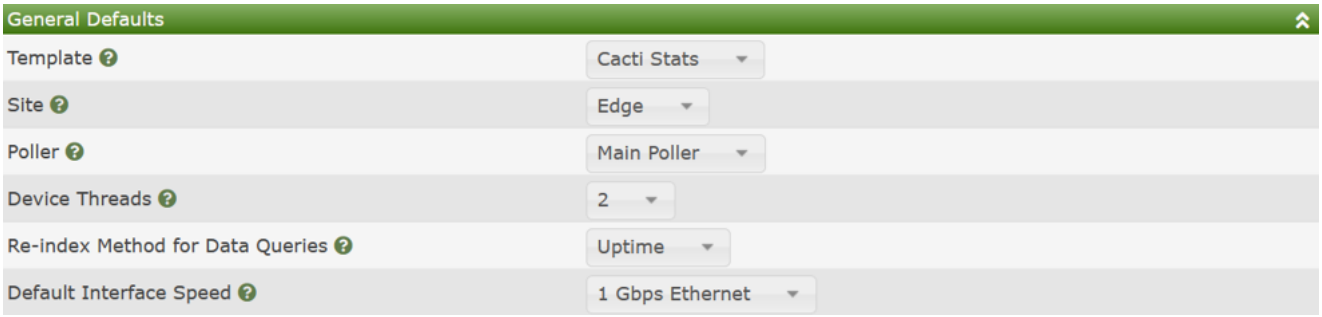
Cacti has multiple ways to create **Devices**. Those methods include:

- **Manual** - Create Devices one at a time through the User Interface
- **CLI** - Through Cacti's Command Line Interfaces (CLI's)
- **Automation** - Through Cacti's Network Discovery

When using any of these methods, Cacti needs a way to apply a default for a new **Device** if the user does not specify on, be it any of the three methods above. That is what this settings sub-tab if for.

General Defaults

In the image below, you can see the **General Defaults**. These settings cover a number of **Device** attributes.



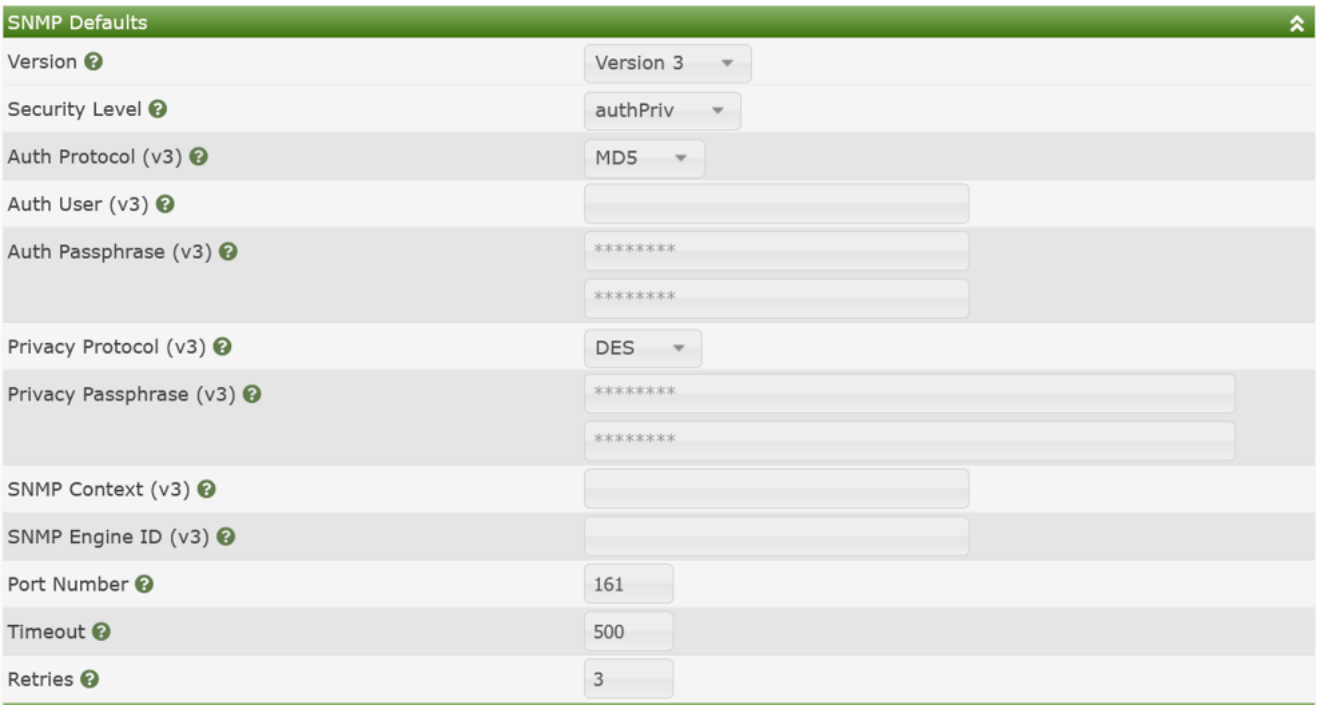
Those settings include:

- **Template** - The default **Device Template**. You might select `Net-SNMP` device if you are monitoring all Linux devices for example.
- **Site** - The default physical location of the **Device**
- **Poller** - The default **Data Collector** to assign new **Devices** to
- **Device Threads** - The default number of Device Threads to assign to a new **Device**
- **Re-index Method for Data Queries** - The default **Re-index" method for new **Device Data Queries**
- **Default Interface Speed** - If the **Device** does not include `ifHighSpeed` what should the maximum speed of the interface be set to.

> **NOTE**: **Device Threads** is only applicable when using the spine binary

SNMP Defaults

Similar to the `General Defaults`, the `SNMP Defaults` control what SNMP settings to use for new **Devices* if the user does not specify any. The image below shows the settings when using first SNMP v2 and then SNMP v3. If you are using SNMP v1 or v2, you will have much less required settings.



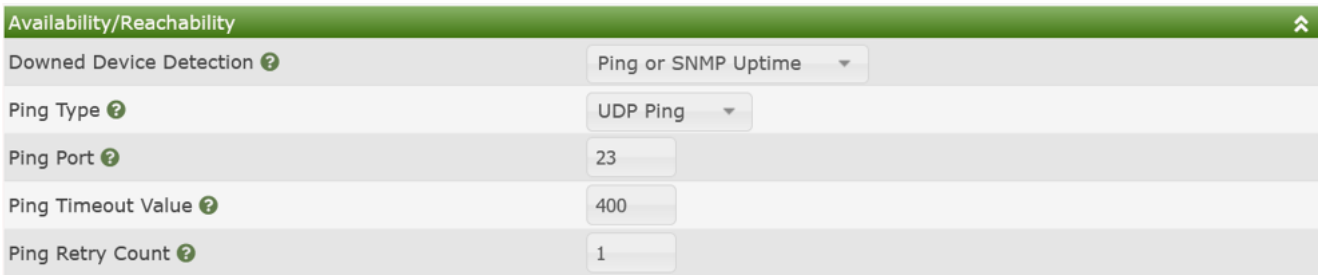
Those settings include:

- **Version** - The default SNMP version to query using (v1, v2, v3)
- **Community** - The default SNMP password to access the device (SNMP v1, v2)
- **Port Number** - The default SNMP port number to poll (SNMP v1, v2, v3)
- **Timeout** - The default SNMP Timeout (SNMP v1, v2, v3)
- **Retries** - The default number of times to retry a query if SNMP times out (SNMP v1, v2, v3)
- **Security Level** - The default Security Level (SNMP v3)
- **Auth Protocol** - The default Authentication Portocol to use (SNMP v3)
- **Auth User** - The default Authentication User to use (SNMP v3)
- **Auth Password** - The default Authentication password to use (SNMP v3)
- **Privacy Protocol** - The default Privacy Protocol to use (SNMP v3)
- **Privacy Pass-phrase** - The default Privacy Pass-phrase to use (SNMP v3)
- **SNMP Context** - The default SNMP Context to use (SNMP v3)
- **SNMP Engine ID** - The default SNMP Engine ID to use (SNMP v3)

Availability / Reachability

Cacti's `Availability / Reachability` define how to perform initial checks prior to polling to see if the **Device** is alive or not. Attempting to gather large amounts of data from a **Device** that is down, is counter-productive.

Availability / Reachability is generally performned using a `ping`. These `pings` come in many formats, and Cacti supports all of the reasonable ones. In the image below, you can see the settings for one of the combination cases. Depending on your `ping` method, you may receive less options.



Those options include:

- **Downed Device Detection** - The method by which Cacti should `ping` the **Device**, SNMP `pings` involve querying a specific OID. Options include:
  - **None** - Only used for devices having their data collected through a back-end agent
  - **Ping and SNMP uptime** - Perform a TCP/UDP/ICMP ping and then check uptime through SNMP. Both methods must work for the device to be **Up**
  - **Ping or SNMP Uptime** - If either method works, consider the **Device Up**
  - **SNMP Uptime** - Only check the SNMP sysUptimeInstance of the **Device**
  - **SNMP Desc** - Only check the SNMP sysDescription of the **Device**
  - **SNMP getNext** - Only perform a basic `getnext` query for the **Device**
  - **Ping** - Either TCP, UDP, or ICMP pings
- **Ping Type** - When using an options that leverages **Ping** above, which method
- **Ping Port** - When using TCP or UDP ping, the port to ping
- **Ping Timeout Value** - The ping timeout value in Milliseconds
- **Ping Retry Count** - The number of times to retry the ping if it fails

Up / Down Settings

The Up / Down Settings define how many polling cycles that a Device must not respond to be considered truly `Down`, and when at device is returning from `Down`, how many polling cycles to keep a device in `Recovering` state before returning it to `Up`. The image below shows the two settings for the **Device**. There is really no reason to explain these two settings in any additional detail.

Copyright (c) 2004-2026 The Cacti Group

Data Settings

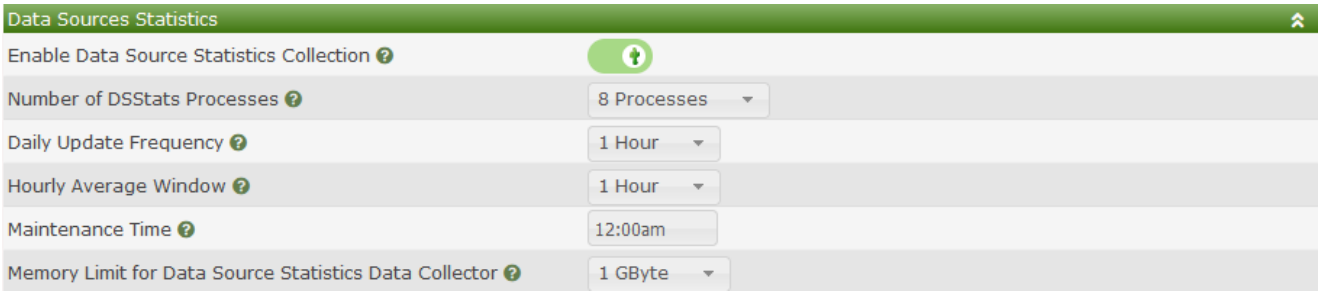
The Cacti Data settings includes settings for gathering statistical data about Cacti's **Data Sources** and about settings around how Cacti stores it's **Round Robin Archives** Time Series Databases (TSDB), otherwise known as RRDfiles.

The sections below cover statistics, file checks, and storage.

Data Source Statistics

Cacti has the ability to store peak and average data for all your **Data Sources**. You can then use that data to easily build dashboards using third party plugins including plugins like Superlinks from Howard Jones.

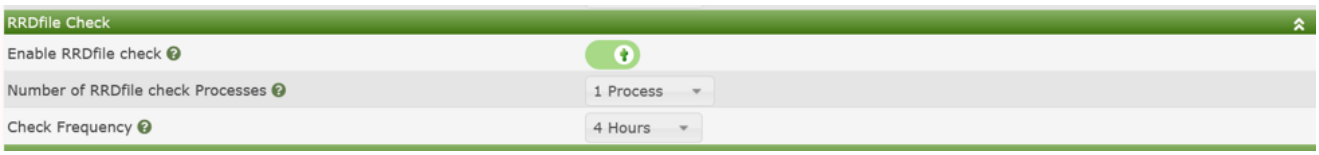
Below you will find an image of the default settings.



Those settings include:

- **Enable Data Source Statistics Collection** - This is setting will either enable or disable the statistics collection.
- **Number of DSStats Processes** - For very large Cacti installs the **Data Source Statistics** can take quite a long time to complete. This setting controls the number of concurrent statistics workers. You can allocate as many as 20 concurrent processes to perform all the updates.
- **Daily Update Frequency** - This setting determines how often daily averages and peaks should be calculated. You can set it to run hourly, up to every 6 hours, or after Cacti's boost process completes.
- **Hourly Average Window** - This is a floating time window where Cacti holds onto all samples for determining the hourly averages for your **Data Sources**. Monitor its size and database load. The current development schema uses InnoDB; older installations may have a different engine and capacity limit.
- **Maintenance Time** - This is the time in the day that Cacti will calculate Monthly and Yearly numbers and perform table maintenance.
- **Memory Limit for Data Source Statistics Data Collector** - This setting controls the peak memory that the statistics poller will be allowed to use. Most systems can work fine using the default 1GByte setting.

RRDfile Checker



RRDfile checker scans RRDfiles and try to find problems like stale values.

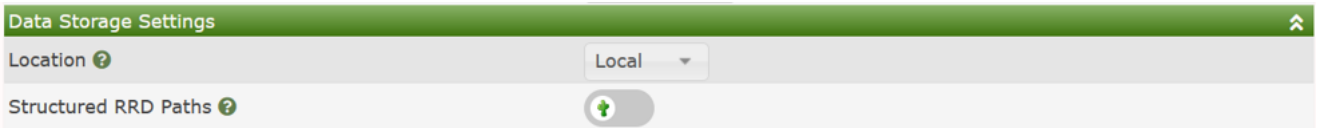
Those settings include:

- **Enable RRDfile Checker** - This setting will either enable or disable the RRD checker.

- **Number of RRDfile Checker Processes** - For very large Cacti installs the **RRDfile Checker** can take quite a long time to complete. More workers increase concurrency and database and storage load.
- **Check Frequency** - This setting determines how often checks should be executed. You can set it to run hourly, up to 24 hours, or after Cacti's boost process completes.

Data Storage Settings

These settings determine how you store your RRDfiles. This sub-section has two layouts as showing in the images below.

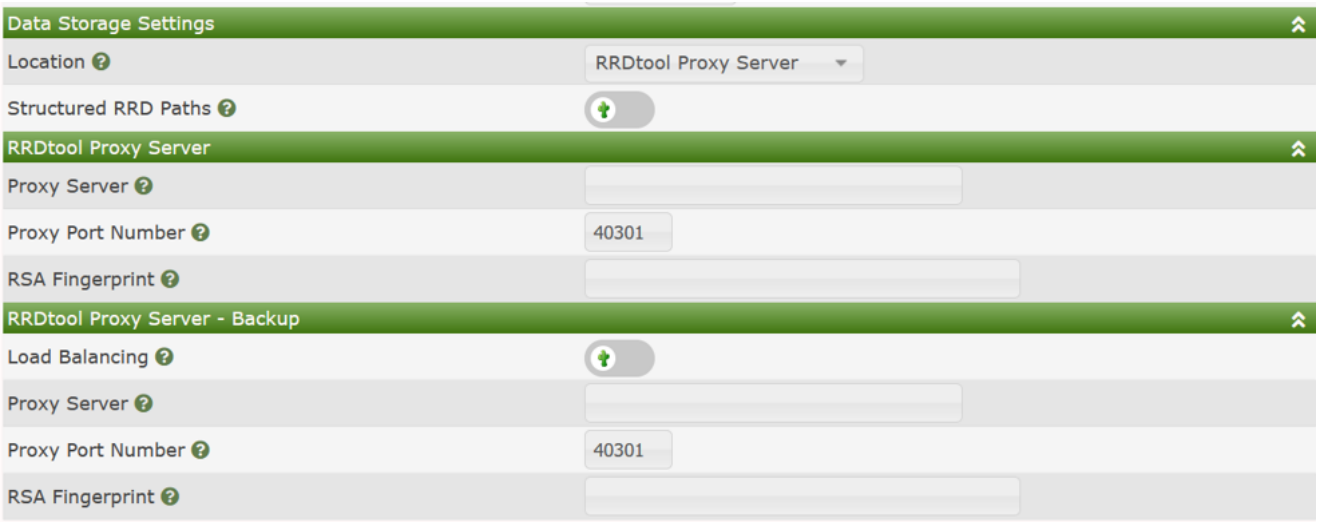


The settings above include:

- **Location** - Either Local or RRDtool Proxy Server. The example above shows the local storage option.
- **Structured RRD Paths** - In very large systems having a flat directory structure has severe performance consequences. However, with the Structured Paths option, Cacti will separate RRDfiles by Device ID and use the Data Source ID as the RRDfile name.

When changing structured-path settings on an existing installation, back up files and database definitions, then use `php cli/structure_rra_paths.php --help` to plan the move. Run the migration after changing the settings, with appropriate file ownership and a controlled collection window. The development branch supports device, query, and hashed directory patterns; changing the pattern requires another migration.

RRD Proxy stores files through a separate service. Configure and verify the proxy deployment independently; Cacti's connection settings do not configure storage replication or guarantee failover of that service.



The settings above include:

- **Proxy Server** - The IP or DNS name of the primary RRDfile proxy server.
- **Proxy Port Number** - The port number to connect to in order to communicate with the RRD Proxy.
- **RSA Fingerprint** - The fingerprint of the current public RSA key the proxy is using. This is required to establish a trusted connection to the RRD Proxy service.

On the development branch, **Load Balancing** selects between the configured primary and backup endpoints and can try the other endpoint after a connection failure. Both must serve the intended data. Confirm compatibility and replication on the proxy side before enabling this option.

- **Load Balancing** - Whether calls to the RRD Proxy should be load balanced.
- **Proxy Server** - The secondary Proxy server IP or DNS name.
- **Proxy Port Number** - The port number on the secondary RRD Proxy to talk to.
- **RSA Fingerprint** - The fingerprint of the current public RSA key the proxy is using. This is required to establish a trusted connection to the RRD Proxy service.

Copyright (c) 2004-2026 The Cacti Group

Visual Settings

Overview

Cacti's Visual settings page is where you can go to globally control the behavior of Cacti's GUI. Settings like the default Theme, the maximum allowed size of a Graph Title, auto-complete settings, as well as a myriad of other settings can be found here.

The Visual page is broken down into several sub-sections. We will cover them and the meaning of the various settings here.

> **NOTE**: This is a popular page for Plugin developers to augment. Therefore, if you have installed Plugins, your Settings page may look different than the default page.

Theme, Table, Tree, and Filter Settings

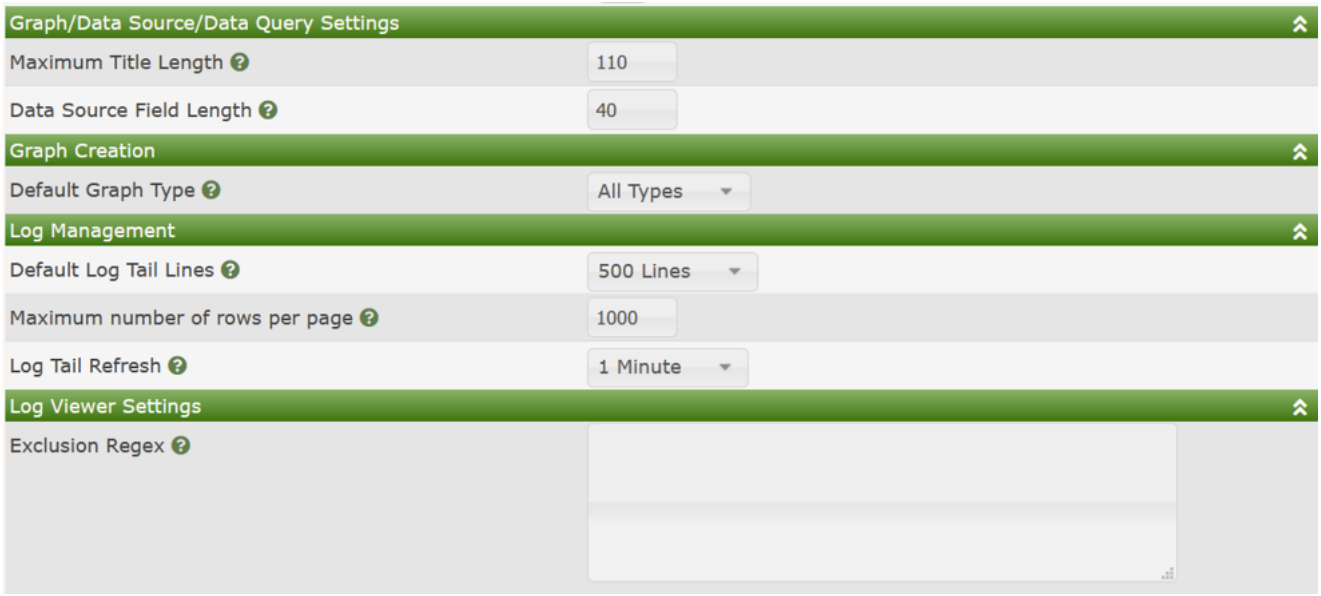
The image below shows the default layout for these sub-sections.

Those settings include:

- **Theme** - The default theme that users logging into Cacti receive
- **Rows Per Page** - For all standard Cacti tables, the number of rows to show per page.
- **Auto-complete Enabled** - Many drop-downs do not scale well with thousands of items. In those cases, Cacti has enabled a drop-down auto-complete feature most large installs should leverage it. However, if you are a small install you can choose to disable auto-complete.
- **Auto-complete Rows** - The maximum rows to show in an auto-complete response.
- **Minimum Tree Width** - The minimum width of the Tree panel in Cacti
- **Maximum Tree Width** - The maximum width of the Tree panel in Cacti. Above this width, tree items will scroll into the panel.
- **Strip Domains from Device Drop-downs** - Strip domain names from Device drop-downs to save horizontal filter space.

Graph, Data Source, Data Query, Log Settings

The image below shows the default layout for these sub-sections.

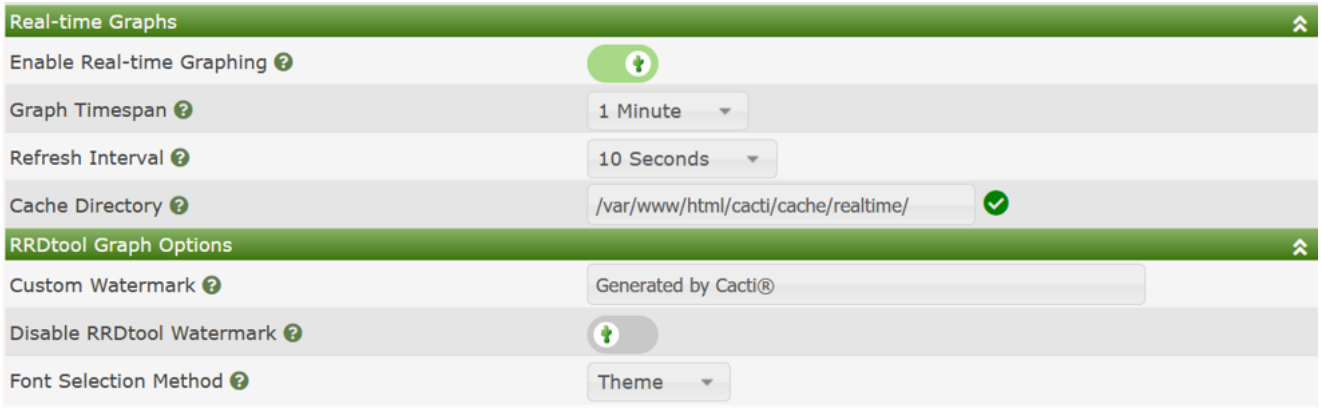


Those settings include:

- **Maximum Title Length** - The maximum length of a Graph or Data Source title. Titles above this length will be clipped.
- **Data Source Field Length** - When applying |query_*| replacement values, the maximum supported length of those variables. Variables longer than this will be clipped.
- **Default Graph Type** - When on the **New Graphs** page, which **Graph Type** filter should be the default. With **Device Templates** with many **Data Queries** or **Graph Templates**, this page can become very long. Allowing you to select your preferred **Graph Type** makes navigating the page easier.
- **Default Log Tail Lines** - When viewing the Cacti Log, how many lines should appear per page.
- **Maximum number of rows per page** - The maximum number of regular expression search matches to display, when searching.
- **Log Tail Refresh** - The frequency that the log pages should auto-refresh.
- **Exclusion Reg-ex** - This setting allows you to exclude certain log messages by regular expression from normal users. Expressions that include login log entries are common here to mask the login accounts of users.

Realtime and RRDtool Graph Options

The image below shows the default layout for these sub-sections.



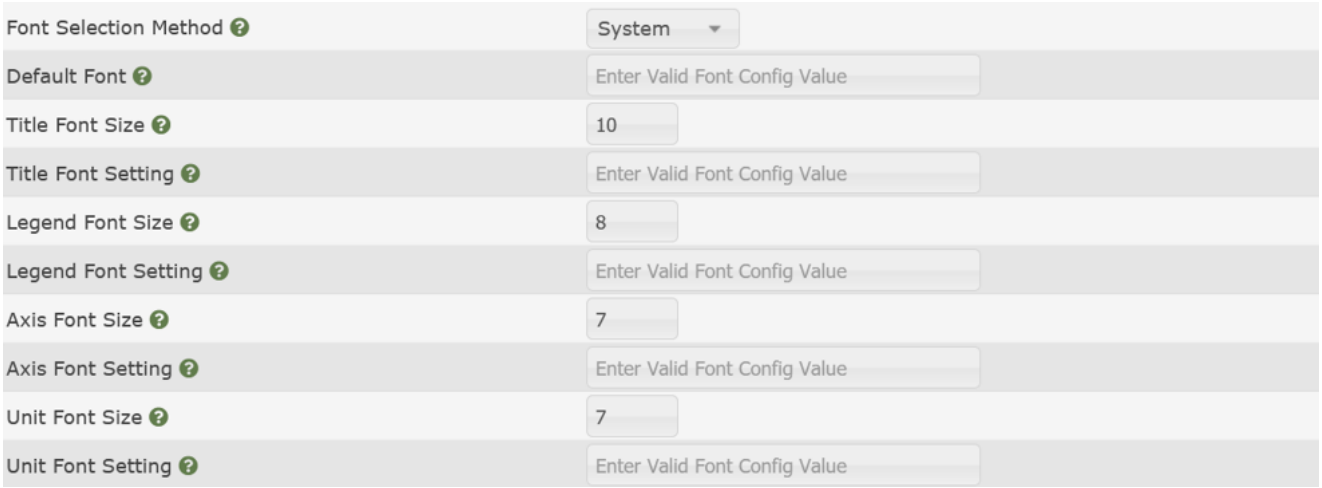
Those settings include:

- **Enable Real-time Graphing** - Allow Realtime graphs to be used in Cacti.
- **Graph Time-span** - The default system level `Graph Timespan` to use when a user opens a Realtime **Graph** for the first time.
- **Refresh Interval** - The default system level `Refresh Interval` for the Realtime interface when a user first opens a Realtime **Graph**.
- **Cacti Directory** - Where should Cacti cache Realtime PNG's and RRDfiles.
- **Custom Watermark** - A watermark that will appear on every Cacti **Graph**
- **Disable RRDtool Watermark** - Every RRDtool Graph includes a vertical watermark. This option allows you to disable that.
- **Font Selection Method** - This allows the system administrator to control, at a global level, how various text elements on RRDtool **Graphs** appear. The settings include: Theme, or System. When choosing `System`, your settings page will expand where you will then have additional control.

> **NOTE**: The RRDtool Watermark control requires a recent version of RRDtool, such as the 1.7 release. If you attempt to use this setting and your **Graphs** break, then upgrade RRDtool.

Font Selection Options

The image below shows the available Font Selection options when you override the Theme settings.



The sizes are in pixels, and the Font Settings can either be a True Type Font file path, or a valid Pango font-config value. For more information on Pango font-config values, see your operating systems documentation. However, you can normally see all the fonts installed on your Linux system by issuing the following command `fc-list`.

Copyright (c) 2004-2026 The Cacti Group

Performance Settings (Boost)

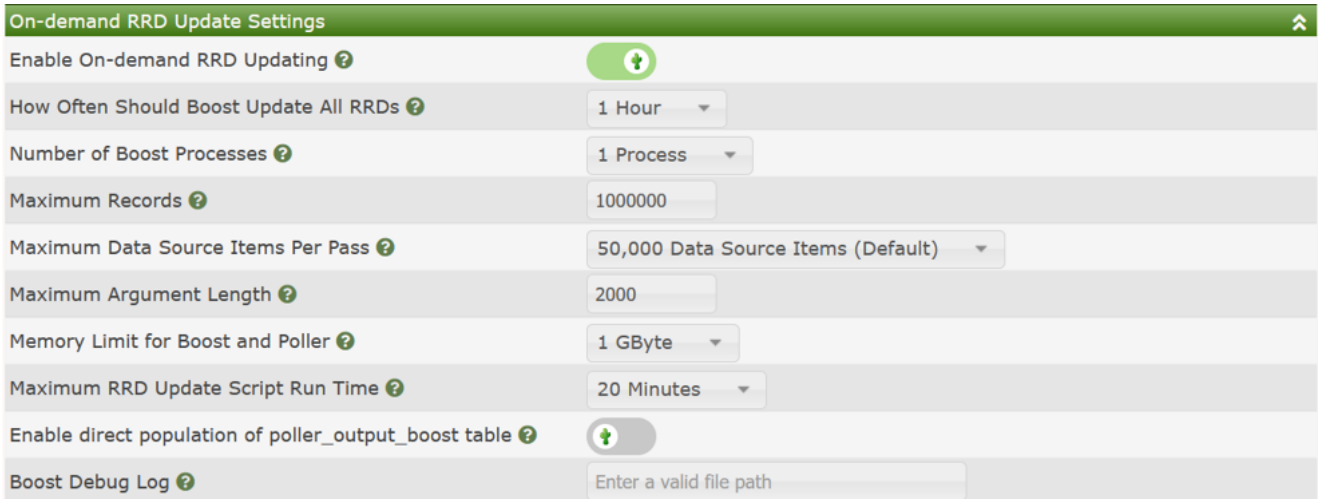
The Performance Settings sub-tab in Cacti contains settings that can help improve the life of your Cacti server and significantly increase the live and lower the I/O wait of your Cacti system through caching of RRDfile disk writes.

It will also reduce the number of CPU cores require to render Cacti **Graphs** due to it's image cache mechanism.

We will explain how these settings work briefly in the next two sections. However, there is a more detailed guide in the Advanced Topics. Please refer to that guide as well.

On-demand RRD Update Settings

These are settings to place RRDfile updates into a database table and cache them. Then, on a periodic basis dump those updates to disk at once. This will reduce the overall wear and tear of your disks, and reduce the time periods where disk I/O will be higher. You can see the available settings in the image below.



The settings include:

- **Enable On-demand RRD Updating** - This check-box will enable on-demand updating and caching of unwritten updates. If you disable it, the poller must first clear the cache before it can be fully disabled. So, disabling is not immediate after it has been activated for some time.
- **How Often Should Boost Update All RRDs** - As it says, what is the desired frequency of performing updates. The idea with boost is to reduce the amount of disk I/O, so if your poller frequency/interval is say 5 minutes, you may want to cache a few hours of updates.
- **Number of Boost Processes** - For systems with shared disk where I/O performance is impacted by file system overhead or latency, the concurrent boost process setting comes in handy. Cacti will spawn the number of collectors specified by this setting.
- **Maximum Records** - This is a threshold in cache records that if that number is exceeded, the cache will immediately begin to be cleared. This setting works together with the **How Often** setting.
- **Maximum Data Sources Items Per Pass** - Cacti will count the number of distinct data sources in your system, and then divide it by this amount, and then make X number of passes through the data. If you have 1M **Data Sources** and 100M records in the cache, and set the setting to 100k, Cacti will make 10 passes through the **Data Sources** each pass processing approximately 10M cache entries.
- **Maximum Argument Length** - This is the longest RRDtool update command to use. Most Linux systems allow for a very long command line. It's advisable to use something less than 32k though as some versions of RRDtool have problems with longer command lines.

- **Memory Limit for Boost and Poller** - In a modern Cacti system, unless you have more than a million **Data Sources** the default of 1GB is likely enough.
- **Maximum RRD Update Script Run Time** - This is the longest to allow the cache clearing script to run before notifying the primary Cacti admin. The process will be allowed to continue to run, but will issue a WARNING.
- **Enable direct population of poller_output_boost table** - Originally a `spine` only setting, this setting is now supported in both `spine` and `cmd.php`. Instead of having to move records inside of the **Data Collector** Cacti simply makes two writes, one into the main output table and another into the boost table. Cacti has to write to the primary output table as some plugins require that data to be there immediately for fault management.
- **Boost Debug Log** - If non-blank and points to a location that is writable, Cacti will log the RRDtool syntax used to update the RRDfiles.

Image Caching

The Image Caching setting on the Performance sub-tab basically defined where to store time sensitive images caches of your various Cacti **Graphs**. These Cached images only live for a few minutes, but can be consumed by multiple users simultaneously.

The image of the settings is shown below. They are fairly self explanatory.

Copyright (c) 2004-2026 The Cacti Group

Spike Kill

Certain types of RRDtool based **Data Sources** can from time to time generate what are look like to the casual observer spikes. These spikes can make your Cacti **Graphs** unreadable.

Spike happen when you either have set no maximum value on your Cacti **Data Template** by using a 'U' as the Maximum Value, or when certain `COUNTER` based RRDfiles have their counter data overflow, and start counting again from zero. When this happens recent versions of RRDtool, like version 1.7, handle these overflows quite well, but for some older RRDtool versions, these changes can result in these spikes.

There are also cases where for some reason the Cacti system goes offline, and therefore your Cacti Graphs will develop what are referred to as gaps.

Spike Kill Settings

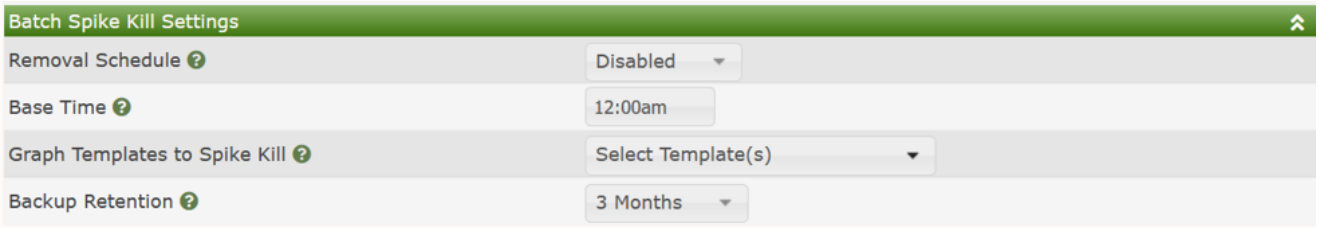
Cacti provides two gap filling options from the user interface. In the image below, you can see the default Settings for Cacti's Spike Killer GUI.

Those settings include:

- **Removal Method** - Cacti supports both Variance and Standard Deviation methods. When calculating those values, you have the ability to filter out outliers such that they don't negatively impact those calculations.
- **Replacement Method** - When filling the Gap, what data point should be used. There is `Last Known Good`, which would be the last good known value, `Average`, which uses the Average value for the RRDfile data source, and `NaN's` which will cause the **Graph** to gap, but the spikes will be removed.
- **Number of Standard Deviations** - This is the number of `Standard Deviations` above the Average that are required before the value is considered a spike. It only applies to the `Standard Deviation` spike removal method.
- **Variance Percentage** - When using the `Variance` spike detection method, the percentage above variance before considering a value a spike.
- **Variance Number of Outliers** - The number of high values to ignore when calculating the Variance.
- **Max Kills Per RRA** - The maximum number of spike to remove from any given RRDfile Data Source.
- **RRDfile Backup Directory** - If you wish to backup RRDfiles that have been altered by Spike Kill, they will be placed in this directory.

Batch Spike Kill Settings

Cacti also allows administrators to kill Cacti spikes on a periodic basis using a batch process. From this batch process, you select the time of day, the **Graph Templates** to perform Spike Kill operations on, and the backup retention for RRDfiles that have spikes removed from them. The image below shows those settings.



Of note is that first of all, the only **Graph Templates** that are shown are **Graph Templates** that include `COUNTER` type **Data Sources**. It is also worth noting that only **Data Query** type **Graph Templates** are supported as of the current Cacti version.

Copyright (c) 2004-2026 The Cacti Group

Mail/Reporting/DNS Settings/Proxy

This area of Cacti is where you configure core notification settings as well as settings to using alternate DNS' servers associated with certain Cacti plugins.

URL Linking

The URL Linking settings provide the Cacti administrator a URL prefix that will point properly to the main Cacti site for both core Cacti functions and Plugins to create URL for their use. Generally this URL should include the full DNS name of the Cacti server, and the correct protocol, for example HTTPS. The image below show an example of how this might be setup.

Mail Settings

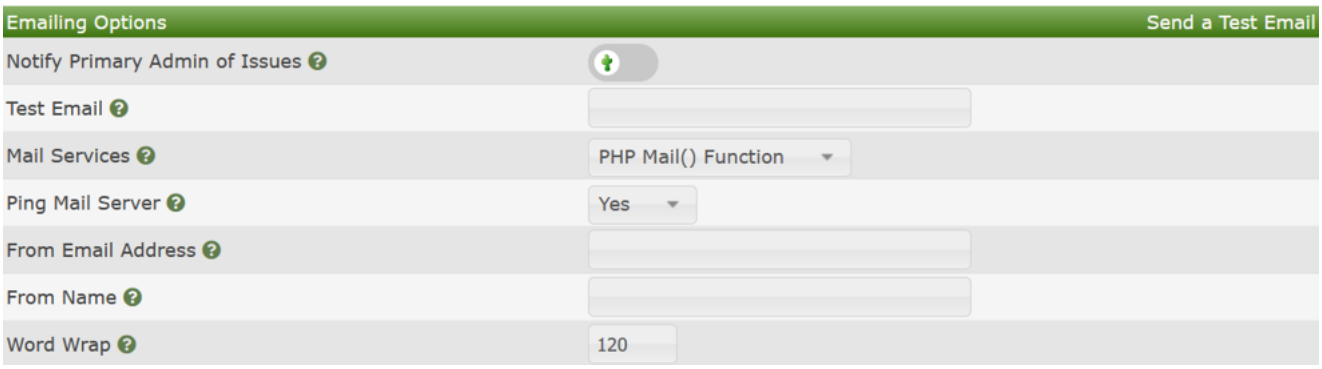
Cacti and its plugins use these settings for notifications and reports. The development branch uses Symfony Mailer through Cacti's mail helpers; older releases use PHPMailer. The available delivery methods include:

- **PHP Mail Function** - PHP has a built-in mail facility where PHP will communicate with the localhost mail facilities to send mail. This is useful if your Cacti system is setup as an anonymous forwarder to a corporate mail system.
- **Sendmail Binary** - This method relies on the `sendmail` binary to properly communicate with your corporate mail systems. Like the **PHP Mail Function**, it requires that your localhost be properly setup to send mail through a forwarding setup.
- **Simple Mail Transfer Protocol (SMTP)** - This is the most configurable option where Cacti can send Emails directly to the Email server.
- **SMTP with OAuth2** - For emailing you can use a mail provider (Google, Microsoft, ...) that requires oauth2 authentication.

In the sections below, we will document each of these options in more details.

Emailing Options

In this section, you specify some general settings around Emailing in general as shown in the image below.



Those settings include:

- **Notify Primary Admin of Issues** - When certain anomalous Cacti events table place that justify the notification of the Cacti administrator, by checking this option, you will receive those notifications.

- **Test Email** - When setting up Cacti for the first time, or when your Email settings change, you can send out a test Email to confirm that the Email setup is working. The Email that you put on this line will receive the test Email.
- **Mail Services** - This is where you pick one of the delivery methods above for providing Mail Services.
- **Ping Mail Server** - When using the SMTP Mail Services provider, Cacti can first ping the Email Server to ensure that it's operational before sending Emails. If the Email Server does not respond, Cacti will issue a warning and continue operations.
- **From Email Address** - The is the Default From Email address that will be used by Cacti and it's plugins, unless the plugin allows overriding the option. The common entry would be something of the form `noreply@mydomain.com` unless you have a desire to receive responses from recipients.
- **From Name** - This setting allows you to provide a name to the given Email address. Something of the form `Cacti Administrator` would be common practice here.
- **Word Wrap** - If not using HTML based Emailing, this will force a word wrap on text data at the length specified.

The next sections will document the various **Mail Services**

PHP Mail Function

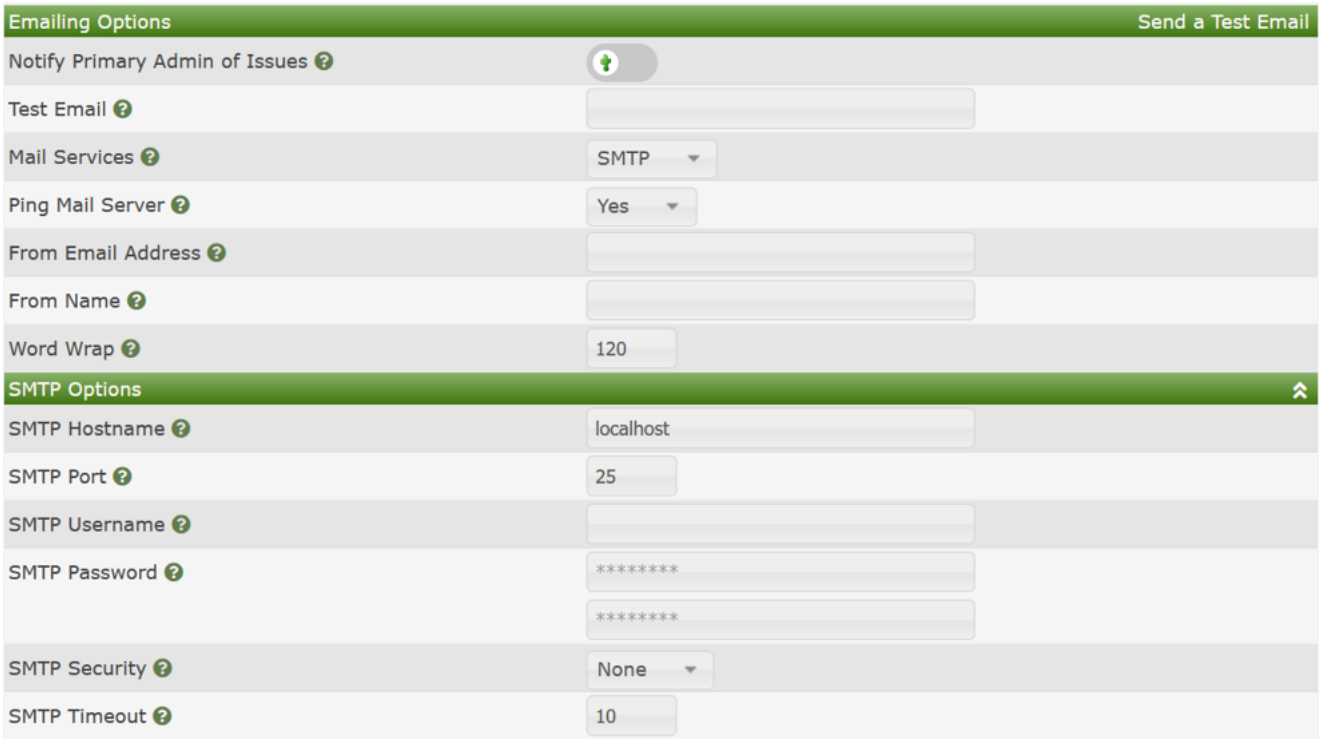
There are no settings for this function. It leverages the Cacti servers mail configuration to send mail notification.

Sendmail Binary

The **Sendmail Binary** is similar to the **PHP Mail Function** with the exception that you can specify the `sendmail` binary path. Cacti actually requires that you include this path.

Simple Mail Transfer Protocol (SMTP)

As mentioned before, the SMTP Mail Services provider provides the most options and will be very familiar and easy to use for Cacti Administrators that may not be as familiar with how to configure the Operating System to properly forward mail to their internal Email servers. Popular mail services like Google, Microsoft, etc. are all supported. Use the authentication and TLS settings required by your provider. The image below shows the **Simple Mail Transfer Protocol (SMTP)** options.

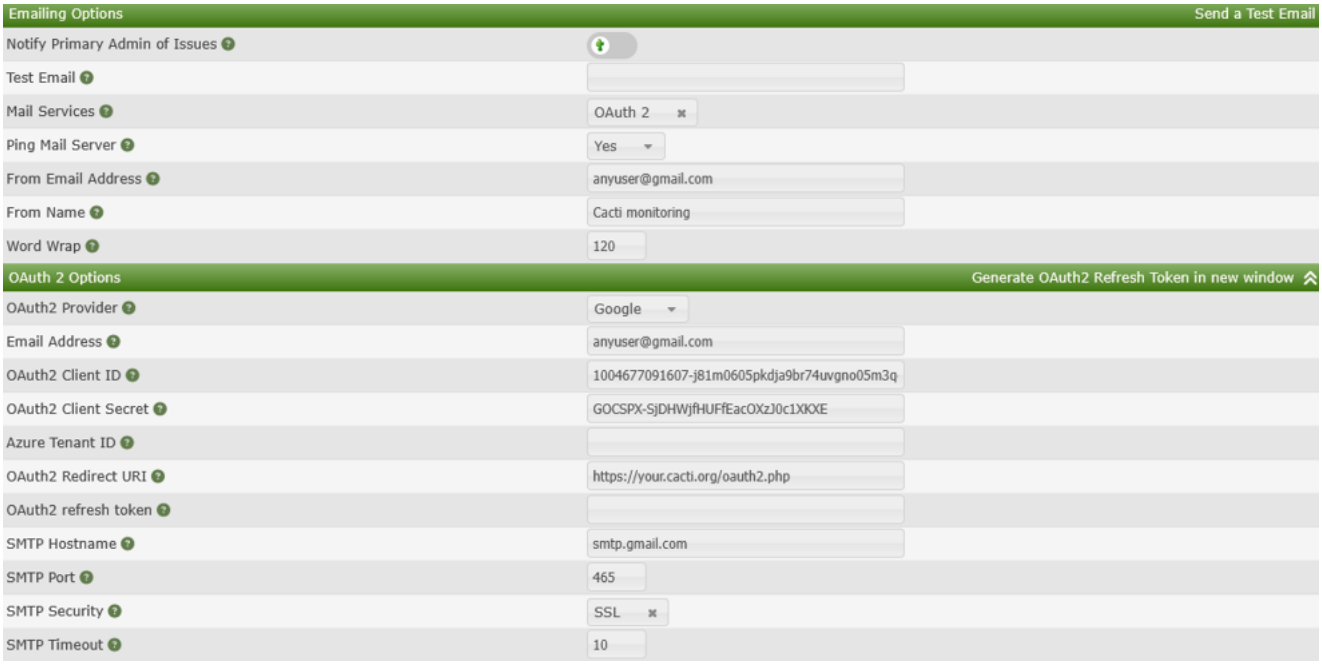


Those options include:

- **SMTP Hostname** - The hostname or DNS name of the SMTP Server
- **SMTP Port** - The TCP Port to use to communicate with the SMTP Server
- **SMTP Username** - The username to use for SMTP communications if your SMTP Server requires authentication.
- **SMTP Password** - The password to use for SMTP communications if you SMTP Server requires authentication.
- **SMTP Security** - The security protocol to use for SMTP communications. The labels distinguish no configured encryption, implicit TLS (**SSL**), and STARTTLS (**TLS**). Choose the mode and port required by the mail server; the **SSL** label does not mean that obsolete SSL protocols should be used.
- **SMTP Timeout** - How long to wait before abandoning connection attempts to the SMTP Server.

SMTP with OAuth2

You can use your email service provider that uses Oauth2. There are ready-made profiles for Google, Microsoft, Yahoo, Azure. Configuration (create an application with offline parameter) may also be required on the email service provider side.

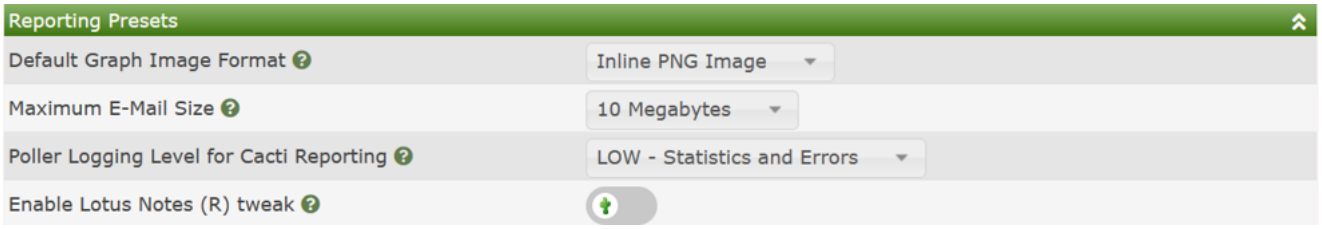


Those options include:

- **OAuth2 Provider** - Select you email provider
- **Email Address** - Email address connected with your OAuth2
- **OAuth2 Client ID** - Client ID of your OAuth2 application
- **OAuth2 Client Secret** - Password for your Client ID
- **Azure Tenant ID** - Used only with Azure
- **OAuth2 Redirect URI** - Register this exact callback URI with the mail provider. The browser returns to it after authorization; Cacti then exchanges the authorization code for tokens. The browser must be able to reach it.
- **OAuth2 Refresh Token** - If you know it from your OAuth2 provider, paste it here. If not, you can use the wizard here to get the token. After successful authorization and the server-side code exchange, Cacti stores the token.
- **SMTP Hostname** - The hostname or DNS name of the SMTP Server
- **SMTP Port** - The TCP Port to use to communicate with the SMTP Server
- **SMTP Security** - The security protocol to use for SMTP communications. The labels distinguish no configured encryption, implicit TLS (**SSL**), and STARTTLS (**TLS**). Choose the mode and port required by the mail server; the **SSL** label does not mean that obsolete SSL protocols should be used.
- **SMTP Timeout** - How long to wait before abandoning connection attempts to the SMTP Server.

Reporting Presets

Cacti Reporting can generate very large Email attachments. Therefore the **Reporting Presets** allow you to control the traffic associated with reporting. From here, you can also set presets for the image type and logging. You can see the settings from the image below.



Since these settings are straightforward, no further explanation is given here.

DNS Options

The DNS Options allow you to specify a primary and secondary DNS to use for Plugin DNS operations. Many plugins allow you to even use their own DNS Options. For plugins that require Alternate/Non-System DNS settings, see their documentation as to how DNS is setup and used.

The image below shows the two settings.

Copyright (c) 2004-2026 The Cacti Group

Authentication

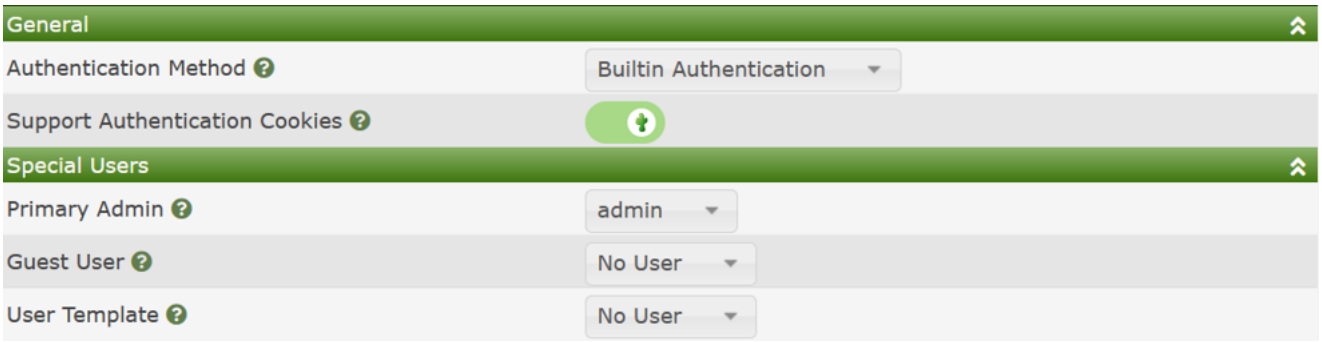
Overview

Authentication options depend on the Cacti version. In the unreleased Cacti 1.3 development branch, choose from:

- **Cacti Builtin Authentication** - Local accounts use password hashes and Cacti's password policies. See Local Authentication.
- **Web Basic Authentication** - The web server authenticates the user and passes the authenticated identity to Cacti. Cacti still controls access to its features and objects. See Basic Authentication.
- **Login Providers** - Configure LDAP, Active Directory, SAML2, or OpenID Connect through Login Providers.

Cacti 1.2 instead provides separate **LDAP Authentication** and **Multiple LDAP/AD Domains** methods. The LDAP and domain guides retain those version-specific details. Do not assume that development interface labels are present in a 1.2 release.

For most of these `Authentication Methods`, there are some common settings shown in the image below.



Those options include:

- **Authentication Method** - One of the options below.
- **Support Authentication Cookies** - Offers a `Keep me signed in` checkbox on the login page. The authentication cookie keeps the user signed in on that browser and expires after 90 days without use. The client must use HTTPS for the cookie to be issued.
- **Primary Admin** - This is the Primary Cacti Admin account. This user account receives Cacti administrative notifications, such as when the Cacti system is experiencing issues.
- **Guest User** - If you select a **Guest User**, people within an organization will be able to access certain Cacti pages without first authenticating. This setting is good for pages like the **Graph Page** where you may wish to embed live Cacti **Graphs** in other corporate websites.
- **User Template** - When creating new accounts, whether LDAP, Web Basic, or new Cacti local accounts, Cacti takes the user's default authorization from this Template User account. For some Cacti **Authentication Methods** you can have more than one Template User, for example **Login Providers**, or **Multiple LDAP/AD Domains** in Cacti 1.2.

In the sections below, we will go over these various **Authentication Methods** and their settings.

Built-in Authentication

Web Basic Authentication

LDAP Authentication

Login Providers (development)

Multiple LDAP/AD Domains (1.2)

Copyright (c) 2004-2026 The Cacti Group

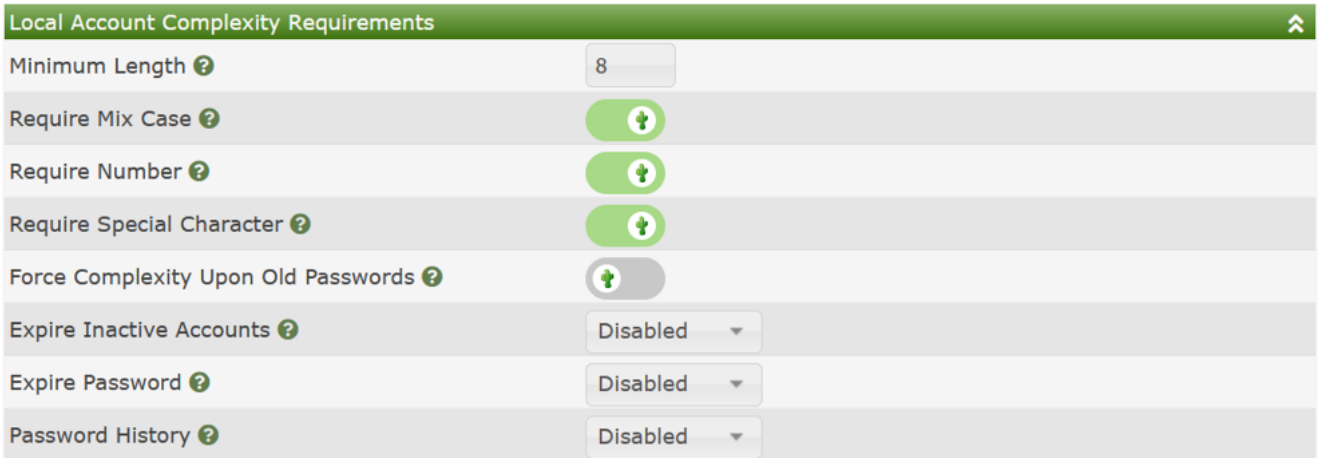
Local Authentication

Overview

Cacti's Built-in Authentication is sufficient for most small work-group Cacti installations. Cacti stores local accounts and password hashes in its database, not recoverable plaintext passwords. Protect database access and backups as part of the installation's credential security.

Local Account Complexity Requirements

Additionally, Cacti has the ability to remember password history, force complexity rules, minimum lengths, force change frequency, etc. on new passwords. These settings are configurable as well. The image below shows the local complexity settings available in Cacti.

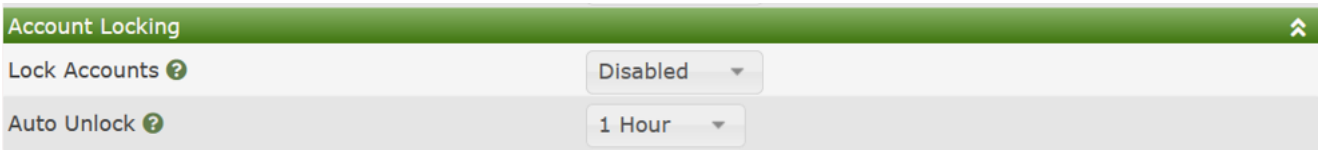


Those options include:

- **Minimum Length** - The minimum length password that is acceptable
- **Require Mix Case** - Require that the password have mixed case
- **Require Number** - The password must includes a number.
- **Require Special Character** - The password must include a Special Character.
- **Force Complexity Upon Old Passwords** - If set, and a user attempts to login with a password that does not meet the current complexity requirements, they will be forced to change their password.
- **Expire Inactive Accounts** - Disable inactive accounts after a certain number of days of inactivity.
- **Password History** - Maintain a list of previous passwords, and do not allow a user to use one of those prior passwords during their password changes.

Account Locking

If a user attempts to use an incorrect password more than a certain number of times without a successful login, Cacti can automatically lock out the user from that account for a certain amount of time. This technique prevents hackers from continually attempting to break a users password from the Cacti interface. The images below shows the two settings associated with this Cacti feature.



The settings are fairly self explanatory, so additional detail is no provided here.

Copyright (c) 2004-2026 The Cacti Group

LDAP Authentication

On the unreleased Cacti 1.3 development branch, configure LDAP and Active Directory under Login Providers. Cacti 1.2 uses the authentication settings and User Domains screens described below. The LDAP terms and connection settings apply to both, but menu labels differ.

Cacti supports using LDAP authentication to access the Cacti GUI. There are several different ways to use LDAP, which will be described in the following sections.

Be Aware

New authentication settings may not take effect immediately. We recommend to open a **new** private browser window to test configuration changes.

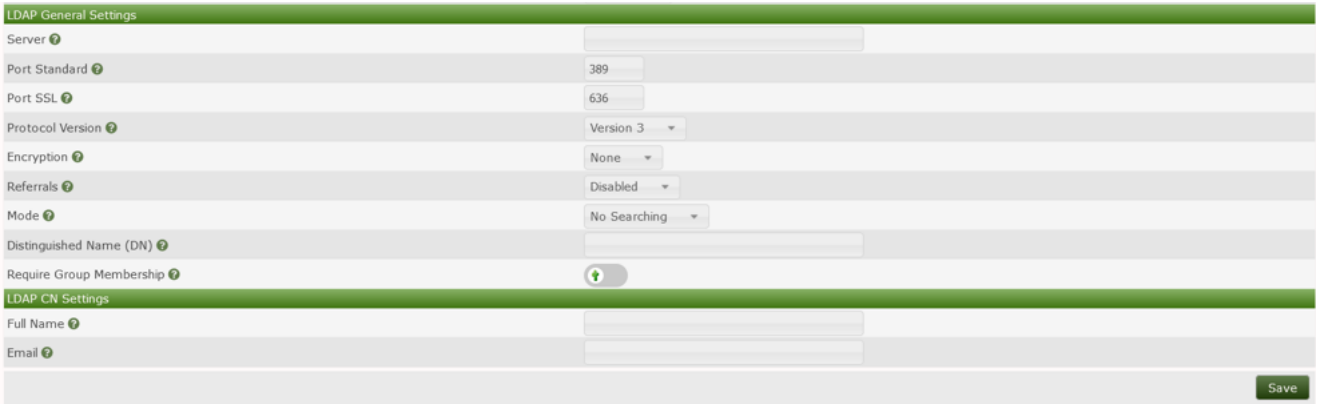
Defining some terms

Before proceeding, please ensure you are familiar with the following terms:

1. *LDAP* - A communication language, or protocol. It is a defined structure of how to ask questions of, and receive responses from, a directory. It also defines what questions there exist to be asked, and what responses there exist to be returned. LDAP is not a directory; it is a way to query a directory.
2. *Directory* - A database of objects, accompanied with other attributes. Commonly, these objects are user identities, accompanied with passwords. Sometimes, the objects are groups, accompanied by a list of members. Usually the objects are stored in some type of hierarchical structure for organizational and management purposes.
3. *DN* - Short for *distinguished name*, a string that represents a unique object in the directory. There can not be two DNs alike in a directory. In Microsoft's *Active Directory*, this is a string like 'CN=Joseph,OU=Accounting,OU=Users,DC=corp,DC=example,DC=com'.
4. *Search* - A type of LDAP request, in which the client sends some criteria to the server in an attempt to receive back a list of directory objects that meet the search criteria.
5. *Bind* - A type of LDAP request, in which the client sends the server a username and a password in an attempt to confirm whether the password is correct for that user.
6. *sAMAccountName* - An attribute found in Microsoft's *Active Directory* directory, and is what we casually know as a *username*. The *sAMAccountName* does not uniquely represent an object in the Active Directory, whereas the *DN* does.

Configuring LDAP

LDAP General Settings



Mode

There are three basic routines that Cacti can execute when handling an LDAP authentication.

1. *No Searching* - In this routine, Cacti simply binds to the LDAP server using the credentials that were entered during login. Cacti performs a transformation upon the username before binding, that which is specified in the *Distinguished Name (DN)* field. This transformation is useful, for example, to add a fully-qualified domain name suffix, or to add a NetBIOS domain name prefix. For example:

```
<username>@example.com
```

or

```
domain\<username>
```

2.*Anonymous Searching* - In this routine, Cacti connects to LDAP with no authentication and attempts to perform a search for the *username*, in an attempt to retrieve the specific DN associated with the username. Once the specific DN of the username is found, Cacti binds to LDAP using the DN of the user and the password that was entered during login. Please note, by default, Microsoft's Active Directory does not permit anonymous searches, and will refuse the search request.

3.Specific Searching_- In this routine, Cacti binds to LDAP using a service account that was provided beforehand, then performs a search for the *username*, in an attempt to retrieve the specific DN associated with the username. Once the specific DN of the username is found, Cacti binds to LDAP using the DN of the user and the password that was entered during login.

Encryption

1. *None* - This is plain-text, normally on port tcp/389. Passwords are transmitted over the network in clear text between Cacti and the directory.
2. *SSL* - Commonly known as LDAPS, this is just like LDAP, but encrypted. This normally runs on port tcp/636. Please note, this is not enabled by default for Microsoft's Active Directory because it requires an encryption key and certificate to be installed on the AD controller(s).
3. *TLS* - This is plain LDAP, but a START_TLS command is sent by the client to the server to initiate encryption. Normally runs on port tcp/389 and can be entered into during a regular LDAP session.

Please note that some directories will limit the request types to which they respond when there is no encryption on the LDAP communication.

Referrals

An LDAP Referral is a field returned in an LDAP response message. The referral provides a list of alternate sources from which directory information can be obtained. In the case of a replicated directory such as Microsoft's Active Directory, the referral may contain an LDAP URL which could resolve to any participating LDAP server, whether correctly configured or not.

LDAP Group Settings

Cacti can enforce that a user be a member of a specific directory group to be granted access to Cacti. This is useful when your directory contains a mix of users who should and should not be granted access to Cacti. For Cacti to require that the user be apart of a group, Cacti will send a *compare* request to the directory server asking the server if a specific group has any parameters that contain a specific username. As an example, a compare requests asks the following type of question:

Does there exist a group named `CN=cacti-admins,OU=Admin_Groups,OU=Groups,DC=corp,DC=company,DC=com`, attached to which there is an attribute named `member`, where the value of the attribute is `CN=Joseph,OU=Admins,DC=example,DC=com`?

The following three settings determine the content of the compare request.

1. *Group Distinguished Name (DN)* - The DN of the group into which the directory server should search for attributes. For example:

```
CN=cacti-admins,OU=Admin_Groups,OU=Groups,DC=corp,DC=company,DC=com
```

2.Group Member Attribute_ - The name of the attribute into which the directory server should search for the user. For example:

```
member
```

3.Group Member Type_ - The format of the text string which should represent the user.

Specific Search Settings

1. *Search Base* - The portion of the directory hierarchy inside which the server should perform the search. When the directory contains many objects, the time and resources required to search the directory are greater. The *search base* setting is useful to reduce the portion of the directory that is searched, thus increasing speed and reducing processor load on the directory server. This setting is also useful if you want to restrict access to Cacti to users who exist in a particular location in the director heierarchy.
2. *Search Filter* - The criteria for which the directory server should search. Any parameters that the server is capable of understanding are permitted here. Cacti will perform a substitution for the phrase `<username>` in the search filter, and will replace it with the username. This field must be in acceptable LDAP syntax. A useful example of a search filter would be:

```
(sAMAccountName=<username>)
```

But more complex filers are also permitted, such as:

```
(&(sAMAccountName=<username>)(memberOf=CN=cacti-admins,OU=Admin_Groups,OU=Groups
,DC=corp,DC=company,DC=com))
```

3.Search Distinguished Name (DN)_ - The username of the service account with which Cacti will bind to the directory.

4.Search Password_ - The password of the service account with which Cacti will bind to the directory.

LDAP CN Settings

Mapping an LDAP user to a Cacti user

Cacti stores many personal preferences and settings into its database that make it easier to use Cacti. For example, the GUI Theme, the Default View Mode, and the User Language are a few of these user settings. These settings are not stored in the LDAP directory, so they are not returned in the LDAP response. Therefore, Cacti will create a local user identity for each LDAP user who logs in. When Cacti creates a local user, it can automatically populate the following two parameters from the LDAP directory. If Cacti sees a local user already exists, it will not attempt to update the user with values from the directory.

1. *Full Name* - The LDAP attribute name which contains a user's full name. Some common examples are 'displayname' or 'cn'.
2. *Email* - The LDAP attribute name which contains a user's email address. One common example is 'mail'.

Always user lower case. No matter if the directory's attribute name contains uppercase characters (e.g. `fullName`), the respective field must be filled out in lower-case (`fullname`).

A few notes

One-Time Passwords (OTP / MFA)

Cacti supports directories where users authenticate with a One-Time Password (OTP) or another Multi-Factor Authentication (MFA) token, but the LDAP *Mode* must be chosen so that the user's single-use credential is presented to the directory only once.

On a single-server authentication path, Cacti binds the user's password exactly once, during the final authentication step. The Distinguished Name (DN) lookup that precedes it never uses the user's password (see the note on multi-server failover below):

- *No Searching* performs no bind during the lookup (the DN is built from the template), then binds once as the user.
- *Anonymous Searching* binds anonymously to locate the DN, then binds once as the user.
- *Specific Searching* binds with the service account to locate the DN, then binds once as the user.

Group membership checks, when enabled, reuse the connection that was already bound with the user's credential and do not perform a second bind.

Because Active Directory refuses anonymous searches by default, an OTP/MFA deployment on AD should use *Specific Searching* with a dedicated service account. The service account performs the directory search so that the user's one-time code is preserved for the single authentication bind. *No Searching* is also safe for OTP when the DN template alone identifies the user, since it performs no directory lookup; choose *Specific* or *Anonymous Searching* only when a lookup is actually required to resolve the DN.

Finally, for OTP/MFA do not use the *Server(s)* space-delimited multi-server list. That field fails over from left to right, so if the first server consumes the one-time password and then returns an error, Cacti re-attempts the bind against the next server with an already-spent code, which will fail. Instead, point the *Server(s)* field at a single, resilient DNS name fronted by a DNS load balancer (VIP). Treat that DNS/load-balancer layer as the resilient, highly-available entry point: it handles backend failover transparently, so Cacti only ever performs one authentication bind per login and the single-use code is never replayed.

Certificate verification

When using LDAPS or STARTTLS, the Cacti server (as an LDAP client) must trust the certificate presented by the directory server. PHP's LDAP functions use the system OpenLDAP library, which reads its certificate configuration from `/etc/ldap/ldap.conf` (Debian/Ubuntu) or `/etc/openldap/ldap.conf` (RHEL/Rocky/AlmaLinux).

Adding a CA certificate on Linux

If the directory server uses a certificate signed by an internal CA or a self-signed certificate, install the CA certificate on the Cacti server and configure the OpenLDAP client library to trust it.

Debian/Ubuntu:

```
# Copy the CA certificate
cp /path/to/internal-ca.crt /usr/local/share/ca-certificates/internal-ca.crt

# Update the system trust store
update-ca-certificates
```

Add the following line to `/etc/ldap/ldap.conf` if not already present:

```
TLS_CACERT /etc/ssl/certs/ca-certificates.crt
```

RHEL/Rocky/AlmaLinux:

```
# Copy the CA certificate
cp /path/to/internal-ca.crt /etc/pki/ca-trust/source/anchors/internal-ca.crt

# Update the system trust store
update-ca-trust extract
```

Add the following line to `/etc/openldap/ldap.conf` if not already present:

```
TLS_CACERT /etc/pki/tls/cert.pem
```

Controlling certificate verification strictness

By default, the OpenLDAP library requires a valid, trusted certificate (`TLS_REQCERT demand`). You can relax this for testing, but do not disable verification in production; it exposes the LDAP bind to interception.

```
# /etc/ldap/ldap.conf or /etc/openldap/ldap.conf

# Require a valid certificate (default, recommended for production)
TLS_REQCERT demand

# Accept certificates signed by the configured CA even if the hostname
# does not match (useful if the LDAP server has an internal hostname)
TLS_REQCERT allow

# Disable certificate verification entirely — DO NOT use in production
TLS_REQCERT never
```

Verifying the configuration

Test the LDAP TLS connection from the Cacti server before enabling it in Cacti:

```
# Test LDAPS (port 636)
openssl s_client -connect ldap.example.com:636 -CAfile /path/to/ca.crt

# Test STARTTLS (port 389)
openssl s_client -connect ldap.example.com:389 -starttls ldap -CAfile /path/to/ca.crt
```

> **Note**: `openssl s_client` verifies the certificate chain but does not check the server hostname. Use `ldapsearch` or a dedicated TLS testing tool (e.g. `testssl.sh`) to confirm hostname validation.

```
# Full LDAP query over TLS using ldapsearch (-W prompts for password)
ldapsearch -H ldaps://ldap.example.com -b "dc=example,dc=com" \
  -D "cn=service,dc=example,dc=com" -W "(cn=testuser)"
```

A successful `CONNECTED` and `Verify return code: 0 (ok)` in the `openssl` output confirms the Cacti server trusts the certificate.

Adding a CA certificate on Windows (IIS)

On Windows, PHP's LDAP extension uses the Windows Certificate Store. Import the CA certificate using the Microsoft Management Console:

1. Open `mmc.exe`, go to **File > Add/Remove Snap-in**, add **Certificates** for the **Computer account**.
2. Expand **Trusted Root Certification Authorities > Certificates**.
3. Right-click, select **All Tasks > Import**, and follow the wizard to import the CA certificate (`.crt` or `.cer` file).
4. Restart the IIS service (`net stop w3svc && net start w3svc`) for the change to take effect.

Search Result Reference (Referrals)

In Microsoft's Active Directory, LDAP response messages may contain a referral to other directory servers. If you are using LDAPS and have installed a certificate on some LDAP servers and not others, LDAP lookups will fail intermittently because LDAPS negotiations will succeed with some directory servers while failing with other directory servers. If you are experiencing intermittent LDAP failures, ensure that the Cacti server has network access to all directory servers, and that all directory servers are properly configured.

Common Scenarios

These following scenarios are designed to help you see the different ways to harness the power of LDAP and build a working LDAP configuration in your environment.

A small office where everyone has access to Cacti

In this scenario, a small LDAP directory contains users who are all granted access to Cacti (there are no users denied from Cacti). Some may be administrators, while others are read-only users.

To handle this scenario, start by creating a new local user in Cacti with read-only permissions. Name the user 'MyLdapTemplate'. Next, in Cacti's Authentication settings, set the `User Template` field to 'MyLdapTemplate'. Set the `Mode` to 'No Searching'. Set the `Distinguished Name (DN)` field to `<username>@example.com'`, where 'example.com' is the name of your domain. When any user logs in to Cacti, their password will be checked against the LDAP directory and they'll be granted access to Cacti with read-only access (from the LDAP template user). Once a new user logs in, Cacti will create a local user where the user's `Realm` is 'LDAP'. You may then optionally promote that user to an administrator by visiting the `Users` page, locating the account for the LDAP user, and granting additional permissions to that user.

A large office where you only want a subset of users to access Cacti

In this scenario, a large LDAP directory contains a mix of users who should, and should not, be granted access to Cacti.

There are two ways to handle this scenario. You may either manually authorize users within Cacti, or you may use a group in the directory to define the authorized users.

To handle this scenario by manually authorizing users within Cacti, navigate to the Authentication settings, and set the `User Template` field to 'No User'. Set the `Mode` to 'No Searching'. Set the `Distinguished Name (DN)` field t `<username>@example.com`, where 'example.com' is the name of your domain. Next, navigate to the `Users` page and create

a new user where the username matches the username in the directory. Set the user's `Realm` to 'LDAP'. Grant your desired permissions to the user. When that user logs in to Cacti, their password will be checked against the LDAP directory and they'll be granted access to Cacti based upon their local account.

To handle the large office scenario by using group membership, start by creating a user in the directory that can be used for searching the directory (this is sometimes referred to as a Service Account). Under normal circumstances, this directory user does not need elevated permissions. Next, create a group in the directory that contains all the users who should be granted access to Cacti. Next, in Cacti, create a new local user named 'MyLdapTemplate', with administrator permissions. In Cacti's Authentication settings, set the `User Template` field to 'MyLdapTemplate'. Set the `Mode` to 'Specific Searching'. Leave the `Distinguished Name (DN)` field blank. Set the `Require Group Membership` field to enabled. Set the `Group Distinguished Name (DN)` to the name of the group. Set `Group Member Attribute` to 'member'. Set `Group Member Type` to 'Distinguished Name'. Set `Search Base` to the DN of the hierarchical structure in the directory which contains all the users, such as 'OU=Users,DC=example,DC=com'. Set `Search Filter` to `'sAMAccountName=<username>'`. Set `Search Distinguished Name (DN)` to the username of the service account you created in the directory. Set `Search Password` to the password for the service account. Optionally, set `Full Name` to the directory attribute that contains the user's name, such as 'cn', and also set `Email` to the directory attribute that contains the user's email address, such as 'mail'.

Copyright (c) 2004-2026 The Cacti Group

Basic Authentication

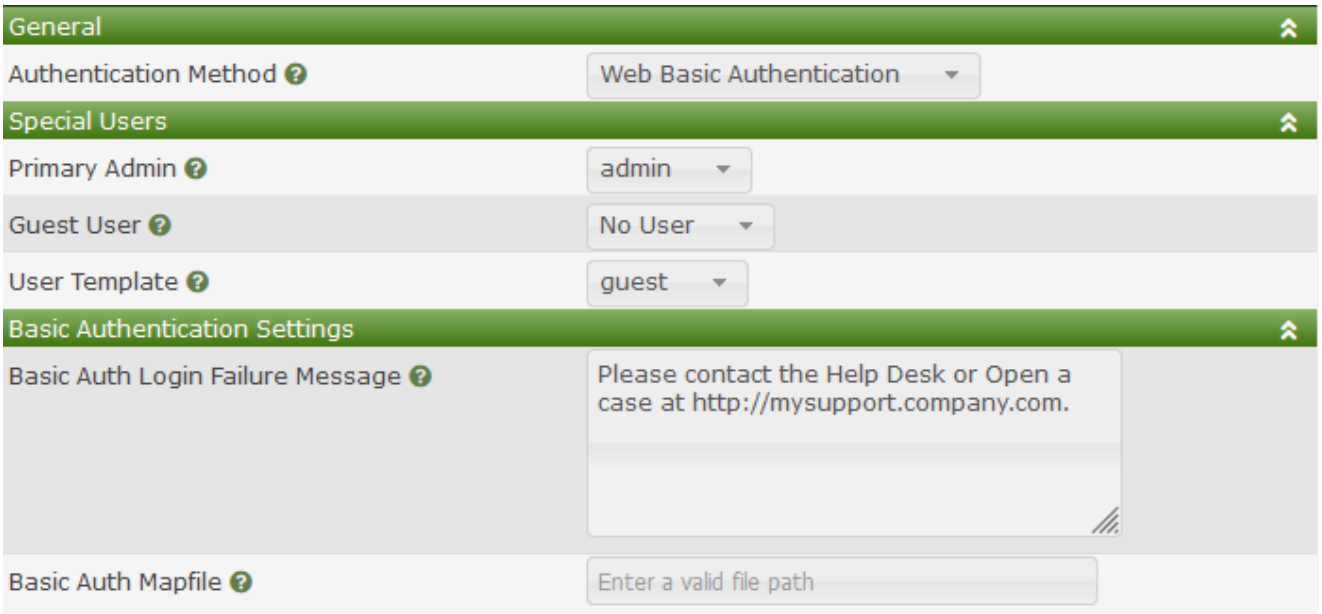
Overview

Basic Auth settings leverage both Apache's and NGINX Authentication modules to grant a user access to the Cacti Website. The various configuration steps for these modules will not be covered in the Cacti documentation. Therefore, if you wish to use this authentication method, it is recommended that you test your configuration on a development server before putting this method into production.

> **NOTE**: Before enabling the **Basic Authentication Method**, you must always verify that you are prompted for a password before getting the normal Cacti login prompt, and that you have setup a new user account with Admin privileges that matches your login account. Otherwise when you enable Basic Authentication, you can lock yourself out of the Cacti Website.

Special Users

The image below shows the settings for **Web Basic Authentication** which includes the `Special Users` sub-section.



Those settings include:

- **Primary Admin** - This is the Cacti primary administrative account this account will receive Cacti notifications and can not be deleted.
- **Guest Account** - This is a special account that allows users to access certain pages without being prompted for a login and password. Setting to `None`, disables this feature.
- **User Template** - All new users who login for the first time will have their initial settings based upon this **User Template**.
- **Basic Auth Login Failure Message** - This textbox can be customized in order to provide a useful message to users who have no access to Cacti via Basic Authentication and how to resolve the issue.
- **Basic Auth Mapfile** - If your basic users do not match OS users you can force a basic account to login with an alternate account.

Basic Auth Mapfile

This settings is important for plugins that grant certain features to accounts, but those accounts do not match the login account. For example, your basic account maybe: joe.schmoe@company.com, and your UNIX login account may be `jschmoe`, which grants you certain plugin permissions, like controlling your own workload.

The mapfile is a file in CSV format, with the first column being the Basic account (aka joe.schmoe@company.com), and the second column being the login account (jschmoe). It is up-to the Cacti administrator to manage and update this file per their local site practices.

Copyright (c) 2004-2026 The Cacti Group

Multiple LDAP/AD Domains

This page describes the Cacti 1.2 authentication interface. The unreleased Cacti 1.3 development branch uses Login Providers instead.

Summary

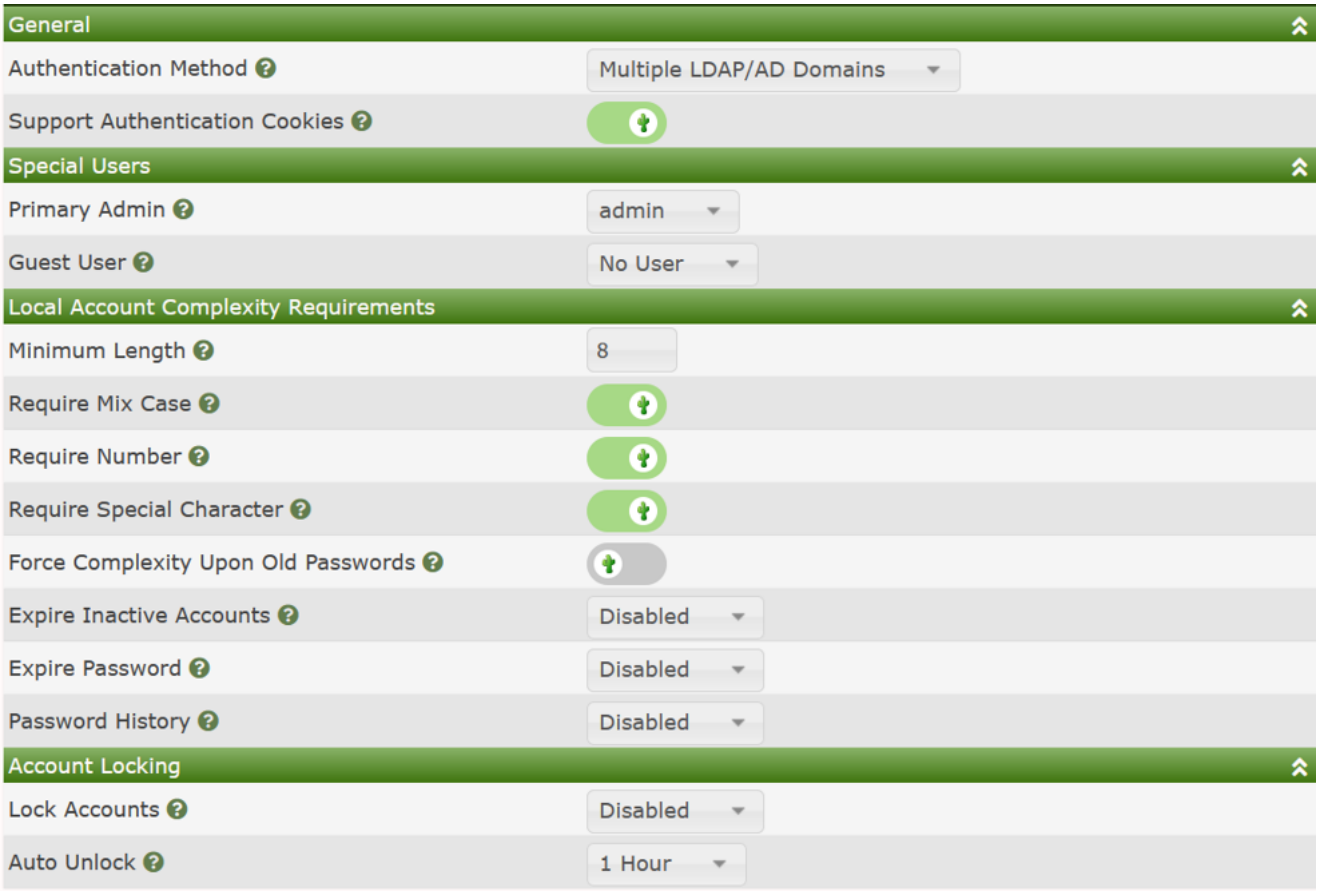
This Authentication method enables the Cacti Administrator to have multiple login level templates for their accounts. This could be used to grant users say for the following three login levels controlled by an LDAP or AD User Group:

- **Admins** - People with full control of Cacti
- **Super Users** - Higher level of access, but below **Admins**
- **Normal Users** - For normal entry level users with limited access.

To use the Multiple Domains setup, you must first setup at least one User Domains from the Cacti console. Once the **Multiple LDAP/AD Domains** are setup, all users will have to provide domain credentials as well as select from which domain they wish to authenticate from, though when enabling this settings, **Built-in** accounts will still be able to login, however, they will have to provide the `Local` option from the drop-down on the login page.

Settings

When you select the **Multiple LDAP/AD Domains** the Authentication sub-tab will look like the image below.



The reason that it shows settings for `Built-in` accounts is that you can still login as a local account when you choose this setting.

Copyright (c) 2004-2026 The Cacti Group

Configuration - Users, Groups and Login Providers

Manage identities and permissions under **Console > Configuration**:

- Users controls individual accounts, realm access, graph permissions, and account settings.
- User Groups assigns shared permissions to users.
- Login Providers configures external authentication on the unreleased Cacti 1.3 development branch.

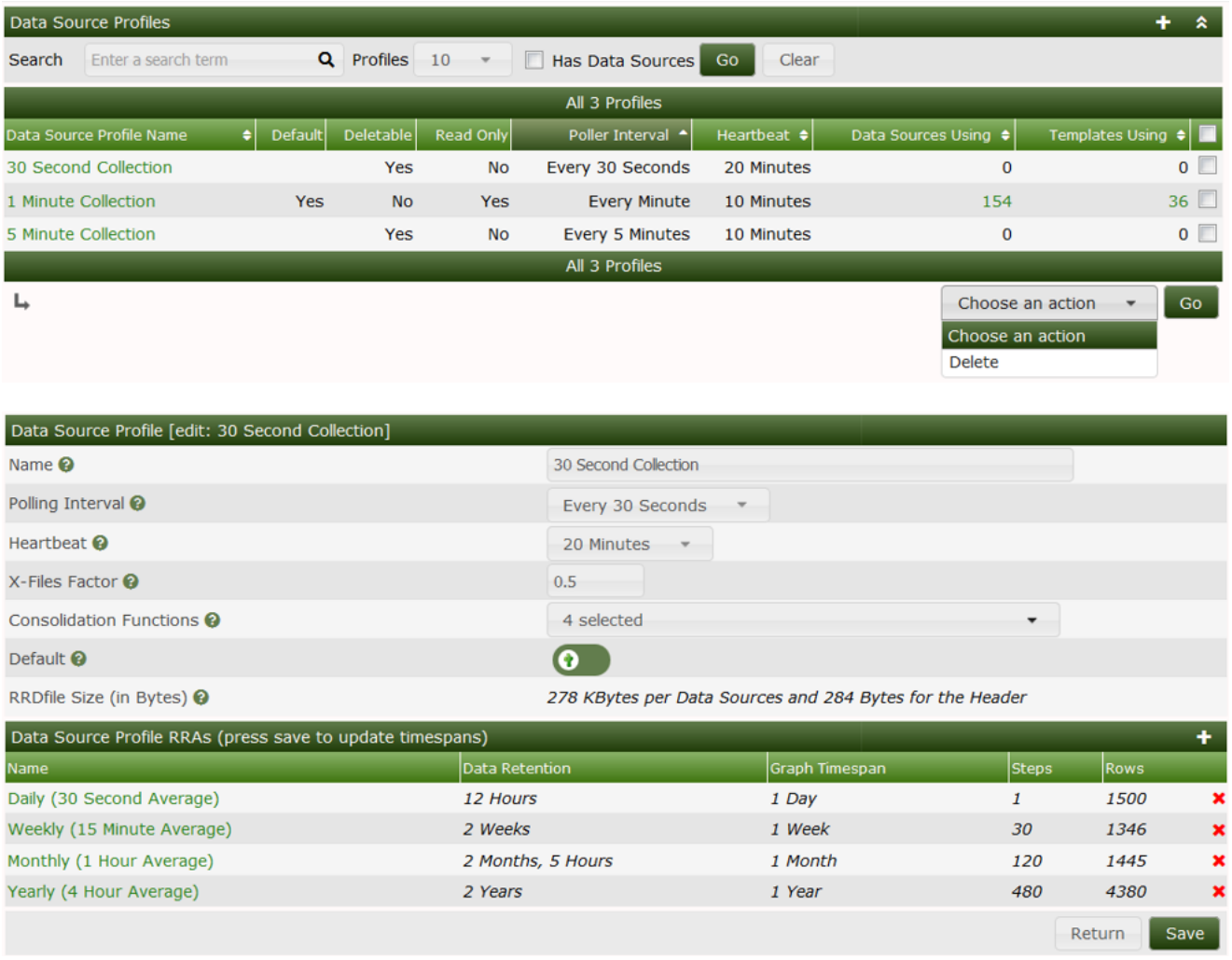
Authentication proves an identity; Cacti's user and group permissions determine what that identity can see and change. Review both when provisioning accounts.

In Cacti 1.2, external directory domains are configured through User Domains. See Authentication for the version-specific options.

Copyright (c) 2004-2026 The Cacti Group

Data Source Profiles

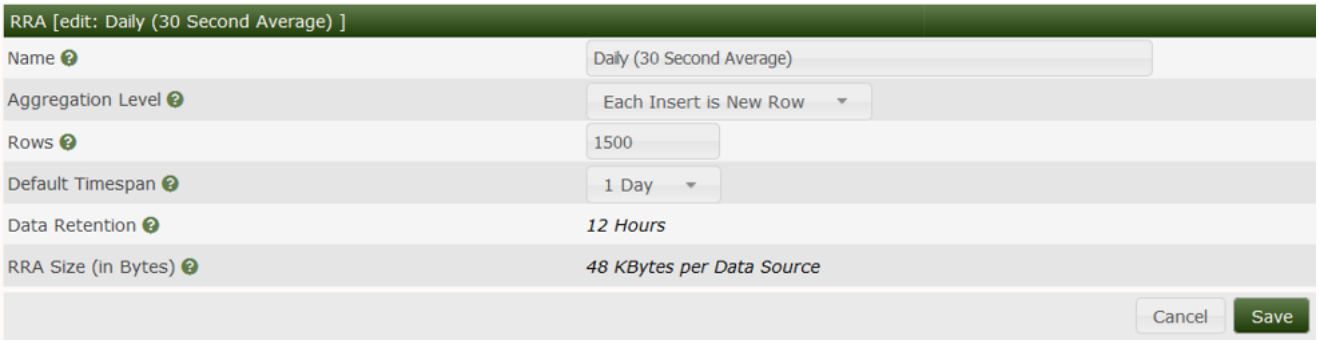
Data Source Profiles define sampling intervals, heartbeats, consolidation functions and archive retention for data sources. A profile controls how an RRD file is created. Selecting another profile does not convert an existing file.



Configure a profile for new data sources

Open **Console > Presets > Data Profiles**. Create a profile or duplicate an existing one, then set its sampling interval, heartbeat and consolidation functions. Edit its archive rows and aggregation durations for the desired retention before assigning it to new templates or data sources.

Archive settings are read-only when existing local sources use that profile. Duplicate the profile and edit the copy for new sources instead of deleting all graphs to unlock the editor. More rows, series, archives and consolidation functions require more disk space; the profile's displayed estimate is more useful than a fixed size table copied from a different configuration.



Calculate retention

Each RRA row covers the base sampling step multiplied by the number of primary data points consolidated into it:

```
row_seconds = profile_step_seconds * archive_steps
retention_seconds = rows * row_seconds
rows = desired_retention_seconds / row_seconds
```

For three days (259,200 seconds) of one-minute rows, use 4,320 rows. For the same retention at five-minute aggregation, use 864 rows. Round up when the division is not exact. The GUI presents aggregation durations; the stored `steps` value is their multiplier of the base step, not a count of rows per hour.

Each archive maintains its own data and retention. Older data does not move from one archive into another when the graph timespan changes. Inspect the existing file's `step`, `pdp_per_row` and row count with `rrdtool info` before planning a conversion.

Change existing collection

By default, **Change Profile** changes the selected template definitions for future sources. On this development branch, **Update database for existing Data Sources?** also updates attached database definitions and launches a poller-cache rebuild. That option still does not change existing RRD files. Use it only as part of a planned migration with matching file structure and collection settings.

Back up the database, configuration and RRD files as a matching recovery set. Record affected sources, their actual file layouts, consolidation functions, retention, ownership and collection schedule. Pause writes while converting files and test the procedure on copies first.

Choose the conversion for the actual change. RRDtool can tune supported fields, resize archives, and, in versions 1.5 and later, add or remove data sources. Changing resolution or rebuilding archives can require a new destination RRD and an explicit data migration. On the main collector, Cacti's `cli/splice_rrd.php` merges an old file with a prepared destination file; it does not choose the new profile, create a migration plan or recover detail already consolidated away. Consult the selected release's `--help` and validate the result before replacing a live file.

Deleting every graph is not a prerequisite for a profile migration. The `cli/remove_graphs.php --all` selection by itself is a dry run; `--force` performs removal, and eligible data sources are removed by default unless `--preserve` is supplied. Do not use this operation as a profile-editing shortcut.

For RRDtool operations, see tune, resize, and info. Changing the collector's cron interval is separate from converting its data.

Copyright (c) 2004-2026 The Cacti Group

CDEFs

Background

CDEF's in Cacti are a one to one analog to CDEF's in RRDtool. Cacti simply provides an interface to create and manage them. Once the CDEF's are created in Cacti they can be imported and exported globally.

CDEF's are mathematical formulas that either modify the numeric data from one to many data sources or VNAMES that you have in your **Graph Template**.

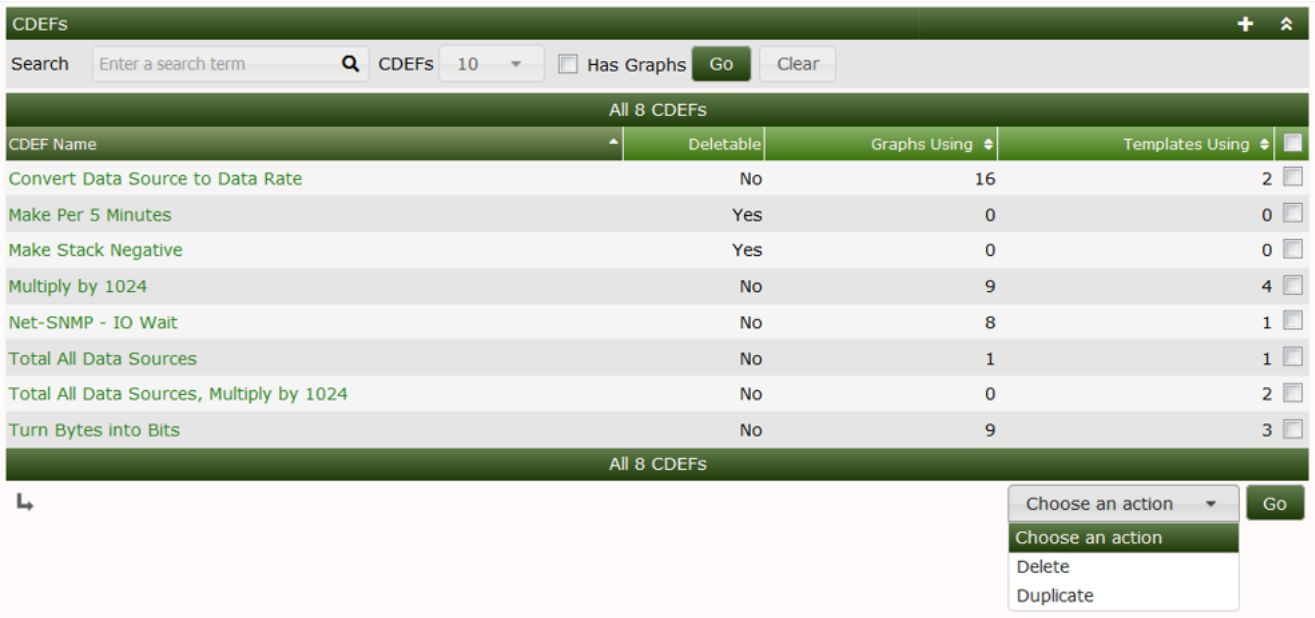
The format of the mathematical formulas is called Reverse Polish Notation (RPN). RPN was and is an early form of how Engineers entered equations into early HP and other Calculators to solve Engineering problems. The reason we still use it today is that it follows a simple Stack principle. In other words, it's not broken.

CDEF's can get very complex as there are several mathematical functions available in the RRDtool command set.

CDEF Interface

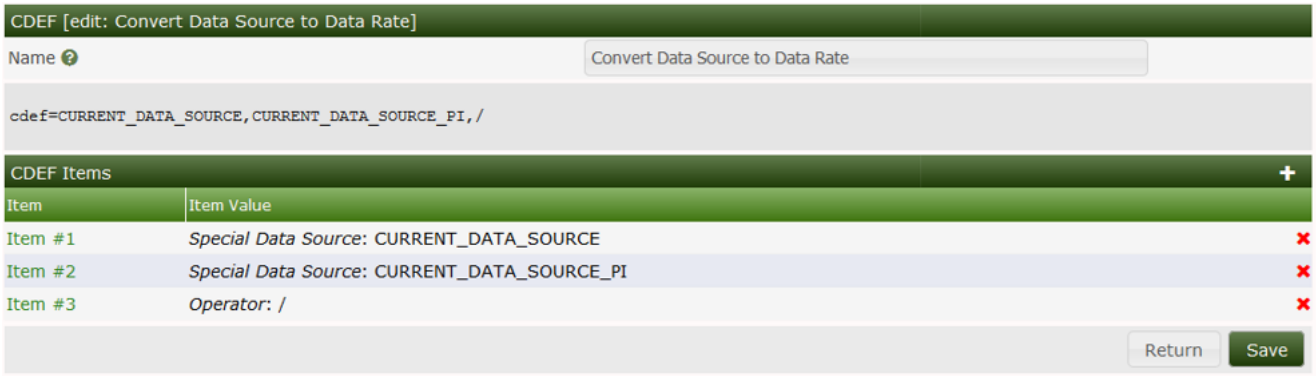
In the image below, you can see all the CDEF's that are included in Cacti by default. There are quite a few of them. Many of the formulas in use are quite simple. If you want to view Tutorials on how to work with CDEF's you should go to the RRDtool Tutorial. There is also documentation at the RRDtool Website.

In this image you can see that you have the ability to Delete or Duplicate a CDEF, but note you will not be able to Delete any CDEF that is associated with a Cacti **Graph** or **Graph Template**.



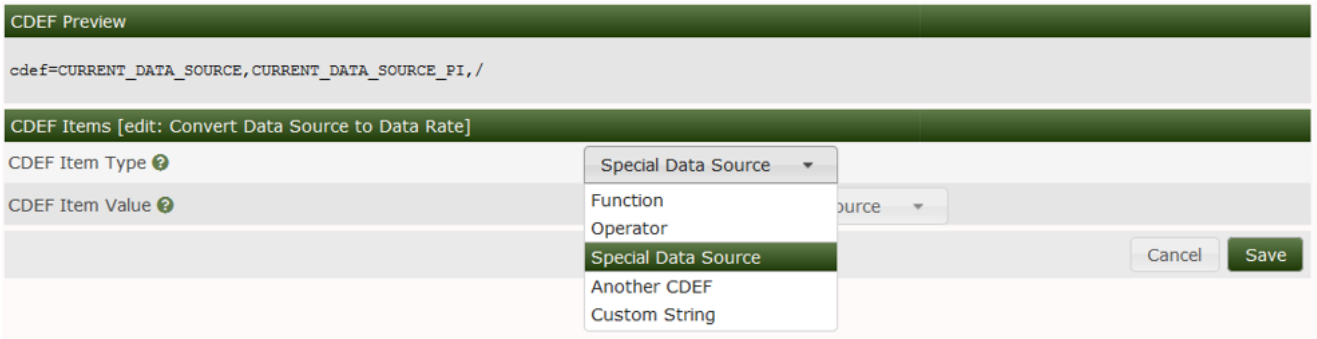
When you Click on the CDEF's name, you will enter into an Edit screen. From there you will see an ordered list of your Stack. It normally will begin with something like the CURRENT_DATA_SOURCE which means that when you Add a **Graph Item** to either a **Graph Template** or **Graph**, you can select a CDEF. The **Data Source** associated with that **Graph Item** is the CURRENT_DATA_SOURCE.

After that, you may see a number, followed by a math operator. That is the simplest form of a CDEF. If you have drag & drop enabled, you can re-order the CDEF items using drag & drop. Otherwise you will see arrows that allow you to move the CDEF Items up and down.



When editing a CDEF, the first decision is what Type of Data you want to put on the Stack. Your options are shown in the image below. They include:

Name	Description
Function	A mathematical function that we will describe below
Operator	Common mathematical operators including (+, -, *, /, and %)
Another CDEF	Another Cacti CDEF. That could be confusing
Special Data Source	A value drawn from the Graph, the Data Source, or the poller, described below
Custom String	Something like a number, a 'U' or 'Nan' for example



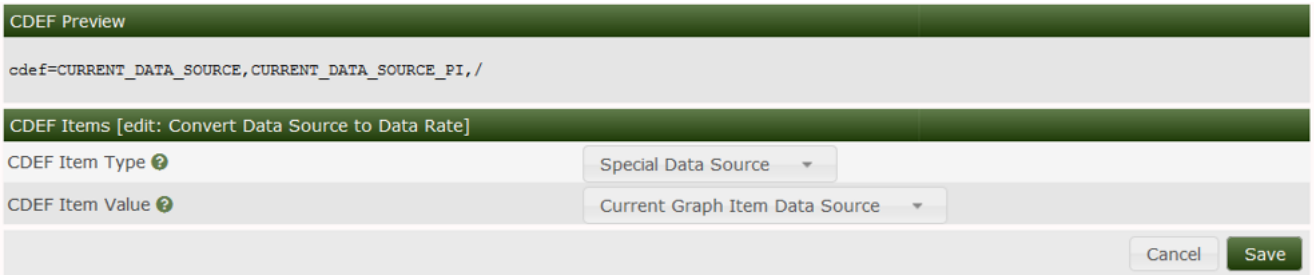
Special Data Sources

In this next image, you will find a CDEF Item in the process of being added. Note that when you pick `Special Data Source` you have a drop-down that appears with the flavor of `Special Data Source`. There are many. They include:

Name	Description
Current Graph Item Data Source	The value of the Data Source associated with the Graph Item
Current Graph Item Polling Interval	This value is otherwise known as the Step in RRDtool terminology
All Data Sources (Do not Include Duplicates)	The total of all Data Sources removing any duplicate DEF's
All Data Sources (Include Duplicates)	Add the values from all the Data Sources whether or not they are duplicated
All Similar Data Sources (Do not Include Duplicates)	Means all Data Sources with the same RRDtool Data Source name like traffic_in, and traffic_out
All Similar Data Sources (Do not Include Duplicates) Polling Interval	The max of the poller intervals returned from all similar Data Sources
All Similar Data Sources (Include Duplicates)	Add the values from all similar Data Sources whether or not they are duplicated
Current Data Source Item: Minimum Value	The RRDtool minimum value of the Current Data Source

Name	Description
Current Data Source Item: Maximum Value	The RRDtool maximum value of the Current Data Source
Current Data Source Item: Least Squares Line Function	The RRDtool LSLINT, LSLSLOPE and LSLCORREL set as a least squares line
Current Data Source Item: Least Squares Line Y-intercept	The y-intercept of the least squares line for the Current Data Source
Current Data Source Item: Least Squares Line Slope	The slope of the least squares line for the Current Data Source
Current Data Source Item: Least Squares Line Correlation Coefficient	The correlation coefficient of that least squares line
Graph: Lower Limit	The lower Limit of the Graph
Graph: Upper Limit	The upper Limit of the Graph
Count of All Data Sources (Do not Include Duplicates)	The total count of all Data Sources without Duplication
Count of All Data Sources (Include Duplicates)	The total count of all Data Sources including Duplicates
Count of All Similar Data Sources (Do not Include Duplicates)	The total count of Data Sources with the same RRDtool Data Source Name, counted once each
Count of All Similar Data Sources (Include Duplicates)	The total count of Data Sources with the same RRDtool Data Source Name, including Duplicates

As you can see there is quite a bit of information that can be pulled from RRDtool for performing Graphical manipulation of Data.



CDEF Functions

This list of CDEF functions is long and it's best to refer directly to the RRDtool Manual for meanings and examples of their use. Cacti supports most of them. If you find one that is not supported, open an Issue on GitHub.

Copyright (c) 2004-2026 The Cacti Group

VDEFs

Background

VDEF's in Cacti are a one to one analog to VDEF's in RRDtool. Cacti simply provides an interface to create and manage them. Once the VDEF's are created in Cacti they can be imported and exported globally.

VDEF's are mathematical formulas that, instead of operating on just one Data Point in the RRDfile, operate on the entire Data Set in the time range selected. So, VDEF's become a useful tool to replace the Nth Percentile and Bandwidth functions in Cacti with the VDEF equivalents.

They are referenced in Cacti graphs as VNAMES. However, as mentioned before Cacti automatically assigns VNAMES. So, to perform complex operations, you will have to experiment to determine the VNAME of your VDEF as you build your **Graph Template**. But in their simplest forms, they are quite easy to use.

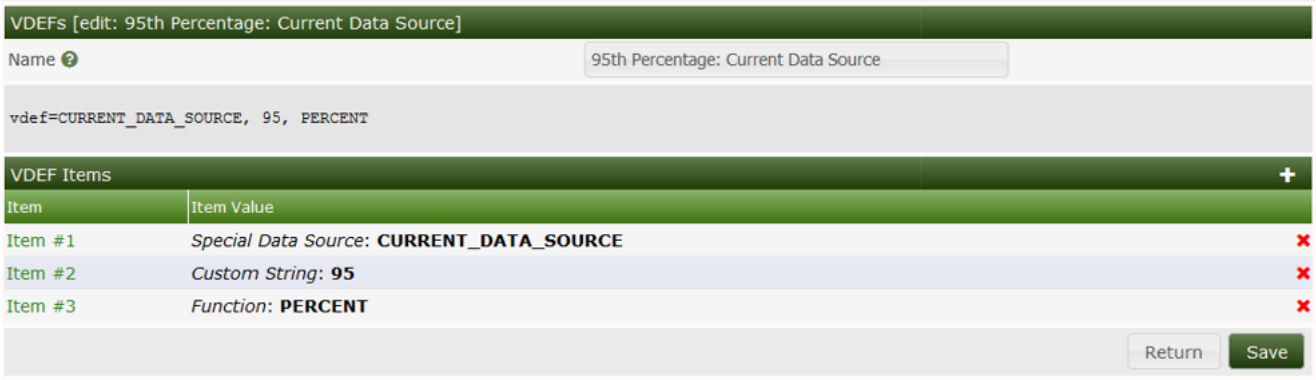
The format of the mathematical formulas is called Reverse Polish Notation (RPN). RPN was and is an early form of how Engineers entered equations into early HP and other Calculators to solve Engineering problems. The reason we still use it today is that it follows a simple Stack principle. In other words, it's not broken.

VDEF Interface

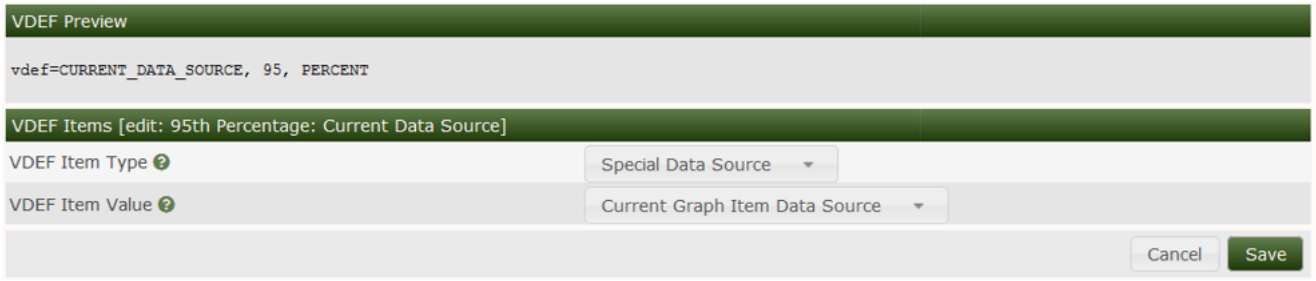
In the image below, you can see all the VDEF's that are included in Cacti by default. They are included just to get you thinking about how you would use them. You can see straight away the first and the last items are 95th Percentile and Bandwidth (or Total) VDEF's. Other examples are pretty self explanatory.

You will also note that you can Delete or Duplicate the VDEF's, but as with CDEF's you can not Delete a VDEF that is being used in a **Graph**.

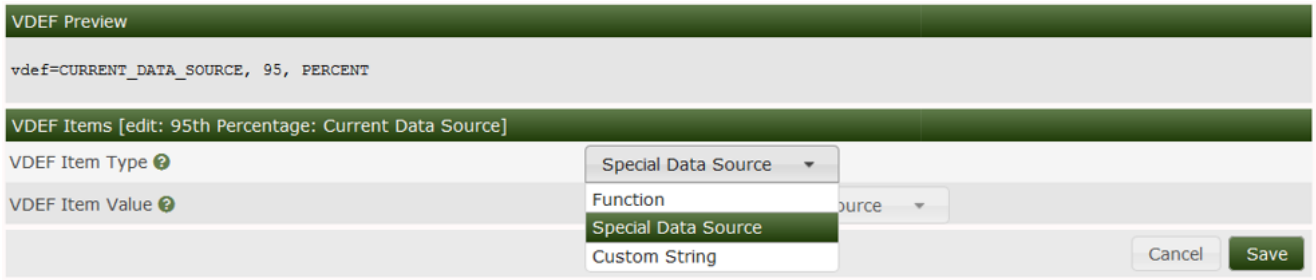
In the image below, you can also see that their interface closely resembles the CDEF Interface. Unlike the CDEF's, there is presently only one `Special Data Source` and that's the Current Data Source, which makes sense when you think about it.

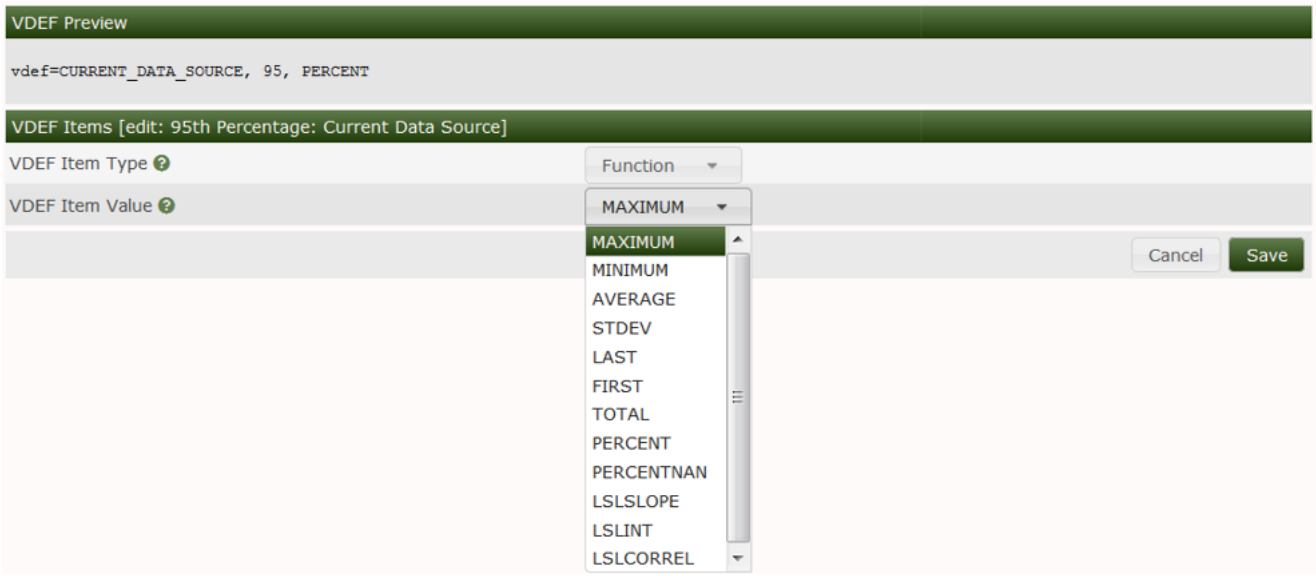


In the image below, you can see an example of a Single completed VDEF for 95th Percentile of the Current Data Source. Pretty straightforward.



In the next two images, you can see the VDEF Item Types and the complete list of VDEF functions. Again, you can reference the RRDtool Documentation for usage examples for more information.





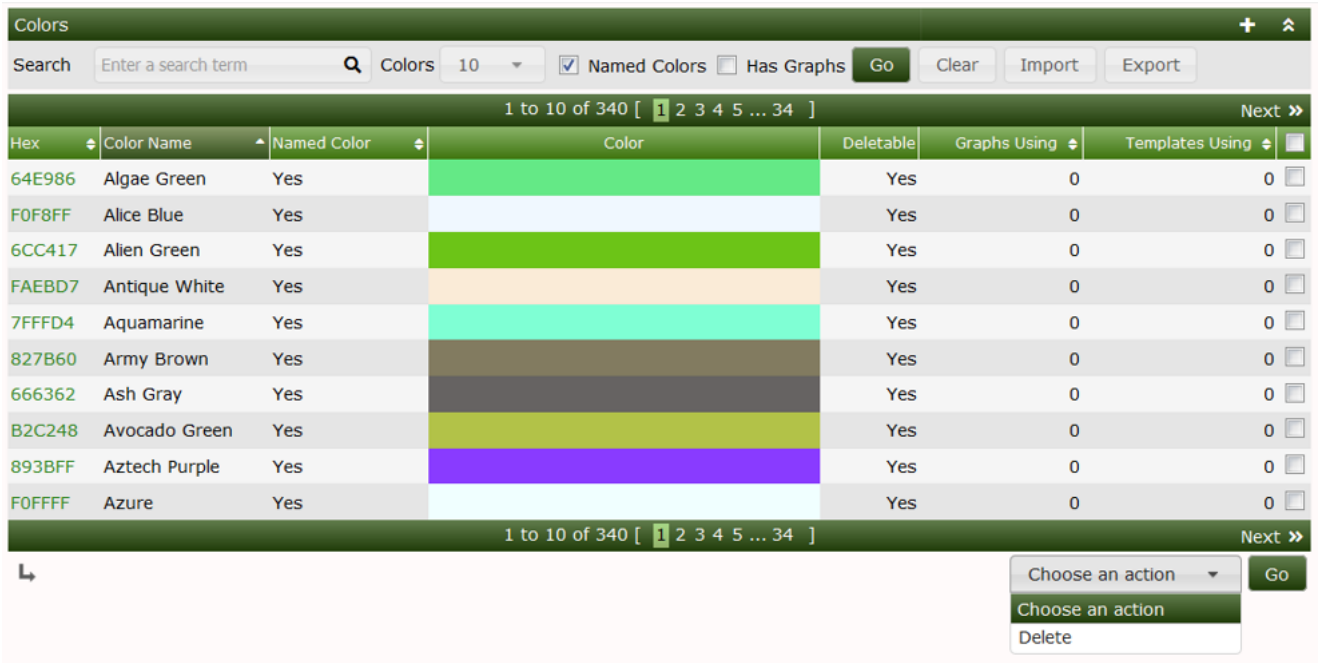
Copyright (c) 2004-2026 The Cacti Group

Colors

Colors in Cacti come in two types. They maintain a list of colors that can be used in Cacti for **Graph Templates** and **Graphs**.

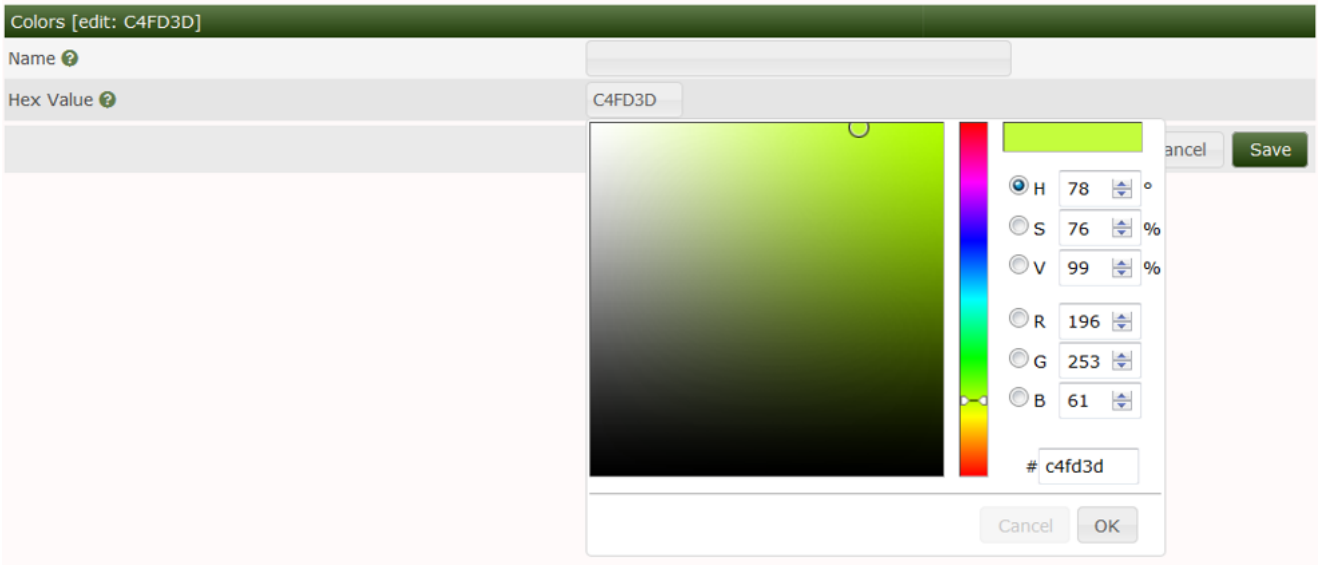
The first type is the legacy Cacti colors imported from earlier versions of Cacti. Starting with Cacti 1.0, Cacti supports the concept of *Named Colors*, which come from a well known dictionary of well defined colors. *Named Colors* are read only in Cacti, while the legacy Cacti colors can be named as the Cacti administrator desires.

The image below shows a list of Cacti *Named Colors*. You can see that there is a search bar and check boxes for showing just *Named Colors* or all colors, and another check box for showing Colors that are used in **Graph Templates** and **Graphs**. You can not remove either the *Named Colors* or Colors that are in use by either a **Graph Template** or **Graph**.



The image below is the Color edit screen. If the color is read only, you can open this page, but not change any of the details. For legacy Cacti colors, or for Colors added by the Cacti Administrator, you can change the name and the Color hex value.

Below, you can see the color picker that allows you to pick a color visually. You can also use the hex values that were supported in earlier Cacti versions.



Copyright (c) 2004-2026 The Cacti Group

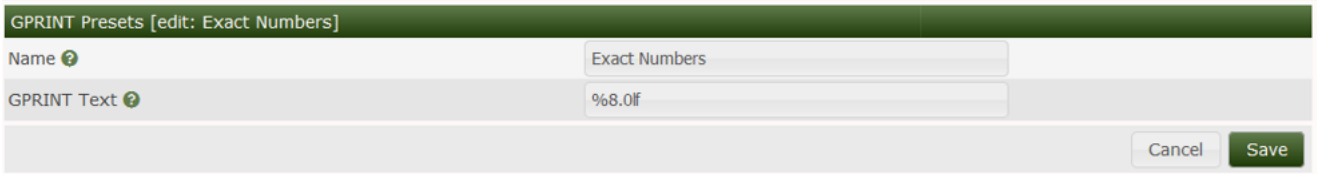
GPRINT Presets

GPRINT Presets are an object in Cacti used for formatting Legend output on RRDtool Graphs. These GPRINT Presets closely follow printf(), sprintf() ANSI syntax. They are simple to create and edit. You can find the format strings on the RRDtool Website.

The stock GPRINT Presets are generally sufficient for most **Graph Templates**, but they can easily be extended, and are a part of the Graph Template Export XML file. So, they are portable once created.

In the image below, you can see the default GPRINT Presets and the actions that can be taken on them. Keep in mind that once a GPRINT Preset is being used by a **Graph** or a **Graph Template**, it cannot be removed.

Below you can see the GPRINT Preset edit screen.



Copyright (c) 2004-2026 The Cacti Group

Variables

Graph Variables

Graphs Variables are special use replacement variables used to take information from the following objects and use them in the context of a Cacti Graph:

- **Cacti Device** - host information
- **Data Query** - input query meta data
- **Data Input** - input variables
- **Nth Percentile** - formulas

It also allows you additional one single use replacement variables like:

- **Data Source Name** - The name of your Data Source
- **Date Time** - The date and time the graph was rendered.

In the examples below, you will be able to see how to use these values. They can be used in:

1. Graph and Graph Template - Titles
2. Graph and Graph Template - Comments
3. Data Query - Suggested names for Graphs and Data Sources
4. Data Source - Min and Max Data Source Limits
5. Aggregate Graphs and Templates - Prefixes

All Graph Variables are enclosed by pipe characters as shown in the examples below.

Date/Time

```
|date_time|
```

This variable will place the date and time of the last poller run on the graph. It can be used to replicate MRTG's "graph last updated" feature.

Data Source Title

```
|data_source_title|
```

This variable, `|data_source_title|`, gets replaced with the name of the data source associated with a particular graph item in a graph template. This change allows the creation of generic graph templates (2-variable line graph, 3-variable stack, etc.) which can be reused with large numbers of data sources. See attached examples to understand usage.

Graph Template Items [edit: An Example Graph Template]

Graph Item	Data Source	Graph Item Type	CF Type	Alpha %	Item Color
Item # 1	**Host MIB - Hard Drive Space (hdd_total): |data_source_title|**	**LINE1**	**AVERAGE**	**100%**	**0000FF**
Item # 2	Host MIB - Hard Drive Space (hdd_total): Cur:	GPRINT	LAST		
Item # 3	Host MIB - Hard Drive Space (hdd_total): Avg:	GPRINT	AVERAGE		
Item # 4	Host MIB - Hard Drive Space (hdd_total): Max:<HR>	GPRINT	MAX		
Item # 5	**Host MIB - Hard Drive Space (hdd_used): |data_source_title|**	**LINE1**	**AVERAGE**	**100%**	**FF0000**
Item # 6	Host MIB - Hard Drive Space (hdd_used): Cur:	GPRINT	LAST		
Item # 7	Host MIB - Hard Drive Space (hdd_used): Avg:	GPRINT	AVERAGE		
Item # 8	Host MIB - Hard Drive Space (hdd_used): Max:<HR>	GPRINT	MAX		

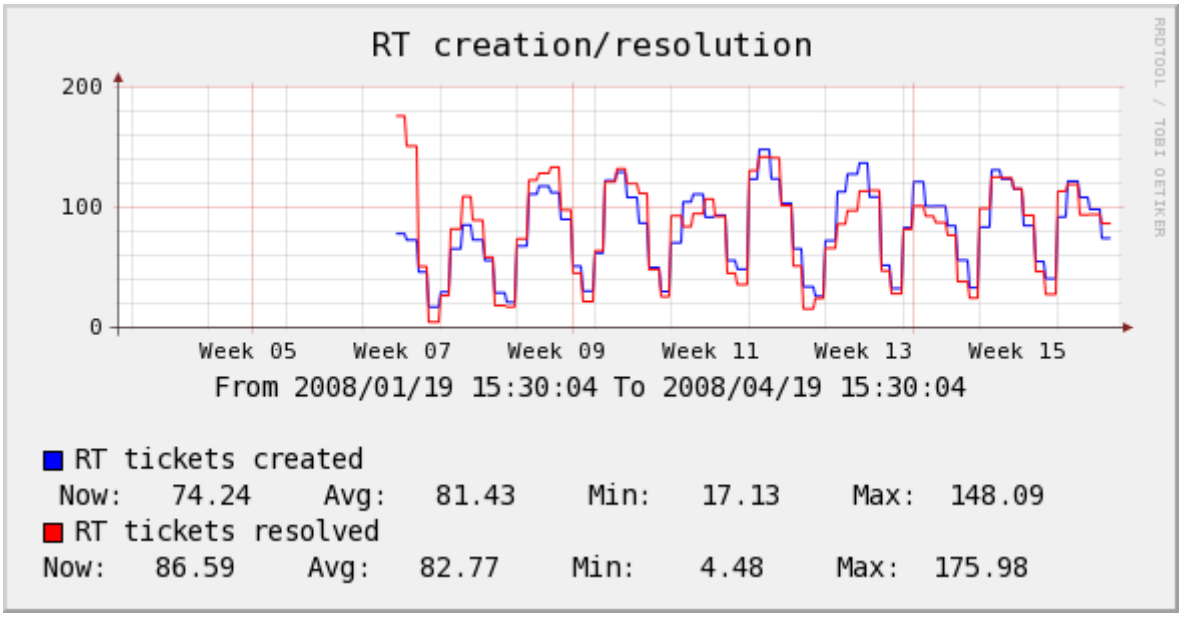
Graph Item Inputs

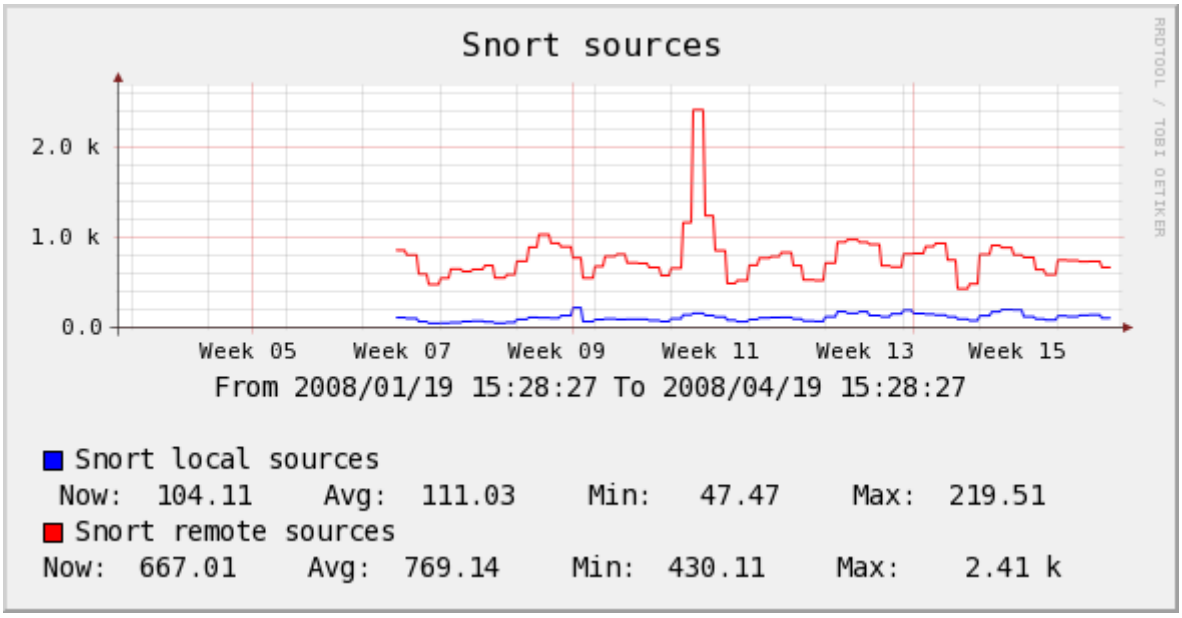
Name
Data Source [hdd_total]
Data Source [hdd_used]

Graph Template [edit: An Example Graph Template]

Name	An Example Graph Template
Multiple Instances	

The example graphs below, show how the output may look, but for a slightly different Case.





Data Query Fields

```
|query_field_name|
```

You can place the value of any data query field on the by including this variable. Make sure to substitute "field_name" with the actual data query field name.

```
|query_ifIP|
```

```
|query_dskPath|
```

Data Input Arguments

```
|input_hostname|
```

```
|input_ip_address|
```

Nth Percentile

```
|[0-9]:(bits|bytes):[0-9]:(current|total|max|total_peak|all_max_current|all_max_peak|aggregate_max|aggregate_sum|
aggregate_current|aggregate):[0-9]|
```

Nth percentile is often used by hosting providers to bill customers based on their peak traffic usage while ignoring their top (100 - Nth) percent. This way if a customer has a fairly consistent traffic pattern and decides to download a huge file one day, the large spike will be ignored. Common Nth percentile is 95, which would cut off the top 5% of the traffic.

The Nth Percentile variables, when placed in Graph Comments will be detected and the syntax will be parsed and the resulting numeric value will replace the formula. They are used in both 95th Percentile and Bandwidth Traffic Graphs today.

In Cacti, Nth percentile works just like any other graph variable. To use this variable you must give it five arguments:

Table 15-1. Nth Percentile Argument Description

Name	Description
[0-9]	The first argument is a number between 1 and 99. This number represents the Nth percentile that you would like to calculate.
(bits | bytes)	Choose whether you want to represent your Nth percentile in bits or bytes. The only valid values for this field are 'bits' and 'bytes'.
[0-9]	This field determines the power of 10 divisor that will be applied to your Nth percentile number. Enter '0' to leave the number alone, '3' for kilo or '6' for mega, etc.
(current | total | max | total_peak | all_max_current | all_max_peak | aggregate_max | aggregate_sum | aggregate)	Nth Percentile Type - Refer to Nth Types below
[0-9]	Enter the number of digits to use for floating point precision when printing out the final number. The default value for this field is two decimal places.

Table 15-2. Nth Percentile Types

Type	Description
current	Calculates the Nth percentile based off the selected data source on the graph item which the variable is used. This type requires a selected Data Source Item in the graph item in which it is defined.
total	Calculates the Nth percentile based off the selected data source on the graph item which the variable is used. But unlike current, this function totals the Nth percentile results of all same named data source names on the graph. Example, all traffic_in would have their Nth percentile calculated and then totaled. This type requires a selected Data Source Item in the graph item in which it is defined.
max	Calculates the Nth percentile based off the selected data source on the graph item which the variable is used, but it is important to note that it selects the higher of the values for each row in data source and then uses the resulting set to calculate the Nth percentile. Example, you have selected a interface data source for traffic_in, this will evaluate traffic_out and traffic_in for that selected data source (RRDtool file) and select the higher of the two values for each row of data. The resulting max values are used to calculate the Nth percentile value. This type requires a selected Data Source Item in the graph item in which it is defined.
total_peak	Calculates the Nth percentile based on the max for each data source defined on the graph. Example, the max is taken for each data source defined on the graph, but it's the max of traffic_in or traffic_out for that data source (RRDtool file). The resulting max values are summed and returned.
all_max_current	Calculates the Nth percentile based off the selected data source on the graph item which the variable is used, but it only selects the highest (max) value of all the data source items on the graph. Example, there are 3 data sources defined on the graph, the selected data source is traffic_in, all data sources for traffic_in will be compared against each other and the highest Nth value of all graph items will be selected as the result. This type requires a selected Data Source Item in the graph item in which it is defined.
all_max_peak	Calculates the Nth percentile by selecting the highest (max) calculated max Nth percentile value for each data source (RRDtool file) selected on the graph. Example, 3 data sources are defined on the graph, for each data source, the max Nth percentile is calculated, and out of the resulting set of values, the highest value is selected and returned.
aggregate_max	Calculates the Nth percentile by selecting the highest value for each summed value of like data sources and selecting the maximum value of that set to calculate the Nth percentile value. Example, you have a graph with 5 traffic_in and 18 traffic_out data sources. The traffic_in rows are summed together, then the traffic_out rows are summed together, then for each row, the higher of the 2 values is selected. The Nth percentile is calculated from the resulting maximum values.
aggregate_sum	

Type	Description
	Calculates the Nth percentile by summing all data sources on the graph row for row and calculates the Nth value on the resulting summed data. Example, you have a graph with 4 traffic_in and 3 traffic_out data sources defined. All the defined traffic_in and traffic_out are summed together and then the Nth percentile value is calculated from that set.
aggregate	Calculates the Nth percentile by summing like data source names row for row from each data source defined on the graph and then returning the highest Nth percent value calculated for like data source names. Example, you have a graph with 3 traffic_out and 5 traffic_in data sources defined. The traffic_in and traffic_out are summed then the Nth value is calculated and the higher of the 2 values is returned as the variable.

> **Note**
>
> All data sources used in a graph MUST have the same RRA definitions. If the number of rows do not match when calculations are attempted, the results may produce errors and invalid results. Please also be aware the aggregate functions take more memory and processor power when used. Recommended minimum of 64 megs memory limit for PHP when querying large ranges or graphs with a large amount of aggregated data sources.

Example 15-3. Output in the following format, for 95th percentile: 42.58 mbit

```
|95:bits:6:max:2|
```

Bandwidth Summation

```
|sum:([0-9]|auto):(current|total):([0-9]):([0-9]+|auto)|
```

Bandwidth summation is useful for summing up all values in an RRDfile for a given time range. This is typically useful on traffic graphs where you can see a total of all traffic that has gone through an interface in a given time period.

In Cacti, bandwidth summation works just like any other graph variable. To use this variable you must give it three arguments:

Table 15-3. Bandwidth Summation Argument Description

Name	Description
([0-9] | auto)	This field determines the power of 10 divisor that will be applied to your number. Enter '0' to leave the number alone, '3' for kilo or '6' for mega, etc. You can also enter 'auto' for this field to have Cacti automatically scale the number and insert the appropriate label.
(current | total)	You can choose to calculate the summation based on the current data source or a total of all data sources used on the graph. The only valid values for this field are 'current' and 'total'.
[0-9]	Enter the number of digits to use for floating point precision when printing out the final number. The default value for this field is two decimal places.
([0-9]+ | auto)	Enter the number of seconds in the past to perform the summation calculation for. For instance '86400' for 24 hours, '172800' for 48 hours, etc. You can also enter 'auto' for this field to have Cacti use the graph timespan.

Example 15-4. Output in the following format: 36.47 GB

```
|sum:auto:current:2:auto|
```

Host Variables

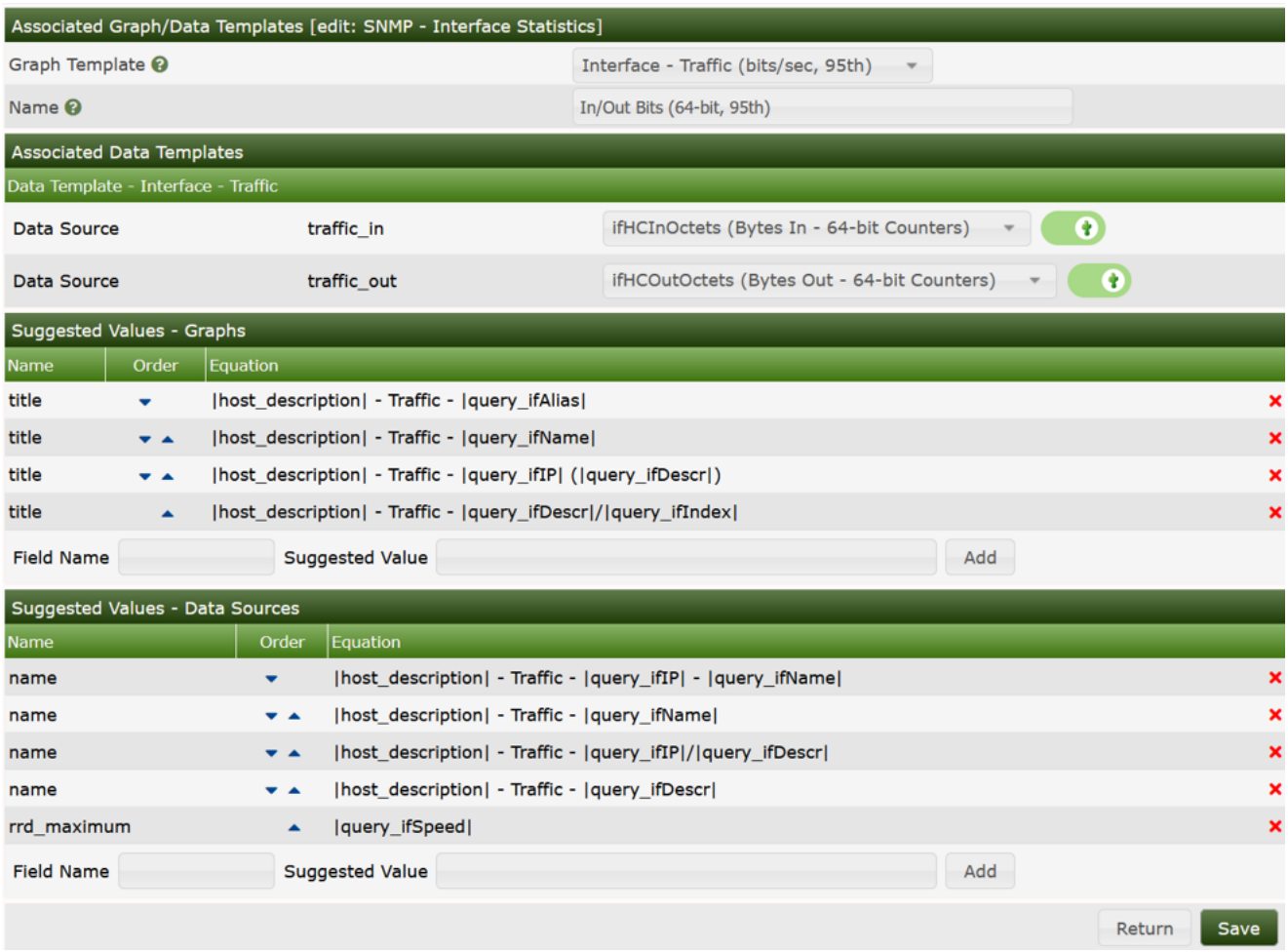
Host variables represent host data and can be placed in graph or data source titles. The following table describes which host variables can be used and which host fields the correspond to.

Table 15-4. Host Variables

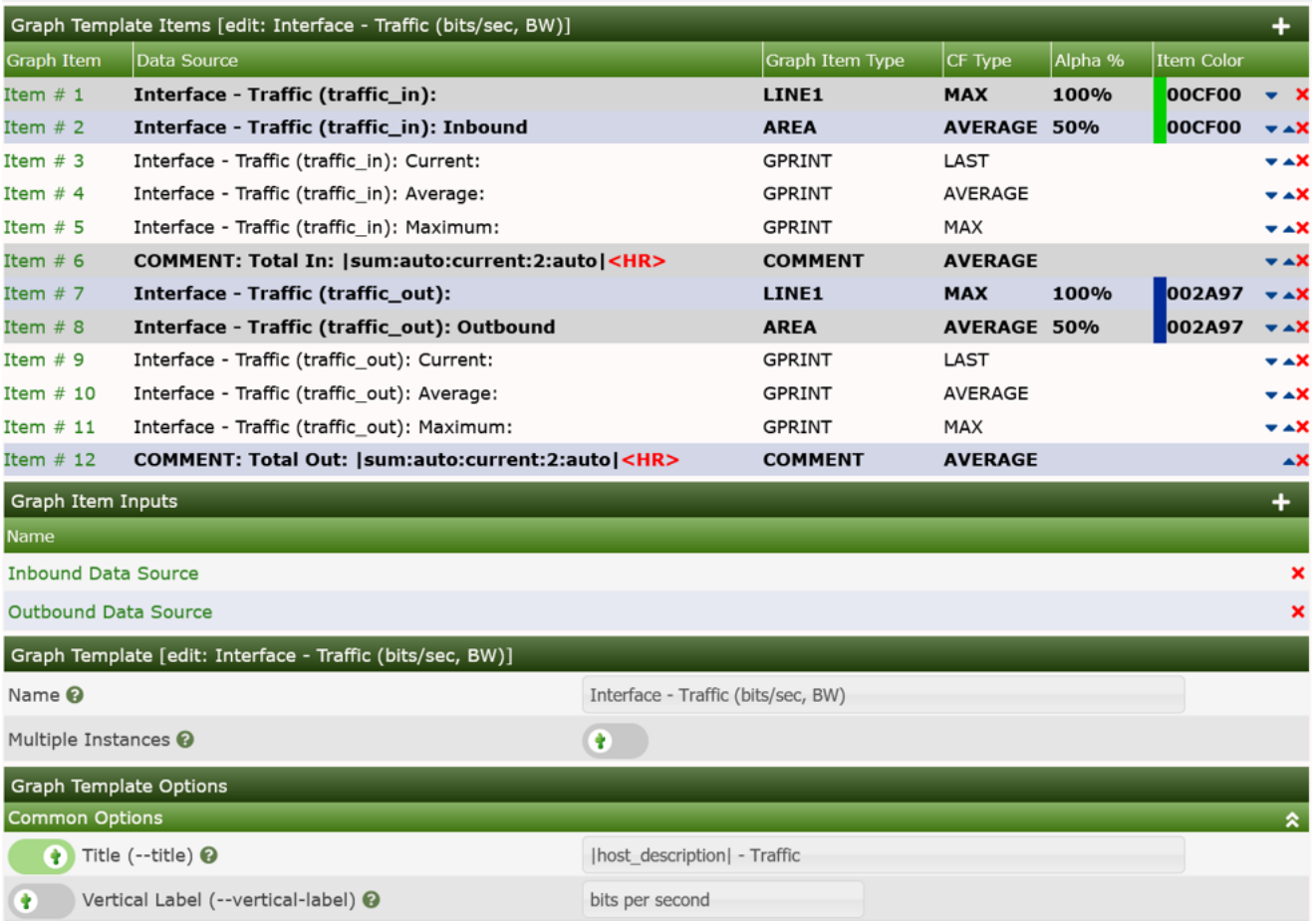
Variable Name	Description
|host_availability|	Availability of host
|host_avg_time|	Average time for host
|host_cur_time|	Current time for host
|host_description|	Description
|host_external_id|	ID associated with external system
|host_hostname|	Hostname
|host_id|	Numerical ID of the Host as it is maintained in the MySQL Tables
|host_location|	Location
|host_max_oids|	SNMP Max OIDs for snmpbulkwalk
|host_notes|	Administrative Notes
|host_ping_retries|	Ping Retries (Downed Host Detection)
|host_polling_time|	Last polling time
|host_snmp_auth_protocol|	SNMP Authentication Protocol (v3)
|host_snmp_community|	SNMP Community
|host_snmp_context|	SNMP Context (v3)
|host_snmp_engine_id|	SNMP Engine Id (v3)
|host_snmp_password|	SNMP Password (v3)
|host_snmp_port|	SNMP Port
|host_snmp_priv_passphrase|	SNMP Privacy Passphrase (v3)
|host_snmp_priv_protocol|	SNMP Privacy Protocol (v3)
|host_snmp_sysContact|	SNMP Contact
|host_snmp_sysDescr|	SNMP Description
|host_snmp_sysLocation|	SNMP Location
|host_snmp_sysName|	SNMP System Name
|host_snmp_sysObjectID|	SNMP ObjectID
|host_snmp_sysUpTimeInstance|	SNMP Up Time
|host_snmp_timeout|	SNMP Timeout
|host_snmp_username|	SNMP Username (v3)
|host_snmp_version|	SNMP Version
|host_uptime|	SNMP Uptime

Example Use

Below you can see the example in the `SNMP - Interface Statistics` Data Query of how they are used for suggested names.



And in Bandwidth Graph Template.



In the images above, you can see both the Title of the Graph and the Comments showing the use of the bandwidth replacement variables.

Copyright (c) 2004-2026 The Cacti Group

RRDtool Specific Features

Cacti supports most RRDtool features. In this chapter, many of those features, and some Cacti implementation specifics around them will be discussed. As RRDtool is a continually changing Times Series Database (TSDB) and **Graph** rendering framework, this guide may be a little behind on all the RRDtool features that Cacti supports.

Database Data Types

There are several different Database **Data Types** that RRDfile supports. They include all those listed below. Cacti claims to support all, but `COMPUTE` remains untested.

Type	Supported	Description
GAUGE	Yes	This is as the term refers to a floating point number that tracks the value reported directly. Unlike the COUNTER, it has not comprehension of past data.
COUNTER	Yes	This is a floating point number that tracks the different between samples to come up with effectively a rate calculation. Used for things like Traffic Graphs where the number reported in SNMP continually increases
DCOUNTER	Yes	This is the same as a COUNTER, but in Double Precision.
DERIVE	Yes	This data type measures the Rate of Change of a value, otherwise known as the first derivative to the value. As such, it relies on previous data in order to report the change.
DDERIVE	Yes	This is the same as the DERIVE, but in Double Precision
ABSOLUTE	Yes	Is for counters that get reset upon reading. This is often used for fast counters. It is also similar to the DERIVE, or acts very much like it.
COMPUTE	Yes	Though Cacti claims to support this Data Type. It's not clear that it's presently working.

> **NOTE**: Presently `Holt-Winters Forecasting` is not supported in Cacti and The Cacti Group has no plans on supporting it. If you would like `Holt-Winters Forecasting` supported in Cacti, you will need to submit a pull request including documentation if you require it's functionality.

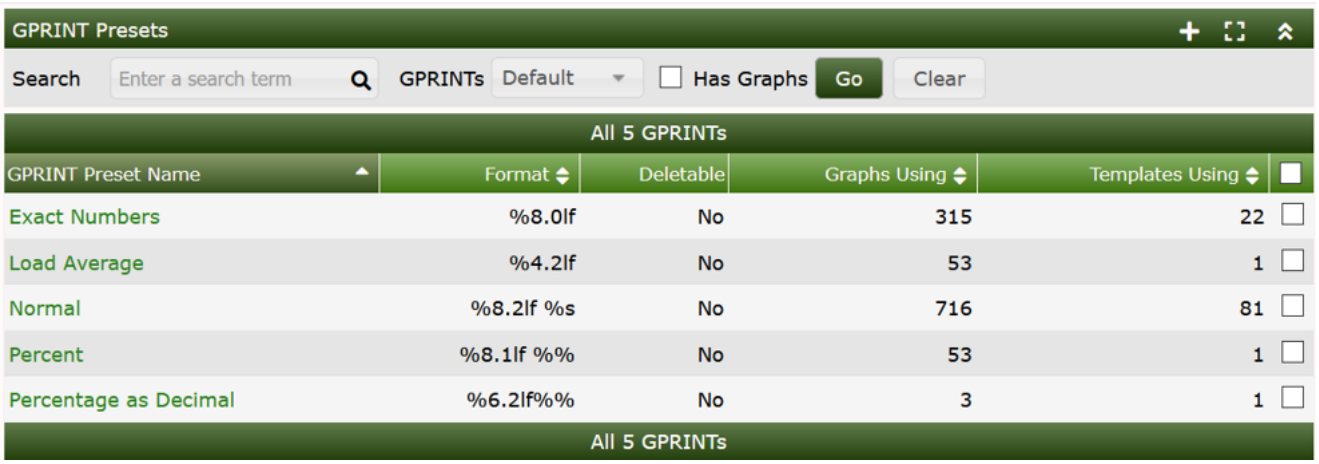
GPRINT Presets

A GPRINT is a graph item type that enables you to print the values of data sources on a graph. They are typically used to represent legend values on the graph. The output format of these numbers are controlled by a printf-like format string. Cacti enables you to keep a global list of these strings that can be applied to any graph item throughout Cacti. You can see the list of available formats described on the RRDtool Website.

Creating a GPRINT Preset

To create a new GPRINT preset, select the Graph Management menu item under the Management heading, and select GPRINT Presets. Click Add to the right and you will be presented with an edit page containing two fields. Enter a name for your GPRINT preset, and the actual printf-like string in the GPRINT Text field. When you are finished, click the Create button to create your new GPRINT preset.

In the image below, you can see the default GPRINT Presets included in the base Cacti. These generally cover just about all use cases presently.



GPRINT Presets

Search | Enter a search term | GPRINTs Default | Has Graphs | Go | Clear

All 5 GPRINTs

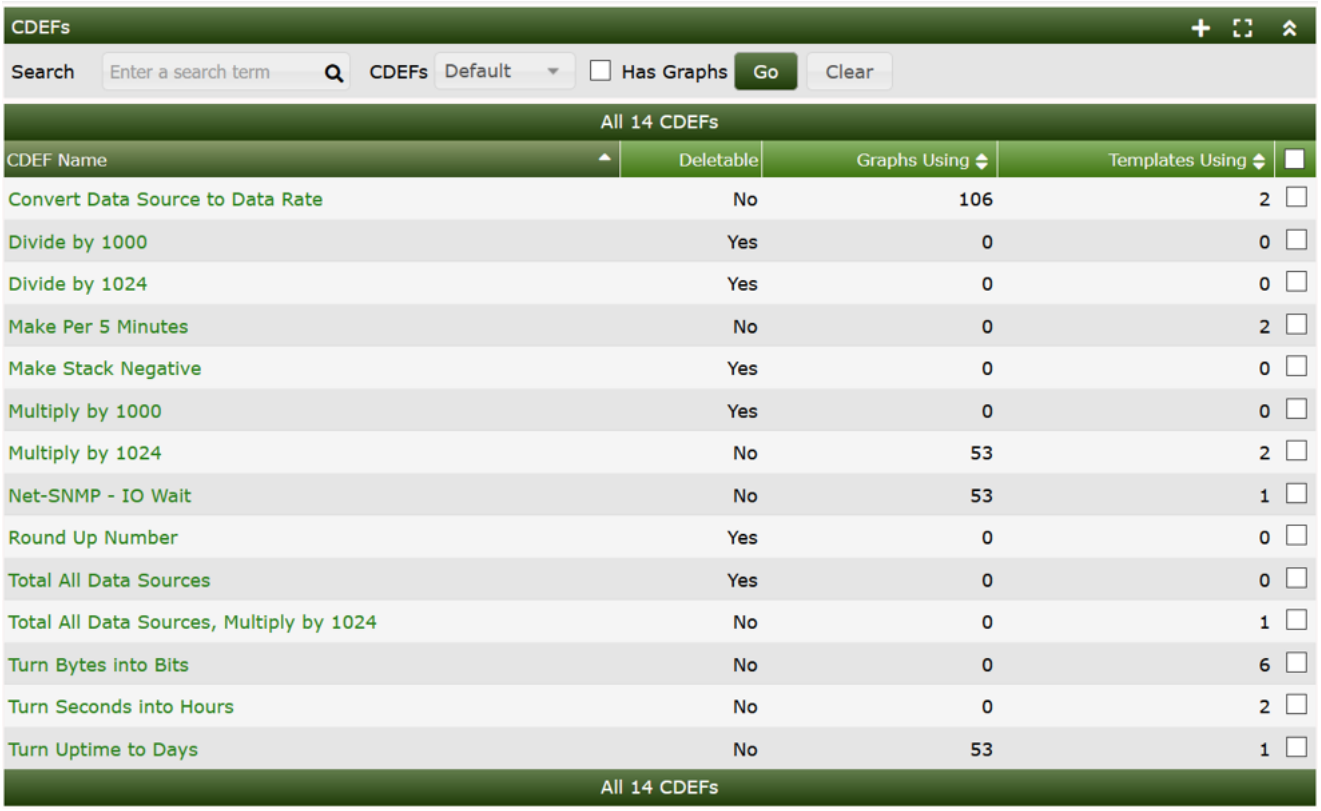
GPRINT Preset Name	Format	Deletable	Graphs Using	Templates Using
Exact Numbers	%8.0lf	No	315	22
Load Average	%4.2lf	No	53	1
Normal	%8.2lf %s	No	716	81
Percent	%8.1lf %%	No	53	1
Percentage as Decimal	%6.2lf%%	No	3	1

All 5 GPRINTs

CDEF's

CDEF's allow you to apply mathematical functions to graph data to alter output. The concept of a CDEF comes straight from RRDtool, and are written in reverse polish notation (RPN). For more information regarding the syntax of CDEF's, check out the CDEF tutorial.

From the image below, you can see that Cacti provides a rich set of CDEF's out of the box. These CDEF's can be imported and exported and are globally assigned for portability.



CDEFs

Search | Enter a search term | CDEFs Default | Has Graphs | Go | Clear

All 14 CDEFs

CDEF Name	Deletable	Graphs Using	Templates Using
Convert Data Source to Data Rate	No	106	2
Divide by 1000	Yes	0	0
Divide by 1024	Yes	0	0
Make Per 5 Minutes	No	0	2
Make Stack Negative	Yes	0	0
Multiply by 1000	Yes	0	0
Multiply by 1024	No	53	2
Net-SNMP - IO Wait	No	53	1
Round Up Number	Yes	0	0
Total All Data Sources	Yes	0	0
Total All Data Sources, Multiply by 1024	No	0	1
Turn Bytes into Bits	No	0	6
Turn Seconds into Hours	No	0	2
Turn Uptime to Days	No	53	1

All 14 CDEFs

Creating a CDEF

To create a new CDEF in Cacti, select CDEF's under `Console > Presets` sub-menu section. Once at this screen, click Add to the right. You will be prompted for a CDEF name, for which you can type anything used to describe your CDEF. Click the Create button so you are redirected back to the edit page, now with an empty CDEF Items box. Construct your CDEF by

adding an item for each element in the CDEF string, common types such as operators and functions are enumerated for your convenience. Below is a basic description of each CDEF item type. You can see from the image below that CDEF's follow a Reverse Polish Notation (RPN), which was a very popular way of computing back in the 70's and 80's when computing devices had much less memory. However, RPN is elegant in it's simplicity.

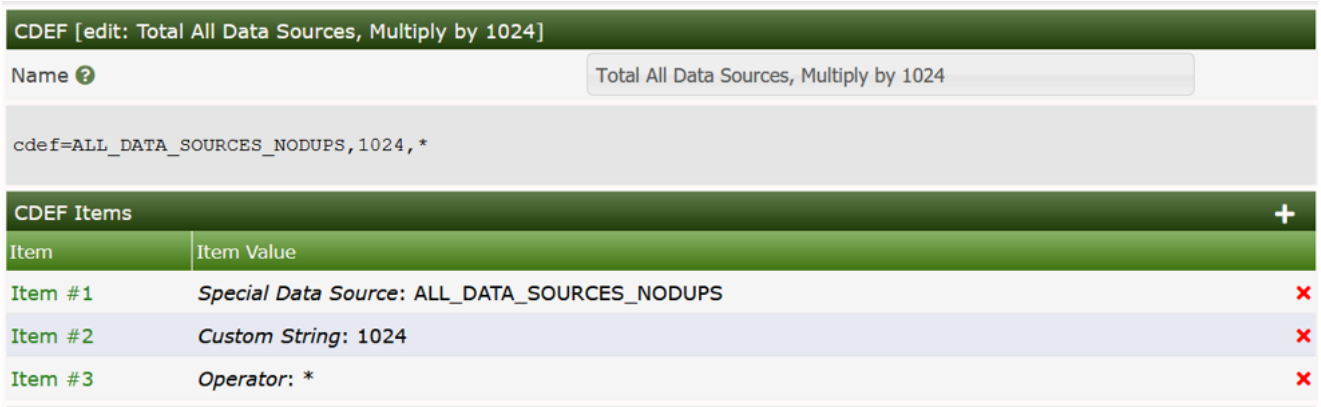


Table 19-1. CDEF Item Types

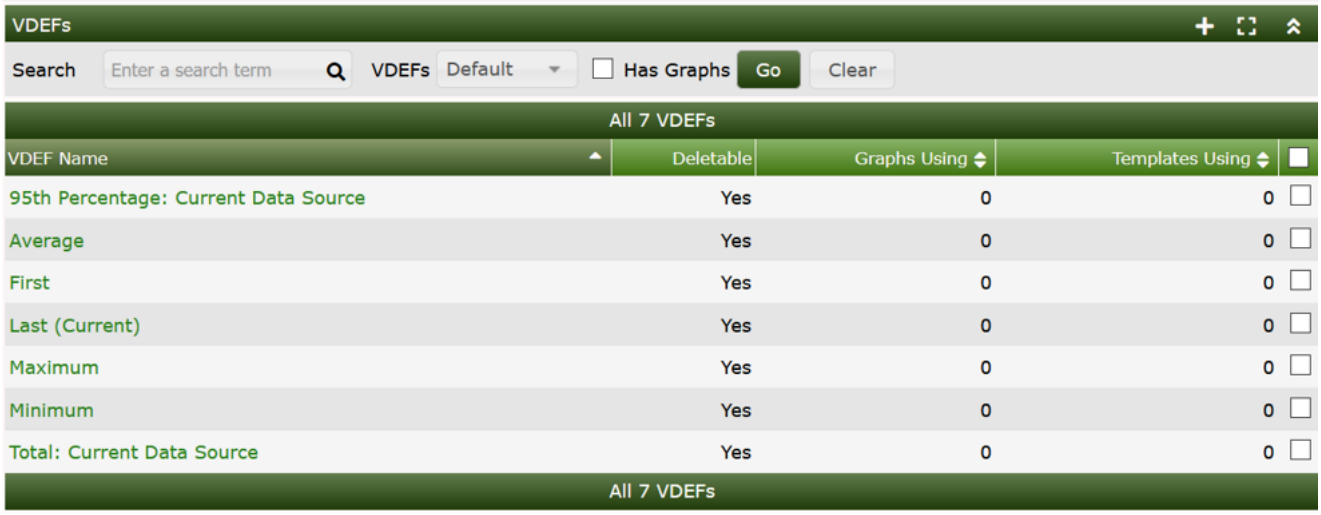
Cacti provides several CDEF Item Types to represent a **Data Source** in order to provide additional functionality. See below for the documentation. Cacti presently does not allow you to specify a VNAME for your DEFS, CDEF's and VDEF's, but instead uses a string pattern to assign them. You will have to take note of then when creating a **Graph Template**. Those Special Data Source types are documented below in the next few sections.

Type	Description
Function	You can choose a CDEF function to use as the item. The RRDtool's graph manual describes the purpose of each CDEF function.
Operator	Just your standard math operators, including modulo (%).
Special Data Source	A special data source is basically a flag to tell Cacti to do some special processing when it encounters this CDEF item. The "Current Graph Item Data Source" type basically inserts the name of the data source that is referenced by the graph item that references to this CDEF. Both of the "All Data Sources" types insert a summation of all data sources used on a graph.
Another CDEF	You can recursively use another CDEF within this CDEF.
Custom String	Sometimes it's just easier to type out the literal CDEF string manually. When referencing to data sources on the graph, remember that Cacti names them 'a', 'b', 'c', '...', starting with the first data source on the graph.

VDEF's

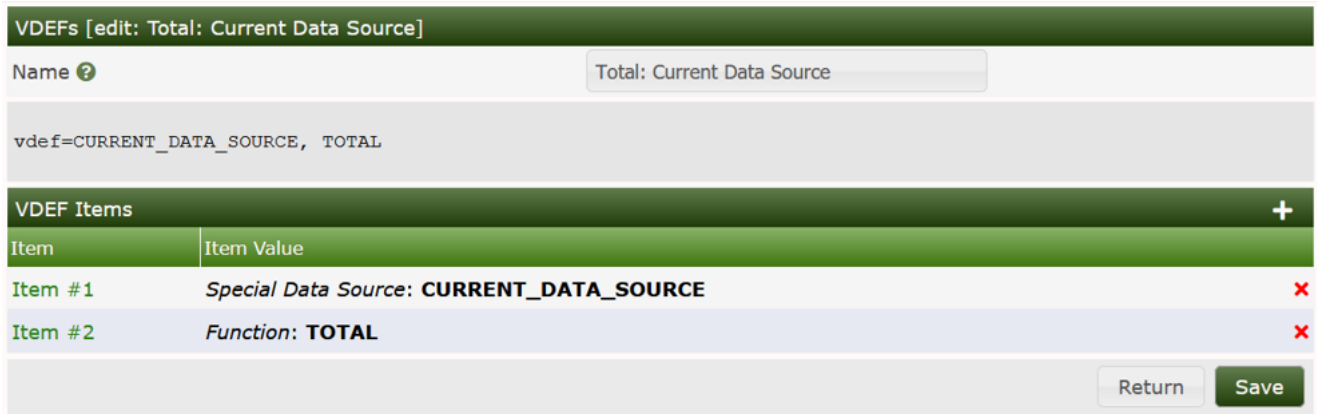
VDEF's provide greater functionality then their ancestor CDEF's and may be able to replace some legacy Cacti functionality like Nth Percentile **Graph Template** logic. They should be used whenever possible for your **Graph Templates**.

You can see from the image below, that 95th percentile and total bandwidth could be replace by using one of the VDEF's below.



Creating a VDEF

To create a new VDEF in Cacti, select VDEF's under `Console > Presets` sub-menu section. Once at this screen, click Add to the right. You will be prompted for a VDEF name, for which you can type anything used to describe your VDEF. Click the Create button so you are redirected back to the edit page, now with an empty VDEF Items box. Construct your VDEF by adding an item for each element in the VDEF string, common types such as operators and functions are enumerated for your convenience. Below is a basic description of each VDEF item type. The image below shows an example VDEF for the Bandwidth type VDEF.



For additional information on CDEF's and VDEF's see that specific section of the documentation. Additionally, you can find examples on the RRDtool Website

Special Data Sources

The Special Data Source selection introduces some variables not known to plain vanilla RRDtool. Let's spend some few words of them to unleash their power.

Table 19-2. CDEF Special Data Source

Special Data Source	Description
Current Graph Item Data Source	Will be replaced by the DEF name of the RRDtool data source referred by the graph item this CDEF is associated to.
All Data Sources (Don't Include Duplicates)	Will add up all data sources of the whole graph to form a total. A data source that appears more than once will be counted only once. Data sources that differ by consolidation functions only are NOT counted as different data sources (e.g. traffic_in:AVERAGE and traffic_in:MAX are counted only once). It is NOT required to associate the graph item to any data source!

Special Data Source	Description
All Data Sources (Include Duplicates)	Will add up all data sources of the whole graph to form a total. A data source that appears more than once will be counted for each time of it's appearance. Data sources that differ by consolidation functions only are NOT counted as different data sources (e.g. traffic_in:AVERAGE and traffic_in:MAX are counted only once). It is NOT required to associate the graph item to any data source!
Similar Data Sources (Don't Include Duplicates)	It is REQUIRED to associate the graph item to the data source that shall be totaled! Let's assume the data source is named "traffic_in". Then, cacti will add up all data sources "traffic_in" of the whole graph to form a data source specific total (e.g. Total traffic In). Data sources with different consolidation functions are counted as same data sources (e.g. traffic_in:AVERAGE and traffic_in:MAX are counted once)
Similar Data Sources (Include Duplicates)	It is REQUIRED to associate the graph item to the data source that shall be totaled! If a data source appears multiple times, it will be added this many times.
Current Data Source Item: Minimum Value	Taken from the Data Template - Data Source Item related to this graph item: fetches the minimum value defined for the given data template. Caution: This is NOT the smallest entry of the given data source!
Current Data Source Item: Maximum Value	Taken from the Data Template - Data Source Item related to this graph item: fetches the maximum value defined for the given data template. Caution: This is NOT the highest entry of the given data source!
Graph: Lower Limit	Taken from the Graph Template: fetches the Lower Limit defined to the Graph Template. This is independent of all --alt-autoscaling options. It is NOT the dynamically determined lower boundary of the graph!
Graph: Upper Limit	Taken from the Graph Template: fetches the Upper Limit defined to the Graph Template. This is independent of all --alt-autoscaling options. It is NOT the dynamically determined upper boundary of the graph!
Count of All Data Sources (Don't Include Duplicates)	Will count the number of all data sources of the whole graph. A data source that appears more than once will be counted only once. Data sources that differ by consolidation functions only are NOT counted as different data sources (e.g. traffic_in:AVERAGE and traffic_in:MAX are counted only once). It is NOT required to associate the graph item to any data source!
Count of All Data Sources (Include Duplicates)	Will count the number of all data sources of the whole graph. A data source that appears more than once will be counted for each time of it's appearance. Data sources that differ by consolidation functions only are NOT counted as different data sources (e.g. traffic_in:AVERAGE and traffic_in:MAX are counted only once). It is NOT required to associate the graph item to any data source!
Count of Similar Data Sources (Don't Include Duplicates)	It is REQUIRED to associate the graph item to the data source that shall be counted! Let's assume the data source is named "traffic_in". Then, cacti will count all data sources "traffic_in" of the whole graph. Data sources with different consolidation functions are counted as same data sources (e.g. traffic_in:AVERAGE and traffic_in:MAX are counted once)
Count of Similar Data Sources (Include Duplicates)	It is REQUIRED to associate the graph item to the data source that shall be counted! If a data source appears multiple times, it will be counted this many times.

While the `All Data Sources/Similar Data Sources` pseudo CDEF variables perform *totaling*, the `Count All Data Sources/Count Similar Data Sources` pseudo CDEF Variables simply *count* the occurrences of the related data sources. Thus, it is easy to compute e.g. an average of all similar data sources by creating the CDEF:

```
CDEF=SIMILAR_DATA_SOURCES_NODUPS,COUNT_SIMILAR_DS_NODUPS,/
```

Using Special Data Source

Let's have some examples:

This is the list of graph items defined. Please note, that Item numbers 17-20 are duplicates of Item numbers 1-4. Even if it usually does not make sense, for this discussion it is very valuable. Item numbers 21-24 are related to a `cdef=ALL_DATA_SOURCES_NODUPS`.

Graph Item	Data Source	Graph Item Type	CF Type
Item # 1	(traffic_in): 1. Target Traffic In	LINE1	AVERAGE
Item # 2	(traffic_in): Current:	GPRINT	LAST
Item # 3	(traffic_in): Average:	GPRINT	AVERAGE
Item # 4	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 5	(traffic_in): 2. Target Traffic In	LINE1	AVERAGE
Item # 6	(traffic_in): Current:	GPRINT	LAST
Item # 7	(traffic_in): Average:	GPRINT	AVERAGE
Item # 8	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 9	(traffic_out): 1. Target Traffic Out	LINE1	AVERAGE
Item # 10	(traffic_out): Current:	GPRINT	LAST
Item # 11	(traffic_out): Average:	GPRINT	AVERAGE
Item # 12	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 13	(traffic_out): 2. Target Traffic Out	LINE1	AVERAGE
Item # 14	(traffic_out): Current:	GPRINT	LAST
Item # 15	(traffic_out): Average:	GPRINT	AVERAGE
Item # 16	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 17	(traffic_in): 1. Target Again Traffic In	LINE1	AVERAGE
Item # 18	(traffic_in): Current:	GPRINT	LAST
Item # 19	(traffic_in): Average:	GPRINT	AVERAGE
Item # 20	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 21	(No Task): AllDsNoDups	LINE1	AVERAGE
Item # 22	(No Task): Current:	GPRINT	LAST
Item # 23	(No Task): Average:	GPRINT	AVERAGE
Item # 24	(No Task): Maximum:[HR]	GPRINT	MAX

And this is the rrd graph statement:

```
/usr/bin/rrdtool graph - \
--imgformat=PNG \
--start=-86400 \
--end=-300 \
--title="Traffic AllDsNoDups" \
--base=1000 \
--height=120 \
--width=500 \
--alt-autoscale-max \
--lower-limit=0 \
--vertical-label="" \
--slope-mode \
--font TITLE:12: \
--font AXIS:8: \
--font LEGEND:8: \
--font UNIT:8: \
DEF:a="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:AVERAGE \
DEF:b="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:MAX \
DEF:c="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:AVERAGE \
DEF:d="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:MAX \
```

```
DEF:e="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:AVERAGE \
DEF:f="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:MAX \
DEF:g="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:AVERAGE \
DEF:h="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:MAX \
CDEF:cdefca=TIME,1202925514,GT,a,a,UN,0,a,IF,IF,TIME,1202925514,GT,c,c,UN,0,c,IF,IF,TIME,1202925514,GT,e,e,UN,0,e
,IF,IF,TIME,1202925514,GT,g,g,UN,0,g,IF,IF,+,+, \
LINE1:a#FFFF00FF:"1. Target Traffic In" \
GPRINT:a:LAST:"   Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:c#FFD660FF:"2. Target Traffic In" \
GPRINT:c:LAST:"     Current:%8.2lf%s" \
GPRINT:c:AVERAGE:"Average:%8.2lf%s" \
GPRINT:d:MAX:"Maximum:%8.2lf%sn" \
LINE1:e#CAF100FF:"1. Target Traffic Out" \
GPRINT:e:LAST:"   Current:%8.2lf%s" \
GPRINT:e:AVERAGE:"Average:%8.2lf%s" \
GPRINT:f:MAX:"Maximum:%8.2lf%sn" \
LINE1:g#CCBB00FF:"2. Target Traffic Out" \
GPRINT:g:LAST:"     Current:%8.2lf%s" \
GPRINT:g:AVERAGE:"Average:%8.2lf%s" \
GPRINT:h:MAX:"Maximum:%8.2lf%sn" \
LINE1:a#FF0000FF:"1. Target Again Traffic In" \
GPRINT:a:LAST:"Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:cdefca#000000FF:"AllDsNoDups" \
GPRINT:cdefca:LAST:"        Current:%8.2lf%s" \
GPRINT:cdefca:AVERAGE:"Average:%8.2lf%s" \
GPRINT:cdefca:MAX:"Maximum:%8.2lf%sn"
```

The data sources are denoted by the letters `a` to `h`. `cdefca` represents the important part. You surely notice, that all data sources using consolidation function `AVERAGE` are taken into account while `MAX` is skipped. Please pay attention to the data source denoted by `a`. Even though the data source appears twice, the cdef shows it only once. This is due to the duplicate suppression.

This is the list of graph items defined. Please note, that Item numbers 17-20 are duplicates of Item numbers 1-4. Even if it usually does not make sense, for this discussion it is very valuable. Item numbers 21-24 are related to a `cdef=ALL_DATA_SOURCES_DUPS`

Graph Item	Data Source	Graph Item Type	CF Type
Item # 1	(traffic_in): 1. Target Traffic In	LINE1	AVERAGE
Item # 2	(traffic_in): Current:	GPRINT	LAST
Item # 3	(traffic_in): Average:	GPRINT	AVERAGE
Item # 4	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 5	(traffic_in): 2. Target Traffic In	LINE1	AVERAGE
Item # 6	(traffic_in): Current:	GPRINT	LAST
Item # 7	(traffic_in): Average:	GPRINT	AVERAGE
Item # 8	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 9	(traffic_out): 1. Target Traffic Out	LINE1	AVERAGE
Item # 10	(traffic_out): Current:	GPRINT	LAST
Item # 11	(traffic_out): Average:	GPRINT	AVERAGE
Item # 12	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 13	(traffic_out): 2. Target Traffic Out	LINE1	AVERAGE
Item # 14	(traffic_out): Current:	GPRINT	LAST
Item # 15	(traffic_out): Average:	GPRINT	AVERAGE
Item # 16	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 17	(traffic_in): 1. Target Again Traffic In	LINE1	AVERAGE

Graph Item	Data Source	Graph Item Type	CF Type
Item # 18	(traffic_in): Current:	GPRINT	LAST
Item # 19	(traffic_in): Average:	GPRINT	AVERAGE
Item # 20	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 21	(No Task): AllDsDups	LINE1	AVERAGE
Item # 22	(No Task): Current:	GPRINT	LAST
Item # 23	(No Task): Average:	GPRINT	AVERAGE
Item # 24	(No Task): Maximum:[HR]	GPRINT	MAX

And this is the rrd graph statement:

```
/usr/bin/rrdtool graph - \
--imgformat=PNG \
--start=-86400 \
--end=-300 \
--title="Traffic AllDsDups" \
--base=1000 \
--height=120 \
--width=500 \
--alt-autoscale-max \
--lower-limit=0 \
--vertical-label="" \
--slope-mode \
--font TITLE:12: \
--font AXIS:8: \
--font LEGEND:8: \
--font UNIT:8: \
DEF:a="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:AVERAGE \
DEF:b="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:MAX \
DEF:c="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:AVERAGE \
DEF:d="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:MAX \
DEF:e="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:AVERAGE \
DEF:f="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:MAX \
DEF:g="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:AVERAGE \
DEF:h="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:MAX \
CDEF:cdefca=TIME,1202925583,GT,a,a,UN,0,a,IF,IF,TIME,1202925583,GT,c,c,UN,0,c,IF,IF,TIME,1202925583,GT,e,e,UN,0,e
,IF,IF,TIME,1202925583,GT,g,g,UN,0,g,IF,IF,TIME,1202925583,GT,a,a,UN,0,a,IF,IF,+,+,+,+ \
LINE1:a#FFFF00FF:"1. Target Traffic In" \
GPRINT:a:LAST:"      Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:c#FFD660FF:"2. Target Traffic In" \
GPRINT:c:LAST:"      Current:%8.2lf%s" \
GPRINT:c:AVERAGE:"Average:%8.2lf%s" \
GPRINT:d:MAX:"Maximum:%8.2lf%sn" \
LINE1:e#CAF100FF:"1. Target Traffic Out" \
GPRINT:e:LAST:"     Current:%8.2lf%s" \
GPRINT:e:AVERAGE:"Average:%8.2lf%s" \
GPRINT:f:MAX:"Maximum:%8.2lf%sn" \
LINE1:g#CCBB00FF:"2. Target Traffic Out" \
GPRINT:g:LAST:"     Current:%8.2lf%s" \
GPRINT:g:AVERAGE:"Average:%8.2lf%s" \
GPRINT:h:MAX:"Maximum:%8.2lf%sn" \
LINE1:a#FF0000FF:"1. Target Again Traffic In" \
GPRINT:a:LAST:"Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:cdefca#000000FF:"AllDsDups" \
GPRINT:cdefca:LAST:"             Current:%8.2lf%s" \
GPRINT:cdefca:AVERAGE:"Average:%8.2lf%s" \
GPRINT:cdefca:MAX:"Maximum:%8.2lf%sn"
```

Again, the data sources are denoted by the letters `a` to `h` and `cdefca` represents the important part. Please pay attention to the data source denoted by `a` appearing twice in this cdef. This is, because it appears twice (Item numbers 17-20) in the graph item list and duplicate suppression is not in effect.

Everything is very much like the above; again Item numbers 17-20 are duplicates of Item numbers 1-4. Item numbers 21-24 are related to a `cdef=SIMILAR_DATA_SOURCES_NODUPS` and are associated with the data source `traffic_in` of the `1.` target (it does not matter, which target is chosen, as long as you only choose a `traffic_in` data source)

Graph Item	Data Source	Graph Item Type	CF Type
Item # 1	(traffic_in): 1. Target Traffic In	LINE1	AVERAGE
Item # 2	(traffic_in): Current:	GPRINT	LAST
Item # 3	(traffic_in): Average:	GPRINT	AVERAGE
Item # 4	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 5	(traffic_in): 2. Target Traffic In	LINE1	AVERAGE
Item # 6	(traffic_in): Current:	GPRINT	LAST
Item # 7	(traffic_in): Average:	GPRINT	AVERAGE
Item # 8	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 9	(traffic_out): 1. Target Traffic Out	LINE1	AVERAGE
Item # 10	(traffic_out): Current:	GPRINT	LAST
Item # 11	(traffic_out): Average:	GPRINT	AVERAGE
Item # 12	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 13	(traffic_out): 2. Target Traffic Out	LINE1	AVERAGE
Item # 14	(traffic_out): Current:	GPRINT	LAST
Item # 15	(traffic_out): Average:	GPRINT	AVERAGE
Item # 16	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 17	(traffic_in): 1. Target Again Traffic In	LINE1	AVERAGE
Item # 18	(traffic_in): Current:	GPRINT	LAST
Item # 19	(traffic_in): Average:	GPRINT	AVERAGE
Item # 20	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 21	(traffic_in): SimilarDsNoDups	LINE1	AVERAGE
Item # 22	(traffic_in): Current:	GPRINT	LAST
Item # 23	(traffic_in): Average:	GPRINT	AVERAGE
Item # 24	(traffic_in): Maximum:[HR]	GPRINT	MAX

And this is the rrd graph statement:

```
/usr/bin/rrdtool graph - \
--imgformat=PNG \
--start=-86400 \
--end=-300 \
--title="Traffic SimilarDsNoDups" \
--base=1000 \
--height=120 \
--width=500 \
--alt-autoscale-max \
--lower-limit=0 \
--vertical-label="" \
--slope-mode \
--font TITLE:12: \
--font AXIS:8: \
--font LEGEND:8: \
--font UNIT:8: \
DEF:a="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:AVERAGE \
DEF:b="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:MAX \
DEF:c="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:AVERAGE \
DEF:d="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:MAX \
```

```
DEF:e="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:AVERAGE \
DEF:f="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:MAX \
DEF:g="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:AVERAGE \
DEF:h="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:MAX \
CDEF:cdefca=TIME,1202924474,GT,a,a,UN,0,a,IF,IF,TIME,1202924474,GT,c,c,UN,0,c,IF,IF,+ \
CDEF:cdefcd=TIME,1202924474,GT,b,b,UN,0,b,IF,IF,TIME,1202924474,GT,d,d,UN,0,d,IF,IF,+ \
LINE1:a#FFFF00FF:"1. Target Traffic In" \
GPRINT:a:LAST:"    Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:c#FFD660FF:"2. Target Traffic In" \
GPRINT:c:LAST:"      Current:%8.2lf%s" \
GPRINT:c:AVERAGE:"Average:%8.2lf%s" \
GPRINT:d:MAX:"Maximum:%8.2lf%sn" \
LINE1:e#CAF100FF:"1. Target Traffic Out" \
GPRINT:e:LAST:" \
Current:%8.2lf%s" \
GPRINT:e:AVERAGE:"Average:%8.2lf%s" \
GPRINT:f:MAX:"Maximum:%8.2lf%sn" \
LINE1:g#CCBB00FF:"2. Target Traffic Out" \
GPRINT:g:LAST:"     Current:%8.2lf%s" \
GPRINT:g:AVERAGE:"Average:%8.2lf%s" \
GPRINT:h:MAX:"Maximum:%8.2lf%sn" \
LINE1:a#FF0000FF:"1. Target Again Traffic In" \
GPRINT:a:LAST:"Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:cdefca#000000FF:"SimilarDsNoDups" \
GPRINT:cdefca:LAST:"           Current:%8.2lf%s" \
GPRINT:cdefca:AVERAGE:"Average:%8.2lf%s" \
GPRINT:cdefcd:MAX:"Maximum:%8.2lf%sn"
```

`cdefca` calculates the `SUM` of `all traffic_in` data sources, but pays attention only to consolidation function `AVERAGE`. Duplicate suppression makes sure, that the second occurrence of the `1`. target is not taken into account. So it's only summing data source `a` and `c`. `cdefca` is used for the `LINE1` graph item with exact match for consolidation function `AVERAGE` as well as for the best matched consolidation functions `LAST` and `MIN`.

`cdefcd` calculates the `SUM` of all `traffic_in` data sources, but pays attention only to consolidation function `MAX`. Again, duplicate suppression is in effect. So it's only summing data source `b` and `d`. `cdefcd` is used for the graph item with exact match for consolidation function `MAX` only. This is, because my rra settings only define `AVERAGE` and `MAX`, where `LAST` and `MIN` are omitted. This may differ for installations defining `LAST` and `MIN` as well.

Again, please note, that Item numbers 17-20 are duplicates of Item numbers 1-4. Item numbers 21-24 are related to a `cdef=SIMILAR_DATA_SOURCES_DUPS` and are associated with the data source `traffic_in` of the `1` target (it does not matter, which target is chosen, as long as you only choose a `traffic_in` data source)

Graph Item	Data Source	Graph Item Type	CF Type
Item # 1	(traffic_in): 1. Target Traffic In	LINE1	AVERAGE
Item # 2	(traffic_in): Current:	GPRINT	LAST
Item # 3	(traffic_in): Average:	GPRINT	AVERAGE
Item # 4	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 5	(traffic_in): 2. Target Traffic In	LINE1	AVERAGE
Item # 6	(traffic_in): Current:	GPRINT	LAST
Item # 7	(traffic_in): Average:	GPRINT	AVERAGE
Item # 8	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 9	(traffic_out): 1. Target Traffic Out	LINE1	AVERAGE
Item # 10	(traffic_out): Current:	GPRINT	LAST
Item # 11	(traffic_out): Average:	GPRINT	AVERAGE
Item # 12	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 13	(traffic_out): 2. Target Traffic Out	LINE1	AVERAGE

Graph Item	Data Source	Graph Item Type	CF Type
Item # 14	(traffic_out): Current:	GPRINT	LAST
Item # 15	(traffic_out): Average:	GPRINT	AVERAGE
Item # 16	(traffic_out): Maximum:[HR]	GPRINT	MAX
Item # 17	(traffic_in): 1. Target Again Traffic In	LINE1	AVERAGE
Item # 18	(traffic_in): Current:	GPRINT	LAST
Item # 19	(traffic_in): Average:	GPRINT	AVERAGE
Item # 20	(traffic_in): Maximum:[HR]	GPRINT	MAX
Item # 21	(traffic_in): SimilarDsDups	LINE1	AVERAGE
Item # 22	(traffic_in): Current:	GPRINT	LAST
Item # 23	(traffic_in): Average:	GPRINT	AVERAGE
Item # 24	(traffic_in): Maximum:[HR]	GPRINT	MAX

And this is the rrd graph statement:

```
/usr/bin/rrdtool graph - \
--imgformat=PNG \
--start=-86400 \
--end=-300 \
--title="Traffic SimilarDsDups" \
--base=1000 \
--height=120 \
--width=500 \
--alt-autoscale-max \
--lower-limit=0 \
--vertical-label="" \
--slope-mode \
--font TITLE:12: \
--font AXIS:8: \
--font LEGEND:8: \
--font UNIT:8: \
DEF:a="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:AVERAGE \
DEF:b="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_in:MAX \
DEF:c="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:AVERAGE \
DEF:d="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_in:MAX \
DEF:e="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:AVERAGE \
DEF:f="/var/www/html/cacti/rra/target1_traffic_in_235.rrd":traffic_out:MAX \
DEF:g="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:AVERAGE \
DEF:h="/var/www/html/cacti/rra/target2_traffic_in_8.rrd":traffic_out:MAX \
CDEF:cdefca=TIME,1202925634,GT,a,a,UN,0,a,IF,IF,TIME,1202925634,GT,c,c,UN,0,c,IF,IF,TIME,1202925634,GT,a,a,UN,0,a
,IF,IF,+,+ \
CDEF:cdefcd=TIME,1202925634,GT,b,b,UN,0,b,IF,IF,TIME,1202925634,GT,d,d,UN,0,d,IF,IF,TIME,1202925634,GT,b,b,UN,0,b
,IF,IF,+,+ \
LINE1:a#FFFF00FF:"1. Target Traffic In" \
GPRINT:a:LAST:"      Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:c#FFD660FF:"2. Target Traffic In" \
GPRINT:c:LAST:"    Current:%8.2lf%s" \
GPRINT:c:AVERAGE:"Average:%8.2lf%s" \
GPRINT:d:MAX:"Maximum:%8.2lf%sn" \
LINE1:e#CAF100FF:"1. Target Traffic Out" \
GPRINT:e:LAST:"     Current:%8.2lf%s" \
GPRINT:e:AVERAGE:"Average:%8.2lf%s" \
GPRINT:f:MAX:"Maximum:%8.2lf%sn" \
LINE1:g#CCBB00FF:"2. Target Traffic Out" \
GPRINT:g:LAST:"   Current:%8.2lf%s" \
GPRINT:g:AVERAGE:"Average:%8.2lf%s" \
GPRINT:h:MAX:"Maximum:%8.2lf%sn" \
LINE1:a#FF0000FF:"1. Target Again Traffic In" \
GPRINT:a:LAST:"Current:%8.2lf%s" \
GPRINT:a:AVERAGE:"Average:%8.2lf%s" \
GPRINT:b:MAX:"Maximum:%8.2lf%sn" \
LINE1:cdefca#000000FF:"SimilarDsDups" \
GPRINT:cdefca:LAST:Current:%8.2lf%s" \
```

```
GPRINT:cdefca:AVERAGE:"Average:%8.2lf%s" \
GPRINT:cdefcd:MAX:"Maximum:%8.2lf%sn"
```

`cdefca` and `cdefcd` represents the important part. You surely notice, that all data sources using consolidation function AVERAGE are taken into account for calculation `cdefca`, while `MAX` is calculated with `cdefcd`. Please pay attention to the data source denoted by `a` and `b`, respectively, appearing twice in those CDEFs as expected.

More CDEF Examples

This example is based on the standard data template `ucd/net - Hard Drive Space`. Cacti ships an associated graph template named `ucd/net - Available Disk Space`. Often, users complain about the fact, that this template prints free and used space as absolute figures only. But instead of knowing, that your data partition has about 10GB used space, you may be interested in the percentage used. So let's create a simple example to show the power of the recently introduced special data sources.

To make it work, we need two new CDEFs. The first one may come in useful for a bunch of different graph templates. It's named `Make Current Data Source 0`. On a first glance, you may wonder why this CDEF is needed. It is used for those data sources, that will be used for calculations but shall not show up themselves. Here's the definition

```
cdef=CURRENT_DATA_SOURCE,0,*
```

Now, the other one, named `Current DS as Percentage of all DS`

```
cdef=CURRENT_DATA_SOURCE,ALL_DATA_SOURCES_NODUPS,/,100,*
```

Here's a short discussion. The first three elements read: Take the current data source and divide it by `Sum of All Data Sources (Don't include Duplicates)`. To form a percentage, you will have to multiply by 100. This is done by adding the elements four and five.

Now, let's build up the new graph

Graph Item	Data Source	Graph Item Type	CF Type	Item Color
Item # 1	(hdd_used):	LINE1	AVERAGE	
Item # 2	(hdd_free):	LINE1	AVERAGE	
Item # 3	(hdd_used): % Used	AREA	AVERAGE	FF0000

Item numbers 1 and 2 in most cases will exceed the maximum percentage of 100 by magnitudes. So they are associated with the CDEF named `Make Current Data Source 0` to make them zero. This adds the data sources to the graph but avoids them showing up. The pseudo-color of `None` is associated to both of them.

Item number 3 is associated to the data source `hdd_used` as we want to print the `% Used` of the partition. The CDEF is `Current DS as Percentage of all DS`, the color is set to `red (FF0000)`.

And this is the rrd graph statement:

```
/usr/bin/rrdtool graph - \
--imgformat=PNG \
--start=-86400 \
--end=-300 \
--title="gandalf - Disk Space - / %used" \
--rigid \
--base=1024 \
--height=120 \
--width=500 \
--alt-autoscale-max \
--lower-limit=0 \
--vertical-label="Percent" \
--slope-mode \
--font TITLE:12: \
--font AXIS:8: \
--font LEGEND:8: \
--font UNIT:8: \
DEF:a="/var/www/html/cacti/rra/target1_hdd_free_236.rrd":hdd_used:AVERAGE \
```

```
DEF:b="/var/www/html/cacti/rra/target1_hdd_free_236.rrd":hdd_free:AVERAGE \
CDEF:cdefa=a,0,* \
CDEF:cdefb=b,0,* \
CDEF:cdefc=a,TIME,1203272123,GT,a,a,UN,0,a,IF,IF,TIME,1203272123,GT,b,b,UN,0,b,IF,IF,+,/,100,* \
LINE1:cdefa:"" \
LINE1:cdefb:"" \
AREA:cdefc#FF0000FF:"% Used"
```

`cdefa` and `cdefb` are used to make the according graph items zero. `cdefc` performs the percentage calculation.

You may of course add `GPRINT` legends as usual. If you do this for the "% Used" entry only, you will get surprising results, if your RRDfile holds either `MAXIMUM`, `LAST` and/or `MINIMUM` consolidation function(s). This is left as an exercise to you :)

This set of CDEFs is used to colorize the background of a graph with different colors for day, night and weekends. The CDEFs are created as usual, we will show the CDEF definition only. Examples are taken from RRDtool-users mailing list courtesy Erik de Mare. Here are the definitions

Background for Daytime

```
cdef=LTIME,86400,%,28800,GT,LTIME,86400,%,64800,LT,INF,UNKN,CURRENT_DATA_SOURCE,*,IF,UNKN,CURRENT_DATA_SOURCE,*,IF
```

Background for Nighttime

cdef=LTIME,86400,%,28800,LT,INF,LTIME,86400,%,64800,GT,INF,UNKN,CURRENT_DATA_SOURCE,*,IF,IF`

Background for Weekend

```
cdef=LTIME,604800,%,172800,GT,LTIME,604800,%,345600,LT,INF,UNKN,CURRENT_DATA_SOURCE,*,IF,UNKN,CURRENT_DATA_SOURCE,*,IF
```

The value of `86400` represents the number of seconds of a day, whereas `28800` represents `8:00`, defined as the start of the day. End of the day, `64800` is assumed at `18:00`. Please replace those values if required. For weekends, same logic applies.

Now, let's apply those new CDEFs to a Graph Template. For this example, I've chosen the `Unix - Processes` that applies to `localhost` only. In turn, please create three new graph items, associate the processes data source, make them AREAs, select a color and opacity. Choose the `daytime`, `nighttime` and `weekend` CDEF in this sequence. As a last step, move those three new graph items to the top. I've chosen opacity of `20%` for Item numbers 1 to 3.

Graph Item	Data Source	Graph Item Type	CF Type	Item Color
Item # 1	(proc):	AREA	AVERAGE	FFFF00
Item # 2	(proc):	AREA	AVERAGE	0000FF
Item # 3	(proc):	AREA	AVERAGE	2E3127
Item # 4	(proc): Running Processes	AREA	AVERAGE	F51D30
Item # 5	(proc): Current:	GPRINT	LAST	
Item # 6	(proc): Average:	GPRINT	AVERAGE	
Item # 7	(proc): Maximum:	GPRINT	MAX	

Copyright (c) 2004-2026 The Cacti Group

Command Line Scripts (CLI)

Cacti supports a number of command line scripts. You will find them in the `./cli` directory. The CLI scripts fall into multiple categories, they include:

- **Automation** - Utilities to help automate adding objects to Cacti
- **Maintenance** - Utilities to help keep Cacti healthy
- **Repair** - Utilities to help with repairing Cacti database components
- **Repair/Migration** - Utilities to help with repairing or migrating Cacti components
- **Install/Upgrade** - Utilities to help with installing and upgrading Cacti

In the table below, each of the CLI scripts are categories and explained briefly.

Automation Scripts

Script	Category	Description
add_device.php	Automation	Allows adding a Device to Cacti. Most device options are included. Some plugin options may not.
change_device.php	Automation	Allows editing device details such as IP/Poller association
add_data_query.php	Automation	Allows adding a Data Query to Cacti.
add_graph_template.php	Automation	Allows adding a Graph Template to Cacti.
add_graphs.php	Automation	Allows adding a Graph to Cacti.
add_perms.php	Automation	Allows adding permissions to Cacti.
add_tree.php	Automation	Allows adding tree's tree branches and objects to Cacti Trees.
copy_user.php	Maintenance	Allows creating new users from Templates

Maintenance Scripts

Script	Category	Description
input_whitelist.php	Maintenance	To onboard new Data Input Methods when using Cacti in a high security environment where new Data Input Methods must always be vetted before enabling for general use.
poller_graphs_reapply_names.php	Maintenance	Allows selecting re-running of Cacti's suggested values engine for Graphs
poller_reindex_hosts.php	Maintenance	Batch method to re-index Cacti Devices
analyze_database.php	Maintenance	Analyzes all Cacti tables rebuilding their index cardinality. Important to run after having added large numbers of Devices and Graphs to Cacti.

Repair Scripts

Script	Category	Description
poller_output_empty.php	Repair	Removes bad entries from Cacti's poller output table.
rebuild_poller_cache.php	Repair	Batch process to re-populate Cacti's poller cache. NOTE: This script can run for a very long time on large systems.
repair_database.php	Repair	Utility to look for common problems with old Cacti databases where objects were removed while still in use. Also rapairs all tables after a system crash.
repair_graphs.php	Repair	Utility to repair Cacti Graphs whose structure is damaged due to legacy Cacti behavior.
repair_templates.php	Repair	Utility to repair Cacti Templates whose structure was damaged due to legacy Cacti behavior.

Repair/Migration Scripts

Script	Category	Description
audit_database.php	Repair/Migration	Script to audit your Cacti schema, and repair any tables that don't match the stock Cacti schema. It will also upgrade the Cacti schema and run plugin upgrade functions. This is a good migration tool to move database from old versions of Cacti.
convert_tables.php	Repair/Migration	Convert all tables from their current form to the Cacti preferred table type, collation and charset
splice_rra.php	Migration	Utility that allows two RRDfiles to be merged, and also assists with resampling an old RRDfile to a new RRA configuration, for example moving a Cacti system from 5 minute polling to 1 minute polling.

Install/Upgrade Scripts

Script	Category	Description
import_package.php	Install/Upgrade	Import a Cacti package from a tgz file.
install_cacti.php	Install/Upgrade	Script to install Cacti from a raw schema. Used primarily by distribution maintainers.
md5sum.php	Install/Upgrade	Utility to verify the MD5SUM of a file.
sqltable_to_php.php	Install/Upgrade	Utility for Plugin Developers to create an object for creating tables in plugins.
structure_rra_paths.php	Install/Upgrade	Utility to convert a system from a flat directory path to a heirarchial one. Important for certain file systems that don't perform well when there are tens or hundreds of thousands of files in a single directory.
upgrade_database.php	Install/Upgrade	Utility to upgrade a Cacti database from a point in time to the current release. Or even rerun the database upgrade from a point in the past to catch up with changes. This script should be used

Script	Category	Description
		in conjunction with the `audit_database.php --repair --upgrade` option.

> **NOTE**: Package maintainers may wish to utilize the CLI methods of install_cacti.php and upgrade_database.php to perform in place installations and upgrades.

In the sub-sections below, we will demonstrate the use of a few of these CLI scripts.

> **Caution**: In the following examples, several numbers are shown as output from various scripts. They will vary between different installations. So don't bother, if your numbers will vary.

Rebuild Poller Cache

The poller cache holds all commands that cacti will issue during the polling process in an internal format. It is possible, to review the current contents of the poller cache by visiting `System Utilities`, `View Poller Cache`. It is possible to apply filters to this view; it will show up like

```
Localhost - Hard Drive Space    Script Server: /var/www/html/cacti/scripts/ss_host_disk.php ss_host_disk
127.0.0.1 1 1:161:500:somesecret:::::: get total 6
                                RRD: /var/www/html/cacti/rra/localhost_hdd_total_61.rrd
Localhost - Load Average        Script: /usr/bin/perl /var/www/html/cacti/scripts/loadavg_multi.pl
                                RRD: /var/www/html/cacti/rra/localhost_load_1min_5.rrd
Localhost - Logged in Users     Script: /usr/bin/perl /var/www/html/cacti/scripts/unix_users.pl
                                RRD: /var/www/html/cacti/rra/localhost_users_6.rrd
Localhost - Memory - Free       Script: /usr/bin/perl /var/www/html/cacti/scripts/linux_memory.pl MemFree:
                                RRD: /var/www/html/cacti/rra/localhost_mem_buffers_3.rrd
Localhost - Memory - Free Swap  Script: /usr/bin/perl /var/www/html/cacti/scripts/linux_memory.pl SwapFree:
                                RRD: /var/www/html/cacti/rra/localhost_mem_swap_4.rrd
Localhost - mtaReceivedMessages SNMP Version: 1, Community: somesecret, OID: .1.3.6.1.2.1.28.1.1.1.1
                                RRD: /var/www/html/cacti/rra/localhost_mtareceivedmessages_47.rrd
Localhost - mtaReceivedVolume   SNMP Version: 1, Community: somesecret, OID: .1.3.6.1.2.1.28.1.1.4.1
                                RRD: /var/www/html/cacti/rra/localhost_mtareceivedvolume_49.rrd
```

There are several circumstances, that may result in the poller cache being out of sync. An example would be a change of the name of a script used in a `Data Input Method`. This change is *not* automatically propagated to the poller cache. It is required, to run `php -q rebuild poller_cache.php` manually.

Calling the script with the parameter `--help` yields

```
shell>php -q rebuild_poller_cache.php --help
Cacti Rebuild Poller Cache Script 1.0, Copyright 2004-2026 - The Cacti Group

usage: rebuild_poller_cache.php [-d] [-h] [--help] [-v] [--version]

-d            - Display verbose output during execution
-v --version  - Display this help message
-h --help     - Display this help message
```

Debug mode lists the data sources items that are worked upon.

```
shell>php -q rebuild_poller_cache.php -d
WARNING: Do not interrupt this script.  Rebuilding the Poller Cache can take quite some time
DEBUG: There are '38' data source elements to update.
DEBUG: Data Source Item '1' of '38' updated
DEBUG: Data Source Item '2' of '38' updated
DEBUG: Data Source Item '3' of '38' updated
DEBUG: Data Source Item '4' of '38' updated
....
```

Without any parameter, some dots are shown as progress indicators.

```
shell>php -q rebuild_poller_cache.php
WARNING: Do not interrupt this script.  Rebuilding the Poller Cache can take quite some time
.....................................
shell>
```

> **Caution**
>
> Rebuilding the poller cache interferes with the poller operation. Make sure that the poller is not running and will not start during a rebuild operation

Re-Index Hosts

Re-Indexing is required only for SNMP/Script Data Queries. Remember, that when applying a Data Query to a Host, a `Re-Index Method` has to be chosen. This reindex method governs the automatic re-indexing based on specific events.

Table 20-1. Re-Index Methods

Re-Index Method	Description
Uptime Goes Backwards	Refers to a system reboot
Index Count Changed	Refers to a change of the number of indexed items
Verify All Fields	All index fields of the according XML file are checked for changes

If the method is set to `Uptime Goes Backwards`, Cacti will detect if the target has been rebooted by querying sysUptime (.1.3.6.1.2.1.1.3.0). If the current value for the uptime is lower than the previous one (uptime goes backwards), a reboot is assumed and a re-index is performed

If the method is set to `Index Count Change`, e.g. the number of interfaces, without checking sysUptime. It is of particular interest in cases, where indexed entries may change without the need of a reboot. As modern operating systems seldom require reboots for system configuration changes, you may want to consider this setting for many cases (e.g. creating of a new filesystem without a reboot). The index to be considered is defined by the according XML file.

If the method is set to `Verify All Fields`, all indexes of the data source are checked for changes. This is of particular interest e.g. in cases, where the index is non-numeric (e.g. a MAC address). It should be noted that in the 1.0 and 1.1 releases, this method was found to be broken and has been corrected since 1.2.

If you feel the need for manual re-indexing, you may run it from CLI. Calling the script with the parameter `--help` yields

```
shell>php -q poller_reindex_hosts.php --help
Cacti Reindex Host Script 1.2, Copyright 2004-2026 - The Cacti Group

usage: poller_reindex_hosts.php --id=[host_id|All] [--qid=[ID|All]] [--host-descr=[description]]
                          [-d] [-h] [--help] [-v] [--version]

--id=host_id             - The host_id to have data queries reindexed or 'All' to reindex all hosts
--qid=query_id           - Only index on a specific data query id; defaults to 'All'
--host-descr=description - The host description to filter by (SQL filters acknowledged)
--debug                  - Display verbose output during execution
-v --version             - Display this help message
-h --help                - Display this help message
```

Running it in debug mode for the host with `id=2` may yield

```
shell>php -q poller_reindex_hosts.php -id=2 -d
WARNING: Do not interrupt this script.  Reindexing can take quite some time
DEBUG: There are '1' data queries to run
DEBUG: Data query number '1' host: '2' SNMP Query Id: '1' starting
DEBUG: Data query number '1' host: '2' SNMP Query Id: '1' ending
```

A silent run for all devices is issued by

```
shell>php -q poller_reindex_hosts.php -id=All
WARNING: Do not interrupt this script.  Reindexing can take quite some time
.....shell>
```

You may run this script against a specific data query id using `--qid=[data query id]` like

```
shell>php -q poller_reindex_hosts.php --id=All --qid=1 -d
WARNING: Do not interrupt this script.  Reindexing can take quite some time
DEBUG: There are '3' data queries to run
DEBUG: Data query number '1' host: '1' SNMP Query Id: '1' starting
DEBUG: Data query number '1' host: '1' SNMP Query Id: '1' ending
DEBUG: Data query number '2' host: '2' SNMP Query Id: '1' starting
DEBUG: Data query number '2' host: '2' SNMP Query Id: '1' ending
DEBUG: Data query number '3' host: '15' SNMP Query Id: '1' starting
DEBUG: Data query number '3' host: '15' SNMP Query Id: '1' ending
```

It is possible, to select a host based on its host description using `--host-descr=[host description]`. It is allowed to apply an SQL filter expression for the description like `--qid=some%descr`. Use this parameter in conjunction with either `--id=[host id]` or `--qid=[data query id]` or both:

```
shell>php -q poller_reindex_hosts.php --id=All --qid=1 --host-descr=ga%f -d
WARNING: Do not interrupt this script.  Reindexing can take quite some time
DEBUG: There are '1' data queries to run
DEBUG: Data query number '1' host: '2' SNMP Query Id: '1' starting
DEBUG: Data query number '1' host: '2' SNMP Query Id: '1' ending
```

> **Caution**
>
> Re-Indexing interferes with the poller operation. Make sure that the poller is not running and will not start during a reindex operation

Empty Poller Output Table

During normal poller operation, all retrieved results are intermediately stored in the table named poller_output After execution of cmd.php or Spine, this table holds all results. The poller.php finally issues all rrdtool update operations. Thus, after polling has completed, the table should be empty.

Conditions may arise, where the table is not (completely) emptied. The most often known issue is lack of php memory. In those cases, the table is bigger than the php memory size, thus not all items are handled correctly. If that issue occurs, you may save all pending updates *after increasing PHP memory appropriately* by running this very script. Calling the script with the parameter `--help` yields

```
shell>php -q poller_output_empty.php --help
Cacti Empty Poller Output Table Script 1.0, Copyright 2004-2026 - The Cacti Group

usage: poller_output_empty.php [-h] [--help] [-v] [--version]

-v --version  - Display this help message
-h --help     - Display this help message
```

You see, no parameter is required for operating successfully. Under normal circumstances, running this script should yield

```
shell>php -q poller_output_empty.php
There were 0, RRD updates made this pass
shell>
```

In case, you hit the poller process or if the table was really not fully processed, you may find

```
shell>php -q poller_output_empty.php
OK u:0.00 s:0.00 r:0.04
```

```
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.04
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.00 r:0.05
OK u:0.00 s:0.01 r:0.06
There were 21, RRD updates made this pass
shell>
```

If logging level verbosity was switched to `DEBUG` you may find additional debug messages that usually show up in `cacti.log`

Reapply Suggested Names to Graphs

For a general understanding of suggested names used with data queries, please see ?. Be aware, that changes to the `Suggested Names` section of a data query will not automatically be propagated to all existing graphs. This is, where poller_graphs_reapply_names.php drops in. Calling the script with the parameter `--help` yields

```
shell>php -q poller_graphs_reapply_names.php --help
Cacti Reapply Graph Names Script 1.0, Copyright 2004-2026 - The Cacti Group

usage: poller_graphs_reapply_names.php -id=[host_id|All][host_id1|host_id2|...] [-s=[search_string] [-d] [-h] [--
help] [-v] [--version]

-id=host_id   - The host_id or 'All' or a pipe delimited list of host_id's
-s=search_str - A graph template name or graph title to search for
-d            - Display verbose output during execution
-v --version  - Display this help message
-h --help     - Display this help message
```

Assume a situation, where the suggested names where changed for the standard data query named `SNMP - Get Mounted Partitions`. In this case, you will want to rework all graphs for this data query only. A verbose run will yield

```
shell>php -q poller_graphs_reapply_names.php -id=All -d -s="Used space"
WARNING: Do not interrupt this script.  Interrupting during rename can cause issues
DEBUG: There are '6' Graphs to rename
DEBUG: Graph Name 'Localhost - Used Space - Memory Buffers' starting
DEBUG: Graph Rename Done for Graph 'Localhost - Used Space - Memory Buffers'
DEBUG: Graph Name 'Localhost - Used Space - Real Memory' starting
DEBUG: Graph Rename Done for Graph 'Localhost - Used Space - Real Memory'
DEBUG: Graph Name 'Localhost - Used Space - Swap Space' starting
DEBUG: Graph Rename Done for Graph 'Localhost - Used Space - Swap Space'
DEBUG: Graph Name 'Localhost - Used Space - /' starting
DEBUG: Graph Rename Done for Graph 'Localhost - Used Space - /'
DEBUG: Graph Name 'Localhost - Used Space - /sys' starting
DEBUG: Graph Rename Done for Graph 'Localhost - Used Space - /sys'
DEBUG: Graph Name 'Localhost - Used Space - /boot' starting
DEBUG: Graph Rename Done for Graph 'Localhost - Used Space - /boot'
```

Notice the miss-spelling of the word “Space”. The `-s=` option is not case sensitive.

Copy Local Cacti Users

For use and understanding the limitation of this script, it is of importance to read User Management

Calling the script with the parameter `--help` yields

```
shell>php -q copy_user.php --help
Cacti Copy User Utility, Version 1.2.0, Copyright (C) 2004-2026 The Cacti Group
usage: copy_cacti_user.php <template user> <new user>

A utility to copy on local Cacti user and their settings to a new one.

NOTE: It is highly recommended that you use the web interface to copy users as
this script will only copy Local Cacti users.
```

Now let's create a new user Harry Potter as follows

```
shell>php -q copy_user.php guest "Harry Potter"
Template User: guest
New User:      Harry Potter

Copying User...
User copied...
```

Find the results of this program call at User Management

User Name**	Full Name	Enabled	Realm	Default Graph Policy	Last Login
admin	Administrator	Yes	Local	ALLOW	Sunday, October 07, 2007 14:59:32
guest	Guest Account	Yes	Local	ALLOW	N/A
Harry Potter	Guest Account	Yes	Local	ALLOW	N/A

Add a New Device

While it is an easy task to add a new device from the panels, this would be a tedious task for creating dozens of hundreds of devices in one turn. This is, where the script `add_device.php` comes in. First, let's have a look at the whole list of features it provides. Calling the script with the parameter `--help` yields

```
shell>php -q add_device.php --help
Add Device Script 1.0, Copyright 2004-2026 - The Cacti Group

A simple command line utility to add a device in Cacti

usage: add_device.php --description=[description] --ip=[IP] --template=[ID] [--notes="[]"] [--disable]
        [--avail=[ping]] --ping_method=[icmp] --ping_port=[N/A, 1-65534] --ping_retries=[2]
        [--version=[1|2|3]] [--community=] [--port=161] [--timeout=500]
        [--username= --password=] [--authproto=] [--privpass= --privproto=] [--context=]
        [--quiet]

    Required:
        --description  the name that will be displayed by Cacti in the graphs
        --ip           self explanatory (can also be a FQDN)
    Optional:
        --template     0, is a number (read below to get a list of templates)
        --notes        '', General information about this host.  Must be enclosed using double quotes.
        --disable      0, 1 to add this host but to disable checks and 0 to enable it
        --avail        pingsnmp, [ping][none, snmp, pingsnmp]
        --ping_method  tcp, icmp|tcp|udp
        --ping_port    '', 1-65534
        --ping_retries 2, the number of time to attempt to communicate with a host
        --version      1, 1|2|3, snmp version
        --community    '', snmp community string for snmpv1 and snmpv2.  Leave blank for no community
        --port         161
        --timeout      500
```

```
        --username     '', snmp username for snmpv3
        --password     '', snmp password for snmpv3
        --authproto    '', snmp authentication protocol for snmpv3
        --privpass     '', snmp privacy passphrase for snmpv3
        --privproto    '', snmp privacy protocol for snmpv3
        --context      '', snmp context for snmpv3

    List Options:
        --list-host-templates
        --list-communities
        --quiet - batch mode value return
```

Wow, that's quite a lot of options. To better understand it's use, let's first stick to the listing options

List all Host Templates

```
shell>php -q add_device.php --list-host-templates
```

The command lists the templates installed in your database. Use those IDs rather than assuming the IDs from another installation.

Why are those `Host Templates` of such importance for this script? Well, when using this script to add a new device, there are 3 required parameters. The `description` and the `device ip address` are of course left to you. But for assigning the correct `Host Template Id`, you should first know the correct id of that very parameter. For the purpose of this chapter we assume to add a device that will be associated to a `ucd/net SNMP Host`, so the id of `3` is required.

If you do not want to associate the host to any `Host Template Id`, known as host template None, either provide the parameter `--template=0` or omit this parameter.

List all Community Strings

```
shell>php -q add_device.php --list-communities

Known communities are: (community)
    public
    snmp-get
```

Of course, your list will vary

Create a New Device

Now, let's set up the most basic command to add a new device. The description shall be `"Device Add Test"`, the ip will be given as a FQDN, `router.mydomain.com`. As a SNMP enabled device, surely a community string has to be provided; in this case given as `public`. In total, this makes the following command

```
shell>php -q add_device.php --description="Device Add Test" --ip="router.mydomain.com" --template=3 --
community="public"

Adding Device Add Test (router.mydomain.com) as "ucd/net SNMP Host" using SNMP v1 with community "public"
Success - new device-id: (11)
```

Go to Devices to see the result:

Description**	Graphs	Data Sources	Status	Hostname	Current (ms)	Average (ms)	Availability
Device Add Test	0	0	Unknown	router.mydomain.com	0	0	100

Use any other combination of parameters in the same way as via the web browser console. You may want to remember the new device id, `11` in this case, for the next steps.

Associate a Data Query to an existing Host

It is recommended to maintain data query associations by maintaining host templates. Each time, a data query is added to a host template, it will automatically associated with all hosts related to that very host template. The data query will be executed as well; this way the snmp cache will be filled with all values related to that query.

Nevertheless, it is sometimes suitable to manually associate a certain data query with a single host only without changing any host template. This is, where the script `add_data_query.php` comes in. First, let's have a look at the whole list of features it provides. Calling the script with the parameter `--help` yields

```
shell>php -q add_data_query.php --help

Add Data Query Script 1.0, Copyright 2004-2026 - The Cacti Group

A simple command line utility to add a data query to an existing device in Cacti

usage: add_data_query.php --host-id=[ID] --data-query-id=[dq_id] --reindex-method=[method] [--quiet]

Required:
        --host-id         the numerical ID of the host
        --data-query-id   the numerical ID of the data_query to be added
        --reindex-method  the reindex method to be used for that data query
                          0|None   = no reindexing
                          1|Uptime = Uptime goes Backwards
                          2|Index  = Index Count Changed
                          3|Fields = Verify all Fields
List Options:
        --list-hosts
        --list-data-queries
        --quiet - batch mode value return

If the data query was already associated, it will be reindexed.
```

Let's first stick to the listing options

List all Hosts

```
shell>php -q add_data_query.php --list-hosts

Known Hosts: (id, hostname, template, description)
1       127.0.0.1       8       Localhost
11      router          3       router.mydomain.com
```

List all Data Queries

```
shell>php -q add_data_query.php --list-data-queries

Known SNMP Queries:(id, name)
1       SNMP - Interface Statistics
2       ucd/net -  Get Monitored Partitions
3       Karlnet - Wireless Bridge Statistics
4       Netware - Get Available Volumes
6       Unix - Get Mounted Partitions
7       Netware - Get Processor Information
8       SNMP - Get Mounted Partitions
9       SNMP - Get Processor Information
```

Add a Data Query

```
shell>php -q add_data_query.php --host-id=11 --data-query-id=1 --reindex-method=fields
Success - Host (11: router.mydomain.com) data query (1: SNMP - Interface Statistics) reindex method (3: Verify
All Fields)
```

This example shows the use of a string `"fields"` for specifying the reindex method. As given by the help text, the use of the numeric value of `"3"` would have given the exact same result. When visiting the device edit panel, you will find non-zero values for the data query status column, e.g. *Success [41 Items, 5 Rows]*.

Associate a Graph Template to an existing Host

It is recommended to maintain graph template associations by maintaining host templates. Each time, a graph template is added to a host template, it will automatically associated with all hosts related to that very host template.

Nevertheless, it is sometimes suitable to manually associate a certain graph template with a single host only without changing any host template. This is, where the script `add_graph_template.php` comes in. First, let's have a look at the whole list of features it provides. Calling the script with the parameter `--help` yields

```
shell>php -q add_graph_template.php --help

Add Graph Template Script 1.0, Copyright 2004-2026 - The Cacti Group

A simple command line utility to associate a graph template with a host in Cacti

usage: add_graph_template.php --host-id=[ID] --graph-template-id=[ID]
        [--quiet]

Required:
        --host-id             the numerical ID of the host
        --graph_template-id   the numerical ID of the graph template to be added

List Options:
        --list-hosts
        --list-graph-templates
        --quiet - batch mode value return
```

Let's first stick to the listing options

List all Hosts for a graph template

```
shell>php -q add_graph_template.php --list-hosts

Known Hosts: (id, hostname, template, description)
1       127.0.0.1       8       Localhost
11      router          3       router.mydomain.com
```

List all Graph Template

```
shell>php -q add_graph_template.php --list-graph-templates

Known Graph Templates:(id, name)
2       Interface - Traffic (bits/sec)
3       ucd/net - Available Disk Space
4       ucd/net - CPU Usage
5       Karlnet - Wireless Levels
6       Karlnet - Wireless Transmissions
7       Unix - Ping Latency
8       Unix - Processes
9       Unix - Load Average
10      Unix - Logged in Users
11      ucd/net - Load Average
...
```

Add a Graph Template

```
shell>php -q add_graph_template.php --host-id=11 --graph-template-id=7

Success: Graph Template associated for host: (11: router) - graph-template: (7: Unix - Ping Latency)
```

Add a New Graph

You won't stop now as you've just created a device from cli. Surely, the task of setting up graphs is the next step. This is done using `add_graphs.php`. Calling the script with the parameter `--help` yields

```
shell>php -q add_graphs.php
Add Graphs Script 1.2, Copyright 2004-2026 - The Cacti Group

A simple command line utility to add graphs in Cacti

usage: add_graphs.php --graph-type=[cg|ds] --graph-template-id=[ID]
        --host-id=[ID] [--graph-title=title] [graph options] [--force] [--quiet]

For cg graphs:
        [--input-fields="[data-template-id:]field-name=value ..."] [--force]

        --input-fields  If your data template allows for custom input data, you may specify that
                        here.  The data template id is optional and applies where two input fields
                        have the same name.
        --force         If you set this flag, then new cg graphs will be created, even though they
                        may already exist

For ds graphs:
        --snmp-query-id=[ID] --snmp-query-type-id=[ID] --snmp-field=[SNMP Field] --snmp-value=[SNMP Value]

        [--graph-title=]       Defaults to what ever is in the graph template/data-source template.

        [--reindex-method=]    the reindex method to be used for that data query
                               if data query already exists, the reindex method will not be changed
                        0|None   = no reindexing
                        1|Uptime = Uptime goes Backwards (Default)
                        2|Index  = Index Count Changed
                        3|Fields = Verify all Fields
List Options:
        --list-hosts
        --list-graph-templates [--host_template=[ID]]
        --list-input-fields --graph-template-id=[ID]
        --list-snmp-queries
        --list-query-types  --snmp-query-id [ID]
        --list-snmp-fields  --host-id=[ID] [--snmp-query-id=[ID]]
        --list-snmp-values  --host-id=[ID] [--snmp-query-id=[ID]] --snmp-field=[Field]

    'cg' graphs are for things like CPU temp/fan speed, while
    'ds' graphs are for data-source based graphs (interface stats etc.)
```

Like the graph creation from the console, this task is split into two different ones:

- **Associated Graph Templates:** These are named `cg` graph_type for this script.
- **Associated Data Queries:** These are named `ds` graph_type for this script. Additional SNMP parameters are required to define those graphs.

> **Caution**
>
> When running this cli script, caution must be taken to avoid typos. Sometimes, list options take an ending “s”, where the option for graph creation comes without “s”. Error messages are not issued to indicate this typo; instead you will see the general help screen.

List Options for Associated Graph Templates

The first list option, `--list-hosts`, is required only if you do not know the `id` of the device you want to add graphs for. It will produce output as follows

```
shell>php -q add_graphs.php --list-hosts

Known Hosts: (id, hostname, template, description)
...
11      router.mydomain.com 3       Device Add Test
```

You surely recognize the device we've just added?

The next list option, `--list-graph-templates`, is required to identify the number of an `Associated Graph Templates`.

```
shell>php -q add_graphs.php --list-graph-templates

Known Graph Templates:(id, name)
2       Interface - Traffic (bits/sec)
3       ucd/net - Available Disk Space
4       ucd/net - CPU Usage
5       Karlnet - Wireless Levels
6       Karlnet - Wireless Transmissions
7       Unix - Ping Latency
8       Unix - Processes
9       Unix - Load Average
...
```

We will need the graph template id of the `ucd/net - CPU Usage` template later. So note the number `4` for it.

It is possible to reduce the listing to the graph templates associated with a specific host template by using `--list-graph-templates --host-template-id=[id]`. For a "ucd/net SNMP Host" this yields

```
shell>php -q add_graphs.php --list-graph-templates --host-template-id=3

Known Graph Templates:(id, name)
4   ucd/net - CPU Usage
11  ucd/net - Load Average
13  ucd/net - Memory Usage...
```

List Options for Associated Data Queries

First task is to find all id's for available data queries.

```
shell>php -q add_graphs.php --host-id=2 --list-snmp-queries

Known SNMP Queries:(id, name)
1       SNMP - Interface Statistics
2       ucd/net -  Get Monitored Partitions
3       Karlnet - Wireless Bridge Statistics
4       Netware - Get Available Volumes
6       Unix - Get Mounted Partitions
7       Netware - Get Processor Information
8       SNMP - Get Mounted Partitions
9       SNMP - Get Processor Information
```

For standard interface statistics, we note the id of `1`

Next task is to find the query_type as done by issuing

```
shell>php -q add_graphs.php --snmp-query-id=1 --list-query-types

Known SNMP Query Types: (id, name)
2       In/Out Errors/Discarded Packets
3       In/Out Non-Unicast Packets
```

```
4       In/Out Unicast Packets
9       In/Out Bytes (64-bit Counters)
13      In/Out Bits
14      In/Out Bits (64-bit Counters)
16      In/Out Bytes
20      In/Out Bits with 95th Percentile
21      In/Out Bits with Total Bandwidth
22      In/Out Bytes with Total Bandwidth
23      In/Out Bits 1 min
24      In/Out Bits 1 min (64-bit Counters)
```

For a standard `In/Out Bits` type, we note the id of `13`

Next issue is to select the `SNMP Field` that shall be used for a new graph.

```
shell>php -q add_graphs.php --host-id=11 --list-snmp-fields

Known SNMP Fields for host-id 11: (name)
ifAlias
ifDescr
ifHwAddr
ifIndex
ifIP
ifName
ifOperStatus
ifSpeed
ifType
```

For our graph, we will use the field `ifOperStatus` to select the correct interfaces.

In general, you will next determine the values available for the `SNMP Field`, in our case `ifOperStatus`. This is done by

```
shell>php -q add_graphs.php --host-id=11 --snmp-field=ifOperStatus --list-snmp-values

Known values for ifOperStatus for host 11: (name)
Down
Up
```

This is no surprise, of course. Now, all parameters required for creating a new graph are determined.

Add Non-Indexed Graphs

We will create a graph for `ucd/net - CPU Usage`. Above, we've identified the graph template it to be `4`. In total, that makes up for the following command

```
shell>php -q add_graphs.php --host-id=11 --graph-type=cg --graph-template-id=4

Graph Added - graph-id: (39) - data-source-id: (63)
```

Find the result when visiting Graph Management

Graph Title**	Template Name	Size
Device Add Test - CPU Usage	ucd/net - CPU Usage	120x500

If the graph template was not associated with that host before, it is now added to the list of Associated Graph Templates.

Add Indexed Graphs

First, let's sum up the id's of all resources required for this task:

Table 20-2. Parameters required for an Interface Traffic Graph

Object	Value	Description
Graph Template Id	2	Interface - Traffic (bits/sec)
SNMP Query Id	1	SNMP - Interface Statistics
SNMP Query Type Id	13	In/Out Bits
SNMP Field	ifOperStatus	Field name
SNMP Value	Up	Field value

This will make up for the following command

```
shell>php -q add_graphs.php --host-id=11 --graph-type=ds --graph-template-id=2 --snmp-query-id=1 --snmp-query-type-id=13 --snmp-field=ifOperStatus --snmp-value=Up

Graph Added - graph-id: (40) - data-source-id: (65)
```

Find the result when visiting Graph Management

Graph Title**	Template Name	Size
Device Add Test - CPU Usage	ucd/net - CPU Usage	120x500
Device Add Test - Traffic - lo	Interface - Traffic (bits/sec)	120x500

If the data query was not associated with that host before, it is now added to the list of Associated Graph Templates. In this case, the Re-Index Method of `Uptime Goes Backwards` is defaulted if not given otherwise. You may provide a different Re-Index Method by entering the optional parameter `--reindex-method`.

Add Items to a Tree

Now, that we've created some nice graphs, they should be put the graph trees. This is done using `add_tree.php`. As you will notice soon, it is possible to even add header items, hosts and graphs to a tree as well as adding a new tree itself. But let us first see the help output. Calling the script with the parameter `--help` yields

```
shell>php -q add_tree.php --help

Add Tree Script 1.0, Copyright 2004-2026 - The Cacti Group

A simple command line utility to add objects to a tree in Cacti

usage: add_tree.php  --type=[tree|node] [type-options] [--quiet]

Tree options:
        --name=[Tree Name]
        --sort-method=[manual|alpha|natural|numeric]

Node options:
        --node-type=[header|host|graph]
        --tree-id=[ID]
        [--parent-node=[ID] [Node Type Options]]

Header node options:
        --name=[Name]

Host node options:
        --host-id=[ID]
        [--host-group-style=[1|2]]
        (host group styles:
         1 = Graph Template,
         2 = Data Query Index)

Graph node options:
        --graph-id=[ID]
```

```
        [--rra-id=[ID]]

List Options:
        --list-hosts
        --list-trees
        --list-nodes --tree-id=[ID]
        --list-rras
        --list-graphs --host-id=[ID]
```

List Hosts

The first try is dedicated to the list option `--list-hosts`. It goes like

```
shell>php -q add_tree.php --list-hosts

Known Hosts: (id, hostname, template, description)
1       127.0.0.1           8       Localhost
2       gandalf         3       gandalf
...
11      router.mydomain.com 3       Device Add Test
```

List Trees

Now, let us `--list-trees`. It goes like

```
shell>php -q add_tree.php --list-trees

Known Trees:
id      sort method                     name
1       Manual Ordering (No Sorting)    Default Tree
```

List Nodes

Listing all existed node of a given tree is done by

```
shell>php -q add_tree.php --list-nodes --tree-id=1

Known Tree Nodes:
type    id      text
Host    7       127.0.0.1       Graph Template
Host    9       gandalf     Graph Template
Host    12      gandalf     Graph Template
```

List RRAs

For special tree add options, you will require the id of the RRA definition to completely specify the add request. That's why the corresponding `--list-rras` option is implemented. It goes this way

```
shell>php -q add_tree.php --list-rras

Known RRAs:
id      steps   rows    timespan        name
1       1       600     86400           Daily (5 Minute Average)
2       6       700     604800          Weekly (30 Minute Average)
3       24      775     2678400         Monthly (2 Hour Average)
4       288     797     33053184        Yearly (1 Day Average)
```

List Graphs for given Hosts

To be able to add a Graph, the id of that very graph is required. Thus, a `--list-graphs --host-id=[id]` option was implemented

```
shell>php -q add_tree.php --list-graphs --host-id=1

Known Host Graphs: (id, name, template)
1       Localhost - Memory Usage            Linux - Memory Usage
2       Localhost - Load Average            Unix - Load Average
3       Localhost - Logged in Users         Unix - Logged in Users
4       Localhost - Processes           Unix - Processes
5       Localhost - Traffic - eth0          Interface - Traffic (bits/sec, 95th Percentile)
33      Localhost - Used Space - Memory Buffers Host MIB - Available Disk Space
34      Localhost - Used Space - Real Memory    Host MIB - Available Disk Space
35      Localhost - Used Space - Swap Space     Host MIB - Available Disk Space
36      Localhost - Used Space - /          Host MIB - Available Disk Space
37      Localhost - Used Space - /sys       Host MIB - Available Disk Space
38      Localhost - Used Space - /boot      Host MIB - Available Disk Space
```

Add a new Tree

Cacti comes with a single tree, named Default Tree. Console entry Graph Trees is used to add more trees. With `add_tree.php`, you may now do so from command line as well:

```
shell>php -q add_tree.php --type=tree --name="Test Tree Add" --sort-method=manual

Tree Created - tree-id: (6)
```

Verify this from console, Graph Trees to find

```
Default Tree
Test Tree Add
```

as expected.

Add a new Header Node to a Tree

Now, that a new tree has been created, you may want to add a new header to that very tree. Use

```
shell>php -q add_tree.php --type=node --node-type=header --tree-id=6 --name="Header Test"

Added Node node-id: (21)
```

You will want to save the id returned if willing to add further nodes to exactly this new Header Node

Please note that it is currently not possible to add another header with the same options even if this is possible from console

But it is possible to add a subheader to an already defined header. Even in this case, the `name` has to be unique

```
shell>php -q add_tree.php --type=node --node-type=header --tree-id=6 --parent-node=21  --name="SubHeader Test"

Added Node node-id: (22)
```

Add a new Host Node to a Tree

We will distinguish several options adding a host to a tree. First, let's add a Host directly to a tree. For this example, we use the tree id returned from adding our own `Test Tree Add` known with `id=6`

```
shell>php -q add_tree.php --type=node --node-type=host --tree-id=6 --host-id=1

Added Node node-id: (23)
```

As no `--parent-node` was given, this host is directly added to the tree itself. If you wish to add a host to a (sub)header specify as follows

```
shell>php -q add_tree.php --type=node --node-type=host --tree-id=6 --parent-node=21 --host-id=1

Added Node node-id: (24)
```

Both example come without a `--host-group-style` option. For those cases, `Graph Template` host group style is default. Here's an example for providing a host group option of `Data Query Index` instead

```
shell>php -q add_tree.php --type=node --node-type=host --tree-id=6 --parent-node=22 --host-id=1 --host-group-
style=2

Added Node node-id: (25)
```

Add a new Graph Node to a Tree

Like above, instead of hosts it is possible to add a single graph to a tree or a (sub)header of any tree. Of course, you again will require the `id` of the tree and optionally of the `header`. This results in

```
shell>php -q add_tree.php --type=node --node-type=graph --tree-id=6 --graph-id=5

Added Node node-id: (26)
```

Like above, this graph now was added directly to the tree itself. To add a graph to a header, proceed as follows

```
shell>php -q add_tree.php --type=node --node-type=graph --tree-id=6 --parent-node=21 --graph-id=5

Added Node node-id: (27)
```

In both cases, no explicit `--rra-id` was given. This will default to the Daily (5 Minute Average). Specify any other `--rra-id` as given

```
shell>php -q add_tree.php --type=node --node-type=graph --tree-id=6 --parent-node=21 --graph-id=1 --rra-id=4

Added Node node-id: (28)
```

to provide the `--rra-id` for a rra of Yearly (1 Day Average).

Add Graph Permissions

This script is used to specify special graph permissions to users. The list of parameters is displayed as usual when calling the script with the parameter `--help`

```
shell>php -q add_perms.php --help

Add Permissions Script 1.0, Copyright 2004-2026 - The Cacti Group

A simple command line utility to add permissions to tree items in Cacti

usage: add_perms.php [ --user-id=[ID] ]
        --item-type=[graph|tree|host|graph_template]
        --item-id [--quiet]

Where item-id is the id of the object of type item-type
List Options:
        --list-users
        --list-trees
```

```
        --list-graph-templates
        --list-graphs --host-id=[ID]
```

List Users

The list of users is retrieved by calling the script with the parameter `--list-users`. It goes like

```
shell>php -q add_perms.php --list-users

Known Users:
id      username        full_name
1       admin   Administrator
3       guest   Guest Account
```

List Trees who accept permissions

The list of trees is retrieved by calling the script with the parameter `--list-trees`. It goes like

```
shell>php -q add_perms.php --list-trees

Known Trees:
id      sort method                     name
1       Manual Ordering (No Sorting)    Default Tree
```

List Graph Templates

The list of available graph templates is retrieved by calling the script with the parameter `--list-graph-templates`. It goes like

```
shell>php -q add_perms.php --list-graph-templates

Known Graph Templates:(id, name)
2       Interface - Traffic (bits/sec)
3       ucd/net - Available Disk Space
4       ucd/net - CPU Usage
5       Karlnet - Wireless Levels
6       Karlnet - Wireless Transmissions
7       Unix - Ping Latency
8       Unix - Processes
9       Unix - Load Average
...
```

List Graphs for given Hosts where permissions can be set

To be able to add a permissions to a specific Graph, the id of that very graph is required. Thus, a `--list-graphs --host-id=[id]` option was implemented

```
shell>php -q add_perms.php --list-graphs --host-id=1

Known Host Graphs: (id, name, template)
1       Localhost - Memory Usage            Linux - Memory Usage
2       Localhost - Load Average            Unix - Load Average
3       Localhost - Logged in Users         Unix - Logged in Users
4       Localhost - Processes           Unix - Processes
5       Localhost - Traffic - eth0          Interface - Traffic (bits/sec, 95th Percentile)
33      Localhost - Used Space - Memory Buffers Host MIB - Available Disk Space
34      Localhost - Used Space - Real Memory    Host MIB - Available Disk Space
35      Localhost - Used Space - Swap Space     Host MIB - Available Disk Space
36      Localhost - Used Space - /          Host MIB - Available Disk Space
37      Localhost - Used Space - /sys       Host MIB - Available Disk Space
38      Localhost - Used Space - /boot      Host MIB - Available Disk Space
```

Add Graph Permissions to specific Users

There are various ways to define graph permissions to specific users. First, we will add graph permissions for a given host. This is done using the parameter `--item-type=host` as follows

```
shell>php -q add_perms.php --user-id=4 --item-type=host --item-id=1
```

`--user-id=4` must specify an existing users. Else, an error message will be printed. As `--item-type=host` is given, `--item-id=1` must specify a valid host. Again, if no host with this specific id exists, an error message will be printed. No output is produced on successful operations.

Next, we will add graph permissions for a given tree. This is done using the parameter `--item-type=tree` as follows

```
shell>php -q add_perms.php --user-id=4 --item-type=tree --item-id=1
```

`--user-id=4` must specify an existing users. Else, an error message will be printed. As `--item-type=tree` is given, `--item-id=1` must specify a valid tree (the Default Tree in this case). Again, if no tree with this specific id exists, an error message will be printed. No output is produced on successful operations.

Adding graph permissions for a given graph template is done using the parameter `--item-type=graph_template` as follows

```
shell>php -q add_perms.php --user-id=4 --item-type=graph_template --item-id=25
```

`--user-id=4` must specify an existing users. Else, an error message will be printed. As `--item-type=graph_template` is given, `--item-id=1` must specify a valid graph_template (the Interface - Traffic (bytes/sec) in this case). Again, if no graph template with this specific id exists, an error message will be printed. No output is produced on successful operations.

Last but not least we add graph permissions for a given graph by using the parameter `--item-type=graph` as follows

```
shell>php -q add_perms.php --user-id=4 --item-type=graph --item-id=8
```

`--user-id=4` must specify an existing users. Else, an error message will be printed. As `--item-type=graph` is given, `--item-id=1` must specify a valid graph. Again, if no graph template with this specific id exists, an error message will be printed. No output is produced on successful operations.

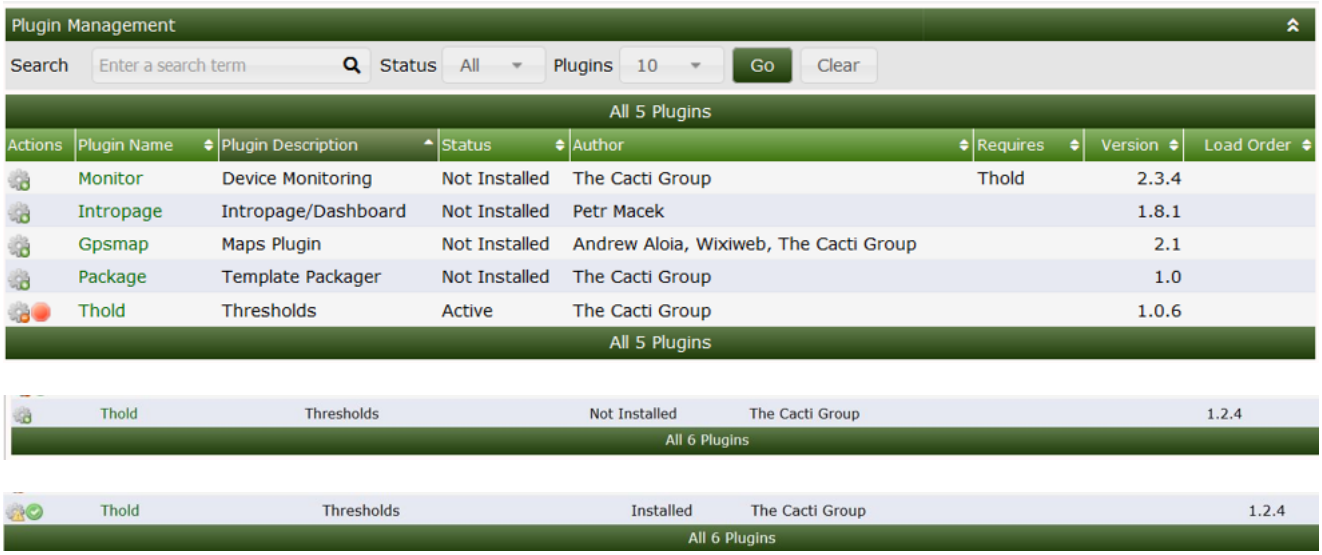
Copyright (c) 2004-2026 The Cacti Group

Plugin Management

Plugins extend Cacti with additional features. Before installing one, read its README and `INFO` file, check its compatibility with your Cacti release, and review required dependencies and database changes.

Installing Plugins

1. Download a compatible release from the plugin's project repository. Place its files under `<path_cacti>/plugins/` using the directory name expected by the plugin. For example, `plugin_thold` is installed in `plugins/thold`, not `plugins/plugin_thold` or an extra nested directory.
2. Make the code readable by the web server and poller. Use the ownership and permissions required by your installation; application code generally does not need to be writable by the web server.
3. Open **Console > Configuration > Plugins** and install the plugin.
4. Enable it, complete its configuration, and grant the intended users access.



Do not assume the repository's development branch is a release suitable for production. Use the plugin's release instructions and verify any prerequisites reported by the management page.

Updating a Plugin

Back up the Cacti database, plugin files, and any plugin-specific data before updating. Read the release's migration instructions, including any required Cacti or plugin dependencies.

1. Disable the plugin without uninstalling it.
2. Replace or update the code in its existing directory, preserving local configuration as directed by the plugin's instructions.
3. Check file ownership and permissions.
4. Re-enable the plugin, complete any required upgrade steps, and verify its functionality and scheduled jobs.

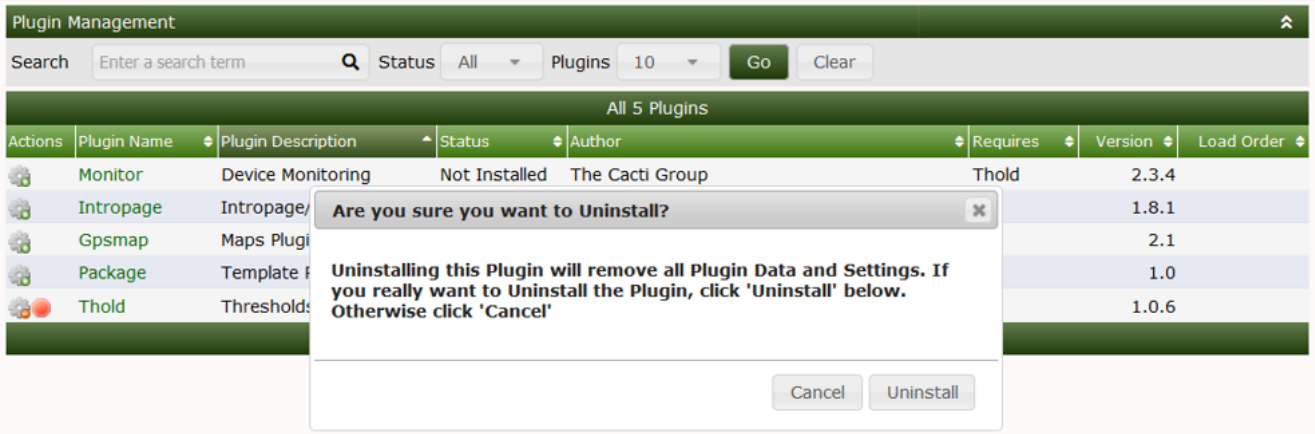
Do not uninstall a plugin as an upgrade step. Uninstalling can remove its tables, settings, and stored data. Disabling and uninstalling are different operations.

Video Walkthrough

[Video Tutorial available online]

Uninstall a Plugin

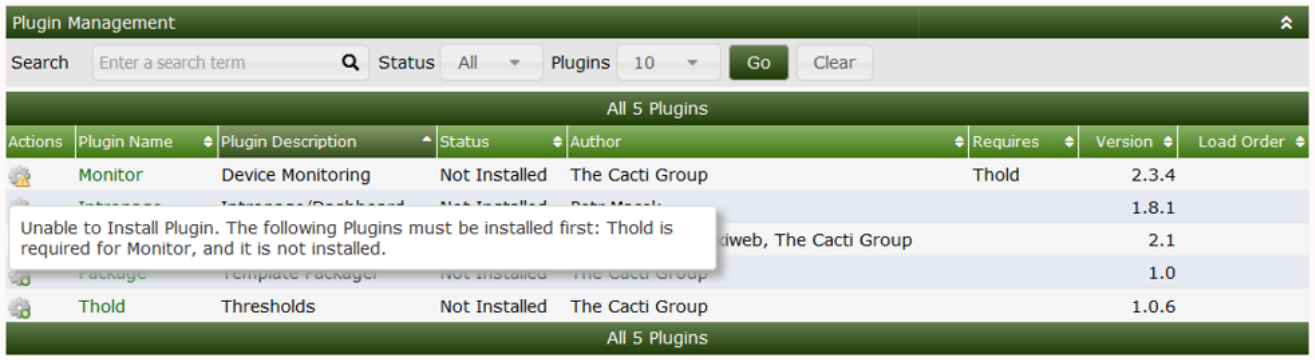
Disable the plugin under **Console > Configuration > Plugins**, then use its uninstall action and review the confirmation. Back up data before proceeding: uninstall behavior is plugin-specific and can delete database tables.



Uninstalling does not necessarily remove the plugin's files. Remove its code from `plugins/` separately when it is no longer needed. Some plugins expose an additional reset or data-removal action; review what it deletes before using it.

Dependencies in plugins

Dependencies vary by plugin and release. Use its `INFO` file, README, and the management page's dependency message to identify what must be installed first. Do not infer a dependency merely because plugins are commonly used together.



Copyright (c) 2004-2026 The Cacti Group

Supported plugins

Cacti group has developed and is still developing a large number of plugins extending cacti. Some of them are adopted from the community.

Of course plugins can be developed by anyone, our documentation contains everything needed. We do our best to keep the plugins we support working with current versions of cacti.

Here is a list of the plugins we support and a short description of them.

- **Thold**

  The Cacti thold plugin is designed to be a fault management system driven by Cacti's Graph information. It provides the facility to inspect data in a Cacti Graph and the underlying RRDfile, and generate alerts for management and operations personnel. It provides Email, Syslog, and SNMP Trap or Inform escalations. In addition, it also can notify personnel of Cacti Device status changes.

  - Hi/Low, Time Based and Baseline deviation thresholds
  - Realerting
  - Multiple Data Manipulation types, including: Exact Value, CDEF, Percentage, and RPN Expression
  - Ability to Generate Tickets to Ticketing Systems through Script Execution
  - Acknowledgment levels

  Thold plugin homepage

- **Intropage**

  User defined dashboards for Cacti. Each user can create own dashboards.

  - Displays useful information about Cacti
  - Add graphs to dashboard
  - Support plugins like thold, syslog, ...
  - Share dashboards between users
  - Respects Cacti's user permissions

  Intropage plugin homepage

- **Maint**

  Plugin is for the scheduling of maintenance so that Thold/Servcheck and other plugins will not alert during that time period.

  - One time or Recurring schedule

  Maint plugin homepage

- **Monitor**

  This plugin allows you to view at a glance all your critical Cacti hosts, and will alert you audibly and via Email when a Device or Devices go down.

  - Data Center Dashboard
  - Audible and Visual Alerting
  - Respects Cacti's user permissions
  - Monitoring can be enabled or disabled at the Device level
  - Supports Monitoring Devices by Criticality

  Monitor plugin homepage

- **Mikrotik**

  This plugin provides support for the MikroTik RouterOS available from MikroTik and other router and switch hardware suppliers. It is not a replacement for MikroTik's GUI interface, but provides nice Cacti Graphs.

  - Automatic Discovery of Cacti Devices that are MikroTik's
  - Enhanced Traffic Graphs
  - Graphs of Wireless and PPPoE Users
  - Table views of Wireless and PPPoE User utilization
  - Graphs of MikroTik environmental data such as temperature and voltage
  - Viewing of Devices and summary statistics, Users, Queue Trees, Access Points, HotSpots, Interfaces, and Wireless Station information

  Mikrotik plugin homepage

- **Routerconfigs**

  The routerconfigs plugin is designed to receive and create backups from your network devices. It also provides the ability to view and diff those router configurations as they change over time. It is designed primarily for Cisco device types, but may work with other device types.

  Routerconfigs plugin homepage

- **ReportIt**

  This plugin creates tabular reports which can be exported to CSV, SML and XML as well.

  - Definition of individual report templates by using measurands and variables in a mathematical way
  - Individual configuration of working days, working time, timezone and subheads per data item
  - Scheduled reporting with sliding time frames
  - Export to CSV, SML and XML
  - Different ways of connecting RRDtool
  - Scheduled reports via email

  Reportit plugin homepage

- **Evidence**

  Plugin provides information about device serial numbers, software/firmware versions and much more information from SNMP Entity MIB

  - Search serial number, problematic firmware version
  - Email notification when changes
  - Saves past configurations, ability to view changes in time
  - Vendor specific information

  Evidence plugin homepage

- **Mactrack**

  Scan network devices (switches, routers, ..) for connected devices

  - Find MACs, IPs, VLANs
  - Associate IP and MAC
  - Where is device connected
  - ARPwatch support
  - Finds Stolen/Lost PC's
  - Tells you when someone is connected who shouldn't be

Mactrack plugin homepage

- **FlowView**

  This plugin allows you to view NetFlow data from inside of Cacti and to generate Email based reports of the data.

  - Ability to view Flow Data in Table Form or three Chart Forms
  - Parallel Query Execution to increase the speed of report generation
  - Support for Shard Query and Horizontal Scaling through use of technologies such as MariaDB MaxScale
  - Tracking of IP addresses that do not resolve through DNS by using ARIN's whois service
  - Customizable Scheduled Reports

  Flowview plugin homepage

- **Servcheck**

  Internet services (http, imap, ldap, ...) monitoring plugin. This plugin allows you to add service monitoring to Cacti. You simply add service check, allow service specific test (like certificate test) and you can add the expected response. Servcheck periodically runs test and notifies if a service check fails. The plugin records statistics about the connection, it's response, and can alert when the status changes.

  - tests for scp, tftp, ftp, smb, ..
  - import own certificate for test
  - notify about soon certificate expiration
  - notify when last result is different

  Servcheck plugin homepage

- **HMIB**

  The hmib plugin is designed to collect SNMP information from Cacti Devices that support the SNMP Host Resources Mib structure. This SNMP information includes performance metrics like CPU utilization, running processes and applications, hardware details, ...

  Hmib plugin homepage

- **Weathermap**

  Weathermap is one of many implementations of the same basic idea - take data from your network devices and use it to provide a single-page overview of the current state of network, like the one to the right.

  Weathermap plugin homepage

- **Syslog**

  It provides a simple Syslog event search an Alert generation and notification interface that can generate both HTML and SMS messages for operations personnel who wish to receive notifications inside of a data or network operations center. When combined by the Linux SNMPTT package, it can be converted into an SNMP Trap and Inform receiver and notification engine as the SNMPTT tool will receive SNMP Traps and Informs and convert them into Syslog messages on your log server. These syslog messages can then be consumed by the syslog plugin. So, this tool is quite handy

  - Message filtering
  - Message searching
  - Message Alerting
  - Alert Methods of Individual and Threshold Based
  - Remote Log Server connection capabilities
  - Ability to Generate Tickets to Ticketing Systems through Script Execution
  - Ability to run alert specific commands at Alert and Re-alert times

Syslog plugin homepage

- **Slowlog**

  The plugin will allow a Cacti Administrator to import their slow Query log and perform analysis of that slow log.

  Slowlog plugin homepage

- **Apcupsd**

  This plugin is to be used to track UPS status from the apcupsd daemon as well as SNMP enabled UPS. The apcupsd daemon is able to provide information on APC and other types of UPS' over various network typologies such as USB, Modbus, Ethernet, etc. We have added basic SNMP support in order to provide direct access to certain models over the SNMP protocol thus removing the need for an apcupsd host to provide the monitoring details as well.

  - Track UPS status
  - Log information to Cacti database

  Apcupsd plugin homepage

- **WMI**

  The WMI Plugin provides data collection services for devices that support the WMI protocol. It operates asyncrhonously to the Cacti data collection and stores the resulting information into cache tables. These cache tables can then be used to generate graphs, thresholds and alerts. It relies on the 'wmic' command in Linux, and WMI services via Microsofts COM protocol from Windows Cacti servers.

  WMI plugin homepage

syslog

The syslog plugin is a Cacti plugin that has been around for more than a decade. It was inspired by the 'aloe' and 'h.aloe' plugins originally developed by the Cacti users sidewinder and Harlequin in the early 2000's. As you will be able to see from the ChangeLog, it has undergone several changes throughout the years, and remains, even today when you have enterprise offering from both Elastic and Splunk, remains a relevant plugin for small to medium sized companies.

It provides a simple Syslog event search an Alert generation and notification interface that can generate both HTML and SMS messages for operations personnel who wish to receive notifications inside of a data or network operations center.

For log events that continue to be generated frequently on a device, such as smartd's feature to notify every 15 minutes of an impending drive failure, can be quieted using syslog's 'Re-Alert' setting.

Syslog also provides multipoller support which allows for scalability and redundancy by leveraging multiple servers rules can be created on the main poller and pushed to the remotes for ease of management or each server can work independently

Features

- Message filter
- Message search
- Output to screen or file
- Date time picker
- Event Alerter
- Event Removal (for Events you don't want to see)
- Filter events by Cacti Graph window from Cacti's Graph View pages
- Use of native MySQL and MariaDB database partitioning for larger installs
- Remote Log Server connection capabilities
- Custom column mappings between Remote Log Server and required Syslog columns
- Remote poller support

Notes on upgrades

If you are upgrading to 2.0 from a prior install, you must first uninstall syslog and ensure both the syslog, syslog_removal, and syslog_incoming tables are removed, and recreated at install time.

In addition, the rsyslog configuration has changed in 2.5. So, for example, to configure modern rsyslog for Cacti, you MUST create a file called cacti.conf in the /etc/rsyslog.d/ directory:

Cacti Configuration for RSYSLOG

Edit /etc/rsyslog.d/cacti.conf

```
$ModLoad imudp
$UDPServerRun 514
$ModLoad ommysql

$template cacti_syslog,"INSERT INTO syslog_incoming(facility_id, priority_id,
  program, logtime, host, message) \
```

```
  values (%syslogfacility%, %syslogpriority%, '%programname%',
  '%timegenerated:::date-mysql%', '%HOSTNAME%', TRIM('%msg%'))", SQL

*.* >localhost,my_database,my_user,my_password;cacti_syslog
```

This is a change from versions 2.0 to 2.4 and below, which had the following file format:

```
$ModLoad imudp
$UDPServerRun 514
$ModLoad ommysql
```

$template cacti_syslog,"INSERT INTO syslog_incoming(facility_id, priority_id, program, date, time, host, message) values (%syslogfacility%, %syslogpriority%, '%programname%', '%timereported:::date-mysql%', '%timereported:::date-mysql%', '%HOSTNAME%', TRIM('%msg%'))", SQL

. >localhost,my_database,my_user,my_password;cacti_syslog

for RHEL, Rocky Linux, or AlmaLinux systems you will all need to install the rsyslog-mysql package

```
yum install rsyslog-mysql
systemctl resatrt rsyslog
```

If you are upgrading to version 2.5 from an earlier version, make sure that you update this template format and restart rsyslog. You may lose some syslog data, but doing this in a timely fashion, will minimize data loss.

Ensure you restart rsyslog after these changes are completed. Other logging servers such as Syslog-NG are also supported with this plugin. Please see some additional documentation in the plugin_syslog repository

We are using the pure integer values that rsyslog provides to both the priority and facility in this version syslog, which makes the data collection must less costly for the database. We have also started including the 'program' syslog column for searching and storage and alert generation.

Device setup

To setup log forwarding from your network switches and routers, and from your various Linux, UNIX, and other operating system devices, please see their respective documentation.

For Cisco routers switches configuration may look something like this

```
SW-3750#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
SW-3750(config)#logging host 192.168.1.194
SW-3750(config)#logging facility local7
SW-3750(config)#logging rate-limit 100
SW-3750(config)#
```

Mariadb/MYSQL Specific configurations

Finally, it's important, especially in more recent versions of MySQL and MariaDB to set a correct SQL Mode. These more recent SQL's prevent certain previously allowable syntax such as an empty data and certain group by limitations in the SQL itself. Therefore, you need to ensure that the SQL mode of the database is correct. To do this, first start by editing either `/etc/my.cnf` or `/etc/my.cnf.d/server.cnf` and inserting the SQL mode variable into the database configuration. For example:

```
[mysqld]
sql_mode=NO_ENGINE_SUBSTITUTION,NO_AUTO_CREATE_USER
```

After this change, you should log into the mysql server and run the following command:

```
mysql> show global variables like 'sql_mode';
```

And ensure that it matches the setting that you placed in the database configuration. If it does not, please search for the configuration that is making this SQL mode other than what you required. More recent versions of MySQL and MariaDB will source multiple database configuration files.

Installing the plugin

1.) Copy the syslog files over to /var/www/html/cacti/plugins

2.) ensure permissions are correct the files should typically be owned by the webserver user

```
chown -R apache:apache syslog
```

3.) copy config.php.dist to config.php

5.) You have two options for storing syslog information you can either use the existing Cacti Database or use a dedicated database for syslog as syslog databases especially for large networks can grow pretty quick it may be wise to create a dedicated database. To use a dedicated DB first create a database in mysql and assign a user you will then change

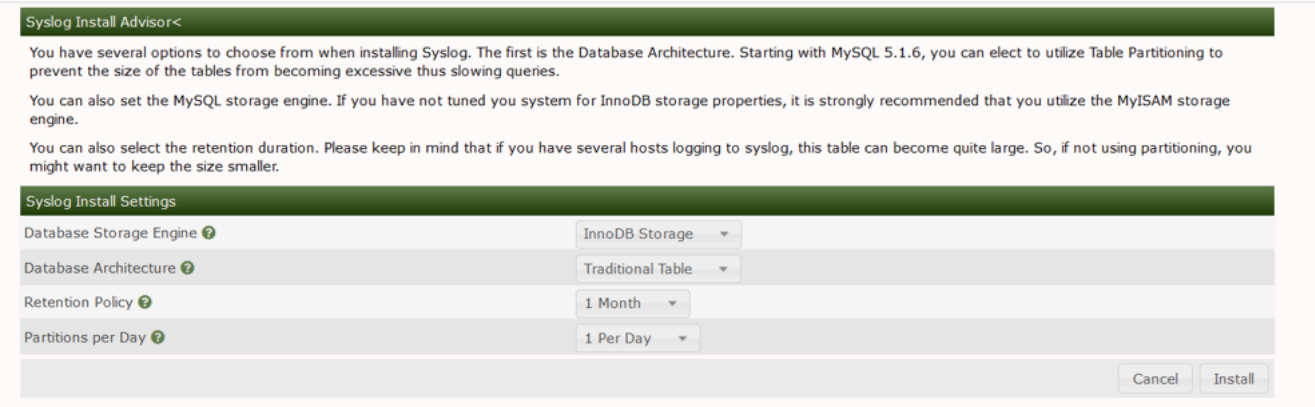
```
$use_cacti_db = true;
to
$use_cacti_db = false;
```

if you intend to use the cacti DB leave config.php as default

6.) install the plugin in the plugins tab located in Console >> configuration >> plugins

7.) You will be presented with the plugin install wizard with options on how you would like the Syslog plugin to be installed options include DB arch and retention time

Note retention settings can be changed after install but db architecture will require a reinstall of the plugin



You will also need to ensure the cacti user is granted select on the Syslog database

```
GRANT SELECT  ON syslog.* TO  'cacti'@'localhost';
```

SNMP Trap configuration

To leverage SNMP traps as mentioned you will need to install SNMPTT. SNMPTT is used to translate OID information passed from the device to a readable format SNMPTT will then write that data into syslog for the Syslog plugin to ingest.

Follow these steps to complete the setup

1.) yum install snmptt

2.) enable snmptt and snmptrapd

```
systemctl enable snmptrapd
systemctl enable snmptt
systemctl start snmptt
systemctl start snmptrapd
```

3.) Now that snmptt and snmptrapd are started and enabled you will first need to modify the snmptrapd systemd file to include -on in the startup options

the file is located in /etc/sysconfig/snmptrapd and should look like this after

```
# snmptrapd command line options
# '-f' is implicitly added by snmptrapd systemd unit file
OPTIONS="-on"
```

4.) Now you need to modify the snmptrapd config file to receive snmptraps by default snmptrapd will not process any traps until you configure it to do so you will also need to tell snmptrapd to forward the received traps over to snmptt

Here is an example of configuring snmptrapd to receive snmptraps from a device using the public snmp string and forwarding it over to snmptt

```
# Example configuration file for snmptrapd
#
# No traps are handled by default, you must edit this file!
#
# authCommunity   log,execute,net public
# traphandle SNMPv2-MIB::coldStart    /usr/bin/bin/my_great_script cold
authCommunity   log,execute,net public
traphandle default /usr/sbin/snmptthandler
```

after making all of the changes to the snmptrapd configuration restart the snmptrapd process for the changes to take effect.

5.) configure snmptt.ini to operate in daemon mode

By default snmptt already writes data to syslog all that is left is to configure daemon mode in snmptt.ini which is located in /etc/snmp

```
# Set to either 'standalone' or 'daemon'
# standalone: snmptt called from snmptrapd.conf
# daemon: snmptrapd.conf calls snmptthandler
# Ignored by Windows.  See documentation
mode = daemon
```

6.) use the snmptt cacti connector

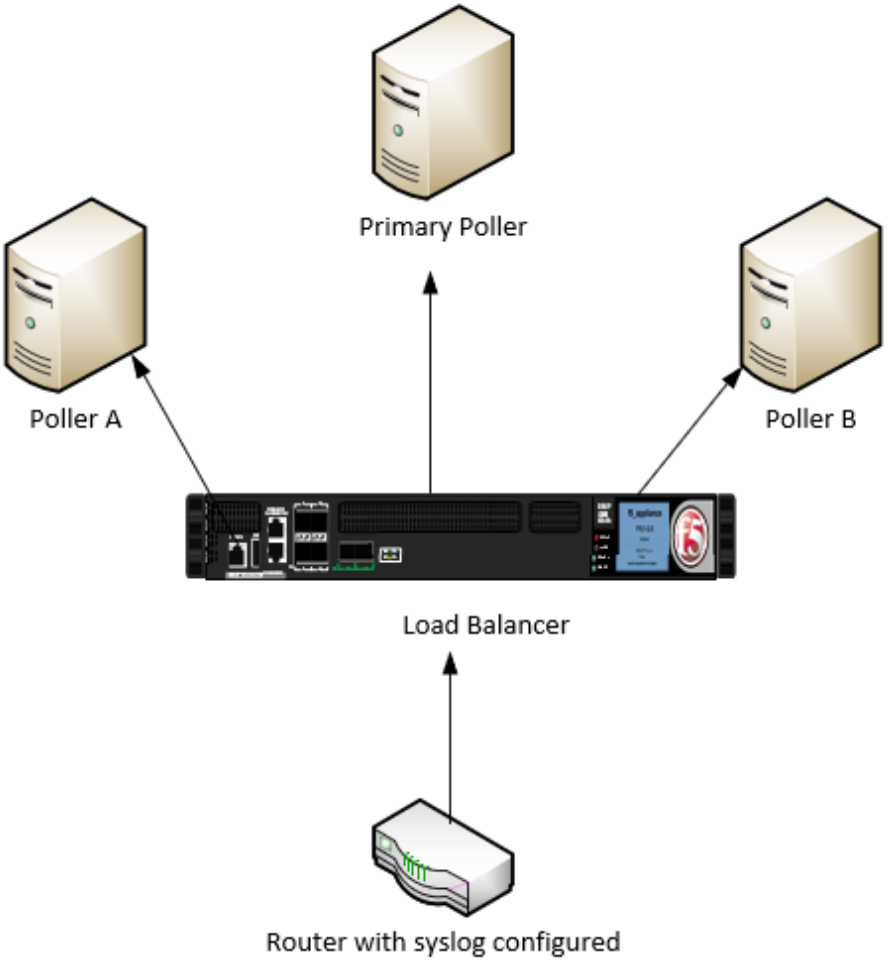
By default since snmptt resides on the syslog server all of the IP information will appear as if its coming from localhost to fix this there is a cacti snmptt connector which will format the trap to be ingested to the syslog db it is located in the contrib folder

Remote poller deployment options

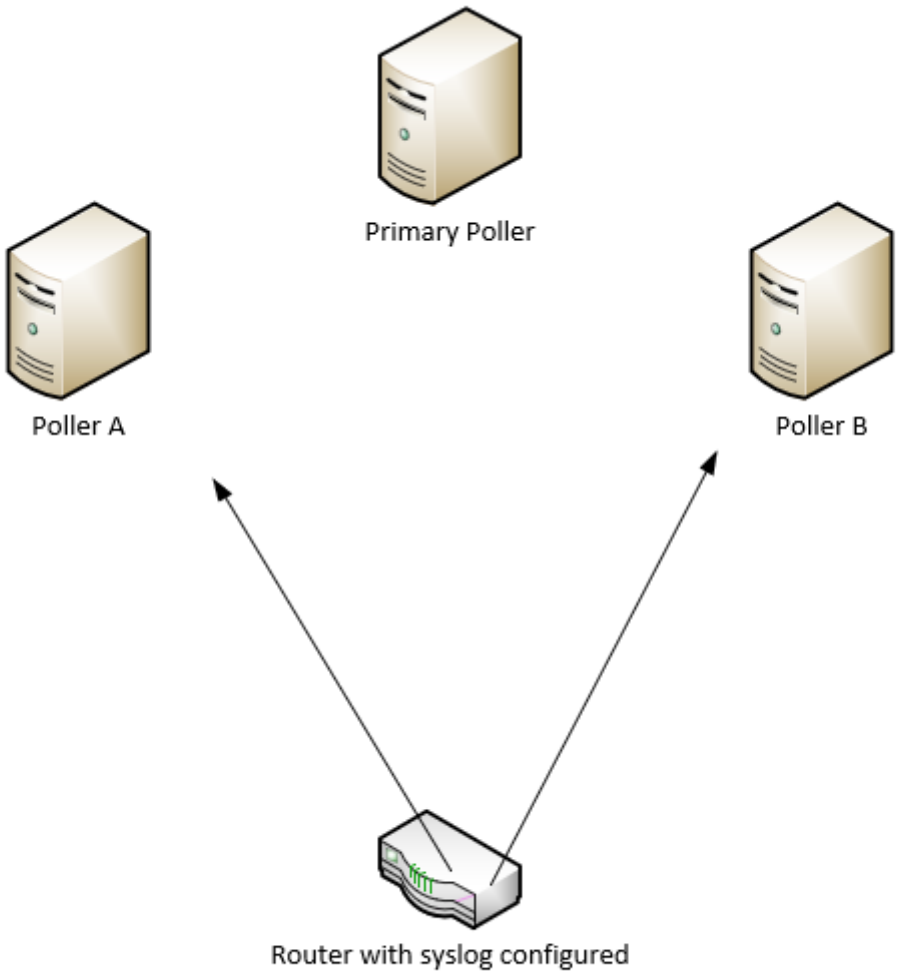
The syslog plugin can be deployed on each poller or select pollers each poller will need to be configured with rsyslog each poller will need to be configured with a syslog config_local file

There are a few deployment approaches

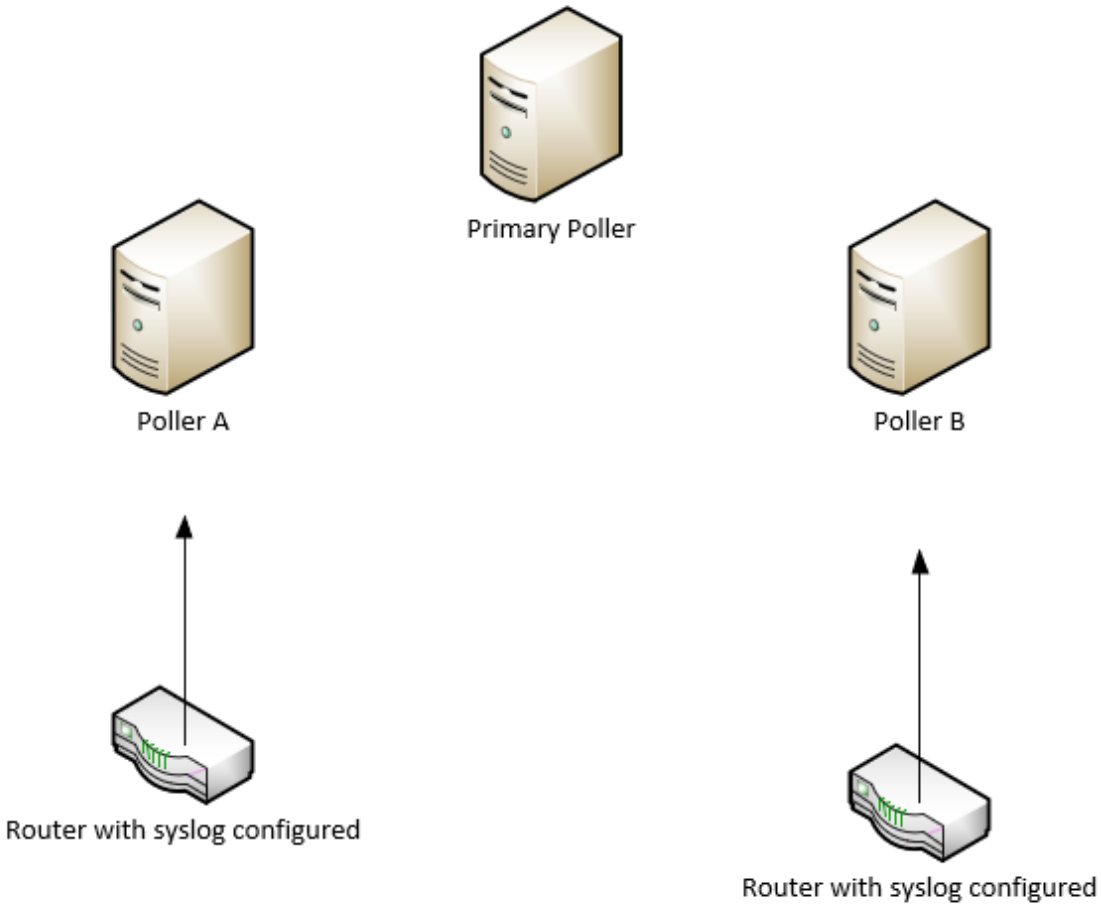
1.) Multiple Pollers with a load balancer for syslog



2.) Multiple pollers no balancer but devices send syslog to multiple pollers



3.) Multiple pollers but devices are assigned to send Syslog to a specific poller for example poller A and device A is at one location



Remote poller setup

To setup each remote poller you will need to enable and configure rsyslog with the following steps

1.) decide if you will use the cacti DB or a separate DB

Note on using the cacti DB as mentioned earlier

You have two options for storing Syslog information you can either use the existing Cacti Database or use a dedicated database for Syslog as Syslog databases especially for large networks can grow pretty quickly it may be wise to create a dedicated database. To use a dedicated DB first create a database in MySQL and assign a user then

```
$use_cacti_db = true;
to
$use_cacti_db = false;
```

2.) If you decide to use the cacti database simply leave config_local.php default

```
$use_cacti_db = true;
```

3.) If you decide to use a separate DB fill out the Database detailing config_local.php

4.) create a cacti.conf file in /etc/rsyslog.d and fill out either your separate DB details or your main cacti DB details

```
$ModLoad imudp
$UDPServerRun 514
$ModLoad ommysql

$template cacti_syslog,"INSERT INTO syslog_incoming(facility_id, priority_id,
  program, date, time, host, message) \
  values (%syslogfacility%, %syslogpriority%, '%programname%', '%timereported:::date-mysql%',
  '%timereported:::date-mysql%', '%HOSTNAME%', TRIM('%msg%'))", SQL

*.* >localhost,my_database,my_user,my_password;cacti_syslog
```

5.) install rsyslog-mysql package to allow rsyslog to write to mysql

```
yum install rsyslog-mysql
```

6.) restart rsyslog

```
systemctl restart rsyslog
```

7.) if you would like for rules to be replicated from the main poller to the remotes you will need to enable the following syslog settings found in configuration >> settings >> syslog

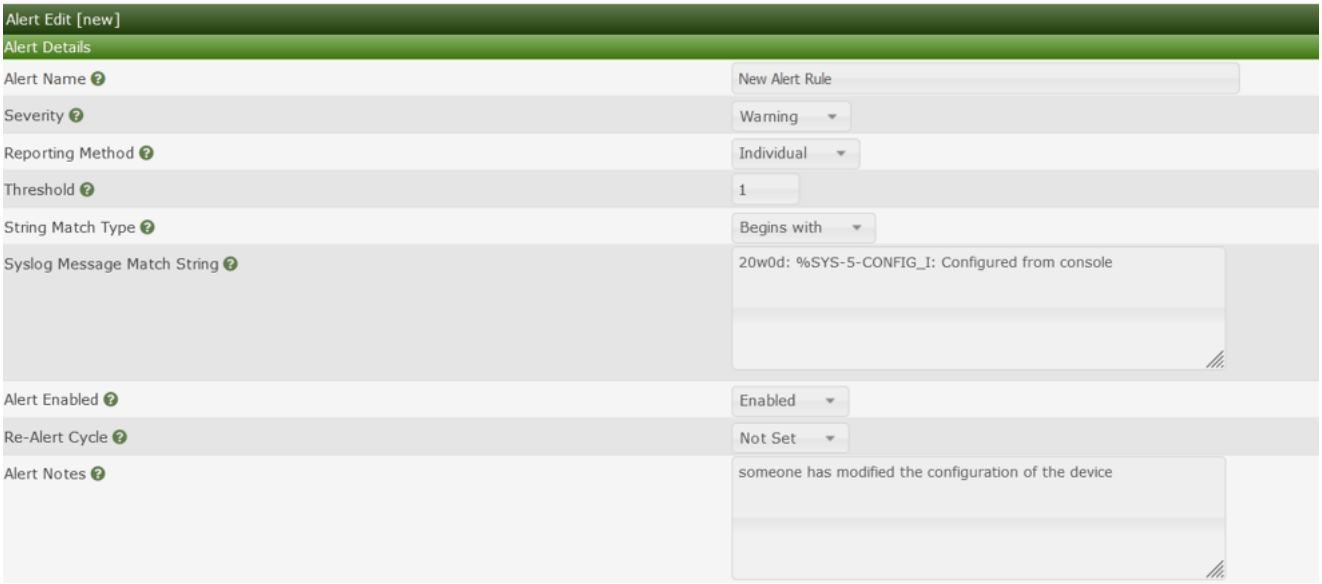
note rules will be replicated within one polling cycle to the remote pollers

if you wish to have each poller operate independently there is no need to enable these options.

Syslog Alert Rules

Alert rules are used to generate alerts in the cacti log they are also used to send out email alerts and command execution to cut tickets to upstream ticketing systems or perform initial troubleshooting scripts can also be leveraged to call on services such as slack or trello for alerting teams.

you can leverage the built in variable substitution to format the input to the script



Syslog Removal Rules

The Syslog plugin allows administrators to delete incoming messages so they are not stored in the DB. For example, if you are not interested in seeing Syslog messages every time a user logs into a device you may add a rule to delete the message.

to do this click on the Syslog settings tab in the console then click removal rules

Possible Bugs and Feature Enhancements

Bug and feature enhancements for the syslog plugin are handled in GitHub. If you find a first search the Cacti forums for a solution before creating an issue in GitHub.

Authors

The syslog plugin has been in development for well over a decade with increasing functionality and stibility over that time. There have been several contributors to thold over the years. Chief amongst them are Jimmy Conner, Larry Adams, SideWinder, and Harlequin. We hope that version 2.0 and beyond are the most stable and robust versions of syslog ever published. We are always looking for new ideas. So, this won't be the last release of syslog, you can rest assured of that.

Copyright (c) 2004-2026 The Cacti Group

MacTrack

The MacTrack plugin is designed to scan network switches, routers and intelligent hubs for connected devices, and record their location either based upon the portname or alias of the switch or hub. It also attempts to discover the ip address of the mac address from the routers included in the MacTrack database. MacTrack can also use arpwatch to gether IP to MAC address associations.

MacTrack has the ability to also notify admins or security personnel when the MAC address of a missing or stolen computer re-appears on the network. This can be helpful in recovering lost equipment. Through MacTrack's interface monitoring feature, a Network Administrator can get a very good idea of where utilization is, where there are errors, etc within their network.

In howto is mentioned menu item "Mactrack" or "Mactrack tab". In older versions is called "Device tracking".

Mactrack setup

- Check default settings Configuration -> Settings -> Mactrack tab (or Device tracking in older version)
- Create sites (mandatory) At this moment Mactrack's sites have no relation to Cacti's sites
- Define Device Types (mandatory) Several device types are provided as an example with the current version. Devices without device type match will not be processes
- Import IEEE OUI Database (optional) Define SNMP set options to use

##Create site One of the mandatory steps to start setting Mactrack up is to create sites where we are going to allocate our Mactrack devices. Console -> Mactrack -> Sites You need add devices to site. You can import some existing Cacti devices into Mactrack or add devices manually on Console -> Mactrack -> Devices

Device types

Device Types

Console -> Mactrack (Device Tracking in previous versions) -> Device Types menu

Create as many device types as you need following this instrunctions. **Description** - give this device type a meaningful description **Vendor** - fill in the name for the vendor of this device type **Device Type** - choose the type of device. What kind of device is it? Meaning that some scanning functions will be available for some of these types only. Eg: If you set a switch/hub as "Switch/Router" you will then be able to pick an IP scanning function up when the device doesn't really supports it. **System Description Match** - provide key information to help Device Tracking detect the type of device. The wildcard character is the % sign. When you open a device in Cacti, you normally see the following on top of the page. `System: Cisco IOS Software, C2960 Software (C2960-LANBASEK9-M), Version 12.2(53)SE2, RELEASE SOFTWARE (fc3)Technical Support: http://www.cisco.com/techsupportCopyright (c) 1986-2010 by Cisco Systems, Inc.Compiled Wed 21-Apr-10 05:52 by prod_rel_team Uptime: 3176562306 Hostname: myswitch101.com ObjectID: .1.3.6.1.4.1.9.1.696` What you want to make Mactrack look at is something like "C2960" which quickly identify the hardware platform. Don't waste your time try to match something different or more complex like "12.2(53)SE2".

Vendor SNMP Object ID Match - this is the SNMP OID on this format: .1.3.6.1.4.1.9.1.696 **MAC Address Scanning Function** - we assume you know what you are looking for here. This function should only be set to switches L2/L3 and not Routers. **IP Address Scanning Function** - this function allows you to look at the ARP table and map IP vs MAC. There is an special function for Cisco devices called get_cisco_dhcpsnooping_table that will look at the DHCP Snooping table for switches if enabled. **802.1x Scanning Function** - this scanning function was introduced on 4.2 version and it's only for Cisco devices. **Serial Number Base OID** - this isn't mandatory but you can specify the SNMP OID to find the serial number to the device. **Serial Number Collection Method** - how is the serial number collected for this OID. If 'SNMP Walk', we

assume multiple serial numbers. If 'Get', it will be only one. **Low User Port Number** - provide the low user port number on this switch. Leave 0 to allow the system to calculate it. Empty assumes 0 **High User Port Number** - provide the low user port number on this switch. Leave 0 to allow the system to calculate it. Empty assumes 0

Export device types option allows you to export the existing devices types to CSV format so you can later on import them onto new installations, to share them or just to back them up. Go to Console -> Mactrack -> Device Types and click the export button. For import Device Types you can use button "Import" on the same page.

##Import OUI Database from IEEE

This function will download and install the latest OUI database from the IEEE Website. Each Network Interface Card (NIC) has a MAC Address. The MAC Address can be broken into two parts. The first part of the MAC Address contains the Vendor MAC. The Vendor MAC identifies who has manufactured the part. This will be helpful to check for rogue devices on your network.

Go to Mactrack -> Tracking Utilities and click on **Refresh IEEE Vendor MAC/OUI Database**

This option requires internet access so if for some reason your server cannot access internet you could manually download the file from OUI Database copy it to your Cacti's box and import it by hand as follow.

Go to **<cacti_path>/plugins/mactrack**

Assuming that you have copied the file to /tmp `[root@mycactiserver mactrack]#php mactrack_import_ouidb.php -f=/tmp/out.txt Getting OUI Database from the IEEE OUI Database Download from IEEE Complete ..... There were '23126' Entries Added/Updated in the database. There were '0' Records Removed from the database.`

##Troubleshooting Running poller in debug mode Go to <cacti_path>/plugins/mactrack php -q poller_mactrack.php -d -sid=1 Run php poller_mactrack.php -h for more options

Run scanner in debug mode for an specific device Go to <cacti_path>/plugins/mactrack php -q mactrack_scanner.php -d -id=23 Run php -q mactrack_scanner.php -h for more options

The scanning function you are looking for does not show up Go to Tracking Tools Menu and click on "Refresh Scanning Functions" Deletes old and potentially stale Device Tracking scanning functions from the drop-down you see when you edit a device type.

Copyright (c) 2004-2026 The Cacti Group

Plugin Development

Overview

Cacti Plugins allow users of Cacti to extend its functionality into areas that the core Cacti developers have not. They were written based off of the work of Jimmy Conner from the SquirrelMail project years ago. They allow developers to augment the Cacti Website, and its polling processes to extend Cacti's reach.

There are a number of popular plugins including:

- **Thold** - A graph based service checking framework that leverages the data on Cacti graphs for performing service checks
- **Syslog** - A plugin that allows you to view log data from hundreds or thousands of hosts and generate faults based upon pattern matching of that log data.
- **Monitor** - A plugin that provides a Device Dashboard highlighting Cacti Devices that are experiencing faults.
- **Intropage** - A plugin that provides a replacement to the Cacti console showing key performance metrics across all of Cacti from a single page.
- **Weathermap** - A plugin that allows you to create architectural diagrams of your network and to animate them with status data.

In theory, these plugins can be integrated into Cacti very simply, and removed without breaking Cacti. However, it must be noted to always review the plugins documentation before you start to use it.

Legacy Plugins Notice

Plugins written for Cacti 0.8.x require modifications in order to be compatible with Cacti 1.x. There are several changes that all plugin developers need to be aware of. Any of the Cacti Group maintained plugin can be used as reference plugins for driving your plugin migration to the 1.x framework and are available on Github.

Plugin Migration

In order to migrate your legacy Cacti plugins, there are several steps that need to be taken. This guide will provide a brief synopsis of changes that are required to make your plugins compatible with Cacti 1.0.

Migration Steps

Each of the following steps should be followed to migrate your plugins. Some of these changes are optional, and some are mandatory. We will start with the list of steps that must be required. Then, in future sections, we will provide details on performing each step.

Required Steps

- Creation of Information (INFO) file (mandatory)
- Applying correct classes to Anchor tags to trigger callbacks (mandatory)
- Remove instances of including top_header.php, top_graph_header.php, top_general_header.php and replacing with function calls top_header(), bottom_footer(), top_graph_header(), and top_general_header(). (mandatory)
- Adding '&header=false' to most header() function calls to drive Ajax rendering. (mandatory)

- Movement of form elements to be W3C compliant as HTML5 requires it (mandatory)
- Moving the $nav variable within the page to ensure tables are formatted correctly. This includes the $nav before the table and the $nav afterwards. (mandatory)
- Remove all instances of $_GET, $_REQUEST, and $_POST and replacing the wrappers get_request_var(), get_filter_request_var(), set_request_var(), isset_request_var(), isempty_request_var() (mandatory)
- Utilize the 'validate_store_request_vars()' function to greatly simplify request validation on your pages. (mandatory)
- Migration of the Plugin to the PIA 2.x+ install and setup process. Cacti PIA 1.x plugins are not supported in Cacti 1.0 and beyond. (mandatory)
- Removal of hooks that inject jQuery, jQueryUI into Cacti's header (mandatory)

Optional Steps

- Remove inline styles, and use Cacti's classes instead (highly recommended)
- Migration of filters to Ajax callbacks (highly recommended)
- Removing references to the $colors array (optional)
- Using new functions for alternate colored rows, sorted, checkbox and other headers (optional)
- Using new options for headers to support tool tips, alignment etc (optional)
- Migration to new pagination functions, which simplify page navigation

INFO File structure

The INFO file follows the INI file specification. An example INFO file includes the following sections:

- **name** - The name of the plugin directory
- **version** - The version number of the plugin
- **longname** - A description for the plugin
- **author** - The plugins author
- **email** - An email contact in order to reach the author
- **homepage** - A link to the plugin authors homepage
- **compat** - Prior to Cacti 1.3, the minimum version of Cacti to support the plugin. After Cacti 1.3, it will represent the range of Cacti versions supported.
- **requires** - The list of plugins and versions that the current plugin needs in order to operate. This is a comma delimited list of Plugins with version attributes similar to the **compat** section.
- **capabilities** - a comma delimited string of capabilities defining how the plugin works with remote data collectors
- **nosync** - a comma delimited string of directories or file extensions to skip during the remote data collector synchronization. File extensions must be formatted as `*.extension` and only are relevant in the base directory of the plugin. The following extensions are automatically excluded regardless of directory: 'tar', 'gz', 'zip', 'tgz', 'ttf', 'z', 'exe', 'pack', 'swp', 'swo'.

Example INFO Files

From the Monitor plugin, we can see the following format:

```
[info]
name = monitor
version = 2.8
longname = Device Monitoring
author = The Cacti Group
email =
homepage = https://www.cacti.net
compat = 1.2.15
requires = thold:1.2.1
capabilities = online_view:1, online_mgmt:1, offline_view:0, offline_mgmt:0, remote_collect:0
```

From the Syslog plugin, we can see a slightly different format.

```
[info]
name = syslog
version = 4.2
longname = Syslog Monitoring
author = The Cacti Group
email =
homepage = https://www.cacti.net
compat = 1.2.23
nosync = config.php
capabilities = online_view:1, online_mgmt:1, offline_view:1, offline_mgmt:1, remote_collect:0, remote_poller:1
```

Here we can see that the config.php file in the plugin directory will not be copied over to remove pollers, and we can see that syslog allow for remote data collection. using the **remote_poller:1** capabilities attribute.

Changes in Cacti 1.3+

Starting with Cacti 1.3 and beyond, the **compat** and **requires** lines can include ranges of support. For example, the following will be supported:

```
compat = >=1.2.15 <1.3.0
requires = thold:>1.2.1 <=2.0, monitor:>2.0
```

Before Cacti 1.3, the **compat** and **requires** attributes assumed always >= there were never provisions for < or <= to for any plugin. Due to changes in the Plugin Architecture, this will change starting with Cacti 1.3.

As mentioned, there are some very good examples of how to create or migrate plugins in the various Cacti plugins included on The Cacti Groups GitHub site. Also, for a very simple Cacti page, you can look at vdef.php as an example. You can use this as a template to manage a table using the Cacti framework. For a simple plugin example, please review the gexport plugin.

Charting Functionality

Several JavaScript based HTML5 Charting packages have been included in Cacti in an effort to assist plugin developers who wish to use graphing API's in their plugins other than RRDtool.

- Billboard.js
- D3
- Chart.js
- DyGraphs
- jQuery Sparklines (note: this project appears unmaintained; evaluate before use)

Logging

For developers using the Cacti framework, it is important to note that additional controls on logging have been added. Debug logging can now be controlled at not only a global level, but now per plugin, per device and even per file.

Copyright (c) 2004-2026 The Cacti Group

Plugin Guidelines

Overview

This section will help the Cacti plugin developer structure their plugins. However, the best way to create a plugin is to borrow someone elses. There are Cacti plugin examples on Cacti's GitHub page and you can find them all over GitHub just by searching.

Plugin Creation

Use the following guidelines for creating your first plugin.

Plugin Namespaces

All plugins must use namespace isolation either through Classes, or using the plugin name in their functions. This prevents name space collisions.

An example plugin is provided here Example Plugin.

Publishing Your Plugin

GitHub is the best place to publish your plugins. Learn to use it.

Copyright (c) 2004-2026 The Cacti Group

Creating a plugin

Your plugin will require a few things to be properly detected by Cacti.

Directory Structure

Your plugin will reside in its own directory in Cacti's plugin directory. You will need to create a new directory, with a unique name. It is recommended that you use all lowercase alpha letters, and try to keep it short (you will be typing it a lot). All plugin names must contain only upper or lowercase letters a-z and digits 0-9.

So to begin, create your directory in the plugins directory like this.

```
/plugins/PLUGINNAME/
```

Setup File

Cacti detects plugins by looking for a 'setup.php' file located inside your plugin's directory. It must be all lowercase. Inside this file should only be functions, no code that runs automatically.

INFO File

Every Cacti plugin must have an INFO file. Any legacy plugins that do not have an INFO file will not work with Cacti 1.x. This was done intentionally so that plugin users could know ahead of time that their plugin was not designed for Cacti 1.x. The info file includes several line including:

setting	description
name	The short name for the plugin. Must be alpha with no spaces and match the directory name.
version	The version of the plugin. Use two to three decimal places.
longname	A user readable short description for the plugin.
author	The author of the plugin.
email	The authors email address for obtaining support.
homepage	If the author maintains a homepage, the address for that page.
compat	The minimum version of Cacti that is required to use this plugin.
depends	A space delimited list of plugins:version that are required to use this plugin.
capabilities	A comma delimited list of capabilities for define what capabilities the plugin has primarily remotely.
nosync	A comma delimited list of relative paths that should not be synchronized with the remote data collectors, for example transient folders that may contain quite a bit of data that changes often.

Mandatory functions

Inside the setup file, there are several functions that are mandatory in order for your plugin to properly be detected, and to properly function.

plugin_PLUGINNAME_install

This first function is the installation function that is ran when a request is received to install your plugin. There are no arguments passed to this function.

You will need to replace PLUGINNAME with the exact folder name (including case sensitivity). Below is an example of this function. We will discuss the register_hook function later in this document.

```
function plugin_PLUGINNAME_install () {
     api_plugin_register_hook('PLUGINNAME', 'top_header_tabs', 'PLUGINNAME_show_tab', 'setup.php');
}
```

plugin_PLUGINNAME_uninstall

The next function is the uninstall function. This is a function that is called when someone requests to uninstall your plugin. You can use this to clean up any extra data that is left by your plugin. If you used the API function to install any tables, they will be cleaned up automatically.

plugin_PLUGINNAME_version

The next function is used for checking the version of your plugin, plus providing a little bit more information.

Below is an example of this function. The few extra things to return are the longname (aka Pretty Name that will be displayed), the author, website, etc... the `url` is a special url used to check for updates of your plugin to be compatible with the 'update' plugin.

```
function plugin_PLUGINNAME_version () {
     global $config;
     $info = parse_ini_file($config['base_path'] . '/plugins/PLUGINNAME/INFO', true);

     return $info['info'];
}
```

plugin_PLUGINNAME_check_config

The last mandatory function is used for determining whether your plugin is ready to be enabled after being installed.

When a plugin is installed, it starts in a disabled state. Only a few of the necessary hooks for the plugin are enabled. These are the config_settings and config_arrays hook. Anything that uses those hooks will still be available while the plugin is disabled. This allows a user to modify any necessary settings for the plugin before the plugin is actually enabled and goes live. You can then use this check_config function to determine whether everything is correctly setup, before you allow the plugin to be enabled. From this function, you will simply need to return TRUE if everything checks out, or return FALSE if something else needs to be setup.

```
function plugin_PLUGINNAME_check_config () {
     if (read_config_option('PLUGINNAME_SETTING') != '') {
          return true;
     }

     return false;
}
```

Optional functions

Some plugins delete all your data and settings when you uninstall them. However, some plugins preserve the data and can be used again in the future. The following 2 functions allow you to delete all data and settings without uninstalling the plugin or manually deleting them. If the plugin does not contain the function, the delete icon will not appear in the settings (console -> configuration -> plugins). These functions are used in version 1.3 and higher.

```
function plugin_PLUGINNAME_has_data() {
    return true;
}

function plugin_PLUGINNAME_remove_data() {
    return false;
}
```

Registering Hooks

The way that your plugin is able to interact within Cacti is via the many hooks that are placed into the interface. As cacti processes the various visual pages and backend functions, it will call specific hook functions which passes control over to any plugins that might want to interact with that aspect of cacti. In order to utilize these hooks, your plugin must register for each hook that you require inside our installation function. This is done via a simple API function.

```
api_plugin_register_hook('PLUGINNAME', 'HOOKNAME', 'CALLBACKFUNCTION', 'FILENAME');
```

Name	Description
PLUGINNAME	The name of your plugin
HOOKNAME	The name of the hook you want to register
CALLBACKFUNCTION	The name of your function to call when the hook is triggered.
FILENAME	The name of the file which contains the above function (path relative to your plugin directory)

Taking our above example, if you wanted to display a new tab at the top of Cacti's Console, your setup.php file may look like this.

```
function plugin_PLUGINNAME_install () {
     api_plugin_register_hook('PLUGINNAME', 'top_header_tabs', 'PLUGINNAME_show_tab', 'setup.php');
}

function PLUGINNAME_show_tab () {
     global $config;
     print '<a href="' . $config['url_path'] . 'plugins/PLUGINNAME/PLUGINNAME.php"><img src="' .
$config['url_path'] . 'plugins/PLUGINNAME/images/tab.gif" align="absmiddle" border="0"></a>';
}
```

Here you can find a full listing of the available hooks

Hook Api Reference

Registering Realms

Cacti allows you to restrict access to various portions of Cacti by utilizing Realm Permissions. Creating a new realm will add a checkbox to the User Permissions form for the page you desire to restrict access to. You can create 1 realm that permits access to multiple pages, or use multiple realms for different functional duties (ex: separating out the viewing and the configuration sections of your plugin).

We need to call this hook in the same function as we registered for hooks. Either of the api calls can come first (there is no order necessary)

```
api_plugin_register_realm('PLUGINNAME', 'FILENAMETORESTRICT', 'DISPLAYTEXT', 1);
```

Name	Description
PLUGINNAME	Your plugins name
FILENAMETORESTRICT	the filename (relative to your plugin directory) to restrict access to, you can separate multiple with a comma
DISPLAYTEXT	This is the text that will display on the User Permissions form for this Realm
1 or 0 (true or false)	This tells it whether to automatically grant permission to the 'admin' user account (true)

So to follow our example, your install function would look like this

```
function plugin_PLUGINNAME_install () {
     api_plugin_register_hook('PLUGINNAME', 'top_header_tabs', 'PLUGINNAME_show_tab', 'setup.php');
     api_plugin_register_realm('PLUGINNAME', 'PLUGINNAME.php,', 'View PLUGINNAME', 1);
}
```

Copyright (c) 2004-2026 The Cacti Group

Reference Plugins

This guide outlines the architecture and standard conventions for reference plugins in Cacti.

The Reference Plugin(s) are generic plugins that implement key pieces of the Plugin Architecture.

They are broken down into several separate plugins, each covering different portions of the Hook API Reference

Standard Files

The following files should be included in every plugin

- CHANGELOG
- INFO
- LICENSE
- index.php
- README.md
- setup.php

In general, the index.php file example below should be included in every plugin directory as it prevents remote file browsing of your Cacti site.

```
<?php

/*
 +-------------------------------------------------------------------------+
 | Copyright (C) 2004-2026 The Cacti Group                                 |
 |                                                                         |
 | This program is free software; you can redistribute it and/or           |
 | modify it under the terms of the GNU General Public License             |
 | as published by the Free Software Foundation; either version 2          |
 | of the License, or (at your option) any later version.                  |
 |                                                                         |
 | This program is distributed in the hope that it will be useful,         |
 | but WITHOUT ANY WARRANTY; without even the implied warranty of          |
 | MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the           |
 | GNU General Public License for more details.                            |
 +-------------------------------------------------------------------------+
 | Cacti: The Complete RRDtool-based Graphing Solution                     |
 +-------------------------------------------------------------------------+
 | This code is designed, written, and maintained by the Cacti Group. See  |
 | about.php and/or the AUTHORS file for specific developer information.   |
 +-------------------------------------------------------------------------+
 | https://www.cacti.net/                                                   |
 +-------------------------------------------------------------------------+
*/

header("Location:../index.php");

?>
```

File Structure

All plugin files should reside under their own respective plugin directory. The example below is for a plugin called 'myplugin'.

```
myplugin/
    images/
        myimage.png
        index.php
    lib/
        index.php
        functions.php
```

```
    include/
        index.php
        settings.php
    CHANGELOG
    INFO
    LICENSE
    README.md
    index.php
    setup.php
```

Contents of Files

The four core sub-directories included in your plugin should be docs, images, lib, and include. As previously mentioned, each of these directories should include an index.php that points to it's parent directory index.php. This way we prevent people from randomly browsing the contents of our web site.

The two main files required for every plugin are the INFO and the setup.php file. Without these two files, the plugin directory will not be recognized as a valid plugin and will be skipped.

The INFO file is formatted as Microsoft standard INI file. It contains information about the plugin including its version, author, dependencies, functions, and minimum version of Cacti and other plugins required. Below, you can see a reference INFO file for the myplugin plugin.

```
[info]
name = mplugin
version = 1.0
longname = My First Plugin
author = Depeche Mode
email = somewhere@outthere.com
homepage = http://www.outthere.com
compat = 1.2
requires = thold monitor
capabilities = online_view:1, online_mgmt:1, offline_view:0, offline_mgmt:0, remote_collect:0
```

The setup file includes key plugin functions that allow the plugin to be installed, uninstalled, upgraded, etc. Those core functions include:

```
<?php
function plugin_myplugin_install() {
}

function plugin_myplugin_uninstall() {
}

function plugin_myplugin_check_config() {
}

function plugin_myplugin_upgrade() {
}

function plugin_myplugin_version() {
}

?>
```

Without these core functions, in your setup.php, your plugin will not fully operate. We will document those functions and what they should include as we continue to document the plugin design principals.

Any core plugin functions should always be placed in the lib/functions.php file, and settings should go into the include/settings.php file. The plugin architecture allows hooks to be defined that reach into the include/settings.php file. Having those hooks point to the include version of the file will keep Cacti page loads fast. Including too much in the setup.php slows Cacti and should be avoided.

Copyright (c) 2004-2026 The Cacti Group

Plugin Hook API Reference

The following table shows all known Cacti plugin hooks and a brief description of their function.

hook	hook	hook
add_graph_template_to_host	api_device_new	api_device_save
auth_alternate_realms	auth_profile_reset	auth_profile_reset_value
auth_profile_run_action	auth_profile_save	auth_profile_tabs
auth_profile_update_data	boost_poller_bottom	bottom_footer
cacti_stats_update	change_password_title	clog_regex_array
config_arrays	config_form	config_insert
config_settings	console_after	console_before
copy_user	create_complete_graph_from_template	custom_denied
custom_login	custom_logout_message	custom_password
custom_version_info	data_input_sql_where	data_source_action_array
data_source_action_bottom	data_source_action_execute	data_source_action_prepare
data_source_edit_bottom	data_source_edit_top	data_source_remove
data_source_to_poller_items	data_sources_table	device_action_array
device_action_bottom	device_action_execute	device_action_prepare
device_display_text	device_edit_pre_bottom	device_edit_top_links
device_filter_end	device_filter_start	device_filters
device_change_javascript	device_remove	device_sql_where
device_table_bottom	device_table_replace	device_template_change
device_template_edit	device_template_top	draw_navigation_text
export_action	fgc_contextoption	get_friendly_name
global_settings_update	graph	graph_buttons
graph_edit_after	graph_image	graph_items_remove
graph_tree_page_buttons	graphs_action_array	graphs_action_bottom
graphs_action_execute	graphs_action_prepare	graphs_item_array
graphs_new_top_links	graphs_remove	graphs_sql_where
hmib_get_cpu	hmib_get_cpu_indexes	hmib_get_disk
host_edit_bottom	host_edit_top	host_save
import_action	is_console_page	login_after
login_before	login_options_navigate	login_process
login_realms	login_title	logout_post_session_destroy
logout_pre_session_destroy	nav_login_after	nav_login_before
page_bottom	page_buttons	page_head
page_title	poller_bottom	poller_command_args
poller_exiting	poller_finishing	poller_output
poller_remote_maint	poller_top	remote_agent

hook	hook	hook
replicate_in	replicate_out	report_filters
resolve_dependencies	rrd_graph_graph_options	run_data_query
snmpagent_cache_install	substitute_host_data	top_graph_header
top_graph_header_tabs	top_graph_jquery_function	top_graph_refresh
top_header	tree_after	tree_view_page_end
ugroup_tabs	update_data_source_title_cache	user_admin_action
user_admin_edit	user_admin_run_action	user_admin_setup_sql_save
user_admin_tab	user_admin_user_save	user_group_admin_action
user_group_admin_edit	user_group_admin_save	user_group_admin_setup_sql_save
user_group_admin_run_action	user_group_admin_tab	user_group_remove
user_remove	utilities_action	utilities_array
utilities_list	valid_host_fields	

add_graph_template_to_host

Allows you to perform additional operations when adding a graph template to host. The parameters passed are `host_id` and `graph_template_id`.

file	introduced	retired
cli/add_graph_template.php	1.0.0	
cli/host_update_template.php	1.0.0	
host.php	1.0.0	
lib/api_device.php	1.0.0	

api_device_new

This hooks allows you to perform additional operation at the end of api_device_save function. Unlike the api_device_save hook, the device ID is already defined here.

file	introduced	retired
lib/api_device.php	1.0.0	

api_device_save

This hook allows you to perform additional operation at the end of api_device_save function. Unlike the api_device_new hook, the device ID is might not be defined here.

This hook allows you to alter the contents to the device table with custom fields. When used properly, it allows you to extend the host table with your plugins custom fields.

file	introduced	retired
lib/api_device.php	1.0.0	

auth_alternate_realms

This hook allows you to handle user login check alternate plugin defined authentication realms.

file	introduced	retired
include/auth.php	1.0.0	

auth_profile_reset

Allows you to reset plugins all user settings

file	introduced	retired
auth_profile.php	1.2.0-beta3	

auth_profile_reset_value

Allows you to reset plugins single user settings

file	introduced	retired
auth_profile.php	1.2.0-beta3	

auth_profile_run_action

Allows you to draw plugins user settings form

file	introduced	retired
auth_profile.php	1.2.0-beta3	

auth_profile_save

Allows you to save plugins all user settings

file	introduced	retired
auth_profile.php	1.2.0-beta3	

auth_profile_tabs

Allows you to draw plugins user setting tabs

file	introduced	retired
auth_profile.php	1.2.0-beta3	

auth_profile_update_data

Allows you to update plugins single user setting

file	introduced	retired
auth_profile.php	1.2.4	

boost_poller_bottom

file	introduced	retired
poller_boost.php	1.0.0	

bottom_footer

Allows you to override the rendering of the bottom_footer html snippet.

file	introduced	retired
include/bottom_footer.php	0.8.7i	1.1.38

cacti_stats_update

file	introduced	retired
poller.php	1.0.0	

change_password_title

file	introduced	retired
auth_changepassword.php	1.2.0-beta1	

clog_regex_array

Allows you to enhance the Logs display with additional information or links.

file	introduced	retired
lib/clog_webapi.php	1.1.34	

config_arrays

Allows your plug-in to define custom array elements to be used throughout cacti. Things like dropdown arrays and the like.

file	introduced	retired
include/global_arrays.php	1.0.0	

config_form

Allows you to define form object to represent data to be stored in your tables. In the case of MacTrack, there is a table called `mac_track_sites` and in order to drive the UI and allow for the user to provide input and save a site, you must provide the Cacti form engine a form array to allow for the form to be presented and contents to be saved.

file	introduced	retired
include/global_form.php	1.0.0	

config_insert

This is a general purpose hook used for connecting to other database, or sourcing common includes that would otherwise not be sourced.

file	introduced	retired
include/global.php	1.0.0	

config_settings

Allows you to extend the Cacti settings page to allow for custom tabs and global settings within them.

file	introduced	retired
include/global_settings.php	1.0.0	

console_after

Allows you to add text after the default message on the console page

file	introduced	retired
index.php	1.0.0	

console_before

Allows you to add text before the default message on the console page. For instance, the Update plug-in will display a message here if there are new versions of plugins.

file	introduced	retired
index.php	1.0.0	

copy_user

Allows a plugin to take part in the Copy User actions. This is important a plugin maintains user settings.

file	introduced	retired
lib/auth.php	1.0.0	

create_complete_graph_from_template

Allows you to perform additional operations when adding new graph to device from graph template.

file	introduced	retired
lib/template.php	1.0.0	

custom_denied

Allows you to override the rendering of the permission denied interface.

file	introduced	retired
permission_denied.php	1.2.2	
auth_login.php	1.2.12	

custom_login

Allows you to override the rendering of the auth_login.php page.

file	introduced	retired
auth_login.php	1.0.0	

custom_logout_message

Allows you to override the rendering of the logout.php page.

file	introduced	retired
logout.php	1.0.0	

custom_password

Allows you to override the rendering of the auth_changepassword.php page.

file	introduced	retired
auth_changepassword.php	1.0.0	

custom_version_info

Allows your plugin to display additional information on the cacti tech support page.

file	introduced	retired
utilities.php	1.0.0	

data_input_sql_where

file	introduced	retired
data_input.php	1.0.0	

data_source_action_array

Allows you to define custom dropdown actions from within the data sources page.

file	introduced	retired
data_sources.php	1.0.0	

data_source_action_bottom

file	introduced	retired
data_sources.php	1.0.0	

data_source_action_execute

Allows you to define custom actions to be taken upon you dropdown selection.

file	introduced	retired
data_sources.php	1.0.0	

data_source_action_prepare

This is the final component to creating a custom action within the data sources. In Cacti's form engine, you must provide the dropdown action and then two follow-up actions to complete the cycle. This is were you actually save the data.

file	introduced	retired
data_sources.php	1.0.0	

data_source_edit_bottom

Allows you to make minor change to the Data Sources page, like add a tabbed interface.

file	introduced	retired
data_sources.php	1.0.0	

data_source_edit_top

Allows you to make minor changes to the Data Sources page, like add a tabbed interface.

file	introduced	retired
data_sources.php	1.0.0	

data_source_remove

This hook is called when the data sources used by a graph are removed. The parameters are the graphs data sources.

file	introduced	retired
data_sources.php	1.0.0	1.1.38
graphs.php	1.0.0	1.1.38
host.php	1.0.0	1.1.38
lib/api_data_source.php	1.2.0-beta1	

data_source_to_poller_items

file	introduced	retired
lib/utility.php	1.0.0	

data_sources_table

Allows you to change the data displayed on the Data Sources table. THold uses this to create a link to that datasources threshold

file	introduced	retired
data_sources.php	1.0.0	

device_action_array

Allows you to define custom dropdown actions from within the device/host page.

file	introduced	retired
host.php	1.0.0	

device_action_bottom

file	introduced	retired
host.php	1.0.0	

device_action_execute

Allows you to define custom actions to be taken upon you dropdown selection.

file	introduced	retired
host.php	1.0.0	

device_action_prepare

This is the final component to creating a custom action within the devices. In Cacti's form engine, you must provide the dropdown action and then two follow-up actions to complete the cycle. This is were you actually save the data.

file	introduced	retired
host.php	1.0.0	

device_display_text

This hook allows you to add custom columns header to the host list page

file	introduced	retired
host.php	1.3.0 - Dev	

device_edit_pre_bottom

file	introduced	retired
host.php	1.0.0	

device_edit_top_links

This hook allows you to add custom actions to the host edit page, below the "Create graphs for this Host".

file	introduced	retired
host.php	1.0.0	

device_filter_end

This hook allows you to add custom filter fields before Cacti built-in filter to the host list page

file	introduced	retired
host.php	1.3.0 - Dev	

device_filter_start

This hook allows you to add custom filter fields after Cacti built-in filter to the host list page

file	introduced	retired
host.php	1.3.0 - Dev	

device_filters

file	introduced	retired
host.php	1.0.0	

device_change_javascript

This hook allows you to add custom javascript to Change device settings page

file	introduced	retired
host.php	1.2.31	

device_remove

This hook allows you to perform additional actions on devices that have been selected for deletion. The parameter is an array of deleted device IDs. Note that at this point, the devices are already deleted from the host table.

file	introduced	retired
host.php	1.0.0	1.1.38
lib/api_device.php	1.2.0-beta1	

device_sql_where

file	introduced	retired
host.php	1.0.0	

device_table_bottom

file	introduced	retired
host.php	1.0.0	

device_table_replace

Allows you to override the rendering of the host list page

file	introduced	retired
host.php	1.3.0 - Dev	

device_template_change

Allows you to add template items such a threshold upon a device template change

file	introduced	retired
lib/ap_device.php	1.2.25	

device_template_edit

file	introduced	retired
host_templates.php	1.0.0	

device_template_top

file	introduced	retired
host_templates.php	1.0.0	

draw_navigation_text

Just below the Cacti "Console", and "Graphs" tabs, cacti provides a Navigation bar that allows you to move backward in a selection tree. Although it is not foolproof, this hook allows you to populate and navigate that tree.

file	introduced	retired
lib/functions.php	1.0.0	

export_action

file	introduced	retired
lib/export.php	1.2.0-beta1	

fgc_contextoption

file	introduced	retired
lib/functions.php	1.1.28	

get_friendly_name

file	introduced	retired
lib/import.php	1.2.0-beta1	

global_settings_update

file	introduced	retired
settings.php	1.0.0	

graph

This hook is called at the top of graph.php page

file	introduced	retired
graph.php	1.0.0	

graph_buttons

You know about the zoom button right? Well this allows you to add any additional custom button to the right of a graph.

file	introduced	retired
graph.php	1.0.0	

graph_edit_after

file	introduced	retired
graphs.php	1.2.0-beta1	

graph_image

This hooks allows displaying of alternative graphics in place of a graph image. This hook can no output text, only modify the image, as it is only called when a graph is required to be outputted.

file	introduced	retired
graph_image.php	1.0.0	
graph_json.php	1.0.0	

graph_items_remove

This hook is called when the data source is deleted, and the option to remove all graph items tied to the data source is selected.

file	introduced	retired
data_sources.php	1.0.0	

graph_tree_page_buttons

Allows you to add custom HTML to the graph tree view page, under the timespan presets and the graph search box.

file	introduced	retired
lib/html_tree.php	0.8.7i	

graphs_action_array

Similar to the host actions and host, prepare and execute hooks. It allows you to have custom dropdown actions within the graph management interface.

file	introduced	retired
graphs.php	1.0.0	

graphs_action_bottom

file	introduced	retired
graphs.php	1.0.0	

graphs_action_execute

file	introduced	retired
graphs.php	1.0.0	

graphs_action_prepare

file	introduced	retired
graphs.php	1.0.0	

graphs_item_array

file	introduced	retired
graphs.php	1.2.0-beta1	

graphs_new_top_links

Add a link at the top of the "Create New Graphs" page.

file	introduced	retired
graphs_new.php	1.0.0	

graphs_remove

This hook is called after one or more graphs are removed. The parameters are the removed graph IDs.

file	introduced	retired
data_sources.php	1.0.0	1.1.38
graphs.php	1.0.0	1.1.38

file	introduced	retired
host.php	1.0.0	1.1.38
lib/api_graph.php	1.2.0-beta1	

graphs_sql_where

This hooks allows you to add additional where clauses to the select statement used to display graphs on the graphs.php page.

file	introduced	retired
graphs.php	1.0.0	

hmib_get_cpu

file	introduced	retired
scripts/ss_host_cpu.php	1.0.0	

hmib_get_cpu_indexes

file	introduced	retired
scripts/ss_host_cpu.php	1.0.0	

hmib_get_disk

file	introduced	retired
scripts/ss_host_disk.php	1.0.0	

host_edit_bottom

Allows you to make minor changes to the Devices page, like add a tabbed interface.

file	introduced	retired
host.php	1.0.0	

host_edit_top

Allows you to make minor changes to the Devices page, like add a tabbed interface.

file	introduced	retired
host.php	1.0.0	

host_save

file	introduced	retired
host.php	1.0.0	

import_action

file	introduced	retired
lib/import.php	1.2.0-beta1	

is_console_page

file	introduced	retired
lib/html.php	1.0.0	

login_after

This allows you to display text or html after the login form.

file	introduced	retired
auth_login.php	1.0.0	

login_before

This allows you to display text or html before the login form. Combined with the above hook, you could draw a box, etc around the form, change its background color, etc to slightly customize the login page.

file	introduced	retired
auth_login.php	1.0.0	

login_options_navigate

Used to support alternate login realm types.

file	introduced	retired
auth_changepassword.php	1.0.0	
auth_login.php	1.0.0	

login_process

Used to authenticate the user logging in to a custom plugin defined realm.

file	introduced	retired
auth_login.php	1.0.0	

login_realms

Used to defined additional custom plugin defined authentication realms. The parameter passed is an array of existing authentication realms.

file	introduced	retired
auth_login.php	1.0.0	

login_title

Allows the plugin to set/modify that page title.

file	introduced	retired
auth_login.php	1.0.0	

logout_post_session_destroy

This pair of hooks (`logout_pre_session_destroy` and `logout_post_session_destroy`) allows you to preserve some session variables that would otherwise get unset, when `session_destroy()` is called.

file	introduced	retired
logout.php	1.0.0	

logout_pre_session_destroy

This pair of hooks (`logout_pre_session_destroy` and `logout_post_session_destroy`) allows you to preserve some session variables that would otherwise get unset, when `session_destroy()` is called.

file	introduced	retired
logout.php	1.0.0	

nav_login_after

This pair of hooks (`nav_login_before` and `nav_login_after`) allows you to add custom HTML to the navigation bar, before and after the "Logged in as " text.

file	introduced	retired
lib/functions.php	1.0.0	

nav_login_before

This pair of hooks (`nav_login_before` and `nav_login_after`) allows you to add custom HTML to the navigation bar, before and after the "Logged in as " text.

file	introduced	retired
lib/functions.php	1.0.0	

page_bottom

This hook allows you to add custom HTML to the bottom of the page, after the main layout table.

file	introduced	retired
include/bottom_footer.php	1.0.0	

page_buttons

Allows you to add custom buttons and HTML to the top of the graph.php page, above the "Viewing graph" row.

file	introduced	retired
graph.php	1.0.0	

page_head

This hook allows you to add custom HTML tags to the page `<head>` tag.

file	introduced	retired
graph_realtime.php	1.0.0	1.1.33
include/top_general_header.php	1.0.0	1.1.33
include/top_graph_header.php	1.0.0	1.1.33
include/top_header.php	1.0.0	1.1.33
lib/html.php	1.1.34	

page_title

Allows the plugin to set/modify the page title

file	introduced	retired
include/top_general_header.php	1.0.0	
include/top_graph_header.php	1.0.0	
include/top_header.php	1.0.0	

poller_bottom

This hook allows you to launch other polling activities once Cacti has complete it's polling activities. Things like MacTrack, Boost, THold, and others use this hook to launch their respective poller.

file	introduced	retired
poller.php	1.0.0	

poller_command_args

Using this hook, your plugin can pass additional command line arguments to the poller.

file	introduced	retired
poller.php	0.8.7i	1.2.8

poller_exiting

file	introduced	retired
poller.php	1.0.0	

poller_finishing

file	introduced	retired
poller.php	1.0.0	

poller_output

This hook was designed specifically for the boost plug-in. However, it has found new life in many plugins making them much faster now. What this hook allows you to do is intercept the contents of the `poller_output` table before the data is deleted. This allows for things like threshold checking and other interesting actions such as checking for up/down events and the like. Previously, plugins were reliant on using things like `rrdtool fetch` to perform these actions. Now they can leverage the pollers output directly.

file	introduced	retired
lib/poller.php	1.0.0	

poller_remote_maint

This hook will fire for plugins that need to perform remote maintenance. It will only fire on the Remote Data Collectors.

file	introduced	retired
poller_maintenance.php	1.2.23	

poller_top

This hook allows you to launch processes concurrently with the Cacti poller for things like THolds fast poller for example.

file	introduced	retired
poller.php	1.0.0	

remote_agent

file	introduced	retired
remote_agent.php	1.0.0	

replicate_in

file	introduced	retired
lib/poller.php	1.0.0	

replicate_out

file	introduced	retired
lib/api_device.php		
lib/poller.php	1.0.0	

report_filters

Allows plugins to add custom filter criteria to report generation interfaces.

File	Release
reports.php	1.0.0

```
function myplugin_report_filters($filters) {
    // Modify or add custom report filters
    return $filters;
}
```

resolve_dependencies

file	introduced	retired
lib/export.php	1.2.0-beta1	

rrd_graph_graph_options

This hook allows you to specify additional options or change parameters regarding graph image rendering. Some of the parameters will be passed to RRDtool.

file	introduced	retired
lib/rrd.php	1.0.0	

run_data_query

This hook allows your plugin to perform additional operations after a data query update was triggered.

file	introduced	retired
lib/data_query.php	1.0.0	

snmpagent_cache_install

file	introduced	retired
lib/snmpagent.php	1.0.0	

substitute_host_data

This allows you to add to the substitution variables that cacti can parse.

file	introduced	retired
lib/variables.php	1.0.0	1.1.5

top_graph_header

Allows you to override the rendering of the top_graph_header.php include.

file	introduced	retired
include/top_graph_header.php	0.8.7i	1.1.38

top_graph_header_tabs

This hook is used to add tab's to Cacti's user interface.

file	introduced	retired
lib/html.php	1.0.0	

top_graph_jquery_function

file	introduced	retired
lib/html_tree.php	1.0.0	

top_graph_refresh

This allows you to change the refresh on the graph (and any plug-in page that calls the graph header)

file	introduced	retired
include/global_session.php	1.0.0	
include/themes/sunrise/global_session.php	1.1.11	1.1.38

top_header

This hook enables you to output custom HTML on top of the graph view page, if navigation tabs are not rendered. Allows you to override the rendering of the top_header.php include.

file	introduced	retired
include/top_general_header.php	1.0.0	1.1.38
include/top_header.php	0.8.7i	1.1.38

tree_after

This hook allows you to add additional HTML at the bottom of the graph tree view page. The HTML is rendered only if a valid graph tree leaf is selected.

file	introduced	retired
lib/html_tree.php	1.0.0	

tree_view_page_end

This hook allows you to add additional HTML at the bottom of the graph tree view page, irrespective of the leaf selection.

file	introduced	retired
graph.php	1.0.0	
lib/html_tree.php	1.0.0	

ugroup_tabs

file	introduced	retired
user_group_admin.php	1.0.0	

update_data_source_title_cache

This hook is called after the data source title cache is updated.

file	introduced	retired
lib/variables.php	1.0.0	

user_admin_action

This hook is part of a series of hooks that allow you to add custom user settings. Consider a plug-in called "abc". In that plug-in, you want to save custom settings on a per user basis. Therefore, following our unpublished ? table naming standards, you would create a table called "abc_settings". When you tab over to the "abc" plug-in, you are provided an "abc settings" on the right similar to "Graphs". The next series of hooks allow you to present those settings to the user, enable or disable the users access to that specific "abc settings" tab and allow an administrator to define customer values for that plug-in on a per user basis.

file	introduced	retired
user_admin.php	1.0.0	

user_admin_edit

See the explanation on `user_admin_action`

file	introduced	retired
user_admin.php	1.0.0	

user_admin_run_action

See the explanation on `user_admin_action`

file	introduced	retired
user_admin.php	1.0.0	

user_admin_setup_sql_save

See the explanation on `user_admin_action`

file	introduced	retired
user_admin.php	1.0.0	

user_admin_tab

See the explanation on `user_admin_action`

file	introduced	retired
user_admin.php	0.8.7i	

file	introduced	retired
utilities.php	1.0.0	1.2.0-beta3

user_admin_user_save

See the explanation on `user_admin_action`

file	introduced	retired
user_admin.php	1.0.0	

user_group_admin_action

Similar to `user_admin_action`

file	introduced	retired
user_group_admin.php	1.0.0	

user_group_admin_edit

See the explanation on `user_admin_action`

file	introduced	retired
user_group_admin.php	1.0.0	

user_group_admin_save

See the explanation on `user_admin_action`

file	introduced	retired
user_group_admin.php	1.0.0	

user_group_admin_run_action

Similar to `user_admin_run_action`

file	introduced	retired
user_group_admin.php	1.0.0	

user_group_admin_tab

Similar to `user_admin_tab`

file	introduced	retired
user_group_admin.php	1.0.0	

user_group_remove

Similar to `user_remove` for user groups.

file	introduced	retired
user_group_admin.php	1.0.0	

user_group_admin_setup_sql_save

file	introduced	retired
user_group_admin.php	1.0.0	

user_remove

Allows the plugin to take part in the user removal process. Important when the plugin maintains user settings.

file	introduced	retired
lib/auth.php	1.0.0	
user_admin.php	1.0.0	

utilities_action

This hook sets up the function to be executed from within Cacti's utilities section.

file	introduced	retired
utilities.php	1.0.0	

utilities_array

file	introduced	retired
utilities.php	1.0.4	

utilities_list

This hook allows you to add a custom utilities pick under Cacti's "Utilities" page.

file	introduced	retired
utilities.php	1.0.0	

utilities_tab

This hook allows you to add a custom utilities tab under Cacti's "Utilities" page.

file	introduced	retired
utilities.php	1.2.0-beta3	

valid_host_fields

Allows your plugin to define additional variables that will be used in substitute_host_data.

Pretty sure that this hook allows you to define custom replacement values in graphs.

file	introduced	retired
include/global_form.php	1.0.0	

Copyright (c) 2004-2026 The Cacti Group

Contributing

Cacti accepts bug reports, code changes, documentation corrections, and translations. Choose the repository that owns the affected component:

- Cacti for the application.
- Documentation for this manual.
- Spine for the C poller.
- The relevant Cacti plugin repository for a plugin.

Report a problem

Search the project's open and closed issues before opening a report. Include versions, reproduction steps, expected and actual behavior, and relevant logs. Remove passwords, SNMP communities, tokens, and other sensitive information. See Known Issues for release notes and existing reports. For installation or usage help, use the Cacti Forums.

Report suspected vulnerabilities privately through the Cacti security policy.

Code contributions

Create a branch for one focused change and submit a pull request to the repository that owns it. Use `develop` unless maintainers request another branch. Check the issue and target branch before starting a maintenance fix; a change needed on one branch may already be fixed on another.

Read the contributor instructions in that branch. In the Cacti development repository, `composer.json` defines PHP requirements and checks, `.nvmrc` specifies the Node version, and `package-lock.json` locks frontend dependencies. For a development checkout, install dependencies from the repository root:

```
composer install
npm ci
```

Use the branch's formatter, static analysis, and relevant tests before submitting. The development branch provides `composer check`; consult its contributor guide for individual commands. Maintenance branches may have different tools. Describe what changed and how you verified it. Add regression coverage when it protects against a realistic failure, and link the associated issue.

Follow the code formatting, PHP, SQL, and security guides. Update relevant documentation and follow the target branch's changelog instructions. See Patch Creation for the submission workflow.

Documentation contributions

Submit manual changes to the documentation repository. Explain the correction and link an existing issue where applicable. Check instructions against the Cacti branch being documented, distinguish released features from development features, and verify commands, links, and images.

Follow the Documentation Standards and link new pages from `README.md`. Updating this repository does not immediately update the public website; publication depends on the project's documentation build and deployment process.

Translations

Use the translation guide to contribute through Cacti's translation service. Changes to translatable application strings must also update the source message template as described in that guide.

Copyright (c) 2004-2026 The Cacti Group

Creating Translations

Translate Cacti and its plugins

Create an account at the Cacti Translation Site, select the project and branch, and choose a language. Review existing strings or add missing translations through the site's Weblate interface.

Weblate manages synchronization of language translations with GitHub. Do not include unrelated language catalog changes in a code pull request.

Update the source message template

Wrap user-facing strings with Cacti's translation helpers, such as `__()`. When adding, changing, or removing translatable strings, update `locales/po/cacti.pot` in the same pull request.

From the Cacti application repository root, use its extraction script:

```
./locales/update-pot.sh
msgfmt --check-format -o /dev/null locales/po/cacti.pot
```

The script requires GNU gettext (`xgettext`); `msgfmt` validates the template. Review the resulting diff and keep it focused on the changed strings. The script and extraction coverage can differ by branch, so confirm that each new or changed message appears in the template.

Commit the source template, `locales/po/cacti.pot`, with your code change. Leave per-language `.po` and compiled `.mo` updates to Weblate.

Catalog locations and local testing

Cacti stores the source template and language `.po` files under `locales/po/`. Compiled runtime catalogs are under `locales/LC_MESSAGES/`.

For local testing, use a separate Cacti installation, compile the relevant catalog using gettext, and select the language in your user profile. Preserve the repository's locale names and directory layout. Do not deploy catalog files from an unrelated branch to a production installation.

You can edit translations with a gettext-compatible editor such as Poedit, then submit them through Weblate. Consult the Weblate upload documentation for supported formats and upload options.

Preserve placeholders, plural forms, and formatting markers when translating. Test messages with substituted values as well as messages containing accented or non-Latin characters.

Copyright (c) 2004-2026 The Cacti Group

Documentation Standards

Write instructions that match the Cacti branch being documented. State when a feature is available only on the development branch or in a particular release. Verify commands, configuration names, links, and screenshots against that version. Keep changes focused on the affected topic.

Source and corrections

This manual is maintained in the Cacti documentation repository. Submit corrections there, linking an existing issue when applicable. Use `develop` unless maintainers request a maintenance branch. A repository change reaches the public website through the project's build and deployment process.

Markdown format

The repository's `.mdlrc` loads `.mdl_style.rb`. These files define the local Markdown checks. Preserve the existing page's style where it does not conflict with those checks.

Titles

Use ATX headings with a space after the hash characters. Use one level-one heading for the page title, then organize sections with levels two through five. Use level six for numbered example, figure, or table captions when needed. Avoid punctuation at the end of headings and keep heading text descriptive.

```
# Page title

## Main section

### Subsection

Paragraph text.
```

Separate headings from surrounding content with a blank line. Preserve existing heading anchors when possible; when renaming a heading, update links to it. Empty HTML anchors may preserve public fragment links when headings move or change. Use Markdown for visible links and ordinary page content.

Text and lists

Wrap prose at 79 characters. Leave one blank line between paragraphs and around lists and code blocks. Do not leave trailing whitespace or repeated blank lines. Do not use raw HTML for ordinary page content.

Use `-` for unordered lists, a number followed by a full stop for ordered lists, and consistent indentation for nested items and continuation text.

```
- First item.
- Second item with a continuation that remains part of the same list item.

1. First step.
2. Second step.
```

Use backticks for file paths, commands, configuration keys, and literal values. Use bold text for interface labels where it improves readability.

Code blocks

Use fenced code blocks with a language label such as `console`, `bash`, `php`, `sql`, or `ini`. Keep example commands reproducible and identify placeholders. Use continuation lines for long commands where the shell supports them. Code and table widths are exempt from the prose line-length rule in the current style configuration.

Do not include real credentials, tokens, private keys, or unredacted logs. Explain required privileges and commands that alter or delete stored data.

Links and images

Use Markdown links with meaningful labels. Use relative `.md` paths for manual pages so the links work in the repository and published manual.

```
[Data Collectors](Data-Collectors.md)
[Collection settings](Settings-Poller.md#general-settings)
![Data Collectors](images/data-collectors.png)
```

Check that target files and heading anchors exist. Add new guides to `Manual-Index.md` and link entry pages from `README.md`. Check image filenames and use alternate text that describes the image. Explain when a screenshot shows an older interface or a specific theme.

Tables

Use Markdown tables with a header and separator row. Keep column names and units explicit. Add a caption when the table needs to be referenced elsewhere.

Copyright

End pages with a separator and the Cacti Group copyright notice, following the target branch's current year:

```
---

Copyright (c) 2004-2026 The Cacti Group
```

Validate changes

Install Ruby and the `mdl` gem, then run checks from the documentation root:

```
gem install mdl
bin/check_markdown_style.sh Changed-Page.md
bin/check_missing_link.sh
```

Omit the filename to run the Markdown check across the repository. Existing files may have unrelated lint failures; distinguish those from failures added by the change. `check_missing_link.sh` checks whether Markdown filenames occur in `README.md`; it does not validate every link target or heading anchor. Check those separately.

The repository's GitHub Actions spelling workflow installs the pinned requirements from `.github/codespell-requirements.txt`. To run the same spelling check in a Python virtual environment:

```
python3 -m venv /tmp/cacti-docs-spelling
/tmp/cacti-docs-spelling/bin/python -m pip install \
  --only-binary :all: --require-hashes \
  -r .github/codespell-requirements.txt
/tmp/cacti-docs-spelling/bin/codespell --config .codespell.cfg
```

Review `git diff --check` and the rendered Markdown before submitting. Verify that commands and menus describe the intended version; passing Markdown and spelling checks does not establish technical accuracy.

Copyright (c) 2004-2026 The Cacti Group

Formatting Guidelines

Overview

Follow the target branch's `.php-cs-fixer.php` and contributor instructions. The development branch exposes `composer php-cs-fixer` for a dry run and `composer php-cs-fixit` to apply formatting. Inspect formatter changes and avoid unrelated formatting in a focused fix.

Cacti uses conventions based on PSR-12 with project-specific exceptions. The checked-in formatter configuration takes precedence over these examples.

The main conventions are:

- Initial white space must be tabs not spaces followed by the use of spaces to align elements after initial tab.
- In a block of variable assignments or array elements (moved to multi-line) it is acceptable, but not required, to use spaces to align the `=` or `=>` for purpose of readability.
- Bracketing/braces should start on the same line of the statement. For example:
  - `if (expression) {`,
  - `} else {`,
  - and `} elseif (expression) {`
- Use lower case constants for things like `false`, `true`, and `null`.
- Use `print` over `echo`.
- Function declarations should include opening braces on the same line as the function. For example:
  - `function name($params ...) {`
- White-space inside of evaluations is discouraged. For example:
  - `if (expression) {` and not `if ( expression ) {`.
- Prefer single quotes for literal strings. Use double quotes for interpolation or escape sequences. Use short array syntax on the development branch; preserve compatibility with the target branch.

All style guidelines should be followed exactly as shown in these examples, unless indicated otherwise.

Indenting

Very simply, always use tabs to indent rather than spaces. Tab stops should be set to 4 spaces.

Below are the vim rules to accomplish this:

```
set noexpandtab
set shiftwidth=4
set softtabstop=4
set tabstop=4
```

Control Structures

Use the following style guidelines for control structures.

if elseif else Construct

```
if ($var == 0) {
    $i++;
} elseif ($var == 1) {
    $i--;
} else {
    $i = 0;
}
```

switch Construct

```
switch ($var) {
    case 0:
        $var = 1;

        break;
    case 1:
        $var = 2;

        break;
    default:
        $var = 3;

        break;
}
```

while Loop

```
while ($i < 10) {
    $i++;
}
```

for Loop

```
for ($i = 0; $i < 10; $i++) {
    $var *= 6;
}
```

foreach Loop

```
foreach ($array as $key => $value) {
    print "$key = $value\n";
}
```

Naming

General Function Naming

All function names should begin with an identifier (see below) and end with an action. The identifier should generally be derived from the name of the file which the function resides.

```
Returns all users
```

```
function user_list() {
    // code
}
```

Returns a specific user

```
function user_get($user_id) {
    // code
}
```

Returns the total number of graphs

```
function graph_total_get() {
    // code
}
```

Returns a specific graph title

```
function graph_title_get() {
    // code
}
```

Constant Naming

Constants should always be all capital letters. In most cases, they begin with some identifier. The name should be verbose enough that it describes **exactly** what the constant represents. Below is the general structure of the name.

```
[identifier]_[type]_[value]
```

Examples of application constants

```
define('DATA_INPUT_TYPE_SCRIPT', 1);
define('DATA_INPUT_TYPE_SNMP', 2);
```

Data input types for data sources

```
define('DATA_INPUT_TYPE_SNMP_QUERY', 3);
define('DATA_INPUT_TYPE_SCRIPT_QUERY', 4);
```

Common Action Types

- list - an array of one or more items
- get - an array with one item or a single value (such as an integer or string)
- remove - some data is to be removed from the database
- update - some pre-existing data is to be updated
- insert - some new data to be saved
- print - information will be echoed to the screen/browser
- generate - data will be generated and returned
- format - data will be passed in, formatted, and then returned

Formatting

Argument Spacing

Always include a single space after each argument in a list except for the last one.

```
function log_insert($facility, $severity, $message) {
    // code
}
```

```
log_insert(LOG_FACILITY_LOCAL0, LOG_LEVEL_WARNING, 'test message');
```

As a general rule of thumb, it is OK to include additional spaces where necessary to improve code readability. The key here is that writing readable code always takes priority.

White space

Always place a single blank line after a control structure unless the following line is an ending brace `}`. There should be no blank lines at the end of the file, and no spaces at the end of line.

```
if ($var == 0) {
    $i = 0;
}

if ($i == 0) {
    $i *= 2;
}
```

```
if ($var ==0) {
    if ($i == 2) {
        $i++;
    }
}
```

Storage Constructs

Constants

Always use constants instead of numbers to represent static values. Instead of doing:

```
if ($poller_verbosity == 2) {
    // code
}
```

Do:

```
if ($poller_verbosity == POLLER_VERBOSITY_LOW) {
    // code
}
```

Variables

Use a local variable when it clarifies intent, avoids repeated work, or makes error handling explicit. Do not sacrifice readability to avoid an assignment.

Global Variables

Use explicit arguments and return values for new shared code where practical. Preserve existing global-state contracts when fixing legacy functions. Do not introduce additional global state as an incidental part of a focused change.

References

Pass arrays by value unless the function is intended to modify the caller's array. PHP uses copy-on-write; adding references is not a general performance optimization. Document intentional mutation and measure performance-sensitive changes against the actual call path.

Database Calls

Ensure that you indent your Database calls by a single tab per indentation level and that you break up database calls by the various provisos. A few well defined database queries can be seen in the examples below

```
$devices = db_fetch_assoc_prepared('SELECT *
  FROM host
  WHERE id = ?',
  [$device_id]);

$vdefs = db_fetch_assoc("SELECT rs.*,
  SUM(CASE WHEN local_graph_id = 0 THEN 1 ELSE 0 END) AS templates,
  SUM(CASE WHEN local_graph_id > 0 THEN 1 ELSE 0 END) AS graphs
  FROM (
    SELECT vd.*, gti.local_graph_id
    FROM vdef AS vd
    LEFT JOIN graph_templates_item AS gti
    ON gti.vdef_id = vd.id
    GROUP BY vd.id, gti.graph_template_id, gti.local_graph_id
  ) AS rs
  $sql_where
  GROUP BY rs.id
  $sql_having
  $sql_order
  $sql_limit");
```

1. Use prepared statements for values, as in the first example.
2. Build dynamic identifiers and SQL fragments from explicit allowlists. Do not concatenate untrusted values into SQL.
3. Always use the term `AS` to define an alias.
4. Ensure that your JOINs are either INNER, LEFT, or RIGHT.
5. Ensure that the ON clause uses table aliases, and that each `AND` or `OR` expression is logically organized for easy reading and interpretation.
6. In your SQL WHERE and SELECT, where functions are involved
    1. Ensure that there are spaces between parameter
    2. Ensure operators and variables are separated by spaces

Comments

Explain intent, constraints, and non-obvious behavior. Keep comments consistent with the code and avoid comments that merely restate an assignment or test.

Whole Line Comments

All single and multi-line comments should follow the C-style syntax.

```
/* this determines if $i is equal to 0 */
if ($i == 0) {
    // code
}
```

```
/* this is an extra long comment that deserves to be
 * split up into multiple lines
 */
if ($i == 0) {
    // code
}
```

End of Line Comments

Shorter comments that follow a line of code should use the familiar `//` style.

```
$var = 0; // assign 0 to $var
```

File Header Comments

All code related files must contain the following notice at the beginning of the file.

```
/*
 +-------------------------------------------------------------------------+
 | Copyright (C) 2004-2026 The Cacti Group                                 |
 |                                                                         |
 | This program is free software; you can redistribute it and/or           |
 | modify it under the terms of the GNU General Public License             |
 | as published by the Free Software Foundation; either version 2          |
 | of the License, or (at your option) any later version.                  |
 |                                                                         |
 | This program is distributed in the hope that it will be useful,         |
 | but WITHOUT ANY WARRANTY; without even the implied warranty of          |
 | MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the           |
 | GNU General Public License for more details.                            |
 +-------------------------------------------------------------------------+
 | Cacti: The Complete RRDtool-based Graphing Solution                     |
 +-------------------------------------------------------------------------+
 | This code is designed, written, and maintained by the Cacti Group. See  |
 | about.php and/or the AUTHORS file for specific developer information.   |
 +-------------------------------------------------------------------------+
 | https://www.cacti.net/                                                   |
 +-------------------------------------------------------------------------+
*/
```

Function Comments

All functions must be preceded by a comment describing the purpose, argument list, and return type of the function. It should be formatted per PHP DocBlock specifications as follow:

```
/**
 * Makes a connection to the database server
 *
 * Given a set of login credentials, an attempt is made to establish a persistent database connection.
 * If the attempt fails, script execution will halt with an error.
 *
 * @param string $host the hostname of the database server, 'localhost' if the database server is running
 *   on this machine
 * @param string $user the username to connect to the database server as
 * @param string $pass the password to connect to the database server with
```

```
 * @param string $db_name the name of the database to connect to
 * @param string $db_type the type of database server to connect to, only 'mysql' is currently supported
 * @param int $retries the number a time the server should attempt to connect before failing
 *
 * @return bool true if successful, false otherwise
 */
```

Place a space between the `@param` and `@return`.

Copyright (c) 2004-2026 The Cacti Group

PHP Specific Constructs

Use the PHP range declared in the target branch's `composer.json`, together with its CI configuration. The development branch requires PHP 8.3 or later within the declared Composer range; maintenance branches can differ. Do not introduce syntax or APIs unsupported by the branch being changed.

Traversing arrays

Use `foreach` to traverse arrays. `each()` was removed in PHP 8.0.

```
foreach ($devices as $device) {
        // Process the device.
}
```

Define the expected type at the function boundary. If an existing API can return `false` or `null`, handle that result before iterating or calling `count()`. Use `is_countable()` when a value can be an array or `Countable`.

Return values

Preserve the function's documented return contract. Compare strictly when `false` signals failure and a successful value could be `0` or an empty array. Do not change a public API's error handling as part of an unrelated fix.

```
$result = file_get_contents($path);

if ($result === false) {
        // Handle the read failure.
}
```

String operations and regular expressions

Use string functions for literal operations and `preg_*` functions when the operation requires a regular expression. The old `ereg*` and `split()` APIs are unavailable on supported development runtimes.

Prefer single quotes for literal strings. Use double quotes when interpolation or escape sequences make the code clearer. Do not claim a performance benefit without measuring the relevant code path.

Including PHP files

Web entry points that require an authenticated user load `include/auth.php`. User-facing CLI entry points load `include/cli_check.php`. These bootstrap files establish the application environment; a CLI entry point must not load the web authentication bootstrap.

After bootstrap, resolve library paths relative to `$config['base_path']`:

```
require_once($config['base_path'] . '/lib/api_device.php');
```

Use `require_once` for required libraries that might already be loaded. Follow the existing entry point's `--help` and `--version` conventions for CLI tools. Keep request validation and authorization at the appropriate boundaries; see Security.

Copyright (c) 2004-2026 The Cacti Group

File System Layout

Use the target branch's existing structure and naming conventions when adding files. Some names, including `host` for device-related tables and pages, are historical compatibility contracts. Do not rename them as incidental cleanup.

Application entry points

PHP pages in the repository root handle web requests, for example `host.php`, `graphs.php`, and `data_sources.php`. Authenticated pages load `include/auth.php`. Administrative command-line tools live under `cli/` and load `include/cli_check.php`.

The root also contains polling entry points such as `poller.php`, `cmd.php`, `script_server.php`, and `cactid.php`. These have different responsibilities; keep scheduling separate from individual data collection scripts.

Shared code and configuration

- `include/` contains bootstrap, authentication, configuration, global arrays, settings, and form definitions. Configure an installation in `include/config.php`, using the supplied distribution file as a reference.
- `lib/` contains shared functions and application classes. Existing `api_*` files provide management operations; `html_*` files provide rendering and form helpers. Keep reusable operations independent of request superglobals.
- `lib/Auth/` and `lib/Console/` contain namespaced classes mapped by Composer on the development branch. Check `composer.json` before adding a namespace.
- `include/vendor/` is the Composer installation directory. Declare PHP dependencies in `composer.json` and commit the lockfile; do not commit installed package trees or edit generated autoload files.

Load dependencies relative to the application base path after bootstrap:

```
require_once($config['base_path'] . '/lib/api_device.php');
```

Frontend assets

- `include/themes/` contains themes.
- `include/js/` contains JavaScript assets used by the application.
- `images/` contains application images.
- `build/` contains frontend build instructions and tooling on the development branch. Follow its README and the locked npm dependencies when rebuilding assets.

Data collection and extensions

- `resource/` contains XML data-query definitions and other resources.
- `scripts/` contains data collection scripts.
- `plugins/` contains one directory per installed plugin. The directory name must match the plugin's expected identifier, and a plugin supplies an `INFO` file describing it.
- `locales/po/` contains `cacti.pot` and language source catalogs; `locales/LC_MESSAGES/` contains compiled catalogs. Follow the translation workflow.

Runtime files

`rra/`, `log/`, and `cache/` are conventional locations for RRD files, logs, and cached output. Actual paths depend on installation settings. Give the poller and web server only the access they need; do not make application code writable to solve a runtime-directory permission problem.

Tests and documentation

The development branch keeps PHP tests under `tests/`, with unit tests grouped by domain under `tests/Unit/`. Follow the target branch's contributor instructions for test placement, bootstrap, and commands.

Application documentation is under `docs/`; this manual is maintained in the separate documentation repository. Link new manual pages from its `README.md` and use existing image paths where appropriate.

Copyright (c) 2004-2026 The Cacti Group

Patch Creation

Submit changes as a GitHub pull request to the repository that owns the code or documentation. See Contributing to choose a repository.

Choose the target branch

Use `develop` unless maintainers request another branch. Confirm the target before starting a maintenance fix. Check that the defect exists on that branch and that the change preserves its runtime and dependency compatibility.

Keep independent changes in separate pull requests. If a change depends on an unmerged pull request, target its branch and describe that dependency. Keep stack branches in the same repository and merge from the bottom up.

Create a working branch

Fork the relevant repository on GitHub, then clone your fork. This example is for the Cacti application; substitute `documentation` or `spine` when needed. Replace `YOUR-USERNAME` with your GitHub username.

```
git clone https://github.com/YOUR-USERNAME/cacti.git
cd cacti
git remote add upstream https://github.com/Cacti/cacti.git
git fetch upstream
git switch -c fix/device-description upstream/develop
```

For a maintenance contribution, create the branch from the agreed upstream maintenance branch instead. Configure your Git author identity locally if it is not already configured; do not copy another contributor's identity.

Make and validate the change

Keep the patch focused on one problem. Update affected documentation and add regression coverage for realistic failures. Review the target branch's contributor instructions, install its locked dependencies, and run the relevant formatter, analysis, and tests.

For the Cacti development branch, the contributor guide and `composer.json` describe the available checks. For this manual, follow Documentation Standards. Maintenance branches may have different checks; do not introduce new tooling solely for a backport.

Follow the target branch's changelog policy. Where `changelog.d/` fragments are used, follow its README. If maintainers require a direct root `CHANGELOG` edit, use the current unreleased section and the real associated issue or advisory identifier. Keep security entries before issues and issues before features, placing a numbered issue among its numeric neighbors. Do not put a new fix in an already released version.

Inspect the patch before committing:

```
git diff --check
git diff
git status --short
```

Stage the intended files explicitly and write a commit message that explains the change. Avoid committing generated dependencies, credentials, or unrelated formatting changes.

Submit the pull request

Push your working branch to your fork:

```
git push -u origin fix/device-description
```

Open a pull request in the upstream repository. Confirm both the base branch and the file diff before submitting. Explain the problem, resulting behavior, and validation, and link the associated issue. Identify version-specific behavior or limitations that matter to reviewers.

A pull request with no reported checks is unverified. Fork workflows may wait for a maintainer's approval; distinguish pending or missing checks from passing checks. Address review feedback on the same branch.

Update the branch

Fetch upstream changes and rebase a branch you own onto its intended base when needed. Resolve conflicts by preserving both changes' behavior, then rerun the checks relevant to the resolution. Recheck changelog placement and duplicate entries after resolving a changelog conflict.

If you must replace a previously pushed branch after a rebase, use `git push --force-with-lease` only for a branch you own and coordinate with anyone else using it. Do not rewrite shared target branches.

Copyright (c) 2004-2026 The Cacti Group

SQL Standards

Cacti uses MySQL or MariaDB through the helpers in `lib/database.php`. Check the target branch's supported database versions before adding SQL syntax.

Quotes

When writing queries do not use quotes around numeric values. While MySQL is not strict about types, most DBMS's assume that any value contained within quotes is a string.

Use single-quoted PHP strings for literal SQL where possible. Bind changing values with placeholders rather than interpolating them into a query.

Database compatibility

Use SQL supported by both MySQL and MariaDB at the versions required by the target branch. Cacti already uses database-specific functions, including `CONCAT()` and `NOW()`; this manual does not imply support for other database engines. Test changes involving collation, grouping, and date handling on the supported engines where their behavior differs.

SQL Injection

Cacti provides support for prepared statements. You should use these prepared statement functions for values to avoid the risk of SQL injection attacks from the Cacti website. Examples include:

```
// Example prepared statements
$hostname = db_fetch_cell_prepared('SELECT hostname FROM host WHERE id = ?', [$id]);
$host     = db_fetch_row_prepared('SELECT * FROM host WHERE id = ?', [$id]);
$graphs   = db_fetch_assoc_prepared('SELECT * FROM graph_local WHERE host_id = ?', [$id]);

db_execute_prepared('UPDATE host SET description = ? WHERE id = ?', ['Local Machines', $id]);
```

Placeholders bind values, not table or column names. Map dynamic identifiers, sort directions, and other SQL fragments from explicit allowlists. Do not concatenate user-supplied SQL, even when ordinary values are parameterized.

Format of SQL statements

- For short statements, SQL calls may be made on a single row.
- If the SQL statement starts to grow and be unreadable without scrolling, it should be wrapped onto a continuation line for ease of use and readability. The following keywords should also be placed at the start of a new line:
    - joins statements (excluding JOIN itself which should not be used) (eg, INNER JOIN, LEFT JOIN, RIGHT JOIN, OUTER JOIN, CROSS JOIN)

      Note: *Any implicit join or JOIN without one of the modifiers should be converted to an explicit join with a modifier. When converting the implicit join statement, remember to remove the WHERE joining clause.
    - AND / OR
    - FROM
    - WHERE

    - ORDER
    - GROUP BY
    - HAVING
    - LIMIT
    - prepared statement arrays
- Indentation should consist of one extra tab when any of the following occur:
    - continuation lines
    - Subqueries
- There should always be 'AS' when defining an alias for any table, field or query to clearly define the definition of said alias.
- There should always be a space following a comma and before/after an operator.

Example of single line SQL

Before formatting

```
$templates = db_fetch_assoc('SELECT DISTINCT gt.id, gt.name FROM graph_templates AS gt INNER JOIN
graph_templates_graph AS gtg ON gt.id = gtg.graph_template_id INNER JOIN graph_templates_item AS gti ON
gtg.graph_template_id=gti.graph_template_id INNER JOIN data_template_rrd AS dtr ON gti.task_item_id=dtr.id INNER
JOIN data_template_data AS dtd ON dtd.data_template_id=dtr.data_template_id AND dtd.local_data_id = 0 WHERE
gtg.local_graph_id=0 AND dtr.local_data_id = 0 AND dtd.local_data_id = 0 AND dtd.data_input_id in (2,11,12) ORDER
BY gt.name;');
```

Corrected formatting with prepared usage

```
$templates = db_fetch_assoc_prepared('SELECT DISTINCT gt.id, gt.name
    FROM graph_templates AS gt
    INNER JOIN graph_templates_graph AS gtg
    ON gt.id = gtg.graph_template_id
    INNER JOIN graph_templates_item AS gti
    ON gtg.graph_template_id=gti.graph_template_id
    INNER JOIN data_template_rrd AS dtr
    ON gti.task_item_id=dtr.id
    INNER JOIN data_template_data AS dtd
    ON dtd.data_template_id=dtr.data_template_id
    AND dtd.local_data_id = 0
    WHERE gtg.local_graph_id=0
    AND dtr.local_data_id = 0
    AND dtd.local_data_id = 0
    AND dtd.data_input_id in (?,?,?)
    ORDER BY gt.name',
    [2, 11, 12]);
```

Example of implicit join

Before formatting

```
$user_realms = db_fetch_assoc('SELECT ua.id, uar.realm_id
    FROM user_auth ua, user_auth_realms uar
    WHERE ua.id = uar.user_id');
```

Corrected formatting

```
$user_realms = db_fetch_assoc('SELECT ua.id, uar.realm_id
    FROM user_auth ua
    INNER JOIN user_auth_realms uar
    ON ua.id = uar.user_id');
```

Copyright (c) 2004-2026 The Cacti Group

Security

Report vulnerabilities privately

Follow the Cacti security policy for suspected vulnerabilities. Keep exploit details and credentials out of public issues and pull requests until disclosure is coordinated.

Authentication and authorization

Authenticated web entry points load `include/auth.php`. Check the user's realm and object permissions before returning protected data or changing state. Hiding a button or filtering a browser view does not enforce authorization. Shared management functions must not assume that a caller owns an object just because its ID is valid.

Protect cookie-authenticated changes with Cacti's CSRF mechanism. Use POST for mutations and include the expected `__csrf_magic` token in forms and AJAX requests. Do not implement state changes through GET requests.

Request validation

Treat request data, device responses, plugin output, and database content as untrusted. Validate each value for its intended use. Cacti provides request helpers in `lib/html_utility.php`:

- `get_filter_request_var()` (also `gfrv()`) validates an integer by default and supports PHP filter constants and options for other types.
- `get_request_var()` (`grv()`) retrieves request data; retrieval alone does not establish that the value is valid for a particular use.
- `set_request_var()` (`srv()`) stores a request value; it is not a validator.

A value already validated in the request cache need not be validated again in every getter. Preserve the validation contract and check lengths, ranges, allowed choices, and permissions where required. The **Log Input Validation Issues** setting helps identify missing validation during development.

SQL statements

Use Cacti's prepared database helpers for values:

```
$hostname = db_fetch_cell_prepared('SELECT hostname
        FROM host
        WHERE id = ?',
        [$device_id]);
```

Placeholders bind values, not identifiers or SQL fragments. Build dynamic column names, sort directions, and table names from an explicit allowlist. See SQL Standards.

OS commands and paths

Prefer an existing API or process helper over constructing shell commands. When a shell command is required, validate values and escape each argument using the target branch's Cacti helpers. On the development branch, use `cacti_escapeshellarg_cmd()` for arguments reaching Windows `cmd.exe`, and `cacti_escapeshellarg()` for the supported non-cmd path. A whole-command escape is not a substitute for argument escaping.

Use configured binary paths and restrict user-selected paths to the intended location. Check ownership and permissions before reading, replacing, or deleting files. Do not solve permission errors by making application code or credential files writable by everyone.

Output escaping

Escape values for the context where they are rendered. HTML text, HTML attributes, JavaScript, URLs, and shell arguments require different handling. On the development branch, use `html_escape()` for HTML text and `html_escape_attr()` for attribute values. Check helper availability before backporting to another branch.

```
print '<td>' . html_escape($description) . '</td>';
```

When building browser content, use `textContent` for plain text and DOM methods for elements. Do not concatenate untrusted values into HTML, inline event handlers, or executable JavaScript. Validate dynamic URLs for the intended protocol and destination before escaping them for an attribute.

For intentionally supported HTML, use the application's reviewed sanitization path. Sanitize after Markdown or other markup conversion and do not append untrusted markup afterward. A content security policy supplements these controls; it does not replace validation, authorization, or output escaping.

Browser and server checks

Browser validation gives prompt feedback, but enforce validation and permissions on the server. Test realistic denied-access cases as well as valid requests when changing a security boundary.

See also

Hardening a Cacti Installation covers installation permissions and operating-system controls.

Copyright (c) 2004-2026 The Cacti Group

Architecture Decision Records

Architecture Decision Records (ADRs) capture decisions that guide Cacti's long-term architecture. They are concise, reviewable documents rather than implementation plans.

Status values

- **Proposed**: under review and not yet binding.
- **Accepted**: approved for new work.
- **Superseded**: replaced by a later ADR.

Decisions

1. ADR-0001: Establish bounded contexts and framework-independent domain code
2. ADR-0002: Model installation as durable, resumable install runs
3. ADR-0003: Provide a versioned plugin extension boundary
4. ADR-0004: Persist workflow state separately from diagnostic artifacts

Contributing

New ADRs should describe the context, decision, consequences, and links to the implementation work. An ADR does not replace user, operator, or API documentation.

ADR-0001: Establish bounded contexts and framework-independent domain code

Status: Proposed

Context

Cacti has grown through a long-lived procedural and class-based codebase. Several workflows combine HTTP handling, CLI parsing, rendering, persistence, shell execution, and business rules in the same implementation. This makes incremental change difficult and exposes internal behavior to plugins.

Decision

New architectural work will identify bounded contexts and keep their domain logic framework-independent. Initial contexts include Installation, Plugin Management, Configuration, Authorization, Data Collection, and Template and Package Management.

Domain code will use PHP 8.1 value objects, backed enums, aggregates, and domain exceptions where they make rules explicit. Constructors must not access the database, filesystem, shell, globals, or request state. Infrastructure, framework components, and legacy helpers will be accessed through application services and narrowly defined ports.

Consequences

- New code can be tested without web, CLI, or database bootstrap.
- Symfony components may implement infrastructure concerns, but Cacti will not require a full Symfony application migration.
- Existing code is migrated through adapters; this decision does not require a flag-day rewrite.
- Public plugin APIs must depend on stable application contracts, not domain internals or legacy globals.

ADR-0002: Model installation as durable, resumable install runs

Status: Proposed

Context

The installer currently keeps workflow state in mutable global settings and mixes wizard rendering with side effects. Background timestamp coordination can hide failures or permit ambiguous ownership of an install attempt.

Decision

Installation and upgrade work will be represented by a durable `InstallRun` with a unique identifier, typed options, plan version, explicit state, operation checkpoints, lease data, progress, and structured failure details.

The lifecycle is `validating`, `ready`, `running`, `succeeded`, `failed`, or `cancelled`. State transitions are validated and recorded atomically. A worker holds a renewable lease for a run and executes one idempotent operation at a time. Web and CLI interfaces submit commands and render run views; they do not perform installation work directly.

Consequences

- Refreshing a wizard page and querying status are side-effect free.
- Failed runs remain failed until an explicit retry or new run is requested.
- Operations can resume from durable checkpoints after interruption.
- Legacy `install_*` settings require a temporary compatibility adapter and a documented removal plan.

Project Credits and Licensing

Return to the documentation front page.

Cacti is maintained by The Cacti Group and its community. These named credits follow the project's About page and preserve the recognition given in this manual. They are not an exhaustive list of contributors.

Active Developers

- Larry Adams (TheWitness)
- Mark Brugnoli-Vinten (netniV)
- Jimmy Conner (cigamit)
- Petr Macek (xmacan)
- Andreas Braun (browniebraun)
- Thomas Urban (phalek)
- Jing Chen (ddb4github)
- Thomas Vincent (somethingwithproof)

Contributors

- Sean Mancini (bmfmancini)
- Chris Bell (Windows) (BSOD2600)
- Paul Gevers (Debian) (paulgevers)
- Morten Stevens (Fedora) (mortenstevens)

Emeritus Members

- Ian Berry (raX)
- Reinhard Scheck (gandalf)
- Tony Roman (rony)
- Andy Blyler (ablyler)

In Memoriam

Jeff Pasnak, credited as **J.P. Pasnak, CD (Linegod)**, was a longtime core contributor. Cacti's 1.2.31 release announcement records his passing and honors his contributions to the project and community.

Community Contributions

Cacti depends on contributors who write and review code, maintain packages, improve documentation, translate the interface, report security issues, test releases, and help other users.

See the maintained Cacti contributor credits, code contribution history, and documentation contribution history for broader recognition. Translation contributions are also recorded on the Cacti translation platform.

Thanks

Thank you to Tobi Oetiker, the creator of RRDtool and MRTG, and to everyone who reports problems, contributes fixes, or supports Cacti and its users.

License

Cacti is licensed under the GNU GPL:

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

The Cacti source repository's license contains the full terms.

Credit Sources

The named roster was checked against Cacti's About page and contributor credits on **26 October 2026**. The release announcement provides the updated recognition for Jeff Pasnak. Follow these project-maintained sources when updating names and roles; commit counts alone do not establish team membership.

Copyright (c) 2004-2026 The Cacti Group

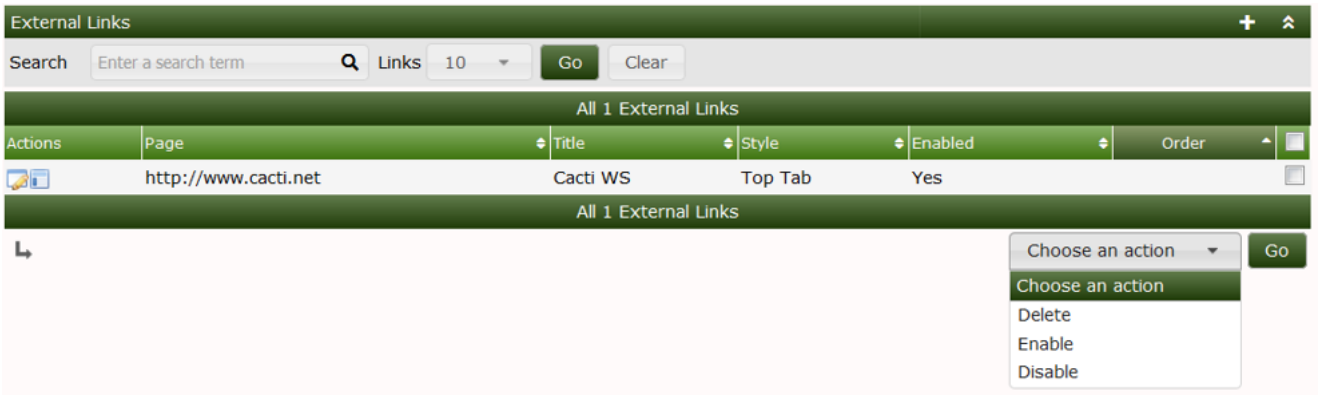
External Links

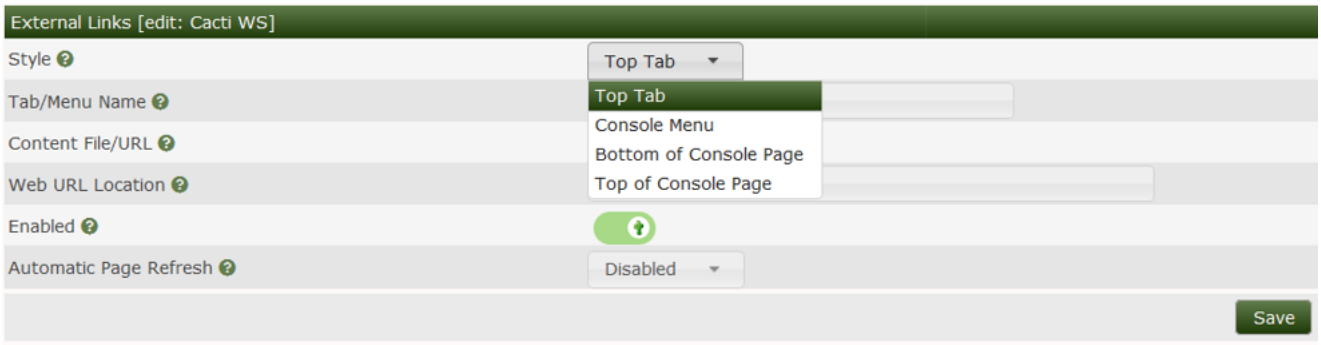
This section will describe **External Links** in Cacti.

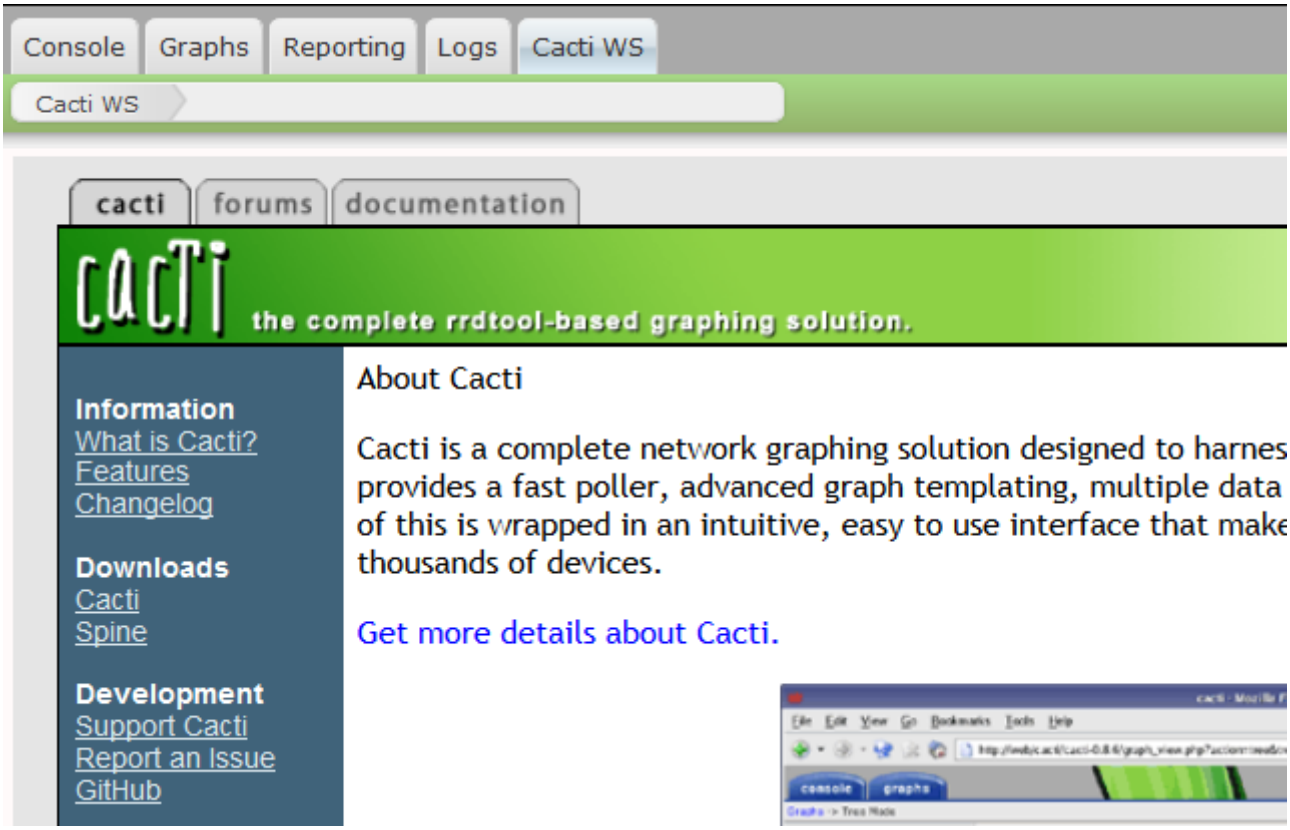
External Links lets administrators add navigation entries for approved URLs. Choose a destination that permits embedding when using an iframe.

An embedded destination can refuse display through its own frame policy. HTTPS Cacti pages also cannot freely embed HTTP content. Check the destination and Cacti content security policy; do not weaken site-wide controls just to embed an unrelated website.

This feature can be very useful to create a single UI for helpdesk/NOC.







Copyright (c) 2004-2026 The Cacti Group

Enterprise Linux LAMP Installation

Use the Enterprise Linux LAMP installation guide and the supported operating systems. This page preserves an older manual URL.

Copyright (c) 2004-2026 The Cacti Group

Enterprise Linux LEMP Installation

Use the Enterprise Linux LEMP installation guide and the supported operating systems. This page preserves an older manual URL.

Copyright (c) 2004-2026 The Cacti Group

Installing on Rocky Linux 9 and 10 with nginx

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

This source-install example covers the unreleased Cacti 1.3 development branch on RHEL, Rocky Linux, or AlmaLinux. Read Requirements first. Run administrative commands as root and adapt paths and accounts to your deployment. For an existing installation, use Upgrading Cacti. For package-managed Cacti, follow the package maintainer's procedure instead of overlaying source files.

Choose the Rocky Linux release and packages

Use the current security-updated minor release of Rocky Linux 9 or 10. This is a fresh-install procedure; do not reset or switch module streams on an existing database or PHP deployment without a separate upgrade plan.

Rocky Linux branch	PHP and database selection
9	Enable the AppStream `php:8.3` and `mariadb:10.11` streams before installing packages
10	Install the AppStream PHP and MariaDB packages directly; this branch does not use module streams

Rocky Linux 9

Inspect the available streams, then select the versions for this example:

```
dnf module list php mariadb
dnf module enable -y php:8.3 mariadb:10.11
```

Rocky Linux 10

Check the packages offered by AppStream; do not run `dnf module` commands:

```
dnf info php-fpm mariadb-server
```

Rocky's PHP/FPM guide explains the difference in packaging, and its release notes list the major-version package changes. Other Enterprise Linux distributions can differ in subscriptions, repositories, and package availability; verify those before adapting this Rocky example.

Install the web and database packages

This example uses MariaDB. MySQL installations should use the package and service names for their selected MySQL repository. Review the database version against Cacti's requirements before installation.

```
dnf install -y nginx php-fpm php-cli php-mysqlnd php-gd php-gmp \
  php-intl php-ldap php-mbstring php-xml php-process php-snmp php-pdo \
  mariadb-server rrdtool net-snmp-utils cronie
php -v
php -m
```

Additional packages can be required by the chosen source revision. Check Requirements, including Composer dependencies where applicable. Do not mix extensions from different PHP versions or repositories.

Configure `date.timezone` in the PHP configuration used by both CLI and FPM. Check `php --ini` for CLI and the installer for web PHP. Review the FPM pool at `/etc/php-fpm.d/www.conf`. For this example, retain the `apache` worker account and allow nginx to connect through the socket ACL:

```
user = apache
group = apache
listen = /run/php-fpm/www.sock
listen.acl_users = apache,nginx
```

Modify the existing entries rather than appending a second pool definition. The socket ACL grants nginx connection access; Cacti's runtime files must be writable by the FPM worker and poller accounts. Keep code owned by the administrator. Validate the pool with `php-fpm -t` before restarting FPM.

Start the database:

```
systemctl enable --now mariadb
```

Follow the installed database vendor's initialization and account-security procedure. Do not set a root password on the shell command line. Configure only variables accepted by that database version; use Cacti's installer recommendations rather than a fixed tuning block. Keep InnoDB doublewrite protection enabled.

Configure nginx

Configure the FPM pool's `listen` path and socket ACLs so nginx can connect. The example uses `/run/php-fpm/www.sock`; verify that it matches your pool. Keep the FPM worker account consistent with the permissions you grant later. Save this example in `/etc/nginx/conf.d/cacti.conf`, replacing the hostname and certificate paths:

```
server {
    listen 80;
    server_name cacti.example.com;
    return 301 https://cacti.example.com$request_uri;
}

server {
    listen 443 ssl;
    server_name cacti.example.com;
    root /usr/share/nginx/html;
    index index.php;
    ssl_certificate /etc/pki/tls/certs/cacti.crt;
    ssl_certificate_key /etc/pki/tls/private/cacti.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    location = / { return 302 /cacti/; }
    location = /cacti { return 301 /cacti/; }

    # nginx does not read Apache .htaccess files.
    location ~ (^|/)\. { deny all; }
    location ~ ^/cacti/(cli|contrib|log|mibs|rra|scripts|service)/ {
        deny all;
    }
    location ~ ^/cacti/cache/(boost|mibcache|realtime|spikekill)/ {
        deny all;
    }
    location ~ ^/cacti/include/(config(_local)?\.php|vendor/.*\.php) {
        deny all;
    }

    location ~ ^/cacti/(.+\.php)$ {
        try_files $uri =404;
        include /etc/nginx/fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_pass unix:/run/php-fpm/www.sock;
    }
    location /cacti/ { try_files $uri $uri/ =404; }
}
```

Keep private-path restrictions before the PHP handler and validate them over HTTP. Static assets under `include/vendor` can be required by the interface; do not deny all vendor assets indiscriminately. Keep backups outside every web root. Restrict setup to administrators until installation is complete.

```
nginx -t
systemctl enable --now php-fpm nginx
```

Install and configure Cacti

Install Composer using its official installation procedure and verify the installer as documented there. Run `composer install --no-dev` and `composer check-platform-reqs --no-dev` as the deployment owner in the staged source tree before deploying it to the web root. Do not substitute `composer update` or run dependency scripts as the runtime service account.

Use General Installing Instructions to obtain the intended source, create a new database and dedicated application account, load timezone data, import `cacti.sql`, and configure `include/config.php`. The examples there use `/var/www/html/cacti`; substitute `/usr/share/nginx/html/cacti` here.

Complete the dependency and filesystem-permission checks before opening the installer. The database account and operating-system service accounts are separate identities. Grant runtime write access to the actual FPM and poller accounts, not automatically to the nginx master process or the database user. Keep executable code and credentials protected during normal operation.

Open `https://cacti.example.com/cacti/` and complete the installer. Configure one scheduler after installation: either a Cacti-specific cron entry or the supplied `cactid` systemd service, using the procedure in the general guide. Do not run both for the same collector. The five-minute cron example is:

```
*/5 * * * * apache /usr/bin/php /PATH/TO/cacti/poller.php >/dev/null 2>&1
```

Replace `/PATH/TO/cacti` and `apache` with the installed path and chosen poller account. Enable `crond` if using cron. Verify a manual poll under that account, then check scheduled runs, logs, collector status, and fresh graph samples.

SELinux and containers

Keep SELinux enforcing. For a custom installation path, use persistent file contexts and allow writes only on Cacti's runtime paths. Review audit denials and authorize the specific network and executable access needed by FPM, RRDtool, SNMP tools, and the collector. Do not use `setenforce 0` as the installation procedure. See the RHEL SELinux guide.

For LXC or Proxmox, retain an unprivileged container and its confinement where possible. Diagnose the specific ICMP/socket permission or mapped-file access failure. Adjust only the required capability or policy; do not make the entire container privileged or set an unconfined AppArmor profile merely to run Cacti.

Optional Spine and proxies

Follow Spine for the optional compiled collector. Select a compatible source or package and the development headers for your OS. Repository names differ: do not apply Rocky/AlmaLinux `crb` commands to RHEL without checking its repository configuration. Configure the Spine database connection with the same application credentials and restrict access to its configuration file.

When using a reverse proxy, see the proxy configuration in the general guide. A header allowlist alone does not authenticate a proxy; prevent clients from bypassing the trusted proxy and require it to replace forwarded client-IP headers.

Copyright (c) 2004-2026 The Cacti Group

Installing Cacti on Ubuntu Server with nginx

This fresh-install guide covers Cacti development (1.3) with nginx, PHP-FPM, and MariaDB. Use the requirements for this Cacti branch. For an existing installation, follow Upgrading Cacti. Do not overlay source files onto a package-managed Cacti installation.

Choose a supported Ubuntu release

As of 26 October 2026, the latest LTS is **Ubuntu Server 26.04 LTS** and the latest interim release is **Ubuntu Server 26.10**. Use fully updated packages. The interim release has a shorter maintenance window; plan its next upgrade before security support expires. See the LTS release notes, interim release notes, and Ubuntu lifecycle.

Ubuntu release	PHP selection	PHP-FPM unit and socket
26.04 LTS	Distribution PHP 8.5 packages	`php8.5-fpm.service`, `/run/php/php8.5-fpm.sock`
26.10	Distribution PHP 8.5 packages	`php8.5-fpm.service`, `/run/php/php8.5-fpm.sock`

The Ubuntu package index documents the LTS FPM package; the interim package index lists the corresponding package for 26.10. Check the package candidates before installation. Vendor OS support and package availability do not establish that every Cacti plugin has been tested with that PHP version. Validate the selected Cacti and plugin set on an isolated installation before production use.

Install nginx, PHP-FPM, and MariaDB

Run administrative commands as root. Enable Ubuntu's Universe repository if it is not already enabled, then install the packages:

```
apt-get update
apt-get install -y software-properties-common
add-apt-repository -y universe
apt-get update
apt-cache policy php-fpm php-cli mariadb-server nginx
apt-get install -y nginx mariadb-server rrdtool snmp cron git \
  php-fpm php-cli php-mysql php-snmp php-xml php-mbstring php-gd php-gmp \
  php-zip php-ldap php-intl php-curl php-sqlite3 composer
php -v
php --ini
php -m
```

Use packages from a consistent Ubuntu release and repository set. Do not install `libapache2-mod-php` for this nginx/FPM setup. If Apache is already bound to the chosen HTTP/HTTPS ports, resolve that conflict before starting nginx rather than running two servers on the same address and port.

Set `date.timezone` for the CLI and FPM configurations, normally under `/etc/php/8.5/cli/php.ini` and `/etc/php/8.5/fpm/php.ini`. Verify the FPM pool at `/etc/php/8.5/fpm/pool.d/www.conf` uses the intended account and socket:

```
user = www-data
group = www-data
listen = /run/php/php8.5-fpm.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660
```

Edit the existing entries. The nginx worker must be able to connect to that socket. Check the actual pool if you changed the account, PHP version, or socket path. Validate FPM configuration and enable the services:

```
php-fpm8.5 -t
systemctl enable --now mariadb php8.5-fpm nginx cron
```

Deploy Cacti and create the database

Run dependency installation as the deployment owner in the source directory, before installing it under the administrator-owned web root:

```
git clone --branch develop --depth 1 https://github.com/Cacti/cacti.git cacti
cd cacti
composer install --no-dev
composer check-platform-reqs --no-dev
```

Record `git rev-parse HEAD` and deploy that complete tree, including its locked dependencies, to `/var/www/html/cacti`. Use administrator privileges only for placement, ownership, and system configuration.

Follow General Installing Instructions to create the new database and dedicated application account, load timezone data, import the matching `cacti.sql`, and copy/configure `include/config.php`. These are fresh-install steps; do not import the default schema over an existing Cacti database.

Use the `www-data` FPM/poller account for this example's runtime permissions. Keep code owned by the administrator, configuration readable only by the required identities, and write access limited to runtime paths. Grant extra installation/import access only where the installer requests it and remove that access afterward. Database credentials and OS service accounts are separate identities.

Configure nginx and HTTPS

Provision a certificate for your hostname using your certificate-management procedure. Replace `cacti.example.com` and the certificate/key paths below. Save this virtual host as `/etc/nginx/sites-available/cacti`:

```
server {
    listen 80;
    server_name cacti.example.com;
    return 301 https://cacti.example.com$request_uri;
}

server {
    listen 443 ssl;
    server_name cacti.example.com;
    root /var/www/html;
    index index.php;
    ssl_certificate /etc/ssl/certs/cacti.crt;
    ssl_certificate_key /etc/ssl/private/cacti.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    location = / { return 302 /cacti/; }
    location = /cacti { return 301 /cacti/; }

    # nginx does not process Cacti's Apache .htaccess restrictions.
    location ~ (^|/)\. { deny all; }
    location ~ ^/cacti/(cli|contrib|log|mibs|rra|scripts|service)/ {
        deny all;
    }
    location ~ ^/cacti/cache/(boost|mibcache|realtime|spikekill)/ {
        deny all;
    }
    location ~ ^/cacti/include/(config(_local)?\.php|vendor/.*\.php) {
        deny all;
    }

    location ~ ^/cacti/(.+\.php)$ {
        try_files $uri =404;
        include /etc/nginx/fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
```

```
        fastcgi_pass unix:/run/php/php8.5-fpm.sock;
    }
    location /cacti/ { try_files $uri $uri/ =404; }
}
```

Enable this site once, without overwriting an existing link or configuration:

```
ln -s /etc/nginx/sites-available/cacti /etc/nginx/sites-enabled/cacti
nginx -t
systemctl reload nginx
```

Review the packaged default site and other virtual hosts for conflicts. Keep private-path rules before the PHP handler, leave required static vendor assets accessible, and store backups outside every web root. Confirm over HTTP that private files are denied and PHP requests execute rather than expose source. Restrict installation access to administrators until complete.

Complete installation and schedule polling

Open `https://cacti.example.com/cacti/`, complete the installer, and check the CLI and web PHP environments separately. Verify executable paths, timezone data, filesystem permissions, and optional Spine before enabling collection.

Configure either cron or `cactid`, using the Ubuntu scheduler guide. Do not run both for one collector. For the default five-minute scheduler, create `/etc/cron.d/cacti` with mode `644` and this system-cron entry:

```
*/5 * * * * www-data /usr/bin/php /var/www/html/cacti/poller.php >/dev/null 2>&1
```

Run a manual polling cycle as the poller account before scheduling, then verify scheduled execution, collector status, logs, and recent RRD samples. Keep AppArmor enforcing and adjust only the specific policy and filesystem access required by the deployed services. For reverse proxies, use the trusted-proxy guidance rather than trusting arbitrary client headers.

Copyright (c) 2004-2026 The Cacti Group

Converting a Debian or Ubuntu Package Installation to Source

Use this procedure only on a supported operating system. Changing installation ownership can affect paths, service accounts, package upgrades, and database management. Test the migration on an isolated copy before changing a production server.

Record the installed layout

Record the Cacti version, database and grants, PHP configuration, web-server configuration, scheduler, service accounts, plugins, Spine configuration, and actual RRD, log, cache, script, and query-resource paths. Distribution packages can keep configuration and data outside the web application tree. Inspect symlinks and package-managed files; do not assume that copying the web directory includes their external targets.

Pause writers and take a complete backup

Follow Upgrading Cacti to pause only Cacti's scheduler and other application writers, and take a matching backup of the database, files, RRD history, configuration, and external state. Do not stop the entire system cron service. Verify the backup and rehearse restoring it before changing the package installation.

Stage and verify the source installation

Use the source installation guide to select and stage the intended Cacti revision and dependencies in a new directory. Its fresh-install schema import is for an empty database. Preserve the existing database for this migration; do not import `cacti.sql` over it or run the fresh-install procedure against production data.

Copy the existing configuration, custom resources, plugins, and RRD data to the intended paths, preserving permissions. Reconcile the new configuration example, URL path, web restrictions, dependency versions, and service accounts. Protect credentials and secret files. Use the package maintainer's documented procedure to relinquish package ownership without deleting application data. Do not purge the package or accept a database-removal prompt merely to switch to source. Keep the old installation and verified backup available.

Cut over and validate

With writers still paused, update the web mapping and the single Cacti scheduler to the source installation. If changing Cacti versions, follow the database-upgrade procedure against the preserved database. Verify paths, permissions, plugins, collector connectivity, and any remote-collector upgrades before resuming collection.

Check scheduled polling, logs, RRD updates, and representative graphs. Document how future source updates will be applied and prevent a later package operation from overwriting the source deployment. If recovery is required, restore the matching files, database, RRDs, and configuration as one set; restoring application files alone does not undo a database migration.

Copyright (c) 2004-2026 The Cacti Group

ADR-0003: Provide a versioned plugin extension boundary

Status: Proposed

Context

Plugins need to extend Cacti without depending directly on legacy installer state, globals, raw persistence, or unstable domain internals. Existing hooks must remain compatible while the platform evolves.

Decision

Cacti will introduce a versioned plugin manifest and extension registry. Plugins will declare identity, compatibility, dependencies, capabilities, and their supported lifecycle contributions. They will receive typed application contexts and least-privilege service interfaces.

Installer extensions will contribute pure plan entries and worker-owned, idempotent operations. Plugin operations are checkpointed by run, plugin, version, and operation key. Required plugin failures prevent core terminal success; optional failures follow an explicit policy.

Consequences

- Plugin authors have a stable compatibility contract and conformance tests.
- Legacy hooks are translated through a compatibility adapter during a published deprecation period.
- Plugins cannot mutate install-run state directly.
- Core can report plugin lifecycle failures with plugin and run identifiers while redacting secrets.

ADR-0004: Persist workflow state separately from diagnostic artifacts

Status: Proposed

Context

Installer state, progress, and diagnostics are currently spread across global settings and logs. This makes recovery, support, retention, and secret redaction difficult to reason about.

Decision

Relational storage is the source of truth for install runs, operation checkpoints, leases, and structured events. Each record is scoped to a run ID and uses atomic state and lease updates. Verbose logs, support bundles, and large generated artifacts are stored separately with metadata and checksums in the relational store.

Run options, checkpoints, and events are schema-versioned and validated. Secrets are not copied into run records, events, logs, or artifacts; protected configuration stores secret values and workflow records retain only references or redacted values.

Consequences

- Operators can diagnose a run without reconstructing state from global logs.
- Retention can prune successful details while preserving failed-run evidence.
- Plugin operation data can use generic namespaced checkpoints instead of creating arbitrary installer-state tables.
- Storage migrations, indexing, artifact cleanup, and retention policy are required parts of the implementation.